

```
import fs from "node:fs";  
  
import os from "node:os";  
  
import path from "node:path";  
  
function 설비읽기() {  
  function 설비쓰기(목록) {  
    function 설비추가(이름, 라인) {  
  
const 목록 = 설비읽기();
```

1 권 · 표와 SQL

데이터베이스

표를 만들고, 넣고, 잇고, 빠르게 만듭니다.

이 책의 모든 출력은 실제로 실행해서 얻은 값입니다.

5

단원

25

개념

148

직접 해보기

0

문제

차례

01	왜 데이터베이스인가	13
01	파일에 저장하면 생기는 일	14
02	데이터베이스는 어떻게 다른가	23
03	어떤 데이터베이스를 고르나	36
04	SQL 은 어떤 말인가	48
05	이 자료를 보는 법	60
02	표 설계하기	81
01	표와 칸 만들기	82
02	타입 고르기	93
03	제약으로 규칙을 박습니다	95
04	기본키를 무엇으로	108
05	NULL 은 값이 아닙니다	122
03	넣고 읽고 고치고 지우기	147
01	넣기	148
02	읽기	162
03	SQL 인젝션을 직접 뚫어 봅니다	177
04	고치고 지우기	191
05	한 번에 여러 건 다루기	204
04	데이터 모델링과 정규화	229
01	한 표에 다 넣으면 생기는 일	230
02	표를 나눕니다	246
03	외래키로 잇습니다	264
04	관계의 모양	277
05	언제 정규화를 깨나	292
05	여러 표를 잇기	315

01	두 표를 잇기	316
02	없는 쪽도 보기	334
03	묶어서 세기	351
04	쿼리 안의 쿼리	371
05	줄마다 계산하기 (윈도우 함수)	390

부록 가	연습문제·종합연습 정답	421
------	---------------------	-----

부록 나	심화문제 정답	493
------	----------------	-----

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작하기

```
npm install
cd 01_왜_데이터베이스인가
node 개념01_파일에_저장하면_생기는_일.js
```

01~06단원은 이게 전부입니다. 데이터베이스를 설치하지 않습니다.

npm install 하나로 진짜 PostgreSQL 18 이 옵니다. PGLite 라는 것인데, Postgres 를 WebAssembly 로 컴파일한 것입니다. 흥내가 아니라 진짜입니다.

```
PGLite 0.5.7 = PostgreSQL 18.3 (wasm32)
```

07단원부터는 진짜 서버가 필요합니다. 동시에 두 사람이 접속하는 실습을 하기 때문입니다.

```
docker compose up -d      # postgres:18 (5434) + mysql:8.4 (3307)
docker compose ps          # 둘 다 healthy 가 될 때까지 기다립니다
```

필요한 것

무엇	언제부터	비고
Node 20 이상	01단원	npm install 만 하면 됩니다
Docker	07단원	★ 01~06단원은 필요 없습니다

포트가 5432 도 5433 도 아닌 **5434** 입니다. 이미 깔린 Postgres 와 안 부딪히게 비켜 뵈었습니다. 그 포트도 쓰고 있으면 docker-compose.yml 의 숫자를 바꾸세요. (port is already allocated 라는 에러가 납니다)

이 자료가 다른 점

모든 출력은 실제로 실행해서 얻은 값입니다

자료에 적힌 // 출력: 은 전부 진짜 실행 결과입니다. 기계가 대조합니다.

```
npm run 검증
```

사람 눈으로는 절대 못 잡습니다. 자료에 적힌 출력이 실제와 다르면 학생이 자기가 틀린 줄 알고 헤맬니다.

틀린 것을 실제로 돌려서 보여 줍니다

- SQL 인젝션을 **진짜로 뚫습니다** (로그인 우회, UNION 유출, 표 이름 캐내기)
- 데드락을 **진짜로 만듭니다** (40P01 deadlock detected)
- UPDATE 에서 WHERE 를 빠뜨려 **전부 바꿔 봅니다**
- 표를 **진짜로 날리고 복구합니다**

감으로 쓰지 않았습니다

아래는 이 자료를 만들면서 감으로 적었다가 측정에 뒤집힌 것들입니다. 전부 다시 재서 고쳤습니다.

★ 실측으로 뒤집힌 것들

타입

무엇	처음 생각	실제
BIGINT 가 JS 로 오면	항상 bigint	★★★ PGlite: 안전정수($2^{53}-1$) 이하면 number, 넘으면 bigint / pg 드라이버: 항상 string
COUNT(*)	bigint	PGlite 는 number, pg 는 string
0.1 + 0.2 !== 0.3 재현	REAL 로 됨	★ REAL 로는 안 됩니다. 정밀도가 낮아 오차가 반올림에 묻힙니다. DOUBLE PRECISION 이어야 합니다
DATE 의 시간대 함정	toISOString()	★ PGlite 는 getDate() 쪽 / pg 는 로컬 자정이라 toISOString() 도 하루 밀립니다

★★ BIGINT 함정이 생각보다 무섭습니다. 개발할 때는 id 가 작아 number 로 오다가, 운영에서 커지면 bigint 로 바뀌어 JSON.stringify 가 터집니다. 테스트로는 절대 안 잡힙니다. 그래서 이 자료는 전부 count(*)::int 를 씁니다.

성능

무엇	처음 생각	실제
----	-------	----

DB 가 파일보다 빠르다	항상	★ 10만 건이면 메모리 배열 filter 가 8배 빠릅니다. 그래도 DB 를 쓰는 이유는 따로 있습니다 (01단원 개념02)
색인 효과	벽시계로 재면 됨	★★ 벽시계로는 3342배인데 Execution Time 으로는 400600배 **, Buffers 로는 220분의 1
복합색인 앞칸이 아니면	절대 안 쓰임	★★ PG18 부터 skip scan 이 생겨서 뒷칸만 줘도 씁니다. 다만 앞칸 값 종류가 많으면 여전히 Seq Scan
OR 는 색인을 못 씀	항상	한 표 안에서 양쪽에 색인이 있으면 BitmapOr 로 잘 씁니다. 두 표에 걸칠 때가 진짜 문제
커버링 색인	만들면 Index Only Scan	★ VACUUM 을 해야 가시성 지도가 채워집니다. 그 전엔 Bitmap Heap Scan
N+1	로컬에서도 크게 느림	로컬 5.5배. 망 왕복 3ms 를 흥내 내면 33~40배
상관 서브쿼리	조금 느림	JOIN 보다 120~220배 (설비 1000 / 점검 20000)
OFFSET 이 커지면 느려짐	항상	★ 정렬키에 색인이 있어야 재현됩니다 (없으면 2.9배, 있으면 65~70배)
10만 건 넣기	한 건씩도 할 만함	한 건씩 16.1초 / 묶음 0.68초 / generate_series 0.34초 / COPY 0.23초 (71배)

동시성

무엇	처음 생각	실제
REPEATABLE READ 와 팬텀	교과서대로 허용	★★★ Postgres 는 막습니다. 스냅샷 격리라서요. 표준은 최소 요구조건입니다
스냅샷이 찍히는 시점	BEGIN	★ 첫 문장입니다. BEGIN 뒤에 남이 커밋한 것이 보입니다
데드락 희생자	규칙이 있음	★ 완전히 비결정적입니다. 4회 재니 가/나/가/나
SERIAL 번호	롤백하면 돌아옴	★ 안 돌아옵니다. 실패한 INSERT 도 번호를 먹습니다
긴 트랜잭션의 영향	좀 느려짐	★★ 청소를 막습니다. 168KB → 1096KB (안 열면 280KB). 3.9배
파일에 20건 동시 쓰기	몇 건 사라짐	1건만 남습니다 (5회 전부)

MySQL

무엇	처음 생각	실제
MySQL 은 관대해서 위험	이상한 값이 조용히 들어감	★★★ MySQL 8.4 는 전부 거절합니다. 타입·길이·범위 ·GROUP BY·CHECK 전부 PG 와 같습니다. 8.0 부터 엄격 모드가 기본입니다
대소문자 무시가 위험	MySQL 이 뚝림	★★★ 반대입니다. 'Admin' 가입 뒤 'admin' 재가입 → PG 는 통과(계정 2개), MySQL 은 1062 로 거절
MySQL 은 뒤 공백을 무시	항상	★ collation 에 따라 다릅니다. 5.7 기본은 무시, 8.0 기본은 구분합니다
'a' 'b' 가 MySQL 에서	0	★ 뒷값이 숫자로 읽히면 1 이 됩니다. "0 이면 문제"로 점검하면 못 잡습니다

★ "MySQL 은 허술하다"고 배웠다면 그건 옛날 이야기입니다. 낡은 정보를 가르치면 안 됩니다.
09단원에서 둘을 나란히 띄워 놓고 직접 재 봅니다.

그 밖에

- **NATURAL JOIN** 은 쓰면 안 됩니다. 칸을 하나씩 추가하니 결과가 $15 \rightarrow 0 \rightarrow 7 \rightarrow 3$ 으로 무너졌습니다. 같은 데이터에 **ON** 조인은 내내 15줄
- **NOT IN** 에 **NULL** 이 섞이면 0건입니다. 에러도 안 납니다
- ★★ **JSONB** 안의 값에는 제약을 못 겁니다. 외래키·UNIQUE·NOT NULL 전부 42601 — "제약 위반" 이 아니라 문법 자체가 없습니다. 그리고 **JSONB** 검색은 생각보다 안 느립니다(20만 줄에서 1.8배). **JSONB** 의 진짜 위험은 느린 게 아니라 틀린 데이터가 들어가는 것입니다
- ★★ 자기 참조에서 "자기가 자기 상위" 인 것을 외래키가 못 막습니다. **UPDATE** 조직 SET 상위번호 = 번호 가 그냥 통과합니다. **CHECK** (상위번호 <> 번호) 가 필요하고, 순환(**A→B→A**)은 **CHECK** 로도 못 막습니다
- ★★ **ON DELETE RESTRICT** 와 **NO ACTION** 은 같은 것이 아닙니다. 에러 코드부터 다릅니다 (23001 vs 23503). **DEFERRABLE INITIALLY DEFERRED** 를 걸면 **NO ACTION** 은 커밋까지 미뤄져 통과하고, **RESTRICT** 는 미뤄도 즉시 막습니다
- **LEFT JOIN** 뒤의 **WHERE** 는 **INNER JOIN** 이 됩니다. ... **OR IS NULL** 로 고치면 절반만 고쳐집니다
- **FROM** 절 서브쿼리 별칭은 **PostgreSQL 16**부터 선택 사항입니다 (MySQL 8.0 은 여전히 에러)
- **SUM(점수) / COUNT(*)** 는 캐스팅 없이 쓰면 정수 나눗셈이라 소수점이 통째로 잘립니다
- **EXPLAIN ANALYZE** 는 **DELETE** 를 진짜로 실행합니다. 66,666건이 진짜 지워졌습니다
- **PGlite** 에서 **COPY** 가 됩니다 (/dev/blob). 10만 건 0.23초
- **console.log** 안의 % 는 **printf** 서식으로 해석됩니다. %95% 가 %95 로 줄어듭니다

표 이름을 한글로 쓰는 이유

```
CREATE TABLE 설비 (  
  id      INTEGER PRIMARY KEY,  
  이름    TEXT NOT NULL,  
  라인    TEXT NOT NULL  
)
```

읽기 쉬우라고 그렇게 했습니다. 회사에서는 영어를 쓰는 곳이 더 많습니다. 팀 규칙을 따르세요.

★ 다만 접속 정보(계정·DB 이름·컨테이너 이름)는 영어입니다. 도구들이 한글을 거절하는 곳이 많습니다. 도커는 컨테이너 이름에 한글이 들어가면 아예 안 만들어 줍니다. `application_name` 에 한글을 넣으면 `pg_stat_activity` 에 16진수로 보입니다.

단 원 01

왜 데이터베이스인가

OPEN 01_왜_데이터베이스인가

```
import fs from "node:fs";
import os from "node:os";
import path from "node:path";
function 설비읽기() {
```

01 파일에 저장하면 생기는 일

02 데이터베이스는 어떻게 다른가

03 어떤 데이터베이스를 고르나

04 SQL 은 어떤 말인가

05 이 자료를 보는 법

- 연습문제

파일에 저장하면 생기는 일

RUN node 개념01_파일에_저장하면_생기는_일.js

★ 이 파일은 10초쯤 걸립니다. 10만 건을 진짜로 만들고 진짜로 잡니다.

데이터베이스를 배우기 전에, 먼저 **없이** 해 봅니다.

설비 목록을 JSON 파일에 저장하는 프로그램입니다. 잘 돌아갑니다. 처음에는요.

이 파일에서 네 가지를 직접 재 봅니다. 전부 실제 측정값입니다.

① 두 사람이 동시에 쓰면 한 사람 것이 사라진다 ② 한 줄 고치려고 전부 다시 쓴다 ③ 하나 찾으려고 전부 읽는다 ④ 파일 크기보다 메모리를 더 먹는다

이 네 가지가 데이터베이스가 존재하는 이유입니다.

```
import fs from "node:fs";
import os from "node:os";
import path from "node:path";
import cp from "node:child_process";
```

실습 파일은 임시 폴더에 만듭니다. 끝나면 지웁니다.

```
const 작업방 = fs.mkdtempSync(path.join(os.tmpdir(), "db자료-01-"));
const 설비파일 = path.join(작업방, "설비.json");
```

잘 돌아가는 것처럼 보입니다

섹션 1

```
fs.writeFileSync(설비파일, JSON.stringify([]));

function 설비읽기() {
  return JSON.parse(fs.readFileSync(설비파일, "utf8"));
}

function 설비쓰기(목록) {
  fs.writeFileSync(설비파일, JSON.stringify(목록));
}

function 설비추가(이름, 라인) {
  const 목록 = 설비읽기();
  const 새번호 = 목록.length === 0 ? 1 : Math.max(...목록.map((설비) => 설비.id)) + 1;
}
```

```

목록.push({ id: 새번호, 이름, 라인, 상태: "정지" });
설비쓰기(목록);

return 새번호;
}

```

```

설비추가("컨베이어 1호", "A");
설비추가("프레스 1호", "B");
설비추가("용접로봇 1호", "C");

```

```

console.log(JSON.stringify(설비읽기()));

```

```

출력      [{"id":1,"이름":"컨베이어 1호","라인":"A","상태":"정지"},
           {"id":2,"이름":"프레스 1호","라인":"B","상태":"정지"},
           {"id":3,"이름":"용접로봇 1호","라인":"C","상태":"정지"}]

```

30줄로 저장 기능이 됐습니다. 라이브러리도 설치 안 했습니다.

★ 이게 나쁜 코드라는 말이 아닙니다. 설정 파일이나 혼자 쓰는 도구라면 이걸로 충분합니다. 문제는 **사람이** **둘 이상** 되거나 **데이터가 늘어날 때** 시작됩니다.

★ 동시에 쓰면 사라집니다

섹션 2

현장에서 이런 일이 벌어집니다.

김반장이 점검 결과를 올립니다. 같은 순간 이반장도 올립니다.

코드로 보면 이렇습니다.

① 김반장: 파일을 읽는다 (3건이 들어 있음) ② 이반장: 파일을 읽는다 (역시 3건) ③ 김반장: 4건으로 만들어 쓴다 ④ 이반장: 4건으로 만들어 쓴다 ← ③을 덮어씁니다

```

출력      둘 다 성공했다고 나오는데 **한 건이 사라집니다.**

```

진짜로 재 봅니다. 20명이 동시에 올리게 합니다.

```

async function 느긋하게추가(값) {
  const 목록 = 설비읽기();

```

★ 읽고 쓰는 사이의 '틈' 입니다. 진짜 서버에서는 이 틈이 저절로 생깁니다. 네트워크를 기다리거나, 다른 요청이 끼어들거나, 디스크를 기다립니다. 여기서는 그 틈을 1밀리초로 흉내 냅니다.

```

await new Promise((resolve) => setTimeout(resolve, 1));

목록.push(값);
설비쓰기(목록);

```

```

}

const 소실실험 = [];

for (let 회차 = 1; 회차 <= 5; 회차 += 1) {
  설비쓰기([]);

  await Promise.all(
    Array.from({ length: 20 }, (_, 번호) => 느긋하게추가({ id: 번호, 이름: `점검 ${번호}` })),
  );

  소실실험.push(설비읽기().length);
}

console.log("20건을 동시에 올린 뒤 남은 건수 (5회):", 소실실험.join(", "));

```

출력 20건을 동시에 올린 뒤 남은 건수 (5회): 1, 1, 1, 1, 1
값이 다를 수 있음

```

console.log("20건 전부 남았나:", 소실실험.every((건수) => 건수 === 20));

```

출력 20건 전부 남았나: false

★★★ 20건을 올렸는데 1건만 남았습니다. 5번 다 그랬습니다.

19건이 **조용히** 사라졌습니다. 에러도 없고, 로그도 없고, 아무도 모릅니다. 김반장은 “올렸는데요?” 라고 하고, 화면에는 없습니다.

★ 이게 가장 무서운 종류의 버그입니다.

터지는 버그는 고칠 수 있습니다. 조용한 버그는 몇 달 뒤에 발견됩니다.

★ “그럼 줄을 세우면 되잖아요” 맞습니다. 한 번에 한 명씩만 쓰게 하면 됩니다. 그런데 그러면 서버가 한 대일 때만 통합니다. 서버를 두 대 띄우는 순간 각자 줄을 서게 되어 다시 깨집니다.

데이터베이스는 이 문제를 **트랜잭션과 잠금**으로 푼다. 07단원에서 합니다.

10만 건을 만듭니다

섹션 3

```

const 건수 = 100000;
const 라인 = ["A", "B", "C"];

const 큰파일 = path.join(작업방, "설비10만.json");

fs.writeFileSync(

```



```

큰파일,
  JSON.stringify(
    Array.from({ length: 건수 }, (_, 자리) => ({
      id: 자리 + 1,
      이름: `설비${자리 + 1}`,
      라인: 라인[자리 % 3],
      상태: "가동",
    })),
  ),
);

const 메가바이트 = fs.statSync(큰파일).size / 1024 / 1024;

console.log(`${건수.toLocaleString()}건 파일 크기: ${메가바이트.toFixed(1)} MB`);

```

출력 100,000건 파일 크기: 6.4 MB
값이 다를 수 있음

6MB 면 별것 아닌 것처럼 보입니다. 그런데 아래를 보세요.

★ 한 줄 고치려고 전부 다시 씁니다

섹션 4

설비 40000번의 상태를 '정지' 로 바꾸겠습니다. **딱 한 글자** 바꿉니다.

```

function 재기(횟수, 할일) {
  const 잔것 = [];

  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    할일(회차);
    잔것.push(performance.now() - 시작);
  }
}

```

중앙값을 씁니다. 평균은 어쩌다 한 번 느린 것에 크게 흔들립니다.

```

return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}

const 파일수정ms = 재기(3, (회차) => {
  const 목록 = JSON.parse(fs.readFileSync(큰파일, "utf8")); // 6.4MB 를 전부 읽고
  목록.find((설비) => 설비.id === 40000 + 회차).상태 = "정지"; // 한 글자 바꾸고
  fs.writeFileSync(큰파일, JSON.stringify(목록)); // 6.4MB 를 전부 다시
  씁니다
});

console.log(`1건 수정: ${파일수정ms.toFixed(0)} ms`);

```

출력 1건 수정: 92 ms
값이 다를 수 있음

★★ 한 글자 바꾸려고 6.4MB 를 읽고 6.4MB 를 썼습니다.

바뀐 양 : 2글자 ("가동" → "정지") 오간 양 : 12.8 MB 낭비 비율 : 약 600만 배

그리고 이건 데이터가 늘면 그대로 느려집니다. 100만 건이면 65MB 를 읽고 65MB 를 씁니다. 아래에서 확인합니다.

★ 하나 찾으려고 전부 읽습니다

섹션 5

```
const 한건조회ms = 재기(3, () => {
  JSON.parse(fs.readFileSync(큰파일, "utf8")).find((설비) => 설비.id === 77777);
});

console.log(`1건 조회: ${한건조회ms.toFixed(0)} ms`);
```

출력 1건 조회: 61 ms
값이 다를 수 있음

설비 하나를 보려고 10만 건을 전부 읽었습니다.

★ "77777번 하나만 읽으면 안 되나요?" 안 됩니다. JSON 파일은 **중간부터 읽을 수가 없습니다.**

77777번이 파일의 어디쯤 있는지 알아내려면 앞에서부터 세어야 하고, 세려면 결국 다 읽어야 합니다.

데이터베이스는 **색인** 으로 이 문제를 푼다. 06단원에서 합니다.

```
const 조건검색ms = 재기(3, () => {
  JSON.parse(fs.readFileSync(큰파일, "utf8")).filter(
    (설비) => 설비.라인 === "A" && 설비.상태 === "가동",
  ).length;
});

console.log(`조건 검색: ${조건검색ms.toFixed(0)} ms`);
```

출력 조건 검색: 59 ms
값이 다를 수 있음

★ 파일 크기보다 메모리를 훨씬 더 먹습니다

섹션 6

6.4MB 파일이니 메모리도 6.4MB 쯤 쓰겠지 — 하고 생각하기 쉽습니다. 재 봅니다.

★★ 그런데 여기서 조심할 게 있습니다. 이 파일은 이미 위에서 10만 건을 여러 번 파싱했습니다. 그 상태에서 heapUsed 를 재면 **0 MB 로 나옵니다.** 앞서 쓴 메모리를 아직 안 치웠거나, 재는 사이에 치워 버리기 때문입니다.

실제로 이 자료를 쓰면서 그 값이 먼저 나왔고, 하마터면 그대로 실을 뻔했습니다.

★ 메모리는 **아무것도 안 한 깨끗한 프로세스**에서 재야 합니다.

그래서 node 를 새로 하나 띄워서 잡니다.

```
const 재는코드 = `
  const fs = require("node:fs");
  const 전 = process.memoryUsage().heapUsed;
  const 목록 = JSON.parse(fs.readFileSync(process.argv[1], "utf8"));
  const 늘어난MB = (process.memoryUsage().heapUsed - 전) / 1024 / 1024;
  console.log(늘어난MB.toFixed(0) + " " + 목록.length);
`;

const 잔것 = [];

for (let 회차 = 0; 회차 < 3; 회차 += 1) {
  잔것.push(cp.execFileSync(process.execPath, ["-e", 재는코드, 큰파일], { encoding:
    "utf8" })).trim());
}

const [늘어난MB, 배열길이] = 잔것[0].split(" ");

console.log("세 번 잔 값:", 잔것.join(" | "));
```

출력 세 번 잔 값: 18 100000 | 18 100000 | 18 100000
값이 다를 수 있음

```
console.log("세 번 다 같은가:", new Set(잔것).size === 1);
```

출력 세 번 다 같은가: true

```
console.log(`배열 길이 ${배열길이} / 파일 ${메가바이트.toFixed(1)} MB → 메모리
${늘어난MB} MB`);
```

출력 배열 길이 100000 / 파일 6.4 MB → 메모리 18 MB
값이 다를 수 있음

```
console.log("메모리가 파일보다 큰가:", Number(늘어난MB) > 메가바이트);
```

출력 메모리가 파일보다 큰가: true

★★ 파일 6.4MB 가 메모리에서 18MB 입니다. **2.8배** 입니다.

자바스크립트 객체는 키 이름, 포인터, 객체 헤더를 각자 들고 있습니다. { id: 1, 이름: "설비1", 라인: "A", 상태: "가동" } 하나가 글자로는 50바이트인데 메모리에서는 180바이트쯤 됩니다.

★ 100만 건으로도 재 봤습니다. 65.6MB 파일 → 힙 185MB. 역시 2.8배였습니다.

비율이 일정합니다. 그러니 ****파일 크기 × 3**** 으로 잡으면 됩니다.

왜 중요한가:

클라우드에서 빌리는 작은 서버는 메모리가 1GB 입니다.
500만 건이면 파일은 330MB 인데 메모리는 900MB 를 넘깁니다.
그 서버에서는 ****파일을 열어 보는 것만으로 죽습니다.****

★ 데이터베이스는 전부 메모리에 올리지 않습니다. 필요한 쪽(page)만 골라서 읽습니다. 그래서 1GB 서버로 100GB 를 다룹니다.

규모가 커지면 어떻게 되나

섹션 7

위에서 켜진 것은 10만 건입니다. 100만 건으로도 재 봤습니다. (이 파일에서는 시간이 오래 걸려 생략합니다)

무엇 10만 건 100만 건 늘어난 배수

파일 크기 6.4 MB 65.6 MB 10배 1건 수정 92 ms 1,467 ms 16배 1건 조회 61 ms 776 ms 13배 조건
검색 59 ms 778 ms 13배 메모리 18 MB 185 MB 10배

★★★ 데이터가 10배 늘면 **모든 작업이 10배 이상 느려집니다.**

이걸 "선형으로 느려진다" 고 합니다. 최악은 아니지만 좋지도 않습니다. 손님이 10배 늘었는데 응답이 10배 느려지면 서비스는 죽은 것입니다.

★ 데이터베이스는 어떤가 같은 실험에서 데이터베이스의 1건 수정은 **0.5ms → 0.4ms** 였습니다. 10배로 늘렸는데 **느려지지 않았습니다.** 개념02 에서 직접 재 봅니다.

그 밖에 파일이 못 하는 것

섹션 8

```
const 못하는것 = [  
  ["규칙을 못 지킴", "라인 칸에 'A' 대신 숫자 999 를 넣어도 파일은 받아 줍니다"],  
  ["중간 실패를 못 되돌림", "설비를 지우다 중간에 죽으면 점검기록만 남습니다"],  
  ["관계를 표현 못 함", "없는 설비의 점검기록을 만들어도 아무도 안 막습니다"],  
  ["여러 대에서 못 씀", "서버 2대가 각자의 파일을 갖게 됩니다"],  
  ["과거를 못 봄", "누가 언제 뭘 바꿨는지 기록이 없습니다"],  
];  
  
for (const [무엇, 설명] of 못하는것) {  
  console.log(`· ${무엇} - ${설명}`);  
}
```

- 출력
- 규칙을 못 지킴 - 라인 칸에 'A' 대신 숫자 999 를 넣어도 파일은 받아 줍니다
 - 중간 실패를 못 되돌림 - 설비를 지우다 중간에 죽으면 점검기록만 남습니다
 - 관계를 표현 못 함 - 없는 설비의 점검기록을 만들어도 아무도 안 막습니다
 - 여러 대에서 못 씀 - 서버 2대가 각자의 파일을 갖게 됩니다
 - 과거를 못 봄 - 누가 언제 뭘 바꿨는지 기록이 없습니다

★ 표처럼 칸을 맞추고 싶어서 padEnd(12) 를 쓰면 한글이 어긋납니다. 한글 한 자는 터미널에서 두 칸을 차지하는데 padEnd 는 한 칸으로 셉니다. 칸을 맞추고 싶으면 글자 폭을 직접 세야 합니다. 이 자료에서는 그냥 · 나 — 로 잇습니다. 표는 주석으로 그립니다.

★ 이걸 전부 코드로 막을 수는 있습니다. 그런데 그렇게 만들다 보면 **데이터베이스를 다시 만들게 됩니다.** 그것도 훨씬 못 만든 것일요.

실습에 쓴 임시 폴더를 지웁니다.

```
fs.rmSync(작업방, { recursive: true, force: true });
```

정리 — 파일 저장의 다섯 가지 한계

한계 무엇이 문제인가 어디서 풀리나

동시에 쓰면 사라짐 20건 중 1건만 남음 07만원 트랜잭션 한 줄 고치려고 전부 씀 6.4MB 를 읽고 다시 씀
개념02 하나 찾으려고 전부 읽음 10만 건을 전부 훑음 06만원 색인 규칙을 못 지킴 이상한 값이 그냥 들어감
02만원 제약 관계를 표현 못 함 고아 데이터가 생김 04만원 외래키

★ 데이터베이스는 이 다섯 가지를 풀려고 만들어진 것입니다. “요즘 다 쓰니까” 가 아니라, **재 보면 이렇게 차이가 나서 씁니다.**

직접 해 볼 것

▫ 직접 해보기 1

섹션 2 의 setTimeout 을 지우면 어떻게 되나요? 20건이 전부 남나요? 왜 그럴까요? (힌트: 틸이 없으면 안 겹칩니다. 그런데 진짜 서버에는 틸이 있습니다)

▫ 직접 해보기 2

건수를 20 에서 200 으로 늘려 보세요. 몇 건이 남나요?

▫ 직접 해보기 3

섹션 3 의 건수를 100만으로 바꿔 보세요. 몇 초나 걸리나요? 노트북이 버티나요? ★ 메모리가 작으면 프로그램이 죽을 수 있습니다. 각오하고 하세요.

▫ 직접 해보기 4

섹션 4 에서 수정 대상을 40000 대신 1 로 바꿔 보세요. 앞쪽이라 더 빠를까요? 재 보세요. (힌트: find 는 빨라지지만 읽고 쓰는 양은 그대로입니다)

▫ 직접 해보기 5

설비 목록을 JSON 대신 CSV 로 저장하면 위 문제 중 몇 개가 풀리나요? (한 줄만 고쳐 쓸 수 있을까요?)

자주 하는 실수

이런 실수를 합니다

“우리 서비스는 작아서 파일로 충분해요”

작을 때는 맞습니다. 문제는 커진 다음에 옮기기가 훨씬 어렵다 는 것입니다. 데이터가 쌓인 뒤에 옮기면 멈춤 시간이 필요합니다.

이런 실수를 합니다

동시성 문제를 테스트로 못 잡음

혼자 눌러 보면 절대 안 납니다. 섹션 2 처럼 일부러 겹쳐야 납니다. 그래서 운영에서 처음 발견됩니다.

이런 실수를 합니다

파일 크기만 보고 메모리를 판단함

6.4MB 파일이 메모리에서 18MB 입니다. 세 배로 잡으세요. 그리고 메모리는 깨끗한 프로세스에서 재세요. 안 그러면 0 이 나옵니다.

이런 실수를 합니다

“락을 걸면 되잖아” 로 끝냄

서버 한 대에서는 됩니다. 두 대가 되면 다시 깨집니다. 그리고 락을 잘못 걸면 데드락이 납니다. 07단원에서 만들어 봅니다.

이런 실수를 합니다

백업을 파일 복사로 함

복사하는 중에 누가 쓰면 반쯤 쓰인 파일 이 복사됩니다. 그 백업은 복구할 때 깨져 있습니다. 10단원에서 제대로 합니다.

데이터베이스는 어떻게 다른가

RUN node 개념02_데이터베이스는_어떻게_다른가.js

01

★ 이 파일은 10초쯤 걸립니다. 10만 건과 100만 건을 진짜로 만들고 진짜로 켜니다.

개념01 에서 파일 저장의 한계를 **재** 봤습니다.

1건 수정 92 ms (6.4MB 를 읽고 6.4MB 를 다시 씀) 1건 조회 61 ms (10만 건을 전부 읽음) 조건 검색 59 ms
동시 쓰기 20건 중 1건만 남음

이번에는 **똑같은 실험을 데이터베이스로** 합니다. 같은 10만 건, 같은 작업, 같은 재는 방법입니다. 숫자만 나란히 놓고 봅니다.

그리고 마지막에 한 가지를 정직하게 짚습니다. **데이터베이스가 언제나 빠른 것은 아닙니다.** 그것도 재서 보여 드립니다.

```
import { PGLite } from "@electric-sql/pglite";
```

첫 데이터베이스를 띄웁니다

섹션 1

설치할 게 없습니다. npm install 은 이미 돼 있습니다. PGLite 는 **PostgreSQL** 을 통째로 **WebAssembly** 로 컴파일한 것입니다.

```
const 시작할때 = performance.now();
const db = await PGLite.create();
const 뜨는데걸린ms = performance.now() - 시작할때;

console.log(`데이터베이스가 뜨는 데: ${뜨는데걸린ms.toFixed(0)} ms`);
```

출력 데이터베이스가 뜨는 데: 2256 ms
값이 다를 수 있음

★ **PGLite.create()** 에 아무것도 안 주면 **메모리에만** 만듭니다. 프로그램이 끝나면 사라집니다. 실습에 딱 맞습니다.

폴더에 남기려면 이름을 줍니다.

```
await PGLite.create("./내디비")
```

★ 파일이 아니라 **폴더**가 생깁니다. 그 안에 PostgreSQL 가 쓰는 것들이 들어갑니다.

진짜 PostgreSQL 인지 물어봅니다. 데이터베이스에게 직접 묻습니다.

```
const 버전 = await db.query("SELECT version()");

console.log(버전.rows[0].version.split(" on ")[0]);
```

출력 PostgreSQL 18.3 (PGLite 0.5.7)

★★ 진짜 PostgreSQL 18.3 입니다. 흥내가 아닙니다.

회사 서버에서 도는 것과 같은 소스코드를 브라우저·Node 에서 돌게 만든 것입니다. 여기서 배운 SQL 은 그대로 회사 서버에서 통합니다.

★ `db.query()` 가 돌려주는 것은 이렇게 생겼습니다.

```
console.log(JSON.stringify(Object.keys(버전)));
```

출력 ["rows", "fields", "command", "affectedRows", "rowCount"]

rows	결과 줄들. 그냥 자바스크립트 객체 배열입니다
fields	칸 이름과 타입 번호
command	무슨 명령이었나 ("SELECT", "UPDATE" ...)
affectedRows	몇 줄이 바뀌었나
rowCount	몇 줄이 나왔나

개념01 과 똑같은 10만 건을 만듭니다

섹션 2

```
await db.exec(`
  CREATE TABLE 설비 (
    id INT PRIMARY KEY,
    이름 TEXT NOT NULL,
    라인 TEXT NOT NULL,
    상태 TEXT NOT NULL
  )
`);
```

★ `db.exec()` 는 세미콜론으로 이은 여러 문장을 받습니다. `db.query()` 는 한 문장만 받습니다. 이 차이는 개념04 에서 자세히 봅니다.

표가 무엇인지, PRIMARY KEY 가 무엇인지는 02단원에서 제대로 합니다. 지금은 “칸 이름과 타입을 미리 정해 둔 것” 정도로 보시면 됩니다.


```
const 넣는데걸린 = performance.now();

await db.exec(`
  INSERT INTO 설비 (id, 이름, 라인, 상태)
  SELECT 번호, '설비' || 번호, (ARRAY['A','B','C'])[1 + (번호 - 1) % 3], '가동'
  FROM generate_series(1, 100000) AS 번호
`);

console.log(`10만 건 넣는 데: ${(performance.now() - 넣는데걸린).toFixed(0)} ms`);
```

```
출력      10만 건 넣는 데: 352 ms
값이 다를 수 있음
```

```
const 건수 = await db.query("SELECT count(*)::int AS 건수 FROM 설비");

console.log("들어간 건수:", 건수.rows[0].건수);
```

```
출력      들어간 건수: 100000
```

★ `count(*)` 뒤에 붙인 `::int` 는 “정수로 바꿔 달라” 는 뜻입니다. 왜 붙일까요?

`count(*)` 의 타입은 **BIGINT** 입니다. 그리고 BIGINT 가 자바스크립트로 올 때는 **값에 따라 타입이 바뀝니다**. 재 봤습니다.

```
const 타입시험 = await db.query(
  "SELECT count(*) AS 작은값, 9007199254740993::bigint AS 큰값 FROM 설비",
);

console.log(`작은값: ${typeof 타입시험.rows[0].작은값} / 큰값: ${typeof
타입시험.rows[0].큰값}`);
```

```
출력      작은값: number / 큰값: bigint
```

★★ 안전 정수(2의 53승 - 1)를 넘으면 뒤에 n 이 붙은 **bigint** 로 옵니다. 그리고 bigint 는 `JSON.stringify` 가 처리하지 못합니다. 그 자리에서 터집니다.

```
"Do not know how to serialize a BigInt"
```

★★★ 무서운 것은 **작을 때는 멀쩡하다가 커지면 터진다** 는 점입니다.

```
테스트 데이터로는 절대 안 걸리고, 몇 년 뒤 운영에서 터집니다.
`::int` 를 붙여서 미리 못을 박아 두세요. 02단원에서 자세히 봅니다.
```

개념01에서는 한 글자 바꾸려고 6.4MB를 읽고 6.4MB를 다시 썼습니다. 데이터베이스에게는 이렇게 말합니다.

```
async function 재기(횟수, 할일) {
  const 잔것 = [];

  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    await 할일(회차);
    잔것.push(performance.now() - 시작);
  }
}
```

개념01과 같은 방식입니다. 중앙값을 씁니다.

```
return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}

const db수정ms = await 재기(9, (회차) =>
  db.query("UPDATE 설비 SET 상태 = $1 WHERE id = $2", ["정지", 40000 + 회차]),
);

console.log(`1건 수정 - DB: ${db수정ms.toFixed(2)} ms (파일은 92 ms 였습니다)`);
```

출력 1건 수정 - DB: 0.34 ms (파일은 92 ms 였습니다)
값이 다를 수 있음

```
console.log("DB 가 파일보다 빠른가:", db수정ms < 92);
```

출력 DB 가 파일보다 빠른가: true

★ \$1, \$2는 “여기에 값이 들어갑니다”라는 자리입니다. 두 번째 인자로 넘긴 배열이 순서대로 채워집니다.

★★★ 값을 글자로 이어 붙이면 절대 안 됩니다.

```
`WHERE id = ${입력값}` 이렇게 쓰면 SQL 인젝션이 뚫립니다.
08단원에서 진짜로 뚫어 보니까.
```

MySQL은 여기가 다릅니다

· 자리 표시가 \$1이 아니라 ?입니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

★ 1건 조회와 조건 검색

섹션 4

```
const db한건ms = await 재기(9, () => db.query("SELECT * FROM 설비 WHERE id = $1", [77777]));
```

```
console.log(`1건 조회 - DB: ${db한건ms.toFixed(2)} ms (파일은 61 ms 였습니다)`);
```

```
출력      1건 조회 - DB: 0.25 ms (파일은 61 ms 였습니다)
값이 다를 수 있음
```

```
const db조건ms = await 재기(9, () =>
  db.query("SELECT count(*) FROM 설비 WHERE 라인 = $1 AND 상태 = $2", ["A", "가동"],
);
```

```
console.log(`조건 검색 - DB: ${db조건ms.toFixed(2)} ms (파일은 59 ms 였습니다)`);
```

```
출력      조건 검색 - DB: 16.37 ms (파일은 59 ms 였습니다)
값이 다를 수 있음
```

```
console.log("셋 다 파일보다 빠른가:", db수정ms < 92 && db한건ms < 61 && db조건ms < 59);
```

```
출력      셋 다 파일보다 빠른가: true
```

★ 여기서 중요한 것을 눈치채셨으면 좋겠습니다.

1건 조회는 0.25 ms 인데 조건 검색은 16 ms 입니다. **60배 넘게** 차이가 납니다. 같은 표, 같은 10만 건인데 왜 이렇게 다를까요?

· WHERE id = 77777 — id 는 PRIMARY KEY 입니다. 색인이 이미 있습니다.

```
10만 건을 안 훑고 바로 찾아갑니다.
```

· WHERE 라인 = 'A' AND 상태 = '가동' — 색인이 없습니다.

```
10만 건을 **전부 훑습니다.** 파일과 똑같은 방식으로요.
```

그런데도 파일(59 ms)보다 3배 빠릅니다. JSON 을 자바스크립트 객체로 바꾸는 일이 없기 때문입니다.

★ 색인을 붙이면 어떻게 되는지, 색인이 어떻게 생겼는지는 06단원에서 합니다.

★★★ 여기가 진짜 차이입니다 — 10배로 늘려도 안 느려집니다

섹션 5

개념01 에서 파일은 10배 늘리자 **16배 느려졌습니다.** (92 ms → 1,467 ms) 같은 일을 데이터베이스에게 시켜 봅니다. 90만 건을 더 넣습니다.

```
const 더넣는데 = performance.now();

await db.exec(`
  INSERT INTO 설비 (id, 이름, 라인, 상태)
  SELECT 번호, '설비' || 번호, (ARRAY['A','B','C'])[1 + (번호 - 1) % 3], '가동'
  FROM generate_series(100001, 1000000) AS 번호
`);

console.log(`90만 건 더 넣는 데: ${performance.now() - 더넣는데}.toFixed(0)} ms`);
```

출력 90만 건 더 넣는 데: 3145 ms
값이 다를 수 있음

```
const 큰건수 = await db.query("SELECT count(*)::int AS 건수 FROM 설비");

console.log("이제 몇 건:", 큰건수.rows[0].건수);
```

출력 이제 몇 건: 1000000

```
const db수정100만ms = await 재기(9, (회차) =>
  db.query("UPDATE 설비 SET 상태 = $1 WHERE id = $2", ["정지", 400000 + 회차]),
);

const db한건100만ms = await 재기(9, () => db.query("SELECT * FROM 설비 WHERE id =
$1", [777777]));

console.log(`1건 수정 - 10만 건: ${db수정ms.toFixed(2)} ms → 100만 건:
${db수정100만ms.toFixed(2)} ms`);
```

출력 1건 수정 - 10만 건: 0.34 ms → 100만 건: 0.32 ms
값이 다를 수 있음

```
console.log(`1건 조회 - 10만 건: ${db한건ms.toFixed(2)} ms → 100만 건:
${db한건100만ms.toFixed(2)} ms`);
```

출력 1건 조회 - 10만 건: 0.25 ms → 100만 건: 0.55 ms
값이 다를 수 있음

```
console.log("데이터가 10배 늘었는데 1건 수정이 3배 안에 머물렀나:", db수정100만ms <
db수정ms * 3);
```

출력 데이터가 10배 늘었는데 1건 수정이 3배 안에 머물렀나: true

★★★ 데이터가 10배 늘었는데 수정 시간은 그대로입니다.

파일 92 ms → 1,467 ms (16배 느려짐) 데이터베이스 0.3 ms → 0.3 ms (안 느려짐)

이게 데이터베이스를 쓰는 **가장 큰 이유**입니다. 빨라서가 아니라, **커져도 안 느려져서**입니다.

왜 그럴까요?

파일은 "id 40000 을 고쳐라" 를 받으면 처음부터 끝까지 읽어야 합니다.
데이터베이스는 색인을 타고 **그 줄이 든 8KB 조각(page) 하나만** 읽습니다.
10만 건이든 1억 건이든 읽는 조각 수가 거의 같습니다.

★ 이 성질을 "규모에 안 흔들린다" 고 합니다.

손님이 10배 늘어도 응답 시간이 그대로라는 뜻입니다.
서비스에서는 이게 전부입니다.

조건 검색은 어떨까요? 이걸 정직하게 말씀드려야 합니다.

```
const db조건100만ms = await 재기(5, () =>
  db.query("SELECT count(*) FROM 설비 WHERE 라인 = $1 AND 상태 = $2", ["A", "가동"]),
);

console.log(`조건 검색 - 10만 건: ${db조건ms.toFixed(0)} ms → 100만 건:
${db조건100만ms.toFixed(0)} ms`);
```

출력 조건 검색 - 10만 건: 16 ms → 100만 건: 162 ms
값이 다를 수 있음

```
console.log("조건 검색은 느려졌나:", db조건100만ms > db조건ms * 3);
```

출력 조건 검색은 느려졌나: true

★★ 조건 검색은 느려졌습니다. 색인이 없으니까요.

색인 없는 검색은 데이터베이스도 전부 훑습니다. 파일과 같습니다. (그래도 파일 778 ms 보다는 빠릅니다)

★ 그래서 06단원이 있습니다. 색인을 붙이면 이 시간이 어떻게 줄어드는지,

그리고 색인을 잘못 붙이면 왜 더 느려지는지 거기서 봅니다.

"데이터베이스를 쓰면 알아서 빨라진다" 는 말은 틀렸습니다. 빠르게 만들 도구가 있는 것뿐입니다. 쓸 줄 알아야 빨라집니다.

★ 동시에 써도 안 사라집니다

섹션 6

개념01 의 그 실험입니다. 20명이 동시에 올리면 1건만 남았습니다. 데이터베이스로 똑같이 해 봅니다.

```
await db.exec("CREATE TABLE 점검기록 (id INT PRIMARY KEY, 내용 TEXT NOT NULL)");

const 남은건수 = [];
```

```

for (let 회차 = 1; 회차 <= 5; 회차 += 1) {
  await db.exec("TRUNCATE 점검기록");

  await Promise.all(
    Array.from({ length: 20 }, (_, 번호) =>
      db.query("INSERT INTO 점검기록 (id, 내용) VALUES ($1, $2)", [번호, `점검
${번호}`])),
    ),
  );

  const 센것 = await db.query("SELECT count(*)::int AS 건수 FROM 점검기록");
  남은건수.push(센것.rows[0].건수);
}

console.log("20건을 동시에 올린 뒤 남은 건수 (5회):", 남은건수.join(", "));

```

출력 20건을 동시에 올린 뒤 남은 건수 (5회): 20, 20, 20, 20, 20

```

console.log("20건 전부 남았나:", 남은건수.every((건수) => 건수 === 20));

```

출력 20건 전부 남았나: true

★★★ 파일은 1건, 데이터베이스는 20건입니다.

코드는 거의 똑같습니다. `Promise.all` 로 20개를 한꺼번에 던진 것도 같습니다. 다른 것은 **받는 쪽**뿐입니다.

파일: 읽는다 → (틈) → 쓴다. 그 틈에 남이 끼어들면 덮어씹니다 데이터베이스: "20번 줄을 넣어라" 를 받고 **그 줄만** 넣습니다

전체를 다시 쓰지 않으니 남의 것을 덮을 일이 없습니다

★ 그리고 진짜 무기는 따로 있습니다. **트랜잭션**입니다.

"이 세 가지를 전부 하거나, 하나도 안 한 것으로 하거나" 를 보장합니다.
07단원에서 합니다.

★★ 솔직하게: PGlite 는 연결이 하나뿐이라 위 실험은 **줄 서서** 처리됐습니다.

진짜 동시성은 아닙니다. 그래도 파일과 달리 ****한 건도 안 사라진다**** 는 것은 보여 줍니다. 진짜 동시 접속 실험은 07단원에서 **Docker** 로 합니다.

★★ 정직하게 — 데이터베이스가 항상 빠르지는 않습니다

섹션 7

여기까지만 보면 "데이터베이스가 무조건 이긴다" 로 읽힙니다. 아닙니다.

개념01 의 파일 측정에는 **파일을 읽어서 파싱하는 시간**이 들어 있습니다. 그 시간을 빼면 어떻게 될까요? 즉, 데이터가 **이미 메모리에 있으면** 어떨까요?

재 봤습니다. 10만 건짜리 자바스크립트 배열을 미리 만들어 놓고 filter 합니다.

```
const 메모리배열 = Array.from({ length: 100000 }, (_, 자리) => ({
  id: 자리 + 1,
  이름: `설비${자리 + 1}`,
  라인: ["A", "B", "C"][자리 % 3],
  상태: "가동",
}));

function 동기재기(횟수, 할일) {
  const 잔것 = [];

  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    할일();
    잔것.push(performance.now() - 시작);
  }

  return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}

const 메모리filterms = 동기재기(9, () => {
  메모리배열.filter((설비) => 설비.라인 === "A" && 설비.상태 === "가동").length;
});

console.log(`조건 검색 - 메모리 배열: ${메모리filterms.toFixed(2)} ms / DB:
${db조건ms.toFixed(2)} ms`);
```

출력 조건 검색 - 메모리 배열: 2.03 ms / DB: 16.37 ms
값이 다를 수 있음

```
console.log("메모리 배열이 DB보다 빠른가:", 메모리filterms < db조건ms);
```

출력 메모리 배열이 DB보다 빠른가: true

★★★ 메모리에 이미 올려 둔 배열의 filter 가 데이터베이스보다 몇 배 빠릅니다.

이 자료를 만들면서 이 값이 나왔고, 하마터면 “DB 가 항상 빠르다” 고 쓸 뻔했습니다. 당연합니다. 배열 filter 는 그냥 메모리를 훑는 것이고, 데이터베이스는 SQL 을 해석하고, 계획을 세우고, 페이지를 읽고, 결과를 돌려줍니다. 그 준비 비용이 있습니다.

★ 그럼 왜 데이터베이스를 쓰나요? 세 가지 때문입니다.

① 메모리에 안 들어갑니다

10만 건은 18MB 라 올라갑니다. 5,000만 건은 9GB 입니다. 안 올라갑니다.
데이터베이스는 필요한 페이지만 읽어서 1GB 서버로 100GB 를 다룹니다.

② 동시에 쓰면 배열도 깨집니다

섹션 6 의 실험을 배열로 해도 결과는 같습니다.
서버가 두 대가 되면 배열은 두 개가 되고, 둘은 영원히 안 맞습니다.

③ 꺼지면 사라집니다

배열은 프로세스가 죽으면 없습니다. 재배포만 해도 없어집니다.

★ 그리고 실제로는 둘 다 씁니다.

자주 보는 것을 메모리에 올려 두고(캐시), 진짜 데이터는 데이터베이스에 둡니다.
"캐시가 틀렸을 때 누구 말이 맞나" 를 정하는 게 설계입니다.

한 건 찾기는 어떨까요? 이걸 뒤집습니다.

```
const 메모리findms = 동기재기(9, () => 메모리배열.find((설비) => 설비.id === 77777));

console.log(`1건 조회 - 메모리 배열: ${메모리findms.toFixed(2)} ms / DB:
${db한건ms.toFixed(2)} ms`);
```

출력 1건 조회 - 메모리 배열: 0.64 ms / DB: 0.25 ms
값이 다를 수 있음

★ 여기서 "DB 가 더 빠르다" 를 그대로 판정으로 쓰면 안 됩니다. 차이가 두 배쯤이라, 컴퓨터가 바쁘면 뒤집힙니다. 실제로 이 자료의 검증기가 70개 파일을 연달아 돌릴 때 뒤집혀서 한 번 깨졌습니다.

그래서 **격차가 뒤집혔는지**를 봅니다. 이걸 안 흔들립니다.

조건 검색 : 배열이 DB 보다 여덟 배쯤 빠릅니다
1건 조회 : DB 가 배열보다 빠릅니다

두 비율은 십수 배 차이라 웬만한 부하로는 안 뒤집힙니다.

```
const 조건검색_배열대비 = 메모리filterms / db조건ms;
const 한건조회_배열대비 = 메모리findms / db한건ms;

console.log("조건 검색에서 벌어졌던 격차가 1건 조회에서 뒤집혔나:", 한건조회_배열대비
> 조건검색_배열대비);
```

출력 조건 검색에서 벌어졌던 격차가 1건 조회에서 뒤집혔나: true

★ 한 건 찾기는 **데이터베이스가 이깁니다**. 배열의 find 는 앞에서부터 하나씩 셉니다. 77,777번이면 77,777번 셉니다. 데이터베이스는 색인을 타고 17번쯤 비교하면 도착합니다.

★★ 정리하면 이렇습니다.

- 전부 훑는 일 → 메모리 배열이 빠름 (준비 비용이 없음)
- 골라 찾는 일 → 데이터베이스가 빠름 (색인이 있음)
- 커지는 일 → 데이터베이스만 버팀
- 안 사라지는 일 → 데이터베이스만 됨

왜 이렇게 되나 — 세 가지 장치

섹션 8

```
const 장치들 = [
  ["필요한 페이지만 읽는다", "8KB 조각 단위로 읽습니다. 전부 안 올립니다", "06만원"],
  ["색인이 있다", "10만 건에서 17번 비교로 찾아갑니다", "06만원"],
  ["트랜잭션이 있다", "전부 하거나 하나도 안 하거나를 보장합니다", "07만원"],
  ["규칙을 지킨다", "이상한 값은 넣는 순간 거절합니다", "02만원"],
  ["관계를 안다", "없는 설비의 점검기록을 못 만들게 합니다", "04만원"],
];

for (const [이름, 설명, 단위] of 장치들) {
  console.log(`· ${이름} - ${설명} (${단위})`);
}
```

출력

- 필요한 페이지만 읽는다 - 8KB 조각 단위로 읽습니다. 전부 안 올립니다 (06만원)
- 색인이 있다 - 10만 건에서 17번 비교로 찾아갑니다 (06만원)
- 트랜잭션이 있다 - 전부 하거나 하나도 안 하거나를 보장합니다 (07만원)
- 규칙을 지킨다 - 이상한 값은 넣는 순간 거절합니다 (02만원)
- 관계를 안다 - 없는 설비의 점검기록을 못 만들게 합니다 (04만원)

★ 이 다섯 가지를 직접 코드로 만들 수는 있습니다. 개념01 마지막에 적은 대로, 그렇게 만들다 보면 데이터베이스를 다시 만들게 됩니다. 그것도 40년 덜 다듬은 것일요.

```
await db.close();
```

★ `db.close()` 를 안 부르면 프로그램이 안 끝나는 경우가 있습니다. 파일 끝에 꼭 넣으세요. 이 자료의 모든 파일이 그렇게 돼 있습니다.

정리 — 같은 실험, 두 가지 결과

무엇 파일(10만) DB(10만) 파일(100만) DB(100만)

1건 수정 92 ms 0.3 ms 1,467 ms 0.3 ms 1건 조회 61 ms 0.3 ms 776 ms 0.6 ms 조건 검색 59 ms 16 ms 778 ms 162 ms 20건 동시 쓰기 1건 남음 20건 남음

★ 읽는 법 · 1건 수정·조회 — **10배 늘려도 안 느려집니다.** 색인이 있기 때문입니다 · 조건 검색 — 색인이 없으면 데이터베이스도 느려집니다 (06만원에서 불입니다) · 동시 쓰기 — 파일은 조용히 읽고,

데이터베이스는 안 읽습니다

★★ 그리고 정직하게 이미 메모리에 올린 배열을 filter 하는 게 더 빠릅니다 (2 ms 대 16 ms).

데이터베이스는 **빨라서** 쓰는 게 아니라 **커져도 버티고, 안 읽고, 안 사라져서** 씁니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 4 의 `WHERE id = 77777` 을 `WHERE 이름 = '설비77777'` 로 바꿔 보세요. 시간이 어떻게 달라지나요? 왜 그럴까요? (힌트: id 에는 색인이 있고 이름에는 없습니다)

▫ 직접 해보기 2

섹션 5 앞에 이 줄을 넣고 조건 검색을 다시 재 보세요. `await db.exec("CREATE INDEX ON 설비 (라인, 상태)");` 몇 배 빨라지나요? (제 노트북에서는 16 ms 가 6 ms 가 됐습니다)

▫ 직접 해보기 3

섹션 6 의 20 을 200 으로 바꿔 보세요. 200건 전부 남나요? 개념01 의 파일은 몇 건 남았나요?

▫ 직접 해보기 4

`PGLite.create()` 대신 `PGLite.create("./내디비")` 로 바꾸고 두 번 실행해 보세요. 두 번째에 무슨 에러가 나나요? (힌트: 표가 이미 있습니다. 에러 코드가 42P07 입니다)

▫ 직접 해보기 5

섹션 7 의 메모리 배열을 100만 건으로 늘려 보세요. 만드는 데 얼마나 걸리나요? 노트북 메모리가 버티나요? 같은 100만 건을 데이터베이스는 4초에 넣었습니다.

▫ 직접 해보기 6

`SELECT current_database(), current_user` 를 찍어 보세요. PGLite 가 기본으로 어떤 이름을 쓰는지 알 수 있습니다.

자주 하는 실수

이런 실수를 합니다

“데이터베이스를 쓰면 알아서 빨라진다”

아닙니다. 색인 없는 조건 검색은 100만 건에서 162 ms 였습니다. 10만 건일 때의 10배입니다. 빠르게 만들 도구가 있는 것이지, 저절로 빨라지지 않습니다. 06단원에서 씁니다.

이런 실수를 합니다

값을 글자로 이어 붙임

`WHERE id = ${입력}` 은 SQL 인젝션 구멍입니다. \$1 을 쓰고 값은 배열로 넘기세요. 예외가 없습니다. 08단원에서 뚫어 봅니다.

이런 실수를 합니다

BIGINT 를 그대로 JSON.stringify 에 넣음

섹션 2 에서 봤듯이 값이 작을 때는 number 라 멀쩡히 넘어갑니다. 안전 정수를 넘는 순간 bigint 가 되고, 거기서 처음 터집니다. `::int` 로 바꾸거나 `Number()` 로 감싸세요. 02단원에서 자세히 봅니다.

이런 실수를 합니다

db.query() 에 세미콜론으로 두 문장을 넣음

`db.query("CREATE TABLE ...; INSERT INTO ...")` 는 에러가 납니다. 여러 문장은 `db.exec()` 입니다. 개념04 에서 실제 에러 메시지를 봅니다.

이런 실수를 합니다

한 번 재고 결론 냄

첫 번째 측정은 거의 항상 느립니다. 캐시도 안 데워졌고 계획도 안 세워졌습니다. 이 파일은 9번 재고 중앙값을 씁니다. 여러분도 그렇게 하세요.

이런 실수를 합니다

메모리 배열과 비교해 놓고 "DB 가 느리다" 고 결론

비교 대상이 틀렸습니다. 배열은 이미 올라와 있고 데이터베이스는 매번 시작합니다. 진짜 질문은 "그 배열이 100배 커져도 올라가나" 입니다. 안 올라갑니다.

이런 실수를 합니다

db.close() 를 빼먹음

프로그램이 안 끝나고 매달려 있습니다. 검증 도구는 이걸 "무한 반복" 으로 보고 3분 뒤에 죽입니다.

어떤 데이터베이스를 고르나

RUN node 개념03_어떤_데이터베이스를_고르나.js

★ PGlite 가 뜨는 데 1초쯤 걸립니다. 잠깐 멈춰 있어도 정상입니다.

개념02 에서 “데이터베이스를 쓰면 이렇게 달라진다” 를 재 봤습니다. 그럼 어떤 데이터베이스를 쓸까요?

여기서 흔히 하는 실수가 있습니다. **제품 이름부터 고르는 것**입니다. “요즘 뭐 쓰나요?” 하고요.

순서가 거꾸로입니다. 먼저 **모양**을 고르고, 그 다음에 제품을 고릅니다. 모양은 크게 셋입니다.

① 관계형 — 표. 칸을 미리 정해 둡니다 ② 문서형 — JSON. 문서마다 생김새가 달라도 됩니다 ③ 키-값 — 열쇠 하나로 값 하나. 그것뿐입니다

이 파일에서 셋을 **같은 데이터로** 만들어 보고, 무엇이 다른지 봅니다. 그리고 이 자료가 PostgreSQL 을 고른 이유를 정직하게 적습니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

같은 데이터를 세 가지 모양으로

섹션 1

설비 두 대를 넣습니다. 컨베이어와 프레스입니다. 그런데 두 대는 **사양이 다릅니다**. 컨베이어는 속도가, 프레스는 압력이 중요합니다. 이 “다름” 을 세 모양이 어떻게 다루는지 보시면 됩니다.

① 관계형 — 칸을 미리 정합니다

```
await db.exec(`
  CREATE TABLE 설비_표 (
    id      INT PRIMARY KEY,
    이름    TEXT NOT NULL,
    라인    TEXT NOT NULL,
    최대속도 INT,
    압력톤  INT
  )
`);

await db.exec(`
  INSERT INTO 설비_표 VALUES
    (1, '컨베이어 1호', 'A', 120, NULL),
```

```
(2, '프레스 1호', 'B', NULL, 250)
`);

const 표결과 = await db.query("SELECT * FROM 설비_표 ORDER BY id");

console.log("① 관계형:", JSON.stringify(표결과.rows));
```

출력 ① 관계형: [{"id":1,"이름":"컨베이어 1호","라인":"A","최대속도":120,"압력톤":null}, {"id":2,"이름":"프레스 1호","라인":"B","최대속도":null,"압력톤":250}]

★ 보이시나요? 빈 칸(**null**)이 생깁니다. 컨베이어에는 압력톤이 없고, 프레스에는 최대속도가 없습니다. 설비 종류가 20가지면 칸이 60개가 되고 대부분이 **null** 이 됩니다. 이걸 제대로 푸는 법은 04단원(모델링)에서 합니다.

★ 대신 얻는 것: 칸이 정해져 있습니다.

오타 난 칸 이름은 넣는 순간 거절당합니다. 나중에 보겠습니다.

② 문서형 — 생김새가 달라도 됩니다

```
await db.exec(`
  CREATE TABLE 설비_문서 (
    id INT PRIMARY KEY,
    이름 TEXT NOT NULL,
    사양 JSONB
  )
`);

await db.query("INSERT INTO 설비_문서 VALUES ($1, $2, $3)", [
  1,
  "컨베이어 1호",
  JSON.stringify({ 제조사: "한성기계", 최대속도: 120, 옵션: ["가변속", "역회전"] }),
]);

await db.query("INSERT INTO 설비_문서 VALUES ($1, $2, $3)", [
  2,
  "프레스 1호",
  JSON.stringify({ 제조사: "대동프레스", 압력톤: 250 }),
]);

const 문서결과 = await db.query("SELECT * FROM 설비_문서 ORDER BY id");

console.log("② 문서형:", JSON.stringify(문서결과.rows));
```

출력 ② 문서형: [{ "id":1, "이름": "컨베이어 1호", "사양": { "옵션": ["가변속", "역회전"], "제조사": "한성기계", "최대속도": 120 } },
 { "id":2, "이름": "프레스 1호", "사양": { "압력톤": 250, "제조사": "대동프레스" } }]

★ `null` 이 없습니다. 있는 것만 넣었습니다. 그리고 컨베이어에는 **옵션**이라는 배열까지 들어 있습니다. 프레스에는 그런 게 없습니다. 그래도 괜찮습니다.

★★ 그런데 넣은 순서대로 안 나옵니다. {제조사, 최대속도, 옵션} 순으로 넣었는데 {옵션, 제조사, 최대속도}로 나왔습니다. JSONB는 읽기 빠른 형태로 바뀌어서 저장하기 때문에 키 순서가 안 지켜집니다. **순서에 의존하는 코드를 쓰면 안 됩니다.** 이걸 나중에 진짜로 사고가 납니다.

③ 키-값 — 열쇠 하나에 값 하나

```
await db.exec(`
  CREATE TABLE 설정 (
    열쇠 TEXT PRIMARY KEY,
    값 TEXT NOT NULL
  )
`);

await db.exec(`
  INSERT INTO 설정 VALUES
    ('라인A_목표수량', '1200'),
    ('라인A_담당', '김반장'),
    ('점검주기_일', '30')
`);

const 하나 = await db.query("SELECT 값 FROM 설정 WHERE 열쇠 = $1", ["라인A_담당"]);

console.log("③ 키-값:", 하나.rows[0].값);
```

출력 ③ 키-값: 김반장

★ 열쇠로 찾는 것 **하나만** 잘합니다. 대신 아주 빠릅니다. “담당이 김반장인 라인을 전부 찾아라”는 못합니다. 값으로는 못 찾습니다. Redis가 이 모양입니다. 메모리에만 두어서 더 빠릅니다.

셋을 나란히 놓으면

섹션 2

```
const 모양비교 = [
  ["관계형", "칸을 미리 정함", "규칙을 지켜야 할 때", "PostgreSQL · MySQL · SQLite"],
  ["문서형", "문서마다 달라도 됨", "생김새가 자주 바뀔 때", "MongoDB"],
  ["키-값", "열쇠로만 찾음", "아주 빠르게 꺼낼 때", "Redis"],
];

for (const [모양, 성질, 언제, 제품] of 모양비교) {
  console.log(`· ${모양} - ${성질} · ${언제} · ${제품}`);
}
```

```
출력      · 관계형 - 칸을 미리 정함 · 규칙을 지켜야 할 때 · PostgreSQL · MySQL ·
          SQLite
          · 문서형 - 문서마다 달라도 됨 · 생김새가 자주 바뀔 때 · MongoDB
          · 키-값 - 열쇠로만 찾음 · 아주 빠르게 꺼낼 때 · Redis
```

★ 어느 쪽이 좋다는 이야기가 아닙니다. **무엇을 지킬 것인가**의 문제입니다.

생산실적·재고·급여처럼 **틀리면 안 되는 것**은 관계형입니다. 숫자가 안 맞으면 회사가 망합니다. 규칙을 데이터베이스가 지켜 줘야 합니다.

로그·센서값·설정 스냅샷처럼 **생김새가 자주 바뀌는 것**은 문서형이 편합니다.

★★ 현장에서는 섞어 씁니다.

```
실적은 PostgreSQL, 세션은 Redis, 로그는 다른 곳 - 이런 식입니다.
"하나만 골라야 한다" 는 생각부터 버리세요.
```

다섯 제품을 언제 쓰나

섹션 3

```
const 제품들 = [
  ["PostgreSQL", "관계형", "새 프로젝트의 기본값. 기능이 가장 많고 표준을 잘 지킴"],
  ["MySQL", "관계형", "국내 SI·제조·공공에 가장 많음. MariaDB 도 사실상 같은 부류"],
  ["SQLite", "관계형", "파일 하나가 데이터베이스 전부. 모바일 앱·데스크톱 앱·
  시험용"],
  ["MongoDB", "문서형", "생김새가 자주 바뀌는 데이터. 스키마를 미리 못 정할 때"],
  ["Redis", "키-값", "세션·캐시·순위표. 메모리에 두어 아주 빠름. 꺼지면 사라질 수
  있음"],
];

for (const [이름, 모양, 언제] of 제품들) {
  console.log(`${이름} (${모양}) - ${언제}`);
}
```

출력	PostgreSQL (관계형) – 새 프로젝트의 기본값. 기능이 가장 많고 표준을 잘 지킴
	MySQL (관계형) – 국내 SI·제조·공공에 가장 많음. MariaDB 도 사실상 같은 부류
	SQLite (관계형) – 파일 하나가 데이터베이스 전부. 모바일 앱·데스크톱 앱·시험용
	MongoDB (문서형) – 생김새가 자주 바뀌는 데이터. 스키마를 미리 못 정할 때
	Redis (키-값) – 세션·캐시·순위표. 메모리에 두어 아주 빠름. 꺼지면 사라질 수 있음

★ 현장 기준으로 한 줄씩 더 붙이면 이렇습니다.

PostgreSQL “뭘 쓸지 모르겠으면 이걸 쓰세요” 가 요즘 답입니다.

JSONB·전문검색·지리정보까지 하나로 됩니다.

MySQL 이미 돌고 있는 시스템이 압도적으로 많습니다.

운영해 본 사람도 많고, 자료도 많고, 관리 도구도 많습니다.

SQLite 서버가 없습니다. 프로그램 안에 들어갑니다.

휴대폰 앱 데이터는 거의 다 이겁니다. 동시 쓰기는 약합니다.

MongoDB “스키마가 없어서 편하다” 로 시작했다가

나중에 “누가 이 칸 이름을 오타 냈지” 로 고생하는 경우가 많습니다.

Redis 진짜 데이터를 여기에만 두면 안 됩니다. 캐시로 쓰세요.

★ 이 자료가 PostgreSQL 을 고른 이유

섹션 4

세 가지입니다. 하나씩 정직하게 적습니다.

① 표준 SQL 을 가장 잘 지킵니다

여기서 배운 문장이 다른 데이터베이스에서도 거의 그대로 통합니다.
반대로 MySQL 에서만 되는 문법을 먼저 배우면 나중에 고생합니다.

② 틀리면 바로 에러를 냅니다

이게 배우는 사람에게 가장 큼니다. 아래에서 직접 봅니다.

③ 새로 시작하는 프로젝트가 많이 고릅니다

최근 개발자 설문에서 PostgreSQL 이 가장 많이 쓰이는 데이터베이스로 나옵니다.
새 프로젝트에서는 격차가 더 큼니다.

★★ ② 를 재 봅니다. 일부러 이상한 것을 넣어 봅니다.

```
await db.exec(`
  CREATE TABLE 점검 (
    id      SERIAL PRIMARY KEY,
    점검자 VARCHAR(5),
    점검일 DATE,
    소요분 INT
  )
`);

const 이상한것들 = [
  ["칸 길이를 넘김", "INSERT INTO 점검 (점검자) VALUES ('김반장님과이반장님')"],
  ["없는 날짜", "INSERT INTO 점검 (점검일) VALUES ('2024-02-31')"],
  ["숫자 칸에 글자", "INSERT INTO 점검 (소요분) VALUES ('삼십분')"],
  ["0 으로 나눔", "SELECT 100 / 0"],
  ["묵지 않은 칸", "SELECT 점검자, 소요분 FROM 점검 GROUP BY 점검자"],
];

for (const [무엇, 문장] of 이상한것들) {
  try {
    await db.query(문장);
    console.log(`${무엇} → 통과했습니다 (?!)`);
  } catch (에러) {
    console.log(`${무엇} → 거절 [${에러.code}]`);
  }
}
```

출력	칸 길이를 넘김 → 거절 [22001]
	없는 날짜 → 거절 [22008]
	숫자 칸에 글자 → 거절 [22P02]
	0 으로 나눔 → 거절 [22012]
	묵지 않은 칸 → 거절 [42803]

★★★ 다섯 개 전부 거절했습니다.

MySQL 은 설정에 따라 이 중 몇 가지를 **조용히 넘어갑니다**. (요즘 버전은 기본값이 엄격해졌지만, 현장의 오래된 서버는 그렇지 않습니다)

칸 길이를 넘김	→ 잘라서 넣습니다. 경고만 뜹니다
없는 날짜	→ '0000-00-00' 으로 넣습니다
0 으로 나눔	→ NULL 을 돌려줍니다. 에러가 아닙니다
묵지 않은 칸	→ 아무 줄이나 하나 골라서 보여 줍니다

★ 배우는 사람에게 이게 왜 나쁠까요?

****틀린 줄 모르고 넘어갑니다.****

"김반장님과이반장님" 을 넣었는데 "김반장님과" 만 들어가 있고,
그걸 몇 달 뒤에 발견합니다. 개념01 의 조용한 버그와 똑같습니다.

★ 09단원에서 MySQL 을 진짜로 띄워 놓고 위 다섯 개를 그대로 해 봅니다.

어느 것이 통과하는지 직접 보시게 됩니다.

MySQL 은 여기가 다릅니다

· 위 다섯 가지의 처리가 sql_mode 설정에 따라 달라집니다 · SERIAL 대신 AUTO_INCREMENT 를 씁니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

★ 그런데 국내 현장은 MySQL 이 훨씬 많습니다

섹션 5

이건 반드시 짚고 넘어가야 합니다.

· **SI·SM 프로젝트** — 발주처 표준이 MySQL 이나 MariaDB 인 곳이 많습니다 · **제조·공장 시스템 (MES/ERP)** — 오래 돌던 것이 그대로 있습니다 · **공공기관** — 국산 DBMS 나 MariaDB 를 쓰는 곳이 많습니다 · **대기업 기간제** — Oracle 이 아직 많습니다

그러니까 "PostgreSQL 이 요즘 대세" 라는 말과 "내가 취업해서 만질 것도 PostgreSQL" 은 **다른 이야기**입니다.

★ 그래서 이 자료는 09단원 하나를 통째로 MySQL 비교에 씁니다. Docker 로 PostgreSQL 과 MySQL 을 **동시에 띄워 놓고** 같은 SQL 을 양쪽에 넣어 보면서 무엇이 다른지 봅니다.

★★ 그리고 각 단원 중간중간에 이런 표시가 나옵니다.

" — MySQL 은 여기가 다릅니다 — "

지금 당장 외우지 마세요. 09단원에서 다시 모아서 봅니다.

★ PostgreSQL 은 문서형 흉내도 냅니다

섹션 6

"그럼 JSON 을 넣고 싶으면 MongoDB 를 따로 써야 하나요?" 아닙니다. 섹션 1 의 ㉠ 가 이미 PostgreSQL 이었습니다. JSONB 라고 합니다.

JSON 안의 값으로 찾을 수 있습니다.

```
const 제조사찾기 = await db.query(
  "SELECT 이름, 사양->>'제조사' AS 제조사 FROM 설비_문서 ORDER BY id",
);

console.log(JSON.stringify(제조사찾기.rows));
```

출력 [{"이름":"컨베이어 1호","제조사":"한성기계"}, {"이름":"프레스 1호","제조사":"대동프레스"}]

★ ->> 는 “이 키의 값을 **글자로** 꺼내라” 입니다. -> 는 “JSON 인 채로 꺼내라” 입니다. 화살표 개수가 다릅니다.

조건으로도 찾습니다.

```
const 한성 = await db.query(`SELECT 이름 FROM 설비_문서 WHERE 사양 @>
'{"제조사":"한성기계"}'`);

console.log("한성기계 설비:", JSON.stringify(한성.rows));
```

출력 한성기계 설비: [{"이름":"컨베이어 1호"}]

@> 는 “이걸 품고 있느냐” 입니다.

없는 키를 꺼내면 어떻게 될까요?

```
const 없는키 = await db.query("SELECT 이름, 사양->>'압력톤' AS 압력톤 FROM 설비_문서
ORDER BY id");

console.log(JSON.stringify(없는키.rows));
```

출력 [{"이름":"컨베이어 1호","압력톤":null}, {"이름":"프레스 1호","압력톤":"250"}]

★★ 두 가지를 보세요.

① 컨베이어에는 압력톤이 없으니 **null** 입니다. 에러가 아닙니다 ② 프레스의 250 이 **문자열 "250"** 으로 나왔습니다. 숫자가 아닙니다

->> 는 무조건 글자로 꺼내기 때문입니다. 계산하려면 (사양->>'압력톤')::int 처럼 바꿔야 합니다.

★★★ 이게 문서형의 대가입니다.

칸을 미리 안 정했으니 **타입도 안 정해져 있습니다.**
 오타 난 키(`압력톤` 대신 `압력톤수`)를 넣어도 아무도 안 막습니다.
 조용히 null 이 됩니다. 개념01 의 그 조용한 버그입니다.

★ 그래서 이렇게 씁니다. **틀리면 안 되는 것은 칸으로**, 자주 바뀌는 부가 정보만 JSONB 로. 전부 JSONB 로 넣으면 관계형을 쓰는 의미가 없습니다.

```
const 통하는것 = [
  ["SELECT · WHERE · ORDER BY · LIMIT", "그대로 통합니다"],
  ["JOIN · GROUP BY · HAVING", "그대로 통합니다"],
  ["표 설계 · 정규화 · 외래키", "생각하는 방식이 같습니다"],
  ["트랜잭션 · 격리수준", "이름과 개념이 같습니다"],
  ["색인 · 실행계획 읽기", "보는 법이 거의 같습니다"],
];
```

```
const 다른것 = [
  ["자리 표시", "$1 (PG) 대 ? (MySQL)"],
  ["자동 번호", "SERIAL (PG) 대 AUTO_INCREMENT (MySQL)"],
  ["표 목록 보기", "\\dt (PG) 대 SHOW TABLES (MySQL)"],
  ["글자 이어붙이기", "|| (PG) 대 CONCAT() (MySQL)"],
];
```

```
console.log("— 그대로 통하는 것 —");
```

```
출력      — 그대로 통하는 것 —
```

```
for (const [무엇, 설명] of 통하는것) console.log(`· ${무엇} - ${설명}`);
```

```
출력      · SELECT · WHERE · ORDER BY · LIMIT - 그대로 통합니다
          · JOIN · GROUP BY · HAVING - 그대로 통합니다
          · 표 설계 · 정규화 · 외래키 - 생각하는 방식이 같습니다
          · 트랜잭션 · 격리수준 - 이름과 개념이 같습니다
          · 색인 · 실행계획 읽기 - 보는 법이 거의 같습니다
```

```
console.log("— 손봐야 하는 것 —");
```

```
출력      — 손봐야 하는 것 —
```

```
for (const [무엇, 설명] of 다른것) console.log(`· ${무엇} - ${설명}`);
```

```
출력      · 자리 표시 - $1 (PG) 대 ? (MySQL)
          · 자동 번호 - SERIAL (PG) 대 AUTO_INCREMENT (MySQL)
          · 표 목록 보기 - \dt (PG) 대 SHOW TABLES (MySQL)
          · 글자 이어붙이기 - || (PG) 대 CONCAT() (MySQL)
```

★★ 배우는 것의 대부분이 그대로 통합니다. 다른 것은 대개 ‘적는 방식’ 이고, 생각하는 방식은 같습니다.

운전면허를 현대차로 따고 기아차를 몰 수 있는 것과 같습니다. 깜빡이 위치가 다를 수는 있어도 운전은 운전입니다.

★ 오히려 PostgreSQL 로 배우는 게 유리한 점이 있습니다. 엄격한 쪽에서 느슨한 쪽으로 가는 것은 쉽습니다. 반대는 어렵습니다. 느슨한 데서 배운 습관이 엄격한 곳에서 전부 걸립니다.

```
await db.close();
```

정리 — 무엇을 언제

질문 답

틀리면 안 되는 데이터인가 관계형 (PostgreSQL · MySQL) 생김새가 자주 바뀌는가 문서형 또는 JSONB
열쇠로 꺼내기만 하나 키-값 (Redis) 앱 안에 넣을 건가 SQLite 뭘 골라야 할지 모르겠다 PostgreSQL

★ 이 자료가 PostgreSQL 인 이유 ① 표준 SQL 을 잘 지켜서 다른 곳에 옮기기 쉽습니다 ② 틀리면 바로
에러를 냅니다 (다섯 개 전부 거절했습니다) ③ 새 프로젝트가 많이 고릅니다

★★ 그래도 국내 현장은 MySQL·MariaDB 가 훨씬 많습니다. 09단원에서 돌을 나란히 띄워 놓고
비교합니다. 거기서 정리됩니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 1 의 **설비_표** 에 설비를 다섯 대 더 넣어 보세요. 종류가 다양해질수록 **null** 이 얼마나 늘어나나요?

▫ 직접 해보기 2

섹션 4 의 **이상한것들** 에 하나를 더 추가해 보세요. 예: **INSERT INTO 점검 (없는칸) VALUES (1)** 어떤
코드가 나오나요? (힌트: 42703)

▫ 직접 해보기 3

섹션 6 에서 **사양->>'압력톤'** 을 (**사양->>'압력톤'**)::**int** 로 바꿔 보세요. 컨베이어(**null**)에서 에러가
나나요, 안 나나요?

▫ 직접 해보기 4

설비_문서 에 키를 일부러 오타 내서 하나 넣어 보세요. {**"제조사": "삼성", "압력톤수": 300**} **사양->>'압
력톤'** 으로 찾으면 어떻게 되나요? 이게 왜 무서운지 한 줄로 적어 보세요.

▫ 직접 해보기 5

SELECT ' {"a":1,"b":2} '::jsonb = ' {"b":2,"a":1} '::jsonb 을 찍어 보세요. 순서가 다른데 같다고
할까요?

여러분 회사(또는 가고 싶은 회사)가 무엇을 쓰는지 찾아보세요. 채용공고에 대부분 적혀 있습니다.

자주 하는 실수

이런 실수를 합니다

제품부터 고름

“요즘 뭐가 좋아요?” 로 시작하면 안 됩니다. “이 데이터가 틀리면 회사가 곤란해지나?” 부터 물으세요. 답이 ‘그렇다’ 면 관계형입니다. 제품은 그 다음입니다.

이런 실수를 합니다

“스키마가 없어서 편하다” 로 문서형을 고름

편한 건 처음 한 달입니다. 6개월 뒤에 압력톤, 압력톤수, `pressure_ton` 이 섞여 있는 걸 발견합니다. 아무도 안 막았기 때문입니다. 규칙은 어딘가에는 있어야 합니다.

이런 실수를 합니다

Redis 에 진짜 데이터를 둬

Redis 는 기본이 메모리입니다. 꺾다 켜면 사라질 수 있습니다. 세션·캐시·순위표는 좋습니다. 주문 내역을 두면 안 됩니다.

이런 실수를 합니다

JSONB 로 전부 넣음

그럴 거면 관계형을 쓰는 의미가 없습니다. 틀리면 안 되는 것은 칸으로 빼세요. 나머지 부가 정보만 JSONB 입니다.

이런 실수를 합니다

“PostgreSQL 배웠으니 MySQL 은 못 한다”

대부분 그대로 통합니다. 다른 건 적는 방식 몇 가지입니다. 09단원에서 그 몇 가지를 모아서 봅니다.

이런 실수를 합니다

JSONB 의 키 순서를 믿음

넣은 순서와 나오는 순서가 다릅니다. 섹션 1 에서 직접 보셨습니다. 순서에 기대는 코드를 쓰면 언젠가 깨집니다.

이런 실수를 합니다

-> 와 ->> 를 헷갈림

-> 는 JSON 인 채로, ->> 는 글자로 꺼냅니다. ->> 로 꺼낸 숫자는 문자열입니다. 더하면 이어 붙습니다.

SQL 은 어떤 말인가

RUN node 개념04_SQL은_어떤_말인가.js

★ PGlite 가 뜨는 데 1초쯤 걸립니다. 잠깐 멈춰 있어도 정상입니다.

개념02·03 에서 SQL 문장을 몇 개 봤습니다. 아직 설명은 안 했습니다. 이번에 SQL 이 **어떤 종류의 말인지** 봅니다.

문법을 외우는 시간이 아닙니다. 문법은 03단원부터 차근차근 합니다. 여기서 잡을 것은 딱 두 가지입니다.

① SQL 은 “어떻게” 가 아니라 “무엇을” 을 적는 말입니다 ② ★★★ **적는 순서와 실행 순서가 다릅니다**

②를 모르면 “WHERE 에서 별칭을 왜 못 쓰지?” 에서 반드시 막힙니다. 오늘 그 에러를 진짜로 내 보겠습니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

실습용 점검기록

섹션 1

```
await db.exec(`
  CREATE TABLE 점검기록 (
    id      SERIAL PRIMARY KEY,
    설비명  TEXT NOT NULL,
    라인    TEXT NOT NULL,
    점검자  TEXT NOT NULL,
    소요분  INT  NOT NULL,
    결과    TEXT NOT NULL
  );

  INSERT INTO 점검기록 (설비명, 라인, 점검자, 소요분, 결과) VALUES
    ('컨베이어 1호', 'A', '김반장', 35, '정상'),
    ('컨베이어 2호', 'A', '김반장', 50, '이상'),
    ('프레스 1호',   'B', '이반장', 20, '정상'),
    ('프레스 2호',   'B', '이반장', 65, '이상'),
    ('용접로봇 1호', 'C', '박반장', 15, '정상'),
    ('용접로봇 2호', 'C', '박반장', 80, '이상'),
    ('도장기 1호',   'A', '김반장', 45, '정상');
`);
```


★ `db.exec()` 를 썼습니다. 세미콜론으로 여러 문장을 이었기 때문입니다. `db.query()` 로 하면 에러가 납니다. 섹션 7 에서 진짜로 내 봅니다.

★ SQL 은 선언형입니다

섹션 2

01

같은 일을 자바스크립트와 SQL 로 각각 해 봅니다.

“소요분이 30분을 넘은 점검을, 오래 걸린 순서로, 위에서 3건만”

```
const 점검배열 = [
  { 설비명: "컨베이어 1호", 라인: "A", 점검자: "김반장", 소요분: 35, 결과: "정상" },
  { 설비명: "컨베이어 2호", 라인: "A", 점검자: "김반장", 소요분: 50, 결과: "이상" },
  { 설비명: "프레스 1호", 라인: "B", 점검자: "이반장", 소요분: 20, 결과: "정상" },
  { 설비명: "프레스 2호", 라인: "B", 점검자: "이반장", 소요분: 65, 결과: "이상" },
  { 설비명: "용접로봇 1호", 라인: "C", 점검자: "박반장", 소요분: 15, 결과: "정상" },
  { 설비명: "용접로봇 2호", 라인: "C", 점검자: "박반장", 소요분: 80, 결과: "이상" },
  { 설비명: "도장기 1호", 라인: "A", 점검자: "김반장", 소요분: 45, 결과: "정상" },
];
```

자바스크립트 — 어떻게 할지를 순서대로 적습니다.

```
const js답 = 점검배열
  .filter((줄) => 줄.소요분 > 30) // ① 걸러라
  .sort((가, 나) => 나.소요분 - 가.소요분) // ② 정렬해라
  .slice(0, 3) // ③ 잘라라
  .map((줄) => 줄.설비명); // ④ 이름만 뽑아라 console.log("자바스크립트:",
JSON.stringify(js답));
```

출력 자바스크립트: ["용접로봇 2호", "프레스 2호", "컨베이어 2호"]

SQL — 무엇을 원하는지만 적습니다.

```
const sql답 = await db.query(`
  SELECT 설비명
  FROM   점검기록
  WHERE  소요분 > 30
  ORDER BY 소요분 DESC
  LIMIT 3
`);

console.log("SQL:", JSON.stringify(sql답.rows.map((줄) => 줄.설비명)));
```

출력 SQL: ["용접로봇 2호", "프레스 2호", "컨베이어 2호"]

```
console.log("두 답이 같은가:", JSON.stringify(js답) ===
JSON.stringify(sql답.rows.map((줄) => 줄.설비명)));
```

출력 두 답이 같은가: true

★★ 같은 답인데 적는 방식이 다릅니다.

자바스크립트: 걸러라 → 정렬해라 → 잘라라 → 뽑아라 (**절차형**) SQL : 이런 걸 이 순서로 이만큼 달라 (**선언형**)

SQL 에는 “어떻게 찾을지” 가 없습니다. 전부 훑을지, 색인을 탈지, 어느 것부터 이을지는 **데이터베이스가 정합니다**. 그 정한 내용을 ‘실행계획’ 이라고 합니다. 06단원에서 들여다봅니다.

★ 왜 이렇게 만들었을까요? 데이터가 늘거나 색인이 생기면 **가장 빠른 방법이 바뀝니다**. 그때마다 코드를 고치지 않아도 되게 하려고, “무엇을” 만 적게 한 것입니다.

★★ 대신 대가가 있습니다. 느릴 때 “왜 느린지” 가 코드에 안 보입니다. 그래서 실행계획을 읽을 줄 알아야 합니다. 06단원입니다.

문장의 구조

섹션 3

SELECT 문장은 이 일곱 조각으로 이루어집니다. 순서가 정해져 있습니다.

SELECT	무엇을 보여 줄 것인가
FROM	어느 표에서
WHERE	어떤 줄만

GROUP BY 무엇으로 묶어서 HAVING 묶은 것 중 어떤 것만 ORDER BY 어떤 순서로 LIMIT 몇 개만 전부 쓸 필요는 없습니다. SELECT 와 FROM 만 있어도 문장이 됩니다.

```
const 묶기 = await db.query(`
  SELECT  라인, count(*)::int AS 건수, round(avg(소요분), 1) AS 평균분
  FROM    점검기록
  GROUP BY 라인
  HAVING  count(*) >= 2
  ORDER BY 라인
`);

console.log(JSON.stringify(묶기.rows));
```

출력 [{"라인":"A","건수":3,"평균분":"43.3"},
 {"라인":"B","건수":2,"평균분":"42.5"},
 {"라인":"C","건수":2,"평균분":"47.5"}]

★ 평균분이 “43.3” 입니다. 따옴표가 붙어 있습니다. **문자열입니다**. round(...) 가 돌려주는 NUMERIC 타입이 자바스크립트로 올 때 문자열이 됩니다. 숫자로 쓰려면 Number() 로 바꾸거나 SQL 에서 ::float 를 붙여야 합니다. ★ 돈 계산에서 이것 때문에 사고가 납니다. 02단원에서 따로 다룹니다.

★ `count(*)::int` 의 `::int` 는 개념02 섹션 2 에서 본 그것입니다. `count(*)` 는 BIGINT 라, 값이 안전 정수를 넘으면 bigint 가 되어 `JSON.stringify` 가 터집니다. 미리 못을 박아 두는 것입니다.

★★★ 적는 순서와 실행 순서가 다릅니다

섹션 4

01

적는 순서 SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT

실행 순서 FROM → WHERE → GROUP BY → HAVING → **SELECT** → ORDER BY → LIMIT

★ **SELECT 가 거의 끝에서 두 번째입니다.** WHERE 나 GROUP BY 가 돌아갈 때는 SELECT 를 아직 안 봤습니다. 그래서 SELECT 에서 만든 별칭을 WHERE 가 모릅니다.

말로 하면 안 와닿습니다. 진짜로 에러를 내 봅니다.

```
try {
  await db.query(`
    SELECT 설비명, 소요분 * 60 AS 소요초
    FROM   점검기록
    WHERE  소요초 > 1000
  `);
} catch (에러) {
  console.log(`WHERE 에서 별칭 → [${에러.code}] ${에러.message}`);
}
```

출력 WHERE 에서 별칭 → [42703] column "소요초" does not exist

★★ “방금 위에서 AS 소요초 라고 만들었는데요?” 만든 것은 맞습니다. 그런데 **WHERE 가 먼저 돕니다.** WHERE 차례에는 아직 소요초라는 게 세상에 없습니다.

★ 그럼 어떻게 쓰나요? 식을 그대로 다시 씁니다.

```
const 다시쓰 = await db.query(`
  SELECT 설비명, 소요분 * 60 AS 소요초
  FROM   점검기록
  WHERE  소요분 * 60 > 1000
  ORDER BY 소요초 DESC
  LIMIT 3
`);

console.log(JSON.stringify(다시쓰.rows));
```

출력 [{"설비명": "용접로봇 2호", "소요초": 4800}, {"설비명": "프레스 2호", "소요초": 3900}, {"설비명": "컨베이어 2호", "소요초": 3000}]

★★ 그런데 **ORDER BY** 에서는 별칭이 됩니다. 위 문장을 보세요. ORDER BY 는 SELECT 다음에 돌기 때문입니다.

실행 순서를 알면 이게 전부 설명됩니다.

WHERE 에서 별칭 → 안 됨 (SELECT 보다 먼저 됨)
 HAVING 에서 별칭 → 안 됨 (SELECT 보다 먼저 됨)
 ORDER BY 에서 별칭 → 됨 (SELECT 다음에 됨)

★ 외우지 마세요. 실행 순서 한 줄만 기억하면 셋 다 따릅니다.

****FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT****

HAVING 에서도 정말 안 되는지 확인합니다.

```
try {
  await db.query(`
    SELECT 라인, count(*) AS 건수
    FROM 점검기록
    GROUP BY 라인
    HAVING 건수 >= 2
  `);
} catch (에러) {
  console.log(`HAVING 에서 별칭 → [${에러.code}] ${에러.message}`);
}
```

출력 HAVING 에서 별칭 → [42703] column "건수" does not exist

실행 순서를 모르면 만나는 에러가 두 개 더 있습니다.

```
try {
  await db.query(`
    SELECT 라인, 설비명, count(*)
    FROM 점검기록
    GROUP BY 라인
  `);
} catch (에러) {
  console.log(`묵지 않은 칸 → [${에러.code}] ${에러.message}`);
}
```

출력 묵지 않은 칸 → [42803] column "점검기록.설비명" must appear in the GROUP BY clause or be used in an aggregate function

★ 라인 'A' 로 묶으면 그 안에 설비명이 셋(컨베이어 1호·2호·도장기 1호) 있습니다. 셋 중 무엇을 보여 줘야 하나요? 알 수 없습니다. 그래서 거절합니다. ★★ MySQL 은 설정에 따라 **아무거나 하나 골라서 보여 줍니다.** 09단원에서 봅니다.

```
try {
  await db.query(`
    SELECT 라인
    FROM 점검기록
    WHERE count(*) > 1
    GROUP BY 라인
  `);
} catch (에러) {
  console.log(`WHERE 에서 집계 → [${에러.code}] ${에러.message}`);
}
```

출력 WHERE 에서 집계 → [42803] aggregate functions are not allowed in WHERE

★ WHERE 는 GROUP BY 보다 **먼저** 돕니다. 아직 안 묶였으니 셀 수가 없습니다. 묶은 다음에 거르는 것이 HAVING 입니다.

★★ 이 한 줄로 정리됩니다.

****WHERE 는 줄을 거르고, HAVING 은 묶음을 거릅니다.****

적는 법 — 대소문자·세미콜론·주석

섹션 5

SQL 은 키워드의 대소문자를 안 가립니다. 이 셋이 전부 같은 문장입니다.

SELECT * FROM 점검기록 select * from 점검기록 SeLeCt * FrOm 점검기록

★ 관례는 **키워드는 대문자, 이름은 그대로** 입니다. 눈으로 훑을 때 문장 구조가 바로 보이라고 그렇게 씁니다. 이 자료는 전부 그 관례를 따릅니다.

★★ 그런데 **이름의 대소문자는 이야기가 다릅니다**. PostgreSQL 은 따옴표 없는 이름을 전부 **소문자로 바꿔서** 뱉니다.

```
const 대소문자 = await db.query("SELECT 소요분 AS Total FROM 점검기록 LIMIT 1");

console.log("돌려준 칸 이름:", JSON.stringify(Object.keys(대소문자.rows[0])));
```

출력 돌려준 칸 이름: ["total"]

★★ Total 이라고 적었는데 total 로 돌아왔습니다. 자바스크립트에서 줄.Total 로 꺼내면 **undefined** 입니다. 여기서 많이 막힙니다. 대문자를 지키려면 AS “Total” 처럼 큰따옴표로 묶어야 합니다. ★ 그런데 그러면 **부를 때마다 큰따옴표를 써야 합니다**. 귀찮습니다.

그래서 실무에서는 칸 이름을 처음부터 소문자로 짓습니다.

★ 한글 이름은 대소문자 문제가 없어서 이 자료에서는 한글을 씁니다. 회사에서는 영어 소문자에 밑줄 (inspection_log)을 많이 씁니다.

세미콜론은 "문장이 여기서 끝났다" 는 표시입니다. 한 문장만 보낼 때는 있어도 되고 없어도 됩니다.

```
const 세미콜론있음 = await db.query("SELECT count(*)::int AS 건수 FROM 점검기록;");

console.log("세미콜론 붙여도:", 세미콜론있음.rows[0].건수);
```

출력 세미콜론 붙여도: 7

주석은 -- 입니다. 그 줄의 끝까지가 주석입니다. 여러 줄은 /* ... */ 입니다.

```
const 주석있는문장 = await db.query(`
  SELECT count(*)::int AS 건수  -- 몇 건인지 셉니다
  FROM  점검기록
  /* 이상만 셉니다 */
  WHERE 결과 = '이상'
`);

console.log("이상 건수:", 주석있는문장.rows[0].건수);
```

출력 이상 건수: 3

★ 자바스크립트의 // 가 아닙니다. SQL 은 -- 입니다. 자주 틀립니다.

SQL 의 네 가족

섹션 6

```
const 가족 = [
  ["DDL", "표를 만들고 바꾸고 지움", "CREATE · ALTER · DROP · TRUNCATE", "02단원"],
  ["DML", "데이터를 넣고 읽고 고치고 지움", "INSERT · SELECT · UPDATE · DELETE",
  "03단원"],
  ["TCL", "여러 작업을 한 덩어리로 묶음", "BEGIN · COMMIT · ROLLBACK", "07단원"],
  ["DCL", "누가 무엇을 할 수 있는지 정함", "GRANT · REVOKE", "10단원"],
];

for (const [이름, 하는일, 명령, 단위] of 가족) {
  console.log(`${이름} - ${하는일} · ${명령} (${단위})`);
}
```

출력 DDL - 표를 만들고 바꾸고 지움 · CREATE · ALTER · DROP · TRUNCATE (02단원)
 DML - 데이터를 넣고 읽고 고치고 지움 · INSERT · SELECT · UPDATE · DELETE
 (03단원)
 TCL - 여러 작업을 한 덩어리로 묶음 · BEGIN · COMMIT · ROLLBACK (07단원)
 DCL - 누가 무엇을 할 수 있는지 정함 · GRANT · REVOKE (10단원)

★ 이름은 안 외워도 됩니다. 시험에나 나옵니다. 중요한 것은 **성격이 다르다** 는 점입니다. DDL 은 표의 모양을 바꾸는 것이라 되돌리기가 어렵고, DML 은 트랜잭션으로 되돌릴 수 있습니다. 07단원에서 확인합니다.

★ query 와 exec 는 다릅니다

섹션 7

섹션 1 에서 `db.exec()` 를 썼습니다. 왜 그랬는지 봅시다.

```
try {
  await db.query("SELECT 1 AS 가; SELECT 2 AS 나;");
} catch (에러) {
  console.log(`query 에 두 문장 → [${에러.code}] ${에러.message}`);
}
```

출력 query 에 두 문장 → [42601] cannot insert multiple commands into a prepared statement

★★ `db.query()` 는 **한 문장만** 받습니다.

왜냐하면 query 는 문장을 '준비된 문장(prepared statement)' 으로 만들기 때문입니다. 준비된 문장은 값을 넣을 자리(\$1, \$2)를 미리 뚫어 두는 방식입니다. 그래야 SQL 인젝션을 막을 수 있습니다. 그런데 준비된 문장은 **한 번에 하나만** 됩니다.

★ 여러 문장을 보내려면 `db.exec()` 입니다.

```
const 여러문장 = await db.exec("SELECT 1 AS 가; SELECT 2 AS 나;");

console.log("exec 결과 개수:", 여러문장.length);
```

출력 exec 결과 개수: 2

```
console.log(JSON.stringify(여러문장.map((하나) => 하나.rows)));
```

출력 [[{"가":1}], [{"나":2}]]

★★★ 대신 `db.exec()` 는 **\$1 을 못 씁니다**. 그래서 사용자가 넣은 값을 exec 로 보내면 안 됩니다. 인젝션이 뚫립니다.

★ 규칙은 이렇게 잡으세요.

- 표를 만들거나 실습 데이터를 미리 넣을 때 → ``exec``
- 사용자 값이 들어가는 모든 문장 → ``query` + `$1``

예외는 없습니다. 08단원에서 이 규칙을 어겼을 때 무슨 일이 나는지 봅시다.

query 가 돌려주는 것을 한 번 더 봅시다.

```
const 조회결과 = await db.query("SELECT 설비명 FROM 점검기록 WHERE 라인 = $1",
["A"]);
```

```
console.log("command:", 조회결과.command, "/ rowCount:", 조회결과.rowCount, "/
affectedRows:", 조회결과.affectedRows);
```

```
출력      command: SELECT / rowCount: 3 / affectedRows: 0
```

```
const 수정결과 = await db.query("UPDATE 점검기록 SET 결과 = $1 WHERE 라인 = $2",
["정상", "C"]);
```

```
console.log("command:", 수정결과.command, "/ affectedRows:", 수정결과.affectedRows,
"/ rows:", JSON.stringify(수정결과.rows));
```

```
출력      command: UPDATE / affectedRows: 2 / rows: []
```

★ UPDATE 는 rows 가 비어 있습니다. 몇 줄이 바뀌었는지는 affectedRows 로 봅니다. 바뀐 줄을 돌려받고 싶으면 RETURNING 을 붙입니다. 03단원에서 합니다.

문법이 틀리면 무엇이 나오나

섹션 8

```
const 틀린문장들 = [
  ["오타", "SELEC * FROM 점검기록"],
  ["없는 표", "SELECT * FROM 없는표"],
  ["없는 칸", "SELECT 없는칸 FROM 점검기록"],
];

for (const [무엇, 문장] of 틀린문장들) {
  try {
    await db.query(문장);
  } catch (에러) {
    console.log(`${무엇} → [${에러.code}] ${에러.message}`);
  }
}
```

```
출력      오타 → [42601] syntax error at or near "SELEC"
          없는 표 → [42P01] relation "없는표" does not exist
          없는 칸 → [42703] column "없는칸" does not exist
```

★★ 에러.code 를 보세요. 다섯 글자짜리 코드입니다. **SQLSTATE** 라고 합니다. 메시지는 버전마다 바뀔 수 있지만 코드는 안 바뀝니다. 그래서 코드로 검색하는 게 정확합니다. 개념05 에서 자주 나오는 코드를 정리합니다.

★ relation 은 '표' 라는 뜻입니다. 관계형에서 표를 relation 이라고 부릅니다. 에러 메시지에 relation 이 나오면 표 이야기라고 보시면 됩니다.


```
await db.close();
```

정리 — SQL 이라는 말

무엇 내용

성격 선언형. “어떻게” 가 아니라 “무엇을” 적는 순서 SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT ★ 실행 순서 FROM WHERE GROUP BY HAVING SELECT ORDER BY LIMIT 별칭이 되는 곳 ORDER BY (O) · WHERE (X) · HAVING (X) WHERE 대 HAVING 줄을 거름 대 묶음을 거름 주석 -- 한 줄 · /* 여러 줄 */ 네 가족 DDL · DML · TCL · DCL query 대 exec 한 문장 + \$1 대 여러 문장 + \$1 없음

★★★ 딱 한 줄만 외운다면 이것입니다. **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT** 이 자료에서 앞으로 만날 에러의 절반쯤이 이 한 줄로 설명됩니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 2 의 SQL 에서 LIMIT 3 을 지우면 몇 건이 나오나요? ORDER BY 소요분 DESC 를 ASC 로 바꾸면 어떻게 되나요?

▫ 직접 해보기 2

점점자별 평균 소요분을 구해 보세요. 평균이 40분을 넘는 사람만 나오게 해 보세요. (힌트: GROUP BY 점점자, 그리고 HAVING)

▫ 직접 해보기 3

섹션 4 의 별칭 에러를 HAVING 이 아니라 GROUP BY 에서 내 보세요. GROUP BY 별칭 — 됩니다. 왜 될까요? (힌트: 표준은 안 되는데 PostgreSQL 이 봐 줍니다. 의존하지 마세요)

▫ 직접 해보기 4

SELECT 설비명 AS Name FROM 점검기록 LIMIT 1 을 돌리고 rows[0].Name 과 rows[0].name 을 각각 찍어 보세요. 어느 쪽이 undefined 인가요?

▫ 직접 해보기 5

db.exec(“SELECT \$1”, [‘A’]) 를 해 보세요. 무슨 에러가 나나요? exec 는 왜 \$1 을 못 받을까요?

▹ 직접 해보기 6

섹션 8의 틀린문장들 에 하나를 더 넣어 보세요. "SELECT * FROM 점검기록 WHERE 소요분 = '삼십'"
어떤 코드가 나오나요?

▹ 직접 해보기 7

섹션 3의 round(avg(소요분), 1) 을 avg(소요분) 으로 바꿔 보세요. 자릿수가 몇 개나 나오나요? 그것도 문자열인가요?

자주 하는 실수

이런 실수를 합니다

WHERE 에서 별칭을 씀

가장 많이 나오는 에러입니다. 42703 이 나옵니다. SELECT 가 WHERE 보다 나중에 돕니다. 식을 그대로 다시 쓰세요.

이런 실수를 합니다

WHERE 에 집계 함수를 씀

WHERE count(*) > 1 은 42803 입니다. 묶기 전이라 셀 수가 없습니다. 묶은 다음 거르는 건 HAVING 입니다.

이런 실수를 합니다

GROUP BY 에 안 넣은 칸을 SELECT 에 씀

42803 입니다. 묶음 하나에 값이 여럿이라 무엇을 보여 줄지 정할 수 없습니다. MySQL 은 아무거나 하나 골라 줍니다. 그래서 더 위험합니다.

이런 실수를 합니다

주석을 // 로 씀

SQL 은 -- 입니다. // 는 문법 에러(42601)가 납니다.

이런 실수를 합니다

db.query() 에 세미콜론으로 여러 문장을 넣음

42601 "cannot insert multiple commands into a prepared statement" 입니다. 여러 문장은 db.exec() 입니다. 대신 exec 에는 \$1 을 못 씁니다.

이런 실수를 합니다

대문자 별칭을 만들고 대문자로 꺼냄

AS Total 은 total 로 돌아옵니다. rows[0].Total 은 undefined 입니다. 지키려면 AS “Total” 처럼 큰따옴표를 쓰거나, 처음부터 소문자로 지으세요.

이런 실수를 합니다

NUMERIC 결과를 숫자로 여김

round(avg(...), 1) 은 “43.3” 이라는 문자열로 옵니다. + 로 더하면 이어 붙습니다. Number() 로 바꾸세요. 02단원에서 자세히 봅니다.

이 자료를 보는 법

RUN node 개념05_이_자료를_보는_법.js

★ PGLite 가 뜨는 데 1초쯤 걸립니다. 잠깐 멈춰 있어도 정상입니다.

여기까지 네 개를 보셨습니다.

개념01 파일에 저장하면 무슨 일이 생기는가 (재 봤습니다)
 개념02 데이터베이스는 어떻게 다른가 (같은 실험을 다시 했습니다)
 개념03 어떤 데이터베이스를 고르나
 개념04 SQL 은 어떤 말인가

이번 파일은 **자료 사용 설명서**입니다. SQL 을 새로 배우지는 않습니다. 앞으로 아홉 단원을 어떻게 볼지, 막히면 무엇을 하면 되는지 정리합니다.

한 번 읽고 넘어가되, **막혔을 때 다시 오세요**. 섹션 6 이 그때 쓸 것입니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

열 단원 지도

섹션 1

```
const 지도 = [
  ["01", "왜 데이터베이스인가", "PGLite", "지금 여기입니다"],
  ["02", "표 설계하기", "PGLite", "칸과 타입, 제약, NUMERIC·BIGINT 함정"],
  ["03", "넣고 읽고 고치고 지우기", "PGLite", "INSERT · SELECT · UPDATE · DELETE"],
  ["04", "데이터 모델링과 정규화", "PGLite", "표를 몇 개로 쪼갤 것인가, 외래키"],
  ["05", "여러 표를 잇기", "PGLite", "JOIN · 서브쿼리 · 집계"],
  ["06", "색인과 실행계획", "PGLite", "왜 느린지 들여다보고 고치기"],
  ["07", "트랜잭션과 동시성", "Docker", "격리수준 · 락 · 데드락"],
  ["08", "애플리케이션과 잇기", "Docker", "연결 풀 · SQL 인젝션 · N+1"],
  ["09", "MySQL 과 무엇이 다른가", "Docker", "둘을 나란히 띄워 놓고 비교"],
  ["10", "운영과 종합 실습", "Docker", "백업 · 복구 · 권한 · 마무리 실습"],
];

for (const [번호, 제목, 환경, 내용] of 지도) {
  console.log(`[${번호}]단원 [${환경}] ${제목} - ${내용}`);
}
```

출력	01단원 [PGLite] 왜 데이터베이스인가 - 지금 여기입니다
	02단원 [PGLite] 표 설계하기 - 칸과 타입, 제약, NUMERIC·BIGINT 함정
	03단원 [PGLite] 넣고 읽고 고치고 지우기 - INSERT · SELECT · UPDATE · DELETE
	04단원 [PGLite] 데이터 모델링과 정규화 - 표를 몇 개로 쪼갤 것인가, 외래키
	05단원 [PGLite] 여러 표를 잇기 - JOIN · 서브쿼리 · 집계
	06단원 [PGLite] 색인과 실행계획 - 왜 느린지 들여다보고 고치기
	07단원 [Docker] 트랜잭션과 동시성 - 격리수준 · 락 · 데드락
	08단원 [Docker] 애플리케이션과 잇기 - 연결 풀 · SQL 인젝션 · N+1
	09단원 [Docker] MySQL 과 무엇이 다른가 - 둘을 나란히 띄워 놓고 비교
	10단원 [Docker] 운영과 종합 실습 - 백업 · 복구 · 권한 · 마무리 실습

★ 순서대로 보세요. 뒤 단원이 앞 단원을 씁니다. 특히 04(모델링) 없이 05(JOIN) 를 보면 “왜 표를 쪼갰지” 가 안 잡힙니다.

★★ 환경이 두 가지입니다. · **01~06 — PGLite.** 설치가 전혀 필요 없습니다. `node 파일이름.js` 면 끝입니다. · **07~10 — Docker.** 진짜 서버가 있어야 되는 것들이라 그렇습니다

PGLite 가 무엇인가

섹션 2

```
const 버전 = await db.query("SELECT version()");

console.log(버전.rows[0].version.split(" on ")[0]);
```

출력 PostgreSQL 18.3 (PGLite 0.5.7)

★ PGLite 는 PostgreSQL 을 WebAssembly 로 컴파일한 것입니다.

흥내가 아닙니다. 진짜 PostgreSQL 18.3 의 소스코드가 그대로 뜬니다. 그래서 여기서 배운 SQL 이 회사 서버에서 그대로 통합니다.

덕분에 이런 게 됩니다.

- ``npm install`` 한 번 말고는 설치가 없습니다
- 서버를 안 켜도 됩니다. 포트도 없습니다
- 실습을 망쳐도 프로그램만 다시 돌리면 새것입니다

★★★ 그런데 한계가 하나 있습니다. 이걸 꼭 아셔야 합니다.

PGLite 는 연결이 하나뿐입니다. 그래서 “두 사람이 동시에 같은 줄을 고치면?” 을 실습할 수가 없습니다. 진짜로 그런지 재 봅니다.

```
await db.exec("CREATE TABLE 재고 (품목 TEXT PRIMARY KEY, 수량 INT NOT NULL)");
await db.exec("INSERT INTO 재고 VALUES ('볼트', 100)");

const 순서 = [];
```

```
await Promise.all([
  db.transaction(async (거래) => {
    순서.push("가 시작");
    await 거래.query("SELECT pg_sleep(0.05)");
    순서.push("가 끝");
  }),
  db.transaction(async (거래) => {
    순서.push("나 시작");
    await 거래.query("SELECT pg_sleep(0.05)");
    순서.push("나 끝");
  }),
]);

console.log("두 거래의 실행 순서:", 순서.join(" → "));
```

출력 두 거래의 실행 순서: 가 시작 → 가 끝 → 나 시작 → 나 끝

```
console.log("겹쳤나:", 순서[1] !== "가 끝");
```

출력 겹쳤나: false

★★ **Promise.all** 로 **동시에** 던졌는데 **줄을 서서** 처리됐습니다. '가' 가 완전히 끝난 다음에 '나' 가 시작했습니다.

연결이 하나뿐이라 그렇습니다. 겹칠 수가 없습니다.

★ 좋은 점: 개념02 의 동시 쓰기 실험에서 한 건도 안 사라진 이유가 이것이기도 합니다 ★ 나쁜 점: **락·데드락·격리수준을 볼 수가 없습니다**

그래서 07단원부터 Docker 로 진짜 PostgreSQL 을 띄웁니다. 거기서는 연결을 여러 개 열어서 진짜로 부딪혀 봅니다.

트랜잭션 자체는 여기서도 됩니다. 되돌리기를 확인합니다.

```
try {
  await db.transaction(async (거래) => {
    await 거래.query("UPDATE 재고 SET 수량 = 0 WHERE 품목 = '볼트'");
    throw new Error("중간에 문제가 생겼습니다");
  });
} catch (에러) {
  console.log("불잡은 에러:", 에러.message);
}
```

출력 불잡은 에러: 중간에 문제가 생겼습니다

```
const 남은수량 = await db.query("SELECT 수량 FROM 재고 WHERE 품목 = '볼트'");

console.log("볼트 수량:", 남은수량.rows[0].수량);
```

출력 볼트 수량: 100

★★ 0 으로 바꿨는데 **100 그대로**입니다. 콜백이 에러를 던지면 `db.transaction()` 이 **자동으로 되돌립니다**. 개념01 의 “중간에 실패하면 되돌릴 수가 없다” 가 이렇게 풀립니다. 자세한 것은 07단원입니다.

07단원부터 필요한 것

섹션 3

07단원부터는 파일 맨 위에 이런 줄이 붙어 있습니다.

```
// * 이 파일은 Docker 가 필요합니다. docker compose up -d
```

자료 폴더에서 이렇게 컵니다.

```
docker compose up -d      켜기
docker compose down       끄기
docker compose down -v     끄고 데이터까지 지우기
```

★ 포트가 흔한 번호가 아닙니다. **일부러 비켜 났습니다**.

```
PostgreSQL 5433   (5432 아님)
MySQL 3307       (3306 아님)
```

이미 노트북에 데이터베이스가 깔려 있는 사람과 안 부딪히게 하려는 것입니다. 접속이 안 되면 포트부터 확인하세요. 이걸로 제일 많이 헤맸니다.

파일을 읽는 법

섹션 4

단원 폴더는 이렇게 생겼습니다.

01_왜_데이터베이스인가/

```
개념01_....js      읽으면서 실행하는 본문 (단원마다 5개)
개념02_....js
...
연습문제.js TODO 를 채우는 문제
연습문제_정답.js   정답과 그 이유
```

★ 읽기만 하지 말고 반드시 돌려 보세요. 이 자료는 그렇게 만들었습니다.

```
node 개념02_데이터베이스는_어떻게_다른가.js
```

★★ 코드 밑의 // 출력: 주석이 무엇인지 아셔야 합니다.

```
console.log("이 줄 아래를 보세요");
```

출력 이 줄 아래를 보세요

바로 위처럼, // 출력: 은 그 코드를 돌리면 실제로 찍히는 값입니다. 감으로 적은 게 아닙니다. 전부 진짜로 돌려서 나온 것을 붙였습니다.

★ 값이 컴퓨터마다 달라지는 것(시간·메모리)은 물음표가 붙습니다.

```
console.log(`지금 이 프로그램이 켜진 지: ${process.uptime().toFixed(1)} 초`);
```

출력 지금 이 프로그램이 켜진 지: 1.4 초

값이 다를 수 있음

// 출력?: 은 “줄은 나오는데 값은 여러분 컴퓨터마다 다릅니다” 라는 뜻입니다.

★★ 그래서 측정하는 곳에서는 판정을 따로 찍습니다. 개념02 에서 이런 줄을 보셨을 겁니다.

```
console.log("DB 가 파일보다 빠른가:", db수정ms < 92);  
// 출력: DB 가 파일보다 빠른가: true
```

값은 달라도 참거짓은 같아야 합니다. 그게 진짜 확인입니다.

★ 여러분도 이 대조를 기계로 돌려볼 수 있습니다. 자료 폴더에서 이렇게요.

```
node _검증도구/출력검증.cjs 01_왜_데이터베이스인가
```

파일을 전부 실행해서 // 출력: 과 진짜 결과를 한 줄씩 맞춰 봅니다. 불일치 0 이 나오면 여러분 컴퓨터에서도 자료가 맞다는 뜻입니다.

★★ 문제를 풀다가 뭔가 이상하면 이걸 먼저 돌려 보세요.

자료가 틀린 건지 내가 틀린 건지 바로 알립니다.

표 이름과 칸 이름이 한글인 것에 대해

섹션 5

이 자료는 표 이름과 칸 이름을 한글로 씁니다. 설비, 점검기록, 소요분 처럼요.

읽기 쉬우라고 그렇게 했습니다. 처음 배울 때 equipment_inspection_log.duration_minutes 를 읽느라 힘을 빼면 손해입니다.

★★ 그런데 회사에서는 대부분 영어를 씁니다.

설비	→ equipment
점검기록	→ inspection_log
소요분	→ duration_min

이유가 몇 가지 있습니다.

- 도구·라이브러리가 영어 이름을 전제로 만들어진 게 많습니다
- 인코딩이 어긋난 서버에서 한글 이름이 깨지는 일이 있습니다
- 외국 개발자와 같이 일하면 필요합니다

★ 문법은 똑같습니다. 이름만 바꿉니다.

이 자료를 다 본 다음에 영어 이름으로 한 번 다시 짜 보시면 좋습니다.

★ 대소문자는 개념04 에서 본 그대로입니다. PostgreSQL 은 따옴표 없는 이름을 소문자로 바꿉니다. 그래서 영어 이름은 **소문자에 밑줄**이 관례입니다.

★ 막혔을 때 무엇을 하나

섹션 6

① 에러 코드로 검색하세요. 메시지가 아니라 **코드**입니다.

```
const 자주보는코드 = [
  ["42601", "문법이 틀렸습니다", "오타. 콤마 빠짐. 괄호 안 닫음"],
  ["42P01", "그런 표가 없습니다", "표 이름 오타. 아직 CREATE 안 함"],
  ["42703", "그런 칸이 없습니다", "칸 이름 오타. WHERE 에서 별칭 씀"],
  ["42803", "묶음 규칙을 어겼습니다", "GROUP BY 빠짐. WHERE 에 집계 함수"],
  ["22P02", "타입이 안 맞습니다", "숫자 칸에 글자를 넣음"],
  ["23505", "이미 있는 값입니다", "PRIMARY KEY·UNIQUE 중복"],
  ["23502", "비워 둘 수 없습니다", "NOT NULL 칸에 NULL"],
  ["23503", "짜이 없습니다", "없는 설비의 점검기록을 만들려 함"],
];

for (const [코드, 뜻, 흔한원인] of 자주보는코드) {
  console.log(`${코드} - ${뜻} · ${흔한원인}`);
}
```

출력	42601 - 문법이 틀렸습니다 · 오타. 콤마 빠짐. 괄호 안 닫음
	42P01 - 그런 표가 없습니다 · 표 이름 오타. 아직 CREATE 안 함
	42703 - 그런 칸이 없습니다 · 칸 이름 오타. WHERE 에서 별칭 씀
	42803 - 묶음 규칙을 어겼습니다 · GROUP BY 빠짐. WHERE 에 집계 함수
	22P02 - 타입이 안 맞습니다 · 숫자 칸에 글자를 넣음
	23505 - 이미 있는 값입니다 · PRIMARY KEY·UNIQUE 중복
	23502 - 비워 둘 수 없습니다 · NOT NULL 칸에 NULL
	23503 - 짜이 없습니다 · 없는 설비의 점검기록을 만들려 함

★ 앞 두 글자만 봐도 갈래가 잡힙니다.

42 로 시작 → 내가 문장을 잘못 썼습니다
 23 으로 시작 → 문장은 맞는데 **규칙에 걸렸습니다**
 22 로 시작 → 값의 타입이나 범위가 문제입니다

★★ 에러를 이렇게 붙잡으면 코드가 보입니다.

```
try {
  await db.query("INSERT INTO 재고 VALUES ('볼트', 50)");
} catch (에러) {
  console.log(`이름:${에러.name} / 코드:${에러.code}`);
  console.log(`메시지:${에러.message}`);
}
```

```
출력      이름:error / 코드:23505
          메시지:duplicate key value violates unique constraint "재고_pkey"
```

★ `에러.name` 은 그냥 “error” 입니다. 쓸모가 없습니다. **코드를 보세요.**

② `\d` 를 쓸 수 없을 때 — 표와 칸을 눈으로 확인하기

책이나 블로그에는 `\dt`, `\d` 표 이름 이 자주 나옵니다. 그건 psql 이라는 **명령줄 프로그램의 기능**이지 SQL 이 아닙니다. PGlite 에서는 안 됩니다. 대신 SQL 로 물어보면 됩니다.

```
const 표목록 = await db.query(`
  SELECT table_name
  FROM   information_schema.tables
  WHERE  table_schema = 'public'
  ORDER BY table_name
`);

console.log("표 목록:", JSON.stringify(표목록.rows.map((줄) => 줄.table_name)));
```

```
출력      표 목록: ["재고"]
```

```
const 칸목록 = await db.query(`
  SELECT column_name, data_type, is_nullable
  FROM   information_schema.columns
  WHERE  table_name = '재고'
  ORDER BY ordinal_position
`);

console.log(JSON.stringify(칸목록.rows));
```

```
출력      [{"column_name":"품목","data_type":"text","is_nullable":"NO"},
           {"column_name":"수량","data_type":"integer","is_nullable":"NO"}]
```

★ `information_schema` 는 “데이터베이스가 자기 자신을 설명해 둔 표” 입니다. 표준이라 MySQL 에서도 거의 같게 됩니다. ★ 이 두 문장은 앞으로 자주 씁니다. 즐겨찾기 해 두세요.

③ 그래도 안 풀리면

- 문장을 반으로 줄여 보세요. WHERE 를 지우고 돌려 보고, 하나씩 다시 붙입니다 · `SELECT * FROM 표 LIMIT 5` 로 진짜 데이터가 어떻게 생겼는지 먼저 보세요 · 검증도구를 돌려서 자료 쪽이 멀쩡한지 확인하세요
- 각 파일 맨 아래 “자주 하는 실수” 를 보세요. 대부분 거기 있습니다

다 보면 무엇을 할 수 있게 되나

섹션 7

```
const 할수있게되는것 = [
  "요구사항을 듣고 표를 몇 개로 나눌지 정하고 CREATE TABLE 로 옮긴다",
  "여러 표를 JOIN 해서 원하는 보고서를 뽑는다",
  "느린 조회를 EXPLAIN 으로 들여다보고 색인으로 고친다",
  "돈이 오가는 작업을 트랜잭션으로 안전하게 묶는다",
  "Node 애플리케이션에서 연결 풀을 쓰고 SQL 인젝션을 막는다",
  "MySQL 로 된 회사 시스템도 큰 어려움 없이 읽고 고친다",
  "백업을 만들고 진짜로 복구해 본다",
];

for (const [자리, 무엇] of 할수있게되는것.entries()) {
  console.log(`${자리 + 1}. ${무엇}`);
}
```

출력	1. 요구사항을 듣고 표를 몇 개로 나눌지 정하고 CREATE TABLE 로 옮긴다 2. 여러 표를 JOIN 해서 원하는 보고서를 뽑는다 3. 느린 조회를 EXPLAIN 으로 들여다보고 색인으로 고친다 4. 돈이 오가는 작업을 트랜잭션으로 안전하게 묶는다 5. Node 애플리케이션에서 연결 풀을 쓰고 SQL 인젝션을 막는다 6. MySQL 로 된 회사 시스템도 큰 어려움 없이 읽고 고친다 7. 백업을 만들고 진짜로 복구해 본다
----	--

★ 이 일곱 가지가 신입 백엔드 개발자에게 실제로 요구되는 것들입니다. “SQL 좀 할 줄 아세요?” 의 ‘좀’ 이 대략 이 범위입니다.

```
await db.close();
```

정리 — 이 자료 사용 설명서

항목 내용

단원 10개. 순서대로 보세요 01~06 환경 PGLite. 설치 없음. node 파일이름.js 07~10 환경 Docker.
 docker compose up -d (PG 5433, MySQL 3307) PGLite WASM 으로 컴파일한 진짜 PostgreSQL
 18.3 ★ PGLite 한계 연결이 하나뿐 → 동시성 실습은 07단원에서

```
// 출력:           실제로 돌려서 나온 값
// 출력?:          줄은 나오지만 값은 컴퓨터마다 다름
```

막혔을 때 에러 코드(SQLSTATE) 로 검색 \d 대신 information_schema.tables / .columns

★ 여기까지가 01단원입니다. 이제 연습문제를 푸세요. 개념01~05 만으로 풀 수 있게 냈습니다. 정답 파일은 **푼 다음에** 여세요.

직접 해 볼 것

▹ 직접 해보기 1

`node _검증도구/출력검증.cjs 01_왜_데이터베이스인가` 를 돌려 보세요. 몇 개 파일이 몇 줄씩 맞나요? 불일치가 0 인가요?

▹ 직접 해보기 2

개념02 파일의 `// 출력:` 한 줄을 일부러 틀리게 고치고 검증도구를 다시 돌려 보세요. 어떻게 잡아 주나요? (확인한 뒤 원래대로 되돌려 놓으세요)

▹ 직접 해보기 3

섹션 2 의 `pg_sleep(0.05)` 를 `pg_sleep(0.3)` 으로 바꿔 보세요. 전체가 0.6초쯤 걸리나요, 0.3초쯤 걸리나요? 그게 무슨 뜻인가요?

▹ 직접 해보기 4

섹션 6 의 `information_schema.columns` 조회에서 `table_name = '재고'` 를 지우고 돌려 보세요. 몇 줄이 나오나요? 왜 그렇게 많을까요?

▹ 직접 해보기 5

일부러 에러를 다섯 개 내고 코드를 모아 보세요. 섹션 6 의 표에 없는 코드도 하나쯤 찾아보세요.

▹ 직접 해보기 6

섹션 2 의 트랜잭션에서 `throw` 를 지우면 볼트 수량이 얼마가 되나요? 왜 그럴까요?

자주 하는 실수

이런 실수를 합니다

읽기만 하고 안 돌림

이 자료는 돌려 보는 것을 전제로 만들었습니다. 숫자를 바꿔 보고, 일부러 틀려 보고, 에러를 읽는 게 공부입니다.

이런 실수를 합니다

\d 나 \dt 를 SQL 인 줄 알고 씀

그건 psql 프로그램의 기능입니다. 42601 문법 에러가 납니다. `information_schema` 를 쓰세요.

이런 실수를 합니다

PGlite 로 동시성을 실습하려 함

연결이 하나뿐이라 안 겹칩니다. 섹션 2 에서 확인하셨습니다. 락-데드락은 07단원에서 Docker 로 합니다.

이런 실수를 합니다

Docker 포트를 5432 로 씀

이 자료는 5433 과 3307 입니다. 일부러 비켜 봤습니다. 접속이 안 되면 포트부터 보세요.

이런 실수를 합니다

에러 메시지 전체를 그대로 검색함

표 이름·칸 이름이 섞여 있어서 검색이 잘 안 됩니다. `postgres 42703` 처럼 **코드로** 찾으세요. 훨씬 정확합니다.

이런 실수를 합니다

정답 파일을 먼저 봄

답을 보면 “아 그렇구나” 하고 넘어가는데, 그건 배운 게 아닙니다. 틀려서 에러를 읽는 과정이 진짜 공부입니다. 30분은 붙들어 보세요.

이런 실수를 합니다

단원을 건너뛴

05단원(JOIN) 이 어렵다고 느껴지면 04단원(모델링)을 안 본 것입니다. 06단원(색인) 이 안 잡히면 05단원을 다시 보세요. 전부 이어져 있습니다.

연습문제

RUN node 연습문제.js

★ PGlite 가 뜨는 데 1초쯤 걸립니다. 잠깐 멈춰 있어도 정상입니다.

개념01~05 에서 배운 것만으로 풀 수 있습니다. 14문제입니다.

푸는 방법

① // TODO: 가 붙은 줄의 null 을 여러분의 답으로 바꿉니다 ② node 연습문제.js 를 돌립니다 ③ 맨 아래 채점표를 봅니다

답의 모양

- SQL 문제 → 문장을 **글자로** 적습니다. 답.문제1 = "SELECT * FROM 설비";
- 값 문제 → 값을 그대로 적습니다. 답.문제9 = "ORDER BY";

★ 안 푼 문제는 null 로 두세요. 채점표에 ☐ 로 표시됩니다. ★ 마지막 세 문제에는 [도전] 이 붙어 있습니다. 나머지를 먼저 푸세요. ★★ 정답 파일은 **다 풀어 본 다음에** 여세요. 30분은 붙들어 보시길 권합니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

실습용 데이터 (고치지 마세요)

```

await db.exec(`
  CREATE TABLE 설비 (
    id INT PRIMARY KEY,
    이름 TEXT NOT NULL,
    라인 TEXT NOT NULL,
    상태 TEXT NOT NULL
  );

  INSERT INTO 설비 VALUES
    (1, '컨베이어 1호', 'A', '가동'),
    (2, '컨베이어 2호', 'A', '정지'),
    (3, '프레스 1호', 'B', '가동'),
    (4, '프레스 2호', 'B', '가동'),
    (5, '용접로봇 1호', 'C', '정지'),
    (6, '도장기 1호', 'A', '가동');

  CREATE TABLE 점검기록 (
    id SERIAL PRIMARY KEY,
    설비명 TEXT NOT NULL,
    라인 TEXT NOT NULL,
    점검자 TEXT NOT NULL,
    소요분 INT NOT NULL,
    결과 TEXT NOT NULL
  );

  INSERT INTO 점검기록 (설비명, 라인, 점검자, 소요분, 결과) VALUES
    ('컨베이어 1호', 'A', '김반장', 35, '정상'),
    ('컨베이어 2호', 'A', '김반장', 50, '이상'),
    ('프레스 1호', 'B', '이반장', 20, '정상'),
    ('프레스 2호', 'B', '이반장', 65, '이상'),
    ('용접로봇 1호', 'C', '박반장', 15, '정상'),
    ('용접로봇 2호', 'C', '박반장', 80, '이상'),
    ('도장기 1호', 'A', '김반장', 45, '정상');

  CREATE TABLE 설비사양 (
    id INT PRIMARY KEY,

```

```

    이름 TEXT NOT NULL,
    사양 JSONB NOT NULL
);

INSERT INTO 설비사양 VALUES
(1, '컨베이어 1호', '{"제조사":"한성기계","최대속도":120}'),
(2, '프레스 1호', '{"제조사":"대동프레스","압력톤":250}');
`);

const 답 = {};

```

문제 1 — 설비를 전부 가져오기

설비 표의 **모든 줄**을 가져오는 SQL 을 적으세요. (칸도 전부 나와야 합니다)

```

답.문제1 = null; // TODO: "SELECT ..." 로 채우세요

```

문제 2 — 조건으로 거르고 순서 정하기

라인이 'A' 인 설비의 **이름만**, **id 가 작은 순서로** 가져오세요. 기대하는 결과: 컨베이어 1호 → 컨베이어 2호
→ 도장기 1호

```

답.문제2 = null; // TODO: "SELECT ..." 로 채우세요

```

문제 3 — 세기

상태가 '가동' 인 설비가 몇 대인지 세는 SQL 을 적으세요. 칸 이름은 **건수** 로 하세요.

★ **count(*)** 는 BIGINT 입니다. 지금은 값이 작아서 그냥 해도 통과하지만, 개념02 섹션 2 에서 본 대로 **::int** 를 붙이는 습관을 들이세요. 왜 그래야 하는지는 정답 파일에서 설명합니다.

```

답.문제3 = null; // TODO: "SELECT ... AS 건수 FROM ..." 로 채우세요

```

문제 4 — 정렬하고 자르기

점검기록에서 **소요분이 큰 순서로 위 3건의 설비명** 을 가져오세요.

```

답.문제4 = null; // TODO: "SELECT ..." 로 채우세요

```

문제 5 — 묶어서 세기

점검기록을 **점검자별로 묶어서** 건수를 세세요. 칸 이름은 **점검자**, **건수** 로 하고, **점검자 이름 오름차순**으로 정렬하세요.

★ 건수에는 문제 3 과 같이 **::int** 를 붙이세요.

```

답.문제5 = null; // TODO: "SELECT ... GROUP BY ..." 로 채우세요

```


문제 6 — 묶은 것 중에서 고르기

점검자별 **평균 소요분이 40분 이상**인 점검자의 이름만, **평균이 큰 순서**로 가져오세요. 칸 이름은 **점검자**입니다.

★ 줄을 거르는 것은 WHERE, 묶음을 거르는 것은 무엇이었나요?

답.문제6 = null; // TODO: "SELECT ... HAVING ..." 로 채우세요

문제 7 — 에러 코드 맞추기

아래 세 문장은 각각 어떤 SQLSTATE 코드를 낼까요? 다섯 글자 **문자열**로 적으세요. (예: "42601")

① SELEC * FROM 설비 ② SELECT * FROM 없는표 ③ SELECT 이름, 소요분 * 60 AS 소요초 FROM 점검기록 WHERE 소요초 > 1000

답의 모양: { 오타: "?????", 없는표: "?????", WHERE별칭: "?????" }

답.문제7 = null; // TODO: 위 모양대로 세 칸을 채운 객체로 바꾸세요

문제 8 — 실행 순서

SELECT 문장의 **실행 순서**를 배열로 적으세요. 아래 일곱 개를 전부 쓰되 순서만 바꾸면 됩니다.

"SELECT", "FROM", "WHERE", "GROUP BY", "HAVING", "ORDER BY", "LIMIT"

답.문제8 = null; // TODO: ["...", "...", ...] 로 채우세요

문제 9 — 별칭이 통하는 곳

SELECT **소요분 * 60 AS 소요초** 라고 만든 별칭 **소요초** 를 **그대로 쓸 수 있는** 절은 WHERE · HAVING · ORDER BY 중 무엇인가요? 문자열로 적으세요.

답.문제9 = null; // TODO: "WHERE" / "HAVING" / "ORDER BY" 중 하나

문제 10 — JSONB 안을 들여다보기

설비사양 표에서 **제조사**가 '한성기계' 인 **설비**의 **이름**을 가져오세요. 칸 이름은 **이름**입니다.

★ 개념03 섹션 6 에서 화살표 두 개짜리 연산자를 봤습니다.

답.문제10 = null; // TODO: "SELECT ..." 로 채우세요

문제 11 — query 인가 exec 인가

세미콜론으로 이은 **두 문장**을 한 번에 보내려고 합니다. db.query() 와 db.exec() 중 무엇을 써야 하나요? 메서드 이름만 문자열로 적으세요. (예: "query")

답.문제11 = null; // TODO: "query" 또는 "exec"

문제 12 [도전] — 묶고, 거르고, 타입까지 맞추기

점검기록을 **라인별로 묶어서** 아래 세 칸을 뽑으세요.

라인 · 건수 · 평균분

조건이 셋입니다. ① 건수가 2건 이상인 라인만 ② 라인 이름 오름차순 ③ ★ **건수** 와 **평균분** 이 **자바스크립트 숫자**로 와야 합니다

평균분은 소수 첫째 자리까지 (예: 43.3)

★ ③ 이 함정입니다. `round(avg(...), 1)` 은 NUMERIC 이라 **문자열**로 옵니다. 개념04 섹션 3 에서 봤습니다. 숫자로 만들려면 하나 더 붙여야 합니다.

답.문제12 = null; // TODO: "SELECT ... GROUP BY ... HAVING ..." 으로 채우세요

문제 13 [도전] — 어느 쪽이 빠를지 예측하기

10만 건짜리 표에서 아래 두 조회를 쥅니다.

① `SELECT * FROM 큰표 WHERE id = 77777` ← id 는 PRIMARY KEY ② `SELECT * FROM 큰표 WHERE 이름 = '설비77777'` ← 이름에는 색인이 없음

어느 쪽이 빠를까요? "id" 또는 "이름" 으로 적으세요.

★ 채점기가 **진짜로 재서** 여러분 예측과 맞춰 봅니다. 답을 채워야 측정이 됩니다. (그래서 이 문제는 채점이 몇 초 걸립니다)

답.문제13 = null; // TODO: "id" 또는 "이름"

문제 14 [도전] — 오타 난 JSON 키

누가 **설비사양** 에 이렇게 넣었습니다. **압력톤** 을 **압력톤수** 로 오타 냈습니다.

`INSERT INTO 설비사양 VALUES (3, '프레스 3호', '{"압력톤수": 300})'`

이때 `SELECT 사양->>'압력톤' FROM 설비사양 WHERE id = 3` 은 무엇을 돌려줄까요?

"에러" → 에러가 난다
"null" → null 이 나온다
"0" → 0 이 나온다

셋 중 하나를 문자열로 적으세요.

답.문제14 = null; // TODO: "에러" / "null" / "0" 중 하나

여기서부터는 채점 코드입니다. 고치지 마세요.

```

const 채점결과 = [];

function 같나(가, 나) {
  return JSON.stringify(가) === JSON.stringify(나);
}

async function 문장돌리기(문장) {
  const 결과 = await db.query(문장);
  return 결과.rows;
}

async function 채점(번호, 제목, 검사) {
  if (답[`문제${번호}`] === null || 답[`문제${번호}`] === undefined) {
    채점결과.push([번호, 제목, " ", "아직 안 풀었습니다"]);
    return;
  }

  try {
    const 판정 = await 검사();
    채점결과.push(판정 === true ? [번호, 제목, "○", "맞았습니다"] : [번호, 제목,
"✗", 판정]);
  } catch (에러) {
    채점결과.push([번호, 제목, "✗", `에러 [${에러.code ?? "?"}] ${에러.message}`]);
  }
}

async function 준비교(번호, 기대) {
  const 실제 = await 문장돌리기(답[`문제${번호}`]);
  return 같나(실제, 기대) ? true : `나온 값: ${JSON.stringify(실제)}`;
}

await 채점(1, "설비 전부 가져오기", async () => {
  const 실제 = await 문장돌리기(답.문제1);
  if (실제.length !== 6) return `6줄이어야 하는데 ${실제.length}줄입니다`;
  return 같나(실제[0], { id: 1, 이름: "컨베이어 1호", 라인: "A", 상태: "가동" })
    ? true
    : `첫 줄이 다릅니다: ${JSON.stringify(실제[0])}`;
});

```

```

});

await 채점(2, "A라인 설비 이름", () =>
  준비교(2, [{ 이름: "컨베이어 1호" }, { 이름: "컨베이어 2호" }, { 이름: "도장기 1호"
  }]),
);

await 채점(3, "가동 중인 설비 수", () => 준비교(3, [{ 건수: 4 }]));

await 채점(4, "오래 걸린 점검 3건", () =>
  준비교(4, [{ 설비명: "용접로봇 2호" }, { 설비명: "프레스 2호" }, { 설비명:
  "컨베이어 2호" }]),
);

await 채점(5, "점검자별 건수", () =>
  준비교(5, [
    { 점검자: "김반장", 건수: 3 },
    { 점검자: "박반장", 건수: 2 },
    { 점검자: "이반장", 건수: 2 },
  ]),
);

await 채점(6, "평균 40분 이상 점검자", () =>
  준비교(6, [{ 점검자: "박반장" }, { 점검자: "김반장" }, { 점검자: "이반장" }]),
);

await 채점(7, "에러 코드 세 개", async () =>
  같나(답.문제7, { 오타: "42601", 없는표: "42P01", WHERE별칭: "42703" })
    ? true
    : `나온 값: ${JSON.stringify(답.문제7)}`,
);

await 채점(8, "실행 순서", async () =>
  같나(답.문제8, ["FROM", "WHERE", "GROUP BY", "HAVING", "SELECT", "ORDER BY",
  "LIMIT"])
    ? true
    : `나온 값: ${JSON.stringify(답.문제8)}`,
);

```

```

await 채점(9, "별칭이 통하는 절", async () =>
  답.문제9 === "ORDER BY" ? true : `나온 값: ${JSON.stringify(답.문제9)}`,
);

await 채점(10, "JSONB 로 제조사 찾기", () => 준비교(10, [{ 이름: "컨베이어 1호" }]));

await 채점(11, "query 인가 exec 인가", async () =>
  답.문제11 === "exec" ? true : `나온 값: ${JSON.stringify(답.문제11)}`,
);

await 채점(12, "[도전] 라인별 건수와 평균", () =>
  준비교(12, [
    { 라인: "A", 건수: 3, 평균분: 43.3 },
    { 라인: "B", 건수: 2, 평균분: 42.5 },
    { 라인: "C", 건수: 2, 평균분: 47.5 },
  ]),
);

await 채점(13, "[도전] 어느 쪽이 빠른가", async () => {
  await db.exec(`
    CREATE TABLE 큰표 (id INT PRIMARY KEY, 이름 TEXT NOT NULL);
    INSERT INTO 큰표 SELECT 번호, '설비' || 번호 FROM generate_series(1, 100000) AS
    번호;
  `);

  const 재기 = async (문장, 값) => {
    const 잔것 = [];
    for (let 회차 = 0; 회차 < 5; 회차 += 1) {
      const 시작 = performance.now();
      await db.query(문장, [값]);
      잔것.push(performance.now() - 시작);
    }
    return 잔것.sort((가, 나) => 가 - 나)[2];
  };

  const idms = await 재기("SELECT * FROM 큰표 WHERE id = $1", 77777);
  const 이름ms = await 재기("SELECT * FROM 큰표 WHERE 이름 = $1", "설비77777");

```

```
const 빠른쪽 = idms < 이름ms ? "id" : "이름";

return 답.문제13 === 빠른쪽 ? true : `실제로 빠른 쪽은 "${빠른쪽}" 입니다`;
});

await 채점(14, "[도전] 오타 난 JSON 키", async () => {
  await db.exec(`INSERT INTO 설비사양 VALUES (3, '프레스 3호', '{"압력톤수":
300}')`);
  const 결과 = await db.query("SELECT 사양->>'압력톤' AS 값 FROM 설비사양 WHERE id =
3");
  const 진짜 = 결과.rows[0].값 === null ? "null" : String(결과.rows[0].값);
  return 답.문제14 === 진짜 ? true : `실제로는 "${진짜}" 입니다`;
});

console.log("===== 01단원 연습문제 채점 =====");
```

```
출력      ===== 01단원 연습문제 채점 =====
```

```
for (const [번호, 제목, 표시, 설명] of 채점결과) {
  console.log(`${String(번호).padStart(2, "0")}. ${표시} ${제목} - ${설명}`);
}
```

```
출력      01. ☐ 설비 전부 가져오기 - 아직 안 풀었습니다
02. ☐ A라인 설비 이름 - 아직 안 풀었습니다
03. ☐ 가동 중인 설비 수 - 아직 안 풀었습니다
04. ☐ 오래 걸린 점검 3건 - 아직 안 풀었습니다
05. ☐ 점검자별 건수 - 아직 안 풀었습니다
06. ☐ 평균 40분 이상 점검자 - 아직 안 풀었습니다
07. ☐ 에러 코드 세 개 - 아직 안 풀었습니다
08. ☐ 실행 순서 - 아직 안 풀었습니다
09. ☐ 별칭이 통하는 절 - 아직 안 풀었습니다
10. ☐ JSONB 로 제조사 찾기 - 아직 안 풀었습니다
11. ☐ query 인가 exec 인가 - 아직 안 풀었습니다
12. ☐ [도전] 라인별 건수와 평균 - 아직 안 풀었습니다
13. ☐ [도전] 어느 쪽이 빠른가 - 아직 안 풀었습니다
14. ☐ [도전] 오타 난 JSON 키 - 아직 안 풀었습니다
```

```
const 맞은수 = 채점결과.filter(([, , 표시]) => 표시 === "○").length;

console.log(`===== ${맞은수} / ${채점결과.length} 맞았습니다 =====`);
```

```
출력      ===== 0 / 14 맞았습니다 =====
```

```
console.log("전부 ○ 가 되면 정답 파일을 열어서 설명을 읽어 보세요.");
```

```
출력      전부 ○ 가 되면 정답 파일을 열어서 설명을 읽어 보세요.
```

★ `padStart` 는 숫자에만 썼습니다. 한글에 `padEnd` 를 쓰면 칸이 어긋납니다. 개념01 섹션 8 에서 그 이유를 봤습니다.

```
await db.close();
```

힌트 (막혔을 때만 보세요) —————

문제 3 · `count(*)` 뒤에 `::int` 를 붙이면 값이 커져도 자바스크립트 숫자입니다.

문제 6 · 묶은 뒤에 거르는 절은 HAVING 입니다.

그리고 HAVING 에서는 별칭을 못 씁니다. 식을 다시 쓰세요.
정렬은 SELECT 다음에 도니까 별칭을 써도 됩니다.

문제 7 · 개념04 섹션 4 와 섹션 8 에 세 코드가 전부 나옵니다.

문제 10 · 사양->>‘제조사’ = ‘한성기계’ 로도 되고

``사양 @> '{"제조사":"한성기계"}'`` 로도 됩니다. 둘 다 정답입니다.

문제 12 · `round(avg(소요분), 1)` 은 NUMERIC 이라 **문자열**로 옵니다.

``count(*)::int`` 처럼 뒤에 하나 더 붙여서 숫자로 만드세요.

표 설계하기

OPEN 02_표_설계하기

```
await db.exec(`
CREATE TABLE 설비 (
설비번호 INT,
이름 TEXT,
```

01 표와 칸 만들기

02 타입 고르기

03 제약으로 규칙을 박습니다

04 기본키를 무엇으로

05 NULL 은 값이 아닙니다

- 연습문제

표와 칸 만들기

RUN node 개념01_표와_칸_만들기.js

★ PGlite 는 시작에 1초쯤 걸립니다. 잠깐 멈춰 있어도 정상입니다.

01단원에서 파일 저장의 한계를 재 봤습니다. 그중 하나가 **규칙을 못 지킨다**였습니다. 라인 칸에 999 를 넣어도 파일은 받아 줍니다.

데이터베이스는 그 규칙을 **표를 만들 때** 적어 둡니다. 그래서 이 단원이 중요합니다.

여기서 꼼꼼히 적으면 → 뒤에 쓰는 코드가 짧아집니다 여기서 대충 적으면 → 그 뒤 전부가 고생합니다

이 개념에서는 표를 만드는 문법만 봅니다. 타입은 개념02, 규칙(제약)은 개념03 에서 합니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

표·줄·칸

섹션 1

엑셀을 떠올리면 거의 맞습니다. 이름만 다릅니다.

엑셀에서 데이터베이스에서 영어

시트	표	table
행	줄	row (record 라고도 함)
열	칸	column (field 라고도 함)
셀	값	value

★ 다른 점이 하나 있습니다. **칸마다 타입이 정해져 있습니다**. 엑셀은 같은 열에 숫자와 글자를 섞어도 됩니다. 표는 안 됩니다. 그 "안 됨" 이 우리가 원하는 것입니다.

```
console.log("표 = 시트 / 줄 = 행 / 칸 = 열");
```

```
출력      표 = 시트 / 줄 = 행 / 칸 = 열
```

CREATE TABLE 을 한 줄씩

섹션 2

설비 표를 만듭니다. 한 줄씩 뜯어봅니다.

CREATE TABLE 설비 (← 설비 라는 이름의 표를 만들어라

설비번호 INT,	← 설비번호 칸, 정수	┌
이름 TEXT,	← 이름 칸, 글자	칸 정의는 쉼표로 있습니다
라인 TEXT,	← 라인 칸, 글자	
가동중 BOOLEAN	← 가동중 칸, 참/거짓	└ * 마지막에는 쉼표 없음

); ← 닫고 세미콜론

규칙은 셋뿐입니다. ① 칸 하나가 "이름 타입" 한 쌍 ② 칸과 칸 사이는 쉼표 ③ 마지막 칸 뒤에는 쉼표를 붙이지 않습니다 ← 여기서 제일 많이 틀립니다

```
await db.exec(`
  CREATE TABLE 설비 (
    설비번호 INT,
    이름 TEXT,
    라인 TEXT,
    가동중 BOOLEAN
  );
`);

console.log("설비 표를 만들었습니다");
```

출력 설비 표를 만들었습니다

마지막에 쉼표를 남기면 이렇게 됩니다. 진짜로 해 봅니다.

```
try {
  await db.exec(`CREATE TABLE 쉼표시험 (가 INT, 나 TEXT,)`);
} catch (에러) {
  console.log(`${에러.code} - ${에러.message}`);
}
```

출력 42601 - syntax error at or near ")"

}

★ 42601 은 **문법 오류** 입니다. "at or near ')" — 닫는 괄호 근처가 이상하다는 뜻입니다. Postgres 는 "쉼표를 지우세요" 라고 말해 주지 않습니다. 위치만 알려 줍니다.

표의 구조는 `information_schema.columns` 에서 볼 수 있습니다. 이건 Postgres 가 자기 정보를 담아 두는 표입니다. 표를 조회하듯이 조회합니다.

```
const 칸들 = await db.query(`
  SELECT column_name, data_type, is_nullable
  FROM information_schema.columns
  WHERE table_name = '설비'
  ORDER BY ordinal_position
`);

for (const 칸 of 칸들.rows) {
  console.log(`${칸.column_name} · ${칸.data_type} · NULL 허용=${칸.is_nullable}`);
}
```

```
출력      설비번호 · integer · NULL 허용=YES
          이름 · text · NULL 허용=YES
          라인 · text · NULL 허용=YES
          가동중 · boolean · NULL 허용=YES
```

★ 두 가지가 눈에 띕니다. ① INT 라고 썼는데 integer 라고 나옵니다. INT 는 integer 의 줄임말입니다 ② NULL 허용이 전부 YES 입니다. **아무것도 안 적으면 다 비워도 됩니다**

이게 사고의 시작입니다. 개념03 에서 막습니다.

★ psql 에서는 \d 설비 한 줄이면 같은 것을 봅니다. PGlite 에는 \d 가 없어서 `information_schema` 를 직접 조회합니다.

★ 이름의 대소문자 — Postgres 는 소문자로 접습니다

이건 반드시 알아야 합니다. 실제로 재 봅니다.

```
await db.exec(`CREATE TABLE Equipment (Id INT, MachineName TEXT)`);
await db.exec(`CREATE TABLE "Equipment2" ("Id" INT, "MachineName" TEXT)`);

const 표들 = await db.query(`
  SELECT table_name FROM information_schema.tables
  WHERE table_schema = 'public'
  ORDER BY table_name
`);

console.log("만들어진 표:", 표들.rows.map((표) => 표.table_name).join(", "));
```

```
출력      만들어진 표: Equipment2, equipment, 설비
```

★★ CREATE TABLE Equipment 라고 대문자로 썼는데 **equipment** 가 됐습니다. 따옴표로 감싼 “Equipment2” 만 대문자가 살아남았습니다.

Postgres 는 따옴표 없는 이름을 **전부 소문자로 접습니다**. 칸 이름도 마찬가지입니다.

```
const 칸이름들 = await db.query(`
  SELECT table_name, column_name FROM information_schema.columns
  WHERE table_name IN ('equipment', 'Equipment2')
  ORDER BY table_name, ordinal_position
`);

for (const 칸 of 칸이름들.rows) {
  console.log(`${칸.table_name} → ${칸.column_name}`);
}
```

```
출력      Equipment2 → Id
          Equipment2 → MachineName
          equipment → id
          equipment → machinename
```

MachineName 이 machinename 이 됐습니다. 낙타등(camelCase)이 뭉개졌습니다.

그래서 이런 일이 벌어집니다.

```
const 대문자로조회 = await db.query(`SELECT * FROM EQUIPMENT`);
console.log("EQUIPMENT 로 조회한 칸:", 대문자로조회.fields.map((칸) =>
  칸.name).join(", "));
```

```
출력      EQUIPMENT 로 조회한 칸: id, machinename
```

대문자로 써도 소문자로 접혀서 equipment 를 찾습니다. 잘 됩니다. 그런데 따옴표로 만든 표는 반대입니다.

```
try {
  await db.query(`SELECT * FROM Equipment2`);
} catch (에러) {
  console.log(`${에러.code} - ${에러.message}`);
}
```

```
출력      42P01 - relation "equipment2" does not exist
```

```
}
```

★★★ 여기가 함정입니다. “Equipment2” 로 만들어 놓으면, 그 뒤로 **영원히 따옴표를 달아야** 합니다. 한 번이라도 빼먹으면 42P01 (없는 표) 입니다.

★ 결론: 따옴표를 쓰지 마세요.

이름은 처음부터 소문자와 밑줄로만 씁니다. `machine\_name` 처럼요.
그러면 대소문자를 신경 쓸 일이 없습니다.

```
console.log("따옴표 없이 만든 이름은 소문자가 된다:", 표들.rows.some((표) =>
표.table_name === "equipment"));
```

출력 따옴표 없이 만든 이름은 소문자가 된다: true

한글 이름은 되나

섹션 5

됩니다. 위에서 이미 **설비**, **설비번호** 로 만들었습니다.

한글에는 대소문자가 없으니 접힐 것도 없습니다. 따옴표 없이 그냥 씁니다.

★ 이 자료는 배우기 쉬우라고 한글 이름을 씁니다. **회사에서는 영어를 훨씬 많이 씁니다.** 이유가 있습니다.

- 도구·라이브러리가 영어 이름을 가정하고 만들어졌습니다
- 자바스크립트 객체 키가 한글이면 `줄.설비번호` 는 되는데 `줄.설비-번호` 같은 건 안 됩니다
- 외국 개발자와 같이 일하면 못 읽습니다

그러니 배울 때는 한글, 실무에서는 팀 규칙을 따르세요.

이름에 쓸 수 없는 것

· 숫자로 시작 → 2호기 (X) 호기2 (O) · 하이픈 → 설비-번호 (X) 설비_번호 (O) · 공백 → 설비 번호 (X) ·
예약어 → order, user, table ← 따옴표 없이는 못 씁니다

실제로 해 봅니다.

```
for (const 나쁜이름 of ["2호기", "설비-번호"]) {
  try {
    await db.exec(`CREATE TABLE ${나쁜이름} (가 INT)`);
    console.log(`${나쁜이름} - 만들어짐`);
  } catch (에러) {
    console.log(`${나쁜이름} - ${에러.code} ${에러.message}`);
  }
}
```

출력 2호기 - 42601 trailing junk after numeric literal at or near "2호기"
설비-번호 - 42601 syntax error at or near "-"

★ **설비-번호** 는 Postgres 가 "설비 빼기 번호" 로 읽습니다. 그래서 문법 오류입니다.

★ 예약어는 조금 다릅니다. **order** 는 표 이름으로 쓸 수 없지만 **user** 는 상황에 따라 됩니다. 외우지 말고
그냥 **피하세요**. **주문**, **사용자** 처럼 다른 이름을 쓰면 됩니다.

이미 있는 표를 또 만들면

섹션 6

```
try {
  await db.exec(`CREATE TABLE 설비 (설비번호 INT)`);
} catch (에러) {
  console.log(`${에러.code} - ${에러.message}`);
}
```

출력 42P07 - relation "설비" already exists

```
}
```

★ 42P07 = “그 이름의 것이 이미 있다”. relation 은 표·뷰·시퀀스를 아우르는 말입니다. 여기서는 표입니다.

실습 파일을 여러 번 돌리면 이 에러를 반드시 만납니다. 대처는 둘입니다.

① 있으면 넘어가라 → CREATE TABLE IF NOT EXISTS ② 있으면 지우고 다시 → DROP TABLE IF EXISTS 뒤에 CREATE

```
const 넘어감 = await db.exec(`CREATE TABLE IF NOT EXISTS 설비 (설비번호 INT)`);
console.log("IF NOT EXISTS 결과:", 넘어감[0].command, ". 에러 없음");
```

출력 IF NOT EXISTS 결과: CREATE · 에러 없음

★★ 여기서 착각하기 쉽습니다. **표를 고쳐 주지 않습니다.** 위에서 설비 표는 칸이 네 개인데, 방금 (설비번호 INT) 하나로 다시 만들려 했습니다. IF NOT EXISTS 는 그냥 아무것도 안 했습니다. 칸은 그대로 네 개입니다.

```
const 칸수 = await db.query(`
  SELECT count(*)::int AS 개수 FROM information_schema.columns WHERE table_name =
  '설비'
`);
console.log("설비 표의 칸 수:", 칸수.rows[0].개수);
```

출력 설비 표의 칸 수: 4

★ 그래서 실습에서는 ② 가 안전합니다.

```
DROP TABLE IF EXISTS 설비;
CREATE TABLE 설비 (...);
```

운영 서버에서는 절대 이러면 안 됩니다. 데이터가 다 날아갑니다.

```
try {
  await db.exec(`DROP TABLE 없는표`);
} catch (에러) {
  console.log(`${에러.code} - ${에러.message}`);
}
```

출력 42P01 – table "없는표" does not exist

```
}

const 조용히 = await db.exec(`DROP TABLE IF EXISTS 없는표`);
console.log("DROP IF EXISTS:", 조용히[0].command, "· 에러 없음");
```

출력 DROP IF EXISTS: DROP · 에러 없음

```
await db.exec(`DROP TABLE IF EXISTS Equipment, "Equipment2"`);
console.log("치웠습니다");
```

출력 치웠습니다

★★★ DROP TABLE 은 되돌릴 수 없습니다. 휴지통이 없습니다. 확인 창도 없습니다. 그냥 사라집니다. 운영에서 표를 지울 일이 있으면 백업부터 하세요. 10단원에서 합니다.

ALTER TABLE — 나중에 고치기

표를 만든 뒤에 “아 도입일도 넣을걸” 하는 일이 반드시 생깁니다. 그때 다시 만들 필요는 없습니다. 고치면 됩니다.

```
await db.exec(`ALTER TABLE 설비 ADD COLUMN 도입일 DATE`); // 칸 추가
await db.exec(`ALTER TABLE 설비 RENAME COLUMN 이름 TO 설비명`); // 칸 이름 바꾸기
await db.exec(`ALTER TABLE 설비 DROP COLUMN 가동중`); // 칸 삭제
await db.exec(`ALTER TABLE 설비 ALTER COLUMN 라인 TYPE VARCHAR(1)`); // 칸 타입
바꾸기 const 고친뒤 = await db.query(`
  SELECT column_name, data_type FROM information_schema.columns
  WHERE table_name = '설비' ORDER BY ordinal_position
`);

for (const 칸 of 고친뒤.rows) {
  console.log(`${칸.column_name} · ${칸.data_type}`);
}
```



```
출력      설비번호 · integer
          설비명 · text
          라인 · character varying
          도입일 · date
```

★ 순서를 보세요. 새로 넣은 도입일이 맨 뒤로 갔습니다. Postgres 에서 칸 순서는 바꿀 수 없습니다. 중간에 끼워 넣을 수도 없습니다. 불편해 보이지만, 사실 SELECT 에서 칸 이름을 적으면 순서는 아무 상관이 없습니다.

표 이름도 바꿉니다.

```
await db.exec(`ALTER TABLE 설비 RENAME TO 설비목록`);
const 있나 = await db.query(`
  SELECT count(*)::int AS 개수 FROM information_schema.tables WHERE table_name =
  '설비목록'
`);
console.log("설비목록 이라는 표가 있나:", 있나.rows[0].개수 === 1);
```

```
출력      설비목록 이라는 표가 있나: true
```

★★ ALTER TABLE 은 운영 중에도 됩니다. 그런데 조심할 게 있습니다.

· ADD COLUMN (기본값 없이) → 거의 즉시. 안전합니다 · DROP COLUMN → 즉시. 다만 데이터가 사라집니다 · ALTER COLUMN TYPE → 표 전체를 다시 씁니다. 큰 표면 몇 분씩 멈춥니다 · RENAME COLUMN → 즉시. 다만 그 이름을 쓰던 코드가 전부 깨집니다

그래서 처음에 잘 만드는 것 이 훨씬 쉽습니다. 이 단원의 목적이 그것입니다.

```
await db.exec(`DROP TABLE 설비목록`);
```

query 와 exec 는 무엇이 다른가

섹션 9

PGlite 에는 두 가지가 있습니다. 헷갈리면 계속 걸립니다.

```
const q = await db.query(`SELECT 1 AS 가, '나' AS 나`);
console.log("query 결과:", JSON.stringify(q.rows), ".", q.command, ". rowCount", q.rowCount);
```

```
출력      query 결과: [{"가":1,"나":"나"}] · SELECT · rowCount 1
```

```
const e = await db.exec(`CREATE TABLE 임시 (a INT); INSERT INTO 임시 VALUES (1), (2);`);
console.log("exec 결과 개수:", e.length, ".", e.map((하나) => `${하나.command}:${하나.affectedRows}`).join(" "));
```

```
출력      exec 결과 개수: 2 · CREATE:0 INSERT:2
```

★ 정리 db.query(sql, [값]) → 문장 **하나**. 파라미터를 넘길 수 있습니다. 결과가 객체 하나 db.exec(sql)
→ 세미콜론으로 이은 **여러 문장**. 파라미터 없음. 결과가 배열

★★ query 에 두 문장을 넣으면 이렇게 됩니다.

```
try {
  await db.query(`SELECT 1; SELECT 2;`);
} catch (에러) {
  console.log("query 에 두 문장:", 에러.message);
}
```

출력 query 에 두 문장: cannot insert multiple commands into a prepared statement

```
}
```

★ 파라미터는 \$1, \$2 입니다. ? 가 아닙니다.

```
const 파라미터 = await db.query(`SELECT $1::int + $2::int AS 합`, [3, 4]);
console.log("파라미터 결과:", 파라미터.rows[0].합);
```

출력 파라미터 결과: 7

```
await db.exec(`DROP TABLE 임시`);
```

MySQL 은 여기가 다릅니다

· 파라미터가 \$1 이 아니라 ? 입니다 · 이름을 소문자로 접지 않습니다. 대신 **운영체제에 따라** 대소문자 구분이 달라집니다

(리눅스는 구분, 맥/윈도우는 구분 안 함 - 그래서 옮길 때 터집니다)

· TEXT 와 VARCHAR 의 성격이 다릅니다. 개념02 에서 잠깐 언급합니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 표를 만들고 고치는 문장

무엇을 문장

표 만들기 CREATE TABLE 이름 (칸 타입, 칸 타입) 있으면 넘어가기 CREATE TABLE IF NOT EXISTS ...
표 지우기 DROP TABLE 이름 없어도 조용히 DROP TABLE IF EXISTS 이름 칸 추가 ALTER TABLE 표
ADD COLUMN 칸 타입 칸 삭제 ALTER TABLE 표 DROP COLUMN 칸 칸 이름 바꾸기 ALTER TABLE
표 RENAME COLUMN 옛 TO 새 칸 타입 바꾸기 ALTER TABLE 표 ALTER COLUMN 칸 TYPE 새타입

표 이름 바꾸기 ALTER TABLE 표 RENAME TO 새이름 구조 보기 SELECT * FROM information_schema.columns WHERE table_name = '표'

에러 코드 뜻

42601	문법 오류 (쉼표, 괄호, 오타)
42P01	그런 표가 없다
42P07	그 이름이 이미 있다

★ 오늘의 핵심 두 가지 ① Postgres 는 따옴표 없는 이름을 **소문자로 접습니다**. 그러니 처음부터 소문자로 쓰세요 ② ALTER 로 나중에 고칠 수 있지만 **싸지 않습니다**. 처음에 잘 만드는 게 낫습니다

직접 해 볼 것

▫ 직접 해보기 1

점검기록 표를 만들어 보세요. 칸: 점검번호(INT), 설비번호(INT), 점검일(DATE), 결과(TEXT) 만든 뒤 information_schema 로 확인하세요.

▫ 직접 해보기 2

섹션 4 를 흉내 내서 “MyTable” 을 만들고, 따옴표 없이 SELECT * FROM MyTable 을 해 보세요. 어떤 에러가 나나요? 왜 그럴까요?

▫ 직접 해보기 3

이 파일을 두 번 연달아 실행하면 어떻게 되나요? (힌트: PGlite.create() 는 메모리에만 만듭니다. 매번 새것입니다) PGlite.create(“./내디비”) 로 바꾸고 두 번 돌려 보세요. ★ 폴더가 생깁니다. 실험이 끝나면 지우세요.

▫ 직접 해보기 4

표를 만들 때 칸 이름을 order 로 해 보세요. 되나요? user 는요? select 는요?

▫ 직접 해보기 5

ALTER TABLE 로 칸을 지웠다가 다시 추가해 보세요. 지웠던 값이 돌아오나요?

▫ 직접 해보기 6

information_schema.tables 대신 pg_tables 를 조회해 보세요. 어떤 칸이 더 있나요? (힌트: SELECT * FROM pg_tables WHERE schemaname='public')

이런 실수를 합니다

마지막 칸 뒤에 쉼표를 남김

가 INT, 나 TEXT,) → 42601. 자바스크립트 배열 버릇이 그대로 나옵니다. JS 는 [1, 2,] 를 받아 주지만 SQL 은 안 받습니다.

이런 실수를 합니다

낙타등 이름을 쓰고 그대로 조회

CREATE TABLE machineLog 하면 실제로는 machinelog 가 됩니다. 나중에 “machineLog” 로 조회하면 42P01 입니다. ★ 처음부터 machine_log 로 쓰세요.

이런 실수를 합니다

IF NOT EXISTS 를 “고쳐 준다” 고 오해

표가 이미 있으면 아무것도 안 합니다. 칸을 늘려 주지 않습니다. 구조를 바꾸려면 ALTER 를 쓰거나 DROP 하고 다시 만들어야 합니다.

이런 실수를 합니다

운영 서버에서 DROP TABLE

휴지통이 없습니다. 되돌릴 수 없습니다. 습관적으로 DROP TABLE IF EXISTS 를 파일 맨 위에 적어 두면 언젠가 사고가 납니다.

이런 실수를 합니다

query 에 세미콜론으로 두 문장

"cannot insert multiple commands into a prepared statement" 가 납니다. 여러 문장은 exec 를 쓰세요. 대신 exec 에는 파라미터를 못 넘깁니다.

이런 실수를 합니다

파라미터를 ? 로 씀

WHERE 라인 = ? 는 Postgres 에서 안 됩니다. \$1 입니다. MySQL·SQLite 예제를 그대로 베끼면 여기서 걸립니다.

```
await db.close();
```

타입 고르기

RUN node 개념02_타입_고르기.js

★★★ 이 단원에서 가장 중요한 파일입니다. 천천히 보세요.

개념01 에서 INT, TEXT 를 아무 설명 없이 썼습니다. 이제 제대로 봅시다. 여기서 잘못 고르면 몇 달 뒤에 돈 계산이 틀어지거나 새벽에 서버가 터집니다.

이 파일에서 실제로 터뜨려 볼 세 가지입니다.

① BIGINT 를 꺼내서 JSON 으로 만들면 **터집니다** ② 돈을 REAL 로 저장하면 **1원씩 틀립니다** ③ NUMERIC 을 그냥 더하면 **글자가 이어붙습니다**

셋 다 에러 메시지가 친절하지 않습니다. 미리 알아야 안 당합니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

타입은 문지기입니다

섹션 1

01단원에서 본 파일 저장의 한계 중 하나가 “규칙을 못 지킨다” 였습니다. 타입이 그 첫 번째 문지기입니다.

```
await db.exec(`CREATE TABLE 문지기 (설비번호 INT, 이름 TEXT)`);

try {
  await db.query(`INSERT INTO 문지기 VALUES ('가나', '컨베이어 1호')`);
} catch (에러) {
  console.log(`${에러.code} - ${에러.message}`);
}
```

출력 22P02 – invalid input syntax for type integer: "가나"

```
}
```

★ 22P02 = “그 타입으로 읽을 수 없는 글자”. JSON 파일이었다면 그냥 들어갔습니다. 반대로 읽을 수 있는 글자는 알아서 바꿔 줍니다.

```
const 바뀜 = await db.query(`SELECT '42'::int AS 숫자로, 42::text AS 글자로`);
console.log("'42'::int →", 바뀜.rows[0].숫자로, typeof 바뀜.rows[0].숫자로);
```

출력 '42'::int → 42 number

```
console.log("42::text →", 바뀜.rows[0].글자로, typeof 바뀜.rows[0].글자로);
```

```
출력      42::text → 42 string
```

★ **::타입** 은 “이 값을 이 타입으로 보라” 는 뜻입니다. **캐스팅(cast)** 이라고 합니다.

전부 한 번에 재 봅니다

섹션 2

표에 담긴 값이 **자바스크립트로 나올 때 무엇이 되는지** 가 이 개념의 핵심입니다. 전부 넣고 한 번에 꺼내 봅니다.

```
await db.exec(`
  CREATE TABLE 타입시험 (
    가_int      INT,
    나_bigint    BIGINT,
    다_smallint  SMALLINT,
    라_numeric   NUMERIC(10,2),
    마_real      REAL,
    바_double    DOUBLE PRECISION,
    사_text      TEXT,
    아_varchar   VARCHAR(10),
    자_char      CHAR(5),
    차_bool      BOOLEAN,
    카_date      DATE,
    타_timestamp TIMESTAMP,
    파_timestampz TIMESTAMPTZ,
    하_jsonb     JSONB,
    거_array     TEXT[],
    너_uuid      UUID
  );
`);

await db.query(`
  INSERT INTO 타입시험 VALUES (
    42, 9007199254740993, 7, 1.5, 1.5, 1.5,
    '글자', '짧게', 'A', true,
    '2026-03-15', '2026-03-15 23:30:00', '2026-03-15 23:30:00+09',
    '{"a":1}', ARRAY['A','B'], '11111111-2222-3333-4444-555555555555'
  )
`);

const 한줄 = (await db.query(`SELECT * FROM 타입시험`)).rows[0];

for (const [칸, 값] of Object.entries(한줄)) {
```

제약으로 규칙을 박습니다

RUN node 개념03_제약으로_규칙을_박습니다.js

개념02 에서 타입이 첫 번째 문지기라고 했습니다. 그런데 타입만으로는 부족합니다.

라인 TEXT → 'A' 도 되고 '999' 도 되고 " 도 됩니다 상태 TEXT → '가동' 도 되고 '가동' 도 됩니다 설비번호 INT → 비워 뒤도 되고, 같은 번호가 열 개 있어도 됩니다

이걸 막는 것이 **제약(constraint)** 입니다.

★ 이 개념의 핵심 메시지는 하나입니다.

제약을 걸면 **애플리케이션 코드가 줄어듭니다.**

“DB 는 저장만 하고 검사는 코드에서” 라고 생각하기 쉽습니다. 그러면 검사 코드를 곳곳에 복사하게 되고, 언젠가 한 군데를 빠뜨립니다. 그날부터 쓰레기 데이터가 들어옵니다. 그리고 아무도 모릅니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

제약이 없으면 무엇이 들어오나

섹션 1

진짜로 넣어 봅니다. 하나도 안 막힙니다.

```
await db.exec(`
  CREATE TABLE 느슨한설비 (
    설비번호 INT,
    이름 TEXT,
    라인 TEXT,
    상태 TEXT,
    도입일 DATE
  );
`);

await db.exec(`
  INSERT INTO 느슨한설비 VALUES
    (1, '컨베이어 1호', 'A', '가동', '2020-01-05'),
    (1, '컨베이어 1호', 'A', '가동', '2020-01-05'),
    (NULL, NULL, '999', '가동', NULL),
    (3, '', '', ' ', '1899-01-01');
`);
```

```

`);

const 쓰레기 = (await db.query(`
  SELECT 설비번호, 이름, 라인, 상태 FROM 느슨한설비 ORDER BY 설비번호 NULLS LAST,
  이름
`)).rows;

for (const 줄 of 쓰레기) {
  console.log(JSON.stringify(줄));
}

```

```

출력      {"설비번호":1,"이름":"컨베이어 1호","라인":"A","상태":"가동"}
          {"설비번호":1,"이름":"컨베이어 1호","라인":"A","상태":"가동"}
          {"설비번호":3,"이름":"","라인":"","상태":""}
          {"설비번호":null,"이름":null,"라인":"999","상태":"가동"}

```

★★★ 네 줄 전부 문제가 있습니다.

· 1번이 두 번 — 같은 설비가 두 대인지, 실수로 두 번 넣은 건지 알 수 없습니다 · 3번은 이름이 **빈 문자열** — 화면에 아무것도 안 나옵니다 · 3번의 상태는 **공백 한 칸** — 상태 === '가동' 도 아니고 빈 문자열도 아닙니다 · NULL 줄은 번호도 이름도 없습니다 — 지울 수도 없습니다 (뭘로 찾나요?) · 라인 '999' 는 존재하지 않는 라인입니다 · '가동' 은 오타입니다. 상태별 집계에서 이것만 빠집니다

★ 이 데이터는 **한 달 뒤에 발견됩니다**. 그때는 이미 수만 건이 섞여 있습니다. 그리고 어느 게 진짜인지 아무도 모릅니다.

★ 같은 검사를 코드로 하면 vs 제약으로 하면

섹션 2

위 데이터를 막는 검사 코드를 자바스크립트로 써 봅니다.

```

const 라인목록 = ["A", "B", "C"];
const 상태목록 = ["가동", "정지", "점검", "고장"];

function 설비검사(설비, 이미있는번호, 이미있는이름) {
  if (설비.설비번호 === null || 설비.설비번호 === undefined) return "설비번호는 필수입니다";
  if (!Number.isInteger(설비.설비번호)) return "설비번호는 정수여야 합니다";
  if (이미있는번호.has(설비.설비번호)) return "이미 있는 설비번호입니다";

  if (typeof 설비.이름 !== "string" || 설비.이름.trim() === "") return "이름은 필수입니다";
  if (설비.이름.length > 20) return "이름이 너무 길니다";
  if (이미있는이름.has(설비.이름)) return "이미 있는 이름입니다";

  if (!라인목록.includes(설비.라인)) return "라인은 A, B, C 중 하나여야 합니다";
  if (!상태목록.includes(설비.상태)) return "상태 값이 올바르지 않습니다";
  if (설비.가격 !== null && 설비.가격 !== undefined && Number(설비.가격) <= 0)

```



```
return "가격은 0보다 커야 합니다";

return null;
}

console.log("검사 함수 줄 수: 약 15줄");
```

출력 검사 함수 줄 수: 약 15줄

★ 문제는 줄 수가 아닙니다. **이 함수를 부르는 걸 잇는 순간 뚫린다**는 것입니다.

· 관리자 화면에서 넣을 때 → 부름 · 엑셀 일괄 등록에서 → 깜빡함 · 배치 스크립트에서 → 몰랐음 · 개발자가 psql 로 직접 → 당연히 안 부름

그리고 **이미있는번호** 를 미리 조회해 오는 것도 **경쟁 조건**이 있습니다. 두 사람이 동시에 같은 번호를 넣으면 둘 다 “없네” 하고 통과합니다. 01단원 섹션 2 에서 본 그 문제입니다.

★★ 같은 규칙을 제약으로 쓰면 이렇습니다.

```
await db.exec(`
  CREATE TABLE 설비 (
    설비번호 INT      PRIMARY KEY,
    이름      TEXT    NOT NULL UNIQUE CHECK (char_length(이름) BETWEEN 1 AND 20),
    라인      TEXT    NOT NULL CHECK (라인 IN ('A','B','C')),
    상태      TEXT    NOT NULL DEFAULT '정지'
    CONSTRAINT 상태_확인 CHECK (상태 IN
('가동','정지','점검','고장')),
    도입일    DATE    NOT NULL DEFAULT CURRENT_DATE,
    가격      NUMERIC(12,2) CHECK (가격 > 0)
  );
`);

console.log("제약으로 쓴 줄 수: 8줄 (표 정의에 포함)");
```

출력 제약으로 쓴 줄 수: 8줄 (표 정의에 포함)

★★★ 그리고 이건 **누가 어떤 경로로 넣든** 통합니다. 관리자 화면이든, 배치든, psql 이든, 동시에 열 명이 들어와도 뚫리지 않습니다. **이미있는번호** 를 미리 조회할 필요도 없습니다. DB 가 원자적으로 처리합니다.

★ 그렇다고 자바스크립트 검사를 없애라는 말이 아닙니다. 화면에서 “이름을 입력하세요” 를 바로 띄우려면 앞단 검사가 필요합니다.

앞단 검사 = **친절**을 위한 것 (빠른 안내) 제약 = **정확**을 위한 것 (마지막 방어선)

앞단 검사는 빠뜨려도 데이터는 안전합니다. 제약을 빠뜨리면 안전하지 않습니다. ★ 그래서 **제약이 먼저**입니다.

```
const 넣기 = async (설명, sql, 값들) => {
  try {
    await db.query(sql, 값들);
    console.log(`${설명} - 들어감`);
  } catch (에러) {
    console.log(`${설명} - ${에러.code} ${에러.constraint ?? "(이름없음)"}`);
  }
};
```

```
await 넣기("정상", `INSERT INTO 설비 (설비번호,이름,라인) VALUES (1,'컨베이어
1호','A')`);
```

출력 정상 - 들어감

```
await 넣기("이름이 NULL", `INSERT INTO 설비 (설비번호,이름,라인) VALUES
(2,NULL,'A')`);
```

출력 이름이 NULL - 23502 (이름없음)

```
await 넣기("이름 중복", `INSERT INTO 설비 (설비번호,이름,라인) VALUES (2,'컨베이어
1호','A')`);
```

출력 이름 중복 - 23505 설비_이름_key

```
await 넣기("설비번호 중복", `INSERT INTO 설비 (설비번호,이름,라인) VALUES (1,'프레스
1호','A')`);
```

출력 설비번호 중복 - 23505 설비_pkey

```
await 넣기("라인이 999", `INSERT INTO 설비 (설비번호,이름,라인) VALUES (3,'프레스
1호','999')`);
```

출력 라인이 999 - 23514 설비_라인_check

```
await 넣기("상태 오타 '가동'", `INSERT INTO 설비 (설비번호,이름,라인,상태) VALUES
(4,'용접로봇 1호','B','가동')`);
```

출력 상태 오타 '가동' - 23514 상태_확인

```
await 넣기("이름이 빈 문자열", `INSERT INTO 설비 (설비번호,이름,라인) VALUES
(5,'','C')`);
```

출력 이름이 빈 문자열 - 23514 설비_이름_check

```
await 넣기("가격이 음수", `INSERT INTO 설비 (설비번호,이름,라인,가격) VALUES
(6,'절단기 1호','C',-100)`);
```

출력 가격이 음수 - 23514 설비_가격_check

★ 섹션 1의 쓰레기가 **전부 막혔습니다**. 자바스크립트 코드는 한 줄도 안 썼습니다.

제약 다섯 가지

섹션 4

① NOT NULL — 비워 둘 수 없다

```
이름 TEXT NOT NULL
```

★ 가장 자주 쓰고, 가장 자주 빠뜨립니다.

의심스러우면 ****일단 NOT NULL 을 걸고****, 정말 모를 수 있는 값만 푸세요.
나중에 푸는 것은 쉽고, 나중에 거는 것은 어렵습니다. (섹션 7에서 봅니다)

② UNIQUE — 같은 값이 두 번 오면 안 된다

```
이름 TEXT UNIQUE
UNIQUE (라인, 이름)      ← 두 칸을 묶어서도 됩니다
```

★★ UNIQUE는 **NULL을 여러 개 허용합니다**. 개념05에서 실측합니다

③ CHECK — 이 조건이 참이어야 한다

```
CHECK (라인 IN ('A','B','C'))
CHECK (가격 > 0)
CHECK (종료일 >= 시작일)                      ← 칸끼리 비교도 됩니다
CHECK (char_length(이름) BETWEEN 1 AND 20)
```

④ DEFAULT — 안 적으면 이 값을 넣어라

```
상태 TEXT NOT NULL DEFAULT '정지'
등록시각 TIMESTAMPTZ NOT NULL DEFAULT now()
```

⑤ PRIMARY KEY — 이 줄을 가리키는 번호

```
설비번호 INT PRIMARY KEY      ← NOT NULL + UNIQUE 를 한꺼번에
```

개념04에서 따로 다룹니다. 여기서는 “NOT NULL + UNIQUE”로만 알아 두세요.

★ DEFAULT 를 확인해 봅시다. 상태와 도입일을 안 적었는데 채워졌습니다.

```
const 기본값들 = (await db.query(`SELECT 상태, 도입일::text AS 도입일 FROM 설비 WHERE
설비번호 = 1`)).rows[0];
console.log("안 적은 상태:", 기본값들.상태);
```

출력 안 적은 상태: 정지

```
console.log("안 적은 도입일:", 기본값들.도입일);
```

출력 안 적은 도입일: 2026-08-26
값이 다를 수 있음

```
console.log("도입일이 오늘인가:", 기본값들.도입일 ===
new Date().toLocaleDateString("en-CA"));
```

출력 도입일이 오늘인가: true

★★ 함정: NULL 을 직접 적으면 DEFAULT 가 안 먹습니다.

```
await db.exec(`CREATE TABLE 기본값시험 (a TEXT DEFAULT '정지')`);
await db.exec(`INSERT INTO 기본값시험 DEFAULT VALUES`);
await db.exec(`INSERT INTO 기본값시험 (a) VALUES (NULL)`);

const 둘 = (await db.query(`SELECT a FROM 기본값시험 ORDER BY a NULLS LAST`)).rows;
console.log("안 적었을 때:", JSON.stringify(둘[0].a), "· NULL 을 적었을 때:",
JSON.stringify(둘[1].a));
```

출력 안 적었을 때: "정지" · NULL 을 적었을 때: null

★ 자바스크립트에서 { 상태: null } 을 그대로 넘기면 DEFAULT 를 지나칩니다. undefined 를 넘겨서 그 칸을 아예 빼거나, NOT NULL 을 같이 걸어 두세요. ★ 그래서 DEFAULT 는 거의 항상 **NOT NULL** 과 짝으로 씁니다.

CHECK 와 NULL — ★ 여기서 많이 틀립니다

섹션 5

```
await db.exec(`CREATE TABLE 널시험 (가격 NUMERIC CHECK (가격 > 0))`);

await 넣기("가격 100", `INSERT INTO 널시험 VALUES (100)`);
```

출력 가격 100 – 들어감

```
await 넣기("가격 -1", `INSERT INTO 널시험 VALUES (-1)`);
```

출력 가격 -1 – 23514 널시험_가격_check

```
await 넣기("가격 NULL", `INSERT INTO 널시험 VALUES (NULL)`);
```

출력 가격 NULL – 들어감

★★★ NULL 이 통과했습니다. NULL > 0 이 거짓이 아니라 **모름(UNKNOWN)** 이기 때문입니다. CHECK 는 “거짓일 때만” 막습니다. 모름은 통과시킵니다.

그래서 **CHECK** 는 **NULL** 을 **막지 않습니다**. 비워 두면 안 되는 칸이면 NOT NULL 을 따로 걸어야 합니다.

이 3값 논리는 개념05 에서 제대로 팝니다. 지금은 이것만 기억하세요.

****CHECK 와 NOT NULL 은 서로 다른 일을 합니다.****

★ 이름 붙인 제약 — 에러를 사용자에게 보여 주려면

섹션 6

위에서 이름 없이 만든 제약은 Postgres 가 알아서 이름을 붙였습니다.

```
const 제약들 = await db.query(`
  SELECT conname, contype, pg_get_constraintdef(oid) AS 정의
  FROM pg_constraint WHERE conrelid = '설비'::regclass AND contype IN ('c','u','p')
  ORDER BY contype, conname
`);

for (const 제약 of 제약들.rows) {
  console.log(`[${제약.contype}] ${제약.conname}`);
}
```

출력 [c] 상태_확인
 [c] 설비_가격_check
 [c] 설비_라인_check
 [c] 설비_이름_check
 [p] 설비_pkey
 [u] 설비_이름_key

★ contype: c = CHECK, p = PRIMARY KEY, u = UNIQUE, f = 외래키(04단원)

★★ 상태_확인 만 우리가 붙인 이름입니다. 나머지는 자동 생성입니다. 자동 이름 규칙: 표_칸_check / 표_칸_key / 표_pkey

자동 이름도 쓸 수는 있습니다. 그런데 **칸 이름을 바꾸면 제약 이름은 안 바뀝니다**. 그러면 코드의 if (e.constraint === “설비_라인_check”) 가 조용히 헛됩니다.

★ 사용자에게 메시지를 보여 줄 제약은 **이름을 직접 붙이세요**.

이제 에러를 사용자 말로 바꿔 봅니다. 이게 제약을 쓰는 진짜 이유입니다.

```
const 메시지 = {
  설비_pkey: "이미 등록된 설비번호입니다.",
  설비_이름_key: "같은 이름의 설비가 이미 있습니다.",
  설비_라인_check: "라인은 A, B, C 중에서 고르세요.",
  상태_확인: "상태는 가동 · 정지 · 점검 · 고장 중 하나여야 합니다.",
  설비_이름_check: "이름은 1자 이상 20자 이하여야 합니다.",
  설비_가격_check: "가격은 0보다 커야 합니다.",
};

function 사람말로(에러) {
  if (에러.code === "23502") return `${에러.column} 은(는) 반드시 입력해야 합니다.`;
  if (메시지[에러.constraint]) return 메시지[에러.constraint];
  if (에러.code === "22P02") return "숫자를 넣어야 할 칸에 글자가 들어왔습니다.";
  return "저장하지 못했습니다. 입력을 확인해 주세요.";
}

const 보여주기 = async (sql) => {
  try {
    await db.query(sql);
    console.log("저장했습니다");
  } catch (에러) {
    console.log(사람말로(에러));
  }
};

await 보여주기(`INSERT INTO 설비 (설비번호,이름,라인) VALUES (7,NULL,'A')`);
```

출력 이름 은(는) 반드시 입력해야 합니다.

```
await 보여주기(`INSERT INTO 설비 (설비번호,이름,라인) VALUES (7,'컨베이어
1호','A')`);
```

출력 같은 이름의 설비가 이미 있습니다.

```
await 보여주기(`INSERT INTO 설비 (설비번호,이름,라인,상태) VALUES (7,'세척기
1호','A','가동')`);
```

출력 상태는 가동 · 정지 · 점검 · 고장 중 하나여야 합니다.

```
await 보여주기(`INSERT INTO 설비 (설비번호,이름,라인) VALUES (7,'세척기 1호','A')`);
```

출력 저장했습니다

★★ 이게 “코드가 줄어든다” 의 진짜 의미입니다.

검사 코드가 사라진 게 아니라, **한 곳으로 모였습니다**. 표 정의에 규칙이 있고, 에러 코드를 사람 말로 바꾸는 표가 하나 있습니다. 규칙이 바뀌면 그 두 곳만 고칩니다.

★ 에러 객체에서 쓸 수 있는 것

```
e.code      → SQLSTATE (23502 / 23505 / 23514 / 22P02 ...)  
e.constraint → 어긴 제약의 이름 (NOT NULL 은 없습니다)  
e.column    → NOT NULL 을 어긴 칸 이름  
e.table     → 표 이름  
e.detail    → "Key (이름)=(컨베이어 1호) already exists." 같은 상세
```

★★ e.detail 과 e.message 는 **사용자에게 그대로 보여 주면 안 됩니다**.

값이 그대로 들어 있습니다. 표 구조와 데이터가 새어 나갑니다.
로그에만 남기고, 화면에는 위처럼 우리 말로 바꿔서 보여 주세요.

★ 나중에 제약을 추가하기

섹션 7

이미 만들어진 표에 제약을 거는 것도 됩니다. **데이터가 깨끗할 때만요**.

섹션 1 의 느슨한설비 표에 걸어 봅니다. 쓰레기가 들어 있습니다.

```
const 걸기 = async (설명, sql) => {  
  try {  
    await db.exec(sql);  
    console.log(`${설명} - 걸렸습니다`);  
  } catch (에러) {  
    console.log(`${설명} - ${에러.code} 실패`);  
  }  
};  
  
await 걸기("NOT NULL", `ALTER TABLE 느슨한설비 ALTER COLUMN 설비번호 SET NOT NULL`);
```

출력 NOT NULL - 23502 실패

```
await 걸기("CHECK 라인", `ALTER TABLE 느슨한설비 ADD CONSTRAINT 라인_확인 CHECK (라인  
IN ('A','B','C'))`);
```

출력 CHECK 라인 - 23514 실패

```
await 걸기("PRIMARY KEY", `ALTER TABLE 느슨한설비 ADD PRIMARY KEY (설비번호)`);
```

출력 PRIMARY KEY - 23505 실패

★★★ 셋 다 실패했습니다. 이미 어긴 데이터가 있으면 제약을 걸 수 없습니다.

이게 이 단원을 처음에 잘하라고 하는 이유입니다. 운영 중인 표에 제약을 나중에 걸려면 이 순서를 밟아야 합니다.

- ① 어긴 줄이 몇 개인지 센다
- ② 그 줄들을 어떻게 고칠지 **업무 담당자와 정한다** ← 여기가 제일 오래 걸립니다
- ③ 고친다
- ④ 제약을 건다
- ⑤ 그 사이에 새로 들어온 어긴 줄이 없는지 다시 본다

②가 몇 주씩 걸립니다. “이 999 라인은 뭐예요?” 를 물어볼 사람이 퇴사했으면 더요.

```
const 어긴수 = (await db.query(`
  SELECT count(*)::int AS 개수 FROM 느슨한설비 WHERE 라인 NOT IN ('A','B','C') OR
  라인 IS NULL
`)).rows[0].개수;
console.log("라인 규칙을 어긴 줄:", 어긴수, "건");
```

출력 라인 규칙을 어긴 줄: 2 건

```
await db.exec(`DELETE FROM 느슨한설비 WHERE 라인 NOT IN ('A','B','C') OR 라인 IS
NULL`);
await 걸기("정리한 뒤 CHECK 라인", `ALTER TABLE 느슨한설비 ADD CONSTRAINT 라인_확인
CHECK (라인 IN ('A','B','C'))`);
```

출력 정리한 뒤 CHECK 라인 - 걸렸습니다

★ 제약을 떼는 것은 언제든 됩니다.

```
await db.exec(`ALTER TABLE 느슨한설비 DROP CONSTRAINT 라인_확인`);
console.log("제약을 떼는 것은 항상 됩니다");
```

출력 제약을 떼는 것은 항상 됩니다

★ NOT VALID — 오늘부터만 막고 싶을 때 —————

옛날 데이터는 손댈 수 없고, 새로 들어오는 것만 막고 싶을 때가 있습니다.

```
await db.exec(`CREATE TABLE 옛날데이터 (값 INT)`);
await db.exec(`INSERT INTO 옛날데이터 VALUES (-1), (5)`);

await 걸기("NOT VALID 로 걸기", `ALTER TABLE 옛날데이터 ADD CONSTRAINT 값_양수 CHECK
(값 > 0) NOT VALID`);
```

출력 NOT VALID 로 걸기 - 걸렸습니다


```
await 넣기("그 뒤에 -9 넣기", `INSERT INTO 옛날데이터 VALUES (-9)`);
```

출력 그 뒤에 -9 넣기 - 23514 값_양수

```
const 남은것 = (await db.query(`SELECT 값 FROM 옛날데이터 ORDER BY 값`)).rows.map((줄) => 줄.값);
console.log("옛날 -1 은 그대로 있나:", JSON.stringify(남은것));
```

출력 옛날 -1 은 그대로 있나: [-1,5]

```
await 걸기("나중에 VALIDATE", `ALTER TABLE 옛날데이터 VALIDATE CONSTRAINT 값_양수`);
```

출력 나중에 VALIDATE - 23514 실패

★ NOT VALID 는 “새 줄만 검사” 입니다. 기존 줄은 안 봅니다. 나중에 데이터를 다 고친 뒤 VALIDATE CONSTRAINT 로 전체 검사를 합니다.

★★ 큰 표에서는 이게 실용적으로도 중요합니다.

그냥 ADD CONSTRAINT 를 하면 표 전체를 훑는 동안 **표가 잠깁니다.**
NOT VALID 로 먼저 걸고, 한가한 시간에 VALIDATE 하는 게 정석입니다.

MySQL 은 여기가 다릅니다

· CHECK 는 MySQL 8.0.16 부터 진짜로 동작합니다. 그 전 버전은 무시했습니다 · NOT VALID 가 없습니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 제약 다섯 가지와 에러 코드

제약 막는 것 에러 코드 e.constraint

NOT NULL 비워 두는 것 23502 없음 (e.column 을 보세요) UNIQUE 같은 값이 두 번 23505 표_칸_key PRIMARY KEY 비움 + 중복 23502/23505 표_pkey

CHECK	조건이 거짓	23514	표_칸_check
DEFAULT	(막지 않음. 채워 줌)	-	-
타입	그 타입으로 못 읽는 값	22P02	-
범위	타입 범위 초과	22003	-
길이	VARCHAR(n) 초과	22001	-

★ 세 줄 요약 ① 제약은 누가 어떤 경로로 넣든 통합니다. 앞단 검사는 빠뜨릴 수 있습니다 ② CHECK 는 NULL 을 막지 않습니다. NOT NULL 을 따로 거세요 ③ 사용자에게 보여 줄 제약은 이름을 붙이세요. 에러 처리가 쉬워집니다

★★ 그리고 가장 중요한 것 — **나중에 걸려면 데이터를 먼저 고쳐야 합니다.** 그 작업은 대개 코드가 아니라 사람 문제라서 몇 주씩 걸립니다.

직접 해 볼 것

▫ 직접 해보기 1

점검기록 표를 제약과 함께 만들어 보세요. · 점검번호는 필수이고 중복 불가 · 설비번호는 필수 · 점검일은 필수이고 기본값은 오늘 · 결과는 '정상' / '주의' / '불량' 중 하나 · 비고는 비워도 됨

▫ 직접 해보기 2

CHECK (종료일 >= 시작일) 처럼 **두 칸을 비교하는 CHECK** 를 걸어 보세요. 시작일만 넣고 종료일을 NULL 로 두면 통과하나요? (힌트: 섹션 5)

▫ 직접 해보기 3

섹션 2 의 **설비검사** 함수를 부르는 걸 잊었다고 가정하고 쓰레기 데이터를 **설비** 표에 넣어 보세요. 몇 개나 들어가나요?

▫ 직접 해보기 4

UNIQUE (라인, 이름) 처럼 **두 칸을 묶은 UNIQUE** 를 걸어 보세요. A라인의 '1호기' 와 B라인의 '1호기' 는 둘 다 들어가나요?

▫ 직접 해보기 5

섹션 6 의 **메시지** 표에 없는 제약을 일부러 어겨 보세요. 어떤 문장이 나오나요? 사용자가 이해할 수 있나요?

▫ 직접 해보기 6

상태_확인 에 '폐기' 를 추가하려면 어떻게 해야 하나요? (힌트: CHECK 는 고칠 수 없습니다. 떼고 다시 걸어야 합니다)

▫ 직접 해보기 7

가격 NUMERIC(12,2) CHECK (가격 > 0) 를 CHECK (가격 >= 0) 로 바꾸면 무엇이 달라지나요? "가격이 0원인 설비" 는 말이 되나요? 업무에 따라 다릅니다.

자주 하는 실수

이런 실수를 합니다

“검사는 코드에서 하니까 DB 는 느슨하게”

경로가 하나일 때만 맞습니다. 배치·엑셀 업로드·psql 이 생기는 순간 뚫립니다. 그리고 동시에 두 명이 들어오면 “중복 확인 후 저장” 은 원자적이지 않아 뚫립니다.

이런 실수를 합니다

CHECK 로 NULL 을 막을 수 있다고 생각

CHECK (가격 > 0) 는 NULL 을 통과시킵니다. NOT NULL 을 따로 거세요.

이런 실수를 합니다

DEFAULT 를 걸어 놓고 코드에서 null 을 넘김

{ 상태: null } 을 넘기면 DEFAULT 가 안 먹고 NULL 이 들어갑니다. 그 칸을 아예 빼거나, NOT NULL 을 같이 거세요.

이런 실수를 합니다

제약 이름을 안 붙이고 자동 이름에 의존

칸 이름을 바꾸면 제약 이름은 안 바뀝니다. 에러 처리 코드가 조용히 헛됩니다.

이런 실수를 합니다

e.message 를 사용자에게 그대로 보여 줌

duplicate key value violates unique constraint “설비_이름_key” 는 사용자에게 아무 의미가 없고, 표 구조를 노출합니다. 우리 말로 바꾸세요.

이런 실수를 합니다

나중에 걸면 되지 하고 미룸

데이터가 쌓인 뒤에는 못 겁니다. 고치는 데 몇 주가 걸립니다. ★ 의심스러우면 **처음에 조이고**, 정말 안 되겠으면 그때 푸세요.

```
await db.close();
```

기본키를 무엇으로

RUN node 개념04_기본키를_무엇으로.js

개념03 에서 PRIMARY KEY 를 “NOT NULL + UNIQUE” 라고만 하고 넘어갔습니다. 이제 제대로 봅니다.

기본키를 정하는 일은 표 설계에서 **가장 되돌리기 어려운 결정**입니다. 칸은 나중에 추가할 수 있습니다. 타입도 (아프지만) 바꿀 수 있습니다. 그런데 기본키는 다른 표들이 그 값을 붙들고 있어서 바꾸기가 아주 어렵습니다.

이 파일에서 실제로 재 볼 것들입니다.

① SERIAL 에 값을 직접 넣으면 **나중에 충돌합니다** — 재현합니다 ② 지운 번호는 재활용되지 않습니다 — 재 봅니다 ③ 롤백해도 번호는 안 돌아옵니다 — 재 봅니다 ④ 자연키는 바뀝니다 — 바뀔 때 무슨 일이 나는지 봅니다

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

PRIMARY KEY 가 하는 일

섹션 1

기본키는 **이 줄 하나를 정확히 가리키는 값**입니다. 세 가지를 한꺼번에 합니다.

```
await db.exec(`
  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름 TEXT NOT NULL
  );
`);

await db.exec(`INSERT INTO 설비 VALUES (1, '컨베이어 1호')`);

const 시도 = async (설명, sql) => {
  try {
    await db.query(sql);
    console.log(`${설명} - 들어감`);
  } catch (에러) {
    console.log(`${설명} - ${에러.code}`);
  }
}
```

```
};
```

```
await 시도("NULL 을 넣으면", `INSERT INTO 설비 VALUES (NULL, '프레스 1호')`);
```

```
출력      NULL 을 넣으면 - 23502
```

```
await 시도("같은 번호를 또 넣으면", `INSERT INTO 설비 VALUES (1, '프레스 1호')`);
```

```
출력      같은 번호를 또 넣으면 - 23505
```

그리고 세 번째, 눈에 안 보이는 일을 합니다.

```
const 색인 = await db.query(`SELECT indexname FROM pg_indexes WHERE tablename =
'설비'`);
console.log("자동으로 생긴 색인:", 색인.rows.map((줄) => 줄.indexname).join(", "));
```

```
출력      자동으로 생긴 색인: 설비_pkey
```

★ PRIMARY KEY = NOT NULL + UNIQUE + 색인(index)

색인 덕분에 WHERE 설비번호 = 1 이 아주 빠릅니다. 100만 줄이 있어도 전부 훑지 않습니다. 06단원에서 재 보입니다.

★★ 표 하나에 PRIMARY KEY 는 **하나뿐**입니다. “이것도 유일한데” 싶은 칸은 UNIQUE 를 걸면 됩니다. UNIQUE 는 여러 개 됩니다.

★ 기본키가 없는 표도 만들 수는 있습니다. 그런데 만들지 마세요. 똑같은 줄이 두 개 있으면 하나만 지울 방법이 없습니다.

번호를 자동으로 매기는 두 가지 방법

섹션 2

설비번호를 사람이 일일이 정하면 곧 충돌합니다. 자동으로 매깁니다. Postgres 에는 방법이 둘 있습니다.

```
await db.exec(`CREATE TABLE 설비_시리얼 (설비번호 SERIAL PRIMARY KEY, 이름 TEXT NOT
NULL)`);
await db.exec(`CREATE TABLE 설비_아이덴티티 (설비번호 INT GENERATED ALWAYS AS
IDENTITY PRIMARY KEY, 이름 TEXT NOT NULL)`);
```

겉보기에는 똑같이 동작합니다. 속을 보면 다릅니다.

```
const 시리얼정의 = (await db.query(`
  SELECT column_default, is_identity FROM information_schema.columns
  WHERE table_name = '설비_시리얼' AND column_name = '설비번호'
`)).rows[0];
```

```
const 아이덴티티정의 = (await db.query(`
  SELECT column_default, is_identity, identity_generation FROM
  information_schema.columns
  WHERE table_name = '설비_아이덴티티' AND column_name = '설비번호'
`)).rows[0];
```

```
console.log("SERIAL 의 기본값:", 시리얼정의.column_default);
```

```
출력      SERIAL 의 기본값: nextval('"설비_시리얼_설비번호_seq"::regclass)
```

```
console.log("SERIAL 이 IDENTITY 인가:", 시리얼정의.is_identity);
```

```
출력      SERIAL 이 IDENTITY 인가: NO
```

```
console.log("IDENTITY 의 기본값:", 아이덴티티정의.column_default);
```

```
출력      IDENTITY 의 기본값: null
```

```
console.log("IDENTITY 종류:", 아이덴티티정의.identity_generation);
```

```
출력      IDENTITY 종류: ALWAYS
```

★★ SERIAL 은 사실 타입이 아닙니다. 줄임말입니다.

```
설비번호 SERIAL
```

는 Postgres 가 이렇게 풀어 씁니다.

- ① 시퀀스(sequence)라는 별도 물건을 만든다 - 설비_시리얼_설비번호_seq
- ② 칸을 INT NOT NULL 로 만든다
- ③ 그 칸의 DEFAULT 를 nextval(시퀀스) 로 건다

시퀀스는 “다음 번호를 뽑아 주는 기계” 입니다. 표와 별개로 존재합니다.

```
const 시퀀스들 = await db.query(`SELECT sequence_name FROM
  information_schema.sequences ORDER BY sequence_name`);
console.log("만들어진 시퀀스:", 시퀀스들.rows.map((줄) => 줄.sequence_name).join(",
  "));
```

```
출력      만들어진 시퀀스: 설비_시리얼_설비번호_seq
```

★ IDENTITY 도 속으로는 시퀀스를 씁니다. 다만 **칸에 붙어 있습니다**. 그래서 표를 지우면 같이 지워지고, 아무나 값을 못 넣게 막을 수 있습니다.

★★ IDENTITY 는 SQL 표준입니다. Postgres 10 부터 됩니다.

새로 만드는 표는 IDENTITY 를 쓰세요. SERIAL 은 오래된 코드에서 만납니다.

왜 그런지를 지금부터 실제로 봅니다.

★★★ SERIAL 에 값을 직접 넣으면 나중에 터집니다

섹션 3

데이터를 옮기거나, 테스트 데이터를 만들 때 번호를 직접 적는 일이 흔합니다.

```
await db.exec(`INSERT INTO 설비_시리얼 (이름) VALUES ('컨베이어 1호')`); //
자동: 1번
await db.exec(`INSERT INTO 설비_시리얼 VALUES (2, '프레스 1호')`); //
직접: 2번
await db.exec(`INSERT INTO 설비_시리얼 VALUES (3, '용접로봇 1호')`); //
직접: 3번 const 지금까지 = (await db.query(`SELECT 설비번호, 이름 FROM 설비_시리얼
ORDER BY 설비번호`)).rows;
console.log("여기까지는 멀쩡합니다:", 지금까지.map((줄) => 줄.설비번호).join(", "));
```

출력 여기까지는 멀쩡합니다: 1, 2, 3

그런데 시퀀스는 그 사이에 아무것도 모릅니다. 물어봅니다.

```
const 시퀀스값 = (await db.query(`SELECT last_value FROM 설비_시리얼_설비번호
_seq`)).rows[0];
console.log("시퀀스가 마지막으로 준 번호:", 시퀀스값.last_value);
```

출력 시퀀스가 마지막으로 준 번호: 1

★★★ 표에는 3번까지 있는데 시퀀스는 아직 1입니다. 직접 넣은 2, 3 을 시퀀스는 본 적이 없습니다.

이 상태에서 다음 설비를 자동으로 넣으면 어떻게 될까요.

```
await 시도("자동으로 다음 설비 넣기", `INSERT INTO 설비_시리얼 (이름) VALUES ('절단기
1호')`);
```

출력 자동으로 다음 설비 넣기 - 23505

★★★ 중복 키 에러입니다. 시퀀스가 2번을 줬는데 이미 2번이 있습니다.

이 사고의 무서운 점은 **시간이 지나서 터진다는** 것입니다.

- 데이터를 옮긴 날에는 멀쩡합니다
- 며칠 뒤 사용자가 새 설비를 등록할 때 처음 터집니다
- 그때는 아무도 이관 작업을 떠올리지 못합니다

그리고 한 번만 터지는 게 아닙니다. 3번도 걸립니다. 4번이 되어야 지나갑니다.

★ 고치는 법: 시퀀스를 표의 최대값에 맞추습니다.

```
const 맞춤 = (await db.query(`
  SELECT setval('설비_시리얼_설비번호_seq', (SELECT max(설비번호) FROM 설비_시리얼))
  AS 맞춤값
`)).rows[0];
console.log("setval 로 맞춤 값:", 맞춤.맞춤값);
```

출력 setval 로 맞춤 값: 3

```
await 시도("다시 자동으로 넣기", `INSERT INTO 설비_시리얼 (이름) VALUES ('절단기
1호')`);
```

출력 다시 자동으로 넣기 - 들어감

```
const 고친뒤 = (await db.query(`SELECT 설비번호 FROM 설비_시리얼 ORDER BY 설비번호
`)).rows;
console.log("고친 뒤:", 고친뒤.map((줄) => 줄.설비번호).join(", "));
```

출력 고친 뒤: 1, 2, 3, 4

★★ 데이터를 옮긴 뒤에는 반드시 **setval** 을 하세요. 이걸 잊어서 나는 장애가 정말 흔합니다.

IDENTITY 는 애초에 못 넣게 막습니다

섹션 4

```
await 시도("IDENTITY 칸에 99 를 직접", `INSERT INTO 설비_아이덴티티 VALUES (99,
'몰래')`);
```

출력 IDENTITY 칸에 99 를 직접 - 428C9

★ 428C9 = "IDENTITY 칸에 직접 값을 넣을 수 없다". 섹션 3 의 사고가 **애초에 일어나지 않습니다**. 이게 IDENTITY 를 쓰는 이유입니다.

정말 필요하다면 명시적으로 뚫을 수 있습니다.

```
await 시도("OVERRIDING SYSTEM VALUE 를 붙이면", `INSERT INTO 설비_아이덴티티
OVERRIDING SYSTEM VALUE VALUES (99, '이관용')`);
```

출력 OVERRIDING SYSTEM VALUE 를 붙이면 - 들어감

★ 길고 눈에 띄는 문장을 일부러 적어야 뚫립니다. 실수로는 안 뚫립니다. (이걸 쓴 뒤에도 시퀀스는 안 따라옵니다. setval 은 여전히 해야 합니다)

★ 두 종류가 있습니다.

GENERATED ALWAYS AS IDENTITY	직접 넣기 금지 (OVERRIDING 으로만) ← 보통 이걸 씁니다
GENERATED BY DEFAULT AS IDENTITY	직접 넣어도 됨 (SERIAL 과 비슷)

```
await db.exec(`CREATE TABLE 설비_기본 (설비번호 INT GENERATED BY DEFAULT AS IDENTITY
PRIMARY KEY, 이름 TEXT)`);
await 시도("BY DEFAULT 에 직접 넣기", `INSERT INTO 설비_기본 VALUES (5, '직접')`);
```

출력 BY DEFAULT 에 직접 넣기 - 들어감

★ BIGINT 로 하려면 이렇게 씁니다.

설비번호 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

(SERIAL 의 BIGINT 판은 BIGSERIAL 입니다)

★★ 개념02 에서 본 대로 키는 **BIGINT** 를 권합니다. INT 는 21억에서 잡니다.

방금 넣은 번호를 알아내기

섹션 5

자동으로 매겨진 번호는 넣기 전에는 모릅니다. RETURNING 으로 받습니다.

```
const 받은것 = await db.query(`INSERT INTO 설비_아이덴티티 (이름) VALUES ('세척기
1호') RETURNING 설비번호, 이름`);
console.log("RETURNING 으로 받은 것:", JSON.stringify(받은것.rows[0]));
```

출력 RETURNING 으로 받은 것: {"설비번호":1,"이름":"세척기 1호"}

★ RETURNING 은 INSERT · UPDATE · DELETE 에 다 붙습니다. Postgres 의 좋은 점입니다.

RETURNING * 으로 줄 전체를 받을 수도 있습니다.

MySQL 은 여기가 다릅니다

· RETURNING 이 없습니다. 넣은 뒤 LAST_INSERT_ID() 를 따로 부릅니다 · SERIAL / IDENTITY 대신 AUTO_INCREMENT 입니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

★ 지운 번호는 재활용되나

섹션 6

많이 궁금해하는 부분입니다. 실제로 재 봅니다.

```

await db.exec(`CREATE TABLE 재활용시험 (번호 INT GENERATED ALWAYS AS IDENTITY, 이름 TEXT)`);
await db.exec(`INSERT INTO 재활용시험 (이름) VALUES ('가'), ('나'), ('다')`);
await db.exec(`DELETE FROM 재활용시험 WHERE 번호 = 3`);
await db.exec(`INSERT INTO 재활용시험 (이름) VALUES ('라')`);

const 재활용 = (await db.query(`SELECT 번호, 이름 FROM 재활용시험 ORDER BY 번호`)).rows;
console.log("3번을 지우고 새로 넣으면:", 재활용.map((줄) => `${줄.번호}:${줄.이름}`).join(" "));

```

출력 3번을 지우고 새로 넣으면: 1:가 2:나 4:라

★ 3번은 영영 비어 있습니다. 4번이 나왔습니다. 시퀀스는 뒤로 돌아가지 않습니다.

★★ 롤백해도 마찬가지로입니다. 실패한 트랜잭션이 쓴 번호도 안 돌아옵니다.

```

await db.exec(`CREATE TABLE 롤백시험 (번호 INT GENERATED ALWAYS AS IDENTITY, 이름 TEXT)`);
await db.exec(`INSERT INTO 롤백시험 (이름) VALUES ('가')`);

try {
  await db.transaction(async (tx) => {
    await tx.query(`INSERT INTO 롤백시험 (이름) VALUES ('실패할것')`);
    throw new Error("일부러 실패");
  });
} catch {

```

롤백됐습니다

```

}

await db.exec(`INSERT INTO 롤백시험 (이름) VALUES ('나')`);

const 롤백뒤 = (await db.query(`SELECT 번호, 이름 FROM 롤백시험 ORDER BY 번호`)).rows;
console.log("롤백 뒤 다음 번호는:", 롤백뒤.map((줄) => `${줄.번호}:${줄.이름}`).join(" "));

```

출력 롤백 뒤 다음 번호는: 1:가 3:나

★★★ 2번이 사라졌습니다. 롤백된 INSERT 도 시퀀스는 이미 써 버렸습니다.

왜 이렇게 만들었을까요? 되돌리면 동시에 들어온 사람과 부딪히기 때문입니다. 시퀀스는 트랜잭션 밖에서 돕니다. 그래야 여러 명이 동시에 넣어도 안 겹칩니다.

★ 그래서 이렇게 기억하세요.

자동 번호는 ****유일함만 보장합니다.**** 연속을 보장하지 않습니다.

★★ 그러니 자동 번호를 "몇 건인가" 로 쓰면 안 됩니다.

"송장번호가 1000번이니 1000건 팔았구나" 는 틀립니다.
세는 것은 `count(\*)` 로 하세요.

★ SQLite 를 쓰던 사람은 여기서 놀랍니다. SQLite 의 AUTOINCREMENT 없는 rowid 는 **빈 번호를 재활용합니다.** Postgres 는 안 합니다.

★ TRUNCATE 도 시퀀스를 안 되돌립니다. 되돌리려면 명시해야 합니다.

```
await db.exec(`TRUNCATE 재활용시험`);
await db.exec(`INSERT INTO 재활용시험 (이름) VALUES ('새로')`);
const 트렁 = (await db.query(`SELECT 번호 FROM 재활용시험`)).rows[0];
console.log("TRUNCATE 뒤 번호:", 트렁.번호);
```

출력 TRUNCATE 뒤 번호: 5

```
await db.exec(`TRUNCATE 재활용시험 RESTART IDENTITY`);
await db.exec(`INSERT INTO 재활용시험 (이름) VALUES ('진짜새로')`);
const 리스트트 = (await db.query(`SELECT 번호 FROM 재활용시험`)).rows[0];
console.log("RESTART IDENTITY 뒤 번호:", 리스트트.번호);
```

출력 RESTART IDENTITY 뒤 번호: 1

UUID 를 키로

섹션 7

```
await db.exec(`
  CREATE TABLE 점검 (
    점검번호 UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    설비번호 INT NOT NULL,
    내용 TEXT NOT NULL
  );
`);

const 새점검 = await db.query(`INSERT INTO 점검 (설비번호, 내용) VALUES (1, '벨트
장력 확인') RETURNING 점검번호`);
console.log("UUID 키 길이:", 새점검.rows[0].점검번호.length, "· 타입:", typeof
새점검.rows[0].점검번호);
```

출력 UUID 키 길이: 36 · 타입: string

```
const 크기비교 = (await db.query(`
  SELECT pg_column_size(gen_random_uuid()) AS uuid바이트, pg_column_size(1::bigint)
  AS bigint바이트
`)).rows[0];
console.log(`UUID ${크기비교.uuid바이트}바이트 vs BIGINT ${크기비교.bigint바이트}
바이트`);
```

출력 UUID 16바이트 vs BIGINT 8바이트

★ UUID 를 키로 쓰면 좋은 점 · DB 에 묻지 않고 애플리케이션에서 미리 만들 수 있습니다 — 이게 가장 큼니다

여러 서버가, 심지어 브라우저가 만들어도 안 겹칩니다

· 주소창에 노출돼도 옆 번호를 추측할 수 없습니다 (/설비/1 → /설비/2 를 못 함) · 여러 DB 를 합칠 때 번호가 안 부딪힙니다

★ 나쁜 점 · 2배 큼니다 (16 vs 8바이트). 색인도 2배가 됩니다 · 순서가 없습니다. 무작위라 색인의 여기저기에 흩어져 들어갑니다

줄이 많아지면 이게 눈에 띄게 느려집니다

· 사람이 못 읽습니다. “설비 3번” 대신 “설비 a3f9-...” 라고 말해야 합니다 · 로그와 화면이 지저분해집니다

★★ 언제 쓰나 · 여러 곳에서 동시에 만드는 데이터 (오프라인 앱, 여러 서버, 브라우저) · 번호를 밖에 노출해야 하는데 추측당하면 곤란한 것 그 외에는 **BIGINT IDENTITY** 가 무난합니다.

★ 절충안: 안에서는 BIGINT 를 쓰고, 밖에 보여 줄 때만 UUID 칸을 따로 두는 방법도 있습니다.

★ UUID 를 TEXT 로 저장하지 마세요. 36바이트가 됩니다. UUID 타입은 16바이트입니다.

자연키 vs 인조키

섹션 8

자연키(natural key) — 업무에 원래 있는 값. 사번, 설비번호, 사업자등록번호, 주민번호 **인조키**

(surrogate key) — 업무와 상관없이 만든 번호. IDENTITY, UUID

자연키가 좋아 보입니다. 이미 있는 값이니 표가 깔끔합니다. 그런데 현장에서는 이런 일이 벌어집니다.

```
await db.exec(`
  CREATE TABLE 작업자_자연키 (
    사번 TEXT PRIMARY KEY,
    이름 TEXT NOT NULL
  );
  CREATE TABLE 배정_자연키 (
    사번 TEXT REFERENCES 작업자_자연키(사번),
    설비 TEXT
```

```
);
`);

await db.exec(`INSERT INTO 작업자_자연키 VALUES ('A-001', '김반장')`);
await db.exec(`INSERT INTO 배정_자연키 VALUES ('A-001', '컨베이어 1호')`);
```

3년 뒤, 인사팀이 사번 체계를 바꿉니다. A-001 → 2026-A-001.

```
await 시도("사번 체계를 바꾸면", `UPDATE 작업자_자연키 SET 사번 = '2026-A-001' WHERE
사번 = 'A-001'`);
```

출력 사번 체계를 바꾸면 - 23503

★★★ 23503 = 외래키 위반. 다른 표가 그 값을 붙들고 있어서 못 바꿉니다.

REFERENCES 는 "이 값은 저 표에 있어야 한다" 는 제약입니다. 04단원에서 제대로 합니다. 여기서는 이것만 보세요. 자연키가 바뀌면 그 값을 쓰는 모든 표를 같이 고쳐야 합니다.

실제 현장에서 자연키가 바뀌는 이유들입니다. 전부 실제로 일어납니다.

- 사번 체계 개편 (회사 합병, 계열사 통합)
- 설비번호를 라인 재배치하면서 다시 붙임
- 사업자등록번호가 바뀜 (법인 전환)
- 처음에 "절대 안 바뀐다" 던 코드값이 바뀜
- 잘못 입력한 사번을 고쳐야 함 ← 이게 제일 흔합니다

★ 인조키로 하면 이렇습니다.

```
await db.exec(`
  CREATE TABLE 작업자 (
    작업자번호 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    사번 TEXT NOT NULL UNIQUE,
    이름 TEXT NOT NULL
  );
`);

await db.exec(`INSERT INTO 작업자 (사번, 이름) VALUES ('A-001', '김반장')`);
await 시도("인조키 표에서 사번 바꾸기", `UPDATE 작업자 SET 사번 = '2026-A-001' WHERE
사번 = 'A-001'`);
```

출력 인조키 표에서 사번 바꾸기 - 들어감

```
const 바뀐것 = (await db.query(`SELECT 작업자번호, 사번, 이름 FROM 작업자`)).rows[0];
console.log("작업자번호는 그대로:", JSON.stringify(바뀐것));
```

출력 작업자번호는 그대로: {"작업자번호":1,"사번":"2026-A-001","이름":"김반장"}

★★ 사번은 바뀌었는데 **작업자번호는 그대로**입니다. 다른 표들은 작업자번호를 붙들고 있으니 아무 영향이 없습니다. 그리고 사번에 UNIQUE 를 걸어 두었으니 중복도 여전히 막힙니다.

★ 정리

인조키(BIGINT IDENTITY) 를 기본키로 ← 기본 선택
자연키(사번)는 UNIQUE 로 ← 규칙은 그대로 지켜집니다

“업무 값을 키로 쓰지 마라” 가 아니라 **“업무 값은 바뀔 수 있으니 키로 삼지 마라”** 입니다.

★★ 주민등록번호는 아예 저장하지 마세요. 키로도, 칸으로도요. 법적으로 아주 까다롭고, 유출되면 회사가 흔들립니다.

복합 기본키

섹션 9

칸 두 개를 묶어서 기본키로 삼을 수도 있습니다.

```
await db.exec(`
  CREATE TABLE 일일생산 (
    설비번호 INT NOT NULL,
    생산일 DATE NOT NULL,
    수량 INT NOT NULL CHECK (수량 >= 0),
    PRIMARY KEY (설비번호, 생산일)
  );
`);

await db.exec(`
  INSERT INTO 일일생산 VALUES (1, '2026-03-15', 100), (1, '2026-03-16', 120), (2, '2026-03-15', 80)
`);

await 시도("같은 설비·같은 날을 또", `INSERT INTO 일일생산 VALUES (1, '2026-03-15', 999)`);
```

출력 같은 설비·같은 날을 또 - 23505

```
await 시도("같은 설비·다른 날", `INSERT INTO 일일생산 VALUES (1, '2026-03-17', 90)`);
```

출력 같은 설비·다른 날 - 들어감

```
const 복합정의 = (await db.query(`
  SELECT pg_get_constraintdef(oid) AS 정의 FROM pg_constraint
  WHERE conrelid = '일일생산'::regclass AND contype = 'p'
`)).rows[0];
console.log("복합 기본키:", 복합정의.정의);
```

출력 복합 기본키: PRIMARY KEY ("설비번호", "생산일")

★ “설비 하나가 하루에 한 줄”이라는 업무 규칙이 표 정의에 박혔습니다. 좋습니다.

★★ 그런데 복합 기본키는 불편한 점이 있습니다. · 다른 표에서 이 줄을 가리키려면 칸을 두 개 들고 다녀야 합니다 · 프로그램에서 /일일생산/1 처럼 주소를 만들 수 없습니다 · 칸이 하나 더 늘면(교대조 같은 것) 기본키를 통째로 바꿔야 합니다

★ 그래서 실무에서는 이렇게 하는 경우가 많습니다.

```
생산번호 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  ← 인조키
UNIQUE (설비번호, 생산일)                                     ← 규칙은 UNIQUE 로
```

기본키는 단순하게 두고, 업무 규칙은 UNIQUE 로 지킵니다.
"설비 하나가 하루에 한 줄" 은 똑같이 지켜집니다.

★★ 복합 기본키가 자연스러운 자리도 있습니다.

04단원에서 배울 **N:M 연결표** (작업자-자격 같은 것) 가 그렇습니다.

정리 — 기본키를 무엇으로 할 것인가

방법 언제 쓰나 주의

BIGINT GENERATED ALWAYS AS ★ 기본 선택 값을 직접 못 넣음

IDENTITY

(그게 장점)

SERIAL / BIGSERIAL 오래된 코드에서 만남 ★ setval 잊으면 터짐 UUID DEFAULT gen_random_uuid()
여러 곳에서 만들 때 2배 큼, 순서 없음 자연키 (사번 등) 거의 쓰지 않음 ★ 바뀌면 전부 고쳐야 함 복합
기본키 N:M 연결표 가리키기 불편

자동 번호에 대해 재 본 것

지운 번호 재활용? ★ 안 합니다 (SQLite 와 다름) 롤백하면 번호가 돌아오나? ★ 안 돌아옵니다

TRUNCATE 하면? 안 돌아옵니다 (RESTART IDENTITY 를 붙여야) 그래서 번호는 연속인가? 아닙니다.
유일할 뿐입니다

에러 코드

23505 중복 (PK · UNIQUE)
 23502 NULL (PK 칸을 비움)
 23503 외래키 위반 (다른 표가 붙들고 있음) - 04단원
 428C9 IDENTITY ALWAYS 칸에 직접 값을 넣음

★ 세 줄 요약 ① 새 표는 `BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY` ② 업무 값(사번·설비코드)은 기본키가 아니라 **UNIQUE** 로 ③ 데이터를 옮긴 뒤에는 **setval** 을 잊지 마세요

직접 해 볼 것

▫ 직접 해보기 1

섹션 3 을 IDENTITY 로 다시 해 보세요. `OVERRIDING SYSTEM VALUE` 로 2, 3 을 넣은 뒤 자동으로 넣으면 SERIAL 과 똑같이 터지나요? (힌트: 시퀀스는 여전히 모릅니다)

▫ 직접 해보기 2

`SELECT nextval('설비_시리얼_설비번호_seq')` 를 세 번 불러 보세요. INSERT 를 안 했는데도 번호가 올라가나요?

▫ 직접 해보기 3

`ALTER SEQUENCE ... RESTART WITH 100` 으로 시퀀스를 100부터 시작하게 해 보세요. 무엇에 쓸 수 있을까요?

▫ 직접 해보기 4

UUID 를 기본키로 쓰는 표와 BIGINT 를 쓰는 표에 각각 1만 건을 넣고 시간을 재 보세요. 차이가 나나요?
 ★ 값은 기계마다 다릅니다. 판정은 `UUID쪽ms > BIGINT쪽ms` 로 찍으세요

▫ 직접 해보기 5

섹션 8 의 `배정_자연키` 에 `ON UPDATE CASCADE` 를 붙여 보세요. 사번을 바꿀 수 있게 되나요? 그러면 자연키를 써도 될까요? (힌트: 되긴 됩니다. 그런데 표가 열 개면 열 개가 다 같이 바뀝니다)

▫ 직접 해보기 6

기본키가 없는 표를 만들고 완전히 똑같은 줄을 두 개 넣어 보세요. 그중 하나만 지울 수 있나요? (힌트: WHERE 에 뭘 쓰죠?)

◀ 직접 해보기 7

복합 기본키 (설비번호, 생산일) 에 교대조를 넣어 (설비번호, 생산일, 교대조) 로 바꾸려면 어떻게 하나요? 이미 데이터가 있으면 어떻게 되나요?

자주 하는 실수

이런 실수를 합니다

데이터를 옮기고 setval 을 안 함

★★★ 이 파일에서 가장 자주 나는 사고입니다. 그날은 멀쩡하고 며칠 뒤에 23505 로 터집니다. 원인을 못 찾습니다. ★ IDENTITY ALWAYS 를 쓰면 애초에 이 상황이 안 생깁니다.

이런 실수를 합니다

자동 번호를 "몇 건" 으로 씀

롤백·삭제 때문에 번호는 건너뛰니다. 세는 것은 count(*) 로 하세요.

이런 실수를 합니다

사번·설비코드를 기본키로 삼음

"이건 절대 안 바뀝니다" 는 3년 안에 틀립니다. 기본키는 인조키로, 업무 값은 UNIQUE 로.

이런 실수를 합니다

UUID 를 TEXT 로 저장

36바이트가 됩니다. UUID 타입은 16바이트입니다. 색인 크기가 두 배 넘게 차이납니다.

이런 실수를 합니다

기본키 없는 표를 만들

똑같은 줄이 두 개 생기면 하나만 지울 방법이 없습니다. 복제·백업 도구도 기본키가 없으면 제대로 못 씁니다.

이런 실수를 합니다

키를 INT 로 잡음

21억은 생각보다 빨리 잡니다. 그리고 다 찬 다음에 바꾸려면 표를 통째로 다시 씁니다. ★ 처음부터 BIGINT 로 잡으세요. 4바이트 아끼려다 새벽에 일어납니다.

```
await db.close();
```

NULL 은 값이 아닙니다

RUN node 개념05_NULL은_값이_아닙니다.js

개념03 에서 “CHECK 는 NULL 을 막지 못한다” 는 것을 봤습니다. 그때 넘어간 이야기를 이제 합니다.

NULL 은 초보자를 가장 많이 넘어뜨리는 것입니다. 문법이 어려워서가 아닙니다. **아무 에러 없이 조용히 틀리기 때문**입니다.

이 파일에서 실제로 재 볼 것들입니다.

① `NULL = NULL` 은 참이 아닙니다 ② ★★★ `NOT IN` 에 NULL 이 섞이면 결과가 **0건**이 됩니다. 에러도 안 납니다 ③ `UNIQUE` 칸에 NULL 은 여러 개 들어갑니다 ④ `SUM` 은 NULL 을 건너뛰고, `AVG` 는 분모에서도 뺍니다

★ 딱 한 문장만 외우세요. **NULL 은 “값이 없다” 가 아니라 “모른다” 입니다.** 0 도, 빈 문자열도, `false` 도 아닙니다. “모르는 것” 끼리는 같은지도 모릅니다. 여기서 전부가 따라 나옵니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

모른다는 것

섹션 1

안이 안 보이는 상자가 둘 있습니다. “내용물이 같습니까?” 답은 “예” 도 “아니오” 도 아닙니다. **모릅니다.** SQL 이 정확히 그렇게 답합니다.

```
const 비교 = (await db.query(`
  SELECT
    (NULL = NULL) AS 널널같나,
    (NULL <> NULL) AS 널널다른가,
    (1 = NULL)     AS 일과널,
    ('' = NULL)    AS 빈문자와널,
    (0 = NULL)     AS 영과널
`)).rows[0];

console.log(JSON.stringify(비교));
```

```
출력      {"널널같나":null,"널널다른가":null,"일과널":null,"빈문자와널":null,"영과널":null}
```

★★★ 전부 `null` 입니다. **false 가 아닙니다.** `NULL = NULL`조차 참이 아닙니다. “모르는 것” 끼리 같은지 모르니까요. 자바스크립트의 `null === null` 이 `true` 인 것과 완전히 다릅니다.

```
console.log("JS 에서 null === null:", null === null);
```

```
출력      JS 에서 null === null: true
```

★ 그리고 NULL 은 계산에도 옮습니다. 한 방울이면 전체가 NULL 이 됩니다.

```
const 계산 = (await db.query(`
  SELECT (1 + NULL)::text AS 더하기, ('가' || NULL) AS 잇기, concat('가', NULL) AS
  concat함수, upper(NULL) AS 대문자
`)).rows[0];
console.log(JSON.stringify(계산));
```

```
출력      {"더하기":null,"잇기":null,"concat함수":"가","대문자":null}
```

★ || 로 이으면 전체가 NULL 이 되는데 concat() 함수는 NULL 을 빈 문자열로 봅니다. 둘이 다릅니다. 이름을 이어 붙일 때 성 || ' ' || 이름을 쓰면 성이 없는 사람의 이름이 통째로 사라집니다. concat 을 쓰세요.

★★ 3값 논리 — TRUE / FALSE / UNKNOWN

섹션 2

자바스크립트의 불리언은 두 가지입니다. SQL 은 세 가지입니다. 진리표를 직접 뽑아 봅니다.

```
const 진리표 = (await db.query(`
  SELECT
    coalesce(x::text, 'NULL') AS x,
    coalesce(y::text, 'NULL') AS y,
    coalesce((x AND y)::text, 'NULL') AS 그리고,
    coalesce((x OR y)::text, 'NULL') AS 또는
  FROM (VALUES (true,true),(true,false),(true,NULL),(false,false),(false,NULL),
  (NULL,NULL)) AS v(x,y)
`)).rows;

for (const 줄 of 진리표) {
  console.log(`${줄.x} AND ${줄.y} = ${줄.그리고}   ·   ${줄.x} OR ${줄.y} = ${줄.
  또는}`);
}
```

```
출력      true AND true = true   ·   true OR true = true
true AND false = false   ·   true OR false = true
true AND NULL = NULL   ·   true OR NULL = true
false AND false = false   ·   false OR false = false
false AND NULL = false   ·   false OR NULL = NULL
NULL AND NULL = NULL   ·   NULL OR NULL = NULL
```

★ 두 줄만 보면 규칙이 보입니다. 나머지는 전부 UNKNOWN 입니다. `false AND NULL = false` ← 하나가 거짓이면 나머지를 몰라도 거짓 `true OR NULL = true` ← 하나가 참이면 나머지를 몰라도 참

★★★ 그리고 **WHERE** 는 **TRUE** 인 줄만 통과시킵니다. FALSE 도 UNKNOWN 도 똑같이 버립니다.
이게 모든 사고의 근원입니다.

그래서 IS NULL 이 필요합니다

섹션 3

```
const 아이에스 = (await db.query(`
  SELECT
    (NULL IS NULL) AS is널,
    (NULL IS NOT NULL) AS isnot널,
    (NULL IS NOT DISTINCT FROM NULL) AS distinct널널,
    (1 IS DISTINCT FROM NULL) AS distinct일널
`)).rows[0];
console.log(JSON.stringify(아이에스));
```

출력 {"is널":true,"isnot널":false,"distinct널널":true,"distinct일널":true}

★ `IS NULL` / `IS NOT NULL` 은 항상 `true` 나 `false` 를 줍니다. UNKNOWN 이 없습니다.

★★ `IS DISTINCT FROM` 은 NULL 을 값처럼 취급해서 비교합니다.

A IS DISTINCT FROM B → 다른가? A IS NOT DISTINCT FROM B → 같은가?

자바스크립트의 `!==` / `===` 처럼 동작합니다. 값이 NULL 일 수 있을 때 유용합니다.

WHERE 에서 줄이 조용히 사라집니다

섹션 4

```
await db.exec(`
  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름 TEXT NOT NULL,
    라인 TEXT,
    담당자 TEXT,
    마지막점검일 DATE
  );
`);

await db.exec(`
  INSERT INTO 설비 VALUES
    (1, '컨베이어 1호', 'A', '김반장', '2026-03-01'),
    (2, '프레스 1호', 'A', NULL, '2026-03-05'),
```

```
(4, '절단기 1호', NULL, NULL, NULL);
`);

const 세기 = async (설명, 조건) => {
  const 결과 = await db.query(`SELECT count(*)::int AS 개수 FROM 설비 WHERE
${조건}`);
  console.log(`${설명} → ${결과.rows[0].개수}건`);
};

console.log("전체 4건 중에서");
```

출력 전체 4건 중에서

```
await 세기("담당자 = '김반장'", `담당자 = '김반장'`);
```

출력 담당자 = '김반장' → 1건

```
await 세기("담당자 <> '김반장'", `담당자 <> '김반장'`);
```

출력 담당자 <> '김반장' → 1건

```
await 세기("담당자 IS NULL", `담당자 IS NULL`);
```

출력 담당자 IS NULL → 2건

```
await 세기("담당자 IS DISTINCT FROM '김반장'", `담당자 IS DISTINCT FROM '김반장'`);
```

출력 담당자 IS DISTINCT FROM '김반장' → 3건

★★★ = '김반장' 1건 + <> '김반장' 1건 = **2건**. 전체는 4건인데요. 담당자가 NULL 인 2건이 **양쪽 어디에도 안 들어갔습니다**. 화면 두 개를 합쳐도 전체가 안 됩니다. "설비가 사라졌어요" 라는 신고가 들어옵니다.

★ 고치는 법은 둘입니다.

```
`담당자 IS DISTINCT FROM '김반장'`          ← NULL 도 "김반장이 아님" 으로 봄
`담당자 <> '김반장' OR 담당자 IS NULL`      ← 같은 뜻, 더 길게
```

★★★ NOT IN 에 NULL 이 섞이면 0건이 됩니다

섹션 5

이 파일에서 가장 중요한 부분입니다. 실무에서 정말 자주 터집니다.

```
await db.exec(`CREATE TABLE 정비중 (설비번호 INT)`);
await db.exec(`INSERT INTO 정비중 VALUES (1), (NULL)`);
```

정비중 표에 NULL 이 한 줄 섞였습니다. (설비번호가 NOT NULL 이 아니라서 들어왔습니다. 개념03 을 안 지킨 표입니다)

```
const 인 = (await db.query(`
  SELECT 설비번호 FROM 설비 WHERE 설비번호 IN (SELECT 설비번호 FROM 정비중) ORDER BY
  설비번호
`)).rows;
console.log("IN - 정비 중인 설비:", JSON.stringify(인.map((줄) => 줄.설비번호)));
```

출력 IN - 정비 중인 설비: [1]

```
const 낫인 = (await db.query(`
  SELECT 설비번호 FROM 설비 WHERE 설비번호 NOT IN (SELECT 설비번호 FROM 정비중) ORDER
  BY 설비번호
`)).rows;
console.log("NOT IN - 정비 중이 아닌 설비:", JSON.stringify(낫인.map((줄) => 줄.
설비번호)));
```

출력 NOT IN - 정비 중이 아닌 설비: []

```
console.log("가동 가능한 설비가 0대로 나왔나:", 낫인.length === 0);
```

출력 가동 가능한 설비가 0대로 나왔나: true

★★★ 0건입니다. 2, 3, 4번은 정비 중이 아닌데도요. 에러도 경고도 없습니다. "오늘 가동 가능한 설비" 화면이 텅 빕니다.

왜 이렇게 되나

설비번호 NOT IN (1, NULL) 은 이렇게 풀립니다.

```
NOT (설비번호 = 1 OR 설비번호 = NULL)
```

2번 설비로 계산해 봅시다.

```
설비번호 = 1      → false
설비번호 = NULL → **UNKNOWN**
false OR UNKNOWN → **UNKNOWN** (섹션 2 의 진리표를 보세요)
NOT UNKNOWN      → **UNKNOWN**
WHERE 는 TRUE 만 통과 → **버려집니다**
```

NULL 이 하나라도 있으면 모든 줄이 UNKNOWN 이 됩니다. 그래서 0건입니다.

직접 값으로도 확인합니다.

```
const 직접 = (await db.query(`SELECT (2 NOT IN (1, NULL)) AS 이번, (1 NOT IN (1, NULL)) AS 일번`)).rows[0];
console.log("2 NOT IN (1, NULL) =", 직접.이번, "· 1 NOT IN (1, NULL) =", 직접.일번);
```

```
출력      2 NOT IN (1, NULL) = null · 1 NOT IN (1, NULL) = false
```

★ 1 NOT IN (1, NULL) 만 false 입니다. 1은 확실히 있으니까요. 나머지는 전부 UNKNOWN 입니다.

어떻게 고치나

방법 A. **NOT EXISTS** 를 쓴다 ← 이게 정답입니다

```
const 낫이그지스트 = (await db.query(`
  SELECT 설비번호 FROM 설비 가
  WHERE NOT EXISTS (SELECT 1 FROM 정비중 나 WHERE 나.설비번호 = 가.설비번호)
  ORDER BY 설비번호
`)).rows;
console.log("NOT EXISTS 로 하면:", JSON.stringify(낫이그지스트.map((줄) => 줄.설비번호)));
```

```
출력      NOT EXISTS 로 하면: [2,3,4]
```

방법 B. 안에서 NULL 을 걸러낸다

```
const 널제거 = (await db.query(`
  SELECT 설비번호 FROM 설비
  WHERE 설비번호 NOT IN (SELECT 설비번호 FROM 정비중 WHERE 설비번호 IS NOT NULL)
  ORDER BY 설비번호
`)).rows;
console.log("IS NOT NULL 을 붙이면:", JSON.stringify(널제거.map((줄) => 줄.설비번호)));
```

```
출력      IS NOT NULL 을 붙이면: [2,3,4]
```

방법 C. 애초에 그 칸에 **NOT NULL** 을 건다 ← 근본 해결

```
`정비중.설비번호 INT NOT NULL` 이었으면 이 사고 자체가 없습니다.
```

★★★ 기억할 것: **NOT IN** 에 서브쿼리를 쓰면 **NOT EXISTS** 로 바꿔 쓰세요. 습관으로 만드세요. 그러면 이 사고를 평생 안 만납니다. (참고로 IN 은 괜찮습니다. NULL 이 섞여도 있는 것은 찾아 줍니다)

```
await db.exec(`CREATE TABLE 생산 (설비번호 INT, 수량 INT)`);
await db.exec(`INSERT INTO 생산 VALUES (1,100), (2,NULL), (3,200)`);

const 집계 = (await db.query(`
  SELECT count(*)::int AS 줄수, count(수량)::int AS 수량있는줄, sum(수량)::int AS 합,
    avg(수량)::text AS 평균, max(수량)::int AS 최대, min(수량)::int AS 최소
  FROM 생산
`)).rows[0];

console.log(`줄수 ${집계.줄수} · 수량있는줄 ${집계.수량있는줄} · 합 ${집계.합}`);
```

출력 줄수 3 · 수량있는줄 2 · 합 300

```
console.log(`평균 ${집계.평균} · 최대 ${집계.최대} · 최소 ${집계.최소}`);
```

출력 평균 150.000000000000000000 · 최대 200 · 최소 100

★★ 여기가 헛갈립니다.

`count(*)` → 줄 수를 셉니다. NULL 과 무관하게 3 `count(수량)` → 수량이 NULL 이 아닌 줄만 셉니다. 2

```
`sum(수량)` → NULL 을 건너뛰고 더합니다. 300
`avg(수량)` → 300 / **2** = 150. ← 3으로 안 나눕니다!
```

★★★ 평균을 손으로 내면 다릅니다.

```
console.log("합 / 줄수 로 계산하면:", 300 / 3);
```

출력 합 / 줄수 로 계산하면: 100

```
console.log("avg 와 같은가:", Number(집계.평균) === 300 / 3);
```

출력 avg 와 같은가: false

★ 100 이냐 150 이냐는 **업무가 정할 문제**입니다. SQL 의 기본은 “뺀다” 입니다. 넣고 싶으면 `avg(coalesce(수량, 0))` 로 씁니다. ★★ 중요한 건 어느 쪽을 원하는지 알고 쓰는 것입니다.

모르고 쓰면 보고서 숫자가 틀리고, 아무도 눈치 못 챵니다.

★ 한 줄도 없으면 sum 은 0 이 아니라 NULL 입니다.


```
const 빈것 = (await db.query(`SELECT sum(수량)::int AS 합, count(*)::int AS 줄수 FROM
생산 WHERE 설비번호 = 999`)).rows[0];
console.log("한 줄도 없을 때 - sum:", 빈것.합, "· count:", 빈것.줄수);
```

```
출력      한 줄도 없을 때 - sum: null · count: 0
```

★★ 이걸 그대로 화면에 쓰면 “총 생산량: null” 이 찍힙니다. `coalesce(sum(수량), 0)` 으로 감싸세요.

COALESCE 와 NULLIF

섹션 7

`COALESCE(a, b, c)` — 앞에서부터 **NULL 이 아닌 첫 값**을 돌려줍니다.

```
const 코알 = (await db.query(`
  SELECT 설비번호,
         coalesce(담당자, '미배정') AS 담당,
         coalesce(마지막점검일::text, '점검이력없음') AS 점검
  FROM 설비 ORDER BY 설비번호
`)).rows;

for (const 줄 of 코알) {
  console.log(`${줄.설비번호}번 · ${줄.담당} · ${줄.점검}`);
}
```

```
출력      1번 · 김반장 · 2026-03-01
          2번 · 미배정 · 2026-03-05
          3번 · 이반장 · 점검이력없음
          4번 · 미배정 · 점검이력없음
```

`NULLIF(a, b)` — a 와 b 가 같으면 NULL, 다르면 a. `COALESCE` 의 반대입니다.

```
const 널이프 = (await db.query(`SELECT nullif(0,0) AS 영영, nullif(1,0) AS 일영, (10
/ nullif(0,0))::text AS 나누기`)).rows[0];
console.log(JSON.stringify(널이프));
```

```
출력      {"영영":null,"일영":1,"나누기":null}
```

★ `10 / nullif(분모, 0)` 는 **0으로 나누기 에러를 피하는 정석**입니다. 결과가 NULL 이 되므로 `coalesce(..., 0)` 로 감싸서 씁니다.

★★ `COALESCE` 를 남발하지 마세요. `coalesce(담당자, '미배정')` 을 조회할 때마다 붙이고 있다면, 그건 **설계 때 정했어야 할 것을 매번 미루고 있는 것**입니다. 담당자가 없을 수 없다면 `NOT NULL DEFAULT '미배정'` 으로 표에 박으세요.

```
await db.exec(`CREATE TABLE 작업자 (사번 TEXT UNIQUE, 이름 TEXT NOT NULL)`);
await db.exec(`INSERT INTO 작업자 VALUES (NULL,'가'), (NULL,'나'), (NULL,'다')`);

const 널개수 = (await db.query(`SELECT count(*)::int AS 개수 FROM 작업자 WHERE 사번
IS NULL`)).rows[0].개수;
console.log("UNIQUE 칸에 NULL 을 세 번 넣었더니:", 널개수, "건 들어감");
```

출력 UNIQUE 칸에 NULL 을 세 번 넣었더니: 3 건 들어감

★★ UNIQUE 는 “같은 값이 두 번 오면 안 된다” 인데, NULL 끼리는 **같은지 모르므로** 중복으로 안 봅니다. 섹션 1 의 규칙 그대로입니다. “사번은 유일하니 UNIQUE” 를 걸어 놓고 사번 안 적은 작업자가 100명 생깁니다. ★ UNIQUE 를 걸 때는 **NOT NULL 도 같이** 거는지 꼭 생각하세요.

★ Postgres 15 부터는 NULL 도 중복으로 보게 할 수 있습니다.

```
await db.exec(`CREATE TABLE 작업자2 (사번 TEXT UNIQUE NULLS NOT DISTINCT, 이름 TEXT
NOT NULL)`);
try {
  await db.query(`INSERT INTO 작업자2 VALUES (NULL,'가'), (NULL,'나')`);
  console.log("NULLS NOT DISTINCT 로 NULL 두 번 - 들어감");
} catch (에러) {
  console.log("NULLS NOT DISTINCT 로 NULL 두 번 -, 에러.code);
```

출력 NULLS NOT DISTINCT 로 NULL 두 번 - 23505

```
}
```

★ 다만 이건 최근 기능이라 다른 DB 로 옮길 때 안 통합니다. 그냥 NOT NULL 을 거는 편이 낫습니다.

ORDER BY 와 GROUP BY 에서 NULL 은 어디에

```
const 정렬 = async (설명, 문구) => {
  const 결과 = await db.query(`SELECT 담당자 FROM 설비 ORDER BY ${문구}`);
  console.log(`${설명} → ${결과.rows.map((줄) => 줄.담당자 ?? "NULL").join(", ")}`);
};
```

```
await 정렬("ORDER BY 담당자 (기본 ASC)", "담당자");
```

출력 ORDER BY 담당자 (기본 ASC) → 김반장, 이반장, NULL, NULL

```
await 정렬("ORDER BY 담당자 DESC", "담당자 DESC");
```

출력 ORDER BY 담당자 DESC → NULL, NULL, 이반장, 김반장

```
await 정렬("ORDER BY 담당자 ASC NULLS FIRST", "담당자 ASC NULLS FIRST");
```

```
출력      ORDER BY 담당자 ASC NULLS FIRST → NULL, NULL, 김반장, 이반장
```

★ Postgres 의 기본값

```
ASC → NULLS LAST (NULL 이 맨 뒤)
```

```
DESC → NULLS FIRST (NULL 이 맨 앞)
```

즉 정렬 방향을 바꾸면 NULL 위치도 바뀝니다. “최근 점검일 순” 화면에서 정렬을 뒤집었더니 점검 안 한 설비가 맨 위로 올라옵니다.

★ 화면에서 쓸 정렬은 NULLS LAST 를 명시하는 습관을 들이세요.

MySQL 은 여기가 다릅니다

· NULL 을 항상 “가장 작은 값” 으로 봅니다. ASC 면 앞, DESC 면 뒤 · NULLS FIRST/LAST 문법이 없습니다

★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

★ GROUP BY 는 반대입니다. NULL 끼리 한 묶음으로 모읍니다.

```
const 그룹 = (await db.query(`
  SELECT coalesce(라인, '(없음)') AS 표시라인, count(*)::int AS 대수
  FROM 설비 GROUP BY 라인 ORDER BY 라인 NULLS LAST
`)).rows;
for (const 줄 of 그룹) console.log(`${줄.표시라인} 라인 - ${줄.대수}대`);
```

```
출력      A 라인 - 2대
          B 라인 - 1대
          (없음) 라인 - 1대
```

★★ 헛갈립니다. 같은 NULL 인데

```
`=` 로 비교하면 → 같지 않음 (UNKNOWN)
```

```
GROUP BY 로 모으면 → 같은 묶음
```

```
UNIQUE 로 보면 → 다른 값
```

상황마다 다릅니다. 그래서 NULL 을 피하는 게 최선입니다.

```
const 널로찾기 = await db.query(`SELECT count(*)::int AS 개수 FROM 설비 WHERE 담당자 = $1`, [null]);
console.log("담당자 = $1 에 null 을 넘기면:", 널로찾기.rows[0].개수, "건");
```

출력 담당자 = \$1 에 null 을 넘기면: 0 건

```
const 제대로 = await db.query(`SELECT count(*)::int AS 개수 FROM 설비 WHERE 담당자 IS NOT DISTINCT FROM $1`, [null]);
console.log("IS NOT DISTINCT FROM $1 에 null 을 넘기면:", 제대로.rows[0].개수, "건");
```

출력 IS NOT DISTINCT FROM \$1 에 null 을 넘기면: 2 건

★★★ 검색 화면에서 “담당자” 를 비워 두고 검색하면 \$1 에 null 이 갑니다. = \$1 은 항상 0건입니다. 에러도 안 납니다. 해결은 둘입니다.

- ① `IS NOT DISTINCT FROM \$1` 을 쓴다
- ② 조건 자체를 안 붙인다 (`WHERE (\$1 IS NULL OR 담당자 = \$1)`)

★ 자바스크립트에서 undefined 를 넘기면 어떻게 될까요.

```
const 언디파인드 = await db.query(`SELECT count(*)::int AS 개수 FROM 설비 WHERE 담당자 = $1`, [undefined]);
console.log("undefined 를 넘기면:", 언디파인드.rows[0].개수, "건");
```

출력 undefined 를 넘기면: 0 건

★ null 로 취급됩니다. 즉 오타로 req.body.담당자 를 잘못 써서 undefined 가 가도 에러 없이 0건입니다. 조용히 틀리는 종류입니다.

★ 그래서 설계 때 무엇을 정해야 하나

칸을 하나 만들 때마다 이 질문에 답하세요.

이 칸이 비어 있는 상태가 업무적으로 말이 되는가?

말이 안 되면 → NOT NULL. 필요하면 DEFAULT 도 같이 말이 되면 → NULL 허용. 그리고 그 뜻을 문서에 적으세요 “아직 모른다” 와 “해당 없음” 이 다르면 → 칸을 나누거나 상태 칸을 따로 두세요

★ 예를 들어 마지막점검일 이 NULL 이면 무슨 뜻입니까?

- 한 번도 점검을 안 했다?
- 점검은 했는데 기록을 못 찾겠다?
- 점검 대상이 아닌 설비다?

셋은 완전히 다른 이야기인데 표에서는 똑같이 NULL 입니다. 그래서 나중에 “점검 안 한 설비 목록” 을 뽑으면 엉뚱한 게 섞입니다.

★★ 어중간하게 두면 평생 이런 코드를 답니다.

```
coalesce(담당자, '미배정')
coalesce(수량, 0)
WHERE (마지막점검일 IS NULL OR 마지막점검일 < ...)
```

조회할 때마다, 화면마다, 개발자마다 붙입니다. 한 군데만 빠뜨리면 숫자가 틀립니다.

★★★ NULL 을 허용할지는 설계 시점에 정하세요. 나중에 정하면 이미 NULL 이 들어와 있어서 못 정합니다. (개념03 섹션 7)

정리 — NULL 이 하는 짓

상황 결과 대처

NULL = NULL NULL (참 아님) IS NULL 1 + NULL NULL coalesce '가' || NULL NULL concat() 을 쓰기 WHERE 담당자 <> '김' NULL 인 줄이 빠짐 IS DISTINCT FROM ★★★ NOT IN (... , NULL) 0건 NOT EXISTS count(*) vs count(칸) 줄 수 vs 값 있는 줄 수 의도한 쪽을 쓰기

sum(칸)	NULL 을 건너뛴	coalesce(sum(.),0)
avg(칸)	분모에서도 뺀	avg(coalesce(칸,0))

UNIQUE 칸의 NULL 여러 개 들어감 NOT NULL 도 같이 ORDER BY ASC NULL 이 맨 뒤 NULLS LAST 명시 ORDER BY DESC NULL 이 맨 앞 NULLS LAST 명시 GROUP BY NULL 끼리 한 묶음 WHERE 칸 = \$1 에 null 0건 IS NOT DISTINCT FROM

★ 세 줄 요약 ① NULL 은 “모른다” 입니다. 0 도 빈 문자열도 아닙니다 ② WHERE 는 TRUE 만 통과시킵니다. UNKNOWN 은 FALSE 와 똑같이 버려집니다 ③ NOT IN 서브쿼리는 NOT EXISTS 로 바꿔 쓰세요. 습관으로 만드세요

★★ 그리고 가장 좋은 대처는 NULL 이 안 들어오게 하는 것입니다. NOT NULL 을 거는 데는 1초가 걸리고, NULL 을 다루는 데는 평생이 걸립니다.

직접 해 볼 것

➤ 직접 해보기 1

섹션 5 의 정비중 표에서 NULL 줄을 지운 뒤 NOT IN 을 다시 해 보세요. 이제 2, 3, 4 가 나오나요? 그다음 NULL 을 한 줄 다시 넣고 해 보세요. 다시 0건이 되나요?

▹ 직접 해보기 2

`SELECT 1 WHERE NULL` 과 `SELECT 1 WHERE false` 를 비교해 보세요. 둘 다 0건인가요? 그럼 뭐가 다른가요? (힌트: `SELECT 1 WHERE NOT NULL` 과 `SELECT 1 WHERE NOT false` 를 해 보세요)

▹ 직접 해보기 3

설비 표에서 “점검한 지 30일이 넘었거나 한 번도 안 한 설비” 를 뽑아 보세요. `WHERE` 마지막 점검일 < `CURRENT_DATE - 30` 만 쓰면 몇 건이 나오나요? 한 번도 안 한 설비가 포함되나요?

▹ 직접 해보기 4

섹션 6 의 생산 표에 수량이 NULL 인 줄을 두 개 더 넣어 보세요. `avg` 가 바뀌나요? `avg(coalesce(수량, 0))` 은요?

▹ 직접 해보기 5

이름 `TEXT NOT NULL` 인 칸에 빈 문자열 '' 을 넣어 보세요. 들어가나요? `NOT NULL` 이 빈 문자열도 막아 주나요? (힌트: 개념03 섹션 2 의 `CHECK` 를 떠올리세요)

▹ 직접 해보기 6

'가' || `NULL` 과 `concat('가', NULL)` 을 각각 해 보세요. 성이 없는 사람의 전체 이름을 만든다면 어느 쪽을 써야 하나요?

▹ 직접 해보기 7

설비 표에 `UNIQUE` (라인, 담당자) 를 걸고 라인과 담당자가 둘 다 NULL 인 줄을 두 개 넣어 보세요. 들어가나요?

자주 하는 실수

이런 실수를 합니다

★★★ NOT IN 서브쿼리에 NULL 이 섞임

결과가 0건이 되는데 에러가 안 납니다. 이 파일에서 가장 중요한 실수입니다. `NOT EXISTS` 를 쓰세요.

이런 실수를 합니다

<> 로 “아닌 것” 을 뽑으면 전부 나올 거라고 생각

NULL 인 줄은 = 에도 <> 에도 안 걸립니다. `IS DISTINCT FROM` 을 쓰거나 `OR ... IS NULL` 을 붙이세요.

이런 실수를 합니다

= NULL 이라고 씀

문법 오류가 안 납니다. 그냥 항상 0건입니다. IS NULL 을 쓰세요.

이런 실수를 합니다

avg 가 전체 줄로 나눌 거라고 생각

NULL 인 줄은 분모에서도 빠집니다. 보고서 숫자가 조용히 틀립니다.

이런 실수를 합니다

UNIQUE 만 걸고 NOT NULL 을 안 걸

NULL 은 몇 개든 들어갑니다. "사번은 유일" 이 안 지켜집니다.

이런 실수를 합니다

파라미터에 null 을 넘기고 = \$1 로 찾을

항상 0건입니다. 검색 화면에서 빈 칸을 넘길 때 자주 납니다.

이런 실수를 합니다

NULL 과 빈 문자열을 섞어 씀

어떤 코드는 '', 어떤 코드는 NULL 을 넣으면 둘 다 검사해야 합니다. ★ 하나로 정하세요. 보통 빈 문자열을 금지(CHECK)하고 NULL 만 씁니다.

```
await db.close();
```

표 설계하기

RUN node 연습문제.js

아래 답.문제N = ... 자리를 채우세요. 지금은 전부 null 입니다. 채운 뒤 실행하면 맞았는지 알려 줍니다.

☐ 아직 안 풀었습니다 → null 그대로 ❌ 틀렸습니다 → 채웠는데 답이 아님 ✅ 정답

★ 순서대로 푸세요. 뒤로 갈수록 어렵습니다. 마지막 세 개는 [도전] 입니다. ★ 막히면 개념01~05 를 다시 보세요. 답은 연습문제_정답.js 에 있습니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();

const 답 = {};
```

문제 1 — 표 만들기

작업자 표를 만드는 CREATE TABLE 문을 문자열로 쓰세요.

작업자번호	정수
이름	글자
입사일	날짜

(제약은 아직 안 겁니다. 칸 이름과 타입만 맞으면 됩니다)

```
답.문제1 = null; // TODO: `CREATE TABLE 작업자 (...)` 를 문자열로
```

문제 2 — 타입 고르기

각 값을 담기에 알맞은 Postgres 타입을 고르세요. 보기: INT · BIGINT · NUMERIC(12,2) · REAL · TEXT · BOOLEAN · DATE · TIMESTAMPTZ

```
답.문제2 = {
  설비단가: null,      // TODO: 원 단위 금액. 1원도 틀리면 안 됩니다
  설비온도: null,      // TODO: 센서가 읽은 섭씨 온도. 소수점 한 자리
  가동여부: null,      // TODO: 돌고 있나 아닌가
  점검기록시각: null,  // TODO: 언제 점검했는지. 해외 지사와 같이 봅니다
};
```

문제 3 — 어느 타입이 정확한가

0.1 을 1,000번 더했을 때 정확히 100 이 나오는 타입은? "NUMERIC" 또는 "REAL" 중 하나를 문자열로 쓰세요.


```
답.문제3 = null; // TODO:
```

문제 4 — BIGINT 를 안전하게 꺼내기

번호창고 표에는 BIGINT 칸 **큰번호** 가 있습니다. 이 값을 `JSON.stringify` 해도 안 터지게 꺼내는 SELECT 문을 쓰세요.

· 결과 칸 이름은 **큰번호** 여야 합니다 · 값은 문자열로 나와야 합니다

```
답.문제4 = null; // TODO: `SELECT ... FROM 번호창고` 를 문자열로
```

문제 5 — 날짜를 하루 안 밀리게 꺼내기

일정 표의 DATE 칸 **마감일** 을 자바스크립트에서 하루도 안 밀리게 받으려면 어떻게 꺼내야 하나요? SELECT 문을 쓰세요. 결과 칸 이름은 **마감일** 입니다.

```
답.문제5 = null; // TODO: `SELECT ... FROM 일정` 을 문자열로
```

문제 6 — 표 고치기

이미 만들어진 **설비목록** 표가 있습니다. 칸은 **번호**, **이름** 두 개입니다. 여기에 아래 두 가지를 하는 문장을 **한 문자열**로 쓰세요. (세미콜론으로 이으세요)

① **도입일** 이라는 DATE 칸을 추가 ② **이름** 칸의 이름을 **설비명** 으로 변경

```
답.문제6 = null; // TODO: `ALTER TABLE ...; ALTER TABLE ...;`
```

문제 7 — 제약 걸기

점검 표를 만드세요. 아래 규칙이 **표 정의만으로** 지켜져야 합니다.

점검번호	정수, 기본키
설비번호	정수, 비울 수 없음
결과	글자, 비울 수 없음, '정상' / '주의' / '불량' 중 하나만
비고	글자, 비워도 됨

```
답.문제7 = null; // TODO: `CREATE TABLE 점검 (...)` 을 문자열로
```

문제 8 — 에러 코드

각 상황에서 나오는 SQLSTATE 코드를 문자열로 쓰세요. (예: "42601")

```
답.문제8 = {
  낫널위반: null, // TODO: NOT NULL 인 칸에 NULL 을 넣었을 때
  유니크위반: null, // TODO: UNIQUE 인 칸에 같은 값을 또 넣었을 때
  체크위반: null, // TODO: CHECK 조건에 안 맞는 값을 넣었을 때
};
```

문제 9 — DEFAULT

주문 표를 만드세요.

주문번호 정수, 기본키
상태 글자, 비울 수 없음, **안 적으면 '접수'** 가 들어가야 함
접수시각 시각, 비울 수 없음, **안 적으면 지금 시각**이 들어가야 함
* 해외에서도 같은 순간으로 보여야 합니다

답.문제9 = null; // TODO: `CREATE TABLE 주문 (...)` 을 문자열로

문제 10 — 기본키를 자동으로

알림 표를 만드세요.

알림번호 **BIGINT**, 자동으로 매겨짐, 직접 값을 넣을 수 없음, 기본키
내용 글자, 비울 수 없음

★ SERIAL 말고 요즘 표준 문법을 쓰세요.

답.문제10 = null; // TODO: `CREATE TABLE 알림 (...)` 을 문자열로

문제 11 — NULL 을 찾기

설비 표에서 담당자가 정해지지 않은 설비의 번호를 오름차순으로 뽑으세요. 결과 칸 이름은 설비번호 입니다.

답.문제11 = null; // TODO: `SELECT 설비번호 FROM 설비 WHERE ... ORDER BY 설비번호`

문제 12 — ★ NOT IN 을 고치기

아래 쿼리는 정비중 표에 NULL 이 한 줄 섞여 있어서 0건이 나옵니다.

```
SELECT 설비번호 FROM 설비
WHERE 설비번호 NOT IN (SELECT 설비번호 FROM 정비중)
ORDER BY 설비번호
```

NULL 이 있어도 제대로 나오게 고쳐서 쓰세요. 결과 칸 이름은 설비번호 입니다.

답.문제12 = null; // TODO:

문제 13 — [도전] 집계와 NULL

생산 표에는 수량 이 NULL 인 줄이 섞여 있습니다. NULL 을 0으로 쳐서 전체 줄 수로 나눈 평균을 구하세요. 결과 칸 이름은 평균 이고, 값은 정수(INT)여야 합니다.

★ 그냥 avg(수량) 을 쓰면 NULL 인 줄이 분모에서도 빠집니다.

```
답.문제13 = null; // TODO: `SELECT ... AS 평균 FROM 생산`
```

문제 14 — [도전] 요구사항대로 표 설계하기

아래 요구사항을 **전부** 표 정의에 담으세요. 애플리케이션 코드는 못 씁니다.

작업지시 · 표

· 지시번호는 BIGINT 로 자동으로 매겨지고, 직접 값을 넣을 수 없다. 기본키다 · 설비번호는 정수이고 반드시 있어야 한다 · 라인은 'A', 'B', 'C' 중 하나여야 하고 반드시 있어야 한다 · 지시일은 날짜이고, 안 적으면 오늘이 들어간다. 비울 수 없다 · 목표수량은 정수이고 0보다 커야 한다. 반드시 있어야 한다 · 완료수량은 정수이고 0 이상이어야 한다. 안 적으면 0 이 들어간다. 비울 수 없다 · 완료수량이 목표수량보다 클 수 없다 ← ★ 칸 두 개를 비교하는 CHECK · 같은 설비에 같은 날짜의 지시가 두 번 있으면 안 된다 ← ★ 두 칸을 묶은 UNIQUE · 상태 확인 제약의 이름은 **수량_확인** 으로 붙일 것 (완료수량 <= 목표수량 검사)

```
답.문제14 = null; // TODO: `CREATE TABLE 작업지시 (...)` 를 문자열로
```

문제 15 — [도전] 더러운 데이터를 정리하고 제약 걸기

옛날설비 표에는 라인이 'A','B','C' 가 아닌 줄이 섞여 있습니다. 그 줄들의 라인을 '**A**' 로 **고친 뒤**, **라인_확인** 이라는 이름의 CHECK 제약을 거세요.

· 라인이 NULL 인 줄도 'A' 로 고칩니다 · 제약: 라인은 'A','B','C' 중 하나 · 한 문자열에 세미콜론으로 이어서 쓰세요 ★ 줄 수는 그대로여야 합니다. 지우면 안 됩니다

```
답.문제15 = null; // TODO: `UPDATE ...; ALTER TABLE ...;`
```

↓↓↓ 아래는 채점 코드입니다. 고치지 마세요 ↓↓↓

```

await db.exec(`
  CREATE TABLE 번호창고 (큰번호 BIGINT);
  INSERT INTO 번호창고 VALUES (9007199254740993);

  CREATE TABLE 일정 (마감일 DATE);
  INSERT INTO 일정 VALUES ('2026-03-15');

  CREATE TABLE 설비목록 (번호 INT, 이름 TEXT);

  CREATE TABLE 설비 (설비번호 INT PRIMARY KEY, 담당자 TEXT);
  INSERT INTO 설비 VALUES (1, '김반장'), (2, NULL), (3, '이반장'), (4, NULL);

  CREATE TABLE 정비중 (설비번호 INT);
  INSERT INTO 정비중 VALUES (1), (NULL);

  CREATE TABLE 생산 (설비번호 INT, 수량 INT);
  INSERT INTO 생산 VALUES (1, 100), (2, NULL), (3, 200), (4, NULL);

  CREATE TABLE 옛날설비 (번호 INT, 라인 TEXT);
  INSERT INTO 옛날설비 VALUES (1, 'A'), (2, '999'), (3, NULL), (4, 'B'), (5, '');
`);

let 맞은수 = 0;

async function 채점(번호, 확인) {
  const 답안 = 답[`문제${번호}`];

  const 안풀었나 =
    답안 === null ||
    답안 === undefined ||
    (typeof 답안 === "object" && Object.values(답안).every((값) => 값 === null));

  if (안풀었나) {
    console.log(`문제 ${번호} -  아직 안 풀었습니다`);
    return;
  }

  try {
    const 통과 = await 확인(답안);
    if (통과) 맞은수 += 1;
    console.log(`문제 ${번호} - ${통과 ? " 정답" : " 틀렸습니다`);
  } catch (에러) {
    console.log(`문제 ${번호} -  에러 (${에러.code ?? 에러.name})`);
  }
}

```

답이 SQL 인 문제를 위해, 실행 뒤 되돌리는 도우미입니다.

```

async function 굴러보기(sql, 확인) {
  let 통과 = false;
  try {
    await db.transaction(async (tx) => {
      await tx.exec(sql);
      통과 = await 확인(tx);
      throw new Error("되돌리기");
    });
  } catch (에러) {
    if (에러.message !== "되돌리기") throw 에러;
  }
  return 통과;
}

async function 칸모양(tx, 표이름) {
  const 결과 = await tx.query(
    `SELECT column_name, data_type, is_nullable, column_default, is_identity
      FROM information_schema.columns WHERE table_name = $1 ORDER BY
ordinal_position`,
    [표이름],
  );
  return 결과.rows;
}

```

성공하면 그 줄을 그대로 둡니다. 뒤에서 중복 검사를 해야 하기 때문입니다.

```

async function 막히나(tx, sql) {
  await tx.query(`SAVEPOINT 시험`);
  try {
    await tx.query(sql);
    await tx.query(`RELEASE SAVEPOINT 시험`);
    return false;
  } catch {
    await tx.query(`ROLLBACK TO SAVEPOINT 시험`);
    return true;
  }
}

console.log("===== 02단원 연습문제 채점 =====");

await 채점(1, (답안) =>
  쿨려보기(답안, async (tx) => {
    const 칸 = await 칸모양(tx, "작업자");
    return (
      칸.length === 3 &&
      칸[0].column_name === "작업자번호" && 칸[0].data_type === "integer" &&
      칸[1].column_name === "이름" && ["text", "character varying"].includes(칸
[1].data_type) &&
      칸[2].column_name === "입사일" && 칸[2].data_type === "date"
    );
  }),
);

await 채점(2, (답안) =>
  답안.설비단가 === "NUMERIC(12,2)" &&
  답안.설비온도 === "REAL" &&
  답안.가동여부 === "BOOLEAN" &&
  답안.점검기록시각 === "TIMESTAMPZ",
);

await 채점(3, (답안) => 답안 === "NUMERIC");

await 채점(4, async (답안) => {

```

```

const 결과 = await db.query(답안);
const 값 = 결과.rows[0].큰번호;
return typeof 값 === "string" && 값 === "9007199254740993" &&
JSON.stringify(결과.rows).length > 0;
});

await 채점(5, async (답안) => {
  const 값 = (await db.query(답안)).rows[0].마감일;
  return 값 === "2026-03-15";
});

await 채점(6, (답안) =>
  둘러보기(답안, async (tx) => {
    const 칸 = await 칸모양(tx, "설비목록");
    const 이름들 = 칸.map((하나) => 하나.column_name);
    return (
      이름들.includes("설비명") && !이름들.includes("이름") &&
      칸.some((하나) => 하나.column_name === "도입일" && 하나.data_type === "date")
    );
  }),
);

await 채점(7, (답안) =>
  둘러보기(답안, async (tx) => {
    const 정상 = !(await 막히나(tx, `INSERT INTO 점검 VALUES (1, 10, '정상',
NULL)`));
    const 결과오타 = await 막히나(tx, `INSERT INTO 점검 VALUES (2, 10, '가동',
NULL)`);
    const 설비널 = await 막히나(tx, `INSERT INTO 점검 VALUES (3, NULL, '정상',
NULL)`);
    const 번호중복 = await 막히나(tx, `INSERT INTO 점검 VALUES (1, 11, '주의',
NULL)`);
    return 정상 && 결과오타 && 설비널 && 번호중복;
  }),
);

await 채점(8, (답안) =>
  답안.낫날위반 === "23502" && 답안.유니크위반 === "23505" && 답안.체크위반 ===
"23514",
);

await 채점(9, (답안) =>

```

```

    쿨러보기(답안, async (tx) => {
      await tx.query(`INSERT INTO 주문 (주문번호) VALUES (1)`);
      const 줄 = (await tx.query(`SELECT 상태, 접수시각 FROM 주문 WHERE 주문번호 =
1`)).rows[0];
      const 칸 = await 칸모양(tx, "주문");
      const 시각칸 = 칸.find((하나) => 하나.column_name === "접수시각");
      return (
        줄.상태 === "접수" &&
        줄.접수시각 instanceof Date &&
        시각칸.data_type === "timestamp with time zone" &&
        시각칸.is_nullable === "NO"
      );
    }),
  )),
);

await 채점(10, (답안) =>
  쿨러보기(답안, async (tx) => {
    const 칸 = await 칸모양(tx, "알림");
    const 번호칸 = 칸.find((하나) => 하나.column_name === "알림번호");
    if (!번호칸 || 번호칸.data_type !== "bigint" || 번호칸.is_identity !== "YES")
    return false;
    await tx.query(`INSERT INTO 알림 (내용) VALUES ('첫 알림')`);
    const 자동 = (await tx.query(`SELECT 알림번호 FROM 알림`)).rows[0].알림번호;
    const 직접막힘 = await 막히나(tx, `INSERT INTO 알림 VALUES (99, '몰래')`);
    return Number(자동) === 1 && 직접막힘;
  )),
);

await 채점(11, async (답안) => {
  const 줄들 = (await db.query(답안)).rows.map((줄) => 줄.설비번호);
  return JSON.stringify(줄들) === JSON.stringify([2, 4]);
});

await 채점(12, async (답안) => {
  const 줄들 = (await db.query(답안)).rows.map((줄) => 줄.설비번호);
  return JSON.stringify(줄들) === JSON.stringify([2, 3, 4]);
});

```



```

await 채점(13, async (답안) => {
  const 값 = (await db.query(답안)).rows[0].평균;
  return Number(값) === 75;
});

await 채점(14, (답안) =>
  둘러보기(답안, async (tx) => {
    const 칸 = await 칸모양(tx, "작업지시");
    const 지시번호 = 칸.find((하나) => 하나.column_name === "지시번호");
    if (!지시번호 || 지시번호.data_type !== "bigint" || 지시번호.is_identity !==
      "YES") return false;

    const 정상 = !(await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (1,'A',100)`));
    const 라인틀림 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (2,'999',100)`);
    const 목표영 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (3,'A',0)`);
    const 완료초과 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량,
완료수량) VALUES (4,'B',10,11)`);
    const 중복 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (1,'A',50)`);

    const 첫줄 = (await tx.query(`SELECT 완료수량, 지시일 FROM 작업지시 WHERE
설비번호 = 1`)).rows[0];
    const 기본값 = 첫줄 && 첫줄.완료수량 === 0 && 첫줄.지시일 instanceof Date;

    const 이름불임 = (await tx.query(
      `SELECT count(*)::int AS 개수 FROM pg_constraint
      WHERE conrelid = '작업지시'::regclass AND conname = '수량_확인'`,
    )).rows[0].개수 === 1;

    return 정상 && 라인틀림 && 목표영 && 완료초과 && 중복 && 기본값 && 이름불임;
  }),
);

await 채점(15, (답안) =>
  둘러보기(답안, async (tx) => {
    const 줄수 = (await tx.query(`SELECT count(*)::int AS 개수 FROM 옛날설비
`)).rows[0].개수;
    const 나쁜것 = (await tx.query(
      `SELECT count(*)::int AS 개수 FROM 옛날설비 WHERE 라인 IS NULL OR 라인 NOT IN
('A','B','C')`,
    )).rows[0].개수;
  });

```

```
const 제약있나 = (await tx.query(
  `SELECT count(*)::int AS 개수 FROM pg_constraint
    WHERE conrelid = '옛날설비'::regclass AND conname = '라인_확인'`,
)).rows[0].개수 === 1;
const 막히나결과 = await 막히나(tx, `INSERT INTO 옛날설비 VALUES (9, 'Z')`);
return 줄수 === 5 && 나쁜것 === 0 && 제약있나 && 막히나결과;
}),
);

console.log(`===== 15문제 중 ${맞은수}개 정답 =====`);
```

```
출력      ===== 02단원 연습문제 채점 =====
문제 1 - ☐ 아직 안 풀었습니다
문제 2 - ☐ 아직 안 풀었습니다
문제 3 - ☐ 아직 안 풀었습니다
문제 4 - ☐ 아직 안 풀었습니다
문제 5 - ☐ 아직 안 풀었습니다
문제 6 - ☐ 아직 안 풀었습니다
문제 7 - ☐ 아직 안 풀었습니다
문제 8 - ☐ 아직 안 풀었습니다
문제 9 - ☐ 아직 안 풀었습니다
문제 10 - ☐ 아직 안 풀었습니다
문제 11 - ☐ 아직 안 풀었습니다
문제 12 - ☐ 아직 안 풀었습니다
문제 13 - ☐ 아직 안 풀었습니다
문제 14 - ☐ 아직 안 풀었습니다
문제 15 - ☐ 아직 안 풀었습니다
===== 15문제 중 0개 정답 =====
```

★ 위 // 출력: 은 아무것도 안 푼 상태의 결과입니다. 문제를 풀면 ☒ 로 바뀌면서 이 주석과 달라집니다. 정상입니다.

```
await db.close();
```

넣고 읽고 고치고 지우기

OPEN 03_넣고_읽고_고치고_지우기

```
await db.exec(`
CREATE TABLE 설비 (
id          SERIAL PRIMARY KEY,
이름        TEXT NOT NULL UNIQUE,
```

- 01 넣기
- 02 읽기
- 03 SQL 인젝션을 직접 뚫어 봅니다
- 04 고치고 지우기
- 05 한 번에 여러 건 다루기
- 연습문제

넣기

RUN node 개념01_넣기.js

★ PGlite 는 시작에 1초쯤 걸립니다. 잠깐 멈춰 있어도 정상입니다.

02단원에서 표를 만들었습니다. 이제 그 표에 데이터를 넣습니다.

데이터를 다루는 명령은 네 개뿐입니다.

```
INSERT  넣기
SELECT  읽기
UPDATE  고치기
DELETE  지우기
```

이 네 개를 묶어서 CRUD 라고 부릅니다. (Create Read Update Delete) 회사에서 하는 일의 90% 는 이 넷입니다. 03단원 전체가 이 넷입니다.

이 파일은 그중 **넣기** 하나만 봅니다. “값을 적어 넣는 것뿐인데 뭐가 더 있나” 싶겠지만, 넣기에서만 실무 사고가 세 가지 납니다. 전부 직접 재현합니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

넣을 표를 만듭니다

섹션 1

02단원에서 한 것과 같습니다. 여기서는 빠르게 지나갑니다.

★ 표 이름과 칸 이름을 한글로 씁니다. 읽기 쉬우라고 그렇게 합니다. 회사에서는 영어를 많이 씁니다. (equipment, line_code, status) 한글로 써도 Postgres 는 잘 돌아갑니다. 큰따옴표가 필요한 경우만 조심하면 됩니다.

```
await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    이름        TEXT NOT NULL UNIQUE,
    라인        TEXT NOT NULL,
    상태        TEXT NOT NULL DEFAULT '정지',
    도입연도    INT,
    등록일      DATE NOT NULL DEFAULT CURRENT_DATE
  );
`);
```

설비가 몇 건 있는지 세어 봅니다. 아직 아무것도 안 넣었습니다.

```
const 처음건수 = await db.query("SELECT count(*)::int AS 건수 FROM 설비");

console.log("처음 건수:", 처음건수.rows[0].건수);
```

출력 처음 건수: 0

★ count(*) 는 그냥 두면 BIGINT 라서 자바스크립트에 **bigint** 로 옵니다. 그러면 `JSON.stringify` 가 터집니다. (02단원에서 다뤘습니다) 그래서 `::int` 를 붙여 number 로 받았습니다. 앞으로도 계속 이렇게 씁니다.

한 건 넣기

섹션 2

가장 기본 형태입니다.

INSERT INTO 표이름 (칸1, 칸2) VALUES (값1, 값2)

칸 목록과 값 목록의 **순서와 개수가 맞아야** 합니다.

```
const 넣기1 = await db.query(`
  INSERT INTO 설비 (이름, 라인, 상태, 도입연도)
  VALUES ('컨베이어 1호', 'A', '가동', 2019)
`);

console.log("command:", 넣기1.command, "/ 넣은 줄 수:", 넣기1.affectedRows);
```

출력 command: INSERT / 넣은 줄 수: 1

★ affectedRows 는 “이 명령이 건드린 줄 수” 입니다. INSERT 면 넣은 수, UPDATE 면 고친 수, DELETE 면 지운 수입니다. 실무에서 이 값을 확인하지 않아서 나는 사고가 아주 많습니다. “지웠습니다” 라고 화면에 띄웠는데 실제로는 0건 지운 경우입니다.

괄호를 쉼표로 이으면 여러 줄을 한 번에 넣습니다.

```
const 넣기2 = await db.query(`
  INSERT INTO 설비 (이름, 라인, 상태, 도입연도)
  VALUES
    ('컨베이어 2호', 'A', '정지', 2021),
    ('프레스 1호', 'B', '가동', 2018),
    ('프레스 2호', 'B', '점검', 2023),
    ('용접로봇 1호', 'C', '가동', 2022)
`);

console.log("한 번에 넣은 줄 수:", 넣기2.affectedRows);
```

출력 한 번에 넣은 줄 수: 4

★★ 이걸 단순히 손이 덜 가는 게 아닙니다. **훨씬 빠릅니다**. 한 건씩 4번 보내면 서버와 4번 왕복합니다. 한 번에 보내면 1번입니다. 10만 건이면 이 차이가 몇십 배가 됩니다. 개념05 에서 진짜로 짭니다.

★ 그리고 한 문장 안의 여러 줄은 **전부 되거나 전부 안 됩니다**. 중간에 하나가 규칙을 어기면 앞의 것도 안 들어갑니다. 한 문장은 그 자체로 하나의 트랜잭션이기 때문입니다. (07단원에서 자세히)

```
const 넣기실패 = await db
  .query(`
    INSERT INTO 설비 (이름, 라인) VALUES
      ('신규설비 A', 'A'),
      ('컨베이어 1호', 'A')
  `)
  .catch((에러) => 에러);

console.log("두 줄 중 하나가 중복이면:", 넣기실패.code);
```

출력 두 줄 중 하나가 중복이면: 23505

```
const 신규있나 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 이름 =
'신규설비 A'");

console.log("멀쩡했던 첫 줄은 들어갔나:", 신규있나.rows[0].건수 > 0);
```

출력 멀쩡했던 첫 줄은 들어갔나: false

★★ 첫 줄은 아무 문제가 없었는데도 안 들어갔습니다. “일부만 들어가는” 일은 없습니다. 이게 데이터베이스가 파일과 다른 점입니다.

★★ 칸 이름을 생략하면 언젠가 터집니다

섹션 4

칸 목록을 빼고 이렇게도 쓸 수 있습니다.

```
INSERT INTO 설비 VALUES (10, '나사기 1호', 'A', '가동', 2020, CURRENT_DATE)
```

짧아서 좋아 보입니다. 실제로 많은 사람이 이렇게 씁니다. 그런데 이건 **표의 칸 순서에 코드가 묶인다**는 뜻입니다.

표의 칸 순서는 바뀝니다. 반드시 바뀝니다. 진짜로 재현해 보겠습니다.

```
await db.exec(`
  CREATE TABLE 점검기록 (
    id      SERIAL PRIMARY KEY,
    설비명  TEXT NOT NULL,
    점검자  TEXT NOT NULL,
    결과    TEXT NOT NULL
  );
`);
```

오늘 짠 코드입니다. 잘 돌아갑니다.

```
await db.query("INSERT INTO 점검기록 VALUES (1, '컨베이어 1호', '김반장', '정상')");

const 처음줄 = await db.query("SELECT 설비명, 점검자, 결과 FROM 점검기록 WHERE id = 1");

console.log("잘 들어갔습니다:", JSON.stringify(처음줄.rows[0]));
```

출력 잘 들어갔습니다: {"설비명":"컨베이어 1호","점검자":"김반장","결과":"정상"}

6개월 뒤. 옆 팀이 점검자 칸을 지웠다 다시 만듭니다. 흔한 일입니다.

```
await db.exec(`
  ALTER TABLE 점검기록 DROP COLUMN 점검자;
  ALTER TABLE 점검기록 ADD COLUMN 점검자 TEXT;
`);
```

★ Postgres 에서 칸을 지웠다 다시 만들면 **맨 뒤로 갑니다**. 이제 표의 칸 순서는 id · 설비명 · 결과 · 점검자 입니다.

아무도 위의 INSERT 코드를 안 고쳤습니다. 그대로 돌립니다.

```
await db.query("INSERT INTO 점검기록 VALUES (2, '프레스 1호', '이기사', '주의')");

const 두번째줄 = await db.query("SELECT 설비명, 점검자, 결과 FROM 점검기록 WHERE id = 2");

console.log("6개월 뒤:", JSON.stringify(두번째줄.rows[0]));
```

출력 6개월 뒤: {"설비명":"프레스 1호","점검자":"주의","결과":"이기사"}

```
console.log("에러가 났나:", false);
```

출력 에러가 났나: false

★★★ 점검자가 '주의' 가 되고, 결과가 '이기사' 가 됐습니다.

에러도 로그도 없습니다. 누가 화면을 보기 전까지 아무도 모릅니다. 왜 에러가 안 나는가: 둘 다 TEXT 칸이라 값이 그냥 들어갑니다. 타입이 달랐으면 에러가 났을 텐데, 그건 **운이 좋은 경우** 입니다.

★ 그래서 규칙은 하나입니다. **INSERT 에는 언제나 칸 이름을 적으세요.** 글자 몇 개 더 치는 대가로 이 사고를 통째로 없앱니다.

칸 이름을 적었으면 어땠을까요. 순서를 뒤죽박죽으로 써 봅니다.

```
await db.query(`
  INSERT INTO 점검기록 (결과, 설비명, 점검자, id)
  VALUES ('불량', '용접로봇 1호', '박주임', 3)
`);

const 세번째줄 = await db.query("SELECT 설비명, 점검자, 결과 FROM 점검기록 WHERE id = 3");

console.log("칸 이름을 적으면:", JSON.stringify(세번째줄.rows[0]));
```

출력 칸 이름을 적으면: {"설비명":"용접로봇 1호","점검자":"박주임","결과":"불량"}

순서를 어떻게 쓰든 제자리로 갑니다. 칸 순서가 바뀌어도 안 깨집니다.

안 적은 칸에는 무엇이 들어가나

섹션 5

설비 표에는 여섯 칸이 있는데 두 칸만 적어 보겠습니다.


```
await db.query("INSERT INTO 설비 (이름, 라인) VALUES ('나사기 1호', 'A')");

const 나사기 = await db.query("SELECT 이름, 상태, 도입연도, 등록일 FROM 설비 WHERE
이름 = '나사기 1호'");
const 나사기줄 = 나사기.rows[0];

console.log("상태:", 나사기줄.상태, "/ 도입연도:", 나사기줄.도입연도);
```

출력 상태: 정지 / 도입연도: null

```
console.log("등록일이 오늘인가:", 나사기줄.등록일 instanceof Date);
```

출력 등록일이 오늘인가: true

★ 안 적은 칸의 값은 이렇게 정해집니다.

DEFAULT 가 있으면	→ 그 값	(상태 → '정지')
DEFAULT 가 없으면	→ NULL	(도입연도 → null)
NOT NULL 인데 DEFAULT 없음 → 에러 23502		

★ DATE 는 자바스크립트에 **Date 객체**로 옵니다. 글자가 아닙니다. 그리고 UTC 자정으로 옵니다. 시간대 함정이 있습니다. (02단원에서 다뤘습니다)

DEFAULT 라는 낱말을 값 자리에 직접 쓸 수도 있습니다.

```
await db.query("INSERT INTO 설비 (이름, 라인, 상태) VALUES ('나사기 2호', 'A',
DEFAULT)");

const 나사기2 = await db.query("SELECT 상태 FROM 설비 WHERE 이름 = '나사기 2호'");

console.log("DEFAULT 라고 쓰면:", 나사기2.rows[0].상태);
```

출력 DEFAULT 라고 쓰면: 정지

★★ 자바스크립트에서 값이 없을 때를 조심하세요. 파라미터로 `undefined` 나 `null` 을 넘기면 **NULL 이 들어갑니다**. DEFAULT 가 아닙니다. 기본값을 쓰고 싶으면 `COALESCE($1, '정지')` 처럼 감싸세요.

```
const 상태값 = null;

await db.query("INSERT INTO 설비 (이름, 라인, 상태) VALUES ($1, 'A', COALESCE($2, '정지'))", [
  "나사기 3호",
  상태값,
]);

const 나사기3 = await db.query("SELECT 상태 FROM 설비 WHERE 이름 = '나사기 3호'");

console.log("COALESCE 로 기본값:", 나사기3.rows[0].상태);
```

출력 COALESCE 로 기본값: 정지

★ RETURNING — 방금 넣은 것을 바로 돌려받기

섹션 6

새 설비를 넣고, 그 설비의 id 로 점검기록을 만들어야 한다고 해 봅시다. id 는 SERIAL 이라 데이터베이스가 정합니다. 넣기 전에는 모릅니다.

다른 데이터베이스에서는 이렇게 합니다.

① INSERT 한다 ② SELECT 로 방금 넣은 것을 다시 찾는다 ← 왕복 2번, 그리고 위험합니다

★ ② 가 왜 위험한가: “방금 넣은 것” 을 찾는 조건이 애매합니다. 이름으로 찾으면 동명이인이 있고, 최대 id 로 찾으면 그 사이에 남이 넣습니다.

Postgres 는 **RETURNING** 한 줄로 끝냅니다.

```
const 새설비 = await db.query(
  `INSERT INTO 설비 (이름, 라인, 상태, 도입연도)
  VALUES ($1, $2, $3, $4)
  RETURNING id, 이름, 등록일`,
  ["3D 프린터 1호", "C", "가동", 2026],
);

console.log("RETURNING 으로 받은 것:", 새설비.rows[0].id, 새설비.rows[0].이름);
```

출력 RETURNING 으로 받은 것: 11 3D 프린터 1호

★ 왕복이 한 번입니다. 그리고 “방금 넣은 그 줄” 이 확실합니다. 남이 동시에 넣어도 내 줄만 돌아옵니다.

RETURNING * 로 전부 받을 수도 있습니다.

```
const 전부받기 = await db.query(`
  INSERT INTO 설비 (이름, 라인) VALUES ('레이저 커터 1호', 'C')
  RETURNING *
`);

console.log("돌려받은 칸들:", Object.keys(전부받기.rows[0]).join(", "));
```

출력 돌려받은 칸들: id, 이름, 라인, 상태, 도입연도, 등록일

여러 줄을 넣으면 여러 줄이 돌아옵니다.

```
const 여러개 = await db.query(`
  INSERT INTO 설비 (이름, 라인) VALUES ('검사기 1호', 'A'), ('검사기 2호', 'B')
  RETURNING id, 이름
`);

console.log("여러 줄 RETURNING:", 여러개.rows.map((줄) => `${줄.id}:${줄.이름}`).join(" · "));
```

출력 여러 줄 RETURNING: 13:검사기 1호 · 14:검사기 2호

RETURNING 안에서 계산도 됩니다.

```
const 계산해서 = await db.query(`
  INSERT INTO 설비 (이름, 라인, 도입연도) VALUES ('포장기 1호', 'A', 2015)
  RETURNING 이름, 2026 - 도입연도 AS 사용연수
`);

console.log("계산해서 돌려받기:", JSON.stringify(계산해서.rows[0]));
```

출력 계산해서 돌려받기: {"이름":"포장기 1호","사용연수":11}

MySQL 은 여기가 다릅니다

· MySQL 8.4 에는 RETURNING 이 없습니다. INSERT 뒤에 `LAST_INSERT_ID()` 를 따로 부릅니다. 여러 줄을 넣으면 `LAST_INSERT_ID()` 는 첫 줄의 id 만 줍니다. 파라미터가 \$1 이 아니라 ? 입니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

INSERT ... SELECT — 다른 표에서 가져와 넣기

섹션 7

값을 손으로 적는 대신, 다른 표를 읽어 온 결과를 그대로 넣을 수 있습니다.

INSERT INTO 받을표 (칸들) SELECT 칸들 FROM 준표 WHERE 조건

★ VALUES 자리에 SELECT 가 들어갈 뿐입니다. 나머지는 같습니다.

```
await db.exec(`
  CREATE TABLE 점검대상 (
    id      SERIAL PRIMARY KEY,
    설비명  TEXT NOT NULL,
    라인    TEXT NOT NULL,
    사유    TEXT NOT NULL
  );
`);

const 옮기기 = await db.query(`
  INSERT INTO 점검대상 (설비명, 라인, 사유)
  SELECT 이름, 라인, '도입 5년 초과'
  FROM 설비
  WHERE 도입연도 IS NOT NULL AND 도입연도 <= 2021
`);

console.log("점검대상으로 옮긴 줄 수:", 옮기기.affectedRows);
```

출력 점검대상으로 옮긴 줄 수: 4

```
const 옮긴것 = await db.query("SELECT 설비명 FROM 점검대상 ORDER BY 설비명");

console.log("옮긴 설비:", 옮긴것.rows.map((줄) => 줄.설비명).join(" · "));
```

출력 옮긴 설비: 컨베이어 1호 · 컨베이어 2호 · 포장기 1호 · 프레스 1호

★ 이게 왜 좋은가: 데이터가 **서버 안에서만** 움직입니다. 자바스크립트로 가져와서 다시 보내면 네트워크를 두 번 탑니다. 4건이면 상관없지만 400만 건이면 완전히 다른 이야기입니다.

백업표 만들기, 오래된 기록 옮기기, 집계표 채우기 — 전부 이걸로 합니다.

넣기가 실패하는 여섯 가지와 에러 코드

섹션 8

★ 에러 코드는 **다섯 글자짜리 표준 번호** 입니다. SQLSTATE 라고 부릅니다. 메시지는 데이터베이스 버전이나 언어 설정에 따라 바뀝니다. 코드는 안 바뀝니다. **코드로 분기하세요.**

```
await db.exec(`
  CREATE TABLE 생산실적 (
    id      SERIAL PRIMARY KEY,
    설비id  INT NOT NULL REFERENCES 설비(id),
    수량    INT NOT NULL,
    생산일  DATE NOT NULL
  );
`);
```

```

async function 넣어보기(설명, sql) {
  try {
    await db.query(sql);
    return `${설명} - 성공`;
  } catch (에러) {
    return `${설명} - ${에러.code} · ${에러.message}`;
  }
}

const 실패들 = [
  await 넣어보기("이미 있는 이름", "INSERT INTO 설비 (이름, 라인) VALUES ('컨베이어 1호', 'A')"),
  await 넣어보기("NOT NULL 칸을 뺐", "INSERT INTO 설비 (이름) VALUES ('라벨기 1호')"),
  await 넣어보기("없는 설비를 가리킴", "INSERT INTO 생산실적 (설비id, 수량, 생산일) VALUES (9999, 10, DATE '2026-08-01')"),
  await 넣어보기("숫자 칸에 글자", "INSERT INTO 설비 (이름, 라인, 도입연도) VALUES ('라벨기 2호', 'A', '작년')"),
  await 넣어보기("없는 칸 이름", "INSERT INTO 설비 (이름, 라인, 제조사) VALUES ('라벨기 3호', 'A', '한국기계')"),
  await 넣어보기("없는 표", "INSERT INTO 설비목록 (이름) VALUES ('라벨기 4호')"),
];

for (const 한줄 of 실패들) {
  console.log(`· ${한줄}`);
}

```

출력

- 이미 있는 이름 - 23505 · duplicate key value violates unique constraint "설비_이름_key"
- NOT NULL 칸을 뺐 - 23502 · null value in column "라인" of relation "설비" violates not-null constraint
- 없는 설비를 가리킴 - 23503 · insert or update on table "생산실적" violates foreign key constraint "생산실적_설비id_fkey"
- 숫자 칸에 글자 - 22P02 · invalid input syntax for type integer: "작년"
- 없는 칸 이름 - 42703 · column "제조사" of relation "설비" does not exist
- 없는 표 - 42P01 · relation "설비목록" does not exist

★★ 23xxx 는 **사용자에게 보여 줄 메시지**로 바뀌어야 하고, 42xxx 와 22P02 는 **개발자에게 알려야 할 버그**입니다. 전부 “오류가 발생했습니다” 로 뭉개면 아무도 못 고칩니다.

실제로 그렇게 나눠 보겠습니다.

```

function 사람말로(코드) {
  const 사용자에게 = {
    "23505": "이미 등록된 설비입니다",
    "23502": "필수 항목이 비어 있습니다",

```

```
};

return 사용자에게[코드] ?? "처리 중 문제가 생겼습니다 (관리자에게 알려짐)";
}

const 중복오류 = await db
  .query("INSERT INTO 설비 (이름, 라인) VALUES ('컨베이어 1호', 'A')")
  .catch((에러) => 에러);

console.log("화면에 띄울 말:", 사람말로(중복오류.code));
```

출력 화면에 띄울 말: 이미 등록된 설비입니다

```
console.log("개발자용 코드:", 중복오류.code);
```

출력 개발자용 코드: 23505

★ 어느 제약이 걸렸는지도 알 수 있습니다. 제약 이름이 에러 객체에 들어 있습니다.

```
console.log("걸린 제약 이름:", 중복오류.constraint);
```

출력 걸린 제약 이름: 설비_이름_key

제약이 여러 개인 표에서는 이게 중요합니다. "이름이 중복" 인지 "사번이 중복" 인지 이걸로 구분합니다.

★ id 가 중간중간 비는 이유

섹션 9

```
const 번호들 = await db.query("SELECT id, 이름 FROM 설비 ORDER BY id");

console.log("id 목록:", 번호들.rows.map((줄) => 줄.id).join(", "));
```

출력 id 목록: 1, 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 14, 15

```
console.log("설비는", 번호들.rows.length, "대인데 마지막 id 는",
번호들.rows.at(-1).id);
```

출력 설비는 13 대인데 마지막 id 는 15

★★ 번호가 중간중간 비어 있습니다. 건수보다 마지막 id 가 큼니다.

실패한 INSERT 도 번호를 하나 가져다 씁니다. 위에서 23505-23502 로 실패한 시도들이 번호를 먹고 사라졌습니다. 되돌려주지 않는 이유는, 되돌리려면 번호를 받을 때마다 줄을 세워야 하기 때문입니다. 그러면 여러 사람이 동시에 넣을 때 거기서 다 막힙니다.

★★★ 그래서 SERIAL 번호에 대해 이렇게 알아 두세요.

- 겹치지 않는다 - 보장합니다
- 커진다 - 보장합니다
- 이어진다 - **보장하지 않습니다**

“오늘 몇 건 들어왔나” 를 `max(id)` - 어제`max(id)` 로 세면 틀립니다. 세려면 `count(*)` 를 쓰세요.

정리 — 넣기

쓰는 법 무엇을 하나

INSERT INTO 표 (칸들) VALUES (...) 한 건 넣기 VALUES (...), (...), (...) 여러 건을 한 번에 (훨씬 빠름)
칸 이름 생략 ★★★★★ 쓰지 마세요. 조용히 어긋납니다 안 적은 칸 DEFAULT → 그 값 / 없으면 NULL
DEFAULT 키워드 기본값을 명시적으로 RETURNING id 방금 넣은 것을 왕복 한 번에 받기 INSERT ...
SELECT 다른 표에서 가져와 넣기

실패 코드 뜻 누구 잘못인가

23505 UNIQUE 중복	사용자
23502 NOT NULL 위반	사용자
23503 외래키 위반	사용자
22P02 타입이 안 맞음	개발자 (검증 누락)
42703 없는 칸	개발자 (배포 사고)
42P01 없는 표	개발자 (배포 사고)

★ 한 문장 안의 여러 줄은 전부 되거나 전부 안 됩니다. 일부만 들어가지 않습니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 4 의 점검기록 표에서, 칸을 하나 더 지웠다 만들어 보세요. (예: 결과 칸) 그 뒤 칸 이름 없는 INSERT 를 또 하면 이번에는 어떤 값이 어디로 갈까요? 먼저 예상하고 돌려 보세요.

▫ 직접 해보기 2

점검기록에 결과 칸을 INT 로 바꿔서 다시 만들어 보세요. 그러면 칸 이름 없는 INSERT 가 에러를 낼까요? (힌트: 22P02. 타입이 다르면 오히려 다행입니다)

▫ 직접 해보기 3

RETURNING 없이 “방금 넣은 설비의 id” 를 구해 보세요. SELECT max(id) FROM 설비 로 하면 될까요? 남이 동시에 넣으면 어떤 값이 나올지 생각해 보세요.

▹ 직접 해보기 4

INSERT ... SELECT 로 라인 C 설비만 점검대상에 넣어 보세요. 사유는 '라인 C 정기점검' 으로 하세요.

▹ 직접 해보기 5

설비 표에 도입연도가 NULL 인 줄이 몇 개인지 세어 보세요. WHERE 도입연도 = NULL 로 하면 왜 0 이 나올까요? (개념02 에서 다룹니다)

▹ 직접 해보기 6

섹션 8 의 넣어보기() 에 여섯 번째 실패를 추가해 보세요. 수량 칸에 음수를 넣으면 어떻게 되나요? CHECK (수량 >= 0) 을 붙이면 어떤 코드가 나오나요? (힌트: 23514)

자주 하는 실수

이런 실수를 합니다

INSERT 에 칸 이름을 안 적음

섹션 4 에서 본 그대로입니다. 오늘은 되고 6개월 뒤에 조용히 어긋납니다. 코드 리뷰에서 이것만은 반드시 잡으세요.

이런 실수를 합니다

affectedRows 를 안 봄

"저장했습니다" 를 띄우기 전에 affectedRows 를 확인하세요. 0 이면 아무것도 안 들어간 것입니다.

이런 실수를 합니다

id 를 직접 정해서 넣음

INSERT INTO 설비 (id, 이름, 라인) VALUES (100, ...) 처럼 쓰면 SERIAL 의 다음 번호는 그대로 1, 2, 3... 입니다. 나중에 100번에 도달하는 순간 23505 가 터집니다. id 는 데이터베이스에 맡기세요.

이런 실수를 합니다

여러 건을 for 문으로 한 건씩 넣음

10만 건이면 왕복 10만 번입니다. 개념05 에서 몇 배인지 잹니다. VALUES 를 묶거나 INSERT ... SELECT 를 쓰세요.

이런 실수를 합니다

에러 메시지로 분기함

if (에러.message.includes("duplicate")) 처럼 쓰면 서버 언어 설정이 바뀌는 순간 전부 깨집니다. e.code 로 분기하세요. 코드는 표준이라 안 바뀝니다.

이런 실수를 합니다

문자열을 이어 붙여서 INSERT 문을 만들

`INSERT INTO 설비 (이름) VALUES ('${입력값}')` — 이게 SQL 인젝션입니다. 이름에 따옴표만 들어가도 문장이 깨지고, 나쁜 마음을 먹으면 표가 사라집니다. 개념03 에서 직접 뚫어 봅니다.

```
await db.close();
```

읽기

RUN node 개념02_읽기.js

★ 이 파일은 10만 건을 만들어 놓고 잡니다. 3초쯤 걸립니다.

개념01 에서 넣었습니다. 이제 읽습니다. 실무에서 쓰는 SQL 의 열에 아홉은 SELECT 입니다.

읽기는 네 가지를 정하는 일입니다. ① 어떤 칸을 (SELECT) ② 어떤 줄을 (WHERE) ③ 어떤 순서로 (ORDER BY) ④ 몇 개만 (LIMIT / OFFSET)

★★ ④ 에서 실무 사고 두 개를 재현합니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

읽을 것을 만들어 둡니다

섹션 0

```
await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    이름        TEXT NOT NULL,
    라인        TEXT NOT NULL,
    상태        TEXT NOT NULL,
    담당자      TEXT,
    도입연도    INT,
    시간당생산량 INT,
    비고        TEXT
  );

  INSERT INTO 설비 (이름, 라인, 상태, 담당자, 도입연도, 시간당생산량, 비고) VALUES
    ('컨베이어 1호', 'A', '가동', '김반장', 2019, 1200, '가동률 95% 이상 유지'),
    ('컨베이어 2호', 'A', '정지', '이기사', 2021, 1100, NULL),
    ('프레스 1호', 'B', '가동', NULL, 2018, 300, '소음 심함'),
    ('프레스 2호', 'B', '점검', '김반장', 2023, 340, NULL),
    ('용접로봇 1호', 'C', '가동', '박주임', 2022, 800, '2026년 교체 예정'),
    ('CNC 선반 1호', 'C', '고장', NULL, 2017, 150, NULL),
    ('cnc 선반 2호', 'C', '가동', '최반장', 2024, 180, NULL),
    ('포장기 1호', 'A', '가동', '이기사', NULL, 900, '불량률 0.95 수준');
`);
```

10만 건짜리 점검기록도 만듭니다. 뒤에서 쪽 나누기를 쥔 때 씁니다. ★ generate_series 는 “서버 안에서 1부터 N 까지 만들어 주는” 함수입니다. 시험 데이터를 만들 때 계속 씁니다. 개념05 에서 제대로 다룹니다.

```
await db.exec(`
  CREATE TABLE 점검기록 (
    id SERIAL PRIMARY KEY, 설비명 TEXT NOT NULL, 라인 TEXT NOT NULL,
    결과 TEXT NOT NULL, 점검일 DATE NOT NULL
  );

  INSERT INTO 점검기록 (설비명, 라인, 결과, 점검일)
  SELECT '설비' || (i % 500 + 1),
    (ARRAY['A','B','C'])[i % 3 + 1],
    (ARRAY['정상','주의','불량'])[i % 3 + 1],
    DATE '2026-01-01' + (i % 200)
  FROM generate_series(1, 100000) AS i;
`);

const 준비 = await db.query("SELECT count(*)::int AS 건수 FROM 점검기록");

console.log("점검기록 건수:", 준비.rows[0].건수);
```

출력 점검기록 건수: 100000

칸 고르기, 그리고 ★ SELECT * 를 쓰면 안 되는 이유

섹션 1

필요한 칸만 이름으로 적습니다.

```
const 고른칸 = await db.query("SELECT 이름, 라인 FROM 설비 WHERE id = 1");

console.log("고른 칸:", JSON.stringify(고른칸.rows[0]));
```

출력 고른 칸: {"이름":"컨베이어 1호","라인":"A"}

* 를 쓰면 전부 옵니다.

```
const 별표 = await db.query("SELECT * FROM 설비 WHERE id = 1");

console.log("* 로 온 칸들:", Object.keys(별표.rows[0]).join(", "));
```

출력 * 로 온 칸들: id, 이름, 라인, 상태, 담당자, 도입연도, 시간당생산량, 비교

★ 편해 보입니다. 그런데 실무 코드에는 쓰면 안 됩니다. 이유가 셋입니다. ① 안 쓰는 데이터를 실어 나릅니다 ② 칸이 늘면 코드가 조용히 깨집니다 ③ 색인만 읽고 끝낼 조회를 표까지 읽게 만듭니다 (06단원)

① 을 진짜로 재 봅니다. 긴 글이 든 표를 하나 만듭니다.

```
await db.exec(`
  CREATE TABLE 점검메모 (id SERIAL PRIMARY KEY, 설비명 TEXT, 결과 TEXT, 비고 TEXT);

  INSERT INTO 점검메모 (설비명, 결과, 비고)
  SELECT '설비' || (i % 300 + 1),
         (ARRAY['정상', '주의', '불량'])[i % 3 + 1],
         repeat('점검 결과 특이사항 없음. ', 20)
  FROM generate_series(1, 20000) AS i;
`);
```

중앙값을 씁니다. 평균은 어쩌다 한 번 느린 것에 크게 흔들립니다.

```
async function 재기(횟수, 할일) {
  const 잔것 = [];
  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    await 할일();
    잔것.push(performance.now() - 시작);
  }
  return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}

const 별표ms = await 재기(5, () => db.query("SELECT * FROM 점검메모"));
const 두칸ms = await 재기(5, () => db.query("SELECT id, 결과 FROM 점검메모"));

console.log(`SELECT * : ${별표ms.toFixed(0)} ms / SELECT id, 결과 :
${두칸ms.toFixed(0)} ms`);
```

```
출력      SELECT * : 217 ms / SELECT id, 결과 : 53 ms
값이 다를 수 있음
```

```
console.log("고른 칸이 더 빠른가:", 두칸ms < 별표ms);
```

```
출력      고른 칸이 더 빠른가: true
```

★★ 필요 없는 '비고' 하나 때문에 몇 배가 느려집니다. 2만 건에서 이만큼이면 200만 건에서는 서비스가 멈춥니다.

② 는 이런 식으로 깨집니다.

```
const [id, 이름] = Object.values(줄); ← 칸 순서에 기댄 코드
```

INSERT INTO 백업 SELECT * FROM 설비; ← 칸이 늘면 개수가 안 맞습니다

```
res.json(줄); ← 새로 생긴 '주민번호' 칸까지 나갑니다
```

★★★ 세 번째가 진짜 사고입니다. API 가 SELECT * 를 그대로 내보내는데 누가 표에 개인정보 칸을 추가하면 그 순간부터 아무도 모르게 밖으로 나갑니다.

★ SELECT * 를 써도 되는 곳: 손으로 들여다볼 때, 그리고 count(*).

WHERE 와 연산자들

섹션 2

```
async function 이름들(sql, 값들) {
  const 결과 = await db.query(sql, 값들);
  return 결과.rows.map((줄) => 줄.이름).join(" · ") || "(없음)";
}

console.log("= 가동:", await 이름들("SELECT 이름 FROM 설비 WHERE 상태 = '가동' ORDER BY id"));
```

출력 = 가동: 컨베이어 1호 · 프레스 1호 · 용접로봇 1호 · cnc 선반 2호 · 포장기 1호

```
console.log("<> 가동:", await 이름들("SELECT 이름 FROM 설비 WHERE 상태 <> '가동' ORDER BY id"));
```

출력 <> 가동: 컨베이어 2호 · 프레스 2호 · CNC 선반 1호

★ '같지 않다' 는 <> 입니다. != 도 통하지만 표준은 <> 입니다.

```
console.log("BETWEEN:", await 이름들("SELECT 이름 FROM 설비 WHERE 도입연도 BETWEEN 2019 AND 2022 ORDER BY id"));
```

출력 BETWEEN: 컨베이어 1호 · 컨베이어 2호 · 용접로봇 1호

★ BETWEEN 은 양쪽 끝을 포함합니다. 2019 도 2022 도 들어갑니다. ★★ 시각까지 있는 칸 (TIMESTAMP)에 쓰면 함정입니다. 1월 1일 AND 1월 31일 은 31일 00시 00분까지라 31일 오후 기록이 빠집니다. 날짜 범위는 >= 와 < 로 쓰세요.

```
console.log("IN:", await 이름들("SELECT 이름 FROM 설비 WHERE 상태 IN ('고장', '점검') ORDER BY id"));
```

출력 IN: 프레스 2호 · CNC 선반 1호

```
console.log("NOT IN:", await 이름들("SELECT 이름 FROM 설비 WHERE 라인 NOT IN ('A', 'B') ORDER BY id"));
```

출력 NOT IN: 용접로봇 1호 · CNC 선반 1호 · cnc 선반 2호

```
console.log("IS NULL:", await 이름들("SELECT 이름 FROM 설비 WHERE 담당자 IS NULL ORDER BY id"));
```

출력 IS NULL: 프레스 1호 · CNC 선반 1호

```
console.log("= NULL:", await 이름들("SELECT 이름 FROM 설비 WHERE 담당자 = NULL"));
```

출력 = NULL: (없음)

★★ 여기서 아주 많이 틀립니다. 담당자 = NULL 은 **에러가 아니라 그냥 0건** 입니다. NULL 은 “값이 없다”가 아니라 “모른다” 입니다. 모르는 것끼리 같은지 물으면 답도 “모른다” 입니다. 참이 아니니 안 나옵니다. 그래서 NULL 은 IS NULL / IS NOT NULL 로만 비교합니다.

★ NOT IN 과 NULL 이 만나면 더 이상합니다.

```
const 낚임 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 담당자 NOT IN ('김반장', NULL)");
```

```
console.log("NOT IN 목록에 NULL 이 섞이면:", 낚임.rows[0].건수, "건");
```

출력 NOT IN 목록에 NULL 이 섞이면: 0 건

★★★ 한 건도 안 나옵니다. 에러도 없습니다. ‘NULL 이 아닌가?’ 의 답이 언제나 ‘모른다’ 라서 전부 탈락합니다. 목록에 NULL 이 들어갈 수 있으면 NOT IN 을 쓰지 마세요.

AND / OR / 괄호

```
console.log(
  "AND:",
  await 이름들("SELECT 이름 FROM 설비 WHERE 라인 = 'C' AND 상태 = '가동' ORDER BY id"),
);
```

출력 AND: 용접로봇 1호 · cnc 선반 2호

```
console.log(
  "괄호 없이:",
  await 이름들("SELECT 이름 FROM 설비 WHERE 라인 = 'A' OR 라인 = 'B' AND 상태 = '가동' ORDER BY id"),
);
```

출력 괄호 없이: 컨베이어 1호 · 컨베이어 2호 · 프레스 1호 · 포장기 1호

```
console.log(
  "괄호 넣고:",
  await 이름들("SELECT 이름 FROM 설비 WHERE (라인 = 'A' OR 라인 = 'B') AND 상태 = '가동' ORDER BY id"),
);
```

출력 괄호 넣고: 컨베이어 1호 · 프레스 1호 · 포장기 1호

★ AND 가 OR 보다 먼저 묶입니다. 헷갈리면 괄호를 치세요. 괄호는 공짜입니다.

★ LIKE 와 와일드카드

섹션 3

LIKE 는 '이런 모양인 글자' 를 찾습니다. 기호가 두 개뿐입니다. % 아무 글자 0개 이상 _ 아무 글자 딱 1개

```
console.log("컨베이어%", await 이름들("SELECT 이름 FROM 설비 WHERE 이름 LIKE '컨베이어%' ORDER BY id"));
```

출력 컨베이어%: 컨베이어 1호 · 컨베이어 2호

```
console.log("%1호:", await 이름들("SELECT 이름 FROM 설비 WHERE 이름 LIKE '%1호' ORDER BY id"));
```

출력 %1호: 컨베이어 1호 · 프레스 1호 · 용접로봇 1호 · CNC 선반 1호 · 포장기 1호

```
console.log("_NC%", await 이름들("SELECT 이름 FROM 설비 WHERE 이름 LIKE '_NC%' ORDER BY id"));
```

출력 _NC%: CNC 선반 1호

★ _ 는 정확히 한 글자입니다. 'C' 자리 하나입니다.

★★ LIKE 는 대소문자를 가립니다. ILIKE 는 안 가립니다.

```
console.log("LIKE cnc%", await 이름들("SELECT 이름 FROM 설비 WHERE 이름 LIKE 'cnc%' ORDER BY id"));
```

출력 LIKE cnc%: cnc 선반 2호

```
console.log("ILIKE cnc%", await 이름들("SELECT 이름 FROM 설비 WHERE 이름 ILIKE 'cnc%' ORDER BY id"));
```

출력 ILIKE cnc%: CNC 선반 1호 · cnc 선반 2호

★ 한글에는 대소문자가 없습니다. 그래서 한글만 찾을 때는 둘이 같습니다.

```
const 한글LIKE = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 이름
LIKE '프레스%");
const 한글ILIKE = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 이름
ILIKE '프레스%");

console.log("한글은 LIKE 와 ILIKE 가 같은가:", 한글LIKE.rows[0].건수 ===
한글ILIKE.rows[0].건수);
```

출력 한글은 LIKE 와 ILIKE 가 같은가: true

★★ 사용자가 검색창에 % 를 치면 그 기호가 와일드카드로 동작해 버립니다. 비고 칸에는 '가동률 95% 이상 유지' 와 '불량률 0.95 수준' 이 있습니다. 사용자가 "95%" 를 쳤습니다. 첫 번째 줄만 나와야 합니다.

```
function 검색어다듬기(입력) {
```

와일드카드 기호 앞에 백슬래시를 붙여 '글자 그대로' 로 만듭니다.

```
return 입력.replace(/[\%_]/g, (글자) => "\\" + 글자);
}

const 그냥넘김 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 비고
LIKE '%' || $1 || '%'", [
    "95%",
]);

const 다듬어넘김 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 비고
LIKE '%' || $1 || '%'", [
    검색어다듬기("95%"),
]);

console.log("그냥 넘기면:", 그냥넘김.rows[0].건수, "건 / 다듬어 넘기면:",
다듬어넘김.rows[0].건수, "건");
```

출력 그냥 넘기면: 2 건 / 다듬어 넘기면: 1 건

★★ 사용자가 친 기호 하나가 와일드카드가 되어 엉뚱한 줄까지 끌고 왔습니다. '불량률 0.95 수준' 에는 퍼센트 기호가 없는데도 걸렸습니다. (값 자체는 언제나 파라미터로 넘깁니다. 개념03 에서 왜 그런지 봅시다)

★ LIKE '%검색어%' 는 앞에 % 가 붙으면 색인을 못 씁니다. 10만 건이면 10만 건을 전부 훑습니다. 06단원에서 왜 그런지 봅시다.

ORDER BY

섹션 4

```
async function 담당자순(sql) {
  const 결과 = await db.query(sql);
  return 결과.rows.map((줄) => `${줄.이름}(${줄.담당자 ?? "없음"})`).join(" · ");
}
```

```
console.log("담당자 ASC:", await 담당자순("SELECT 이름, 담당자 FROM 설비 ORDER BY
담당자, id LIMIT 4"));
```

출력 담당자 ASC: 컨베이어 1호(김반장) · 프레스 2호(김반장) · 용접로봇 1호
(박주임) · 컨베이어 2호(이기사)

```
console.log("담당자 DESC:", await 담당자순("SELECT 이름, 담당자 FROM 설비 ORDER BY
담당자 DESC, id LIMIT 4"));
```

출력 담당자 DESC: 프레스 1호(없음) · CNC 선반 1호(없음) · cnc 선반 2호(최반장)
· 컨베이어 2호(이기사)

```
console.log(
  "DESC NULLS LAST:",
  await 담당자순("SELECT 이름, 담당자 FROM 설비 ORDER BY 담당자 DESC NULLS LAST, id
LIMIT 4"),
);
```

출력 DESC NULLS LAST: cnc 선반 2호(최반장) · 컨베이어 2호(이기사) · 포장기 1호
(이기사) · 용접로봇 1호(박주임)

★★ Postgres 의 기본 규칙입니다.

ASC (오름차순) → NULL 이 **맨 뒤** · DESC (내림차순) → NULL 이 **맨 앞**

“NULL 은 가장 큰 값” 이라고 외우면 편합니다. 목록 맨 위에 값 없는 줄이 오면 이게 원인입니다. NULLS LAST 로 직접 정하세요.

여러 칸으로 정렬합니다. 앞의 칸이 먼저, 같으면 뒤의 칸으로 갑니다.

```
const 여러칸정렬 = await db.query(`
  SELECT 라인, 이름, 도입연도
  FROM 설비
  ORDER BY 라인 ASC, 도입연도 DESC NULLS LAST
`);
```

```
console.log(여러칸정렬.rows.map((줄) => `${줄.라인}:${줄.이름}(${줄.도입연도 ??
"미상"})`).join(" · "));
```

출력 A: 컨베이어 2호(2021) · A: 컨베이어 1호(2019) · A: 포장기 1호(미상) · B: 프레스 2호(2023) · B: 프레스 1호(2018) · C: cnc 선반 2호(2024) · C: 용접로봇 1호(2022) · C: CNC 선반 1호(2017)

LIMIT / OFFSET, 그리고 ★★ 그 함정 두 개

섹션 5

목록 화면은 보통 이렇게 만듭니다. 1쪽 → LIMIT 20 OFFSET 0 · 2쪽 → OFFSET 20 · ... · 4501쪽 → OFFSET 90000

```
const 첫쪽 = await db.query("SELECT id FROM 점검기록 ORDER BY id LIMIT 5 OFFSET 0");
const 둘째쪽 = await db.query("SELECT id FROM 점검기록 ORDER BY id LIMIT 5 OFFSET 5");

console.log("1쪽:", 첫쪽.rows.map((줄) => 줄.id).join(","), "/ 2쪽:",
둘째쪽.rows.map((줄) => 줄.id).join(","));
```

출력 1쪽: 1,2,3,4,5 / 2쪽: 6,7,8,9,10

★★ 함정 ①: OFFSET 이 커지면 느려집니다. OFFSET 90000 은 “90000개를 건너뛰다” 인데, 건너뛰려고 **그 90000개를 실제로 읽고** 나서 버립니다. 뒤로 갈수록 읽고 버리는 양이 늘어납니다. 10만 건으로 재 봅니다.

```
const 앞쪽ms = await 재기(7, () =>
  db.query("SELECT id, 설비명 FROM 점검기록 ORDER BY id LIMIT 20 OFFSET 0"),
);

const 뒤쪽ms = await 재기(7, () =>
  db.query("SELECT id, 설비명 FROM 점검기록 ORDER BY id LIMIT 20 OFFSET 90000"),
);

console.log(`OFFSET 0 : ${앞쪽ms.toFixed(2)} ms / OFFSET 90000 :
${뒤쪽ms.toFixed(2)} ms`);
```

출력 OFFSET 0 : 0.27 ms / OFFSET 90000 : 17.70 ms
값이 다를 수 있음

```
console.log(`몇 배 : ${((뒤쪽ms / 앞쪽ms).toFixed(0))} 배`);
```

출력 몇 배 : 66 배
값이 다를 수 있음

```
console.log("뒤쪽이 더 느린가:", 뒤쪽ms > 앞쪽ms);
```

출력 뒤쪽이 더 느린가: true

★★ 같은 20건을 가져오는데 수십 배가 걸립니다. 1000만 건짜리 표라면 뒷쪽은 아예 안 열립니다.

★ 해결: OFFSET 대신 **마지막으로 본 값** 을 조건으로 씁니다. 이걸 키셋 페이지 나누기(keyset pagination) 라고 합니다.

```
const 키셋ms = await 재기(7, () =>
  db.query("SELECT id, 설비명 FROM 점검기록 WHERE id > $1 ORDER BY id LIMIT 20",
    [90000]),
);

console.log(`키셋 : ${키셋ms.toFixed(2)} ms`);
```

출력 키셋 : 0.32 ms
값이 다를 수 있음

```
console.log("키셋이 OFFSET 90000 보다 빠르냐:", 키셋ms < 뒤쪽ms);
```

출력 키셋이 OFFSET 90000 보다 빠르냐: true

★ 키셋은 “몇 쪽으로 건너뛰기” 를 못 합니다. 다음 쪽만 됩니다. 무한 스크롤이나 ‘더 보기’ 버튼에는 딱 맞습니다.

★★★ 함정 ②: 같은 줄이 두 쪽에 나옵니다. 정렬 기준 칸의 값이 겹치면 그 안의 순서는 **정해져 있지 않습니다**. 점검기록의 라인은 A·B·C 뿐이라 3만 건씩 값이 같습니다. 라인만으로 쪽을 나눠 봅니다.

```
const 겹침1쪽 = await db.query("SELECT id FROM 점검기록 ORDER BY 라인 LIMIT 10 OFFSET 0");
const 겹침2쪽 = await db.query("SELECT id FROM 점검기록 ORDER BY 라인 LIMIT 10 OFFSET 10");

const 앞쪽id = 겹침1쪽.rows.map((줄) => 줄.id);
const 뒷쪽id = 겹침2쪽.rows.map((줄) => 줄.id);
const 겹친것 = 앞쪽id.filter((id) => 뒷쪽id.includes(id));

console.log("1쪽:", 앞쪽id.join(","));
```

출력 1쪽: 24,27,15,21,6,3,12,9,18,30
값이 다를 수 있음

```
console.log("2쪽:", 뒷쪽id.join(","));
```

출력 2쪽: 12,6,3,15,21,27,18,9,24,60
값이 다를 수 있음

```
console.log("두 쪽에 겹쳐 나온 줄:", 겹친것.length, "개");
```

출력 두 쪽에 겹쳐 나온 줄: 9 개
값이 다를 수 있음

```
console.log("겹친 줄이 있는가:", 겹친것.length > 0);
```

출력 겹친 줄이 있는가: true

★★★ 10개 중 9개가 1쪽에도 2쪽에도 나왔습니다.

화면에서는 "1쪽에서 본 기록이 2쪽에도 또 있어요", "분명 목록에 있었는데 넘겨도 안 보여요"로 나타납니다. 개발 DB 에는 데이터가 적어 안 겹치니 재현도 안 됩니다.

★ 왜 그런가: 라인 값이 같은 3만 건의 순서는 정해져 있지 않습니다. 매번 달라도 규칙 위반이 아닙니다.

★★ 해결은 한 줄입니다. **정렬에 고유한 칸을 덧붙입니다.**

```
const 고침1쪽 = await db.query("SELECT id FROM 점검기록 ORDER BY 라인, id LIMIT 10 OFFSET 0");
const 고침2쪽 = await db.query("SELECT id FROM 점검기록 ORDER BY 라인, id LIMIT 10 OFFSET 10");

const 고침1 = 고침1쪽.rows.map((줄) => 줄.id);
const 고침2 = 고침2쪽.rows.map((줄) => 줄.id);

console.log("고침 1쪽:", 고침1.join(","));
```

출력 고침 1쪽: 3,6,9,12,15,18,21,24,27,30

```
console.log("고침 2쪽:", 고침2.join(","));
```

출력 고침 2쪽: 33,36,39,42,45,48,51,54,57,60

```
console.log("이제 겹치는가:", 고침1.some((id) => 고침2.includes(id)));
```

출력 이제 겹치는가: false

★ 규칙: **ORDER BY** 의 마지막에는 언제나 고유한 칸(보통 id)을 넣으세요.

DISTINCT

섹션 6

```
const 라인목록 = await db.query("SELECT DISTINCT 라인 FROM 설비 ORDER BY 라인");

console.log("라인 종류:", 라인목록.rows.map((줄) => 줄.라인).join(", "));
```

출력 라인 종류: A, B, C

★ DISTINCT 는 **고른 칸 전체의 조합**에 대해 중복을 없앱니다. 한 칸이 아닙니다.

```
const 조합 = await db.query("SELECT DISTINCT 라인, 상태 FROM 설비 ORDER BY 라인, 상태");

console.log("라인+상태 조합:", 조합.rows.map((줄) => `${줄.라인}/${줄.상태}`).join(" · "));
```

출력 라인+상태 조합: A/가동 · A/정지 · B/가동 · B/점검 · C/가동 · C/고장

★ DISTINCT 는 공짜가 아닙니다. 10만 건을 전부 훑어 중복을 지웁니다. “종류가 몇 개인지” 만 필요하면 count(DISTINCT 칸) 이 낮습니다.

```
const 설비종류 = await db.query("SELECT count(DISTINCT 설비명)::int AS 종류 FROM 점검기록");

console.log("점검기록에 나온 설비 종류:", 설비종류.rows[0].종류);
```

출력 점검기록에 나온 설비 종류: 500

별칭(AS)과 계산 칸

섹션 7

읽어 오면서 계산할 수 있습니다. 그 결과에 이름을 붙이는 게 별칭입니다.

```
const 계산 = await db.query(`
  SELECT
    이름,
    시간당생산량 * 8           AS 하루생산량,
    2026 - 도입연도          AS 사용연수,
    상태 = '가동'            AS 돌고있음
  FROM 설비
  WHERE 시간당생산량 IS NOT NULL
  ORDER BY 하루생산량 DESC
  LIMIT 3
`);

for (const 줄 of 계산.rows) {
  console.log(`${줄.이름} - 하루 ${줄.하루생산량}개 · ${줄.사용연수} ?? "?"년차 · 가동중=${줄.돌고있음}`);
}
```

출력 컨베이어 1호 - 하루 9600개 · 7년차 · 가동중=true
 컨베이어 2호 - 하루 8800개 · 5년차 · 가동중=false
 포장기 1호 - 하루 7200개 · ?년차 · 가동중=true

★ 별칭에 띄어쓰기를 쓰려면 큰따옴표가 필요합니다. AS “하루 생산량” 처럼요. 작은따옴표는 값이라서 안 됩니다.

★★ 별칭은 ORDER BY 에서는 쓸 수 있는데 WHERE 에서는 못 씁니다.

```
const 별칭실패 = await db
  .query("SELECT 이름, 시간당생산량 * 8 AS 하루생산량 FROM 설비 WHERE 하루생산량 > 5000")
  .catch((에러) => 에러);

console.log("WHERE 에서 별칭:", 별칭실패.code, ".", 별칭실패.message);
```

출력 WHERE 에서 별칭: 42703 · column "하루생산량" does not exist

★ 왜 그런가: 실행 순서가 이렇기 때문입니다.

FROM → WHERE → GROUP BY → SELECT → ORDER BY → LIMIT

WHERE 를 볼 때는 별칭이 아직 안 만들어져 있습니다. ORDER BY 때는 있습니다. 이 순서를 외워 두면 “왜 여기서 되고 저기선 안 되지” 가 거의 다 풀립니다.

```
const 별칭해결 = await db.query(
  "SELECT 이름 FROM 설비 WHERE 시간당생산량 * 8 > 5000 ORDER BY 이름",
);

console.log("계산식을 그대로 쓰면:", 별칭해결.rows.map((줄) => 줄.이름).join(" · "));
```

출력 계산식을 그대로 쓰면: 용접로봇 1호 · 컨베이어 1호 · 컨베이어 2호 · 포장기 1호

정리 — 읽기

무엇 쓰는 법 주의할 점

칸 고르기 SELECT 이름, 라인 * 는 코드에 쓰지 마세요

같다/다르다	= , <>	!= 도 되지만 <> 가 표준
범위	BETWEEN 가 AND 나	양 끝을 포함합니다
목록	IN ('가', '나')	목록에 NULL 이 있으면 NOT IN 금지

빈 값 IS NULL = NULL 은 언제나 0건

모양	LIKE / ILIKE	% 와 _ 가 와일드카드
정렬	ORDER BY 칸 ASC/DESC	ASC→NULL 뒤, DESC→NULL 앞
자르기	LIMIT n OFFSET m	* OFFSET 이 크면 느립니다

중복 없애기 DISTINCT 고른 칸 '조합' 기준 이름 붙이기 AS WHERE 에서는 못 씁니다

실행 순서: FROM → WHERE → GROUP BY → SELECT → ORDER BY → LIMIT

★★ 재현한 사고 두 개 ① OFFSET 90000 이 OFFSET 0 보다 수십 배 느립니다 → 키셋으로 바꾸세요 ② 정렬키가 겹치면 같은 줄이 두 쪽에 나옵니다 → ORDER BY 끝에 id 를 붙이세요

직접 해 볼 것

▫ 직접 해보기 1

담당자가 없는 설비를 담당자 = " 로 찾아보세요. 왜 안 나올까요? 빈 글자와 NULL 은 다릅니다.

▫ 직접 해보기 2

섹션 5 의 겹침 실험에서 LIMIT 을 10 대신 100 으로 해 보세요. 겹치는 줄이 늘어나나요, 줄어드나요?

▫ 직접 해보기 3

OFFSET 을 90000 대신 99980 으로 바꿔 재 보세요. 더 느려지나요? 어디까지 느려지나요?

▫ 직접 해보기 4

점검기록에서 결과가 '불량' 인 것만 최근 순으로 20건 뽑아 보세요. ORDER BY 에 id 를 꼭 붙이세요.

▫ 직접 해보기 5

설비 이름에 '선반' 이 들어간 것을 대소문자 상관없이 찾아보세요. 그리고 그 검색어를 파라미터(\$1)로 넘겨 보세요.

자주 하는 실수

이런 실수를 합니다

NULL 을 = 로 비교함

담당자 = NULL 은 애러가 아니라 0건입니다. IS NULL 을 쓰세요. NOT IN 목록에 NULL 이 섞이면 결과가 통째로 0건이 됩니다.

이런 실수를 합니다

ORDER BY 없이 LIMIT 을 씀

"아무 20건" 이 나옵니다. 매번 다를 수 있습니다. LIMIT 을 쓸 때는 반드시 ORDER BY 를 쓰세요.

이런 실수를 합니다

ORDER BY 에 고유한 칸을 안 붙임

ORDER BY 점검일 DESC 가 아니라 ORDER BY 점검일 DESC, id DESC 로 쓰세요.

이런 실수를 합니다

코드에서 **SELECT *** 를 씀

칸이 하나 늘면 응답에 그 칸이 딸려 나갑니다. 개인정보 칸이면 그대로 유출입니다.

이런 실수를 합니다

페이지가 뒤로 갈수록 느린 걸 서버 탓으로 돌림

OFFSET 때문입니다. 서버를 키워도 안 낫습니다. 쿼리를 바꿔야 합니다.

이런 실수를 합니다

검색창 입력을 그대로 **LIKE** 에 넣음

% 를 치면 전부 검색되고 _ 는 아무 한 글자가 됩니다. 다듬어서 넘기세요.

```
await db.close();
```


SQL 인젝션을 직접 뚫어 봅니다

RUN node 개념03_SQL_인젝션을_직접_뚫어봅니다.js

★ 이 파일에는 일부러 취약하게 만든 코드가 들어 있습니다.

그런 줄에는 // 검증무시: 라고 표시해 두었습니다. 실무에 복사하지 마세요.

개념02 까지는 SQL 문장을 손으로 적었습니다. 실제 프로그램에서는 조건이 **사용자가 친 값**으로 정해집니다. 로그인은 사번과 비밀번호로, 검색은 검색어로, 목록은 정렬 기준으로요.

그 값을 SQL 문장에 어떻게 넣느냐 — 여기서 웹 보안 사고의 절반이 납니다.

이 파일은 설명만 하지 않습니다. **진짜로 뚫습니다.** ① 비밀번호 없이 관리자로 로그인하고 ② 못 보는 표를 끌어내고 ③ 표를 지웁니다. 그다음 같은 공격을 파라미터 바인딩으로 막습니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

공장 시스템을 하나 만듭니다

섹션 0

```
await db.exec(`
  CREATE TABLE 작업자 (
    id      SERIAL PRIMARY KEY,
    사번    TEXT NOT NULL UNIQUE,
    이름    TEXT NOT NULL,
    비밀번호 TEXT NOT NULL,
    등급    TEXT NOT NULL
  );

  INSERT INTO 작업자 (사번, 이름, 비밀번호, 등급) VALUES
    ('A1001', '김반장', 'gongjang1', '관리자'),
    ('A1002', '이기사', 'press2024', '작업자'),
    ('A1003', '박주임', 'welding77', '작업자');

  CREATE TABLE 급여 (
    사번    TEXT NOT NULL,
    월급    INT NOT NULL,
    계좌    TEXT NOT NULL
  );
```

```
( 'A1001', 5200000, '111-22-333333' ),
    ( 'A1002', 3800000, '444-55-666666' ),
    ( 'A1003', 3500000, '777-88-999999' );
`);
```

★ 실제로는 비밀번호를 이렇게 그대로 저장하면 안 됩니다. 해시로 저장합니다. 여기서는 인젝션만 보려고 단순하게 뒀습니다.

흔하디흔한 취약 코드

섹션 1

로그인 기능입니다. 아주 자연스러워 보입니다. 실제로 이런 코드가 수없이 많이 돌아가고 있습니다.

```
async function 취약한로그인(사번, 비밀번호) {
    const 문장 = `SELECT 사번, 이름, 등급 FROM 작업자
                  WHERE 사번 = '${사번}' AND 비밀번호 = '${비밀번호}'`;

    try {
        const 결과 = await db.query(문장);
        return { 문장, 줄들: 결과.rows };
    } catch (에러) {
        return { 문장, 에러: `${에러.code} ${에러.message}` };
    }
}
```

정상 사용자가 제대로칩니다.

```
const 정상 = await 취약한로그인("A1001", "gongjang1");

console.log("정상 로그인:", 정상.줄들.length, "건 .", 정상.줄들[0]?.이름 ?? "실패");
```

출력 정상 로그인: 1 건 . 김반장

비밀번호를 틀립니다.

```
const 틀림 = await 취약한로그인("A1001", "아무거나");

console.log("틀린 비밀번호:", 틀림.줄들.length, "건");
```

출력 틀린 비밀번호: 0 건

여기까지는 완벽하게 잘 돌아갑니다. 테스트도 다 통과합니다.

★★★ 비밀번호 없이 로그인합니다

섹션 2

비밀번호 칸에 이걸칩니다.

```
' OR '1'='1
```

무슨 일이 일어나는지 만들어진 문장을 직접 보겠습니다.

```
const 뚫기1 = await 취약한로그인("A1001", "' OR '1'='1");

console.log("만들어진 문장:", 뚫기1.문장.replace(/\s+/g, " "));
```

출력 만들어진 문장: SELECT 사번, 이름, 등급 FROM 작업자 WHERE 사번 = 'A1001'
AND 비밀번호 = '' OR '1'='1'

```
console.log("로그인 결과:", 뚫기1.줄들.length, "건");
```

출력 로그인 결과: 3 건

```
console.log("등급들:", 뚫기1.줄들.map((줄) => `${줄.이름}/${줄.등급}`).join(" · "));
```

출력 등급들: 김반장/관리자 · 이기사/작업자 · 박주임/작업자

★★★ 비밀번호를 모르는데 전 직원이 로그인됐습니다.

```
WHERE 사번 = 'A1001' AND 비밀번호 = '' OR '1'='1'
└─ 거짓일 수도 있는 부분 ─┘ └─ 언제나 참 ┘
```

AND 가 OR 보다 먼저 묶이므로 결국 "...또는 참" 이 됩니다. 전부 통과입니다. 대부분의 로그인 코드는 첫 줄을 그대로 로그인 처리합니다. 즉 **관리자 김반장으로 들어갑니다.**

사번 칸으로도 됩니다. 이번에는 주석으로 뒤를 통째로 없앱니다.

```
const 뚫기2 = await 취약한로그인("' OR 1=1 --", "비밀번호따위");

console.log("주석으로 자른 문장:", 뚫기2.문장.replace(/\s+/g, " "));
```

출력 주석으로 자른 문장: SELECT 사번, 이름, 등급 FROM 작업자 WHERE 사번 = ''
OR 1=1 --' AND 비밀번호 = '비밀번호따위'

```
console.log("결과:", 뚫기2.줄들.length, "건 · 첫 줄:", 뚫기2.줄들[0]?.이름);
```

출력 결과: 3 건 · 첫 줄: 김반장

★ SQL 에서 -- 는 주석입니다. 그 뒤는 전부 무시됩니다. 비밀번호를 검사하는 부분 자체가 사라졌습니다.

★★★ 볼 수 없는 표를 끌어냅니다

섹션 3

설비 검색 기능입니다. 역시 문자열을 이어 붙입니다.

```

async function 취약한검색(검색어) {
  const 문장 = `SELECT 이름, 등급 FROM 작업자 WHERE 이름 LIKE '%${검색어}%' ORDER BY 1`;

  try {
    return (await db.query(문장)).rows;
  } catch (에러) {
    return [{ 이름: `에러 ${에러.code}`, 등급: 에러.message }];
  }
}

const 정상검색 = await 취약한검색("김");

console.log("정상 검색:", JSON.stringify(정상검색));

```

출력 정상 검색: [{"이름":"김반장","등급":"관리자"}]

★ UNION 은 두 SELECT 의 결과를 위아래로 이어 붙입니다. 칸 개수만 맞으면 **전혀 다른 표의 내용도** 붙일 수 있습니다.

먼저 어떤 표가 있는지 캐냅니다. information_schema 는 Postgres 가 스스로 갖고 있는 “표 목록이 든 표” 입니다. 누구나 읽을 수 있습니다.

```

const 표캐내기 = await 취약한검색(
  "김' UNION SELECT table_name, '' FROM information_schema.tables WHERE table_schema = 'public' ORDER BY 1 --",
);

console.log("알아낸 표 이름들:", 표캐내기.map((줄) => 줄.이름).join(", "));

```

출력 알아낸 표 이름들: 급여, 작업자

★★ ‘급여’ 라는 표가 있다는 것을 알아냈습니다. 이제 안을 봅니다.

```

const 유출 = await 취약한검색(
  "김' UNION SELECT 사번 || ' ' || 계좌, 월급::text FROM 급여 ORDER BY 1 --",
);

for (const 줄 of 유출) {
  console.log(`유출 · ${줄.이름} / ${줄.등급}`);
}

```

출력 유출 · A1001 111-22-333333 / 5200000
 유출 · A1002 444-55-666666 / 3800000
 유출 · A1003 777-88-999999 / 3500000

★★★ 계좌번호와 월급이 그대로 나왔습니다. 검색 기능 하나 때문에 **로그인도 안 한 사람이** 전 직원 급여를 봤습니다. 화면에는 “이름 / 등급” 칸으로 표시되니 개발자는 눈치채기 어렵습니다.

★ 진짜 공격은 여기서 더 나갑니다. 비밀번호 해시를 끌어내고, 파일을 읽습니다.

★★★ 표를 지웁니다

섹션 4

“SELECT 만 하는 자리니까 지우지는 못하겠지” — 그렇지 않습니다.

PGlite 의 db.query 는 세미콜론으로 이은 두 문장을 거부합니다.

```
const 두문장 = await db
  .query("SELECT 1; DROP TABLE 급여")
  .catch((에러) => 에러);

console.log("query 로 두 문장:", 두문장.code, ".", 두문장.message);
```

```
출력      query 로 두 문장: 42601 · cannot insert multiple commands into a prepared
          statement
```

★ 이걸 PGlite 나 node-postgres 가 **준비된 문장(prepared statement)** 으로 보내기 때문입니다. 다행히 여기서 한 겹 막힙니다.

★★★ 그런데 db.exec 는 여러 문장을 그대로 실행합니다. 스키마를 만들 때 쓰라고 있는 함수인데, 여기에 사용자 입력이 닿으면 끝입니다.

```
await db.exec(`
  CREATE TABLE 작업일지 (id SERIAL PRIMARY KEY, 내용 TEXT NOT NULL);
  INSERT INTO 작업일지 (내용) VALUES ('오전 점검 완료');
`);

async function 취약한일지쓰기(내용) {
  const 문장 = `INSERT INTO 작업일지 (내용) VALUES ('${내용}')`;

  try {
    await db.exec(문장);
    return "저장됨";
  } catch (에러) {
    return `${에러.code}`;
  }
}

console.log("정상 저장:", await 취약한일지쓰기("오후 점검 완료"));
```

```
출력      정상 저장: 저장됨
```

공격자가 일지 칸에 이렇게 씁니다.

```
const 공격문 = "청소 완료'); DROP TABLE 급여; --";

console.log("공격 결과:", await 취약한일지쓰기(공격문));
```

출력 공격 결과: 저장됨

```
const 급여남았나 = await db.query("SELECT to_regclass('급여') AS 있나");

console.log("급여 표가 아직 있나:", 급여남았나.rows[0].있나 !== null);
```

출력 급여 표가 아직 있나: false

★★★ 표가 사라졌습니다. 되돌릴 수 없습니다. 백업에서 복구해야 합니다.

그리고 앱 로그에는 “일지 저장 성공” 이라고 찍혔습니다.

왜 뚫리는가

섹션 5

★★★ 데이터와 명령을 같은 글자 줄에 섞었기 때문입니다.

데이터베이스가 받은 것은 그냥 긴 글자 한 줄입니다. 어디까지가 명령이고 어디부터가 값인지 구분할 방법이 없습니다. 따옴표 하나가 그 경계인데, 사용자가 따옴표를 칠 수 있으면 경계를 옮깁니다.

같은 모양의 사고가 여기저기 있습니다. 셸 명령에 이어 붙이면 명령 주입, HTML 에 이어 붙이면 XSS 입니다.

★ 파라미터 바인딩으로 막습니다

섹션 6

값을 문장에 끼워 넣지 않습니다. 자리만 표시하고 값은 따로 보냅니다.

```
Postgres → $1, $2, $3
MySQL → ?
```

```
async function 안전한로그인(사번, 비밀번호) {
  const 결과 = await db.query(
    "SELECT 사번, 이름, 등급 FROM 작업자 WHERE 사번 = $1 AND 비밀번호 = $2",
    [사번, 비밀번호],
  );

  return 결과.rows;
}

console.log("정상 로그인:", (await 안전한로그인("A1001", "gongjang1")).length, "건");
```

출력 정상 로그인: 1 건

아까 통했던 공격을 그대로 다시 합니다.

```
console.log("' OR '1'='1 공격:", (await 안전한로그인("A1001", "' OR '1'='1")).length, "건");
```

출력 ' OR '1'='1 공격: 0 건

```
console.log("주석 공격:", (await 안전한로그인("' OR 1=1 --", "아무거나")).length, "건");
```

출력 주석 공격: 0 건

```
console.log("UNION 공격:", (await 안전한로그인("A1001' UNION SELECT 1,2,3 --", "x")).length, "건");
```

출력 UNION 공격: 0 건

★ 전부 0건입니다. 예러도 안 납니다. 그냥 그런 사변이 없을 뿐 입니다.

★ 왜 파라미터는 안전한가

섹션 7

파라미터로 보낸 값은 문장으로 해석되지 않습니다.

① 클라이언트가 서버에 문장 틀을 보냅니다

```
"SELECT ... WHERE 사번 = $1 AND 비밀번호 = $2"
```

② 서버가 이 틀을 먼저 파싱합니다. 이 시점에 문장의 모양이 확정됩니다 ③ 그다음 값을 따로 보냅니다 ④ 값은 이미 확정된 자리에 그냥 들어왔습니다

★★ ② 에서 문장 모양이 이미 정해졌기 때문에, 값에 따옴표가 있든 DROP TABLE 이 있든 문장이 바뀌지 않습니다. 그건 그냥 "그런 글자로 된 사번" 입니다.

진짜로 그런지 확인해 봅니다. 공격 문자열을 이름으로 저장해 보겠습니다.

```
await db.query("INSERT INTO 작업자 (사번, 이름, 비밀번호, 등급) VALUES ($1, $2, $3, $4)", [
  "A1004",
  "' OR '1'='1; DROP TABLE 작업자; --",
  "test1234",
  "작업자",
]);

const 저장된것 = await db.query("SELECT 이름 FROM 작업자 WHERE 사번 = $1", ["A1004"]);

console.log("저장된 이름:", 저장된것.rows[0].이름);
```

출력 저장된 이름: ' OR '1'='1; DROP TABLE 작업자; --

```
const 작업자남았나 = await db.query("SELECT count(*)::int AS 건수 FROM 작업자");

console.log("작업자 표는 몇평한가:", 작업자남았나.rows[0].건수, "건");
```

출력 작업자 표는 몇평한가: 4 건

★ 공격 문자열이 그냥 **이름 데이터**가 됐습니다. 이게 정답입니다. “위험한 글자를 지운다”가 아니라 “값은 값일 뿐이게 만든다”입니다.

위의 실행 결과가 지저분해지니 이 줄은 지우고 갑니다.

```
await db.query("DELETE FROM 작업자 WHERE 사번 = $1", ["A1004"]);
```

★ 값이 여러 개일 때도 파라미터로 됩니다. IN 은 = ANY 로 바꿉니다.

```
const 여럿 = await db.query("SELECT 이름 FROM 작업자 WHERE 사번 = ANY($1) ORDER BY 사번", [
  ["A1001", "A1003"],
]);

console.log("여러 값 조회:", 여럿.rows.map((줄) => 줄.이름).join(", "));
```

출력 여러 값 조회: 김반장, 박주임

★ LIKE 도 파라미터로 됩니다. % 는 값 쪽에 붙입니다.

```
const 좋은검색 = await db.query("SELECT 이름 FROM 작업자 WHERE 이름 LIKE $1", ["김%"]);

console.log("안전한 LIKE:", 좋은검색.rows.map((줄) => 줄.이름).join(", "));
```

출력 안전한 LIKE: 김반장

★★ 파라미터로 막을 수 없는 자리가 있습니다

섹션 8

파라미터는 **값이 오는 자리**에만 쓸 수 있습니다. 표 이름, 칸 이름, 정렬 방향은 문장의 **구조**라서 안 됩니다.

하나씩 확인해 봅니다.

```
const 표이름시도 = await db.query("SELECT count(*) FROM $1", ["작업자"]).catch((에러) => 에러);
```

```
console.log("표 이름을 파라미터로:", 표이름시도.code, ".", 표이름시도.message);
```

출력 표 이름을 파라미터로: 42601 · syntax error at or near "\$1"

```
const 방향시도 = await db
  .query("SELECT 이름 FROM 작업자 ORDER BY 이름 $1", ["DESC"])
  .catch((에러) => 에러);
```

```
console.log("정렬 방향을 파라미터로:", 방향시도.code, ".", 방향시도.message);
```

출력 정렬 방향을 파라미터로: 42601 · syntax error at or near "\$1"

★★★ 여기가 진짜 함정입니다. 칸 이름은 **에러도 안 납니다**.

```
const 칸이름시도 = await db.query("SELECT $1 FROM 작업자 LIMIT 3", ["이름"]);
```

```
console.log("칸 이름을 파라미터로:", JSON.stringify(칸이름시도.rows));
```

출력 칸 이름을 파라미터로: [{"column?": "이름"}, {"column?": "이름"}, {"column?": "이름"}]

★★ 칸 값이 아니라 “이름”이라는 **글자**가 세 번 나왔습니다. 에러가 없으니 테스트도 통과합니다. 화면에만 이상하게 나옵니다.

```
const 정렬시도 = await db.query("SELECT 이름 FROM 작업자 ORDER BY $1", ["이름"]);
```

```
console.log("정렬 기준을 파라미터로:", 정렬시도.rows.map((줄) => 줄.이름).join(", "));
```

출력 정렬 기준을 파라미터로: 김반장, 이기사, 박주임

```
const 진짜정렬 = await db.query("SELECT 이름 FROM 작업자 ORDER BY 이름");
```

```
console.log("진짜로 정렬하면:", 진짜정렬.rows.map((줄) => 줄.이름).join(", "));
```

출력 진짜로 정렬하면: 김반장, 박주임, 이기사

```
const 같은순서 = 정렬시도.rows.map((줄) => 줄.이름).join() === 진짜정렬.rows.map((줄) => 줄.이름).join();

console.log("두 결과가 같은가:", 같은순서);
```

출력 두 결과가 같은가: false

★★ 정렬이 **조용히 안 됐습니다**. 상수 하나로 정렬한 셈이라 순서가 그대로입니다.

★★★ 그러면 어떻게 하나요. **허용 목록(allowlist)** 을 씁니다. 사용자가 보낸 글자를 SQL 에 넣는 게 아니라, **내가 미리 정해 둔 목록에서 고릅니다**.

```
const 정렬가능칸 = { 이름: "이름", 사번: "사번", 등급: "등급" };
const 정렬방향 = { 오름: "ASC", 내림: "DESC" };

async function 작업자목록(정렬칸요청, 방향요청) {
```

★ 목록에 없으면 기본값으로 떨어뜨립니다. 사용자 글자는 SQL 에 절대 안 닿습니다.

```
const 정렬칸 = 정렬가능칸[정렬칸요청] ?? "사번";
const 방향 = 정렬방향[방향요청] ?? "ASC";
```

허용목록에서 고른 값이므로 이어 붙여도 안전합니다.

```
const 결과 = await db.query(`SELECT 사번, 이름 FROM 작업자 ORDER BY
${정렬칸} ${방향}, id`);

return 결과.rows.map((줄) => 줄.이름).join(", ");
}

console.log("이름 오름차순:", await 작업자목록("이름", "오름"));
```

출력 이름 오름차순: 김반장, 박주임, 이기사

```
console.log("사번 내림차순:", await 작업자목록("사번", "내림"));
```

출력 사번 내림차순: 박주임, 이기사, 김반장

```
console.log("공격 시도:", await 작업자목록("이름; DROP TABLE 작업자 --", "내림"));
```

출력 공격 시도: 박주임, 이기사, 김반장

★ 공격 문자열은 목록에 없으므로 기본값 '사번 ASC' 로 떨어졌습니다. 표는 멀쩡합니다.

★★ 목록의 **키**를 사용자에게 노출하고, **값**만 SQL 에 넣는 것이 핵심입니다. "정규식으로 위험한 글자를 걸러 내는" 방식은 언젠가 뚫립니다. 허용 목록은 뚫릴 수가 없습니다. 목록에 없으면 그냥 안 되니까요.

★ 이스케이프로 막으려는 시도는 왜 실패하는가

섹션 9

“따옴표를 두 개로 바꾸면 되지 않나요?” — 흔한 생각입니다. 해 봅니다.

```
function 어설픈이스케이프(값) {
  return String(값).replace(/'/g, "'");
}
```

값이 따옴표 안에 있는 자리에서는 이게 통하는 것처럼 보입니다.

```
async function 이스케이프로그인(사번, 비밀번호) {
  const 문장 = `SELECT 이름 FROM 작업자
    WHERE 사번 = '${어설픈이스케이프(사번)}'
    AND 비밀번호 = '${어설픈이스케이프(비밀번호)}'`;

  return (await db.query(문장)).rows;
}

console.log("이스케이프 후 공격:", (await 이스케이프로그인("A1001", "' OR
'1'='1')).length, "건");
```

출력 이스케이프 후 공격: 0 건

막힌 것처럼 보입니다. 그런데 **숫자 자리**에는 따옴표가 없습니다.

```
async function 이스케이프상세(id) {
  const 문장 = `SELECT 이름, 등급 FROM 작업자 WHERE id = ${어설픈이스케이프(id)}`;

  return (await db.query(문장)).rows;
}

console.log("정상 조회:", (await 이스케이프상세("1")).length, "건");
```

출력 정상 조회: 1 건

```
const 숫자뚫기 = await 이스케이프상세("1 OR 1=1");

console.log("숫자 자리 공격:", 숫자뚫기.length, "건 .", 숫자뚫기.map((줄) => 줄.
이름).join(", "));
```

출력 숫자 자리 공격: 3 건 · 김반장, 이기사, 박주임

★★★ 따옴표를 한 개도 안 쓰고 전부 끌어냈습니다. 이스케이프 함수는 **따옴표 안에 있을 때만** 의미가 있습니다.

★ 직접 만든 이스케이프가 실패하는 이유는 더 있습니다.

- 숫자·식별자·LIMIT 자리에는 따옴표가 없습니다
- 백슬래시 처리 방식이 설정마다 다릅니다
- 문자 인코딩에 따라 따옴표로 되살아나는 바이트 조합이 있습니다
- 함수를 한 군데라도 빼먹으면 그 한 군데로 뚫립니다

★★★ 규칙: 이스케이프 함수를 직접 만들지 마세요. 값은 파라미터로, 구조는 허용 목록으로. 이 둘이면 끝납니다.

MySQL 은 여기가 다릅니다

· 파라미터 자리 표시가 ? 입니다 · mysql2 는 기본이 '클라이언트에서 문자열로 만들어 보내기' 라서

`execute()` (진짜 준비된 문장) 를 쓰는 편이 낫습니다

· `multipleStatements` 옵션을 켜면 세미콜론 공격이 그대로 통합니다. 켜지 마세요 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — SQL 인젝션

어디에 넣나 파라미터가 되나 어떻게 하나

WHERE 값 됩니다 \$1, \$2 여러 개의 값 됩니다 = ANY(\$1) 에 배열 LIKE 검색어 됩니다 \$1 에 '%검색어%' 표 이름 안 됨 · 42601 에러 ★ 허용 목록 정렬 방향(ASC/DESC) 안 됨 · 42601 에러 ★ 허용 목록 칸 이름 ★★ 에러 없이 글자 상수 ★ 허용 목록 ORDER BY 기준 ★★ 에러 없이 정렬 안 됨 ★ 허용 목록 여러 문장 (exec) 해당 없음 입력이 닿지 않게

★★★ 외울 것 두 줄 ① 값은 절대 문장에 이어 붙이지 않습니다. 파라미터로 보냅니다. ② 파라미터가 안 되는 자리는 허용 목록으로 고릅니다.

직접 해 볼 것

▫ 직접 해보기 1

취약한로그인 에 비밀번호로 ' OR 등급 = '관리자' 를 넣어 보세요. 관리자만 골라서 로그인할 수 있나요?

▫ 직접 해보기 2

취약한검색 으로 작업자 표의 비밀번호 칸을 끌어내 보세요. UNION SELECT 사변, 비밀번호 FROM 작업자 를 써 보세요.

▫ 직접 해보기 3

섹션 8 의 작업자목록 에 정렬칸으로 '비밀번호' 를 넣어 보세요. 왜 안 되나요? 허용 목록에 '비밀번호' 를 넣으면 어떤 일이 생길까요?

▹ 직접 해보기 4

안전한로그인 에 사번으로 A1001 앞뒤에 공백을 넣어 보세요. 파라미터는 공백까지 값으로 봅니다. 어떻게 되나요?

▹ 직접 해보기 5

이스케이프상세 를 파라미터(\$1)로 고쳐 보세요. 1 OR 1=1 을 넣으면 이번에는 어떤 에러가 나나요?
(힌트: 22P02)

자주 하는 실수

이런 실수를 합니다

“우리는 내부망이라 괜찮아요”

인젝션은 대부분 내부 직원이나 협력사 계정을 통해 들어옵니다. 그리고 내부망 시스템이 언젠가 외부에 열립니다. 반드시 열립니다.

이런 실수를 합니다

프런트엔드에서 검증했으니 됐다고 생각함

화면을 거치지 않고 API 를 직접 부르면 그만입니다. 검증은 서버에서, 방어는 쿼리에서 합니다.

이런 실수를 합니다

위험한 낱말을 막는 방식(blocklist)

‘DROP’ 이나 ‘--’ 를 막아도 우회 방법은 끝없이 많습니다. 막는 목록이 아니라 허용 목록입니다.

이런 실수를 합니다

ORM 을 쓰니 안전하다고 믿음

ORM 도 raw query 를 지원하고, 거기에 이어 붙이면 똑같이 뚫립니다. 정렬 칸을 문자열로 받는 API 도 흔한 구멍입니다.

이런 실수를 합니다

에러 메시지를 그대로 화면에 뿌림

relation “급여” does not exist 한 줄이면 표 이름이 넘어갑니다. 공격자는 에러 메시지로 구조를 알아냅니다. 사용자에게는 뭉뚱그려 보여 주세요.

이런 실수를 합니다

exec 나 **multi-statement** 를 편해서 그냥 씀

섹션 4 에서 본 그대로입니다. 한 번 닿으면 표가 사라집니다. 스키마 만들 때만 쓰고, 사용자 입력은 절대 닿지 않게 하세요.

```
await db.close();
```

고치고 지우기

RUN node 개념04_고치고_지우기.js

★ 20만 건을 지우는 실험이 있어 3초쯤 걸립니다.

넣었고(개념01) 읽었습니다(개념02). 이제 고치고 지웁니다.

UPDATE 와 DELETE 는 문법이 아주 짧습니다. 반나절이면 다 배웁니다. 그런데 **운영 사고의 대부분이 이 둘에서 납니다.**

이유는 하나입니다. SELECT 는 틀려도 화면만 이상하지만, UPDATE 와 DELETE 는 **틀리는 순간 데이터가 사라집니다.**

그래서 이 파일은 문법보다 '안 틀리는 습관'에 무게를 둡니다. WHERE 를 빠뜨렸을 때 무슨 일이 나는지 진짜로 돌려 봅니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

설비 표를 만듭니다

섹션 0

```
await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    이름        TEXT NOT NULL,
    라인        TEXT NOT NULL,
    상태        TEXT NOT NULL,
    점검횟수    INT NOT NULL DEFAULT 0,
    삭제일시    TIMESTAMPTZ
  );

  INSERT INTO 설비 (이름, 라인, 상태) VALUES
    ('컨베이어 1호', 'A', '가동'),
    ('컨베이어 2호', 'A', '정지'),
    ('프레스 1호', 'B', '가동'),
    ('프레스 2호', 'B', '점검'),
    ('용접로봇 1호', 'C', '가동'),
    ('CNC 선반 1호', 'C', '고장');
`);
```

```
async function 상태보기() {
  const 결과 = await db.query("SELECT 이름, 상태 FROM 설비 ORDER BY id");
  return 결과.rows.map((줄) => `${줄.이름}=${줄.상태}`).join(" · ");
}
```

```
console.log("처음:", await 상태보기());
```

```
출력      처음: 컨베이어 1호=가동 · 컨베이어 2호=정지 · 프레스 1호=가동 · 프레스
          2호=점검 · 용접로봇 1호=가동 · CNC 선반 1호=고장
```

UPDATE 의 기본

섹션 1

UPDATE 표이름 SET 칸1 = 값1, 칸2 = 값2 WHERE 조건

★ SET 은 쉼표로 여러 칸을 한 번에 바꿉니다. ★ WHERE 는 SELECT 의 WHERE 와 완전히 똑같습니다. 개념02 에서 배운 것 그대로입니다.

```
const 한건고침 = await db.query(
  "UPDATE 설비 SET 상태 = $1 WHERE 이름 = $2",
  ["점검", "컨베이어 1호"],
);

console.log("바뀐 줄 수:", 한건고침.affectedRows);
```

```
출력      바뀐 줄 수: 1
```

여러 칸을 한 번에 바꿉니다. 지금 값을 계산에 쓸 수도 있습니다.

```
await db.query("UPDATE 설비 SET 상태 = '가동', 점검횟수 = 점검횟수 + 1 WHERE 라인 = 'A'");

const 라인A = await db.query("SELECT 이름, 상태, 점검횟수 FROM 설비 WHERE 라인 = 'A' ORDER BY id");

console.log("라인 A:", 라인A.rows.map((줄) => `${줄.이름}(${줄.상태}/${줄.점검횟수}회)`).join(" · "));
```

```
출력      라인 A: 컨베이어 1호(가동/1회) · 컨베이어 2호(가동/1회)
```

★★ 점검횟수 = 점검횟수 + 1 은 **데이터베이스 안에서** 더합니다. 읽어 와서 자바스크립트로 더한 뒤 다시 쓰면, 그 사이에 남아 더한 값이 사라집니다. 01단원에서 파일로 겪은 그 사고와 같은 모양입니다. 숫자를 늘릴 때는 언제나 이 형태로 쓰세요.

조건에 맞는 줄이 없으면 그냥 0건입니다. 에러가 아닙니다.


```
const 없는것 = await db.query("UPDATE 설비 SET 상태 = '가동' WHERE id = 9999");

console.log("없는 id 를 고치면:", 없는것.affectedRows, "건 (에러 아님)");
```

출력 없는 id 를 고치면: 0 건 (에러 아님)

★★ 그래서 “수정했습니다” 를 띄우기 전에 affectedRows 를 봐야 합니다. 0 이면 그 줄은 없거나, 남이 이미 지웠거나, 조건이 틀린 것입니다.

★★★ WHERE 를 빠뜨리면 전부 바뀝니다

섹션 2

이런 일이 실제로 벌어집니다.

개발자가 SQL 편집기에 이렇게 씁니다.

```
UPDATE 설비 SET 상태 = '정지'
WHERE 라인 = 'C'
```

그런데 실행 버튼을 누를 때 **첫 줄만 선택된 상태**였습니다. 또는 WHERE 를 쓰기 전에 실수로 실행했습니다. 진짜로 해 보겠습니다.

```
const 전체건수 = await db.query("SELECT count(*)::int AS 건수 FROM 설비");

console.log("설비는 모두", 전체건수.rows[0].건수, "대입니다");
```

출력 설비는 모두 6 대입니다

```
const 사고 = await db.query("UPDATE 설비 SET 상태 = '정지'");

console.log("바뀐 줄 수:", 사고.affectedRows);
```

출력 바뀐 줄 수: 6

```
console.log("지금 상태:", await 상태보기());
```

출력 지금 상태: 컨베이어 1호=정지 · 컨베이어 2호=정지 · 프레스 1호=정지 ·
프레스 2호=정지 · 용접로봇 1호=정지 · CNC 선반 1호=정지

★★★ 여섯 대가 전부 ‘정지’ 가 됐습니다.

에러도 경고도 없습니다. SQL 문법상 아무 잘못이 없습니다. “WHERE 를 안 쓰면 전부” 가 규칙이기 때문입니다.

그리고 **되돌릴 수 없습니다**. 바뀌기 전 값이 어디에도 안 남습니다. 컨베이어 1호가 원래 ‘점검’ 이었는지 ‘가동’ 이었는지 아무도 모릅니다.

복구하려면 백업에서 표 전체를 되살려야 하는데, 그 사이에 들어온 정상 데이터도 같이 날아갑니다.

지금 이 어떤 상태였는지 아무도 모르니, 실습을 위해 다시 만들어 둡니다.

```
await db.exec(`
  UPDATE 설비 SET 상태 = '가동' WHERE id IN (1, 3, 5);
  UPDATE 설비 SET 상태 = '정지' WHERE id = 2;
  UPDATE 설비 SET 상태 = '점검' WHERE id = 4;
  UPDATE 설비 SET 상태 = '고장' WHERE id = 6;
`);
```

안 틀리는 습관 세 가지

섹션 3

★ 습관 ① — 같은 WHERE 로 SELECT 를 먼저 돌립니다.

UPDATE 를 쓰기 전에 SELECT 로 바뀌어서 돌려 봅니다. 몇 건이 나오는지 눈으로 확인하고, 그 숫자가 예상과 같을 때만 UPDATE 로 바꿉니다.

```
const 미리보기 = await db.query("SELECT id, 이름 FROM 설비 WHERE 라인 = 'A'");

console.log("바뀔 줄:", 미리보기.rowCount, "건 .", 미리보기.rows.map((줄) => 줄.이름).join(", "));
```

출력 바뀔 줄: 2 건 · 컨베이어 1호, 컨베이어 2호

숫자가 맞으면 그때 UPDATE 로 바꿉니다.

```
const 확인후 = await db.query("UPDATE 설비 SET 상태 = '점검' WHERE 라인 = 'A'");

console.log("실제로 바뀐 줄:", 확인후.affectedRows, "· 미리 본 것과 같은가:",
  확인후.affectedRows === 미리보기.rowCount);
```

출력 실제로 바뀐 줄: 2 · 미리 본 것과 같은가: true

★ 습관 ② — 트랜잭션 안에서 하고, 숫자가 이상하면 되돌립니다.

트랜잭션은 “묶어서 전부 되거나 전부 안 되게” 하는 장치입니다. 자세한 것은 07단원에서 합니다. 여기서는 되돌리기만 씁니다.

```
const 되돌린결과 = await db
  .transaction(async (tx) => {
    const 바뀜 = await tx.query("UPDATE 설비 SET 상태 = '폐기'");

    if (바뀜.affectedRows > 3) {
```

예상보다 많이 바뀌었습니다. 던지면 통째로 되돌아갑니다.

```

throw new Error(`${바뀜.affectedRows}건이나 바뀝니다`);
}

return "커밋했습니다";
})
.catch((에러) => `되돌렸습니다 - ${에러.message}`);

console.log(되돌린결과);

```

출력 되돌렸습니다 - 6건이나 바뀝니다

```

const 폐기건수 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 상태 = '폐기'");

console.log("폐기 상태로 남은 줄:", 폐기건수.rows[0].건수, "건");

```

출력 폐기 상태로 남은 줄: 0 건

★★ 여섯 건이 바뀌었다가 통째로 되돌아왔습니다. 아무 흔적도 안 남았습니다. 운영 데이터베이스에서 손으로 UPDATE 를 칠 때는 반드시 이 형태로 하세요.

```

BEGIN;
UPDATE ...;      ← 몇 건인지 봅니다
ROLLBACK;        ← 이상하면 되돌리고
COMMIT;          ← 맞으면 확정합니다

```

★ 습관 ③ — RETURNING 으로 무엇이 바뀌었는지 눈으로 봅니다. 다음 섹션에서 합니다.

RETURNING 으로 바뀐 것 확인하기

섹션 4

UPDATE 와 DELETE 에도 RETURNING 을 붙일 수 있습니다. 바뀐 줄이 그대로 돌아옵니다.

```

const 바뀐것 = await db.query(`
  UPDATE 설비 SET 점검횟수 = 점검횟수 + 1
  WHERE 라인 = 'B'
  RETURNING id, 이름, 점검횟수
`);

console.log("바뀐 줄들:", 바뀐것.rows.map((줄) => `${줄.이름}→${줄.점검횟수}회`)
  .join(" · "));

```

출력 바뀐 줄들: 프레스 1호→1회 · 프레스 2호→1회

★★ RETURNING 은 **바뀐 뒤의 값**을 줍니다. 바뀌기 전 값이 아닙니다. 바뀌기 전 값이 필요하다면 미리 SELECT 해 두거나, 07단원의 트랜잭션 안에서 처리합니다.

★ 로그를 남길 때 아주 유용합니다. "누가 무엇을 몇 건 바꿨는지" 를 왕복 한 번으로 기록할 수 있습니다.

DELETE 와 TRUNCATE

섹션 5

```
const 지움 = await db.query("DELETE FROM 설비 WHERE 상태 = '고장' RETURNING 이름");

console.log("지운 것:", 지움.affectedRows, "건 .", 지움.rows.map((줄) => 줄.이름).join(", "));
```

출력 지운 것: 1 건 · CNC 선반 1호

★★★ DELETE 도 WHERE 를 빠뜨리면 전부 지웁니다. UPDATE 와 똑같습니다. DELETE FROM 설비 한 줄이면 표가 빕니다.

TRUNCATE 는 표를 통째로 비웁니다. 둘이 어떻게 다른지 재 봅니다.

```
await db.exec(`
  CREATE TABLE 로그1 AS SELECT i AS id, '점검' AS 종류 FROM generate_series(1,
200000) AS i;
  CREATE TABLE 로그2 AS SELECT i AS id, '점검' AS 종류 FROM generate_series(1,
200000) AS i;
`);

const 지우기시작 = performance.now();
const 델리트 = await db.query("DELETE FROM 로그1");
const 델리트ms = performance.now() - 지우기시작;

const 자르기시작 = performance.now();
await db.query("TRUNCATE 로그2");
const 트렁케이트ms = performance.now() - 자르기시작;

console.log(`20만 건 · DELETE ${델리트ms.toFixed(0)} ms / TRUNCATE
${트렁케이트ms.toFixed(0)} ms`);
```

출력 20만 건 · DELETE 107 ms / TRUNCATE 3 ms
값이 다를 수 있음

```
console.log("TRUNCATE 가 더 빠른가:", 트렁케이트ms < 델리트ms);
```

출력 TRUNCATE 가 더 빠른가: true

```
console.log("DELETE 는 몇 건인지 알려 주는가:", 델리트.affectedRows);
```

출력 DELETE 는 몇 건인지 알려 주는가: 200000

디스크 자리도 다릅니다.

```
const 자리 = await db.query(`
  SELECT pg_total_relation_size('로그1')::int AS 델리트후,
         pg_total_relation_size('로그2')::int AS 트렁케이트후
`);

console.log("남은 자리 - DELETE 후:", 자리.rows[0].델리트후, "바이트 / TRUNCATE 후:",
자리.rows[0].트렁케이트후, "바이트");
```

출력 남은 자리 - DELETE 후: 8921088 바이트 / TRUNCATE 후: 8192 바이트
값이 다를 수 있음

```
console.log("DELETE 후에도 자리가 남아 있는가:", 자리.rows[0].델리트후 > 0);
```

출력 DELETE 후에도 자리가 남아 있는가: true

★★ 왜 이렇게 다른가

DELETE 는 줄을 하나씩 '지웠다고 표시' 합니다.

옛 줄이 그대로 남아 있어서 자리를 차지합니다. (VACUUM 이 나중에 치웁니다)
그 대신 WHERE 를 쓸 수 있고, 트랜잭션 안에서 되돌릴 수 있고,
RETURNING 으로 무엇을 지웠는지 받을 수 있습니다.

TRUNCATE 는 파일을 통째로 새로 만듭니다. 그래서 순식간입니다.

그 대신 WHERE 가 없고, RETURNING 도 없습니다. 전부 아니면 아무것도 아닙니다.

★ TRUNCATE ... RESTART IDENTITY 를 쓰면 SERIAL 번호도 1 부터 다시 시작합니다.

그냥 TRUNCATE 만 하면 번호는 이어집니다.

```
const 잘못된시도 = await db.query("TRUNCATE 로그2 RETURNING id").catch((에러) =>
에러);

console.log("TRUNCATE 에 RETURNING:", 잘못된시도.code);
```

출력 TRUNCATE 에 RETURNING: 42601

★ 실무에서는: 시험 데이터를 갈아엎을 때만 TRUNCATE 를 씁니다. 운영 표에는 거의 안 씁니다. 되돌릴 여지가 너무 적습니다.

★ UPSERT — 있으면 고치고 없으면 넣기

섹션 6

이런 요구가 아주 흔합니다.

“설비별 일일 생산량을 기록하는데, 같은 날 두 번 오면 더해 주세요”

이걸 순진하게 짜면 이렇게 됩니다.

① SELECT 로 있는지 본다 ② 있으면 UPDATE, 없으면 INSERT

★★ ① 과 ② 사이에 남이 넣으면 깨집니다. 그리고 왕복이 두 번입니다. Postgres 는 이걸 한 문장으로 합니다.

```
await db.exec(`
  CREATE TABLE 일일생산 (
    설비명 TEXT NOT NULL,
    날짜 DATE NOT NULL,
    수량 INT NOT NULL,
    PRIMARY KEY (설비명, 날짜)
  );
`);

async function 생산기록(설비명, 수량) {
  const 결과 = await db.query(
    `INSERT INTO 일일생산 (설비명, 날짜, 수량)
    VALUES ($1, DATE '2026-08-01', $2)
    ON CONFLICT (설비명, 날짜)
    DO UPDATE SET 수량 = 일일생산.수량 + EXCLUDED.수량
    RETURNING 수량`,
    [설비명, 수량],
  );

  return 결과.rows[0].수량;
}

console.log("첫 기록:", await 생산기록("컨베이어 1호", 100));
```

출력 첫 기록: 100

```
console.log("같은 날 또:", await 생산기록("컨베이어 1호", 30));
```

출력 같은 날 또: 130

```
console.log("또 한 번:", await 생산기록("컨베이어 1호", 5));
```

출력 또 한 번: 135

★ EXCLUDED 가 무엇인가

“넣으려다 부딪힌 그 값들” 을 담고 있는 가상의 줄입니다.

일일생산.수량	→ 이미 표에 들어 있던 값	(100)
EXCLUDED.수량	→ 이번에 넣으려던 값	(30)

그래서 일일생산.수량 + EXCLUDED.수량 이 “쌓기” 가 됩니다. EXCLUDED.수량 만 쓰면 “덮어쓰기” 가 됩니다.

```
await db.query(
  `INSERT INTO 일일생산 (설비명, 날짜, 수량) VALUES ('컨베이어 1호', DATE '2026-08-01', 7)
  ON CONFLICT (설비명, 날짜) DO UPDATE SET 수량 = EXCLUDED.수량`,
);

const 덮어쓴것 = await db.query("SELECT 수량 FROM 일일생산 WHERE 설비명 = '컨베이어 1호'");

console.log("덮어쓰기 뒤:", 덮어쓴것.rows[0].수량);
```

출력 덮어쓰기 뒤: 7

★ DO NOTHING — 부딪히면 그냥 넘어갑니다. 에러도 안 냅니다.

```
const 아무것도안함 = await db.query(
  `INSERT INTO 일일생산 (설비명, 날짜, 수량) VALUES ('컨베이어 1호', DATE '2026-08-01', 999)
  ON CONFLICT DO NOTHING
  RETURNING 수량`,
);

console.log("DO NOTHING 결과:", 아무것도안함.affectedRows, "건 · 값은 그대로", (await db.query("SELECT 수량 FROM 일일생산 WHERE 설비명 = '컨베이어 1호'")).rows[0].수량);
```

출력 DO NOTHING 결과: 0 건 · 값은 그대로 7

★★ 언제 무엇을 쓰나

DO UPDATE 설정값 저장, 일별 집계 쌓기, 외부 시스템에서 받은 데이터 동기화 DO NOTHING 같은 요청이 두 번 와도 한 번만 처리해야 할 때 (중복 방지)

★ 주의: ON CONFLICT 의 괄호에는 **UNIQUE** 나 **PRIMARY KEY** 가 걸린 칸만 쓸 수 있습니다. 그냥 아무 칸이나 쓰면 42P10 에러가 납니다.

UPDATE ... FROM — 다른 표를 보고 고치기

섹션 7

“라인별 기본 상태표대로 설비 상태를 맞춰 주세요” 같은 요구입니다.

```
await db.exec(`
  CREATE TABLE 라인정책 (
    라인      TEXT PRIMARY KEY,
    기본상태  TEXT NOT NULL
  );
```

```
INSERT INTO 라인정책 VALUES ('A', '가동'), ('B', '정지');
`);
```

```
const 정책적용 = await db.query(`
  UPDATE 설비
  SET 상태 = 라인정책.기본상태
  FROM 라인정책
  WHERE 설비.라인 = 라인정책.라인
  RETURNING 설비.이름, 설비.상태
`);
```

```
console.log(
  "정책 적용:",
  정책적용.rows.map((줄) => `${줄.이름}=${줄.상태}`).sort().join(" · "),
);
```

```
출력      정책 적용: 컨베이어 1호=가동 · 컨베이어 2호=가동 · 프레스 1호=정지 ·
          프레스 2호=정지
```

★ FROM 에 다른 표를 적고 WHERE 로 이어 줍니다. JOIN 과 같은 모양입니다. 라인정책에 없는 라인 C 는 안 바뀌었습니다. WHERE 로 이어지지 않았으니까요.

★★ 실수 주의: WHERE 의 이어 주는 조건을 빼먹으면 모든 조합이 만들어져서 **전부 아무 값으로나 바뀝니다**. WHERE 없는 UPDATE 보다 나쁩니다.

소프트 삭제 vs 진짜 삭제

섹션 8

지우는 대신 '지웠다는 표시'만 하는 방법입니다. 설비 표에 만들어 둔 삭제일시 칸을 씁니다.

```
await db.query("UPDATE 설비 SET 삭제일시 = now() WHERE 이름 = $1", ["프레스 2호"]);

const 살아있는것 = await db.query("SELECT 이름 FROM 설비 WHERE 삭제일시 IS NULL ORDER
BY id");
const 전부 = await db.query("SELECT 이름 FROM 설비 ORDER BY id");

console.log("살아 있는 설비:", 살아있는것.rows.length, "대 / 표에 있는 줄:",
전부.rows.length, "줄");
```

```
출력      살아 있는 설비: 4 대 / 표에 있는 줄: 5 줄
```

★★ 트레이드오프

소프트 삭제가 좋은 점

- 실수로 지워도 되살립니다 (삭제일시 = NULL 로)
- 언제 누가 지웠는지 남습니다
- 지워진 설비를 가리키던 점검기록이 깨지지 않습니다

소프트 삭제의 대가

- **** 모든 조회에 `AND 삭제일시 IS NULL` 을 붙여야 합니다.****
한 군데라도 빠뜨리면 지운 설비가 화면에 나옵니다. 이게 제일 자주 나는 버그입니다
- UNIQUE 제약이 깨집니다. 같은 이름을 지웠다 또 만들면 중복이 됩니다
(부분 색인으로 풀 수 있습니다. 06단원에서 다룹니다)
- 데이터가 계속 쌓입니다. 지워도 안 줄어듭니다
- 개인정보는 법적으로 진짜로 지워야 하는 경우가 많습니다

★ 실무 판단 기준

- 되살릴 일이 있는가 → 있으면 소프트 삭제
- 다른 표가 이 줄을 가리키는가 → 가리키면 소프트 삭제
- 개인정보인가 → 그러면 진짜 삭제 (또는 값만 비우기)
- 로그·이력 성격인가 → 애초에 지우지 말고 기간별로 옮기세요

```
const 되살리기 = await db.query(  
  "UPDATE 설비 SET 삭제일시 = NULL WHERE 이름 = $1 RETURNING 이름",  
  ["프레스 2호"],  
);  
  
console.log("되살린 설비:", 되살리기.affectedRows, "건");
```

출력

되살린 설비: 1 건

MySQL 은 여기가 다릅니다

- UPSERT 문법이 INSERT ... ON DUPLICATE KEY UPDATE 입니다 · EXCLUDED 대신 VALUES(칸) 또는 new.칸 을 씁니다 · UPDATE ... FROM 이 없습니다. UPDATE 설비 JOIN 라인정책 ... 형태로 씁니다
- ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 고치고 지우기

무엇 쓰는 법 주의

고치기 UPDATE 표 SET 칸=값 WHERE 조건 ★★★ WHERE 없으면 전부 숫자 늘리기 SET 횟수 = 횟수 + 1 읽어 와서 더하지 마세요 지우기 DELETE FROM 표 WHERE 조건 ★★★ WHERE 없으면 전부 통째로 비우기 TRUNCATE 표 빠르지만 되돌리기 어려움 바뀐 것 보기 ... RETURNING 칸들 바뀐 '뒤' 값입니다 있으면 고치고 없으면 넣기 INSERT ... ON CONFLICT DO UPDATE EXCLUDED = 넣으려던 값 중복이면 넘어가기 ON CONFLICT DO NOTHING 에러 안 남 다른 표 보고 고치기 UPDATE ... FROM ... WHERE 이어 주는 조건 필수 지운 척하기 UPDATE SET 삭제일시 = now() 모든 조회에 조건 추가

DELETE 와 TRUNCATE

DELETE TRUNCATE

WHERE 됩니다

안 됩니다

RETURNING 됩니다

42601 에러

속도(20만 건) 느립니다 수십 배 빠릅니다 디스크 자리 그대로 남습니다 바로 돌려줍니다 되돌리기 트랜잭션 안에서 됩니다 사실상 못 합니다

★★★ 이 파일에서 꼭 가져갈 것 UPDATE 와 DELETE 는 먼저 **SELECT** 로 몇 건인지 확인하고 씁니다. 운영 데이터에 손으로 칠 때는 트랜잭션으로 감쌉니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 2 의 사고를 DELETE 로 재현해 보세요. DELETE FROM 설비 를 트랜잭션 안에서 돌리고 되돌려 보세요.

▫ 직접 해보기 2

점검횟수를 “읽어 와서 +1 해서 쓰기” 로 바꿔 보세요. Promise.all 로 열 번 동시에 부르면 몇이 되나요? SET 점검횟수 = 점검횟수 + 1 로 하면 몇이 되나요?

▫ 직접 해보기 3

일일생산에 DO UPDATE 대신 DO NOTHING 을 쓰면 같은 날 두 번째 기록은 어떻게 되나요? 어느 쪽이 맞는 설계일까요?

▫ 직접 해보기 4

섹션 7 의 UPDATE ... FROM 에서 WHERE 줄을 지워 보세요. (트랜잭션 안에서 하세요!) 몇 건이 바뀌고 값은 어떻게 되나요?

▫ 직접 해보기 5

소프트 삭제한 설비를 빼먹고 조회하는 코드를 하나 만들어 보세요. 그다음 그 조건을 뷰(VIEW)로 감싸면 어떻게 편해질까요?

▫ 직접 해보기 6

TRUNCATE ... RESTART IDENTITY 로 표를 비운 뒤 새로 넣으면 id 가 몇 번부터 시작하나요? 그냥 TRUNCATE 와 비교해 보세요.

자주 하는 실수

이런 실수를 합니다

WHERE 를 안 쓰거나, 쓴 줄이 실행 범위에서 빠짐

SQL 편집기에서 여러 줄 중 일부만 선택해 실행하면 이렇게 됩니다. 습관: UPDATE 를 칠 때 WHERE 를 먼저 쓰고 SET 을 나중에 채우세요.

이런 실수를 합니다

affectedRows 를 안 보고 성공 처리

0건이어도 에러가 아닙니다. "수정되었습니다" 만 뜨고 아무 일도 안 일어납니다.

이런 실수를 합니다

읽어 와서 계산한 뒤 다시 씀

```
const 줄 = await SELECT...; await UPDATE SET 횟수 = 줄.횟수 + 1;
```

동시에 두 번 들어오면 한 번이 사라집니다. SET 횟수 = 횟수 + 1 로 쓰세요.

이런 실수를 합니다

운영 DB 에서 트랜잭션 없이 손으로 UPDATE 를 침

되돌릴 방법이 없습니다. BEGIN 을 먼저 치는 습관을 들이세요.

이런 실수를 합니다

TRUNCATE 를 **DELETE** 대신 습관적으로 씀

빠르다고 좋은 게 아닙니다. WHERE 도 없고 되돌리기도 어렵습니다.

이런 실수를 합니다

소프트 삭제를 도입하고 조회 조건을 빠뜨림

지운 설비가 목록에 다시 나타납니다. 표를 직접 조회하지 말고 삭제일시 IS NULL 을 넣은 뷰를 만들어 그것만 쓰게 하세요.

```
await db.close();
```

한 번에 여러 건 다루기

RUN node 개념05_한번에_여러_건_다루기.js

★★ 이 파일은 25초쯤 걸립니다. 10만 건을 네 가지 방법으로 진짜로 넣습니다.

그중 하나(한 건씩 넣기)가 아주 느려서 그렇습니다. 그게 이 파일의 요점입니다.

지금까지는 몇 건씩 다뤘습니다. 실무에서는 이런 일이 옵니다.

· 옛 시스템에서 설비 이력 200만 건을 옮겨야 합니다 · 매일 새벽에 어제 생산실적 30만 건을 넣습니다 · 잘못 들어간 값 50만 건을 고쳐야 합니다

한 건 넣는 코드를 for 문으로 감싸면 될 것 같습니다. **안 됩니다.** 얼마나 안 되는지 재 보겠습니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();

const 건수 = 100000;

await db.exec(`
  CREATE TABLE 방법1 (id SERIAL PRIMARY KEY, 이름 TEXT NOT NULL, 라인 TEXT NOT NULL);
  CREATE TABLE 방법2 (id SERIAL PRIMARY KEY, 이름 TEXT NOT NULL, 라인 TEXT NOT NULL);
  CREATE TABLE 방법3 (id SERIAL PRIMARY KEY, 이름 TEXT NOT NULL, 라인 TEXT NOT NULL);
  CREATE TABLE 방법4 (id SERIAL PRIMARY KEY, 이름 TEXT NOT NULL, 라인 TEXT NOT NULL);
`);

const 라인들 = ["A", "B", "C"];
```

방법 ①: 한 건씩 10만 번

가장 먼저 떠오르는 방법입니다. 그리고 가장 많이 쓰입니다.

```
const 시작1 = performance.now();

for (let 번호 = 1; 번호 <= 건수; 번호 += 1) {
  await db.query("INSERT INTO 방법1 (이름, 라인) VALUES ($1, $2)", [
    `설비${번호}`,
    라인들[번호 % 3],
  ]);
}

const 걸린1 = performance.now() - 시작1;

console.log(`① 한 건씩 ${건수.toLocaleString()}번 : ${(걸린1 / 1000).toFixed(1)} 초`);
```

출력 ① 한 건씩 100,000번 : 19.4 초
값이 다를 수 있음

방법 ②: 값을 묶어서 한 문장으로

VALUES 뒤에 괄호를 여러 개 이어 붙입니다. 1000건씩 끊어서 보냅니다.

★ 왜 1000건씩 끊나: Postgres 의 파라미터 개수 상한이 65535 개입니다. 칸이 2개면 한 문장에 32767 건까지입니다. 그리고 너무 크게 묶으면 문장을 만드는 쪽이 메모리를 많이 씁니다. 1000 ~ 5000 이 무난합니다.

```
const 묶음크기 = 1000;
const 시작2 = performance.now();

for (let 시작번호 = 1; 시작번호 <= 건수; 시작번호 += 묶음크기) {
  const 값들 = [];
  const 자리표시 = [];

  for (let 자리 = 0; 자리 < 묶음크기; 자리 += 1) {
    const 번호 = 시작번호 + 자리;

    값들.push(`설비${번호}`, 라인들[번호 % 3]);
    자리표시.push(`(${값들.length - 1}, ${값들.length})`);
  }
}
```

★ 이어 붙이는 것은 자리표시(\$1, \$2)뿐입니다. 값은 전부 파라미터로 갑니다.

```
const 문장 = `INSERT INTO 방법2 (이름, 라인) VALUES ${자리표시.join(", ")}`;

await db.query(문장, 값들);
}

const 걸린2 = performance.now() - 시작2;

console.log(`② ${묶음크기}건씩 묶어서 : ${(걸린2 / 1000).toFixed(2)} 초`);
```

출력 ② 1000건씩 묶어서 : 0.70 초
값이 다를 수 있음

방법 ③: 서버 안에서 만들기 (generate_series) —————

값이 규칙적이면 아예 서버가 만들게 합니다. 오가는 데이터가 **문장 하나**뿐입니다.

```
const 시작3 = performance.now();

await db.query(`
  INSERT INTO 방법3 (이름, 라인)
  SELECT '설비' || i, (ARRAY['A','B','C'])[i % 3 + 1]
  FROM generate_series(1, $1) AS i
`, [건수]);

const 걸린3 = performance.now() - 시작3;

console.log(`③ generate_series : ${(걸린3 / 1000).toFixed(2)} 초`);
```

출력 ③ generate_series : 0.31 초
값이 다를 수 있음

방법 ④: COPY —————

COPY는 “표에 데이터를 통째로 부어 넣는” 전용 명령입니다. 한 줄씩 파싱하는 INSERT와 달리 CSV를 그대로 받아 넣습니다.

★ PGlite에서는 파일 대신 '/dev/blob'이라는 특별한 자리를 씁니다. 진짜 서버에서는 파일 경로를 쓰거나, 프로그램에서 스트림으로 보냅니다. (08단원)

```
const 줄들 = [];

for (let 번호 = 1; 번호 <= 건수; 번호 += 1) {
  줄들.push(`설비${번호},${라인들[번호 % 3]}`);
}
```

```
const 시작4 = performance.now();

const 복사결과 = await db.query(
  "COPY 방법4 (이름, 라인) FROM '/dev/blob' WITH (FORMAT csv)",
  [],
  { blob: new Blob([줄들.join("\n") + "\n"]) },
);

const 걸린4 = performance.now() - 시작4;

console.log(`④ COPY : ${((걸린4 / 1000).toFixed(2))} 초 ·
${복사결과.affectedRows.toLocaleString()}건`);
```

출력 ④ COPY : 0.26 초 · 100,000건
값이 다를 수 있음

결과

```
const 센것 = await db.query(`
  SELECT (SELECT count(*)::int FROM 방법1) AS 하나,
         (SELECT count(*)::int FROM 방법2) AS 둘,
         (SELECT count(*)::int FROM 방법3) AS 셋,
         (SELECT count(*)::int FROM 방법4) AS 넷
`);

const 센줄 = 센것.rows[0];

console.log("네 방법 모두 10만 건이 들어갔나:", [센줄.하나, 센줄.둘, 센줄.셋, 센줄.넷].every((수) => 수 === 건수));
```

출력 네 방법 모두 10만 건이 들어갔나: true

```
console.log(`② 는 ① 보다 ${((걸린1 / 걸린2).toFixed(0))} 배 빠릅니다`);
```

출력 ② 는 ① 보다 28 배 빠릅니다
값이 다를 수 있음

```
console.log(`③ 은 ① 보다 ${((걸린1 / 걸린3).toFixed(0))} 배 빠릅니다`);
```

출력 ③ 은 ① 보다 63 배 빠릅니다
값이 다를 수 있음

```
console.log(`④ 는 ① 보다 ${((걸린1 / 걸린4).toFixed(0))} 배 빠릅니다`);
```

출력 ④ 는 ① 보다 75 배 빠릅니다
값이 다를 수 있음

```
console.log("한 건씩이 가장 느린가:", 걸린1 > 걸린2 && 걸린1 > 걸린3 && 걸린1 > 걸린4);
```

출력 한 건씩이 가장 느린가: true

★★★ 왜 이렇게까지 차이가 나는가 — 왕복 횟수 때문입니다.

한 건 넣는 일 자체는 아주 빠릅니다. 0.01밀리초도 안 걸립니다. 시간을 잡아먹는 것은 그 앞뒤에 붙는 고정 비용입니다.

문장을 보낸다 → 서버가 파싱한다 → 계획을 세운다 → 실행한다
→ 결과를 만든다 → 돌려보낸다 → 클라이언트가 받는다

이 고정 비용이 한 번에 0.2밀리초라고 하면, 10만 번이면 그것만 20초입니다. 데이터를 넣는 시간이 아니라 오가는 시간입니다.

★ 여기는 PGlite 라서 네트워크가 아예 없습니다. 그런데도 27배가 났습니다.

진짜 서버가 네트워크 건너에 있으면 왕복 하나가 0.5~2밀리초입니다.
그러면 10만 건에 1~3분입니다. 차이가 훨씬 커집니다.

★ 정리하면

- ① 한 건씩 – 사용자가 화면에서 한 건 넣을 때. 그때만.
- ② 묶음 INSERT – 값이 프로그램 쪽에 있을 때의 기본. 가장 많이 씁니다
- ③ INSERT SELECT – 값이 이미 데이터베이스 안에 있거나 규칙적으로 만들 수 있을 때
- ④ COPY – 파일에서 대량으로 부어 넣을 때. 가장 빠릅니다

COPY 를 조금 더

COPY 는 반대 방향도 됩니다. 표를 CSV 로 뽑아냅니다.

```
const 내보내기 = await db.query("COPY (SELECT 이름, 라인 FROM 방법4 LIMIT 3) TO  
'/dev/blob' WITH (FORMAT csv);");  
const 나온글 = await 내보내기.blob.text();  
  
console.log("COPY 로 뽑은 CSV:", JSON.stringify(나온글));
```

출력 COPY 로 뽑은 CSV: "설비1,B\n설비2,C\n설비3,A\n"

★★ COPY 의 한계

- 한 줄이라도 잘못되면 **전부 실패합니다.** 어느 줄인지는 알려 줍니다
- ON CONFLICT 를 못 씁니다. 중복이 있으면 그냥 터집니다
- 트리거나 기본값 계산이 있으면 INSERT 만큼 느려질 수 있습니다

★ 실무에서는 이렇게 합니다.

- ① 임시 표에 COPY 로 부어 넣고
- ② INSERT ... SELECT ... ON CONFLICT 로 진짜 표에 옮깁니다

이러면 빠르면서 중복도 처리됩니다.

대량 UPDATE 를 한 번에 하면 무슨 일이 하나

10만 건을 한 문장으로 고쳐 봅니다.

```
const 전크기 = await db.query("SELECT pg_total_relation_size('방법3')::int AS 크기");

const 시작UP = performance.now();
const 한번에 = await db.query("UPDATE 방법3 SET 라인 = 'D'");
const 걸린UP = performance.now() - 시작UP;

const 후크기 = await db.query("SELECT pg_total_relation_size('방법3')::int AS 크기");

console.log(`한 번에 ${한번에.affectedRows.toLocaleString()}건 UPDATE :
${걸린UP.toFixed(0)} ms`);
```

출력 한 번에 100,000건 UPDATE : 525 ms
값이 다를 수 있음

```
console.log(`표 크기 : ${전크기.rows[0].크기 / 1024 / 1024}.toFixed(1)} MB →
${후크기.rows[0].크기 / 1024 / 1024}.toFixed(1)} MB`);
```

출력 표 크기 : 7.2 MB → 14.3 MB
값이 다를 수 있음

```
console.log("표가 커졌는가:", 후크기.rows[0].크기 > 전크기.rows[0].크기);
```

출력 표가 커졌는가: true

★★★ 10만 건을 고쳤을 뿐인데 표가 두 배가 됐습니다.

Postgres 는 줄을 고칠 때 그 자리를 덮어쓰지 않습니다. 새 줄을 뒤에 쓰고, 옛 줄에 '죽었다' 고 표시합니다. 그래야 같은 순간에 읽고 있던 다른 사람이 옛 값을 그대로 볼 수 있습니다. (07단원)

그래서 대량 UPDATE 는 이런 일을 함께 일으킵니다.

- 디스크가 갑자기 두 배로 납니다. 꼭 차면 데이터베이스가 멈춥니다
- 트랜잭션이 길어져 그동안 그 표를 건드리는 다른 작업이 밀립니다
- 중간에 실패하면 10만 건이 통째로 되돌아갑니다. 그 시간이 또 걸립니다
- 복제본으로 보내는 로그가 한꺼번에 몰립니다

★★ 그래서 나눠서 합니다.

```

await db.exec("CREATE TABLE 나눠서 AS SELECT i AS id, '설비' || i AS 이름, 'A' AS
라인 FROM generate_series(1, 100000) AS i");
await db.exec("ALTER TABLE 나눠서 ADD PRIMARY KEY (id)");

const 시작나눔 = performance.now();
let 고친수 = 0;
let 번째 = 0;

for (let 시작id = 1; 시작id <= 100000; 시작id += 10000) {
  const 결과 = await db.query(
    "UPDATE 나눠서 SET 라인 = 'D' WHERE id >= $1 AND id < $2",
    [시작id, 시작id + 10000],
  );

  고친수 += 결과.affectedRows;
  번째 += 1;
}

```

진짜 서버에서는 여기서 잠깐 쉽니다. 다른 작업에 자리를 내주려고요.

```
await new Promise((r) => setTimeout(r, 100));
```

```
}
```

```
const 걸린나눔 = performance.now() - 시작나눔;
```

```
console.log(`1만 건씩 ${번째}번에 나눠서 : ${걸린나눔.toFixed(0)} ms ·
${고친수.toLocaleString()}건`);
```

```
출력          1만 건씩 10번에 나눠서 : 529 ms · 100,000건
값이 다를 수 있음
```

```
console.log(`한 번에 : ${걸린UP.toFixed(0)} ms / 나눠서 : ${걸린나눔.toFixed(0)}
ms`);
```

```
출력          한 번에 : 525 ms / 나눠서 : 529 ms
값이 다를 수 있음
```

★ 총 시간은 비슷하거나 나눠서 하는 쪽이 조금 더 걸립니다. 그래도 실무에서는 나누는 쪽이 맞습니다.

한 번에 하면 : 300초 동안 아무도 그 표를 못 씁니다. 실패하면 처음부터입니다 나눠서 하면 : 3초짜리 작업이 100번입니다. 사이사이 남이 끼어들 수 있고,

중간에 실패해도 거기서부터 다시 하면 됩니다

★★ 나누는 기준은 **정렬된 키의 범위**로 잡으세요. LIMIT 만 쓰면 매번 같은 줄을 다시 볼 수 있습니다 (개념02 의 그 문제입니다).

대량 DELETE 도 같습니다. 지울 것을 id 범위로 끊어서 지웁니다.

```
const 시작삭제 = performance.now();
let 지운수 = 0;
let 회차 = 0;

for (;;) {
  const 결과 = await db.query(
    "DELETE FROM 나눠서 WHERE id IN (SELECT id FROM 나눠서 ORDER BY id LIMIT 10000)",
  );

  지운수 += 결과.affectedRows;
  회차 += 1;

  if (결과.affectedRows === 0) break;
}

console.log(`1만 건씩 지우기 : ${회차}번 돌아서 ${지운수.toLocaleString()}건 ·
${(performance.now() - 시작삭제).toFixed(0)} ms`);
```

출력 1만 건씩 지우기 : 11번 돌아서 100,000건 · 608 ms
값이 다를 수 있음

★ 마지막 한 번은 0건이 나옵니다. 그게 '다 지웠다' 는 신호입니다.

★★ 아주 많이 지울 거면 다른 방법이 낫습니다.

- 표의 90% 이상을 지운다 → 남길 것만 새 표에 넣고 표를 바꿔치기
- 기간별로 계속 지운다 → 파티션을 나눠서 통째로 떼어 내기 (10단원)

generate_series 로 시험 데이터 만들기

앞으로 계속 쓸 도구입니다. 이 함수 하나면 시험 데이터를 마음대로 만듭니다.

```
const 맞보기 = await db.query("SELECT i FROM generate_series(1, 5) AS i");

console.log("1부터 5까지:", 맞보기.rows.map((줄) => 줄.i).join(", "));
```

출력 1부터 5까지: 1, 2, 3, 4, 5

```
const 건너뛰기 = await db.query("SELECT i FROM generate_series(0, 20, 5) AS i");

console.log("0부터 20까지 5씩:", 건너뛰기.rows.map((줄) => 줄.i).join(", "));
```

출력 0부터 20까지 5씩: 0, 5, 10, 15, 20

```
const 날짜 = await db.query(`
  SELECT to_char(날, 'YYYY-MM-DD') AS 날짜
  FROM generate_series(DATE '2026-08-01', DATE '2026-08-05', INTERVAL '1 day') AS 날
`);
```

```
console.log("날짜도 됩니다:", 날짜.rows.map((줄) => 줄.날짜).join(", "));
```

```
출력      날짜도 됩니다: 2026-08-01, 2026-08-02, 2026-08-03, 2026-08-04, 2026-08-05
```

★ 이 셋을 섞으면 그럴듯한 시험 데이터가 나옵니다.

```
await db.exec(`
  CREATE TABLE 생산실적 (
    id      SERIAL PRIMARY KEY,
    설비명  TEXT NOT NULL,
    라인    TEXT NOT NULL,
    생산일  DATE NOT NULL,
    수량    INT NOT NULL,
    불량    INT NOT NULL
  );
`);
```

```
await db.query(`
  INSERT INTO 생산실적 (설비명, 라인, 생산일, 수량, 불량)
  SELECT
    '설비' || (i % 50 + 1),
    (ARRAY['A','B','C'])[i % 3 + 1],
    DATE '2026-01-01' + (i % 180),
    800 + (i % 400),
    (i % 17)
  FROM generate_series(1, 50000) AS i
`);
```

```
const 요약 = await db.query(`
  SELECT 라인,
         count(*)::int AS 건수,
         sum(수량)::int AS 총생산,
         sum(불량)::int AS 총불량
  FROM 생산실적
  GROUP BY 라인
  ORDER BY 라인
`);
```

```
for (const 줄 of 요약.rows) {
  console.log(`라인 ${줄.라인} - ${줄.건수}건 · 생산 ${줄.총생산.toLocaleString()} ·
```

```
불량 ${줄.총불량.toLocaleString()}`);
}
```

```
출력      라인 A - 16666건 · 생산 16,657,933 · 불량 133,326
          라인 B - 16667건 · 생산 16,658,600 · 불량 133,333
          라인 C - 16667건 · 생산 16,658,467 · 불량 133,323
```

★ 무작위가 필요하면 `random()` 을 씁니다. 단, 무작위를 쓰면 돌릴 때마다 값이 달라져서 결과를 비교하기 어렵습니다. 그래서 이 자료에서는 `i % 나머지` 를 씁니다. **언제 돌려도 같은 데이터**가 나옵니다.

★ 색인이 있을 때와 없을 때의 차이를 재려면 이런 데이터가 꼭 필요합니다. 06단원에서 이 방식으로 만든 표를 계속 씁니다.

정리 — 한 번에 여러 건

10만 건 넣기 (이 컴퓨터에서 켜 값)

① 한 건씩 10만 번 가장 느립니다 (수십 배) ② 1000건씩 묶은 INSERT ① 보다 수십 배 빠름 · 가장 많이 씁니다 ③ INSERT ... SELECT ② 와 비슷하거나 조금 빠름 · 서버 안에서 만들 때 ④ COPY 가장 빠름 · 파일에서 부어 넣을 때

왜 그런가 : 한 건을 넣는 시간이 아니라 **왕복하는 시간**이 대부분입니다

대량 UPDATE / DELETE

한 번에 하면 표가 두 배로 붓는다 · 그동안 다른 작업이 밀립니다

실패하면 처음부터입니다

나눠서 하면 총 시간은 더 걸리지만 사이사이 남이 끼어들 수 있고

실패해도 이어서 할 수 있습니다

나누는 기준 ★ 정렬된 키의 범위로. LIMIT 만 쓰면 같은 줄을 또 붓는다

`generate_series`

`generate_series(1, 100)` 1부터 100까지 `generate_series(0, 20, 5)` 5씩 건너뛰며
`generate_series(날짜, 날짜, INTERVAL '1 day')` 날짜도 됩니다

직접 해 볼 것

▫ 직접 해보기 1

방법 ② 의 묶음크기를 100 · 1000 · 5000 으로 바꿔 재 보세요. 계속 빨라지나요? 어디서 더 안 빨라지나요?

▫ 직접 해보기 2

방법 ① 을 트랜잭션 하나로 감싸 보세요. db.transaction 안에서 10만 번 넣으면 얼마나 빨라지나요? (힌트: 문장마다 커밋하지 않게 되어 꽤 빨라집니다)

▫ 직접 해보기 3

묶음크기를 40000 으로 해 보세요. 어떤 에러가 나나요? (힌트: 칸이 2개면 파라미터가 8만 개입니다. 상한은 65535 입니다)

▫ 직접 해보기 4

COPY 로 넣을 CSV 한 줄에 칸을 세 개 넣어 보세요. 어떤 에러가 나고, 다른 줄은 들어가나요?

▫ 직접 해보기 5

나눠서 UPDATE 하는 반복문에 100밀리초 쉬는 줄을 넣어 보세요. 전체 시간은 얼마나 늘어나나요? 그만한 값어치가 있을까요?

▫ 직접 해보기 6

generate_series 와 random() 으로 시험 데이터를 만들어 보세요. 두 번 돌리면 결과가 같나요? setseed() 를 쓰면 어떻게 되나요?

자주 하는 실수

이런 실수를 합니다

for 문 안에서 await 로 한 건씩 넣음

이 파일에서 켜 그대로입니다. 10만 건이면 수십 배가 됩니다. 묶어서 보내거나 INSERT ... SELECT 로 바꾸세요.

이런 실수를 합니다

Promise.all 로 한꺼번에 10만 개를 던짐

빨라지지 않습니다. 연결은 하나뿐이라 줄을 서고, 메모리만 터집니다. 동시에 던지는 게 아니라 **한 문장에 묶는 것**이 답입니다.

이런 실수를 합니다

묶음 INSERT 를 문자열 이어 붙이기로 만듦

값이 10만 개면 인젝션 위험도 10만 배입니다. 자리표시(\$1, \$2)만 이어 붙이고 값은 파라미터 배열로 보내세요. (개념03)

이런 실수를 합니다

운영 시간에 대량 UPDATE 를 한 방에 돌림

표가 두 배로 붓고 다른 작업이 전부 밀립니다. 나눠서, 되도록 한가한 시간에 하세요.

이런 실수를 합니다

나눌 때 LIMIT 만 씀

ORDER BY 없이 LIMIT 으로 끊으면 같은 줄을 또 보거나 어떤 줄을 건너뜁니다. id 범위로 끊거나, 마지막으로 본 id 를 기억하세요.

이런 실수를 합니다

대량 작업 뒤에 통계를 안 고침

많이 넣고 지우면 계획을 세우는 근거가 낡습니다. ANALYZE 를 한 번 돌려 주세요. 왜 그게 중요한지는 06단원에서 실행계획을 보며 다룹니다.

```
await db.close();
```

넣고 읽고 고치고 지우기

```
RUN node 연습문제.js
```

// TODO: 를 채우고 다시 돌려세요. 통과하면  로 바뀝니다.

정답과 “왜 그런지” 는 연습문제_정답.js 에 있습니다.

★ 처음에는 15문제 전부  입니다. 정상입니다.


```

import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();

await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    이름        TEXT NOT NULL UNIQUE,
    라인        TEXT NOT NULL,
    상태        TEXT NOT NULL DEFAULT '정지',
    담당자      TEXT,
    도입연도    INT
  );

  INSERT INTO 설비 (이름, 라인, 상태, 담당자, 도입연도) VALUES
    ('컨베이어 1호', 'A', '가동', '김반장', 2019),
    ('컨베이어 2호', 'A', '정지', NULL, 2021),
    ('프레스 1호', 'B', '가동', '이기사', 2018),
    ('프레스 2호', 'B', '점검', '김반장', 2023),
    ('용접로봇 1호', 'C', '가동', '박주임', 2022),
    ('CNC 선반 1호', 'C', '고장', NULL, 2017);

  CREATE TABLE 점검기록 (
    id          SERIAL PRIMARY KEY,
    설비명      TEXT NOT NULL,
    결과        TEXT NOT NULL,
    점검일      DATE NOT NULL
  );

  INSERT INTO 점검기록 (설비명, 결과, 점검일)
  SELECT '설비' || (i % 10 + 1),
         (ARRAY['정상', '주의', '불량'])[i % 3 + 1],
         DATE '2026-06-01' + (i % 30)
  FROM generate_series(1, 300) AS i;

  CREATE TABLE 일일생산 (

```

```

    설비명 TEXT NOT NULL,
    날짜 DATE NOT NULL,
    수량 INT NOT NULL,
    PRIMARY KEY (설비명, 날짜)
);

CREATE TABLE 작업자 (
    사번 TEXT PRIMARY KEY,
    이름 TEXT NOT NULL,
    비밀번호 TEXT NOT NULL
);

INSERT INTO 작업자 VALUES
    ('A1001', '김반장', 'gongjang1'),
    ('A1002', '이기사', 'press2024');
`);

const 결과들 = [];

async function 채점(번호, 제목, 검사) {
    let 통과 = false;

    try {
        통과 = (await 검사()) === true;
    } catch {
        통과 = false;
    }

    결과들.push(통과);
    console.log(`${String(번호).padStart(2)}. ${제목} - ${통과 ? "✅ 통과" : "❌
아직"}`);
}

```

문제 1 — INSERT 한 건

설비 표에 아래 설비를 넣으세요. 칸 이름을 반드시 적으세요. 이름 '포장기 1호' · 라인 'A' · 상태 '가동' · 도입연도 2025

```
const 문제1 = `
  -- TODO: 여기에 INSERT 문을 쓰세요
  SELECT 1
`;

await 채점(1, "INSERT 한 건", async () => {
  await db.query(문제1);
  const 확인 = await db.query("SELECT 라인, 상태, 도입연도 FROM 설비 WHERE 이름 = '포장기 1호'");
  const 줄 = 확인.rows[0];
  return 줄?.라인 === "A" && 줄?.상태 === "가동" && 줄?.도입연도 === 2025;
});
```

출력 1. INSERT 한 건 - ❌ 아직

문제 2 — 여러 건을 한 문장으로

아래 세 대를 **한 문장**으로 넣으세요. 상태는 적지 말고 기본값이 들어가게 하세요. ('검사기 1호','A',2024) ('검사기 2호','B',2024) ('검사기 3호','C',2025)

```
const 문제2 = `
  -- TODO: VALUES 를 세 벌 이어 붙인 INSERT 문 하나를 쓰세요
  SELECT 1
`;

await 채점(2, "여러 건을 한 문장으로", async () => {
  const 넣기 = await db.query(문제2);
  const 확인 = await db.query("SELECT 상태 FROM 설비 WHERE 이름 LIKE '검사기%' ORDER BY 이름");
  return 넣기.affectedRows === 3 && 확인.rows.length === 3 && 확인.rows.every((줄) => 줄.상태 === "정지");
});
```

출력 2. 여러 건을 한 문장으로 - ❌ 아직

문제 3 — RETURNING 으로 id 받기

설비를 넣고 **방금 넣은 줄의 id 와 이름**을 바로 돌려받으세요. 이름과 도입연도는 파라미터 \$1, \$2 로 받습니다. (문자열을 이어 붙이지 마세요)

```
async function 설비추가(이름, 도입연도) {
```

INSERT ... RETURNING 을 써서 { id, 이름 } 을 돌려주세요

```
return null;
}

await 채점(3, "RETURNING 으로 id 받기", async () => {
  const 돌아온것 = await 설비추가("레이저 커터 1호", 2020);
  const 확인 = await db.query("SELECT id FROM 설비 WHERE 이름 = '레이저 커터 1호'");
  return typeof 돌아온것?.id === "number" && 돌아온것.id === 확인.rows[0]?.id;
});
```

출력 3. RETURNING 으로 id 받기 - ❌ 아직

문제 4 — WHERE 와 IN

상태가 '점검' 이거나 '고장' 인 설비의 **이름만** 골라 id 순으로 돌려주세요. IN 을 쓰세요.

```
const 문제4 = `
  -- TODO: 여기에 SELECT 문을 쓰세요
  SELECT 이름 FROM 설비 WHERE false
`;

await 채점(4, "WHERE 와 IN", async () => {
  const 결과 = await db.query(문제4);
  const 이름들 = 결과.rows.map((줄) => 줄.이름).join(",");
  const 칸들 = Object.keys(결과.rows[0] ?? {});
  return 이름들 === "프레스 2호,CNC 선반 1호" && 칸들.length === 1;
});
```

출력 4. WHERE 와 IN - ❌ 아직

문제 5 — NULL 찾기

담당자가 정해지지 않은 설비가 몇 대인지 세어 주세요. 결과 칸 이름은 건수, 타입은 number 여야 합니다.
(힌트: ::int)

```
const 문제5 = `
  -- TODO: 여기에 SELECT 문을 쓰세요
  SELECT -1 AS 건수
`;

await 채점(5, "NULL 찾기", async () => {
  const 결과 = await db.query(문제5);
  const 기대 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 담당자 IS NULL");
  return 결과.rows[0]?.건수 === 기대.rows[0].건수 && 기대.rows[0].건수 > 0;
});
```

출력 5. NULL 찾기 - ❌ 아직

문제 6 — LIKE 로 검색하기

이름에 'cnc' 가 들어간 설비를 **대소문자 상관없이** 찾으세요. 검색어는 파라미터 \$1 로 받습니다.

```
async function 설비검색(검색어) {
```

ILIKE 와 파라미터를 써서 이름 배열을 돌려주세요

```
return [];
}

await 채점(6, "LIKE 로 검색하기", async () => {
  const 찾은것 = await 설비검색("cnc");
  const 없는것 = await 설비검색("zzz");
  return 찾은것.join(",") === "CNC 선반 1호" && 없는것.length === 0;
});
```

출력 6. LIKE 로 검색하기 - ❌ 아직

문제 7 — 정렬과 자르기

도입연도가 오래된 순으로 3대를 뽑으세요. ★ 도입연도가 같아도 순서가 흔들리지 않게 **id** 를 정렬에 덧붙이세요.

```
const 문제7 = `
  -- TODO: 여기에 SELECT 문을 쓰세요
  SELECT 이름 FROM 설비 WHERE false
`;

await 채점(7, "정렬과 자르기", async () => {
  const 결과 = await db.query(문제7);
  const 이름들 = 결과.rows.map((줄) => 줄.이름).join(",");
  return 이름들 === "CNC 선반 1호,프레스 1호,컨베이어 1호" &&
    /ORDER\s+BY[\s\S]*,\s*id/i.test(문제7);
});
```

출력 7. 정렬과 자르기 - ❌ 아직

문제 8 — 별칭과 계산 칸

설비마다 '이름' 과 '사용연수'(2026 - 도입연도)를 뽑되, 사용연수가 많은 순으로 2대만 보여 주세요.
별칭은 사용연수 로 하세요. ★ 도입연도가 비어 있는 설비는 빼세요.

```
const 문제8 = `
  -- TODO: 여기에 SELECT 문을 쓰세요
  SELECT 이름, 0 AS 사용연수 FROM 설비 WHERE false
`;

await 채점(8, "별칭과 계산 칸", async () => {
  const 결과 = await db.query(문제8);
  const 줄들 = 결과.rows.map((줄) => `${줄.이름}:${줄.사용연수}`).join(",");
  return 줄들 === "CNC 선반 1호:9,프레스 1호:8";
});
```

출력 8. 별칭과 계산 칸 - ❌ 아직

문제 9 — UPDATE 와 affectedRows

라인 'C' 설비의 상태를 전부 '점검' 으로 바꾸고, **바뀐 줄 수**를 돌려주세요. ★ WHERE 를 빠뜨리지 마세요.

```
async function 라인점검(라인) {
```

UPDATE 를 실행하고 바뀐 줄 수(number)를 돌려주세요

```
return -1;
}

await 채점(9, "UPDATE 와 affectedRows", async () => {
  const 대상 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 라인 = 'C'");
  const 바뀐수 = await 라인점검("C");
  const 확인 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 라인 = 'C' AND 상태 = '점검'");
  const 다른라인 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 라인 <> 'C' AND 상태 = '점검'");
```

★ WHERE 를 빠뜨렸으면 다른 라인까지 '점검' 이 되어 여기서 걸립니다.

```
return 바뀐수 === 대상.rows[0].건수 && 확인.rows[0].건수 === 대상.rows[0].건수 &&
다른라인.rows[0].건수 === 1;
});
```

출력 9. UPDATE 와 affectedRows - ❌ 아직

문제 10 — DELETE 와 RETURNING

점검기록에서 결과가 '불량' 인 줄을 지우고, **지운 줄 수**를 돌려주세요.

```
async function 불량기록지우기() {
```

DELETE ... RETURNING 을 쓰고 지운 줄 수를 돌려주세요

```
return -1;
}

await 채점(10, "DELETE 와 RETURNING", async () => {
  const 지운수 = await 불량기록지우기();
  const 남은것 = await db.query("SELECT count(*)::int AS 건수 FROM 점검기록 WHERE 결과 = '불량'");
  return 지운수 === 100 && 남은것.rows[0].건수 === 0;
});
```

출력 10. DELETE 와 RETURNING - ❌ 아직

문제 11 — UPSERT

같은 설비·같은 날짜로 여러 번 들어오면 수량을 **더하세요**. 처음이면 그냥 넣습니다. 한 문장으로 하세요.
(힌트: ON CONFLICT · EXCLUDED)

```
async function 생산기록(설비명, 날짜, 수량) {
```

INSERT ... ON CONFLICT ... DO UPDATE 를 써서 최종 수량(number)을 돌려주세요

```
    return -1;
}
```

```
await 채점(11, "UPSERT", async () => {
    const 첫번 = await 생산기록("컨베이어 1호", "2026-08-01", 100);
    const 두번 = await 생산기록("컨베이어 1호", "2026-08-01", 30);
    const 다른날 = await 생산기록("컨베이어 1호", "2026-08-02", 7);
    const 줄수 = await db.query("SELECT count(*)::int AS 건수 FROM 일일생산");
    return 첫번 === 100 && 두번 === 130 && 다른날 === 7 && 줄수.rows[0].건수 === 2;
});
```

출력 11. UPSERT - ❌ 아직

문제 12 — ★ SQL 인젝션 막기

아래 로그인 함수는 문자열을 이어 붙여서 뚫립니다. **파라미터 바인딩**으로 고치세요. 동작(정상 로그인)은 그대로여야 합니다.

원래 코드:

```
const 문장 = `SELECT 이름 FROM 작업자
              WHERE 사번 = '${사번}' AND 비밀번호 = '${비밀번호}'`;
return (await db.query(문장)).rows;
```

```
async function 로그인(사번, 비밀번호) {
```

\$1, \$2 를 써서 안전하게 고치세요. 줄 배열을 돌려줍니다


```
return [];  
}  
  
await 채점(12, "★ SQL 인젝션 막기", async () => {  
  const 정상 = await 로그인("A1001", "gongjang1");  
  const 공격1 = await 로그인("A1001", "' OR '1'='1");  
  const 공격2 = await 로그인("' OR 1=1 --", "아무거나");  
  return 정상.length === 1 && 정상[0].이름 === "김반장" && 공격1.length === 0 &&  
    공격2.length === 0;  
});
```

출력 12. ★ SQL 인젝션 막기 - ❌ 아직

문제 13 — [도전] ★ 허용 목록으로 정렬 막기

정렬 기준과 방향은 파라미터로 못 넣습니다. **허용 목록**으로 고르세요.

- 정렬칸은 '이름' · '라인' · '도입연도' 만 허용합니다 · 방향은 '오름' → ASC, '내림' → DESC 만 허용합니다
- 허용 목록에 없으면 정렬칸은 id, 방향은 ASC 로 떨어뜨립니다 · ★ 같은 값이 있어도 순서가 흔들리지 않게 정렬 끝에 id 를 붙이세요

```
async function 설비목록(정렬칸, 방향) {
```

허용 목록에서 고른 값만 SQL 에 넣으세요. 이름 배열을 돌려줍니다

```
return [];  
}  
  
await 채점(13, "[도전] 허용 목록으로 정렬 막기", async () => {  
  const 뽑기 = async (sql) => (await db.query(sql)).rows.map((줄) => 줄.  
    이름).join(",");  
  
  const 기대이름순 = await 뽑기("SELECT 이름 FROM 설비 ORDER BY 이름, id");  
  const 기대연도역순 = await 뽑기("SELECT 이름 FROM 설비 ORDER BY 도입연도 DESC,  
id");  
  const 기대기본 = await 뽑기("SELECT 이름 FROM 설비 ORDER BY id");  
  
  const 이름순 = (await 설비목록("이름", "오름")).join(",");  
  const 연도역순 = (await 설비목록("도입연도", "내림")).join(",");  
  const 공격 = (await 설비목록("이름; DROP TABLE 설비 --", "; DROP TABLE 설비 --
```

```

    })).join(",");
    const 기본 = (await 설비목록(undefined, undefined)).join(",");
    const 표살아있나 = await db.query("SELECT count(*)::int AS 건수 FROM 설비");

    return (
      이름순 === 기대이름순 &&
      연도역순 === 기대연도역순 &&
      공격 === 기대기본 &&
      기본 === 기대기본 &&
      표살아있나.rows[0].건수 > 0
    );
  });
}

```

출력 13. [도전] 허용 목록으로 정렬 막기 - ❌ 아직

문제 14 — [도전] 키셋 페이지 나누기

OFFSET 을 쓰지 않고 다음 쪽을 가져오세요. 마지막으로 본 id 보다 큰 것 중 앞에서 몇 개를 가져옵니다.

· 마지막id 가 null 이면 처음부터 · id 오름차순, 한 쪽에 개수만큼

· 돌려줄 것: { ids: [...], 다음: 마지막 id 또는 null }

async function 다음쪽(마지막id, 개수) {

WHERE 와 ORDER BY 와 LIMIT 으로 만드세요. OFFSET 은 쓰지 마세요

```

return { ids: [], 다음: null };
}

await 채점(14, "[도전] 키셋 페이지 나누기", async () => {
  const 첫째 = await 다음쪽(null, 5);
  const 둘째 = await 다음쪽(첫째.다음, 5);
  const 겹침 = 첫째.ids.filter((id) => 둘째.ids.includes(id));

  return (
    첫째.ids.length === 5 &&
    둘째.ids.length === 5 &&
    겹침.length === 0 &&
    첫째.다음 === 첫째.ids[4] &&
    둘째.ids[0] > 첫째.ids[4]
  );
});

```

```
);
});
```

출력 14. [도전] 키셋 페이지 나누기 - ❌ 아직

문제 15 — [도전] 대량으로 넣기

점검기록에 1000건을 넣으세요. **왕복은 한 번**이어야 합니다. · 설비명 : '대량설비' || i (i 는 1부터 1000까지)
· 결과 : '정상' · 점검일 : 2026-09-01 부터 하루씩 (i % 30 일을 더하세요)

힌트: generate_series 를 쓰면 INSERT ... SELECT 한 문장으로 끝납니다.

```
async function 대량넣기() {
```

한 문장으로 1000건을 넣고 넣은 줄 수를 돌려주세요

```
return -1;
}

await 채점(15, "[도전] 대량으로 넣기", async () => {
  const 넣은수 = await 대량넣기();
  const 확인 = await db.query(`
    SELECT count(*)::int AS 건수,
           min(점검일) AS 처음,
           max(점검일) AS 마지막
    FROM 점검기록 WHERE 설비명 LIKE '대량설비%'
  `);
  const 줄 = 확인.rows[0];

  return (
    넣은수 === 1000 &&
    줄.건수 === 1000 &&
    줄.처음.toISOString().slice(0, 10) === "2026-09-01" &&
    줄.마지막.toISOString().slice(0, 10) === "2026-09-30"
  );
});
```

출력 15. [도전] 대량으로 넣기 - ❌ 아직

```
console.log(`\n맞힌 문제: ${결과들.filter(Boolean).length} / ${결과들.length}`);
```

출력

맞힌 문제: 0 / 15

```
await db.close();
```

데이터 모델링과 정규화

OPEN 04_데이터_모델링과_정규화

04

```
await db.exec(`
CREATE TABLE 점검대장 (
점검번호      SERIAL PRIMARY KEY,
설비이름      TEXT NOT NULL,
```

01 한 표에 다 넣으면 생기는 일

02 표를 나눕니다

03 외래키로 있습니다

04 관계의 모양

05 언제 정규화를 깨나

- 연습문제

한 표에 다 넣으면 생기는 일

RUN node 개념01_한_표에_다_넣으면_생기는_일.js

02단원에서 표를 만들었고 03단원에서 넣고 읽고 고치고 지웠습니다. 표는 하나였고 하나로 충분해 보였습니다. 이번 단원은 **표를 몇 개로 나눌 것인가**를 다룹니다. 나누는 법부터 배우면 외우기만 하게 됩니다. 그래서 순서를 뒤집습니다. 한 표에 다 넣고, 현장에서 흔한 일을 해 보고, **사고를 냅니다**. 이름은 그 다음입니다.

★ 오늘 낼 사고 네 가지입니다. 이 파일은 **아프기만 하고 끝납니다**.

- 한 사람의 연락처가 ****두 개****가 됩니다
- 새 라인과 새 사람을 ****등록할 방법이 없습니다****
- 점검기록 한 줄을 지웠더니 ****설비가 통째로 사라집니다****
- 같은 내용인데 ****저장 공간을 더 먹습니다****

고치는 것은 개념02가 합니다.

```
import { PGlite } from "@electric-sql/pglite";
const db = await PGlite.create();
```

잘 돌아가는 것처럼 보입니다

섹션 1

어느 공장의 「점검대장」입니다. 점검을 하면 한 줄 적습니다. 엑셀을 그대로 옮겼습니다. 칸이 13개인데, 설비 이야기·라인 이야기·사람 이야기·점검 이야기가 한 줄에 다 있습니다.

```
await db.exec(`
  CREATE TABLE 점검대장 (
    점검번호      SERIAL PRIMARY KEY,
    설비이름      TEXT NOT NULL,
    설비도입년도  INT  NOT NULL,
    라인코드      TEXT NOT NULL,
    라인이름      TEXT NOT NULL,
    라인동        TEXT NOT NULL,
    담당자사번    INT  NOT NULL,
    담당자이름    TEXT NOT NULL,
    담당자연락처  TEXT NOT NULL,
    점검일        DATE NOT NULL,
    결과          TEXT NOT NULL,
    점수          INT,
    점검내용      TEXT NOT NULL
  );
```

```

INSERT INTO 점검대장
    (설비이름, 설비도입년도, 라인코드, 라인이름, 라인동,
     담당자사번, 담당자이름, 담당자연락처, 점검일, 결과, 점수, 점검내용) VALUES
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-01-05', '정상', 92, '벨트 장력 정상'),
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-02-05', '정상', 88, '롤러 윤활 완료'),
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-03-05', '주의', 71, '벨트 마모 관찰'),
    ('프레스 1호', 2019, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-01-12', '정상', 95, '유압 압력 정상'),
    ('프레스 1호', 2019, 'A', '조립1라인', '1동', 103, '박주임', '010-5555-6666',
     '2024-02-12', '정상', 90, '금형 교체'),
    ('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-01-20', '정상', 97, '토치 팁 교체'),
    ('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-02-20', '주의', 68, '아크 불안정'),
    ('검사기 1호', 2018, 'B', '가공2라인', '1동', 103, '박주임', '010-5555-6666',
     '2024-01-25', '정상', 85, '센서 교정'),
    ('포장기 1호', 2022, 'C', '포장3라인', '2동', 104, '최사원', '010-7777-8888',
     '2024-02-28', '정상', 91, '실링 온도 조정');
`;

const 처음줄수 = await db.query(`SELECT count(*)::int AS 줄수 FROM 점검대장`);

console.log("점검대장 줄 수:", 처음줄수.rows[0].줄수);

```

출력 점검대장 줄 수: 9

“1동에서 3월 이전에 한 점검을 담당자 연락처까지 붙여서 보여 주세요” 를 해 봅니다. 표가 하나라서 **SELECT 한 방**이면 끝납니다.

```

const 한방 = await db.query(`
    SELECT 설비이름, 라인이름, 담당자이름, 담당자연락처, 점검일, 결과
    FROM 점검대장
    WHERE 라인동 = '1동' AND 점검일 < DATE '2024-03-01'
    ORDER BY 점검일
    LIMIT 5
`);

for (const 줄 of 한방.rows) {
    const 날짜 = 줄.점검일.toISOString().slice(0, 10);
    console.log(`· ${날짜} · ${줄.설비이름} · ${줄.라인이름} · ${줄.담당자이름} (${줄.
    담당자연락처}) · ${줄.결과}`);
}

```

출력

- 2024-01-05 · 컨베이어 1호 · 조립1라인 · 김반장(010-1111-2222) · 정상
- 2024-01-12 · 프레스 1호 · 조립1라인 · 김반장(010-1111-2222) · 정상
- 2024-01-20 · 용접로봇 1호 · 가공2라인 · 이기사(010-3333-4444) · 정상
- 2024-01-25 · 검사기 1호 · 가공2라인 · 박주임(010-5555-6666) · 정상
- 2024-02-05 · 컨베이어 1호 · 조립1라인 · 김반장(010-1111-2222) · 정상

★ 진심으로 인정하고 갑시다. **이건 편합니다.** 어디를 볼지 고민할 필요가 없고, 여러 표를 잊지 않아도 되고 (05단원의 JOIN 이 필요 없습니다), 엑셀에서 바로 옮겨 답습니다. 그래서 처음엔 대부분 이렇게 시작합니다. 잘못이 아닙니다. 문제는 **이 표를 고치기 시작할 때** 나타납니다.

★ 연락처 하나 바꿨을 뿐인데

섹션 2

김반장이 휴대폰을 바꿨습니다. 010-1111-2222 → 010-9999-0000 사람 한 명의 연락처 하나입니다. 간단해 보입니다. 그런데 먼저 물어봅시다. **그 번호가 표에 몇 번 적혀 있습니까?**

```
const 김반장줄수 = await db.query(`
  SELECT count(*)::int AS 줄수 FROM 점검대장 WHERE 담당자사번 = 101
`);

console.log("김반장(101)의 연락처가 적힌 줄 수:", 김반장줄수.rows[0].줄수);
```

출력 김반장(101)의 연락처가 적힌 줄 수: 3

★ 한 사람의 연락처가 **3곳**에 적혀 있습니다. 점검을 30번 했으면 30곳, 3년 뒤면 300곳입니다. 같은 사실이 여러 곳에 — 이것부터가 사고의 씨앗입니다.

```
const 김반장어디 = await db.query(`
  SELECT 점검번호, 설비이름, 점검일, 담당자연락처
  FROM 점검대장 WHERE 담당자사번 = 101 ORDER BY 점검번호
`);

for (const 줄 of 김반장어디.rows) {
  const 날짜 = 줄.점검일.toISOString().slice(0, 10);
  console.log(`· 점검번호 ${줄.점검번호} · ${줄.설비이름} · ${날짜} · ${줄.
    담당자연락처}`);
}
```

출력

- 점검번호 1 · 컨베이어 1호 · 2024-01-05 · 010-1111-2222
- 점검번호 2 · 컨베이어 1호 · 2024-02-05 · 010-1111-2222
- 점검번호 4 · 프레스 1호 · 2024-01-12 · 010-1111-2222

이제 바꿉니다. ★★★ 여기서 현실적인 사고를 재현합니다. 화면에서 “컨베이어 1호 담당 김반장”을 클릭했고, 프로그램이 그 조건을 그대로 WHERE 에 넣습니다. 개발자는 잘못했다고 생각하지 않습니다. **화면에서 고른 대로 넣었을 뿐입니다.**


```
const 고침 = await db.query(`
  UPDATE 점검대장
  SET 담당자연락처 = '010-9999-0000'
  WHERE 담당자이름 = '김반장' AND 설비이름 = '컨베이어 1호'
`);

console.log("고쳐진 줄 수:", 고침.affectedRows);
```

출력 고쳐진 줄 수: 2

에러는 없습니다. “수정되었습니다” 가 화면에 뜹니다. 아무도 이상하다고 생각하지 않습니다. 그런데 김반장의 연락처를 다시 뽑아 봅시다.

```
const 번호들 = await db.query(`
  SELECT DISTINCT 담당자연락처 FROM 점검대장 WHERE 담당자사번 = 101 ORDER BY
  담당자연락처
`);

console.log("김반장의 연락처:", 번호들.rows.map((줄) => 줄.담당자연락처).join(" /
"));
```

출력 김반장의 연락처: 010-1111-2222 / 010-9999-0000

```
console.log("김반장 번호가 하나인가:", 번호들.rows.length === 1);
```

출력 김반장 번호가 하나인가: false

★★★ 번호가 두 개가 됐습니다. 이제 김반장의 진짜 번호가 무엇인지 데이터베이스는 모릅니다. 둘 다 들어 있고 둘 다 “김반장” 인데, 어느 쪽이 옛날 것인지 알려 주는 칸이 없습니다. ★ “많이 적힌 쪽이 맞겠지” 도 안 됩니다. 세어 보면 이렇습니다.

```
const 표결 = await db.query(`
  SELECT 담당자연락처, count(*)::int AS 줄수
  FROM 점검대장 WHERE 담당자사번 = 101
  GROUP BY 담당자연락처 ORDER BY 줄수 DESC, 담당자연락처
`);

for (const 줄 of 표결.rows) {
  console.log(`· ${줄.담당자연락처} - ${줄.줄수}줄`);
}
```

출력 · 010-9999-0000 - 2줄
 · 010-1111-2222 - 1줄

다수결이면 새 번호가 이깁니다. 그런데 김반장이 컨베이어 점검을 한 번만 했었다면 반대가 됩니다. 다수결이 점검 횟수에 따라 뒤집힙니다. 판단 근거가 못 됩니다.

★★ 진짜 무서운 곳은 여기입니다. 설비가 멈춰서 담당자에게 전화를 걸어야 하는데, 프로그램은 점검기록 아무 줄이나 읽어서 그 번호로 겁니다. **어느 줄을 읽느냐에 따라 옛 번호로 걸립니다.** 전화는 안 받고 대응이 늦어집니다. 원인은 몇 달 전의 UPDATE 한 줄인데, 아무도 거기까지 못 거슬러 올라갑니다. ★ 이런 사고를 **갱신 이상(update anomaly)** 이라고 합니다. 같은 사실이 여러 줄에 흩어져 있어서, 일부만 고쳐지면 앞뒤가 안 맞게 되는 것입니다.

```
await db.query(`UPDATE 점검대장 SET 담당자연락처 = '010-1111-2222' WHERE 담당자사번 = 101`);
```

★ 아직 점검을 안 했으면 등록할 수가 없습니다

섹션 3

공장에 두 가지 일이 생겼습니다. ① 신설4라인(코드 'D', 2동) 이 생겼습니다. 설비도 점검도 아직 없습니다. ② 정신입(사번 105, 010-1212-3434) 이 입사했습니다. 점검은 아직 안 했습니다. 둘 다 **지금 등록해 줘야** 라인 목록에 뜨고 배정도 할 수 있습니다.

```
const 라인목록 = await db.query(`
  SELECT DISTINCT 라인코드, 라인이름, 라인동 FROM 점검대장 ORDER BY 라인코드
`);

for (const 줄 of 라인목록.rows) {
  console.log(`· ${줄.라인코드} · ${줄.라인이름} · ${줄.라인동}`);
}
```

```
출력      · A · 조립1라인 · 1동
          · B · 가공2라인 · 1동
          · C · 포장3라인 · 2동
```

★★ 라인 목록을 뽑는 방법이 **점검대장을 훑는 것밖에 없습니다.** 라인이라는 사실이 따로 없고 점검기록에 묻어 있어서, 점검이 없는 라인은 **존재할 수가 없습니다.** 그래도 억지로 넣어 봅니다. 점검 관련 칸은 값이 없으니 NULL 로 둡니다.

```
try {
  await db.query(`
    INSERT INTO 점검대장
      (설비이름, 설비도입년도, 라인코드, 라인이름, 라인동,
       담당자사번, 담당자이름, 담당자연락처, 점검일, 결과, 점수, 점검내용)
    VALUES (NULL, NULL, 'D', '신설4라인', '2동', NULL, NULL, NULL, NULL, NULL, NULL, NULL, NULL)
  `);
  console.log("신설4라인이 들어갔습니다");
} catch (에러) {
  console.log("신설4라인 등록 실패 -", 에러.code);
}
```

```
출력      신설4라인 등록 실패 - 23502
```

```
console.log("사유:", 에러.message);
```

```
출력      사유: null value in column "설비이름" of relation "점검대장" violates
not-null constraint
```

```
}

try {
  await db.query(`
    INSERT INTO 점검대장
      (설비이름, 설비도입년도, 라인코드, 라인이름, 라인동,
       담당자사번, 담당자이름, 담당자연락처, 점검일, 결과, 점수, 점검내용)
    VALUES (NULL, NULL, NULL, NULL, NULL, 105, '정신입', '010-1212-3434', NULL, NULL,
            NULL, NULL)
  `);
  console.log("정신입이 들어갔습니다");
} catch (에러) {
  console.log("정신입 등록 실패 -", 에러.code);
}
```

```
출력      정신입 등록 실패 - 23502
```

```
}
```

★ 23502 는 NOT NULL 위반입니다. “점검기록이니까 설비 이름은 당연히 있어야지” 하고 NOT NULL 을 걸었고, 그 판단 자체는 옳았습니다. ★★ 문제는 이 표가 **점검기록만 담는 표가 아니라는 것**입니다. 라인 정보도 사람

정보도 여기 있는데, 그것들에는 설비 이름이라는 게 애초에 없습니다.

그럼 NOT NULL 을 풀면 되지 않을까요? 풀어 봅니다.

```
await db.exec(`
  ALTER TABLE 점검대장 ALTER COLUMN 설비이름      DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 설비도입년도 DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 라인코드      DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 라인이름      DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 라인동        DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 담당자사번    DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 담당자이름    DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 담당자연락처  DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 점검일        DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 결과          DROP NOT NULL;
  ALTER TABLE 점검대장 ALTER COLUMN 점검내용      DROP NOT NULL;
`);
```

```
INSERT INTO 점검대장 (라인코드, 라인이름, 라인동) VALUES ('D', '신설4라인', '2동')
RETURNING 점검번호
`);
const 사람넣기 = await db.query(`
  INSERT INTO 점검대장 (담당자사번, 담당자이름, 담당자연락처) VALUES (105, '정신입',
  '010-1212-3434')
  RETURNING 점검번호
  `);

console.log("신설4라인이 받은 점검번호:", 라인넣기.rows[0].점검번호);
```

출력 신설4라인이 받은 점검번호: 12

```
console.log("정신입이 받은 점검번호:", 사람넣기.rows[0].점검번호);
```

출력 정신입이 받은 점검번호: 13

★ 번호가 10, 11 이 아니라 12, 13 입니다. 위에서 실패한 INSERT 두 번이 SERIAL 번호를 이미 먹었습니다. **실패해도 번호는 소비됩니다.** 어쨌든 들어가긴 했습니다. 그런데 대가가 있습니다.

```
const 전체줄수 = await db.query(`SELECT count(*)::int AS 줄수 FROM 점검대장`);
const 진짜점검 = await db.query(`SELECT count(*)::int AS 줄수 FROM 점검대장 WHERE
점검일 IS NOT NULL`);

console.log("점검대장 줄 수:", 전체줄수.rows[0].줄수);
```

출력 점검대장 줄 수: 11

```
console.log("실제 점검 건수:", 진짜점검.rows[0].줄수);
```

출력 실제 점검 건수: 9

```
console.log("줄 수가 곧 점검 건수인가:", 전체줄수.rows[0].줄수 === 진짜점검.rows[0].
줄수);
```

출력 줄 수가 곧 점검 건수인가: false

★★★ 점검이 아닌 줄이 점검대장에 섞였습니다. “이번 달 점검 몇 건 했나” 를 세는 코드는 대부분 `count(*)::int` 한 줄인데, 이제 그 값이 틀립니다. 라인 등록 줄과 사람 등록 줄까지 세니까요. 보고서 숫자가 조용히 부풀어 오릅니다. 고치려면 **모든** 조회에 `WHERE 점검일 IS NOT NULL` 을 붙여야 합니다. 한 군데라도 빠뜨리면 그 화면만 숫자가 다르고, 어디서 빠뜨렸는지는 아무도 모릅니다.

```
const 설비목록 = await db.query(`SELECT DISTINCT 설비이름 FROM 점검대장 ORDER BY
설비이름`);

console.log("설비 이름 목록:", 설비목록.rows.map((줄) => 줄.설비이름 ?? "
(빈칸)").join(" / "));
```

출력 설비 이름 목록: 검사기 1호 / 용접로봇 1호 / 컨베이어 1호 / 포장기 1호 /
프레스 1호 / (빈칸)

★ 설비 목록에도 "(빈칸)" 이 하나 끼었습니다. 화면에 빈 줄이 뜹니다. ★ 이런 사고를 **삽입 이상(insert anomaly)** 이라고 합니다. 같이 있으면 안 되는 것을 같은 표에 넣어서, 한쪽을 등록하려면 없는 다른 쪽을 억지로 만들어 내야 하는 것입니다.

```
await db.query(`DELETE FROM 점검대장 WHERE 점검일 IS NULL`);
```

★ 점검기록 하나 지웠는데 설비가 사라집니다

섹션 4

포장기 1호를 봅시다. 점검기록이 몇 줄일까요.

```
const 포장기줄수 = await db.query(`
  SELECT count(*)::int AS 줄수 FROM 점검대장 WHERE 설비이름 = '포장기 1호'
`);

console.log("포장기 1호의 점검기록 줄 수:", 포장기줄수.rows[0].줄수);
```

출력 포장기 1호의 점검기록 줄 수: 1

딱 한 줄입니다. 그 한 줄이 무엇을 들고 있는지 먼저 봐 둡시다.

```
const 지우기전 = await db.query(`
  SELECT 설비이름, 설비도입년도, 라인코드, 라인이름, 라인동,
         담당자사번, 담당자이름, 담당자연락처
  FROM 점검대장 WHERE 설비이름 = '포장기 1호'
`);
const 그줄 = 지우기전.rows[0];

console.log(`설비 - ${그줄.설비이름} · ${그줄.설비도입년도}년 도입 · ${그줄.라인코드}
라인`);
```

출력 설비 - 포장기 1호 · 2022년 도입 · C라인

```
console.log(`라인 - ${그줄.라인코드} · ${그줄.라인이름} · ${그줄.라인동}`);
```

출력 라인 - C · 포장3라인 · 2동

```
console.log(`사람 - ${그줄.담당자사번} · ${그줄.담당자이름} · ${그줄.담당자연락처}`);
```

```
출력      사람 - 104 · 최사원 · 010-7777-8888
```

이제 사고가 납니다. "2월 28일 포장기 점검, 다른 설비 걸 잘못 적은 겁니다. 지워 주세요." 지우려는 것은 **점검기록 하나**입니다. 설비를 없애자는 게 아닙니다.

```
const 지움 = await db.query(`
  DELETE FROM 점검대장 WHERE 설비이름 = '포장기 1호' AND 점검일 = DATE '2024-02-28'
`);
```

```
console.log("지운 줄 수:", 지움.affectedRows);
```

```
출력      지운 줄 수: 1
```

한 줄만 지웠습니다. 요청대로 했습니다. 확인해 봅시다.

```
const 설비남았나 = await db.query(`SELECT count(*)::int AS 줄수 FROM 점검대장 WHERE
설비이름 = '포장기 1호'`);
const 사람남았나 = await db.query(`SELECT count(*)::int AS 줄수 FROM 점검대장 WHERE
담당자사번 = 104`);
const 라인남았나 = await db.query(`SELECT count(*)::int AS 줄수 FROM 점검대장 WHERE
라인코드 = 'C'`);
```

```
console.log("포장기 1호가 남아 있는 줄 수:", 설비남았나.rows[0].줄수);
```

```
출력      포장기 1호가 남아 있는 줄 수: 0
```

```
console.log("최사원(104) 이 남아 있는 줄 수:", 사람남았나.rows[0].줄수);
```

```
출력      최사원(104) 이 남아 있는 줄 수: 0
```

```
console.log("C라인(포장3라인) 이 남아 있는 줄 수:", 라인남았나.rows[0].줄수);
```

```
출력      C라인(포장3라인) 이 남아 있는 줄 수: 0
```

★★★ 세 가지가 한꺼번에 사라졌습니다. · 포장기 1호라는 설비가 **2022년에 들어왔다** 는 사실 · 최사원 (104) 이라는 사람과 그 연락처 010-7777-8888 · C라인이 '포장3라인' 이고 2동에 있다는 사실 지우려던 것은 점검기록 하나인데, 그 한 줄이 **유일한 보관처**였습니다. ★ 되돌릴 수 없습니다. "몇 년에 들어왔더라" 를 물어볼 곳이 이 표뿐이었으니까요. ★★ 그리고 아무도 못 알아챈다. DELETE 는 "1 row deleted" 라고만 합니다.

최사원이 사라진 것은 다음 달 인원 집계에서 발견됩니다.

★ 이런 사고를 **삭제 이상(delete anomaly)** 이라고 합니다. 지우려던 것과 같이 떨어져 나가면 안 되는 것이 함께 사라지는 것입니다.

```
const 라인다시 = await db.query(`SELECT DISTINCT 라인코드 FROM 점검대장 ORDER BY
라인코드`);

console.log("지금 남은 라인:", 라인다시.rows.map((줄) => 줄.라인코드).join(", "));
```

출력 지금 남은 라인: A, B

회사에 라인은 그대로 세 개인데, 표에는 두 개만 남았습니다.

★ 저장 공간도 낭비합니다

섹션 5

04

지금까지는 정확성 문제였습니다. 이번에는 크기입니다. 점검이 5000건 쌓였다고 하고 통짜 표를 만듭니다. 설비 200대, 라인 4개, 작업자 30명이 돌아가며 나옵니다. ★ 이 SQL 을 지금 이해할 필요는 없습니다. 크기만 보면 됩니다.

```
await db.exec(`
  CREATE TABLE 통짜5000 AS
  SELECT
    n
    AS 점검번호,
    '컨베이어 ' || ((n % 200) + 1) || '호'
    AS 설비이름,
    2010 + ((n % 200) % 15)
    AS 설비도입년도,
    chr(65 + ((n % 200) % 4))
    AS 라인코드,
    '조립' || (((n % 200) % 4) + 1) || '라인'
    AS 라인이름,
    (((n % 200) % 2) + 1) || '동'
    AS 라인동,
    101 + (n % 30)
    AS 담당자사번,
    '작업자' || ((n % 30) + 1) || '번'
    AS 담당자이름,
    '010-' || lpad(((n % 30) + 1)::text, 4, '0') || '-0000' AS 담당자연락처,
    DATE '2024-01-01' + (n % 300)
    AS 점검일,
    CASE WHEN n % 7 = 0 THEN '주의' ELSE '정상' END
    AS 결과,
    60 + (n % 40)
    AS 점수,
    (ARRAY['벨트 장력 정상', '롤러 윤활 완료', '토치 팁 교체', '센서 교정', '금형 교체'])
    [(n % 5) + 1] AS 점검내용
  FROM generate_series(1, 5000) AS n;
`);

const 통짜줄수 = await db.query(`SELECT count(*)::int AS 줄수 FROM 통짜5000`);

console.log("통짜 표 줄 수:", 통짜줄수.rows[0].줄수);
```

출력 통짜 표 줄 수: 5000

```
const 통짜크기 = (await db.query(
  `SELECT pg_total_relation_size('통짜5000')::int AS 바이트`,
)).rows[0].바이트;

const 한줄크기 = (await db.query(
  `SELECT pg_column_size(t.*)::int AS 바이트 FROM 통짜5000 AS t LIMIT 1`,
)).rows[0].바이트;

console.log(`통짜 5000줄 전체: ${통짜크기} 바이트`);
```

출력 통짜 5000줄 전체: 1056768 바이트
값이 다를 수 있음

```
console.log(`한 줄: ${한줄크기} 바이트`);
```

출력 한 줄: 145 바이트
값이 다를 수 있음

같은 내용을 주제별로 나눠 담으면 얼마나 될까요. 나누는 방법은 개념02에서 합니다. 여기서는 크기만 재 봅니다.

```
await db.exec(`
  CREATE TABLE 나눔_라인 AS
  SELECT chr(65 + (n - 1)) AS 라인코드, '조립' || n || '라인' AS 이름, (((n - 1) %
  2) + 1) || '동' AS 동
  FROM generate_series(1, 4) AS n;

  CREATE TABLE 나눔_설비 AS
  SELECT n AS 설비번호, '컨베이어 ' || n || '호' AS 이름,
         chr(65 + ((n - 1) % 4)) AS 라인코드, 2010 + ((n - 1) % 15) AS 도입년도
  FROM generate_series(1, 200) AS n;

  CREATE TABLE 나눔_작업자 AS
  SELECT 100 + n AS 사번, '작업자' || n || '번' AS 이름,
         '010-' || lpad(n::text, 4, '0') || '-0000' AS 연락처,
         chr(65 + ((n - 1) % 4)) AS 소속라인
  FROM generate_series(1, 30) AS n;

  CREATE TABLE 나눔_점검 AS
  SELECT n AS 점검번호, ((n % 200) + 1) AS 설비번호,
         DATE '2024-01-01' + (n % 300) AS 점검일,
         CASE WHEN n % 7 = 0 THEN '주의' ELSE '정상' END AS 결과,
         60 + (n % 40) AS 점수, 101 + (n % 30) AS 담당사번,
         (ARRAY['벨트 장력 정상', '롤러 윤활 완료', '토치 팁 교체', '센서 교정', '금형
  교체'][(n % 5) + 1]) AS 점검내용
  FROM generate_series(1, 5000) AS n;
```



```
`);

const 나눔크기 = (await db.query(`
  SELECT ( pg_total_relation_size('나눔_라인')
    + pg_total_relation_size('나눔_설비')
    + pg_total_relation_size('나눔_작업자')
    + pg_total_relation_size('나눔_점검') )::int AS 바이트
`)).rows[0].바이트;

console.log(`나눠 담은 쪽 전체: ${나눔크기} 바이트`);
```

출력 나눠 담은 쪽 전체: 589824 바이트
값이 다를 수 있음

```
console.log(`줄어든 양: ${통짜크기 - 나눔크기} 바이트`);
```

출력 줄어든 양: 466944 바이트
값이 다를 수 있음

```
console.log("나눈 쪽이 더 작은가:", 나눔크기 < 통짜크기);
```

출력 나눈 쪽이 더 작은가: true

```
const 줄인비율 = Math.round(((통짜크기 - 나눔크기) / 통짜크기) * 100);

console.log(`통짜 대비 ${줄인비율}% 줄었습니다`);
```

출력 통짜 대비 44% 줄었습니다
값이 다를 수 있음

★ 방금 %를 하나만 썼는데 그대로 나왔습니다. 인자가 하나뿐이면 그렇습니다. 인자를 하나 더 붙이면 첫 인자가 printf 서식이 되어 %%를 써야 합니다.

```
console.log(`통짜 대비 ${줄인비율}%% 줄었습니다 -`, "나눈 쪽 승");
```

출력 통짜 대비 44% 줄었습니다 - 나눈 쪽 승
값이 다를 수 있음

★★ 왜 줄어들까요. 통짜 표는 '컨베이어 137호'라는 글자를 그 설비가 나올 때마다 들고 있습니다. 나눠 담으면 그 이름은 **200줄에 한 번씩만** 있습니다. 같은 사실을 몇 번 적었느냐의 차이입니다. ★★★ 그리고 낭비되는 것은 **디스크만이 아닙니다**. · 메모리 — 조회할 때 읽어 올리는 양이 그만큼 큼 · 백업 — 매일 뜨는 백업이 매일 그만큼 큼 · 네트워크 — 서버에서 앱으로 보내는 양이 그만큼 큼 · 캐시 — 같은 메모리에 담을 수 있는 줄 수가 그만큼 줄어들습디다 디스크는 요즘 쌉니다. 하지만 **메모리와 네트워크는 안쌉니다**. ★ "그래도 통짜가 조회는 빠르지 않나요?" — 항상 그렇지는 않습니다. 읽을 바이트가 많으면 더 많은 쪽(page)을 읽습니다. 무엇이 빠르지는 **06단원**에서 재 봅니다.

여기까지 오면서 사고를 세 번 냈습니다. 각각 이름이 있고, 셋을 묶어서 **이상(anomaly)** 이라고 부릅니다.

```
const 이상세가지 = [
  ["갱신 이상", "김반장 연락처가 두 개가 됐습니다", "같은 사실이 3줄에 흩어져 있어서"],
  ["삽입 이상", "신설4라인과 정신입을 못 넣었습니다", "점검 없이는 라인도 사람도 존재할 수 없어서"],
  ["삭제 이상", "점검 1줄 지웠더니 설비가 사라졌습니다", "그 줄이 설비 정보의 유일한 보관처라서"],
];

for (const [이름, 무슨일, 왜] of 이상세가지) {
  console.log(`· ${이름} - ${무슨일} (${왜})`);
}
```

출력	· 갱신 이상 - 김반장 연락처가 두 개가 됐습니다 (같은 사실이 3줄에 흩어져 있어서) · 삽입 이상 - 신설4라인과 정신입을 못 넣었습니다 (점검 없이는 라인도 사람도 존재할 수 없어서) · 삭제 이상 - 점검 1줄 지웠더니 설비가 사라졌습니다 (그 줄이 설비 정보의 유일한 보관처라서)
----	--

★★★ 뿌리 원인은 하나입니다 — **한 줄에 서로 다른 주제가 섞여 있어서.**

점검대장 한 줄에는 네 가지 이야기가 들어 있습니다.

설비 이야기	- 설비이름 · 설비도입년도
라인 이야기	- 라인코드 · 라인이름 · 라인동
사람 이야기	- 담당자사번 · 담당자이름 · 담당자연락처
점검 이야기	- 점검일 · 결과 · 점수 · 점검내용

이 넷은 **각자 생기고 각자 없어집니다**. 라인은 설비보다 먼저 생기고, 사람은 점검보다 먼저 입사하고, 설비는 점검기록이 지워져도 남아 있어야 합니다. 시점이 다른 것들을 한 줄에 묶어 댔으니 하나를 넣으려면 다른 것이 필요하고, 하나를 지우면 다른 것이 딸려 나갑니다. ★ 그래서 **주제별로 나눕니다**. 이것을 **정규화(normalization)** 라고 합니다. ★★ 오늘은 여기까지입니다. 다음 파일(개념02)에서 이 표를 실제로 나눕니다.

그리고 나눈 뒤에 오늘 낸 사고 세 개를 다시 일으켜 봅니다. 안 납니다.

MySQL 은 여기가 다릅니다 —————

· 파라미터가 \$1 이 아니라 ? 입니다 · SERIAL 대신 AUTO_INCREMENT 입니다 · 크기는 pg_total_relation_size 대신 information_schema.TABLES 로 봅니다 ★ 이상(anomaly) 이 생기는

원리는 **완전히 똑같습니다**. DBMS 문제가 아닙니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 한 표에 다 넣으면 생기는 세 가지 이상

이상 무엇이 문제인가 어디서 풀리나

갱신 이상 한 사람 연락처가 3줄 → 2줄만 고쳐짐 개념02 (작업자 표로 분리) 삽입 이상 점검이 없으면 라인·사람을 못 넣음 개념02 (라인·작업자 표) 삭제 이상 점검 1줄 지우니 설비·사람·라인이 사라짐 개념02 (설비 표로 분리) 공간 낭비 같은 글자를 5000번 반복 저장 개념02 (참조로 대체)

뿌리 원인 한 줄에 서로 다른 주제가 섞여 있음 04단원 전체

★ 오늘 배운 것은 “정규형이 몇 개다”가 아니라 **나누지 않으면 이런 사고가 난다**입니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 2의 UPDATE에서 AND 설비이름 = ‘컨베이어 1호’를 지워 보세요. 몇 줄이 고쳐지나요? 조건을 빠뜨렸는지 어떻게 알아챌 수 있을까요?

▫ 직접 해보기 2

섹션 2의 UPDATE를 WHERE 담당자이름 = ‘김반장’으로 바꿔 보세요. 사번이 아니라 이름으로 찾은 것입니다. 김반장이 두 명이면요?

▫ 직접 해보기 3

섹션 4에서 지운 것이 ‘용접로봇 1호’의 1월 점검이었다면 설비 정보가 사라졌을까요? (힌트: 용접로봇은 점검기록이 몇 줄인가요)

▫ 직접 해보기 4

섹션 5의 5000을 50000으로 바꿔 보세요. 통짜와 나눔의 차이가 10배로 커지나요? 재서 확인하세요.

▫ 직접 해보기 5

종이에 점검대장 한 줄을 적고 칸마다 색을 칠해 보세요. 설비=빨강, 라인=파랑, 사람=초록, 점검=검정. 몇 덩어리로 뭉치나요? 그 덩어리가 개념02의 답입니다.

이런 실수를 합니다

“UPDATE 에 WHERE 를 잘 쓰면 되잖아요”

이번 한 번은 잘 쓰면 됩니다. 그런데 그 표를 고치는 코드가 몇 군데 있나요? 화면, 배치, 엑셀 업로드, 관리자 도구... 전부가 몇 년 동안 한 번도 안 틀려야 합니다. ★ 사람이 조심해서 지켜야 하는 규칙은 **언젠가 깨집니다**. 구조가 지켜 주는 규칙만 안 깨집니다.

이런 실수를 합니다

“우리는 리뷰를 하니까 괜찮습니다”

섹션 2 의 UPDATE 를 다시 보세요. 화면에서 고른 조건을 그대로 넣은 코드입니다. **읽어도 이상하지 않습니다**. 이상한 것은 코드가 아니라 표의 생김새입니다.

이런 실수를 합니다

“통짜가 빠르니까 일부러 그렇게 둔 겁니다”

반은 맞습니다. 일부러 합치는 것을 **역정규화(denormalization)** 라 하고 실제로 씁니다. ★★ 하지만 그건 **먼저 나눠 보고, 재 보고, 느려서** 하는 것입니다. 안 나눠 본 채로 “빠를 것 같아서” 두는 것은 그냥 안 나눈 것입니다.

이런 실수를 합니다

COUNT(*) 를 그냥 쓰기

이 파일은 전부 `count(*)::int` 로 썼습니다. PGlite 의 `COUNT(*)` 는 값이 작으면 `number`, 안전정수를 넘으면 `bigint` 를 줍니다. ★★ 개발할 때는 `number` 로 오다가 운영에서 데이터가 쌓이면 `bigint` 로 바뀌어 `JSON.stringify` 가 터집니다. **테스트로는 절대 안 잡힙니다**.

이런 실수를 합니다

NOT NULL 을 풀어서 해결했다고 생각하기

섹션 3 에서 풀었더니 들어가긴 했고, 점점 건수가 늘어졌습니다. ★ NOT NULL 이 막은 것은 **버그가 아니라 설계 문제의 신호**였습니다. 제약을 풀어서 신호를 꺾을 뿐 문제는 그대로이고, 다음에는 더 조용한 곳에서 터집니다.

이런 실수를 합니다

console.log 에서 % 가 사라지거나 두 개가 됨

규칙이 **인자를 몇 개 넘겼느냐**로 갈립니다. · 인자가 하나 — 서식 해석을 안 합니다. “40%” → 40%, “40%” → 40%. 인자가 둘 이상 — 첫 인자를 서식으로 보냅니다. % 가 % 한 개로 줄고 %s %d %i %j %o 는 뒤 인자로 바뀝니다. ★ “무조건 %%” 도 틀리고 “그냥 %” 도 틀립니다. 섹션 5 의 두 줄을 다시 보세요.

이런 실수를 합니다

“정규화는 이론이고 실무는 다르다”

오늘 이론을 한 줄도 안 배웠습니다. 사고만 냈습니다. 정규화는 시험 문제가 아니라 **위 세 가지 사고를 안 나게 하는 방법**입니다.

```
await db.close();
```

표를 나눅니다

RUN node 개념02_표를_나눅니다.js

개념01에서 「점검대장」 한 표에 전부 집어넣었고, 사고가 셋 났습니다.

갱신 이상 — 김반장 연락처를 고치려고 여러 줄을 고쳐야 했습니다 삽입 이상 — 점검을 안 한 신설4라인은 적을 곳이 아예 없었습니다 삭제 이상 — 포장기 1호의 점검기록 한 줄을 지웠더니 설비가 통째로 사라졌습니다

이 파일은 그 표를 **단계적으로 쪼갭니다**. ★★★ 단계마다 “이 이상이 사라집니다” 라고 **말로 하지 않습니다**. 개념01과 **똑같은 사고를 다시 일으켜 보고**, 안 난다는 것을 숫자로 찍습니다. ★ 이름을 먼저 외우지 마세요. 사고를 먼저 보고, 그 다음에 이름을 붙입니다.

```
import { PGlite } from "@electric-sql/pglite";
const db = await PGlite.create();
```

개념01의 통째 표입니다. “나누기 전 / 나눈 뒤” 를 나란히 재야 하니 끝까지 남겨 둡니다.

```
await db.exec(`
  CREATE TABLE 점검대장 (
    점검번호 SERIAL PRIMARY KEY,
    설비이름 TEXT NOT NULL, 설비도입년도 INT NOT NULL,
    라인코드 TEXT NOT NULL, 라인이름 TEXT NOT NULL, 라인동 TEXT NOT NULL,
    담당자사번 INT NOT NULL, 담당자이름 TEXT NOT NULL, 담당자연락처 TEXT NOT NULL,
    점검일 DATE NOT NULL, 결과 TEXT NOT NULL, 점수 INT, 점검내용 TEXT NOT NULL
  );
  INSERT INTO 점검대장
    (설비이름, 설비도입년도, 라인코드, 라인이름, 라인동,
     담당자사번, 담당자이름, 담당자연락처, 점검일, 결과, 점수, 점검내용) VALUES
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-01-05', '정상', 92, '벨트 장력 정상'),
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-02-05', '정상', 88, '롤러 윤활 완료'),
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-03-05', '주의', 71, '벨트 마모 관찰'),
    ('프레스 1호', 2019, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-01-12', '정상', 95, '유압 압력 정상'),
    ('프레스 1호', 2019, 'A', '조립1라인', '1동', 103, '박주임', '010-5555-6666',
     '2024-02-12', '정상', 90, '금형 교체'),
    ('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-01-20', '정상', 97, '토치 팁 교체'),
    ('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
```

```
( '검사기 1호', 2018, 'B', '가공2라인', '1동', 103, '박주임', '010-5555-6666',
  '2024-01-25', '정상', 85, '센서 교정'),
  ( '포장기 1호', 2022, 'C', '포장3라인', '2동', 104, '최사원', '010-7777-8888',
  '2024-02-28', '정상', 91, '실링 온도 조정')
);
```

나누기 전에 — 무엇을 보고 나누나

섹션 1

★ 기준은 “이론” 이 아닙니다. 질문 하나입니다.

이 표의 **한 줄이 무엇 하나를 설명하고 있는가?**

```
const 줄수 = await db.query("SELECT count(*)::int AS 값 FROM 점검대장");
console.log("점검대장 줄 수:", 줄수.rows[0].값);
```

출력 점검대장 줄 수: 9

```
const 칸수 = await db.query("SELECT count(*)::int AS 값 FROM
information_schema.columns WHERE table_name='점검대장'");
console.log("점검대장 칸 수:", 칸수.rows[0].값);
```

출력 점검대장 칸 수: 13

이 13개 칸을 “무엇을 설명하는 칸인가” 로 묶으면 이렇게 갈립니다.

```
const 주제별칸 = [
  ["설비", "설비이름 · 설비도입년도"],
  ["라인", "라인코드 · 라인이름 · 라인동"],
  ["사람", "담당자사번 · 담당자이름 · 담당자연락처"],
  ["점검", "점검번호 · 점검일 · 결과 · 점수 · 점검내용"],
];
for (const [주제, 칸들] of 주제별칸) console.log(`· ${주제} - ${칸들}`);
```

출력 · 설비 - 설비이름 · 설비도입년도
· 라인 - 라인코드 · 라인이름 · 라인동
· 사람 - 담당자사번 · 담당자이름 · 담당자연락처
· 점검 - 점검번호 · 점검일 · 결과 · 점수 · 점검내용

```
console.log("한 줄이 설명하는 주제 개수:", 주제별칸.length);
```

출력 한 줄이 설명하는 주제 개수: 4

★★ 한 줄이 네 가지를 동시에 설명합니다. 그래서 설비 이름 하나만 고치려 해도 점검 줄까지 건드리게 됩니다. 나눌 자리는 이 경계입니다.

함수 종속

★ 무엇을 알면 무엇이 정해지는가. 이게 전부입니다. 설비이름을 알면 도입년도가 정해집니다 → 설비이름 → 도입년도. 기호 놀이 대신 **데이터로 확인**합니다. 왼쪽으로 묶어 오른쪽이 몇 가지인지 세면 됩니다.

```
const 설비종속 = await db.query(`
  SELECT 설비이름, count(DISTINCT 설비도입년도)::int AS 가짓수
  FROM 점검대장 GROUP BY 설비이름 ORDER BY 설비이름
`);
for (const 줄 of 설비종속.rows) console.log(`${줄.설비이름} → 도입년도 ${줄.가짓수}
가지`);
```

출력 검사기 1호 → 도입년도 1가지
 용접로봇 1호 → 도입년도 1가지
 컨베이어 1호 → 도입년도 1가지
 포장기 1호 → 도입년도 1가지
 프레스 1호 → 도입년도 1가지

```
console.log("설비이름을 알면 도입년도가 하나로 정해지나:", 설비종속.rows.every((줄) => 줄.가짓수 === 1));
```

출력 설비이름을 알면 도입년도가 하나로 정해지나: true

반대로 **안 정해지는** 것도 봐야 합니다. 라인코드로 묶어 봅니다.

```
const 라인종속 = await db.query(`
  SELECT 라인코드,
         count(DISTINCT 라인이름)::int AS 라인이름가짓수,
         count(DISTINCT 설비이름)::int AS 설비이름가짓수
  FROM 점검대장 GROUP BY 라인코드 ORDER BY 라인코드
`);
for (const 줄 of 라인종속.rows) console.log(`${줄.라인코드} → 라인이름 ${줄.
라인이름가짓수}가지 · 설비이름 ${줄.설비이름가짓수}가지`);
```

출력 A → 라인이름 1가지 · 설비이름 2가지
 B → 라인이름 1가지 · 설비이름 2가지
 C → 라인이름 1가지 · 설비이름 1가지

```
console.log("라인코드 → 라인이름 인가:", 라인종속.rows.every((줄) => 줄.
라인이름가짓수 === 1));
```

출력 라인코드 → 라인이름 인가: true

```
console.log("라인코드 → 설비이름 인가:", 라인종속.rows.every((줄) => 줄.
설비이름가짓수 === 1));
```


출력 라인코드 → 설비이름 인가: false

★ A라인에 설비가 두 가지라 “라인코드 → 설비이름” 은 성립하지 않습니다. 성립하는 것만 모으면 그게 나눌 표의 밑그림입니다.

설비번호 → 설비이름, 도입년도 / 라인코드 → 라인이름, 라인동 / 사번 → 이름, 연락처

★ 제1정규형 — 한 칸에 여러 값을 넣지 않기

섹션 2

“컨베이어는 A라인에도 B라인에도 걸쳐 있어요. 둘 다 적어 주세요.” 가장 쉬운 답은 콤마로 잇는 것입니다. 실제로 아주 많이 봅니다.

```
await db.exec(`
  CREATE TABLE 설비_나뿔 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL, 라인목록 TEXT NOT NULL);
  INSERT INTO 설비_나뿔 VALUES
    (1, '컨베이어', 'A,B'), (2, '프레스', 'B'), (3, '용접로봇', 'A,BC'), (4, '검사기', 'BC'),
    (5, '포장기', 'C');
`);
```

★ ‘BC’ 는 오타가 아닙니다. **BC 라인**이라는 다른 라인입니다. 현장 코드에는 혼합니다. 이제 “B라인 설비를 찾아 주세요” 를 해 봅니다. 칸에 값이 여러 개니 = 로는 못 찾습니다.

```
const 오탐 = await db.query("SELECT 번호, 이름, 라인목록 FROM 설비_나뿔 WHERE
라인목록 LIKE '%B%' ORDER BY 번호");
for (const 줄 of 오탐.rows) console.log(`걸림: ${줄.번호}번 ${줄.이름} - 라인목록
'${줄.라인목록}'`);
```

출력 걸림: 1번 컨베이어 - 라인목록 'A,B'
 걸림: 2번 프레스 - 라인목록 'B'
 걸림: 3번 용접로봇 - 라인목록 'A,BC'
 걸림: 4번 검사기 - 라인목록 'BC'

```
const 진짜B = 오탐.rows.filter((줄) => 줄.라인목록.split(",").includes("B"));
const 가짜B = 오탐.rows.filter((줄) => !줄.라인목록.split(",").includes("B"));

console.log("LIKE 로 걸린 개수:", 오탐.rows.length);
```

출력 LIKE 로 걸린 개수: 4

```
console.log("실제 B라인 개수:", 진짜B.length);
```

출력 실제 B라인 개수: 2

```
console.log("B라인이 아닌데 걸린 번호:", 가짜B.map((줄) => 줄.번호).join(", "));
```

출력 B라인이 아닌데 걸린 번호: 3, 4

★★★ 3번('A,BC') 과 4번('BC') 은 **B라인이 아닌데 걸렸습니다**. LIKE '%B%' 는 "B 라는 글자가 어딘가에 있으면" 입니다. 'BC' 안에도 B 가 있습니다. **한 칸에 값을 여러 개 넣은 순간부터 정확히 찾을 방법이 없어진 것**입니다. 에러도 안 납니다. 틀린 목록이 조용히 나옵니다. ★ 그 밖에 못 하는 것 — 개수를 못 세고, 정렬을 못 하고, 외래키를 못 겁니다. 걸어 봅니다.

```
await db.exec(`
  CREATE TABLE 라인_간이 (라인코드 TEXT PRIMARY KEY);
  INSERT INTO 라인_간이 VALUES ('A'), ('B'), ('BC'), ('C');
`);
try {
  await db.exec("ALTER TABLE 설비_나뭇 ADD FOREIGN KEY (라인목록) REFERENCES 라인_
간이(라인코드);");
  console.log("외래키가 걸렸습니다");
} catch (에러) {
  console.log("외래키 걸기 실패:", 에러.code);
}
```

출력 외래키 걸기 실패: 23503

'A,B' 라는 라인코드는 라인 표에 없으니 거부당합니다. ★★ 즉 **없는 라인을 적어도 아무도 못 막는 상태**입니다. 고치는 법은 간단합니다. **칸을 늘리지 말고 줄을 늘립니다**.

```
await db.exec(`
  CREATE TABLE 설비라인 (
    설비번호 INT NOT NULL,
    라인코드 TEXT NOT NULL REFERENCES 라인_간이(라인코드),
    PRIMARY KEY (설비번호, 라인코드)
  );
  INSERT INTO 설비라인 VALUES (1, 'A'), (1, 'B'), (2, 'B'), (3, 'A'), (3, 'BC'), (4, 'BC'),
(5, 'C');
`);

const 정확 = await db.query("SELECT 설비번호 FROM 설비라인 WHERE 라인코드 = $1 ORDER
BY 설비번호", ["B"]);
console.log("라인코드 = 'B' 로 걸린 번호:", 정확.rows.map((줄) => 줄.
설비번호).join(", "));
```

출력 라인코드 = 'B' 로 걸린 번호: 1, 2

```
console.log("오탐이 사라졌나:", 정확.rows.length === 진짜B.length);
```

출력 오탐이 사라졌나: true

LIKE 로는 못 하던 **개수 세기**도 이제 됩니다.

```
const 라인개수 = await db.query("SELECT 설비번호, count(*)::int AS 라인수 FROM 설비라인 GROUP BY 설비번호 ORDER BY 설비번호");
for (const 줄 of 라인개수.rows) console.log(`설비 ${줄.설비번호}번 → 라인 ${줄.라인수}개`);
```

출력	설비 1번 → 라인 2개
	설비 2번 → 라인 1개
	설비 3번 → 라인 2개
	설비 4번 → 라인 1개
	설비 5번 → 라인 1개

★ 여기에 이름을 붙입니다. **한 칸에는 값 하나. 이것을 제1정규형(1NF) 이라고 합니다.** ★ TEXT[] 나 JSONB 는 어떤가 — 예외가 있습니다. 개념05 에서 다룹니다.

★ 제2정규형 — 기본키 일부에만 딸린 칸 떼어내기

섹션 3

★★ 2정규형 문제는 **기본키가 칸 두 개 이상일 때만** 생깁니다. 기본키가 칸 하나면 볼 필요도 없습니다. 이미 2정규형입니다. 기본키가 (설비번호, 점검일) 인 점검 표를 만들어 봅시다.

```
await db.exec(`
  CREATE TABLE 점검_1차 (
    설비번호 INT NOT NULL, 점검일 DATE NOT NULL,
    설비이름 TEXT NOT NULL, 설비도입년도 INT NOT NULL, 결과 TEXT NOT NULL,
    PRIMARY KEY (설비번호, 점검일)
  );
  INSERT INTO 점검_1차 VALUES
    (1, '2024-01-05', '컨베이어 1호', 2015, '정상'), (1, '2024-02-05', '컨베이어
1호', 2015, '정상'),
    (1, '2024-03-05', '컨베이어 1호', 2015, '주의'), (2, '2024-01-12', '프레스
1호', 2019, '정상'),
    (2, '2024-02-12', '프레스 1호', 2019, '정상'), (3, '2024-01-20', '용접로봇
1호', 2021, '정상'),
    (3, '2024-02-20', '용접로봇 1호', 2021, '주의'), (4, '2024-01-25', '검사기
1호', 2018, '정상'),
    (5, '2024-02-28', '포장기 1호', 2022, '정상');
`);
```

칸마다 물어봅니다. **기본키 전체가 있어야 정해지는가?** 결과 — (설비번호, 점검일) 둘 다 있어야 정해집니다. 정상입니다 설비이름 — 설비번호만 알면 정해집니다. 점검일은 필요 없습니다 ★ 설비도입년도 — 역시 설비번호만 알면 정해집니다 ★ 기본키의 **일부에만** 딸린 것. 이걸 **부분 종속** 이라고 합니다.

```
const 부분종속 = await db.query(`
  SELECT 설비번호, count(DISTINCT 설비이름)::int AS 이름가짓수, count(*)::int AS 줄수
  FROM 점검_1차 GROUP BY 설비번호 ORDER BY 설비번호
`);
for (const 줄 of 부분종속.rows) console.log(`설비 ${줄.설비번호}번 → 이름 ${줄.
이름가짓수}가지가 ${줄.줄수}줄에 반복`);
```

출력

```
설비 1번 → 이름 1가지가 3줄에 반복
설비 2번 → 이름 1가지가 2줄에 반복
설비 3번 → 이름 1가지가 2줄에 반복
설비 4번 → 이름 1가지가 1줄에 반복
설비 5번 → 이름 1가지가 1줄에 반복
```

이름은 한 가지인데 3줄에 적혀 있습니다. **같은 사실이 3번** 있는 것입니다. 개념01의 갱신 이상을 재현해 봅니다. 조건을 잘못 잡아 3월 점검 줄을 빠뜨렸습니다.

```
await db.exec(`
  UPDATE 점검_1차 SET 설비이름 = '컨베이어 1호(개조)'
  WHERE 설비번호 = 1 AND 점검일 < DATE '2024-03-01';
`);
const 갈라짐 = await db.query("SELECT count(DISTINCT 설비이름)::int AS 값 FROM 점검
_1차 WHERE 설비번호 = 1");
console.log("나누기 전 - 설비 1번의 이름 가짓수:", 갈라짐.rows[0].값);
```

출력

```
나누기 전 - 설비 1번의 이름 가짓수: 2
```

★★★ 같은 설비의 이름이 **두 가지**가 됐습니다. 어느 쪽이 진짜인지 알 수 없습니다. 되돌리고, 이번엔 제대로 고칩니다. 손대는 줄이 몇 개인지 셉니다.

```
await db.exec("UPDATE 점검_1차 SET 설비이름 = '컨베이어 1호' WHERE 설비번호 = 1;");
const 통짜갱신 = await db.query("UPDATE 점검_1차 SET 설비이름 = $1 WHERE 설비번호 =
$2", ["컨베이어 1호(개조)", 1]);
console.log("나누기 전 - 이름 하나 고치는 데 손댄 줄 수:", 통짜갱신.affectedRows);
```

출력

```
나누기 전 - 이름 하나 고치는 데 손댄 줄 수: 3
```

```
await db.exec("UPDATE 점검_1차 SET 설비이름 = '컨베이어 1호' WHERE 설비번호 = 1;");
```

3줄입니다. 설비 200대에 점검 10만 건이면 수백 줄이 됩니다. **줄이 많아질수록 빠뜨릴 확률이 올라갑니다.** 그러니 부분 종속인 칸을 떼어냅니다.

```
await db.exec(`
  CREATE TABLE 설비_2차 (설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL, 도입년도 INT
  NOT NULL);
  INSERT INTO 설비_2차 SELECT DISTINCT 설비번호, 설비이름, 설비도입년도 FROM 점검
  _1차;

  CREATE TABLE 점검_2차 (
    설비번호 INT NOT NULL REFERENCES 설비_2차(설비번호),
    점검일 DATE NOT NULL, 결과 TEXT NOT NULL,
    PRIMARY KEY (설비번호, 점검일)
  );
  INSERT INTO 점검_2차 SELECT 설비번호, 점검일, 결과 FROM 점검_1차;
`);
```

★ 이제 똑같은 사고를 다시 시도합니다.

```
const 나눈뒤갱신 = await db.query("UPDATE 설비_2차 SET 이름 = $1 WHERE 설비번호 =
  $2", ["컨베이어 1호(개조)", 1]);
console.log("나눈 뒤 - 이름 하나 고치는 데 손댄 줄 수:", 나눈뒤갱신.affectedRows);
```

출력 나눈 뒤 - 이름 하나 고치는 데 손댄 줄 수: 1

```
const 나눈뒤가짓수 = await db.query("SELECT count(DISTINCT 이름)::int AS 값 FROM 설비
  _2차 WHERE 설비번호 = 1");
console.log("나눈 뒤 - 설비 1번의 이름 가짓수:", 나눈뒤가짓수.rows[0].값);
```

출력 나눈 뒤 - 설비 1번의 이름 가짓수: 1

```
console.log("빠뜨릴 줄이 남아 있나:", 나눈뒤갱신.affectedRows > 1);
```

출력 빠뜨릴 줄이 남아 있나: false

★★ 고칠 줄이 **하나뿐**이라 반쪽만 고쳐질 수가 없습니다. “조심해서 고치자”가 아니라 **틀릴 수 있는 구조를 없앤** 것입니다.

★ 이름을 붙입니다.

****기본키의 일부에만 딸린 칸을 떼어낸 상태. 이것을 제2정규형(2NF) 이라고 합니다.****

```
await db.exec("UPDATE 설비_2차 SET 이름 = '컨베이어 1호' WHERE 설비번호 = 1;");
```

```
await db.exec(`
  CREATE TABLE 설비_3차 (
    설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
    라인코드 TEXT NOT NULL, 라인이름 TEXT NOT NULL, 라인동 TEXT NOT NULL
  );
  INSERT INTO 설비_3차 VALUES
    (1, '컨베이어 1호', 'A', '조립1라인', '1동'), (2, '프레스 1호', 'A', '조립1라인', '1동'),
    (3, '용접로봇 1호', 'B', '가공2라인', '1동'), (4, '검사기 1호', 'B', '가공2라인', '1동'),
    (5, '포장기 1호', 'C', '포장3라인', '2동');
`);
```

기본키는 설비번호 하나뿐이니 2정규형 문제는 없습니다. 그런데도 같은 값이 반복됩니다. 라인이름·라인동 — 설비번호가 아니라 **라인코드**를 알아야 정해집니다

즉 이런 사슬입니다. 설비번호 → 라인코드 → 라인이름 키를 거쳐 **한 다리 건너** 딸려 있습니다. 이걸 **이행 종속** 이라고 합니다.

```
const 이행종속 = await db.query(`
  SELECT 라인코드, count(DISTINCT 라인이름)::int AS 이름가짓수, count(*)::int AS 줄수
  FROM 설비_3차 GROUP BY 라인코드 ORDER BY 라인코드
`);
for (const 줄 of 이행종속.rows) console.log(`라인 ${줄.라인코드} → 이름 ${줄.이름가짓수}가지가 ${줄.줄수}줄에 반복`);
```

출력 라인 A → 이름 1가지가 2줄에 반복
 라인 B → 이름 1가지가 2줄에 반복
 라인 C → 이름 1가지가 1줄에 반복

조립1라인의 이름이 '조립1라인(신)' 으로 바뀌었습니다.

```
const 라인갱신전 = await db.query("UPDATE 설비_3차 SET 라인이름 = $1 WHERE 라인코드 = $2", ["조립1라인(신)", "A"]);
console.log("나누기 전 - 라인 이름 하나 고치는 데 손댄 줄 수:", 라인갱신전.affectedRows);
```

출력 나누기 전 - 라인 이름 하나 고치는 데 손댄 줄 수: 2

```
await db.exec("UPDATE 설비_3차 SET 라인이름 = '조립1라인' WHERE 라인코드 = 'A'");
```

설비 5대라 2줄입니다. 200대면 수십 줄입니다. ★ 그리고 여기서는 **삽입 이상**도 납니다. “신설4라인(D, 2동) 을 만들었습니다. 설비는 아직 한 대도 없습니다.”

```
try {
  await db.query("INSERT INTO 설비_3차 (라인코드, 라인이름, 라인동) VALUES ('D', '신설4라인', '2동')");
  console.log("설비 없이 라인만 등록 성공");
} catch (에러) {
  console.log("설비 없이 라인만 등록 실패:", 에러.code);
}
```

출력 설비 없이 라인만 등록 실패: 23502

23502 는 NOT NULL 위반입니다. ★★ 설비가 없는 라인은 **적을 자리 자체가 없습니다**. 그래서 가짜 설비를 만들어 넣는 편법을 쓰게 되고, 그때부터 데이터가 썩습니다.

답은 같습니다. **라인코드가 정하는 칸들을 라인 표로 떼어냅니다**.

```
await db.exec(`
  CREATE TABLE 라인_4차 (라인코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL, 동 TEXT NOT NULL);
  INSERT INTO 라인_4차 SELECT DISTINCT 라인코드, 라인이름, 라인동 FROM 설비_3차;

  CREATE TABLE 설비_4차 (
    설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
    라인코드 TEXT NOT NULL REFERENCES 라인_4차(라인코드)
  );
  INSERT INTO 설비_4차 SELECT 설비번호, 이름, 라인코드 FROM 설비_3차;
`);
```

★ 갱신을 다시 시도합니다.

```
const 라인갱신후 = await db.query("UPDATE 라인_4차 SET 이름 = $1 WHERE 라인코드 = $2", ["조립1라인(신)", "A"]);
console.log("나눈 뒤 - 라인 이름 하나 고치는 데 손댄 줄 수:", 라인갱신후.affectedRows);
```

출력 나눈 뒤 - 라인 이름 하나 고치는 데 손댄 줄 수: 1

```
console.log("손댄 줄이 줄었나:", 라인갱신후.affectedRows < 라인갱신전.affectedRows);
```

출력 손댄 줄이 줄었나: true

```
await db.exec("UPDATE 라인_4차 SET 이름 = '조립1라인' WHERE 라인코드 = 'A'");
```

★ 삽입도 다시 시도합니다. 이번엔 진짜로 넣습니다.

```
const 신설라인 = await db.query("INSERT INTO 라인_4차 (라인코드, 이름, 동) VALUES ($1, $2, $3)", ["D", "신설4라인", "2동"]);
console.log("설비 없는 라인 등록 - 넣은 줄 수:", 신설라인.affectedRows);
```

출력 설비 없는 라인 등록 - 넣은 줄 수: 1

```
const 라인표줄수 = await db.query("SELECT count(*)::int AS 값 FROM 라인_4차");
console.log("라인 표 줄 수:", 라인표줄수.rows[0].값);
```

출력 라인 표 줄 수: 4

```
const D설비 = await db.query("SELECT count(*)::int AS 값 FROM 설비_4차 WHERE 라인코드 = $1", ["D"]);
console.log("D라인에 달린 설비 수:", D설비.rows[0].값);
```

출력 D라인에 달린 설비 수: 0

★★★ 설비가 **0대인 라인**이 아무 문제 없이 등록됐습니다. 삽입 이상이 풀린 것입니다. ★ 이름을 붙입니다.

****키가 아닌 칸에 딸린 칸을 떼어낸 상태. 이것을 제3정규형(3NF) 이라고 합니다.****

한 번 더: 사람도 같은 모양입니다

점검번호 → 담당자사번 → 담당자이름, 담당자연락처 — 역시 이행 종속입니다. 떼어냅니다.

```
await db.exec(`
  CREATE TABLE 작업자_4차 (사번 INT PRIMARY KEY, 이름 TEXT NOT NULL, 연락처 TEXT NOT NULL);
  INSERT INTO 작업자_4차 SELECT DISTINCT 담당자사번, 담당자이름, 담당자연락처 FROM 점검대장;
`);
const 사람수 = await db.query("SELECT count(*)::int AS 값 FROM 작업자_4차");
console.log("작업자 표 줄 수:", 사람수.rows[0].값);
```

출력 작업자 표 줄 수: 4

점검대장 9줄에 사람 이름이 9번 적혀 있었는데 사람 표는 4줄입니다. 한 곳으로 모였습니다.

도착점 — 네 개의 표

섹션 5

★★ 이 네 표가 **05단원의 출발 데이터**입니다. 칸 이름까지 그대로입니다. 05단원을 열면 맨 앞에 이 네 표가 그대로 나옵니다. “아, 04단원에서 만든 그 표구나” 하면 됩니다. 여기서 나눈 결과를 05단원에서 다시 이어서(JOIN) 봅니다.

★ 여기서 **작업자**에 **연락처**가 없는 것이 이상해 보일 수 있습니다. 위 섹션 4에서 만든 **작업자_4차**에는 있었습니다. 뻔 이유는 단순합니다. 05단원이 쓰는 표를 최대한 단순하게 두려는 것뿐입니다. 현장에서는 연락처도 당연히 **작업자** 표에 둡니다. 있어야 할 자리는 거기입니다. ★ 중요한 것은 **“연락처는 사람 표에 한 번만 적는다”**는 것이고, 그건 이미 배웠습니다.

```
await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    동        TEXT NOT NULL
  );
  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드),
    도입년도 INT NOT NULL
  );
  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드)
  );
  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호),
    점검일    DATE NOT NULL,
    결과      TEXT NOT NULL,
    점수      INT,
    담당사번 INT REFERENCES 작업자(사번)
  );

  INSERT INTO 라인 VALUES ('A','조립1라인','1동'), ('B','가공2라인','1동'),
  ('C','포장3라인','2동');
  INSERT INTO 설비 VALUES
    (1,'컨베이어 1호','A',2015), (2,'프레스 1호','A',2019), (3,'용접로봇
  1호','B',2021),
    (4,'검사기 1호','B',2018), (5,'포장기 1호','C',2022);
  INSERT INTO 작업자 VALUES
    (101,'김반장','A'), (102,'이기사','B'), (103,'박주임','B'), (104,'최사원','C');
  INSERT INTO 점검기록 VALUES
    (1,1,'2024-01-05','정상',92,101), (2,1,'2024-02-05','정상',88,101),
    (3,1,'2024-03-05','주의',71,102), (4,2,'2024-01-12','정상',95,101),
    (5,2,'2024-02-12','정상',90,103), (6,3,'2024-01-20','정상',97,102),
    (7,3,'2024-02-20','주의',68,102), (8,4,'2024-01-25','정상',85,103),
    (9,5,'2024-02-28','정상',91,104);
`);
```

★★★ 이제 개념01 의 세 가지 이상을 전부 다시 시도합니다.

① 갱신 이상 — 김반장 연락처가 바뀌었습니다. 통짜 표와 나란히 켜봅니다.

```
const 통짜연락처 = await db.query("UPDATE 점검대장 SET 담당자연락처 = $1 WHERE  
담당자사번 = $2", ["010-9999-0000", 101]);  
console.log("점검대장 - 김반장 연락처를 고친 줄 수:", 통짜연락처.affectedRows);
```

출력 점검대장 - 김반장 연락처를 고친 줄 수: 3

```
const 표연락처 = await db.query("UPDATE 작업자_4차 SET 연락처 = $1 WHERE 사번 = $2",  
["010-9999-0000", 101]);  
console.log("작업자 표 - 김반장 연락처를 고친 줄 수:", 표연락처.affectedRows);
```

출력 작업자 표 - 김반장 연락처를 고친 줄 수: 1

```
const 연락처가짓수 = await db.query("SELECT count(DISTINCT 연락처)::int AS 값 FROM  
작업자_4차 WHERE 사번 = 101");  
console.log("김반장 연락처 가짓수:", 연락처가짓수.rows[0].값);
```

출력 김반장 연락처 가짓수: 1

최종 작업자 표도 성질이 같습니다. 소속라인을 옮겨 봅니다.

```
const 소속변경 = await db.query("UPDATE 작업자 SET 소속라인 = $1 WHERE 사번 = $2",  
["B", 101]);  
console.log("작업자 표 - 김반장 소속을 고친 줄 수:", 소속변경.affectedRows);
```

출력 작업자 표 - 김반장 소속을 고친 줄 수: 1

```
await db.exec("UPDATE 작업자 SET 소속라인 = 'A' WHERE 사번 = 101;");  
  
console.log("갱신 이상이 사라졌나:", 표연락처.affectedRows === 1 &&  
연락처가짓수.rows[0].값 === 1);
```

출력 갱신 이상이 사라졌나: true

★ 사람에 딸린 칸은 작업자 표 한 곳에만 있습니다. 고칠 줄은 언제나 1줄입니다.

② 삽입 이상 — 점검을 한 번도 안 한 라인과 사람을 넣어 켜봅니다.

```
const 라인D = await db.query("INSERT INTO 라인 VALUES ($1, $2, $3)", ["D",  
"신설4라인", "2동"]);  
console.log("신설4라인 등록 - 넣은 줄 수:", 라인D.affectedRows);
```

출력 신설4라인 등록 - 넣은 줄 수: 1

```
const 정신입 = await db.query("INSERT INTO 작업자 VALUES ($1, $2, $3)", [105,
"정신입", "D"]);
console.log("정신입 등록 - 넣은 줄 수:", 정신입.affectedRows);
```

출력 정신입 등록 - 넣은 줄 수: 1

```
const 점검없는것 = await db.query(`
SELECT
  (SELECT count(*)::int FROM 점검기록
   WHERE 설비번호 IN (SELECT 설비번호 FROM 설비 WHERE 라인코드 = 'D')) AS
라인점검수,
  (SELECT count(*)::int FROM 점검기록 WHERE 담당사번 = 105) AS 사람점검수
`);
console.log("D라인 점검 수:", 점검없는것.rows[0].라인점검수);
```

출력 D라인 점검 수: 0

```
console.log("정신입 점검 수:", 점검없는것.rows[0].사람점검수);
```

출력 정신입 점검 수: 0

```
console.log("삽입 이상이 사라졌나:", 라인D.affectedRows === 1 && 정신입.affectedRows
=== 1);
```

출력 삽입 이상이 사라졌나: true

★ 점검이 0건인데도 라인과 사람이 멀쩡히 등록됐습니다. 개념01 에서는 불가능했습니다.

③ 삭제 이상 — 포장기 1호의 **유일한** 점검기록을 지웁니다.

```
const 지우기전 = await db.query("SELECT count(*)::int AS 값 FROM 점검기록 WHERE
설비번호 = 5");
console.log("포장기 1호의 점검기록 수:", 지우기전.rows[0].값);
```

출력 포장기 1호의 점검기록 수: 1

```
const 삭제 = await db.query("DELETE FROM 점검기록 WHERE 설비번호 = $1", [5]);
console.log("지운 줄 수:", 삭제.affectedRows);
```

출력 지운 줄 수: 1

```
const 설비남음 = await db.query("SELECT 이름, 도입년도 FROM 설비 WHERE 설비번호 =
5");
const 남은설명 = 설비남음.rows.length === 0 ? "없음" : `${설비남음.rows[0].이름}
(${설비남음.rows[0].도입년도}년)`;
console.log("설비 표에 남은 포장기 1호:", 남은설명);
```

출력 설비 표에 남은 포장기 1호: 포장기 1호 (2022년)

```
console.log("삭제 이상이 사라졌나:", 설비남음.rows.length === 1);
```

출력 삭제 이상이 사라졌나: true

★★★ 점검기록만 사라지고 **설비는 남았습니다**. 개념01에서는 그 한 줄이 곧 설비였기 때문에 설비가 통째로 증발했습니다.

```
await db.exec("INSERT INTO 점검기록 VALUES (9, 5, '2024-02-28', '정상', 91, 104);");
```

★ **BCNF** — 3정규형을 조금 더 조인 것입니다. 실무에서 3정규형까지 하면 대개 BCNF도 만족합니다. 이름만 알아 두세요. 4정규형·5정규형도 있습니다.

★ 정규화의 대가

섹션 6

★★ 개념01에서는 `SELECT * FROM 점검대장`; 한 줄이면 다 나왔습니다.

```
const 한줄로 = await db.query("SELECT count(*)::int AS 값 FROM 점검대장");
console.log("통짜 표는 표 1개 - 줄 수:", 한줄로.rows[0].값);
```

출력 통짜 표는 표 1개 - 줄 수: 9

지금은 같은 내용을 보려면 네 표를 이어야 합니다.

```
const 이어보기 = await db.query(`
  SELECT 설.이름 AS 설비, 라.이름 AS 라인, 작.이름 AS 담당자, 점.점검일, 점.결과
  FROM 점검기록 점
  JOIN 설비   설 ON 점.설비번호 = 설.설비번호
  JOIN 라인   라 ON 설.라인코드 = 라.라인코드
  JOIN 작업자 작 ON 점.담당사번 = 작.사번
  ORDER BY 점.점검번호 LIMIT 3
`);
for (const 줄 of 이어보기.rows) console.log(`${줄.점검일.toISOString().slice(0,10)} ·
${줄.설비} · ${줄.라인} · ${줄.담당자} · ${줄.결과}`);
```

출력 2024-01-05 · 컨베이어 1호 · 조립1라인 · 김반장 · 정상
 2024-02-05 · 컨베이어 1호 · 조립1라인 · 김반장 · 정상
 2024-03-05 · 컨베이어 1호 · 조립1라인 · 이기사 · 주의

★ 이게 **JOIN**입니다. 지금은 “이렇게 생겼구나”만 보면 됩니다. 문법은 05단원에서 제대로 합니다.

여기서 외우려 하지 마세요.

대가는 이렇습니다. · 조회가 길어집니다 — 한 줄이던 `SELECT`가 `JOIN` 세 개짜리가 됩니다 · 표가 늘어 한눈에 안 들어옵니다 — 1개에서 4개가 됐습니다 · 코드가 길어집니다 — 저장할 때도 표마다 따로 넣어야 합니다

★ 그래도 나눕니다. 조화가 길어지는 비용보다, 틀린 데이터를 고치는 비용이 훨씬 큼니다. JOIN 은 길어도 결과가 항상 맞습니다. 중복된 연락처 중 진짜를 찾는 일은 답이 없을 때가 많습니다. 그건 코드로 못 고칩니다. 사람이 하나하나 전화를 걸어야 합니다. ★ “그럼 언제 일부러 안 나누나요?” — 일부러 중복을 남기는 것을 **비정규화** 라고 합니다. **개념05** 에서 언제 그렇게 하는지, 대가가 무엇인지 재 봅니다.

MySQL 은 여기가 다릅니다

- 정규형은 이론이라 데이터베이스를 가리지 않습니다. 그대로 통합니다 · 파라미터가 \$1 이 아니라 ? 입니다
- SERIAL 대신 AUTO_INCREMENT 입니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 세 번 나누면 무엇이 사라지나

정규형 무엇을 떼어내나 사라지는 이상

1NF	한 칸의 여러 값을 여러 줄로	틀린 검색 - LIKE 오탐 2건
2NF	기본키 **일부**에만 딸린 칸 (복합 기본키일 때만 생깁니다)	갱신 이상 - 3줄 → 1줄

3NF 키가 아닌 칸에 딸린 칸 갱신 이상 — 2줄 → 1줄

(설비번호 → 라인코드 → 라인이름)	삽입 이상 - 설비 0대 라인 등록
----------------------	---------------------

표를 나눈 결과 삭제 이상도 함께 사라집니다. 점검기록을 지워도 설비는 남습니다.

★ 나누는 기준은 이론이 아니라 질문 하나입니다. **한 줄이 무엇 하나를 설명하고 있는가.** ★ 대가는 JOIN 입니다. 05단원에서 배웁니다.

직접 해 볼 것

직접 해보기 1

섹션 2 의 설비_나뭇 에 (6, '적재기', 'AB') 를 넣어 보세요. LIKE '%B%' 로 몇 개가 걸리나요? LIKE '%A%' 는요? 왜 그런지 설명해 보세요.

직접 해보기 2

섹션 3 의 점검_1차 에서 기본키를 점검번호 하나로 바꾸면 2정규형 문제가 남을까요? 대신 어느 정규형 문제가 생길까요?

▫ 직접 해보기 3

섹션 4의 라인_4차에서 이름을 고친 뒤 설비_4차 를 조회해 보세요. 라인 이름이 바뀌어 보입니까? (설비_4차 에는 라인이름 칸이 없습니다)

▫ 직접 해보기 4

섹션 5의 최종 표에서 라인 'A' 를 DELETE 해 보세요. 에러 코드를 찍어 보세요. (막는 이유는 개념03에서 합니다)

▫ 직접 해보기 5

설비 하나가 라인 두 곳에 걸쳐 있다면 최종 표를 어떻게 고칠까요? 섹션 2의 설비라인 표를 떠올려 보세요. 설비.라인코드 칸은 어떻게 됩니까?

▫ 직접 해보기 6

점검기록에 '점검내용' 칸을 되살린다면 어느 표에 넣어야 합니까? (무엇을 알아야 점검내용이 정해지는지 물어보세요)

자주 하는 실수

이런 실수를 합니다

정규형을 외우려고 함

"제2정규형은 부분 함수 종속이 없는 릴레이션이다" 를 외워도 도움이 안 됩니다. ★ 물을 것은 하나입니다. "이 칸은 무엇을 알면 정해지는가?" 그 답이 기본키 전체가 아니면 떼어냅니다.

이런 실수를 합니다

복합 기본키가 아닌데 2정규형을 걱정함

기본키가 칸 하나면 부분 종속이 생길 수가 없습니다. 이미 2정규형입니다. 그런 표의 중복은 3정규형 문제입니다. 섹션 4를 보세요.

이런 실수를 합니다

여러 값을 콤마 문자열로 넣음

★ LIKE '%B%' 가 'BC' 를 잡습니다. 개수도 못 세고 외래키도 못 겁니다. 줄을 늘리세요. 칸을 늘리지 마세요.

이런 실수를 합니다

무조건 잘게 쪼갬

정규화는 함수 종속이 있을 때 하는 것입니다. '주소' 를 무작정 시/군/구로 쪼개는 것은 정규화가 아닙니다. 쓸 일이 있을 때만 쪼개세요.

이런 실수를 합니다

LIKE '%값%' 로 다중값 칸을 조회함

실수 3 의 결과입니다. 그리고 이건 **에러가 안 납니다**. 틀린 목록이 조용히 나옵니다. 몇 달 뒤에 “왜 이 설비가 여기 있죠?” 로 발견됩니다.

이런 실수를 합니다

표가 늘어나는 게 손해라고 생각함

★ 표 개수는 사람이 읽는 비용이고, 틀린 데이터는 **고칠 수 없는 비용**입니다.

```
await db.close();
```

외래키로 잇습니다

RUN node 개념03_외래키로_잇습니다.js

개념02 에서 「점검대장」 한 표를 넷으로 나눴습니다. 라인 · 설비 · 작업자 · 점검기록. 나누고 나니 새 문제가 생겼습니다.

점검기록에 적힌 설비번호 3번이 **진짜 있는 설비인가?**

한 표였을 때는 물어볼 필요가 없었습니다. 설비 이름이 그 줄에 같이 있었으니까요. 나누고 나서는 **아무도 안 지켜 줍니다.** 그걸 지켜 주는 것이 **외래키(foreign key)** 입니다.

① 고아 데이터 ② 어떻게 거는가, 언제 못 거는가 ③ ON DELETE 다섯 가지 ④ CASCADE 가 왜 위험한가 ⑤ 외래키에는 색인이 자동으로 안 생깁니다

```
import { PGLite } from "@electric-sql/pglite";
const db = await PGLite.create();
```

이 파일에서 계속 쓰는 잔손입니다. 막히는 것을 보여 주는 게 목적이라 SQLSTATE 만 찍습니다.

```
async function 해보기(설명, 할일) {
  try {
    await 할일();
    console.log(`${설명} → 통과`);
  } catch (에러) {
    console.log(`${설명} → 막힘 (${에러.code})`);
  }
}

async function 세기(표, 조건 = "TRUE") {
  const 결과 = await db.query(`SELECT count(*)::int AS 수 FROM ${표} WHERE ${조건}`);
  return 결과.rows[0].수;
}
```

★ 외래키가 없으면 — 고아 데이터

섹션 1

개념02 가 만들어 낸 모양 그대로입니다. 단, **외래키는 아직 안 걸었습니다.**

```
await db.exec(`
  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름 TEXT NOT NULL,
```



```

도입년도 INT NOT NULL
);
CREATE TABLE 점검기록 (
  점검번호 INT PRIMARY KEY,
  설비번호 INT NOT NULL,      -- * 그냥 숫자 칸입니다. 아무 숫자나 들어갑니다
  점검일 DATE NOT NULL,
  결과 TEXT NOT NULL,
  점수 INT
);
INSERT INTO 설비 VALUES
(1, '컨베이어 1호', 'A', 2015), (2, '프레스 1호', 'A', 2019),
(3, '용접로봇 1호', 'B', 2021), (4, '검사기 1호', 'B', 2018),
(5, '포장기 1호', 'C', 2022);
INSERT INTO 점검기록 VALUES
(1, 1, '2024-01-05', '정상', 92), (2, 1, '2024-02-05', '정상', 88),
(3, 1, '2024-03-05', '주의', 71), (4, 2, '2024-01-12', '정상', 95),
(5, 2, '2024-02-12', '정상', 90), (6, 3, '2024-01-20', '정상', 97),
(7, 3, '2024-02-20', '주의', 68), (8, 4, '2024-01-25', '정상', 85),
(9, 5, '2024-02-28', '정상', 91);
`);

console.log("설비", await 세기("설비"), "대 · 점검기록", await 세기("점검기록"),
"줄");

```

출력 설비 5 대 · 점검기록 9 줄

이제 사고를 냅니다. 설비 999번은 없습니다. 그런데 999번의 점검기록을 넣어 봅니다.

```

await 해보기("없는 설비번호 999 로 점검기록 넣기", () =>
  db.exec(`INSERT INTO 점검기록 VALUES (10, 999, '2024-03-01', '정상', 80)`));

```

출력 없는 설비번호 999 로 점검기록 넣기 → 통과

★★★ 그냥 들어갔습니다. 에러도 경고도 없습니다. 눈으로 확인해 봅니다.

```

console.log("설비 999번:", await 세기("설비", "설비번호 = 999"), "대");

```

출력 설비 999번: 0 대

```

console.log("설비 999번의 점검기록:", await 세기("점검기록", "설비번호 = 999"),
"줄");

```

출력 설비 999번의 점검기록: 1 줄

★ 설비는 없는데 그 설비의 점검기록은 있습니다. 이런 줄을 **고아 데이터(orphan)** 라고 부릅니다. 부모가 없는 자식이라는 뜻입니다. 세는 쿼리는 NOT EXISTS 로 씁니다. (JOIN 으로도 되지만 JOIN 은 05단원 것입니다)

```
const 고아수 = await db.query(`
  SELECT count(*)::int AS 수 FROM 점검기록 기록
  WHERE NOT EXISTS (SELECT 1 FROM 설비 WHERE 설비.설비번호 = 기록.설비번호)
`);
```

```
console.log("고아 데이터:", 고아수.rows[0].수, "줄");
```

출력 고아 데이터: 1 줄

★★★ 이게 왜 무서운가 — 에러가 안 납니다. 로그도 안 남습니다. 테스트도 통과합니다. **몇 달 뒤 보고서를 뺄 때** 발견됩니다. 건수 합이 안 맞거나, 목록 화면의 설비 이름 칸이 비어 나옵니다. 그때 “언제부터였지?” 를 알아내야 하는데 알 방법이 없습니다. ★ 개념01 정리표의 “관계를 표현 못 함 → 04단원 외래키” 가 이것입니다.

외래키를 겁니다

섹션 2

문법은 두 가지입니다. 하는 일은 똑같습니다. ① 칸 뒤에 짧게 붙이는 형태 : 설비번호 INT REFERENCES 설비(설비번호) ② 표 끝에 따로 쓰는 형태 : FOREIGN KEY (설비번호) REFERENCES 설비(설비번호) ★ ② 를 꼭 써야 하는 경우가 있습니다. 칸이 **둘 이상** 인 외래키입니다.

```
FOREIGN KEY (라인코드, 설비번호) REFERENCES 설비(라인코드, 설비번호)
```

★ 걸기 전에 알아야 할 것 (1) — 부모 쪽 칸은 PRIMARY KEY 나 UNIQUE 여야 합니다.

```
await 해보기("설비(이름) 을 가리키는 외래키 - 이름은 PK 가 아닙니다", () =>
  db.exec(`CREATE TABLE 시험표 (설비이름 TEXT REFERENCES 설비(이름))`));
```

출력 설비(이름) 을 가리키는 외래키 - 이름은 PK 가 아닙니다 → 막힘 (42830)

```
try {
  await db.exec(`CREATE TABLE 시험표 (설비이름 TEXT REFERENCES 설비(이름))`);
} catch (에러) {
  console.log("메시지:", 에러.message);
}
```

출력 메시지: there is no unique constraint matching given keys for referenced table "설비"

```
}
```

★ 가리키는 쪽이 **한 줄로 딱 정해져야** 검사를 합니다. 이름이 같은 설비가 둘이면 “어느 쪽?” 이 되니까요. 부모 칸에 UNIQUE 라도 붙어 있어야 합니다.

★★ 걸기 전에 알아야 할 것 (2) — 이미 고아가 있으면 못 겁니다. 지금 점검기록에는 아까 넣은 999 번 줄이 그대로 있습니다.

```
await 해보기("고아가 남은 표에 외래키 걸기", () =>
  db.exec(`ALTER TABLE 점검기록 ADD CONSTRAINT 점검기록_설비_fk
    FOREIGN KEY (설비번호) REFERENCES 설비(설비번호)`));
```

출력 고아가 남은 표에 외래키 걸기 → 막힘 (23503)

★★★ 운영에서 진짜 겪는 일입니다. “외래키를 안 걸었네, 지금이라도 걸자” 하고 ALTER 를 날리면 이렇게 막힙니다. 몇 년치 데이터에 고아가 쌓여 있기 때문입니다. 순서가 정해져 있습니다. **고아를 먼저 치우고, 그다음에 겁니다.**

```
const 치운것 = await db.query(`
  DELETE FROM 점검기록 기록
  WHERE NOT EXISTS (SELECT 1 FROM 설비 WHERE 설비.설비번호 = 기록.설비번호)
`);

console.log("치운 고아:", 치운것.affectedRows, "줄");
```

출력 치운 고아: 1 줄

```
await 해보기("고아를 치운 뒤 다시 외래키 걸기", () =>
  db.exec(`ALTER TABLE 점검기록 ADD CONSTRAINT 점검기록_설비_fk
    FOREIGN KEY (설비번호) REFERENCES 설비(설비번호)`));
```

출력 고아를 치운 뒤 다시 외래키 걸기 → 통과

★ 진짜 운영에서는 고아를 그냥 지우면 안 됩니다. 먼저 **따로 떼 놓고** 무슨 데이터였는지 확인하세요. 쓰레기일 수도 있고, 부모를 잘못 지운 사고의 흔적일 수도 있습니다.

이제 아까 통과했던 그 INSERT 를 똑같이 다시 해 봅니다.

```
await 해보기("다시 999 번으로 점검기록 넣기", () =>
  db.exec(`INSERT INTO 점검기록 VALUES (10, 999, '2024-03-01', '정상', 80)`));
```

출력 다시 999 번으로 점검기록 넣기 → 막힘 (23503)

★★ 막혔습니다. SQLSTATE 23503 — 외래키 위반입니다. 같은 SQL 인데 결과가 뒤집혔습니다. 표에 규칙이 생겼기 때문입니다.

짧은 형태(①)도 실제로 써 봅니다. 담당자 칸을 붙입니다.

```
await db.exec(`
  CREATE TABLE 작업자 (사번 INT PRIMARY KEY, 이름 TEXT NOT NULL);
  INSERT INTO 작업자 VALUES (101,'김반장'), (102,'이기사'), (103,'박주임'),
  (104,'최사원');
  ALTER TABLE 점검기록 ADD COLUMN 담당사번 INT REFERENCES 작업자(사번);
`);

await 해보기("없는 사번 777 을 담당자로 넣기", () =>
  db.exec(`UPDATE 점검기록 SET 담당사번 = 777 WHERE 점검번호 = 1`));
```

출력 없는 사번 777 을 담당자로 넣기 → 막힘 (23503)

걸린 제약을 확인합니다. ★★ 여기에 PG18 함정이 하나 있습니다

```
const 제약전부 = await db.query(`
  SELECT conname AS 이름, contype AS 종류 FROM pg_constraint
  WHERE conrelid = '점검기록'::regclass ORDER BY contype, conname
`);

for (const 줄 of 제약전부.rows) console.log(` ${줄.종류} - ${줄.이름}`);
```

출력

- f - 점검기록_담당사번_fkey
- f - 점검기록_설비_fk
- n - 점검기록_결과_not_null
- n - 점검기록_설비번호_not_null
- n - 점검기록_점검번호_not_null
- n - 점검기록_점검일_not_null
- p - 점검기록_pkey

★★ NOT NULL 이 contype='n' 으로 같이 잡힙니다. PostgreSQL 17 까지는 안 들어왔습니다. 18 부터 들어옵니다. 예전 자료의 쿼리를 그대로 쓰면 개수가 달라집니다. ★ 제약을 조회할 때는 종류를 좁혀 주세요.

```
contype IN ('c','u','p','f') -- CHECK · UNIQUE · PRIMARY KEY · FOREIGN KEY
```

```
const 외래키만 = await db.query(`
  SELECT conname AS 이름 FROM pg_constraint
  WHERE conrelid = '점검기록'::regclass AND contype = 'f' ORDER BY conname
`);

for (const 줄 of 외래키만.rows) console.log(`외래키: ${줄.이름}`);
```

출력

외래키: 점검기록_담당사번_fkey

외래키: 점검기록_설비_fk

★★ 부모를 지우면 자식은 어떻게 되나 — ON DELETE 다섯 가지

섹션 3

설비를 폐기했습니다. 그 설비의 점검기록 40줄은 어떻게 됩니까? 답이 하나가 아니어서, 외래키를 걸 때 정책을 골라 적습니다. 전부 직접 돌려 봅니다.

```
await db.exec(`
  CREATE TABLE 실험설비 (설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL);
  INSERT INTO 실험설비 VALUES (1,'가'), (2,'나'), (3,'다'), (4,'라'), (5,'마'),
  (9,'미배정');
  CREATE TABLE 기록_막기      (점검번호 INT PRIMARY KEY, 설비번호 INT REFERENCES
  실험설비(설비번호) ON DELETE RESTRICT);
  CREATE TABLE 기록_기본      (점검번호 INT PRIMARY KEY, 설비번호 INT REFERENCES
  실험설비(설비번호) ON DELETE NO ACTION);
  CREATE TABLE 기록_같이삭제  (점검번호 INT PRIMARY KEY, 설비번호 INT REFERENCES
  실험설비(설비번호) ON DELETE CASCADE);
  CREATE TABLE 기록_널로      (점검번호 INT PRIMARY KEY, 설비번호 INT REFERENCES
  실험설비(설비번호) ON DELETE SET NULL);
  CREATE TABLE 기록_기본값으로 (점검번호 INT PRIMARY KEY, 설비번호 INT DEFAULT 9
  REFERENCES 실험설비(설비번호) ON DELETE SET DEFAULT);
  INSERT INTO 기록_막기 VALUES (1,1);
  INSERT INTO 기록_기본 VALUES (1,2);
  INSERT INTO 기록_같이삭제 VALUES (1,3), (2,3), (3,3);
  INSERT INTO 기록_널로 VALUES (1,4);
  INSERT INTO 기록_기본값으로 VALUES (1,5);
`);

await 해보기("RESTRICT - 설비 1 지우기", () => db.exec(`DELETE FROM 실험설비 WHERE
설비번호 = 1`));
```

출력 RESTRICT - 설비 1 지우기 → 막힘 (23001)

```
await 해보기("NO ACTION - 설비 2 지우기", () => db.exec(`DELETE FROM 실험설비 WHERE
설비번호 = 2`));
```

출력 NO ACTION - 설비 2 지우기 → 막힘 (23503)

★★★ 둘 다 막혔는데 **SQLSTATE** 가 다릅니다.

RESTRICT → 23001 (foreign_key_violation 이 아닙니다) / NO ACTION → 23503

e.code === "23503" 만 보고 분기하는 코드는 RESTRICT 를 놓칩니다. 자주 나는 버그입니다.

```
console.log("같이삭제 - 지우기 전 자식:", await 세기("기록_같이삭제"), "줄");
```

출력 같이삭제 - 지우기 전 자식: 3 줄

```
await 해보기("CASCADE - 설비 3 지우기", () => db.exec(`DELETE FROM 실험설비 WHERE
설비번호 = 3`));
```

출력 CASCADE - 설비 3 지우기 → 통과

```
console.log("같이삭제 - 지운 뒤 자식:", await 세기("기록_같이삭제"), "줄");
```

출력 같이삭제 - 지운 뒤 자식: 0 줄

```
await 해보기("SET NULL - 설비 4 지우기", () => db.exec(`DELETE FROM 실험설비 WHERE
설비번호 = 4`));
```

출력 SET NULL - 설비 4 지우기 → 통과

```
const 널로 = await db.query(`SELECT 설비번호 FROM 기록_널로 WHERE 점검번호 = 1`);
console.log("널로 - 자식의 설비번호:", 널로.rows[0].설비번호);
```

출력 널로 - 자식의 설비번호: null

```
await 해보기("SET DEFAULT - 설비 5 지우기", () => db.exec(`DELETE FROM 실험설비 WHERE
설비번호 = 5`));
```

출력 SET DEFAULT - 설비 5 지우기 → 통과

```
const 기본값으로 = await db.query(`SELECT 설비번호 FROM 기록_기본값으로 WHERE
점검번호 = 1`);
console.log("기본값으로 - 자식의 설비번호:", 기본값으로.rows[0].설비번호);
```

출력 기본값으로 - 자식의 설비번호: 9

★ DEFAULT 9 ('미배정') 로 바뀌었습니다. "기록을 버리긴 아까우니 미배정에 모아 둔다" 입니다.

★ SET DEFAULT 의 함정 — 그 기본값이 부모에 없으면

```
await db.exec(`
  CREATE TABLE 실험설비2 (설비번호 INT PRIMARY KEY);
  INSERT INTO 실험설비2 VALUES (5);
  CREATE TABLE 기록_기본값없음 (점검번호 INT PRIMARY KEY,
    설비번호 INT DEFAULT 77 REFERENCES 실험설비2(설비번호) ON DELETE SET DEFAULT);
  INSERT INTO 기록_기본값없음 VALUES (1, 5);
`);

await 해보기("SET DEFAULT 인데 기본값 77 이 부모에 없음", () =>
  db.exec(`DELETE FROM 실험설비2 WHERE 설비번호 = 5`));
```

출력 SET DEFAULT 인데 기본값 77 이 부모에 없음 → 막힘 (23503)

★★ 자식을 77 로 바꾸려는 순간 **그 77 이 또 외래키 검사에 걸립니다**. 그래서 23503 이고, 부모 삭제 자체가 통째로 실패합니다. SET DEFAULT 를 쓸 거면 **그 기본값에 해당하는 부모 줄을 반드시 미리 만들어 두세요**.

★ SET NULL 의 함정 — 그 칸이 NOT NULL 이면

```
await db.exec(`
  CREATE TABLE 실험설비3 (설비번호 INT PRIMARY KEY);
  INSERT INTO 실험설비3 VALUES (5);
  CREATE TABLE 기록_널못됨 (점검번호 INT PRIMARY KEY,
    설비번호 INT NOT NULL REFERENCES 실험설비3(설비번호) ON DELETE SET NULL);
  INSERT INTO 기록_널못됨 VALUES (1, 5);
`);

await 해보기("SET NULL 인데 그 칸이 NOT NULL", () =>
  db.exec(`DELETE FROM 실험설비3 WHERE 설비번호 = 5`));
```

출력 SET NULL 인데 그 칸이 NOT NULL → 막힘 (23502)

★ 23502 — NOT NULL 위반. 표는 아무 말 없이 만들어지고 **지울 때가 되어서야** 터집니다.

★★ RESTRICT 와 NO ACTION 의 진짜 차이

위에서는 둘 다 막혔습니다. 차이는 **검사 시점** 입니다. NO ACTION → 검사를 트랜잭션 끝까지 **미룰 수 있습니다** RESTRICT → 미루라고 해도 **그 자리에서 막습니다** DEFERRABLE INITIALLY DEFERRED 를 붙이면 차이가 드러납니다. (트랜잭션 자체는 07단원 것입니다. 여기서는 문법만 쓰고 넘어갑니다)

```
await db.exec(`
  CREATE TABLE 미룸설비 (설비번호 INT PRIMARY KEY);
  INSERT INTO 미룸설비 VALUES (1), (2);
  CREATE TABLE 미룸_기본 (점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 미룸설비(설비번호) ON DELETE NO ACTION DEFERRABLE
  INITIALLY DEFERRED);
  CREATE TABLE 미룸_막기 (점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 미룸설비(설비번호) ON DELETE RESTRICT DEFERRABLE
  INITIALLY DEFERRED);
  INSERT INTO 미룸_기본 VALUES (1, 1);
  INSERT INTO 미룸_막기 VALUES (1, 2);
`);
```

한 트랜잭션 안에서 **부모를 먼저 지우고 자식을 지웁니다**. 중간 한순간만 보면 고아지만, 끝나고 보면 아무 문제가 없습니다.

```
await 해보기("NO ACTION + 미루기 - 부모 먼저, 자식 나중", () =>
  db.transaction(async (트) => {
    await 트.query(`DELETE FROM 미룸설비 WHERE 설비번호 = 1`);
    await 트.query(`DELETE FROM 미룸_기본 WHERE 설비번호 = 1`);
  }));
```

출력 NO ACTION + 미루기 - 부모 먼저, 자식 나중 → 통과

```
await 해보기("RESTRICT + 미루기 - 똑같이 해 보기", () =>
  db.transaction(async (트) => {
    await 트.query(`DELETE FROM 미룸설비 WHERE 설비번호 = 2`);
    await 트.query(`DELETE FROM 미룸_막기 WHERE 설비번호 = 2`);
  }));
```

출력 RESTRICT + 미루기 - 똑같이 해 보기 → 막힘 (23001)

```
console.log("남은 미룸설비:", await 세기("미룸설비"), "줄");
```

출력 남은 미룸설비: 1 줄

★★ 이게 둘의 진짜 차이입니다. NO ACTION 은 “끝날 때 앞뒤가 맞으면 된다” 라서 1번이 지워졌고, RESTRICT 는 “자식이 달려 있으면 그 순간 안 된다” 라서 2번이 남았습니다. ★ RESTRICT 가 더 엄격합니다. 아무것도 안 적으면 기본값인 NO ACTION 이 걸립니다.

ON UPDATE 도 같은 다섯 가지가 있습니다

```
await db.exec(`
  CREATE TABLE 라인코드표 (라인코드 TEXT PRIMARY KEY);
  INSERT INTO 라인코드표 VALUES ('A');
  CREATE TABLE 설비_코드추적 (설비번호 INT PRIMARY KEY,
    라인코드 TEXT REFERENCES 라인코드표(라인코드) ON UPDATE CASCADE);
  INSERT INTO 설비_코드추적 VALUES (1, 'A');
  UPDATE 라인코드표 SET 라인코드 = 'A1' WHERE 라인코드 = 'A';
`);

const 따라감 = await db.query(`SELECT 라인코드 FROM 설비_코드추적 WHERE 설비번호 = 1`);
console.log("부모 키를 A → A1 로 바꾸니 자식은:", 따라감.rows[0].라인코드);
```

출력 부모 키를 A → A1 로 바꾸니 자식은: A1

★ ON UPDATE 는 덜 씁니다. 기본키를 바꾸는 일 자체가 드물기 때문입니다.

★ CASCADE 의 위험

섹션 4

CASCADE 는 편합니다. 그래서 위험합니다. 04단원의 도착점인 네 표를 CASCADE 로 이어 놓고 실제로 사고를 내 봅니다.

```
await db.exec(`
  DROP TABLE 점검기록;
  DROP TABLE 설비;
  CREATE TABLE 라인 (라인코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL, 동 TEXT NOT NULL);
  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름 TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드) ON DELETE CASCADE,
    도입년도 INT NOT NULL
  );
  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호) ON DELETE CASCADE,
    점검일 DATE NOT NULL,
    결과 TEXT NOT NULL,
    점수 INT,
    담당사번 INT REFERENCES 작업자(사번)
  );
`);
```

```
const 데이터넣기 = `
  DELETE FROM 라인;
  INSERT INTO 라인 VALUES ('A','조립1라인','1동'), ('B','가공2라인','1동'),
  ('C','포장3라인','2동'), ('D','신설4라인','2동');
  INSERT INTO 설비 VALUES
    (1,'컨베이어 1호','A',2015), (2,'프레스 1호','A',2019),
    (3,'용접로봇 1호','B',2021), (4,'검사기 1호','B',2018), (5,'포장기
1호','C',2022);
  INSERT INTO 점검기록 VALUES
    (1,1,'2024-01-05','정상',92,101), (2,1,'2024-02-05','정상',88,101), (3,1,'2024-
03-05','주의',71,102),
    (4,2,'2024-01-12','정상',95,101), (5,2,'2024-02-12','정상',90,103),
    (6,3,'2024-01-20','정상',97,102), (7,3,'2024-02-20','주의',68,102),
    (8,4,'2024-01-25','정상',85,103), (9,5,'2024-02-28','정상',91,104);
`;
await db.exec(데이터넣기);
```

★ DELETE 한 줄이 몇 줄을 지울지 **미리 세어 보는 것**이 먼저입니다.

```
console.log("설비 1 을 지우면 같이 사라질 점검기록:", await 세기("점검기록",
"설비번호 = 1"), "줄");
```

출력 설비 1 을 지우면 같이 사라질 점검기록: 3 줄

```
console.log("지우기 전 점검기록:", await 세기("점검기록"), "줄");
```

출력 지우기 전 점검기록: 9 줄

```
await db.exec(`DELETE FROM 설비 WHERE 설비번호 = 1`);
console.log("설비 1 대를 지운 뒤 점검기록:", await 세기("점검기록"), "줄");
```

출력 설비 1 대를 지운 뒤 점검기록: 6 줄

★★ 설비 한 줄을 지웠는데 점검기록 세 줄이 무더기로 사라졌습니다. DELETE 문에는 '점검기록'이라는 글자가 없었습니다.

★★★ 더 무서운 것: 연쇄가 이어집니다 ————

라인 → 설비 → 점검기록. 3단으로 CASCADE 가 걸려 있습니다. 라인 하나를 지우면?

```
await db.exec(데이터넣기);

console.log("되돌림 - 라인", await 세기("라인"), " · 설비", await 세기("설비"), " ·
점검기록", await 세기("점검기록"));
```

출력 되돌림 - 라인 4 · 설비 5 · 점검기록 9

```
await db.exec(`DELETE FROM 라인 WHERE 라인코드 = 'A'`);
console.log("라인 A 하나 지운 뒤 - 라인", await 세기("라인"), "· 설비", await 세기("설비"), "· 점검기록", await 세기("점검기록"));
```

출력 라인 A 하나 지운 뒤 - 라인 3 · 설비 3 · 점검기록 4

★★★ 라인 한 줄을 지웠는데 설비 두 대와 점검기록 다섯 줄이 사라졌습니다. 지운 사람은 자기가 무엇을 지웠는지 모릅니다. 그리고 되돌릴 방법이 없습니다. 현장 사고는 대개 이런겁니다 — “안 쓰는 라인 정리할게요” → 3년치 점검 이력이 통째로. ★ 지우기 전에 반드시 세어 보세요. 한 줄이면 됩니다.

```
const 라인B파장 = await db.query(`
  SELECT count(*)::int AS 수 FROM 점검기록 기록
  WHERE EXISTS (SELECT 1 FROM 설비
                WHERE 설비.설비번호 = 기록.설비번호 AND 설비.라인코드 = 'B')
`);

console.log("라인 B 를 지우면 사라질 점검기록:", 라인B파장.rows[0].수, "줄");
```

출력 라인 B 를 지우면 사라질 점검기록: 3 줄

★ 실무 조언

사람 데이터 · 돈 데이터에는 CASCADE 를 걸지 마세요. 회원 · 주문 · 결제 · 급여 · 점검 이력 같은 것들입니다. 대신 둘 중 하나를 씁니다.

- ① RESTRICT 로 막고 **손으로 정리**합니다. 귀찮은 게 안전한 겁니다
- ② **소프트 삭제** - 진짜로 안 지우고 '지워진 것으로 표시' 만 합니다

```
await db.exec(`
  ALTER TABLE 설비 ADD COLUMN 삭제여부 BOOLEAN NOT NULL DEFAULT FALSE;
  UPDATE 설비 SET 삭제여부 = TRUE WHERE 설비번호 = 4; -- 지우는 대신 표시만
`);

console.log("소프트 삭제 - 살아 있는 설비:", await 세기("설비", "삭제여부 = FALSE"), "대");
```

출력 소프트 삭제 - 살아 있는 설비: 2 대

```
console.log("소프트 삭제 - 점검기록은:", await 세기("점검기록"), "줄 (그대로)");
```

출력 소프트 삭제 - 점검기록은: 4 줄 (그대로)

★ 소프트 삭제의 대가도 있습니다. 모든 조회에 WHERE 삭제여부 = FALSE 를 빠뜨리면 안 됩니다. 한 군데만 빠뜨려도 지운 설비가 화면에 다시 나타납니다. ★ CASCADE 는 주문 → 주문상세처럼 부모 없이는 의미가 없는 자식에만 씁니다.

★ 외래키에는 색인이 자동으로 안 생깁니다

섹션 5

PRIMARY KEY 를 걸면 색인이 자동으로 생깁니다. 외래키도 그럴 것 같습니다. 확인해 봅니다.

```
const 자식색인 = await db.query(`SELECT indexname FROM pg_indexes WHERE tablename =
'점검기록' ORDER BY indexname`);
for (const 줄 of 자식색인.rows) console.log(`자식(점검기록) 색인: ${줄.indexname}`);
```

출력 자식(점검기록) 색인: 점검기록_pkey

```
const 부모색인 = await db.query(`SELECT indexname FROM pg_indexes WHERE tablename =
'설비' ORDER BY indexname`);
for (const 줄 of 부모색인.rows) console.log(`부모(설비) 색인: ${줄.indexname}`);
```

출력 부모(설비) 색인: 설비_pkey

★★ 자식 표에는 _pkey 하나뿐입니다. 설비번호 칸에도 담당사번 칸에도 색인이 없습니다. ★ 헛갈리는 지점입니다. 정리하면

부모 쪽(설비.설비번호) - 색인 **있습니다**. PRIMARY KEY 니까요
자식 쪽(점검기록.설비번호) - 색인 **없습니다**. 아무도 안 만들어 줍니다

★ 없으면 뭐가 문제인가 — 부모를 지울 때 느려집니다. 설비 한 대를 지우려면 “이 설비를 가리키는 점검기록이 있나?” 를 확인해야 하는데, 자식 쪽에 색인이 없으면 점검기록을 처음부터 끝까지 훑습니다. 10만 줄로 재 봅니다.

```
await db.exec(`
  CREATE TABLE 설비_대량 (설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL);
  INSERT INTO 설비_대량 SELECT 번호, '설비' || 번호 FROM generate_series(1, 220) AS
  번호;
  CREATE TABLE 점검_대량 (점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비_대량(설비번호), 점수 INT);
  INSERT INTO 점검_대량 SELECT 번호, (번호 % 200) + 1, 번호 % 100 FROM
  generate_series(1, 100000) AS 번호;
  ANALYZE;
`);
```

console.log("점검_대량:", await 세기("점검_대량"), "줄");

출력 점검_대량: 100000 줄

설비 201~220 번에는 점검기록이 하나도 없습니다. 그래도 지우려면 자식 표를 확인해야 합니다.

```
async function 재기(할일) {
  const 시작 = performance.now();
```

관계의 모양

RUN node 개념04_관계의_모양.js

개념02 에서 통짜 표를 나눴고, 개념03 에서 외래키로 다시 이었습니다. 그런데 잇는 모양이 늘 같지는 않습니다. 네 가지가 있습니다.

1:N	라인 하나에 설비가 여럿	← 가장 흔합니다
N:M	작업자 여럿 ↔ 자격 여럿	← 중간 표가 필요합니다
1:1	설비 하나에 상세스펙 하나	← 드뭅니다

자기 참조 조직이 자기 표를 가리킴 ← 계층을 한 표로

★★★ 이 파일의 진짜 목표는 마지막 섹션입니다. ***"A 하나에 B 가 여럿일 수 있나?" "B 하나에 A 가 여럿일 수 있나?"** 이 두 질문이면 모양이 저절로 정해집니다.

```
import { PGLite } from "@electric-sql/pglite";
const db = await PGLite.create();
```

1:N — 가장 흔한 모양

섹션 1

라인 하나에 설비가 여럿 있고, 설비 하나는 라인 하나에만 속합니다. 이걸 **1:N** 이라고 합니다. 1 쪽이 라인, N 쪽이 설비입니다.

★★★ 물어야 할 것은 딱 하나입니다. **외래키를 어느 쪽에 두는가**. 답: **N 쪽(자식)에 둡니다**. 먼저 반대로 해 봅니다. 봐야 기억에 남습니다.

```
await db.exec(`
  CREATE TABLE 라인_잘못 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    설비번호 INT          -- * 1 쪽에 외래키를 걸어 봤습니다
  );
  INSERT INTO 라인_잘못 VALUES ('A', '조립1라인', 1);
`);
```

설비 1번을 달았습니다. 여기까지는 됩니다. 이제 설비 2번을 하나 더 답니다.

방법 ① 그 줄을 고친다 → 1번이 사라집니다.

```
await db.query("UPDATE 라인_잘못 SET 설비번호 = 2 WHERE 라인코드 = $1", ["A"]);

const 덮어씀 = await db.query("SELECT 설비번호 FROM 라인_잘못 WHERE 라인코드 = 'A'");
console.log("고쳤더니 남은 설비번호:", 덮어씀.rows[0].설비번호);
```

출력 고쳤더니 남은 설비번호: 2

★ 1번 컨베이어가 조용히 사라졌습니다. 에러도 없습니다.

방법 ② 줄을 하나 더 넣는다 → 기본키가 막습니다.

```
try {
  await db.query("INSERT INTO 라인_잘못 VALUES ($1, $2, $3)", ["A", "조립1라인", 1]);
  console.log("두 번째 줄 들어감");
} catch (에러) {
  console.log("두 번째 줄 실패:", 에러.code);
}
```

출력 두 번째 줄 실패: 23505

}

★★ 막다른 길입니다. 라인 표의 한 줄은 라인 하나를 뜻합니다. 그 줄에 설비 칸이 하나뿐이면 **라인 하나에 설비 하나밖에** 못 답니다.

방법 ③ “그럼 콤마로 이어 붙이면?” — 여기가 더 무섭습니다.

```
await db.exec(`
  CREATE TABLE 라인_콤마 (라인코드 TEXT PRIMARY KEY, 설비목록 TEXT);
  INSERT INTO 라인_콤마 VALUES ('A', '1,2'), ('B', '3,4'), ('C', '12');
`);

const 오탐 = await db.query("SELECT 라인코드 FROM 라인_콤마 WHERE 설비목록 LIKE '%2%'");
console.log("설비 2번이 있는 라인:", 오탐.rows.map((줄) => 줄.라인코드).join(", "));
```

출력 설비 2번이 있는 라인: A, C

★★★ C라인에는 설비 2번이 없습니다. 12번이 있을 뿐인데 '12' 안의 '2'가 걸렸습니다. 이게 **1정규형을 어킨 대가**입니다. 개념02에서 본 그 사고입니다.

2: 제대로 — 외래키는 N 쪽에

섹션 1

```
await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
```

```

동      TEXT NOT NULL
);

CREATE TABLE 설비 (
  설비번호 INT PRIMARY KEY,
  이름      TEXT NOT NULL,
  라인코드 TEXT REFERENCES 라인(라인코드),  -- * 외래키가 N 쪽에 있습니다
  도입년도 INT NOT NULL
);

INSERT INTO 라인 VALUES
  ('A', '조립1라인', '1동'), ('B', '가공2라인', '1동'),
  ('C', '포장3라인', '2동'), ('D', '신설4라인', '2동');

INSERT INTO 설비 VALUES
  (1, '컨베이어 1호', 'A', 2015), (2, '프레스 1호', 'A', 2019),
  (3, '용접로봇 1호', 'B', 2021), (4, '검사기 1호', 'B', 2018),
  (5, '포장기 1호', 'C', 2022);
`);

```

이제 A라인에 설비를 몇 대든 답니다. 줄을 늘리기만 하면 됩니다.

```

const A설비 = await db.query("SELECT count(*)::int AS 대수 FROM 설비 WHERE 라인코드 =
$1", ["A"]);

console.log("A라인 설비 대수:", A설비.rows[0].대수);

```

출력 A라인 설비 대수: 2

★ `count(*)::int` 를 씁니다. 그냥 `count(*)` 는 줄이 많아지면 `bigint` 로 바뀝니다.

```

const 라인별 = await db.query(`
  SELECT 라인.라인코드,
         (SELECT count(*)::int FROM 설비 WHERE 설비.라인코드 = 라인.라인코드) AS 대수
  FROM 라인
  ORDER BY 라인.라인코드
`);

for (const 줄 of 라인별.rows) {
  console.log(`· ${줄.라인코드}라인 - 설비 ${줄.대수}대`);
}

```

출력 · A라인 - 설비 2대
 · B라인 - 설비 2대
 · C라인 - 설비 1대
 · D라인 - 설비 0대

★★ D라인(신설4라인)은 **0대**입니다. 1:N 의 N 은 “1개 이상” 이 아니라 “**0개 이상**” 입니다. 설비 표에는 D라인 줄이 아예 없습니다 — 그게 정상입니다. ★ 이걸 놓치면 “라인 목록에 신설라인이 안 나와요” 하는 버그가 납니다.

설비 쪽에서 라인을 뽑으면 D가 빠집니다. 라인 표에서 뽑아야 나옵니다.

1:N 은 한 번 더 이어집니다. 설비 하나에 점검기록이 여럿입니다.

```
await db.exec(`
  CREATE TABLE 작업자 (
    사번      INT  PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인  TEXT REFERENCES 라인(라인코드)
  );

  CREATE TABLE 점검기록 (
    점검번호 INT  PRIMARY KEY,
    설비번호 INT  REFERENCES 설비(설비번호),  -- ★ 여기도 N 쪽입니다
    점검일    DATE NOT NULL,
    결과      TEXT NOT NULL,
    점수      INT,
    담당사번 INT  REFERENCES 작업자(사번)
  );

  INSERT INTO 작업자 VALUES
    (101, '김반장', 'A'), (102, '이기사', 'B'),
    (103, '박주임', 'B'), (104, '최사원', 'C'), (105, '정신입', 'D');

  INSERT INTO 점검기록 VALUES
    (1, 1, '2024-01-05', '정상', 92, 101), (2, 1, '2024-02-05', '정상', 88, 101),
    (3, 1, '2024-03-05', '주의', 71, 102), (4, 2, '2024-01-12', '정상', 95, 101),
    (5, 2, '2024-02-12', '정상', 90, 103), (6, 3, '2024-01-20', '정상', 97, 102),
    (7, 3, '2024-02-20', '주의', 68, 102), (8, 4, '2024-01-25', '정상', 85, 103),
    (9, 5, '2024-02-28', '정상', 91, 104);
`);

const 설비1점검 = await db.query("SELECT count(*)::int AS 건수 FROM 점검기록 WHERE
설비번호 = $1", [1]);

console.log("설비 1번의 점검기록:", 설비1점검.rows[0].건수, "건");
```

출력 설비 1번의 점검기록: 3 건

★ 점검기록에는 외래키가 두 개(설비번호·담당사번) 있습니다. 한 표가 부모를 둘 가질 수 있습니다. “설비 → 점검기록” 도 1:N, “작업자 → 점검기록” 도 1:N 입니다.

★★ N:M — 중간 표가 필요한 모양

섹션 2

02단원에서 예고해 둔 예제입니다. **작업자와 자격**입니다.

· 작업자 한 명이 자격을 여럿 가질 수 있습니다. (김반장: 용접 + 안전) · 자격 하나를 여러 명이 가질 수 있습니다. (용접: 김반장 + 이기사)

양쪽 다 “여럿” 입니다. 이게 **N:M** 입니다. 먼저 외래키만으로 해 봅니다.

```
await db.exec(`
  CREATE TABLE 자격 (
    자격코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL
  );
  INSERT INTO 자격 VALUES
    ('WELD', '용접기능사'), ('CRANE', '지게차운전기능사'),
    ('SAFE', '산업안전기사'), ('ELEC', '전기기능사');

  CREATE TABLE 작업자_잘못 (
    사번      INT   PRIMARY KEY,
    이름      TEXT NOT NULL,
    자격코드 TEXT REFERENCES 자격(자격코드)  -- * 칸이 하나뿐입니다
  );
  INSERT INTO 작업자_잘못 VALUES (101, '김반장', 'WELD');
`);
```

김반장이 산업안전기사도 땀습니다. 넣어 봅니다.

```
try {
  await db.query("INSERT INTO 작업자_잘못 VALUES ($1, $2, $3)", [101, "김반장",
    "SAFE"]);
  console.log("두 번째 자격 들어감");
} catch (에러) {
  console.log("두 번째 자격 실패:", 에러.code);
}
```

출력 두 번째 자격 실패: 23505

```
}
```

고치면? 용접기능사가 사라집니다. 섹션 1 과 똑같은 막다른 길입니다.

★ “칸을 자격코드1, 자격코드2 로 늘리면?” 자격이 3개인 사람이 오면 다시 막히고, “용접 보유자” 를 찾으려면 칸을 전부 뒤져야 합니다. 언제나 잘못된 설계입니다.

답은 표를 하나 더 만드는 것입니다. 작업자도 자격도 아닌, **관계만 담는 표**입니다.

```
await db.exec(`
  CREATE TABLE 작업자자격 (
    사번      INT REFERENCES 작업자(사번),
    자격코드 TEXT REFERENCES 자격(자격코드),
    취득일    DATE NOT NULL,
    만료일    DATE,
    PRIMARY KEY (사번, 자격코드)      -- ★★ 복합 기본키
  );

  INSERT INTO 작업자자격 VALUES
    (101, 'WELD', '2020-03-01', '2025-03-01'), (101, 'SAFE', '2021-06-15', NULL),
    (102, 'WELD', '2022-01-10', '2027-01-10'), (102, 'ELEC', '2023-05-20', '2028-05-20'),
    (103, 'CRANE', '2019-11-01', '2024-11-01');
`);
```

★ 중간 표는 **양쪽으로 외래키**를 가집니다. 이게 중간 표의 정의입니다.

작업자 —1:N—> 작업자자격 <—N:1— 자격

N:M 하나가 1:N 두 개로 쪼개집니다. DB 는 N:M 을 직접 표현하지 못합니다.

```
const 김반장자격 = await db.query(
  "SELECT 자격코드 FROM 작업자자격 WHERE 사번 = $1 ORDER BY 자격코드", [101]);

console.log("김반장(101)의 자격:", 김반장자격.rows.map((줄) => 줄.자격코드).join(", "));
```

출력 김반장(101)의 자격: SAFE, WELD

반대 방향도 같은 표 하나로 풀니다. 용접기능사를 가진 사람을 찾습니다.

```
const 용접보유 = await db.query(
  `SELECT 이름 FROM 작업자
   WHERE 사번 IN (SELECT 사번 FROM 작업자자격 WHERE 자격코드 = $1) ORDER BY 사번`,
  ["WELD"]);

console.log("용접기능사 보유자:", 용접보유.rows.map((줄) => 줄.이름).join(", "));
```

출력 용접기능사 보유자: 김반장, 이기사

★ **IN (SELECT ...)** 를 썼습니다. 자격 이름까지 뽑으려면 JOIN 이 편한데, **JOIN 문법은 05단원에서 제대로 합니다.**

```
const 무자격 = await db.query(`
  SELECT 이름 FROM 작업자 가
  WHERE NOT EXISTS (SELECT 1 FROM 작업자자격 나 WHERE 나.사번 = 가.사번) ORDER BY 가.
  사번
`);

console.log("자격이 없는 사람:", 무자격.rows.map((줄) => 줄.이름).join(", "));
```

출력 자격이 없는 사람: 최사원, 정신입

★ 1:N 의 N 이 0 일 수 있듯 N:M 도 양쪽 다 0 일 수 있습니다. 중간 표에 줄이 없다는 것은 “관계가 없다” 는 뜻입니다.

3: ★★ 왜 복합 기본키인가

섹션 2

PRIMARY KEY (사번, 자격코드) 는 두 칸을 묶어서 기본키로 씁니다. 사번만으로는 못 됩니다 — 김반장이 두 줄이니깐요. 자격코드만으로도 못 됩니다. **둘을 합치면** 겹치지 않습니다.

★ 그리고 이게 규칙 하나를 공짜로 만들어 줍니다. **같은 사람에게 같은 자격을 두 번 줄 수 없습니다.**

```
try {
  await db.query("INSERT INTO 작업자자격 VALUES ($1, $2, $3, $4)", [101, "WELD",
    "2024-08-01", null]);
  console.log("중복 자격 들어감");
} catch (에러) {
  console.log("중복 자격 실패:", 에러.code);
```

출력 중복 자격 실패: 23505

```
console.log("막은 것:", 에러.message.includes("작업자자격_pkey"));
```

출력 막은 것: true

```
}
```

★★★ 복합 기본키를 안 걸면 이 INSERT 가 **그냥 들어갑니다**. 그러면 “용접기능사 보유자” 를 셀 때 김반장이 두 번 나오고, 몇 달 뒤에 발견됩니다. 중간 표에 복합 기본키를 안 거는 것이 N:M 에서 가장 흔한 사고입니다.

4: ★ 중간 표가 가진 정보

섹션 2

여기가 중간 표의 진짜 값어치입니다. 위 표의 **취득일** 과 **만료일** 을 어디에 둘까요?

· 작업자 표에? → 김반장은 자격이 둘인데 취득일 칸은 하나입니다. 못 답습니다. · 자격 표에? →
용접기능사는 보유자가 둘인데 취득일이 다릅니다. 못 답습니다.

★★★ 이 정보는 작업자의 것도 자격의 것도 아닙니다. “김반장이 용접기능사를 가진 그 관계” 가 가진
정보입니다. 관계 자체가 정보를 가집니다. 그래서 중간 표에 답니다.

```
const 만료임박 = await db.query(`
  SELECT 사번, 자격코드, 만료일::text AS 만료일
  FROM 작업자자격
  WHERE 만료일 IS NOT NULL AND 만료일 < DATE '2025-06-01'
  ORDER BY 만료일
`);

for (const 줄 of 만료임박.rows) {
  console.log(`· ${줄.사번} / ${줄.자격코드} - ${줄.만료일} 만료`);
}
```

출력

· 103 / CRANE - 2024-11-01 만료
· 101 / WELD - 2025-03-01 만료

★ 만료일이 NULL 인 줄(김반장의 SAFE)은 안 걸렸습니다. NULL 은 < 비교에서 참이 아닙니다. “평생
자격” 을 NULL 로 돌지는 설계 판단입니다.

관계에 붙는 정보는 이 밖에도 많습니다. 수강(학번, 과목코드) → 성적, 수강신청일 주문항목(주문번호,
상품) → 수량, ★ 주문 당시의 단가 (상품 표의 현재 단가와 다릅니다) 출고(설비, 부품) → 개수, 출고일

5: 중간 표 이름 짓기

섹션 2

양쪽 이름을 이어 붙이기 작업자자격, 설비부품 ← 흔하고 무난합니다 ★ 의미 있는 이름이 있으면 수강,
주문항목, 예약 ← 이게 더 낫습니다

★ “수강” 은 “학생과목” 보다 좋은 이름입니다. 현장 사람들이 쓰는 말이니깐요. 이름이 떠오르면 그 관계가
독립된 개념이라는 신호입니다.

★ 언제 복합키 대신 대리키(SERIAL)를 쓰나 — 중간 표가 자기 자식을 가질 때입니다. 주문항목에
‘배송추적’ 이 딸리면 자식이 부모 키 두 칸을 들고 다녀야 해 번거롭습니다. 그럴 때 항목번호 SERIAL
PRIMARY KEY 를 두고 UNIQUE (주문번호, 상품코드) 를 따로 겁니다. ★★ 그 UNIQUE 를 빼먹으면 중복을
막던 장치가 사라집니다. 여기가 함정입니다.

1:1 — 드문 모양

섹션 3

★ 솔직하게 시작합니다. 1:1 은 드뭅니다. “A 하나에 B 하나, B 하나에 A 하나” 라면 대개는 한 표로 합치는
게 맞습니다. 표를 나누면 찾을 때마다 두 표를 봐야 합니다. 그래도 나누는 경우가 셋 있습니다.

```

await db.exec(`
  -- ① 칸이 너무 많아서 나눌 때 (자주 안 보는 칸을 밀어냅니다)
  CREATE TABLE 설비상세스펙 (
    설비번호 INT PRIMARY KEY REFERENCES 설비(설비번호), -- * 외래키를 그대로
    기본키로
    정격전력W INT, 무게kg INT, 제조사 TEXT, 시리얼 TEXT
  );

  -- ② 가끔만 있는 정보 (폐기된 설비만 줄이 생깁니다)
  CREATE TABLE 설비폐기정보 (
    설비번호 INT PRIMARY KEY REFERENCES 설비(설비번호),
    폐기일 DATE NOT NULL, 사유 TEXT NOT NULL
  );

  -- ③ 접근 권한을 나눌 때 (급여는 아무나 보면 안 됩니다)
  CREATE TABLE 작업자급여 (사번 INT PRIMARY KEY REFERENCES 작업자(사번), 기본급 INT
  NOT NULL);

  INSERT INTO 설비상세스펙 VALUES (1, 2200, 450, '한성기계', 'HS-2015-0001');
  INSERT INTO 설비폐기정보 VALUES (4, '2024-06-30', '노후');
  INSERT INTO 작업자급여 VALUES (101, 3800000), (102, 3200000);
`);

const 스펙수 = await db.query("SELECT count(*)::int AS 수 FROM 설비상세스펙");
const 폐기수 = await db.query("SELECT count(*)::int AS 수 FROM 설비폐기정보");
console.log(`설비 5대 중 · 스펙 등록 ${스펙수.rows[0].수}대 · 폐기 ${폐기수.rows[0].
수}대`);

```

출력 설비 5대 중 · 스펙 등록 1대 · 폐기 1대

★★ 자식 쪽이 **0줄일 수도 있습니다**. 그래서 정확히는 “1:0 또는 1” 입니다. ②번이 그렇습니다. 한 표로 합쳤다면 설비 표의 폐기일 칸이 대부분 NULL 이었을 겁니다.

★★★ 1:1 을 강제하는 장치는 **UNIQUE 또는 기본키**입니다. 위에서는 외래키 칸을 그대로 **PRIMARY KEY** 로 썼습니다. 두 번째 줄을 넣어 봅니다.

```

try {
  await db.query("INSERT INTO 설비상세스펙 VALUES ($1, $2, $3, $4, $5)", [1, 3000,
  500, "대한기계", "DH-0002"]);
  console.log("두 번째 스펙 들어감");
} catch (에러) {
  console.log("두 번째 스펙 실패:", 에러.code);
}

```

출력 두 번째 스펙 실패: 23505

}

★ 대리키(메모번호 SERIAL PRIMARY KEY)를 따로 두고 싶으면, 외래키 칸에 **설비번호 INT UNIQUE REFERENCES 설비(설비번호)** 처럼 **UNIQUE** 를 겁니다. 효과는 같습니다. ★★ 그 UNIQUE 를 빼면 위 INSERT 가 그냥 들어가고, **1:1 이 아니라 1:N 이 됩니다.** “하나만 있겠지” 하고 코드가 첫 줄만 읽으면 둘째 줄은 영영 안 보입니다.

★ 자기 참조 — 자기 표를 가리키는 외래키

섹션 4

현장의 계층은 이렇습니다.

공장 — 조립1라인 — 압입공정 / 검사공정

└ 가공2라인 — 용접공정

단계마다 표를 만들 수는 없습니다. **한 표에 담고 자기 표를 가리키게** 합니다.

```
await db.exec(`
  CREATE TABLE 조직 (
    번호      INT  PRIMARY KEY,
    이름      TEXT NOT NULL,
    상위번호 INT  REFERENCES 조직(번호)    -- ★ 자기 표를 가리킵니다
  );

  INSERT INTO 조직 VALUES
    (1, '공장', NULL),                -- ★ 맨 위는 NULL 입니다
    (2, '조립1라인', 1), (3, '가공2라인', 1),
    (4, '압입공정', 2), (5, '검사공정', 2), (6, '용접공정', 3);
`);
```

★★ 맨 위(최상위)의 상위번호 는 **NULL** 입니다. NOT NULL 을 걸면 뿌리를 못 넣습니다.

```
const 뿌리 = await db.query("SELECT 이름 FROM 조직 WHERE 상위번호 IS NULL");
console.log("최상위:", 뿌리.rows.map((줄) => 줄.이름).join(", "));
```

출력 최상위: 공장

한 단계 아래를 찾습니다. 그냥 조건 하나면 됩니다.

```
const 아래 = await db.query("SELECT 이름 FROM 조직 WHERE 상위번호 = $1 ORDER BY
번호", [2]);
console.log("조립1라인 바로 아래:", 아래.rows.map((줄) => 줄.이름).join(", "));
```

출력 조립1라인 바로 아래: 압입공정, 검사공정

한 단계 위도 같은 표에서 찾습니다.

```
const 위 = await db.query(
  "SELECT 이름 FROM 조직 WHERE 번호 IN (SELECT 상위번호 FROM 조직 WHERE 번호 = $1)",
  [4]);

console.log("압입공정의 상위:", 위.rows[0].이름);
```

출력 압입공정의 상위: 조립1라인

★ 여러 단계를 한 번에 타고 내려가려면 — “공장 아래 전부” 같은 것 — **재귀 쿼리(WITH RECURSIVE)** 가 필요합니다. 이름만 알아 두세요.

★★★ 함정: 자기가 자기 상위가 될 수 있습니다. 외래키는 이걸 못 막습니다. 6번이 6번을 가리켜도 “조직에 6번이 있으니” 통과입니다.

```
await db.query("UPDATE 조직 SET 상위번호 = 번호 WHERE 번호 = $1", [6]);

const 자기자신 = await db.query("SELECT 번호, 상위번호 FROM 조직 WHERE 번호 = 6");
console.log("용접공정의 상위번호:", 자기자신.rows[0].상위번호);
```

출력 용접공정의 상위번호: 6

★ 통과했습니다. 이 줄을 타고 올라가면 **영원히 6번을 맴돕니다**. 막는 것은 CHECK 제약입니다. 먼저 데이터를 되돌리고 겁니다.

```
await db.query("UPDATE 조직 SET 상위번호 = 3 WHERE 번호 = $1", [6]);
await db.exec("ALTER TABLE 조직 ADD CONSTRAINT 자기상위금지 CHECK (상위번호 <>
번호)");

try {
  await db.query("UPDATE 조직 SET 상위번호 = 번호 WHERE 번호 = $1", [6]);
  console.log("자기 상위 지정 성공");
} catch (에러) {
  console.log("자기 상위 지정 실패:", 에러.code);
}
```

출력 자기 상위 지정 실패: 23514

```
}
```

★★ 순환(A→B→A)은 CHECK 로도 못 막습니다. 두 줄을 각각 보면 둘 다 정상이니깐요.

```
await db.exec("UPDATE 조직 SET 상위번호 = 4 WHERE 번호 = 2");
const 순환 = await db.query("SELECT 번호, 상위번호 FROM 조직 WHERE 번호 IN (2, 4)
ORDER BY 번호");

console.log("순환:", 순환.rows.map((줄) => `${줄.번호}→${줄.상위번호}`).join(" / "));
```

★ 2번의 상위가 4번, 4번의 상위가 2번인데 DB 는 통과시킵니다. 막으려면 트리거를 짜거나 애플리케이션에서 검사해야 합니다. 자기 참조는 순환을 누가 막을지 미리 정하세요.

```
await db.exec("UPDATE 조직 SET 상위번호 = 1 WHERE 번호 = 2");
```

★★★ 설계할 때 묻는 두 질문

섹션 5

여기가 이 파일의 결론입니다. 모양을 외우지 마세요. 두 번 물으면 됩니다.

★ 질문 1. "A 하나에 B 가 여럿일 수 있나?" ★ 질문 2. "B 하나에 A 가 여럿일 수 있나?"

질문1 질문2 모양 외래키를 어디에

아니오	아니오	1:1	자식 쪽 + UNIQUE(또는 기본키)
예	아니오	1:N	B(N 쪽)에
아니오	예	1:N (방향만 반대)	A 쪽에
예	예	N:M	중간 표를 새로 만들

```
function 관계모양(A에B가여럿, B에A가여럿) {
  if (A에B가여럿 && B에A가여럿) return "N:M - 중간 표";
  if (A에B가여럿) return "1:N - 외래키는 B 쪽에";
  if (B에A가여럿) return "1:N - 외래키는 A 쪽에";
  return "1:1 - 대개는 한 표로";
}

const 사례 = [
  ["라인", "설비", true, false],
  ["설비", "점검기록", true, false],
  ["작업자", "자격", true, true],
  ["설비", "상세스펙", false, false],
  ["주문", "상품", true, true],
  ["부서", "부서장", false, false],
];

for (const [가, 나, 질문1, 질문2] of 사례) {
  console.log(`${가} ↔ ${나} - ${관계모양(질문1, 질문2)}`);
}
```


출력 라인 ↔ 설비 - 1:N - 외래키는 B 쪽에
 설비 ↔ 점검기록 - 1:N - 외래키는 B 쪽에
 작업자 ↔ 자격 - N:M - 중간 표
 설비 ↔ 상세스펙 - 1:1 - 대개는 한 표로
 주문 ↔ 상품 - N:M - 중간 표
 부서 ↔ 부서장 - 1:1 - 대개는 한 표로

★★ 그리고 질문이 하나 더 있습니다. 이게 제일 중요합니다.

★★★ “지금은 하나인데, 나중에 여럿이 될 수 있나?”

점검기록에는 담당사번이 하나입니다. 2인 1조로 바뀌면 어떻게 될까요?

```
await db.exec(`
  CREATE TABLE 점검담당 (
    점검번호 INT REFERENCES 점검기록(점검번호),
    사번      INT REFERENCES 작업자(사번),
    PRIMARY KEY (점검번호, 사번)
  );
  INSERT INTO 점검담당 SELECT 점검번호, 담당사번 FROM 점검기록 WHERE 담당사번 IS NOT
  NULL;
  ALTER TABLE 점검기록 DROP COLUMN 담당사번;
`);

const 옮긴수 = await db.query("SELECT count(*)::int AS 수 FROM 점검담당");
console.log("옮긴 관계 줄 수:", 옮긴수.rows[0].수);
```

출력 옮긴 관계 줄 수: 9

★★★ 방금 한 일: ① 표를 새로 만들고 ② 데이터를 전부 옮기고 ③ 원래 칸을 지웠습니다. 9줄이라 순식간이지만 **운영 중인 1000만 줄이면 서비스를 멈춰야 합니다.** 그리고 담당사번을 읽던 코드와 화면이 전부 깨집니다. ★ 여기서는 시범입니다. 현장이 아직 1인 담당이면 04단원의 최종 점검기록 표는

“담당사번” 을 그대로 둡니다. 05단원은 그 모양에서 시작합니다.

★ 그렇다고 반대로 가면 안 됩니다. “확실히 않으니 전부 N:M 으로” 하면 표가 두 배가 되고 조회할 때마다 중간 표를 거칩니다. ★★ “여럿이 될 수 있나?” 에 현장이 “아니오” 라고 잘라 말하면 1:N 으로 가세요.

“글쎄요” 라고 하면 그때 N:M 을 생각하세요. 균형은 거기에 있습니다.

MySQL 은 여기가 다릅니다 —————

· 파라미터가 \$1 이 아니라 ? 이고, 관계 표현 방식은 **완전히 같습니다** · 단, 오래된 MyISAM 엔진은 외래키를 무시합니다. InnoDB 를 쓰세요 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 관계의 네 가지 모양

1:N	N 쪽(자식)에	REFERENCES	라인 ↔ 설비
N:M	중간 표를 새로 만들어 양쪽	복합 기본키	작업자 ↔ 자격
1:1	자식 쪽에	UNIQUE / 기본키	설비 ↔ 상세스펙

자기 참조 같은 표의 다른 줄을 CHECK(<> 자기) 조직 ↔ 상위조직

★ 두 질문이 전부입니다. “A 하나에 B 가 여럿?” “B 하나에 A 가 여럿?” 그리고 하나 더: “**나중에 여럿이 될 수 있나?**”

직접 해 볼 것

 1 — 섹션 1 의 라인_코마 에 ‘D’ 를 ‘2,20’ 으로 넣어 보세요.

``LIKE '%2%'`` 로 몇 줄이 걸리나요? 몇 줄이 오탐인가요?

 2 — 작업자자격에서 PRIMARY KEY (사번, 자격코드) 를 지우고 다시 돌려 보세요.

중복 INSERT 가 통과합니까? 그 뒤 “용접기능사 보유자” 를 세면 몇 명입니까?

 3 — 조직에서 CHECK (상위번호 <> 번호) 를 지운 뒤 1번(공장)의 상위번호를

6번으로 바꿔 보세요. 통과합니까? 그럼 최상위는 몇 개입니까?

 4 — “설비 하나에 부품이 여럿?” “부품 하나가 여러 설비에?” 두 질문을 해 보고,

답에 따라 표를 직접 만들어 데이터를 넣어 보세요.

 5 — 점검담당으로 옮긴 뒤 점검 1번에 이기사(102)를 한 명 더 넣어 보세요.

이제 점검 1번의 담당자는 몇 명입니까? 옛날 구조로는 가능했을까요?

자주 하는 실수

이런 실수를 합니다

외래키를 1 쪽에 검

라인 표에 설비번호 칸을 두면 라인 하나에 설비 하나뿐입니다. ★ 외래키는 언제나 **N 쪽(자식)** 에 둡니다. 예외가 없습니다.

이런 실수를 합니다

N:M 을 콤마 문자열로 처리

'1,2' 로 담으면 LIKE '%2%' 가 '12' 를 잡습니다. 섹션 1 에서 실제로 봤습니다. ★ 자격코드1, 자격코드2 처럼 칸을 번호로 늘리는 것도 같은 실수입니다.

이런 실수를 합니다

중간 표에 복합 기본키를 안 검

같은 사람에게 같은 자격이 두 줄 생겨 인원이 부풀려집니다. ★ 에러가 안 나서 몇 달 뒤에 발견됩니다. 가장 비싼 실수입니다.

이런 실수를 합니다

1:1 인데 UNIQUE 를 안 검

막는 장치가 없으면 그건 1:1 이 아니라 1:N 입니다. 코드가 첫 줄만 읽으면 둘째 줄은 안 보입니다.

이런 실수를 합니다

자기 참조에서 순환을 안 막음

외래키도 CHECK 도 A→B→A 를 못 막습니다. 재귀로 타는 코드가 무한 반복에 빠집니다.

이런 실수를 합니다

성급한 N:M — 그리고 그 반대

“혹시 모르니 전부 중간 표로” 하면 표가 두 배, 쿼리가 두 배 복잡해집니다. 반대로 “나중에 바꾸지 뭐” 도 위험합니다. 1:N → N:M 은 칸 추가가 아니라 **데이터 이사**입니다.

```
await db.close();
```

언제 정규화를 깨나

RUN node 개념05_언제_정규화를_깨나.js

★ 이 파일은 20초쯤 걸립니다. 점검기록 10만 줄을 진짜로 만들고 진짜로 켵니다.

개념01~04 는 계속 “나누세요” 였습니다. 이 파일은 그 반대입니다. 가끔은 일부러 안 나눕니다.

★★★ 그런데 순서를 절대 뒤집으면 안 됩니다.

먼저 정규화하고 → 재 보고 → 그 다음에 켵니다.

처음부터 “빠를 것 같아서” 안 나누는 것은 최적화가 아닙니다. 그냥 틀린 설계입니다. 개념01 의 통짜 표로 되돌아가는 것입니다.

★ “정규화 안 해도 된다” 는 이야기가 아닙니다. 정확히 반대입니다. 정규화를 알아야 켵 수 있습니다.

무엇이 어긋날지 모르면 켵는 게 아니라 그냥 사고를 내는 것입니다. 세 가지를 봅시다. ① 집계 칼럼 — 성능 때문에 켵니다. 대가가 있습니다 ② 이력 데이터 복사 — 성능이 아니라 그게 진짜 맞는 설계입니다 ③ JSONB — 편한데, 읽는 것이 생각보다 많습니다

```
import { PGLite } from "@electric-sql/pglite";
const db = await PGLite.create();
```

여러 번 재고 중앙값을 켵니다. 한 번 재고 결론 내리면 안 됩니다.

```
async function 재기(횟수, 할일) {
  const 잔것 = [];
  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    await 할일();
    잔것.push(performance.now() - 시작);
  }
  return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}
```

섹션 1 — ★★ 집계 칼럼

현장에서 이런 요청이 옵니다. “설비 목록 화면에 점검을 몇 번 받았는지 같이 보여 주세요.” 정규화된 표에서 점검 횟수는 어디에도 저장돼 있지 않습니다. 세어야 나옵니다. 같은 사실을 두 군데에 두지 않는 게 정규화니까요. 진짜 규모로 만들어 봅시다.

```

await db.exec(`
  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
    라인코드 TEXT NOT NULL, 도입년도 INT NOT NULL
  );
  INSERT INTO 설비 (설비번호, 이름, 라인코드, 도입년도)
  SELECT 번호,
    (ARRAY['컨베이어', '프레스', '용접로봇', '검사기', '포장기'])[1 + 번호 % 5] ||
    ' ' || 번호 || '호',
    (ARRAY['A', 'B', 'C'])[1 + 번호 % 3], 2010 + 번호 % 15
  FROM generate_series(1, 200) AS 번호;

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT NOT NULL REFERENCES 설비(설비번호),
    점검일 DATE NOT NULL, 결과 TEXT NOT NULL, 점수 INT
  );
  INSERT INTO 점검기록 (점검번호, 설비번호, 점검일, 결과, 점수)
  SELECT 번호, 1 + 번호 % 200, DATE '2024-01-01' + (번호 % 365),
    CASE WHEN 번호 % 7 = 0 THEN '주의' ELSE '정상' END, 60 + 번호 % 40
  FROM generate_series(1, 100000) AS 번호;
`);
const 규모 = await db.query(`
  SELECT (SELECT count(*)::int FROM 설비) AS 설비수,
    (SELECT count(*)::int FROM 점검기록) AS 점검수`;
console.log(`설비 ${규모.rows[0].설비수}대 · 점검기록 ${규모.rows[0].점검수}줄`);

```

출력 설비 200대 · 점검기록 100000줄

방법 A: 매번 썹니다 (정규화 유지)

```

const 매번세기 = `
  SELECT 설.설비번호, 설.이름,
    (SELECT count(*)::int FROM 점검기록 기 WHERE 기.설비번호 = 설.설비번호) AS
  점검횟수
  FROM 설비 설 ORDER BY 설.설비번호`;
const 첫화면 = await db.query(매번세기);
console.log("첫 줄:", JSON.stringify(첫화면.rows[0]));

```

출력 첫 줄: {"설비번호":1,"이름":"프레스 1호","점검횟수":500}

```

const 매번ms = await 재기(5, () => db.query(매번세기));
console.log(`매번 COUNT: ${매번ms.toFixed(1)} ms`);

```

출력 매번 COUNT: 2118.7 ms

값이 다를 수 있음

★★ 2초입니다. 화면 하나 여는 데 2초면 사람은 “고장났다” 합니다. 설비 200대마다 점검기록 10만 줄을 흘립니다. 2천만 번을 봅니다.

★ 제일 먼저 할 일은 “칸을 만들자” 가 아니라 **색인**입니다. 06단원에서 합니다. 이 파일은 “색인으로도 안 되면” 다음 이야기입니다.

방법 B: 설비 표에 점검횟수 칸을 둡니다 (정규화를 깬다) ———

```
await db.exec(`
  -- ★ 여기가 일부러 깬 곳입니다. 이 세 줄을 반드시 남기세요.
  -- [역정규화 · 집계 칼럼]
  -- 왜: 설비 목록 화면이 매번 COUNT 로 2초 걸림 (2024-05 측정)
  -- 대가: 점검기록이 바뀔 때 같이 안 고치면 조용히 어긋납니다
  -- 지키는 법: 트리거 + 밤마다 배치 재계산 + 어긋남 검사 쿼리
  ALTER TABLE 설비 ADD COLUMN 점검횟수 INT NOT NULL DEFAULT 0;
  UPDATE 설비 SET 점검횟수 =
    (SELECT count(*)::int FROM 점검기록 기 WHERE 기.설비번호 = 설비.설비번호);
`);
const 칸에서읽기 = `SELECT 설비번호, 이름, 점검횟수 FROM 설비 ORDER BY 설비번호`;
const 칸ms = await 재기(5, () => db.query(칸에서읽기));
console.log(`집계 칼럼: ${칸ms.toFixed(1)} ms`);
```

출력 집계 칼럼: 2.2 ms
값이 다를 수 있음

```
console.log("집계칼럼이 더 빠르거나, 칸ms < 매번ms);
```

출력 집계칼럼이 더 빠르거나: true

```
console.log(`몇 배 빠르거나: 약 ${Math.round(매번ms / 칸ms)}배`);
```

출력 몇 배 빠르거나: 약 959배
값이 다를 수 있음

빨라도 값이 다르면 소용없습니다. 같은지 확인합니다.

```
const 칸결과 = await db.query(칸에서읽기);
console.log("두 방법의 값이 같은가:", 칸결과.rows[0].점검횟수 === 첫화면.rows[0].점검횟수);
```

출력 두 방법의 값이 같은가: true

★★★ 대가: 어긋납니다 ———

점검을 하나 더 넣습니다. 그런데 **점검횟수 갱신을 깜빡합니다**. 상상이 아닙니다. · 새로 온 사람이 INSERT 만 하고 UPDATE 가 있는 줄 모릅니다 · 배치 스크립트가 다른 경로로 데이터를 밀어 넣습니다 · 관리자

화면에서 직접 SQL 을 돌립니다

```
const 사고전 = await db.query("SELECT 이름, 점검횟수 FROM 설비 WHERE 설비번호 = 7");
console.log(`${사고전.rows[0].이름} 화면 표시: ${사고전.rows[0].점검횟수}건`);
```

출력 용접로봇 7호 화면 표시: 500건

```
await db.query(`
  INSERT INTO 점검기록 (점검번호, 설비번호, 점검일, 결과, 점수)
  VALUES (200001, 7, DATE '2024-12-30', '정상', 88)`);
```

↑ UPDATE 설비 SET 점검횟수 = 점검횟수 + 1 을 안 했습니다. 깜빡한 것입니다.

```
const 칸값 = await db.query("SELECT 점검횟수 FROM 설비 WHERE 설비번호 = 7");
const 진짜값 = await db.query("SELECT count(*)::int AS 진짜 FROM 점검기록 WHERE
설비번호 = 7");
console.log(`화면(집계칼럼): ${칸값.rows[0].점검횟수}건 · 목록(진짜):
${진짜값.rows[0].진짜}건`);
```

출력 화면(집계칼럼): 500건 · 목록(진짜): 501건

```
console.log("집계칼럼이 실제와 맞는가:", 칸값.rows[0].점검횟수 === 진짜값.rows[0].
진짜);
```

출력 집계칼럼이 실제와 맞는가: false

★★★ “화면에는 500건이라고 나오는데 목록을 열면 501건입니다.” 에러도 안 나고, 로그도 안 남고, 아무도 모릅니다. 몇 달 뒤에 현장 반장이 “숫자가 안 맞는데요” 하고 전화합니다.

★ 정규화는 이 사고를 **구조적으로 불가능하게** 만든 것이었습니다.

사실이 한 군데에만 있으면 어긋날 데가 없으니까요.
깡다는 것은 그 보호막을 스스로 벗는 일입니다.

★ **어긋남 검사 쿼리 (깡 때 반드시 같이 만드세요)**

```
const 검사쿼리 = `
  SELECT 설.설비번호, 설.이름, 설.점검횟수 AS 칸값,
         (SELECT count(*)::int FROM 점검기록 기 WHERE 기.설비번호 = 설.설비번호) AS
진짜값
  FROM 설비 설
  WHERE 설.점검횟수 <> (SELECT count(*)::int FROM 점검기록 기 WHERE 기.설비번호 = 설.
설비번호)
  ORDER BY 설.설비번호`;
const 어긋난것 = await db.query(검사쿼리);
console.log("어긋난 설비 수:", 어긋난것.rows.length);
```

출력 어긋난 설비 수: 1

```
for (const 줄 of 어긋난것.rows) {  
  console.log(`· ${줄.이름} - 칸 ${줄.칸값} / 진짜 ${줄.진짜값}`);  
}
```

출력 · 용접로봇 7호 - 칸 500 / 진짜 501

★ 이 쿼리를 매일 새벽에 돌리고, 한 줄이라도 나오면 알림을 보내세요.

어긋나지 않게 하는 세 가지

① 트랜잭션 — INSERT 와 UPDATE 를 묶어서 둘 다 되거나 둘 다 안 되게 합니다 ★ 장점: 가장 정확하고 코드가 눈에 보입니다 ★ 단점: **INSERT 하는 곳이 여러 군데면 전부 고쳐야 합니다.** 하나만 빠져도 어긋납니다 ★ 07단원에서 제대로 합니다. 여기서는 이름만 알아 두세요.

② 트리거 — 데이터베이스가 알아서 고쳐 줍니다 ★ 문법은 외우지 마세요. “자동으로 해 주는 장치가 있다”만 아세요.

```
await db.exec(`  
  CREATE FUNCTION 점검횟수_맞추기() RETURNS TRIGGER AS $$  
  BEGIN  
    IF TG_OP = 'INSERT' THEN  
      UPDATE 설비 SET 점검횟수 = 점검횟수 + 1 WHERE 설비번호 = NEW.설비번호;  
    ELSIF TG_OP = 'DELETE' THEN  
      UPDATE 설비 SET 점검횟수 = 점검횟수 - 1 WHERE 설비번호 = OLD.설비번호;  
    END IF;  
    RETURN NULL;  
  END;  
  $$ LANGUAGE plpgsql;  
  
  CREATE TRIGGER 점검횟수_자동 AFTER INSERT OR DELETE ON 점검기록  
  FOR EACH ROW EXECUTE FUNCTION 점검횟수_맞추기();  
`);  
const 트전 = await db.query("SELECT 이름, 점검횟수 FROM 설비 WHERE 설비번호 = 9");  
console.log(`${트전.rows[0].이름} 트리거 걸기 직후: ${트전.rows[0].점검횟수}건`);
```

출력 포장기 9호 트리거 걸기 직후: 500건

이번에는 UPDATE 를 **일부러 안 씁니다.** INSERT 만 합니다.


```
await db.query(`
  INSERT INTO 점검기록 (점검번호, 설비번호, 점검일, 결과, 점수)
  VALUES (200002, 9, DATE '2024-12-31', '정상', 90)`);
const 트후 = await db.query("SELECT 점검횟수 FROM 설비 WHERE 설비번호 = 9");
const 트진짜 = await db.query("SELECT count(*)::int AS 진짜 FROM 점검기록 WHERE
설비번호 = 9");
console.log(`INSERT 만 했는데 칸: ${트후.rows[0].점검횟수}건 · 진짜:
${트진짜.rows[0].진짜}건`);
```

출력 INSERT 만 했는데 칸: 501건 · 진짜: 501건

```
console.log("트리거가 맞춰 졌나:", 트후.rows[0].점검횟수 === 트진짜.rows[0].진짜);
```

출력 트리거가 맞춰 졌나: true

DELETE 도 처리해야 합니다. 안 그러면 지울 때 어긋납니다.

```
await db.query("DELETE FROM 점검기록 WHERE 점검번호 = 200002");
const 삭후 = await db.query("SELECT 점검횟수 FROM 설비 WHERE 설비번호 = 9");
const 삭진짜 = await db.query("SELECT count(*)::int AS 진짜 FROM 점검기록 WHERE
설비번호 = 9");
console.log(`DELETE 후 칸: ${삭후.rows[0].점검횟수}건 · 진짜: ${삭진짜.rows[0].진짜}
건`);
```

출력 DELETE 후 칸: 500건 · 진짜: 500건

★ 트리거의 단점 (공짜가 아닙니다) · **눈에 안 보입니다**. 코드에 UPDATE 가 없는데 값이 바뀌어 디버깅이 어렵습니다 · **성능도 공짜가 아닙니다**. INSERT 한 줄마다 UPDATE 가 한 번 더 듭니다 · **과거는 못 고칩니다**. 트리거 걸기 전에 어긋난 것은 그대로입니다

```
const 여전히 = await db.query(검사쿼리);
console.log("트리거를 걸었는데도 어긋난 설비 수:", 여전히.rows.length);
```

출력 트리거를 걸었는데도 어긋난 설비 수: 1

★★ 7번 설비는 여전히 틀렸습니다. 트리거는 **앞으로만** 지켜 줍니다.

③ 배치 재계산 — 밤에 한 번 전부 다시 셉니다

```
const 배치시작 = performance.now();
const 배치결과 = await db.query(`
  UPDATE 설비 SET 점검횟수 =
  (SELECT count(*)::int FROM 점검기록 기 WHERE 기.설비번호 = 설비.설비번호)`);
const 배치ms = performance.now() - 배치시작;
console.log(`배치 재계산: ${배치ms.toFixed(1)} ms`);
```

출력
값이 다를 수 있음

배치 재계산: 1885.6 ms

```
console.log("고친 줄 수:", 배치결과.affectedRows);
```

출력 고친 줄 수: 200

```
const 배치후 = await db.query(검사쿼리);  
console.log("배치 후 어긋난 설비 수:", 배치후.rows.length);
```

출력 배치 후 어긋난 설비 수: 0

★ 배치 재계산이 셋 중 가장 **단순하고 튼튼합니다**. 무슨 일이 있었든 다시 세니까요. 대신 **밤에 한 번 돌 때까지 하루 동안은 틀립니다**. ★★ 현실적인 답은 “하나만 고르기” 가 아닙니다. **트리거로 앞을 지키고 + 배치로 뒤를 닦고 + 검사 쿼리로 감시합니다**. 셋 다 합니다.

★ 언제 집계 칼럼을 쓰나 — 아래 셋이 전부 맞을 때만 —————

① 읽기가 훨씬 많은가 — 목록 화면은 하루 수천 번, 점검 등록은 하루 수십 번 ② 재 봤더니 진짜 느린가 — 감이 아니라 숫자로. 색인으로 먼저 풀어 봤는가 ③ 약간 틀려도 되는 값인가 — 점검 횟수는 하루 늦어도 됩니다. 잔액은 안 됩니다

★★★ 특히 세 번째. **돈이 걸린 값에는 집계 칼럼을 쓰지 마세요**. “계좌 잔액이 하루 동안 틀렸습니다” 는 사고 보고서에 올라갑니다.

섹션 2 — ★ 이력 데이터는 일부러 복사합니다

★★ 섹션 1 의 집계 칼럼은 **성능 때문에** 어쩔 수 없이 깐 것입니다. 이번 것은 다릅니다. **복사해 두는 게 진짜 맞는 설계입니다**. 성능과 아무 상관이 없습니다. 안 하면 그냥 **틀린 데이터**가 됩니다.

```
await db.exec(`  
  CREATE TABLE 작업자 (사번 INT PRIMARY KEY, 이름 TEXT NOT NULL);  
  INSERT INTO 작업자 VALUES (101, '김반장'), (102, '이기사'), (103, '박주임');  
  
  CREATE TABLE 점검이력 (  
    점검번호 INT PRIMARY KEY, 설비이름 TEXT NOT NULL, 점검일 DATE NOT NULL,  
    담당사번 INT REFERENCES 작업자(사번) ON DELETE SET NULL  
  );  
  INSERT INTO 점검이력 VALUES  
    (1, '컨베이어 1호', DATE '2024-01-05', 101),  
    (2, '프레스 1호', DATE '2024-01-12', 101),  
    (3, '검사기 1호', DATE '2024-01-25', 103);  
`);
```

이름은 작업자 표에만 있습니다. 개념02~04 가 가르친 그대로입니다. ★ 아래 JOIN 문법은 지금 이해 못 해도 됩니다. **05단원에서 제대로 합니다**.

```
const 이력조회 = `
  SELECT 기.점검일::text AS 점검일, 기.설비이름, 자.이름 AS 담당자
  FROM 점검이력 기 LEFT JOIN 작업자 자 ON 자.사번 = 기.담당사번
  WHERE 기.점검번호 = $1`;
const 개명전 = await db.query(이력조회, [1]);
console.log("개명 전:", JSON.stringify(개명전.rows[0]));
```

```
출력      개명 전: {"점검일":"2024-01-05","설비이름":"컨베이어
1호","담당자":"김반장"}
```

김반장이 승진했습니다. 이름이 바뀝니다.

```
await db.query("UPDATE 작업자 SET 이름 = '김공장장' WHERE 사번 = 101");
const 개명후 = await db.query(이력조회, [1]);
console.log("개명 후:", JSON.stringify(개명후.rows[0]));
```

```
출력      개명 후: {"점검일":"2024-01-05","설비이름":"컨베이어
1호","담당자":"김공장장"}
```

```
console.log("과거 기록이 그대로인가:", 개명후.rows[0].담당자 === 개명전.rows[0].
담당자);
```

```
출력      과거 기록이 그대로인가: false
```

★★★ 2024년 1월 5일에 점검한 사람은 **김반장**이었습니다. 그날 “김공장장” 이라는 사람은 없었습니다. 기록이 바뀐 것입니다. 감사에서 이 표를 뽑으면 존재하지 않았던 사람이 점검한 것으로 나옵니다.

★ 더 확실한 예: 퇴사입니다.

```
await db.query("DELETE FROM 작업자 WHERE 사번 = 103");
const 퇴사후 = await db.query(이력조회, [3]);
console.log("퇴사 후:", JSON.stringify(퇴사후.rows[0]));
```

```
출력      퇴사 후: {"점검일":"2024-01-25","설비이름":"검사기 1호","담당자":null}
```

★★ 담당자가 **null** 입니다. 누가 점검했는지 영영 알 수 없습니다. RESTRICT 를 걸면 삭제는 막히지만 퇴사자를 영원히 못 지웁니다. 둘 다 답이 아닙니다. 문제가 다른 데 있기 때문입니다.

```
await db.exec(`
  -- [일부러 복사 · 이력 보존]
  -- 왜: 점검 시점의 담당자 이름은 '그때의 사실'입니다.
  --   작업자 이름이 바뀌거나 퇴사해도 절대 바뀌면 안 됩니다.
  -- * 이건 성능 최적화가 아닙니다. 중복이라고 지우지 마세요.
  ALTER TABLE 점검이력 ADD COLUMN 담당자이름_당시 TEXT;
`);
```

★★ 그런데 이미 늦었습니다. 101번은 바뀌었고 103번은 지워졌습니다. 아래는 우리가 옛 이름을 알고 있어서 손으로 채운 것입니다. 실제 운영에서는 못 채웁니다. 그래서 처음부터 복사해야 합니다.

```
await db.exec(`
  UPDATE 점검이력 SET 담당자이름_당시 = '김반장' WHERE 담당사번 = 101;
  UPDATE 점검이력 SET 담당자이름_당시 = '박주임' WHERE 점검번호 = 3;
`);
```

이제 이름을 또 바꿔 봅니다.

```
await db.query("UPDATE 작업자 SET 이름 = '김대표' WHERE 사번 = 101");
const 복사본 = await db.query(`
  SELECT 점검번호, 점검일::text AS 점검일, 담당자이름_당시, 담당사번
  FROM 점검이력 ORDER BY 점검번호`);
for (const 줄 of 복사본.rows) {
  console.log(`${줄.점검일} · 그때 담당: ${줄.담당자이름_당시} · 지금 사번: ${줄.
  담당사번}`);
}
```

```
출력      2024-01-05 · 그때 담당: 김반장 · 지금 사번: 101
          2024-01-12 · 그때 담당: 김반장 · 지금 사번: 101
          2024-01-25 · 그때 담당: 박주임 · 지금 사번: null
```

```
const 지금이름 = await db.query("SELECT 이름 FROM 작업자 WHERE 사번 = 101");
console.log("지금 101번 이름:", 지금이름.rows[0].이름);
```

```
출력      지금 101번 이름: 김대표
```

```
console.log("과거 기록이 안 바뀌었나:", 복사본.rows[0].담당자이름_당시 === "김반장");
```

```
출력      과거 기록이 안 바뀌었나: true
```

★ 퇴사한 박주임의 이름도 남았습니다. 사번은 null 이 됐지만 기록은 살았습니다. ★ 같은 원리가 적용되는 것들 · 주문의 상품가격 — 가격이 올라도 작년 주문 금액이 바뀌면 안 됩니다 · 계약서의 회사 주소 — 이사 갔다고 옛 계약서 주소가 바뀌면 안 됩니다 · 세금계산서의 상호 — 발행 시점의 상호가 그대로 남아야 합니다 · 급여명세서의 직급 — 승진했다고 작년 명세서 직급이 바뀌면 안 됩니다

★★ 판단 기준은 한 줄입니다.

```
**"이 값은 지금의 사실인가, 그때의 사실인가?"**
```

· 지금의 사실 → 이어서 봅니다 (외래키). 설비의 현재 소속 라인 · 그때의 사실 → 복사해 둡니다. 점검 당시의 담당자 이름

★ 헷갈리면 물어보세요. "원본이 바뀌면 이 값도 같이 바뀌어야 하나?"

★★★ 섹션 1 과 헛갈리지 마세요. 집계 칼럼이 원본과 다르면 **버그**입니다. 맞춰야 합니다. 담당자이름_당시가 다른 것은 **정상**입니다. 맞추면 기록이 망가집니다.

섹션 3 — JSONB: 언제 쓰고 언제 쓰면 안 되나

설비마다 스펙이 제각각입니다. 컨베이어는 길이와 속도, 프레스는 톤수와 금형. 전부 칸으로 만들면 수십 칸이 되고 대부분 NULL 입니다. 이럴 때 JSONB 를 씁니다.

```
await db.exec(`
  CREATE TABLE 설비스펙 (설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL, 스펙 JSONB NOT
  NULL);
  INSERT INTO 설비스펙 VALUES
    (1, '컨베이어 1호', '{"제조사":"한성","길이m":12.5,"옵션":["비전"]}' ),
    (2, '프레스 1호', '{"제조사":"대영","톤수":200,"금형":{"번호":"M-
7","수명":50000}}' ),
    (3, '용접로봇 1호', '{"제조사":"한성","축수":6,"옵션":["비전","자동보정"]}' );
`);
const 한줄 = await db.query("SELECT 스펙 FROM 설비스펙 WHERE 설비번호 = 2");
```

★ PGlite 는 JSONB 를 **자동으로 자바스크립트 object** 로 파싱해서 줍니다. `JSON.parse` 를 따로 부르지 않습니다.

```
console.log("타입:", typeof 한줄.rows[0].스펙);
```

```
출력      타입: object
```

```
console.log("바로 꺼내 쓰기:", 한줄.rows[0].스펙.금형.번호);
```

```
출력      바로 꺼내 쓰기: M-7
```

SQL 안에서 꺼낼 때는 화살표를 씁니다. 하나(→)는 JSON 을 그대로, 둘(→>)은 글자로 꺼냅니다.

```
const 연산자 = await db.query(`
  SELECT 스펙 -> '금형' AS 화살, 스펙 ->> '금형' AS 두화살
  FROM 설비스펙 WHERE 설비번호 = 2`);
console.log("-> 결과 타입:", typeof 연산자.rows[0].화살);
```

```
출력      -> 결과 타입: object
```

```
console.log("->> 결과 타입:", typeof 연산자.rows[0].두화살);
```

```
출력      ->> 결과 타입: string
```

@> 는 "이 내용을 포함하는가" 입니다.

```
const 포함검색 = await db.query(`
  SELECT 이름 FROM 설비스펙 WHERE 스펙 @> '{"제조사":"한성"}' ORDER BY 설비번호`);
console.log("한성 제조:", 포함검색.rows.map((줄) => 줄.이름).join(", "));
```

출력 한성 제조: 컨베이어 1호, 용접로봇 1호

★ 써도 되는 때

· 설비마다 제각각인 스펙 — 칸으로 만들면 대부분 NULL 인 칸이 수십 개 · 외부 API 응답 원본 보관 —
나중에 뭐가 필요할지 몰라 통째로 남겨 둬 · 로그·이벤트 기록 — 검색보다 쌓는 게 목적 · 자주 바뀌는
설정값 — 칸을 추가할 때마다 배포하기 싫음

★★ 쓰면 안 되는 때 (이쪽이 더 중요합니다)

(1) 자주 검색하거나 정렬하는 값 — 느립니다. 재 봅니다.

```
await db.exec(`
  CREATE TABLE 스펙속도 (번호 INT PRIMARY KEY, 제조사 TEXT NOT NULL, 스펙 JSONB NOT
  NULL);
  INSERT INTO 스펙속도
  SELECT 번호, (ARRAY['한성','대영','신광','동방'])[1 + 번호 % 4],
    jsonb_build_object(
      '일련', 번호, '길이m', 번호 % 30, '속도', '1.2m/s', '점검주기', 30,
      '메모', '정기 점검 대상 설비입니다',
      '제조사', (ARRAY['한성','대영','신광','동방'])[1 + 번호 % 4])
  FROM generate_series(1, 200000) AS 번호;
`);
```

같은 값을 일반 칸에도, JSONB 안에도 넣어 뒀습니다. 똑같은 조건으로 세어 봅니다.

```
const 일반칸ms = await 재기(5, () =>
  db.query("SELECT count(*)::int AS 개수 FROM 스펙속도 WHERE 제조사 = '한성'"));
const JSONB칸ms = await 재기(5, () =>
  db.query("SELECT count(*)::int AS 개수 FROM 스펙속도 WHERE 스펙 ->> '제조사' =
  '한성'"));
console.log(`일반 칸: ${일반칸ms.toFixed(1)} ms · JSONB: ${JSONB칸ms.toFixed(1)}
ms`);
```

출력 일반 칸: 36.5 ms · JSONB: 64.6 ms
값이 다를 수 있음

```
console.log("일반 칸이 더 빠른가:", 일반칸ms < JSONB칸ms);
```

출력 일반 칸이 더 빠른가: true

★ JSONB 는 줄마다 문서를 열어서 키를 찾아야 합니다. 일반 칸은 그냥 읽습니다. ★ JSONB 에도 색인 (GIN)을 걸 수 있습니다. **06단원**에서 합니다. (2) ★ 제약을 못 겁니다. 해 봅니다.

```
await db.exec(`
  CREATE TABLE 라인 (라인코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL);
  INSERT INTO 라인 VALUES ('A', '조립1라인'), ('B', '가공2라인');
`);
const 못거는것 = [
  ["외래키", `ALTER TABLE 설비스펙 ADD FOREIGN KEY ((스펙 ->> '라인')) REFERENCES
라인(라인코드)`],
  ["UNIQUE", `ALTER TABLE 설비스펙 ADD UNIQUE ((스펙 ->> '제조사'))`],
  ["NOT NULL", `ALTER TABLE 설비스펙 ALTER COLUMN (스펙 ->> '제조사') SET NOT NULL`],
];
for (const [무엇, 문장] of 못거는것) {
  try {
    await db.query(문장);
    console.log(`${무엇}: 걸렸습니다`);
  } catch (오류) {
    console.log(`${무엇}: 실패 (${오류.code})`);
  }
}
```

```
출력      외래키: 실패 (42601)
          UNIQUE: 실패 (42601)
          NOT NULL: 실패 (42601)
```

★★ 42601 은 **문법 오류**입니다. “값이 틀렸다” 가 아닙니다. PostgreSQL 은 JSONB 안의 값에 제약을 거는 문법을 아예 갖고 있지 않습니다. 즉 **처음부터 못 하는 일**입니다. 02단원의 제약이 전부 사라집니다.

```
await db.query(`INSERT INTO 설비스펙 VALUES (7, '절단기 1호', '{}')`);
const 빈것 = await db.query("SELECT 스펙 ->> '제조사' AS 제조사 FROM 설비스펙 WHERE
설비번호 = 7");
console.log("스펙 NOT NULL 인데 빈 객체가 통과, 제조사:", 빈것.rows[0].제조사);
```

```
출력      스펙 NOT NULL 인데 빈 객체가 통과, 제조사: null
```

(3) ★ 오타를 못 막습니다. 셋 다 “A라인 설비” 라고 넣은 것입니다.

```
await db.exec(`
  INSERT INTO 설비스펙 VALUES (8, '검사기 1호', '{"제조사":"한성","라인":"A"}');
  INSERT INTO 설비스펙 VALUES (9, '포장기 1호', '{"제조사":"한성","line":"A"}');
  INSERT INTO 설비스펙 VALUES (10, '도장기 1호', '{"제조사":"한성","라인":"A"}');
`);
const A라인 = await db.query(`
  SELECT count(*)::int AS 개수 FROM 설비스펙 WHERE 스펙 @> '{"라인":"A"}';
console.log(`A라인으로 넣은 것 3개 · 검색되는 것 ${A라인.rows[0].개수}개`);
```

출력 A라인으로 넣은 것 3개 · 검색되는 것 1개

```
const 키목록 = await db.query(`
  SELECT 설비번호, jsonb_object_keys(스펙) AS 키
  FROM 설비스펙 WHERE 설비번호 IN (8, 9, 10) ORDER BY 설비번호, 키`);
console.log("들어간 키:", 키목록.rows.map((줄) => `${줄.설비번호}=${줄.키}`).join("
"));
```

출력 들어간 키: 8=라인 8=제조사 9=line 9=제조사 10=라인 10=제조사

★★ 9번은 키 이름을 'line' 으로 썼고, 10번은 값에 공백이 붙었습니다. 데이터베이스는 **아무 말도 안 했습니다**. 일반 칸이었다면 없는 칸 이름은 42703 으로 즉시 거절당했을 것입니다. ★ 느린 건 색인으로 고칩니다. 틀린 데이터는 못 고칩니다.

★★★ 결론: **칸으로 정할 수 있으면 칸으로 하세요**. JSONB 는 정말 정할 수 없을 때만. 자주 쓰는 값은 칸으로 꺼내 두세요.

MySQL 은 여기가 다릅니다

· JSONB 가 아니라 JSON 입니다 (내부 저장 방식이 다릅니다) · @> 대신 `JSON_CONTAINS()` 를 씁니다. - >> 는 같습니다 · 트리거 문법이 다릅니다 (CREATE FUNCTION 없이 바로 BEGIN ... END) ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

섹션 4 — ★★★ 깨기 전 체크리스트

```
const 체크리스트 = [
  "① 진짜로 느린가? 재 봤는가? (감이 아니라 숫자로)",
  "② 색인으로 먼저 풀 수 있나? (06단원 - 대개 이걸로 풀립니다)",
  "③ 깨면 무엇이 어긋날 수 있나? 어긋나면 누가 아픈가?",
  "④ 어긋난 것을 어떻게 찾아낼 것인가? (검사 쿼리를 같이 만드세요)",
  "⑤ 되돌릴 수 있나? (칸을 지우면 원래대로 돌아오나)",
];
for (const 항목 of 체크리스트) {
  console.log(항목);
}
```

출력 ① 진짜로 느린가? 재 봤는가? (감이 아니라 숫자로)
 ② 색인으로 먼저 풀 수 있나? (06단원 - 대개 이걸로 풀립니다)
 ③ 깨면 무엇이 어긋날 수 있나? 어긋나면 누가 아픈가?
 ④ 어긋난 것을 어떻게 찾아낼 것인가? (검사 쿼리를 같이 만드세요)
 ⑤ 되돌릴 수 있나? (칸을 지우면 원래대로 돌아오나)

```
console.log("다섯 개를 다 통과했는가:", 체크리스트.length === 5);
```

출력 다섯 개를 다 통과했는가: true

★★★ 다시 한 번. **먼저 정규화하고, 재 보고 나서 깨세요.** 깨는 것은 정규화를 몰라도 되는 게 아닙니다. **알아야 할 수 있는 일입니다.** 무엇이 어긋날지 아는 사람만 그걸 감시할 수 있습니다. ★ 깬 곳에는 **반드시 주석으로 이유를 남기세요.** 안 적으면 반년 뒤 다음 사람이 “여기 중복 데이터 있네요? 정리했습니다” 하고 지웁니다. 그리고 화면이 2초가 됩니다.

정리 — 깨는 방법 세 가지

깨는 방법 얻는 것 잃는 것 지키는 법

집계 칼럼 목록 화면 수백 배 갱신을 깜빡하면 트리거 + 밤 배치

(2119ms → 2.2ms) 조용히 어긋남 + 검사 쿼리 알림

이력 값 복사 기록이 안 바뀔 칸이 늘고 원본과 달라지는 게

(원본이 바뀌어도) 용량이 조금 늘어남 정상. 맞추지 말 것

JSONB 칸 안 만들고 넣음 제약이 전부 없음 자주 쓰는 값은

(스펙이 제각각) 오타를 못 막음 칸으로 꺼내 두기

★ 셋 중 **이력 값 복사만 성능과 무관합니다.** 그건 원래 그렇게 하는 게 맞습니다. 나머지 둘은 재 보고 나서, 대가를 알고 하는 거래입니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 1의 설비를 200대에서 20대로 줄여 보세요. 여전히 느린가요? (힌트: 안 느리면 **깬 이유가 없습니다.** 그게 이 파일의 요점입니다)

▫ 직접 해보기 2

점검기록의 설비번호를 UPDATE 로 바꿔 보세요. 점검횟수가 맞나요? (힌트: 트리거는 INSERT 와 DELETE 만 처리하게 만들었습니다)

▫ 직접 해보기 3

섹션 2에서 담당자이름_당시 를 NOT NULL 로 만들면 어떻게 될까요? 지금 데이터에서 되나요? 왜 안 될까요?

▫ 직접 해보기 4

주문 표를 설계해 보세요. 상품 표에 가격이 있습니다. 복사할까요, 이어서 볼까요? “지금의 사실인가 그때의 사실인가” 로 답해 보세요.

▫ 직접 해보기 5

섹션 3 의 JSONB 에서 '제조사' 만 일반 칸으로 꺼내 보세요. (힌트: ADD COLUMN 뒤 UPDATE ... = 스펙 →> '제조사') 얼마나 빨라지나요?

자주 하는 실수

이런 실수를 합니다

재 보지도 않고 껌

“COUNT 는 느리니까” 하고 처음부터 집계 칼럼을 둡니다. 설비가 20대면 COUNT 는 1ms 도 안 걸립니다. 어긋남만 얻고 속도는 못 얻습니다. ★ 재기 전에는 깨지 마세요. 순서는 하나뿐입니다. **정규화 → 측정 → 필요하면 깨기.**

이런 실수를 합니다

집계 칼럼 갱신을 깜빡함

INSERT 하는 곳이 화면·배치·관리자 SQL 세 군데인데 한 군데만 고칩니다. 에러도 안 납니다. ★ 그래서 검사 쿼리를 **꺄 때 같이** 만듭니다. 나중에 만들면 안 만듭니다.

이런 실수를 합니다

트리거만 믿고 배치 검사를 안 함

트리거는 **앞으로만** 지킵니다. 걸기 전에 어긋난 것, 잠깐 끄고 대량 적재한 것은 남습니다. ★ 트리거 + 배치 재계산 + 검사 쿼리. 셋 다 하세요.

이런 실수를 합니다

이력 복사와 성능 역정규화를 헛갈림

담당자이름_당시가 지금 이름과 다르면 “어긋났다” 며 맞추는 배치를 만듭니다. ★★ 그건 어긋난 게 아닙니다. **그게 맞는 값입니다.** 맞추면 기록이 망가집니다. 반대로 집계 칼럼이 다른 것은 진짜 버그입니다. 둘을 반드시 구분하세요.

이런 실수를 합니다

JSONB 를 만능으로 씀

“칸 추가가 귀찮으니 전부 JSONB 로” 로 시작합니다. 반년 뒤 키 이름이 세 종류가 되고, 외래키가 없어 없는 라인코드가 들어가 있고, 검색은 느립니다. 이미 100만 줄입니다.

이런 실수를 합니다

깡 이유를 안 적음

다음 사람이 보면 그냥 중복 데이터입니다. “정리했습니다” 하고 지웁니다. ★ 스키마 옆에 **왜 · 대가 · 지키는 법** 세 줄을 남기세요. 커밋 메시지가 아니라 **스키마 옆**입니다.

```
await db.close();
```

데이터 모델링과 정규화

RUN node 연습문제.js

// TODO: 를 채우고 다시 돌려세요. 통과하면  로 바뀝니다.

정답과 “왜 그런지” 는 연습문제_정답.js 에 있습니다.

★ 처음에는 13문제 전부  입니다. 정상입니다.

★ 정답을 보기 전에 꼭 직접 해 보세요. 답을 읽는 것과 손으로 표를 나눠 보는 것은

전혀 다릅니다. 이 단원은 특히 그렇습니다.

★★ 순서는 “먼저 사고를 내고, 그 다음에 이름을 붙인다” 입니다.

1~2번에서 사고를 겪고, 3~5번에서 이름을 붙이고, 6~13번에서 나눠 다시 잇습니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

공통 예제 데이터

주인공은 「점검대장」 한 표입니다. 설비도 라인도 사람도 점검도 전부 여기 들어 있습니다.

```

await db.exec(`
CREATE TABLE 점검대장 (
    점검번호    SERIAL PRIMARY KEY,
    설비이름    TEXT NOT NULL,
    설비도입년도 INT  NOT NULL,
    라인코드    TEXT NOT NULL,
    라인이름    TEXT NOT NULL,
    라인동      TEXT NOT NULL,
    담당자사번  INT   NOT NULL,
    담당자이름  TEXT NOT NULL,
    담당자연락처 TEXT NOT NULL,
    점검일      DATE NOT NULL,
    결과        TEXT NOT NULL,
    점수        INT,
    점검내용    TEXT NOT NULL
);
INSERT INTO 점검대장
(설비이름, 설비도입년도, 라인코드, 라인이름, 라인동,
담당자사번, 담당자이름, 담당자연락처, 점검일, 결과, 점수, 점검내용) VALUES
('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
'2024-01-05', '정상', 92, '벨트 장력 정상'),
('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
'2024-02-05', '정상', 88, '롤러 윤활 완료'),
('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 102, '이기사', '010-3333-4444',
'2024-03-05', '주의', 71, '벨트 마모 관찰'),
('프레스 1호', 2019, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
'2024-01-12', '정상', 95, '유압 압력 정상'),
('프레스 1호', 2019, 'A', '조립1라인', '1동', 103, '박주임', '010-5555-6666',
'2024-02-12', '정상', 90, '금형 교체'),
('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
'2024-01-20', '정상', 97, '토치 팁 교체'),
('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
'2024-02-20', '주의', 68, '아크 불안정'),
('검사기 1호', 2018, 'B', '가공2라인', '1동', 103, '박주임', '010-5555-6666',
'2024-01-25', '정상', 85, '센서 교정'),
('포장기 1호', 2022, 'C', '포장3라인', '2동', 104, '최사원', '010-7777-8888',
'2024-02-28', '정상', 91, '실링 온도 조정');

-- 3번용: 한 칸에 값을 여러 개 쑤셔 넣은 표
CREATE TABLE 작업자라인 (
    사번 INT PRIMARY KEY, 이름 TEXT NOT NULL, 라인목록 TEXT NOT NULL
);
INSERT INTO 작업자라인 VALUES
(101, '김반장', 'A,B'), (102, '이기사', 'B'), (103, '박주임', 'A,BC'),
(104, '최사원', 'BC'), (105, '정신입', 'C');

```

-- 4번용: 기본키가 두 칸인 표

```
CREATE TABLE 설비부품 (
  설비번호 INT NOT NULL, 부품코드 TEXT NOT NULL, 부품이름 TEXT NOT NULL,
  부품단가 INT NOT NULL, 수량 INT NOT NULL,
  PRIMARY KEY (설비번호, 부품코드)
);

INSERT INTO 설비부품 VALUES
(1, 'P-100', '베어링', 12000, 4), (1, 'P-200', '벨트', 35000, 2),
(2, 'P-100', '베어링', 12000, 6), (3, 'P-300', '토치팁', 8000, 10),
(3, 'P-100', '베어링', 12000, 2);
```

-- 5번·11번·13번용: 설비 한 대가 한 줄인 표

```
CREATE TABLE 설비대장 (
  설비번호 INT PRIMARY KEY, 설비이름 TEXT NOT NULL, 도입년도 INT NOT NULL,
  라인코드 TEXT NOT NULL, 라인이름 TEXT NOT NULL, 라인동 TEXT NOT NULL
);

INSERT INTO 설비대장 VALUES
(1, '컨베이어 1호', 2015, 'A', '조립1라인', '1동'),
(2, '프레스 1호', 2019, 'A', '조립1라인', '1동'),
(3, '용접로봇 1호', 2021, 'B', '가공2라인', '1동'),
(4, '검사기 1호', 2018, 'B', '가공2라인', '1동'),
(5, '포장기 1호', 2022, 'C', '포장3라인', '2동');
```

-- 11번용: 외래키를 안 걸고 굴린 표 (고아가 들어 있습니다)

```
CREATE TABLE 생산실적 (
  실적번호 SERIAL PRIMARY KEY, 설비번호 INT NOT NULL, 날짜 DATE NOT NULL, 수량
  INT NOT NULL
);

INSERT INTO 생산실적 (설비번호, 날짜, 수량) VALUES
( 1, '2024-03-01', 120), ( 2, '2024-03-01', 80), (99, '2024-03-01', 45),
( 3, '2024-03-02', 150), (77, '2024-03-02', 30), (99, '2024-03-03', 60),
( 5, '2024-03-03', 95);
```

-- 6번·8번용: ON DELETE RESTRICT 가 걸린 표

```
CREATE TABLE 정책라인 (라인코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL);
CREATE TABLE 정책설비 (
```

```

    설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
    라인코드 TEXT REFERENCES 정책라인(라인코드) ON DELETE RESTRICT
);
INSERT INTO 정책라인 VALUES ('A', '조립1라인'), ('B', '가공2라인');
INSERT INTO 정책설비 VALUES (1, '컨베이어 1호', 'A'), (2, '프레스 1호', 'A');

-- 10번·13번용: 사람과 자격증
CREATE TABLE 사원 (사번 INT PRIMARY KEY, 이름 TEXT NOT NULL);
INSERT INTO 사원 VALUES
    (101, '김반장'), (102, '이기사'), (103, '박주임'), (104, '최사원'), (105,
'정신입');
CREATE TABLE 자격증 (자격코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL);
INSERT INTO 자격증 VALUES
    ('WELD', '용접기능사'), ('CRANE', '크레인운전'), ('SAFE', '산업안전기사');

-- 12번용: 집계 칼럼(캐시)을 따로 들고 있는 표
CREATE TABLE 설비집계 (설비이름 TEXT PRIMARY KEY, 점검횟수 INT NOT NULL);
INSERT INTO 설비집계 VALUES
    ('컨베이어 1호', 3), ('프레스 1호', 2), ('용접로봇 1호', 5),
    ('검사기 1호', 0), ('포장기 1호', 1);
`);

const 결과들 = [];

async function 채점(번호, 제목, 검사) {
    let 통과 = false;

    try {
        통과 = (await 검사()) === true;
    } catch {
        통과 = false;
    }

    결과들.push(통과);
    console.log(`${String(번호).padStart(2)}. ${제목} - ${통과 ? "✅ 통과" : "❌
아직"}`);
}

```

판단 문제 채점용. 띄어쓰기와 대소문자를 무시하고 견줄니다.

```

const 다듬기 = (값) => String(값 ?? "").replace(/\s+/g, "").toUpperCase();
const 같은목록 = (내답, 정답) => Array.isArray(내답) &&
    내답.length === 정답.length &&
    내답.every((값, 자리) => 다듬기(값) === 다듬기(정답[자리]));

```

문제 1 — 연락처 하나 바꾸는 데 몇 줄을 고쳐야 하나

담당자는 네 명뿐인데 점검이 있을 때마다 이름과 연락처가 다시 적혔습니다. 김반장(101)의 연락처가 '010-9999-0000' 으로 바뀌었습니다. **전부** 바꾸고 바뀐 줄 수를 돌려주세요.

★ 한 줄만 고치면 같은 사람의 연락처가 두 개가 됩니다. 그게 갱신 이상입니다.

```
async function 연락처바꾸기(사번, 새번호) {
```

UPDATE 를 실행하고 바뀐 줄 수(number)를 돌려주세요

```
    return -1;
}

await 채점(1, "연락처 하나 바꾸는 데 몇 줄인가", async () => {
    const 바뀐수 = await 연락처바꾸기(101, "010-9999-0000");
    const 가짓수 = await db.query(
        "SELECT count(DISTINCT 담당자연락처)::int AS 가짓수 FROM 점검대장 WHERE 담당자사번 = 101",
    );
    const 남의것 = await db.query(
        "SELECT count(*)::int AS 건수 FROM 점검대장 WHERE 담당자사번 <> 101 AND 담당자연락처 = '010-9999-0000'",
    );
    return 바뀐수 === 3 && 가짓수.rows[0].가짓수 === 1 && 남의것.rows[0].건수 === 0;
});
```

출력 1. 연락처 하나 바꾸는 데 몇 줄인가 - ❌ 아직

문제 2 — 이 사고들의 이름은 무엇인가

(가) 김반장 연락처를 바꾸려면 3줄을 다 고쳐야 합니다.

한 줄이라도 놓치면 같은 사람의 연락처가 둘이 됩니다.

(나) 아직 점검을 한 번도 안 한 신설4라인('D', '2동')을 점검대장에 넣을 수가 없습니다.

점검일도 결과도 없기 때문입니다.

(다) 포장기 1호의 점검 줄은 딱 하나입니다. 그 줄을 지우면

'포장기 1호' 라는 설비가 세상에서 사라집니다.

'갱신 이상' · '삽입 이상' · '삭제 이상' 중에서 (가)(나)(다) 순서대로 고르세요.

여기에 답을 쓰세요

```
const 답2 = null;

await 채점(2, "이 사고들의 이름은 무엇인가", async () =>
    같은목록(답2, ["갱신 이상", "삽입 이상", "삭제 이상"]),
);
```

출력 2. 이 사고들의 이름은 무엇인가 - ✖ 아직

문제 3 — 한 칸에 값을 여러 개 넣으면

작업자라인 표의 라인목록 칸에 'A,B' 처럼 쉼표로 이어 넣었습니다. B라인 작업자를 찾으려고 라인목록 LIKE '%B%' 를 쓰면 네 명이 걸립니다. 그런데 진짜 B라인은 두 명뿐입니다. 'A,BC' 와 'BC' 는 BC라인이지 B라인이 아닙니다.

칸 하나에 값 하나만 들어가게 표를 다시 만드세요. · 표 이름은 작업자라인_1정규형 · 칸은 사번(INT) · 라인코드(TEXT), 기본키는 두 칸을 묶은 복합 기본키 · 작업자라인의 값을 쉼표로 나눠 전부 넣으세요 (모두 7줄이 됩니다)

그 다음 B라인 작업자의 사번을 **오름차순 배열**로 돌려주세요. (손으로 적어 넣어도 됩니다)

```
async function B라인작업자() {
```

표를 만들고, 값을 나눠 넣고, B라인 사번 배열을 돌려주세요

```
return [];
}

await 채점(3, "한 칸에 값을 여러 개 넣으면", async () => {
    const 사번들 = await B라인작업자();
    const 출수 = await db.query("SELECT count(*)::int AS 건수 FROM 작업자라인
_1정규형");
    const 오탐 = await db.query("SELECT count(*)::int AS 건수 FROM 작업자라인 WHERE
라인목록 LIKE '%B%'");
    return (
```


여러 표를 잇기

OPEN 05_여러_표를_잇기

```
await db.exec(`
CREATE TABLE 라인 (
라인코드 TEXT PRIMARY KEY,
이름      TEXT NOT NULL,
```

01 두 표를 잇기

02 없는 쪽도 보기

03 묶어서 세기

04 쿼리 안의 쿼리

05 줄마다 계산하기 (윈도우 함수)

- 연습문제

두 표를 잇기

```
RUN node 개념01_두_표를_잇기.js
```

04단원에서 표를 나눴습니다. 설비 이름을 점검기록마다 적지 않고 설비 표에 한 번만 적었습니다. 저장할 때는 그게 맞습니다.

그런데 **사람이 보고 싶은 것은 이어진 모습**입니다. "3월에 주의 난 설비 이름이 뭐죠?" 라고 묻지, "설비번호 3번의 3월 점검 결과요" 라고 묻지 않습니다.

나눠 놓은 표를 다시 이어서 보는 것 — 그게 JOIN 입니다. 이 파일은 가장 기본인 INNER JOIN 하나만 제대로 팝니다. 짝 없는 줄은 개념02 의 LEFT JOIN 이 살립니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

예제 데이터

05단원 전체가 이 네 표를 씁니다. 사이사이 일부러 구멍을 뚫어 놔습니다. · 설비 5(포장기 1호) 는 점검기록이 없습니다 · 라인 D(신설4라인) 는 설비가 없습니다 · 점검기록 11·15 는 담당사번이 NULL 입니다

```

await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    동        TEXT NOT NULL
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드),
    도입년도 INT NOT NULL
  );

  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드)
  );

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호),
    점검일   DATE NOT NULL,
    결과     TEXT NOT NULL,
    점수     INT,
    담당사번 INT REFERENCES 작업자(사번)
  );

  INSERT INTO 라인 VALUES
    ('A', '조립1라인', '1동'),
    ('B', '가공2라인', '1동'),
    ('C', '포장3라인', '2동'),
    ('D', '신설4라인', '2동');

  INSERT INTO 설비 VALUES

```

```
(1, '컨베이어 1호', 'A', 2015),
(2, '프레스 1호', 'A', 2019),
(3, '용접로봇 1호', 'B', 2021),
(4, '검사기 1호', 'B', 2018),
(5, '포장기 1호', 'C', 2022);
```

```
INSERT INTO 작업자 VALUES
```

```
(101, '김반장', 'A'),
(102, '이기사', 'B'),
(103, '박주임', 'B'),
(104, '최사원', 'C'),
(105, '정신입', NULL);
```

```
INSERT INTO 점검기록 VALUES
```

```
( 1, 1, '2024-01-05', '정상', 92, 101),
( 2, 1, '2024-02-05', '정상', 88, 101),
( 3, 1, '2024-03-06', '주의', 71, 102),
( 4, 1, '2024-04-05', '정상', 90, 101),
( 5, 2, '2024-01-12', '주의', 65, 102),
( 6, 2, '2024-02-14', '불량', 48, 103),
( 7, 2, '2024-03-15', '정상', 88, 101),
( 8, 3, '2024-01-20', '정상', 95, 103),
( 9, 3, '2024-02-20', '정상', NULL, 103),
(10, 3, '2024-03-21', '주의', 74, 102),
(11, 3, '2024-04-22', '정상', 90, NULL),
(12, 3, '2024-05-23', '불량', 52, 103),
(13, 4, '2024-01-28', '정상', NULL, 104),
(14, 4, '2024-03-29', '정상', 88, 104),
(15, 4, '2024-05-30', '주의', 69, NULL);
`);
```

왜 이어야 하는가

섹션 1

```
const 기록만 = await db.query("SELECT 점검번호, 설비번호, 결과, 점수 FROM 점검기록
ORDER BY 점검번호 LIMIT 3");
for (const 줄 of 기록만.rows) {
  console.log(` · 점검 ${줄.점검번호} · 설비번호 ${줄.설비번호} · ${줄.결과} · ${줄.
점수}점`);
}
```

```
출력      · 점검 1 · 설비번호 1 · 정상 · 92점
          · 점검 2 · 설비번호 1 · 정상 · 88점
          · 점검 3 · 설비번호 1 · 주의 · 71점
```

```
console.log("이 결과에 설비 이름이 있나:", "이름" in 기록만.rows[0]);
```

출력 이 결과에 설비 이름이 있나: false

★ “설비번호 1” 이 무슨 설비인지 이 표만 보고는 모릅니다. 현장에 “설비번호 1이 주의입니다” 라고 하면 못 알아듣습니다. “컨베이어 1호” 라고 해야 알아듣습니다. ★ “그럼 프로그램에서 두 번 조회해 합치면 되잖아요” — 많이들 이렇게 짭니다.

```
let 던진쿼리수 = 0;

async function 프로그램에서합치기() {
  던진쿼리수 = 0;
  const 기록 = await db.query("SELECT 점검번호, 설비번호, 결과 FROM 점검기록 ORDER BY
  점검번호");
  던진쿼리수 += 1;
  const 합친것 = [];
  for (const 줄 of 기록.rows) {
```

★ 점검기록 한 줄마다 설비를 한 번씩 더 물어봅니다

```
const 설비 = await db.query("SELECT 이름 FROM 설비 WHERE 설비번호 = $1", [줄.
설비번호]);
  던진쿼리수 += 1;
  합친것.push({ 점검번호: 줄.점검번호, 설비이름: 설비.rows[0].이름, 결과: 줄.결과
});
}
return 합친것;
}

const 손으로합친것 = await 프로그램에서합치기();
console.log("던진 쿼리 수:", 던진쿼리수, "/ 얻은 줄 수:", 손으로합친것.length);
```

출력 던진 쿼리 수: 16 / 얻은 줄 수: 15

★★ 15줄을 얻으려고 쿼리를 16번 던졌습니다. 목록 1번 + 줄마다 1번씩 15번. 이것 **N+1 문제** 라고 합니다. 5만 줄이면 5만 1번입니다. JOIN 은 한 번에 가져옵니다.

```
async function 한번에가져오기() {
  const 결과 = await db.query(`
    SELECT p.점검번호, s.이름 AS 설비이름, p.결과
    FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
    ORDER BY p.점검번호
  `);
  return 결과.rows;
}

console.log("JOIN 결과가 같은가:", JSON.stringify(손으로합친것) ===
JSON.stringify(await 한번에가져오기()));
```

출력 JOIN 결과가 같은가: true

시간을 잹니다. 다섯 번 재서 중앙값을 씁니다. 한 번만 재면 어쩌다 느린 것에 휘돌립니다.

```
async function 재기(횟수, 할일) {
  const 잔것 = [];
  for (let 회차 = 0; 회차 < 횟수; 회차 += 1) {
    const 시작 = performance.now();
    await 할일();
    잔것.push(performance.now() - 시작);
  }
  return 잔것.sort((가, 나) => 가 - 나)[Math.floor(횟수 / 2)];
}

const N더하기1ms = await 재기(5, 프로그램에서합치기);
const 조인ms = await 재기(5, 한번에가져오기);
console.log(`쿼리 16번: ${N더하기1ms.toFixed(1)} ms / 쿼리 1번: ${조인ms.toFixed(1)} ms`);
```

출력 쿼리 16번: 4.4 ms / 쿼리 1번: 0.4 ms
값이 다를 수 있음

```
console.log("N+1 이 더 느린가:", N더하기1ms > 조인ms);
```

출력 N+1 이 더 느린가: true

★ 왜 이렇게까지 차이가 나는지는 06단원(색인과 실행계획) 에서 봅니다. 여기서는 “잇는 일은 DB 에게 시켜라” 만 가져가면 됩니다.

INNER JOIN ... ON

섹션 2

문법은 이렇습니다. FROM 왼쪽표 JOIN 오른쪽표 ON 짝짓는조건 ★ JOIN 이 하는 일은 **두 표의 줄을 짝지어 옆으로 붙이는 것** 입니다. 위아래로 쌓는 게 아닙니다. 개념 모델은 이렇습니다. (진짜 이렇게 도는 건 아닙니다 — 06단원) ① 설비 5줄 × 점검기록 15줄 = 75가지 짝을 전부 만들어 보고 ② 조건이 참인 것만 남긴다

1 컨베이어 1호 — 점검 1 (설비번호 1) ✓ 남김 1 컨베이어 1호 —X— 점검 5 (설비번호 2) ✕ 버림 5 포장기 1호 —X— (짝이 하나도 없음) ✕ 통째로 사라짐

```
const 조인전부 = await db.query("SELECT * FROM 설비 JOIN 점검기록 ON 설비.설비번호 = 점검기록.설비번호");
const 설비만 = await db.query("SELECT * FROM 설비");
const 점검기록만 = await db.query("SELECT * FROM 점검기록");
console.log("설비 칸 수:", 설비만.fields.length, "/ 점검기록 칸 수:", 점검기록만.fields.length);
```


출력 설비 칸 수: 4 / 점검기록 칸 수: 6

```
console.log("이은 칸 수:", 조인전부.fields.length, "/ 4 + 6 인가:",
  조인전부.fields.length === 4 + 6);
```

출력 이은 칸 수: 10 / 4 + 6 인가: true

```
console.log("설비 줄 수:", 설비만.rows.length, "/ 점검기록 줄 수:",
  점검기록만.rows.length);
```

출력 설비 줄 수: 5 / 점검기록 줄 수: 15

```
console.log("이은 줄 수:", 조인전부.rows.length);
```

출력 이은 줄 수: 15

★ 칸 수 = 왼쪽 칸 수 + 오른쪽 칸 수. 옆으로 붙였으니 당연합니다. 이어진 모습을 봅니다. 길어서 앞의 두 대만 찍습니다.

```
const 조인보기 = await db.query(`
  SELECT 설비.이름 AS 설비이름, 점검기록.점검번호, 점검기록.결과
  FROM 설비 JOIN 점검기록 ON 설비.설비번호 = 점검기록.설비번호
  WHERE 설비.설비번호 <= 2 ORDER BY 점검기록.점검번호
`);
for (const 줄 of 조인보기.rows) console.log(` · ${줄.설비이름} · 점검 ${줄.점검번호} ·
${줄.결과}`);
```

출력 · 컨베이어 1호 · 점검 1 · 정상
 · 컨베이어 1호 · 점검 2 · 정상
 · 컨베이어 1호 · 점검 3 · 주의
 · 컨베이어 1호 · 점검 4 · 정상
 · 프레스 1호 · 점검 5 · 주의
 · 프레스 1호 · 점검 6 · 불량
 · 프레스 1호 · 점검 7 · 정상

★ 컨베이어 1호가 네 번, 프레스 1호가 세 번 나왔습니다. 이게 섹션 5 의 씨앗입니다. ★ 그리고 이은 결과 15줄 전체에 설비는 4대뿐입니다. 포장기 1호는 점검기록이 없어서 **짜을 못 찾아 통째로 빠졌습니다**. INNER JOIN 은 양쪽에 다 있는 것만 남깁니다.

```
const 빠진설비 = await db.query("SELECT 설비번호, 이름 FROM 설비 WHERE 설비번호 NOT
  IN (SELECT 설비번호 FROM 점검기록);");
for (const 줄 of 빠진설비.rows) console.log(` · 빠진 설비: ${줄.설비번호} ${줄.
  이름}`);
```

출력 · 빠진 설비: 5 포장기 1호

★ 이렇게 빠진 줄까지 살리는 것이 LEFT JOIN 입니다. 개념02 에서 합니다. 아래는 INNER 확인.

```
const INNER붙임 = await db.query("SELECT COUNT(*) AS 줄수 FROM 설비 INNER JOIN
점검기록 ON 설비.설비번호 = 점검기록.설비번호");
console.log("INNER 를 붙여도 같은가:", INNER붙임.rows[0].줄수 ===
조인전부.rows.length);
```

```
출력      INNER 를 붙여도 같은가: true
```

★ INNER 는 생략할 수 있습니다. 안 쓰면 INNER 로 칩니다. 이 자료에서는 생략합니다.

별칭(alias)

섹션 3

표 뒤에 짧은 이름을 붙입니다. FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호 필요한 이유는 둘입니다. ① 표 이름이 길어서 ② ★ 같은 이름의 칸이 양쪽에 있을 때 — 설비에도 이름, 작업자에도 이름 이 있습니다

```
try {
  await db.query("SELECT 이름 FROM 설비 s JOIN 작업자 w ON s.라인코드 = w.소속라인");
  console.log("에러가 안 났습니다");
} catch (에러) {
  console.log("e.code:", 에러.code);
  console.log("e.message:", 에러.message);
}
```

```
출력      e.code: 42702
          e.message: column reference "이름" is ambiguous
```

★ 42702 = ambiguous_column. "어느 표의 이름인지 모르겠다" 는 뜻입니다. 별칭으로 고칩니다.

```
const 별칭사용 = await db.query(`
  SELECT s.이름 AS 설비이름, w.이름 AS 담당자이름, p.결과
  FROM 설비 s
  JOIN 점검기록 p ON s.설비번호 = p.설비번호
  JOIN 작업자 w ON p.담당사번 = w.사번
  ORDER BY p.점검번호 LIMIT 3
`);
console.log("결과 칸 이름:", 별칭사용.fields.map((칸) => 칸.name).join(", "));
```

```
출력      결과 칸 이름: 설비이름, 담당자이름, 결과
```

```
for (const 줄 of 별칭사용.rows) console.log(`· ${줄.설비이름} · ${줄.담당자이름} ·
${줄.결과}`);
```

```
출력      · 컨베이어 1호 · 김반장 · 정상
          · 컨베이어 1호 · 김반장 · 정상
          · 컨베이어 1호 · 이기사 · 주의
```

★ AS 설비이름 은 결과 칸의 이름, 설비 s 는 표의 별칭 입니다. 둘 다 AS 를 쓸 수 있어 헷갈립니다. 표 별칭에는 AS 를 안 쓰는 게 관례입니다. ★★ 별칭을 붙이면 원래 표 이름은 더 이상 못 씁니다. 가려집니다.

```
try {
  await db.query("SELECT 설비.이름 FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.
설비번호");
  console.log("에러가 안 났습니다");
} catch (에러) {
  console.log("e.code:", 에러.code);
  console.log("e.message:", 에러.message);
}
```

```
출력      e.code: 42P01
          e.message: invalid reference to FROM-clause entry for table "설비"
```

★ 42P01 은 “없는 표” 에러입니다. 설비 표는 분명히 있는데도 이게 납니다. 별칭 s 를 붙이는 순간 그 표의 이름은 s 가 되기 때문입니다. **끝까지 별칭으로** 쓰세요.

표 세 개 이상 잇기

섹션 4

JOIN 은 왼쪽부터 차례로 붙입니다. 앞에서 이은 결과가 다음 JOIN 의 왼쪽이 됩니다. 라인 —JOIN→ (라인 +설비) —JOIN→ (+점검기록) —JOIN→ (+작업자)

```
const 네표 = await db.query(`
  SELECT *
  FROM 라인 l
  JOIN 설비 s    ON l.라인코드 = s.라인코드
  JOIN 점검기록 p ON s.설비번호 = p.설비번호
  JOIN 작업자 w  ON p.담당사번 = w.사번
`);
console.log("네 표를 이은 칸 수:", 네표.fields.length, "/ 3+4+6+3 인가:",
네표.fields.length === 3 + 4 + 6 + 3);
```

```
출력      네 표를 이은 칸 수: 16 / 3+4+6+3 인가: true
```

```
console.log("네 표를 이은 줄 수:", 네표.rows.length);
```

```
출력      네 표를 이은 줄 수: 13
```

★ 점검기록은 15줄인데 13줄이 나왔습니다. 두 줄이 어디로 갔을까요.

```
const 담당없음 = await db.query("SELECT COUNT(*) AS 건수 FROM 점검기록 WHERE 담당사번
IS NULL");
console.log("담당사번이 NULL 인 점검기록:", 담당없음.rows[0].건수);
```

```
출력      담당사번이 NULL 인 점검기록: 2
```

★★ 담당자가 안 정해진 2건이 작업자와 짝을 못 지어 사라졌습니다. NULL 은 무엇과도 같지 않습니다. NULL = 105 는 참도 거짓도 아니라 ON 조건이 참이 안 됩니다. ★ 표를 하나 더 이을 때마다 줄이 **조용히** 줄어들 수 있습니다.

```
const 네표보기 = await db.query(`
  SELECT l.이름 AS 라인이름, s.이름 AS 설비이름, w.이름 AS 담당자, p.결과
  FROM 라인 l
  JOIN 설비 s      ON l.라인코드 = s.라인코드
  JOIN 점검기록 p ON s.설비번호 = p.설비번호
  JOIN 작업자 w    ON p.담당사번 = w.사번
  ORDER BY p.점검번호 LIMIT 3
`);
for (const 줄 of 네표보기.rows) console.log(`· ${줄.라인이름} · ${줄.설비이름} ·
${줄.담당자} · ${줄.결과}`);
```

출력

- 조립1라인 · 컨베이어 1호 · 김반장 · 정상
- 조립1라인 · 컨베이어 1호 · 김반장 · 정상
- 조립1라인 · 컨베이어 1호 · 이기사 · 주의

★★ 1:N 을 이으면 줄 수가 늘어납니다

섹션 5

이 파일에서 제일 중요한 부분입니다. 설비 한 대에 점검기록이 여러 건 붙습니다(1:N). 이걸 이으면 **설비가** 점검기록 건수만큼 반복해서 나옵니다.

```
const 반복확인 = await db.query(`
  SELECT s.이름, COUNT(*) AS 나온횟수
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.설비번호, s.이름 ORDER BY s.설비번호
`);
for (const 줄 of 반복확인.rows) console.log(`· ${줄.이름} · 결과에 ${줄.나온횟수}번
나옴`);
```

출력

- 컨베이어 1호 · 결과에 4번 나옴
- 프레스 1호 · 결과에 3번 나옴
- 용접로봇 1호 · 결과에 5번 나옴
- 검사기 1호 · 결과에 3번 나옴

★★ 여기서 사고가 납니다. 모르고 세면 틀립니다.

```
const 설비수 = await db.query("SELECT COUNT(*) AS 대수 FROM 설비");
const 이은뒤수 = await db.query("SELECT COUNT(*) AS 대수 FROM 설비 s JOIN 점검기록 p
ON s.설비번호 = p.설비번호");
const 중복뻥수 = await db.query("SELECT COUNT(DISTINCT s.설비번호) AS 대수 FROM 설비
s JOIN 점검기록 p ON s.설비번호 = p.설비번호");
console.log("설비 표만 세면:", 설비수.rows[0].대수);
```

출력 설비 표만 세면: 5

```
console.log("이은 뒤에 세면:", 이은뒤수.rows[0].대수);
```

출력 이은 뒤에 세면: 15

```
console.log("DISTINCT 로 세면:", 중복뺀수.rows[0].대수);
```

출력 DISTINCT 로 세면: 4

★★★ 설비는 15대가 아닙니다. 5대입니다. `COUNT(*)` 는 **결과의 줄 수** 를 셉니다. ★ 5도 맞고 15도 맞고 4도 맞습니다. 셋 다 **다른 질문의 답** 입니다.

"설비가 몇 대인가" 5 / "점검을 몇 번 했나" 15 / "점검받은 설비가 몇 대인가" 4.
쿼리를 쓰기 전에 ****무엇을 세는지**** 부터 정하세요.

표를 하나 더 끼우면 더 크게 틀립니다. 라인별 설비 대수를 세 봅니다.

```
const 라인별진짜 = await db.query("SELECT 라인코드, COUNT(*) AS 대수 FROM 설비 GROUP BY 라인코드 ORDER BY 라인코드");
for (const 줄 of 라인별진짜.rows) console.log(`· 라인 ${줄.라인코드} · 설비 ${줄.대수}대`);
```

출력 · 라인 A · 설비 2대
 · 라인 B · 설비 2대
 · 라인 C · 설비 1대

```
const 라인별틀림 = await db.query(`
  SELECT l.라인코드, COUNT(*) AS 대수
  FROM 라인 l JOIN 설비 s ON l.라인코드 = s.라인코드
           JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY l.라인코드 ORDER BY l.라인코드
`);
for (const 줄 of 라인별틀림.rows) console.log(`· 라인 ${줄.라인코드} · 설비 ${줄.대수}대(?)`);
```

출력 · 라인 A · 설비 7대(?)
 · 라인 B · 설비 8대(?)

★★★ 라인 A 에 설비가 7대라고 나왔습니다. 실제로는 2대입니다. 점검기록을 이어 붙였더니 설비가 점검 건수만큼 부풀었습니다. 라인 C 와 D 는 아예 사라졌습니다. 보고서에 이 숫자가 올라가면 아무도 눈치 못 칩니다. 그럴듯하니까요.

```
const 라인별고침 = await db.query(`
  SELECT l.라인코드, COUNT(DISTINCT s.설비번호) AS 대수
  FROM 라인 l JOIN 설비 s ON l.라인코드 = s.라인코드
        JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY l.라인코드 ORDER BY l.라인코드
`);
for (const 줄 of 라인별고침.rows) console.log(` 라인 ${줄.라인코드} · 설비 ${줄.대수}대`);
```

```
출력      · 라인 A · 설비 2대
          · 라인 B · 설비 2대
```

★ 부풀린 것은 DISTINCT 로 고쳤습니다. 그래도 라인 C·D 는 여전히 없습니다. 그건 DISTINCT 로 못 고칩니다. LEFT JOIN 이 필요합니다. 개념02·03 에서 합니다. ★ SUM 도 똑같이 틀립니다. 이걸 더 눈치채기 어렵습니다.

```
const 년도합진짜 = await db.query("SELECT SUM(도입년도) AS 합 FROM 설비");
const 년도합틀림 = await db.query("SELECT SUM(s.도입년도) AS 합 FROM 설비 s JOIN
점검기록 p ON s.설비번호 = p.설비번호");
console.log("설비 표만 SUM:", 년도합진짜.rows[0].합, "/ 이은 뒤 SUM:",
년도합틀림.rows[0].합);
```

```
출력      설비 표만 SUM: 10095 / 이은 뒤 SUM: 30276
```

★★ 설비 1이 4번 나오니 2015 를 네 번 더했습니다. 금액이면 매출이 뺄뵈기됩니다. ★ 1:N 을 이은 결과에서 1쪽 값을 집계하면 거의 항상 틀립니다.

제대로 세는 법 (GROUP BY + LEFT JOIN)은 개념03 에서 합니다.

★ 세는 값의 타입도 한 번 짚고 갑니다. 여기서 학생들이 잘 걸립니다.

```
const 큰수 = await db.query("SELECT 9007199254740993::BIGINT AS 큰값");
console.log("COUNT 결과의 타입:", typeof 설비수.rows[0].대수);
```

```
출력      COUNT 결과의 타입: number
```

```
console.log("큰 BIGINT 의 타입:", typeof 큰수.rows[0].큰값, String(큰수.rows[0].큰값));
```

```
출력      큰 BIGINT 의 타입: bigint 9007199254740993
```

★★ COUNT(*) 의 SQL 타입은 BIGINT 입니다. PGlite 는 안전하게 담기는 크기면 number 로, 넘어가면 **bigint** 로 줍니다. bigint 는 JSON.stringify 가 터지고 number 와 + 로 못 더합니다. ★ 세는 값은 습관적으로 Number(...) 로 감싸세요.

USING 과 NATURAL JOIN

섹션 6

양쪽 칸 이름이 똑같으면 USING 으로 짧게 씁니다. ON 설비.설비번호 = 점검기록.설비번호 → USING (설비번호)

```
const ON버전 = await db.query("SELECT * FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호");
const USING버전 = await db.query("SELECT * FROM 설비 JOIN 점검기록 USING (설비번호)");
console.log("ON 칸 수:", ON버전.fields.length, "/ USING 칸 수:", USING버전.fields.length);
```

출력 ON 칸 수: 10 / USING 칸 수: 9

```
console.log("줄 수는 같은가:", ON버전.rows.length === USING버전.rows.length);
```

출력 줄 수는 같은가: true

```
console.log("ON 칸 이름:", ON버전.fields.map((칸) => 칸.name).join(","));
```

출력 ON 칸 이름: 설비번호,이름,라인코드,도입년도,점검번호,설비번호,점검일,결과,점수,담당사번

```
console.log("USING 칸 이름:", USING버전.fields.map((칸) => 칸.name).join(","));
```

출력 USING 칸 이름: 설비번호,이름,라인코드,도입년도,점검번호,점검일,결과,점수,담당사번

★ ON 은 설비번호가 두 번, USING 은 한 번만 나옵니다. 그래서 한 칸 적습니다. ★★ 여기 자바스크립트 쪽 함정이 하나 붙어 있습니다.

```
console.log("ON 버전 fields:", ON버전.fields.length, "/ rows[0] 키:", Object.keys(ON버전.rows[0]).length);
```

출력 ON 버전 fields: 10 / rows[0] 키: 9

★★ SQL 은 10칸을 줬는데 자바스크립트 객체에는 9개뿐입니다. 키는 중복될 수 없어서 뒤에 온 설비번호가 앞것을 덮어썼습니다. ★ 그래서 JOIN 에서 SELECT * 를 쓰면 안 됩니다. 필요한 칸만 AS 로 골라 쓰세요. NATURAL JOIN 은 이름이 같은 칸을 알아서 다 찾아 짝지어 줍니다.

```
const 자연조인1 = await db.query("SELECT * FROM 설비 NATURAL JOIN 점검기록");
console.log("NATURAL 줄 수:", 자연조인1.rows.length, "/ 칸 수:", 자연조인1.fields.length);
```

출력 NATURAL 줄 수: 15 / 칸 수: 9

잘 됩니다. USING (설비번호) 와 똑같이 ON 도 안 썼습니다. 편해 보입니다. ★★★ 그런데 쓰지 마세요. 반년 뒤 누가 점검기록에 라인코드를 복사해 둡니다. 조회할 때마다 설비를 잇기 귀찮아서요. 현실에서 아주 흔합니다.

```
await db.exec("ALTER TABLE 점검기록 ADD COLUMN 라인코드 TEXT;");
const 자연조인2 = await db.query("SELECT * FROM 설비 NATURAL JOIN 점검기록");
const ON조인2 = await db.query("SELECT * FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호");
console.log("칸 하나 늘린 뒤 NATURAL 줄 수:", 자연조인2.rows.length);
```

출력 칸 하나 늘린 뒤 NATURAL 줄 수: 0

```
console.log("같은 상황에서 ON 줄 수:", ON조인2.rows.length);
```

출력 같은 상황에서 ON 줄 수: 15

★★★ 15줄이 0줄이 됐습니다. **애러도 안 납니다.** NATURAL JOIN 이 라인코드까지 조건에 넣었고, 새 칸은 전부 NULL 이라 다 탈락합니다. 칸 하나 늘렸을 뿐인데 목록이 사라집니다.

```
await db.exec("UPDATE 점검기록 SET 라인코드 = 'A' WHERE 설비번호 IN (1, 2);");
const 자연조인3 = await db.query("SELECT * FROM 설비 NATURAL JOIN 점검기록");
console.log("설비 1·2 에 라인코드를 채운 뒤 NATURAL 줄 수:", 자연조인3.rows.length);
```

출력 설비 1·2 에 라인코드를 채운 뒤 NATURAL 줄 수: 7

복사해 둔 값은 이렇게 들어옵니다. 한 건만 잘못 넣어 봅니다.

```
await db.exec("UPDATE 점검기록 SET 라인코드 = 'Z' WHERE 설비번호 = 1;");
const 자연조인4 = await db.query("SELECT * FROM 설비 NATURAL JOIN 점검기록");
console.log("한 설비를 틀리게 바꾼 뒤 NATURAL 줄 수:", 자연조인4.rows.length);
```

출력 한 설비를 틀리게 바꾼 뒤 NATURAL 줄 수: 3

★★★ 15 → 0 → 7 → 3. 쿼리는 한 글자도 안 바꿨습니다. NATURAL JOIN 은 **표가 바뀌면 조용히 뜻이 바뀝니다.** 조건이 쿼리에 없으니 코드를 읽어도 안 보입니다. ★ USING 은 괜찮습니다. 칸이 늘어도 USING (설비번호) 는 설비번호만 봅니다.

```
await db.exec("ALTER TABLE 점검기록 DROP COLUMN 라인코드;");
const 되돌림확인 = await db.query("SELECT * FROM 설비 NATURAL JOIN 점검기록");
console.log("칸을 지워 원래대로 돌린 뒤 NATURAL 줄 수:", 되돌림확인.rows.length);
```

출력 칸을 지워 원래대로 돌린 뒤 NATURAL 줄 수: 15

옛날 문법과 카테시안 곱

섹션 7

오래된 코드는 JOIN 이라는 낱말 없이 이렇게 씁니다. FROM 설비, 점검기록 WHERE 설비.설비번호 = 점검기록.설비번호

```
const 옛날문법 = await db.query("SELECT s.이름, p.점검번호 FROM 설비 s, 점검기록 p
WHERE s.설비번호 = p.설비번호 ORDER BY p.점검번호");
const 요즘문법 = await db.query("SELECT s.이름, p.점검번호 FROM 설비 s JOIN 점검기록
p ON s.설비번호 = p.설비번호 ORDER BY p.점검번호");
console.log("옛날 문법 줄 수:", 옛날문법.rows.length);
```

출력 옛날 문법 줄 수: 15

```
console.log("결과가 똑같은가:", JSON.stringify(옛날문법.rows) ===
JSON.stringify(요즘문법.rows));
```

출력 결과가 똑같은가: true

★★ 그런데 이 문법에는 함정이 있습니다. 조건을 빠뜨려도 에러가 안 납니다.

```
const 조건빠뜨림 = await db.query("SELECT * FROM 설비, 점검기록");
console.log("조건 없이 이은 줄 수:", 조건빠뜨림.rows.length, "/ 5 × 15 인가:",
조건빠뜨림.rows.length === 5 * 15);
```

출력 조건 없이 이은 줄 수: 75 / 5 × 15 인가: true

★★★ 모든 짝을 다 만든 것입니다. 이걸 **카테시안 곱(cartesian product)** 이라고 합니다. 섹션 2 에서 “75가지 짝을 만들어 보고 조건에 맞는 것만 남긴다” 고 했는데, 조건을 안 주면 **거르는 단계 없이 75가지가 전부 나옵니다.** 설비 1만 대 × 점검기록 10만 건이면 **10억 줄** 입니다. 서버 메모리가 날아갑니다. 일부러 작게 만들어 시간을 재 봅니다. 300 × 300 입니다.

```
await db.exec(`
  CREATE TABLE 큰왼쪽 AS SELECT g AS 번호, 'L' || g AS 이름 FROM
  generate_series(1, 300) g;
  CREATE TABLE 큰오른쪽 AS SELECT g AS 번호, 'R' || g AS 이름 FROM
  generate_series(1, 300) g;
`);
const 곱줄수 = await db.query("SELECT COUNT(*) AS 줄수 FROM 큰왼쪽, 큰오른쪽");
const 조건줄수 = await db.query("SELECT COUNT(*) AS 줄수 FROM 큰왼쪽 가, 큰오른쪽 나
WHERE 가.번호 = 나.번호");
console.log("조건 없이:", 곱줄수.rows[0].줄수, "줄 / 조건 있게:", 조건줄수.rows[0].
줄수, "줄");
```

출력 조건 없이: 90000 줄 / 조건 있게: 300 줄

```
const 곱ms = await 재기(5, () => db.query("SELECT COUNT(*) AS 줄수 FROM 큰왼쪽,
큰오른쪽"));
const 조건ms = await 재기(5, () => db.query("SELECT COUNT(*) AS 줄수 FROM 큰왼쪽 가,
큰오른쪽 나 WHERE 가.번호 = 나.번호"));
console.log(`조건 없이: ${곱ms.toFixed(1)} ms / 조건 있게: ${조건ms.toFixed(1)} ms`);
```

출력 조건 없이: 8.8 ms / 조건 있게: 0.5 ms
값이 다를 수 있음

```
console.log("조건을 빠뜨린 쪽이 더 느린가:", 곱ms > 조건ms);
```

출력 조건을 빠뜨린 쪽이 더 느린가: true

★ 표가 300줄짜리 둘뿐인데도 벌써 차이가 납니다. 왜 그런지는 06단원에서 합니다. ★★ 그래서 JOIN ... ON 을 씁니다. ON 을 빠뜨리면 **문법 에러로 바로 잡힙니다.**

```
try {
  await db.query("SELECT * FROM 설비 JOIN 점검기록");
  console.log("에러가 안 났습니다");
} catch (에러) {
  console.log("e.code:", 에러.code);
  console.log("e.message:", 에러.message);
}
```

출력 e.code: 42601
e.message: syntax error at end of input

★ 42601 = 문법 에러. 사람이 실수했을 때 **기계가 대신 잡아 주는 것** 이 좋은 문법입니다. 심표 문법은 안 잡아 줍니다. 정말로 모든 짝이 필요할 때는 뜻을 분명히 적습니다.

```
const 일부러곱 = await db.query("SELECT COUNT(*) AS 줄수 FROM 설비 CROSS JOIN
점검기록");
console.log("CROSS JOIN 줄 수:", 일부러곱.rows[0].줄수);
```

출력 CROSS JOIN 줄 수: 75

★ CROSS JOIN 은 “일부러 곱한 것” 이라고 코드에 적어 두는 것입니다. 달력 날짜 × 설비 목록처럼 빈칸까지 다 만들어야 할 때 씁니다.

MySQL 은 여기가 다릅니다

· MySQL 에서는 JOIN / INNER JOIN / CROSS JOIN 이 서로 바꿔 쓸 수 있는 낱말입니다. 그래서 FROM 설비 JOIN 점검기록 처럼 ON 을 빼도 에러가 안 나고 카테시안 곱이 됩니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

정리 — 잇는 문법과 이 파일에서 실측한 값

문법 칸 수 조건을 빠뜨리면 쓸 것인가

JOIN ... ON 10 문법 예러 42601 ★ 이걸 쓰세요 JOIN ... USING (설비번호) 9 칸 이름 틀리면 예러 ★ 이름 같으면 좋음 NATURAL JOIN 9 조용히 결과가 바뀜 ✕ 쓰지 마세요 FROM 설비, 점검기록 10 75줄 곱, 예러 없음 ✕ 옛날 문법 CROSS JOIN 10 원래 곱하는 문법 ○ 일부러 곱할 때만

실측값 값 왜

설비 JOIN 점검기록 15줄 설비 5는 짝이 없어 빠짐 (칸은 4+6=10) 네 표를 이으면 13줄 담당사번 NULL 2건이 더 빠짐 (칸은 16) 이은 뒤 COUNT(*) / COUNT(DISTINCT) 15 / 4 15는 설비 대수가 아닙니다 조건 없이 실패로 이으면 75줄 5 × 15

★★★ 딱 하나만 가져간다면 이것입니다. 1:N 을 이으면 1쪽이 반복됩니다. 그 상태로 세거나 더하면 틀립니다.

직접 해 볼 것

▫ 직접 해보기 1

섹션 2 의 JOIN 을 뒤집어 보세요. FROM 점검기록 JOIN 설비 ON ... 줄 수가 달라지나요? 칸 순서는요? (힌트: INNER 는 좌우를 바꿔도 같습니다)

▫ 직접 해보기 2

설비와 작업자를 ON s.라인코드 = w.소속라인 으로 이어 보세요. 몇 줄일까요? 손으로 먼저 세고 돌려서 맞춰 보세요. (힌트: 라인 A 는 설비 2대 × 작업자 1명)

▫ 직접 해보기 3

섹션 5 의 SUM(s.도입년도) 를 SUM(p.점수) 로 바꿔 보세요. 점수는 이어도 안 틀립니다. (힌트: 점수는 N쪽, 도입년도는 1쪽 칸)

▫ 직접 해보기 4

라인·설비·점검기록을 잇고 1등 라인만 골라 보세요. 조건을 WHERE 에 쓸 때와 ON 에 쓸 때 결과가 같나요? (LEFT JOIN 은 다릅니다 — 개념02)

▫ 직접 해보기 5

설비 NATURAL JOIN 작업자 는 몇 줄일까요? 예상하고 돌려 보세요. ★ 두 표에 이름 칸이 둘 다 있습니다. 그것도 짝짓기 조건이 됩니다.

▹ 직접 해보기 **6**

섹션 7 의 300 을 1000 으로 바꿔 보세요. 줄 수는 100만이 됩니다. 시간은 몇 배가 되나요? ★ 3000 이상은 하지 마세요. 노트북이 멈춥니다.

▹ 직접 해보기 **7**

섹션 1 의 N+1 코드에서 점검기록을 `generate_series` 로 1000건 만들어 돌려 보세요. JOIN 과 몇 배 차이인가요?

자주 하는 실수

이런 실수를 합니다

이은 결과에서 COUNT(*) 로 개수를 셈

설비 JOIN 점검기록 의 COUNT(*) 는 15입니다. 설비는 5대인데요. 1:N 을 이으면 1쪽이 반복됩니다. COUNT(DISTINCT 설비번호) 를 쓰세요. SUM·AVG 도 똑같이 부릅니다.

이런 실수를 합니다

표를 하나 더 이었더니 줄이 줄어든 걸 못 알아챈

15줄이 13줄이 됐습니다. 담당사번이 NULL 인 2건이 빠진 것입니다. INNER JOIN 은 짝 없는 줄을 **말없이** 버립니다. 이를 때마다 줄 수를 확인하세요.

이런 실수를 합니다

JOIN 에서 SELECT * 를 씀

설비번호가 두 번 나오는데 객체 키는 하나뿐이라 뒤엎것이 덮어씹니다. fields 는 10개인데 rows[0] 키는 9개였습니다. 필요한 칸만 AS 로 골라 쓰세요.

이런 실수를 합니다

NATURAL JOIN 을 편하다고 씀

지금은 잘 돛니다. 반년 뒤 누가 칸 하나를 늘리면 조용히 0줄이 됩니다. 에러도 로그도 안 남습니다. USING 이나 ON 을 쓰세요.

이런 실수를 합니다

임표 문법으로 잇다가 조건을 빠뜨림

FROM 설비, 점검기록 은 75줄을 그냥 돌려줍니다. 에러가 안 납니다. JOIN ... ON 이었으면 42601 로 잡혔습니다. 표가 세 개면 조건도 두 개라 더 자주 빠뜨립니다.

이런 실수를 합니다

별칭을 붙여 놓고 원래 표 이름을 씀

FROM 설비 s 라고 해 놓고 설비.이름 이라고 쓰면 42P01 이 납니다. “표가 있는데 왜 없다고 하지?” 하고 헤맬니다. 끝까지 별칭으로 쓰세요.

이런 실수를 합니다

프로그램에서 반복문으로 이어 붙임

점검기록 줄마다 설비를 조회하면 쿼리를 1 + N 번 던집니다. 여기서는 16번, 5만 건이면 5만 1번입니다. 잇는 일은 DB 에게 시키세요. 그러라고 있는 기능입니다.

```
await db.close();
```

없는 쪽도 보기

```
RUN node 개념02_없는_쪽도_보기.js
```

개념01 에서 INNER JOIN 을 했습니다. 그런데 INNER JOIN 은 **짜이 있는 것만** 남깁니다. 설비는 5대인데 점검기록과 이어 보면 4대만 나옵니다. 포장기 1호(설비5)는 점검기록이 하나도 없어서 **결과에서 조용히 사라집니다**. 하필 그게 관리자가 제일 보고 싶은 것입니다. "점검을 한 번도 안 한 설비" 야말로 당장 손봐야 할 설비니까요.

이 파일에서는 **짜이 없는 쪽도 살려 두는 방법**을 봅니다. 그리고 그 방법을 한 줄로 망가뜨리는 실수를 아주 자세히 봅니다. 실무에서 제일 자주 터지는 조인 사고입니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

예제 데이터

```

await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    동        TEXT NOT NULL
  );
  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드),
    도입년도 INT NOT NULL
  );
  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드)
  );
  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호),
    점검일   DATE NOT NULL,
    결과      TEXT NOT NULL,
    점수      INT,
    담당사번 INT REFERENCES 작업자(사번)
  );
  INSERT INTO 라인 VALUES
    ('A', '조립1라인', '1동'),
    ('B', '가공2라인', '1동'),
    ('C', '포장3라인', '2동'),
    ('D', '신설4라인', '2동');
  INSERT INTO 설비 VALUES
    (1, '컨베이어 1호', 'A', 2015),
    (2, '프레스 1호', 'A', 2019),
    (3, '용접로봇 1호', 'B', 2021),
    (4, '검사기 1호', 'B', 2018),
    (5, '포장기 1호', 'C', 2022);

```

```

INSERT INTO 작업자 VALUES
  (101, '김반장', 'A'),
  (102, '이기사', 'B'),
  (103, '박주임', 'B'),
  (104, '최사원', 'C'),
  (105, '정신입', NULL);
INSERT INTO 점검기록 VALUES
  ( 1, 1, '2024-01-05', '정상', 92, 101),
  ( 2, 1, '2024-02-05', '정상', 88, 101),
  ( 3, 1, '2024-03-06', '주의', 71, 102),
  ( 4, 1, '2024-04-05', '정상', 90, 101),
  ( 5, 2, '2024-01-12', '주의', 65, 102),
  ( 6, 2, '2024-02-14', '불량', 48, 103),
  ( 7, 2, '2024-03-15', '정상', 88, 101),
  ( 8, 3, '2024-01-20', '정상', 95, 103),
  ( 9, 3, '2024-02-20', '정상', NULL, 103),
  (10, 3, '2024-03-21', '주의', 74, 102),
  (11, 3, '2024-04-22', '정상', 90, NULL),
  (12, 3, '2024-05-23', '불량', 52, 103),
  (13, 4, '2024-01-28', '정상', NULL, 104),
  (14, 4, '2024-03-29', '정상', 88, 104),
  (15, 4, '2024-05-30', '주의', 69, NULL);
`);

```

NULL 을 그대로 찍으면 "null" 이라 눈에 잘 안 들어옵니다. 보기 좋게 바꿉니다.

```
const 없으면 = (값) => (값 === null ? "(없음)" : 값);
```

INNER JOIN 이 빠뜨리는 것

섹션 1

★ 먼저 짚고 갈 것이 있습니다. count(*) 는 Postgres 에서 **BIGINT** 입니다. BIGINT 는 자바스크립트 number 에 다 안 들어갑니다. 그래서 타입이 왔다 갔다 합니다.

```
const 개수확인 = await db.query(`SELECT count(*) AS 설비수, 9007199254740993::BIGINT
AS 아주큰수 FROM 설비`);
```

```
console.log("설비 수:", 개수확인.rows[0].설비수, " · 타입:", typeof 개수확인.rows[0].
설비수);
```

출력 설비 수: 5 · 타입: number

```
console.log("아주 큰 수:", 개수확인.rows[0].아주큰수, " · 타입:", typeof
개수확인.rows[0].아주큰수);
```

출력 아주 큰 수: 9007199254740993n · 타입: bigint


```
try {
  JSON.stringify(개수확인.rows[0]);
} catch (에러) {
  console.log("bigint 를 JSON.stringify 하면:", 에러.message);
}
```

출력 bigint 를 JSON.stringify 하면: Do not know how to serialize a BigInt

★★ 같은 BIGINT 인데 하나는 number, 하나는 bigint 로 왔습니다. PGlite 는 number 로 정확히 담기는 값이면 number 로, 넘어가면 bigint 로 줍니다. **데이터가 커지면 타입이 바뀝니다.** 개발할 때는 number 였다가 운영에서 bigint 가 오고, JSON 응답을 만들다가 위치를 터집니다. ★ 그래서 세는 값에는 **항상 Number(...)** 를 씁니다.

```
const 안쪽 = await db.query(`
  SELECT s.설비번호, s.이름, p.점검일, p.결과
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  ORDER BY s.설비번호, p.점검번호`);
console.log("INNER JOIN 결과 줄 수:", 안쪽.rows.length);
```

출력 INNER JOIN 결과 줄 수: 15

```
const 나온설비 = await db.query(`
  SELECT DISTINCT s.설비번호, s.이름
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  ORDER BY s.설비번호`);
console.log("INNER JOIN 에 나온 설비:", 나온설비.rows.length, "대");
```

출력 INNER JOIN 에 나온 설비: 4 대

```
for (const 줄 of 나온설비.rows) console.log(`· ${줄.설비번호} · ${줄.이름}`);
```

출력 · 1 · 컨베이어 1호
 · 2 · 프레스 1호
 · 3 · 용접로봇 1호
 · 4 · 검사기 1호

```
console.log("포장기 1호가 결과에 있는가:", 나온설비.rows.some((줄) => 줄.이름 === "포장기 1호"));
```

출력 포장기 1호가 결과에 있는가: false

★★ 설비는 5대인데 4대만 나왔습니다. INNER JOIN 은 **양쪽 다 짝이 있는 줄만** 남깁니다. 포장기 1호는 점검기록이 0건이라 짝이 없고, 그래서 사라졌습니다.

이게 왜 사고인가: 이 결과를 “전체 설비 점검 현황” 으로 보고하면 4대가 전부 점검받은 것으로 보입니다. **점검을 한 번도 안 받은 설비만 골라서 안 보입니다.** 에러도 경고도 없습니다. 개념01 에서 본 조용한 버그와

같은 종류입니다.

★ 조인 결과의 줄 수가 예상보다 적으면 **먼저 짝 없는 줄을 의심하세요.**

LEFT JOIN — 왼쪽은 무조건 살립니다

섹션 2

```
const 왼쪽 = await db.query(`
  SELECT s.설비번호, s.이름, p.점검번호, p.결과
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  ORDER BY s.설비번호, p.점검번호`);
console.log("LEFT JOIN 결과 줄 수:", 왼쪽.rows.length);
```

출력 LEFT JOIN 결과 줄 수: 16

15건(점검기록 전부) + 1줄(짝 없는 포장기 1호) = 16줄입니다. 그 1줄을 그대로 봅니다.

```
console.log("마지막 줄:", JSON.stringify(왼쪽.rows[왼쪽.rows.length - 1]));
```

출력 마지막 줄: {"설비번호":5,"이름":"포장기 1호","점검번호":null,"결과":null}

★ 짝을 못 찾으면 **오른쪽 표의 칸이 전부 NULL로 채워진 가짜 줄**이 하나 생깁니다. 줄을 버리는 게 아니라, 빈 줄을 붙여서 왼쪽을 살려 두는 것입니다. ★ LEFT JOIN은 LEFT OUTER JOIN의 줄임말입니다. OUTER는 생략해도 됩니다. ★ 왼쪽/오른쪽이 어느 쪽인가: **FROM에 먼저 쓴 표가 왼쪽**입니다. 위 질의에서는 설비가 왼쪽, 점검기록이 오른쪽입니다. 그래서 “설비는 다 나오고 점검기록은 있으면 붙는다”가 됩니다.

```
const 세보기 = await db.query(`
  SELECT s.설비번호, s.이름, count(*) AS 전체줄수, count(p.점검번호) AS 진짜점검수
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.설비번호, s.이름
  ORDER BY s.설비번호`);
for (const 줄 of 세보기.rows) {
  console.log(`· ${줄.이름} · count(*)=${줄.전체줄수} · count(점검번호)=${줄.진짜점검수}`);
}
```

출력 · 컨베이어 1호 · count(*)=4 · count(점검번호)=4
· 프레스 1호 · count(*)=3 · count(점검번호)=3
· 용접로봇 1호 · count(*)=5 · count(점검번호)=5
· 검사기 1호 · count(*)=3 · count(점검번호)=3
· 포장기 1호 · count(*)=1 · count(점검번호)=0

```
console.log("포장기 1호를 count(*)로 세면 0인가:", Number(세보기.rows[4].전체줄수) === 0);
```

출력 포장기 1호를 count(*)로 세면 0인가: false

★★ 여기가 함정입니다. 포장기 1호는 점검이 0건인데 count(*) 는 1 이라고 합니다. NULL 로 채워진 가짜 줄도 "줄" 이기 때문입니다. 0으로 세려면 **오른쪽 표의 칸을 지정해서** 세야 합니다. count(p.점검번호) 는 0 입니다. count(칸) 은 그 칸이 NULL 인 줄을 안 셉니다. ★ 집계는 이 단원 개념03 에서 합니다.

RIGHT JOIN 과 FULL OUTER JOIN

섹션 3

RIGHT JOIN 은 LEFT JOIN 의 좌우를 뒤집은 것입니다. 줄 수가 같은지 봅니다.

```
const 오른쪽 = await db.query(`
  SELECT s.설비번호, s.이름, p.점검번호
  FROM 점검기록 p RIGHT JOIN 설비 s ON s.설비번호 = p.설비번호
  ORDER BY s.설비번호, p.점검번호`);
console.log("RIGHT JOIN 결과 줄 수:", 오른쪽.rows.length);
```

출력 RIGHT JOIN 결과 줄 수: 16

```
console.log("LEFT 로 만든 것과 줄 수가 같은가:", 오른쪽.rows.length ===
  왼쪽.rows.length);
```

출력 LEFT 로 만든 것과 줄 수가 같은가: true

★ 실무에서는 거의 LEFT 만 씁니다. RIGHT 를 보면 FROM 에 쓴 표가 기준이라는 감각이 깨져서 읽는 사람이 한 번 멈춥니다. 표 순서를 바꿔 LEFT 로 쓸 수 있으면 그렇게 쓰세요.

FULL OUTER JOIN 은 양쪽 다 짝 없는 것까지 살립니다. 그런데 지금은 짝 없는 쪽이 라인 D(설비 없음) 하나뿐이라, 라인이 없는 설비를 하나 넣어 봅니다.

```
await db.exec(`INSERT INTO 설비 VALUES (6, '이동대차 1호', NULL, 2023);`);

const 라인왼쪽 = await db.query(`SELECT count(*) AS 개수 FROM 라인 l LEFT JOIN 설비
  s ON l.라인코드 = s.라인코드`);
const 라인오른쪽 = await db.query(`SELECT count(*) AS 개수 FROM 라인 l RIGHT JOIN
  설비 s ON l.라인코드 = s.라인코드`);

console.log("라인 LEFT JOIN 설비:", Number(라인왼쪽.rows[0].개수), "줄");
```

출력 라인 LEFT JOIN 설비: 6 줄

```
console.log("라인 RIGHT JOIN 설비:", Number(라인오른쪽.rows[0].개수), "줄");
```

출력 라인 RIGHT JOIN 설비: 6 줄

```
const 양쪽 = await db.query(`
  SELECT l.라인코드, l.이름 AS 라인이름, s.이름 AS 설비이름
  FROM 라인 l FULL OUTER JOIN 설비 s ON l.라인코드 = s.라인코드
  ORDER BY l.라인코드, s.설비번호`);
console.log("라인 FULL OUTER JOIN 설비:", 양쪽.rows.length, "줄");
```

출력 라인 FULL OUTER JOIN 설비: 7 줄

```
for (const 줄 of 양쪽.rows) {
  console.log(`· ${없으면(줄.라인코드)} · ${없으면(줄.라인이름)} · ${없으면(줄.
설비이름)}`);
}
```

출력 · A · 조립1라인 · 컨베이어 1호
· A · 조립1라인 · 프레스 1호
· B · 가공2라인 · 용접로봇 1호
· B · 가공2라인 · 검사기 1호
· C · 포장3라인 · 포장기 1호
· D · 신설4라인 · (없음)
· (없음) · (없음) · 이동대차 1호

★ 마지막 두 줄을 보세요. 라인 D 는 설비 쪽이 (없음) — 왼쪽만 있는 줄이고, 이동대차 1호는 라인 쪽이 (없음) — 오른쪽만 있는 줄입니다. FULL 은 둘 다 살립니다. ★ FULL OUTER JOIN 은 자주 안 씁니다. 보통 기준 표가 정해져 있어서 LEFT 로 충분합니다. 두 시스템에서 받은 목록을 맞춰 볼 때(대사 작업) 정도가 쓸모 있는 자리입니다.

MySQL 은 여기가 다릅니다

- MySQL 8.0 에는 FULL OUTER JOIN 이 없습니다. LEFT 와 RIGHT 를 UNION 으로 이어 흉내 냅니다
- ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

실험이 끝났으니 되돌립니다. 안 그러면 뒤 섹션 결과가 흔들립니다.

```
await db.exec(`DELETE FROM 설비 WHERE 설비번호 = 6;`);
const 되돌림 = await db.query(`SELECT count(*) AS 개수 FROM 설비`);
console.log("되돌린 뒤 설비 수:", Number(되돌림.rows[0].개수));
```

출력 되돌린 뒤 설비 수: 5

★★★ LEFT JOIN 뒤의 WHERE 가 INNER JOIN 으로 바뀌 버립니다

섹션 4

이 파일에서 제일 중요한 섹션입니다. 실무에서 정말 자주 터집니다. 하고 싶은 일: "설비별 **불량** 점검이 몇 건인지 보고 싶다. 불량인 한 건도 없는 설비도, 점검을 안 한 설비도 나와야 한다." 세 가지 방법을 나란히 돌려 봅니다.

① INNER JOIN + WHERE

```
const 방법1 = await db.query(`
  SELECT s.이름, p.점검번호
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.결과 = '불량'`);
console.log("① INNER + WHERE:", 방법1.rows.length, "줄");
```

출력 ① INNER + WHERE: 2 줄

② LEFT JOIN + WHERE ← 흔히 이렇게 씁니다. 그런데 틀립니다.

```
const 방법2 = await db.query(`
  SELECT s.이름, p.점검번호
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.결과 = '불량'`);
console.log("② LEFT + WHERE:", 방법2.rows.length, "줄");
```

출력 ② LEFT + WHERE: 2 줄

```
console.log("①과 ②의 줄 수가 같은가:", 방법1.rows.length === 방법2.rows.length);
```

출력 ①과 ②의 줄 수가 같은가: true

★★★ LEFT 라고 써 놓았는데 INNER 와 결과가 똑같습니다. 실행 순서로 보면 답이 나옵니다.

1단계 ON 으로 짝을 짓습니다 → 16줄 (설비5는 NULL 줄로 살아 있음)
2단계 WHERE 로 줄을 버립니다 → p.결과 = '불량' 이 아닌 줄을 전부 버립니다

2단계에서 설비5의 줄은 p.결과 가 **NULL** 입니다. NULL = '불량' 은 거짓이 아니라 **UNKNOWN** 이고, WHERE 는 참인 줄만 남깁니다. (02단원에서 배운 NULL 3값 논리입니다. NULL 은 "모른다" 라서 비교가 안 됩니다) 그래서 LEFT JOIN 이 살려 둔 줄을 WHERE 가 다시 죽입니다. 불량이 없는 설비1~4 도 사라집니다.

★★★ LEFT JOIN 을 써 놓고 오른쪽 표 칸을 WHERE 에 걸면 INNER JOIN 이 됩니다.

③ 조건을 ON 절로 옮깁니다. ← 이게 정답입니다.

```
const 방법3 = await db.query(`
  SELECT s.설비번호, s.이름, p.점검번호
  FROM 설비 s
  LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호 AND p.결과 = '불량'
  ORDER BY s.설비번호`);
console.log("③ LEFT + ON:", 방법3.rows.length, "줄");
```

출력 ③ LEFT + ON: 5 줄

```
for (const 줄 of 방법3.rows) console.log(`· ${줄.이름} · 불량점검번호 ${없으면(줄.
점검번호)}`);
```

```
출력      · 컨베이어 1호 · 불량점검번호 (없음)
          · 프레스 1호 · 불량점검번호 6
          · 용접로봇 1호 · 불량점검번호 12
          · 검사기 1호 · 불량점검번호 (없음)
          · 포장기 1호 · 불량점검번호 (없음)
```

★ 설비 5대가 전부 나왔습니다. 불량인 설비는 점검번호가 (없음) 입니다.

★ ON 과 WHERE 는 이렇게 다릅니다.

```
ON      - **짝을 지을 때** 쓰는 조건. 안 맞으면 왼쪽 줄에 NULL 을 붙여 살려 둡니다.
WHERE   - **짝을 다 지은 다음** 줄을 버리는 조건. 살려 둔 줄도 가리지 않고 버립니다.
```

INNER JOIN 에서는 둘의 결과가 같습니다(살려 둘 줄이 없으니까요). **OUTER JOIN**에서만 갈립니다. 그래서 헷갈립니다.

④ WHERE 를 쓰면서 NULL 을 봐주면 어떻게 되나

```
const 방법4 = await db.query(`
  SELECT s.설비번호, s.이름, p.결과
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.결과 IS NULL OR p.결과 = '불량'
  ORDER BY s.설비번호`);
console.log("④ LEFT + WHERE(NULL 봐주기):", 방법4.rows.length, "줄");
```

```
출력      ④ LEFT + WHERE(NULL 봐주기): 3 줄
```

```
for (const 줄 of 방법4.rows) console.log(`· ${줄.이름} · ${없으면(줄.결과)}`);
```

```
출력      · 프레스 1호 · 불량
          · 용접로봇 1호 · 불량
          · 포장기 1호 · (없음)
```

```
console.log("불량이 없는 설비10이 살아났는가:", 방법4.rows.some((줄) => 줄.설비번호
=== 1));
```

```
출력      불량이 없는 설비10이 살아났는가: false
```

★ 절반만 고쳐졌습니다. 짝이 아예 없던 포장기 1호는 살아났습니다(결과가 NULL 이니까요). 그런데 **불량만 없는** 설비1~4 는 여전히 안 나옵니다. '정상' 줄이 있어서 NULL 도 아니고 '불량' 도 아니기 때문입니다. ★ 조건을 WHERE 에 두는 한 제대로 안 됩니다. **ON 으로 옮기는 게 정답입니다.**

```
읽기도 ON 쪽이 쉽습니다. `IS NULL OR` 이 붙으면 다음 사람이 한참 들여다봅니다.
```

⑤ ★ 반대로 왼쪽 표 조건은 WHERE 에 써도 안전합니다.

```
const 방법5 = await db.query(`
  SELECT s.설비번호, s.이름, p.점검번호
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE s.도입년도 >= 2019`);
console.log("⑤ 왼쪽 표 조건을 WHERE 에:", 방법5.rows.length, "줄");
```

출력 ⑤ 왼쪽 표 조건을 WHERE 에: 9 줄

```
console.log("짜 없는 포장기 1호가 살아있는가:", 방법5.rows.some((줄) => 줄.설비번호
=== 5));
```

출력 짜 없는 포장기 1호가 살아있는가: true

★ 왼쪽 표 칸은 NULL 로 채워지지 않으니 WHERE 에 걸어도 UNKNOWN 이 안 생깁니다. ★ 외울 것은 한 줄입니다. 오른쪽 표 조건 → ON. 왼쪽 표 조건 → WHERE.

LEFT JOIN ... WHERE 오른쪽.칸 IS NULL — “짜이 없는 것만”

섹션 5

방금 WHERE 가 줄을 죽인다고 했습니다. 그걸 일부러 쓰는 관용구가 있습니다.

```
const 점검없는설비 = await db.query(`
  SELECT s.설비번호, s.이름
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.점검번호 IS NULL
  ORDER BY s.설비번호`);
console.log("점검을 한 번도 안 한 설비:", 점검없는설비.rows.length, "대");
```

출력 점검을 한 번도 안 한 설비: 1 대

```
for (const 줄 of 점검없는설비.rows) console.log(`· ${줄.설비번호} · ${줄.이름}`);
```

출력 · 5 · 포장기 1호

★ 원리: LEFT JOIN 은 짜 없는 줄에만 NULL 을 채웁니다. 그러니 “오른쪽 칸이 NULL 인 줄” 은 곧 “짜이 없던 줄” 입니다.

```
const 설비없는라인 = await db.query(`
  SELECT l.라인코드, l.이름
  FROM 라인 l LEFT JOIN 설비 s ON l.라인코드 = s.라인코드
  WHERE s.설비번호 IS NULL`);
for (const 줄 of 설비없는라인.rows) console.log(`· 설비 없는 라인 - ${줄.
라인코드} ${줄.이름}`);
```

출력 · 설비 없는 라인 - D 신설4라인

★★★ NULL 검사는 반드시 **오른쪽 표의 NOT NULL 인 칸**(보통 기본키)으로 하세요. NULL 이 들어갈 수 있는 칸으로 검사하면 엉뚱한 줄이 떨어집니다. 재현해 봅니다.

```
const 잘못된검사 = await db.query(`
  SELECT s.설비번호, s.이름, p.점검번호
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.점수 IS NULL
  ORDER BY s.설비번호`);
console.log("점수 칸으로 검사했을 때:", 잘못된검사.rows.length, "줄");
```

출력 점수 칸으로 검사했을 때: 3 줄

```
for (const 줄 of 잘못된검사.rows) console.log(`· ${줄.이름} · 점검번호 ${없으면(줄.
점검번호)}`);
```

출력 · 용접로봇 1호 · 점검번호 9
 · 검사기 1호 · 점검번호 13
 · 포장기 1호 · 점검번호 (없음)

★ 1대만 나와야 하는데 3줄이 나왔습니다. 점검은 했는데 **점수만 안 적은** 기록(점검 9, 점검 13)이 떨어져 왔습니다. 점수는 NULL 이 허용된 칸이라 “짜이 없다” 와 “값을 안 적었다” 가 구별이 안 됩니다. 점검번호는 기본키라 NULL 이 없습니다. NULL 이면 짜이 없는 것이 확실합니다. ★ 같은 것을 **NOT EXISTS** 로도 쓸 수 있습니다. 그쪽이 뜻이 더 분명할 때가 많습니다. **WHERE NOT EXISTS (SELECT 1 FROM 점검기록 p WHERE p.설비번호 = s.설비번호)** 서버쿼리는 이 단원 개념04 에서 합니다.

CROSS JOIN — 일부러 모든 조합 만들기

섹션 6

개념01 에서 ON 을 빠뜨리면 카테시안 곱이 되어 사고가 난다고 했습니다. CROSS JOIN 은 그것을 **일부러** 쓰겠다고 밝히는 문법입니다. 제일 쓸모 있는 자리는 **빈칸 채우기** 입니다. 월별 표에서 점검이 없던 달은 줄이 안 생깁니다.

```
const 달목록 = await db.query(`
  SELECT to_char(월, 'YYYY-MM') AS 월
  FROM generate_series('2024-01-01'::date, '2024-05-01'::date, '1 month') AS 달(월)
  ORDER BY 월`);
for (const 줄 of 달목록.rows) console.log(`· ${줄.월}`);
```

출력 · 2024-01
 · 2024-02
 · 2024-03
 · 2024-04
 · 2024-05

★ generate_series 로 만든 날짜는 자바스크립트에 Date 객체로 옵니다. 그대로 찍으면 시간대 때문에 하루 어긋나 보입니다. SQL 안에서 to_char 로 문자열을 만들어 받는 쪽이 안전합니다.


```
const 달곱설비 = await db.query(`
  SELECT count(*) AS 개수
  FROM generate_series('2024-01-01'::date, '2024-05-01'::date, '1 month') AS 달(월)
  CROSS JOIN 설비 s`);
console.log("5개월 × 설비 5대 =", Number(달곱설비.rows[0].개수), "줄");
```

출력 5개월 × 설비 5대 = 25 줄

```
const 월별표 = await db.query(`
  SELECT to_char(달.월, 'YYYY-MM') AS 월, s.이름, count(p.점검번호) AS 점검수
  FROM generate_series('2024-01-01'::date, '2024-05-01'::date, '1 month') AS 달(월)
  CROSS JOIN 설비 s
  LEFT JOIN 점검기록 p
    ON p.설비번호 = s.설비번호 AND date_trunc('month', p.점검일) = 달.월
  WHERE s.설비번호 = 1
  GROUP BY 달.월, s.이름
  ORDER BY 달.월`);
for (const 줄 of 월별표.rows) console.log(`· ${줄.월} · ${줄.이름} · ${Number(줄.
점검수)}건`);
```

출력 · 2024-01 · 컨베이어 1호 · 1건
 · 2024-02 · 컨베이어 1호 · 1건
 · 2024-03 · 컨베이어 1호 · 1건
 · 2024-04 · 컨베이어 1호 · 1건
 · 2024-05 · 컨베이어 1호 · 0건

달력 없이 그냥 묶으면 어떻게 되는지 대조해 봅니다.

```
const 달력없이 = await db.query(`
  SELECT to_char(date_trunc('month', 점검일), 'YYYY-MM') AS 월, count(*) AS 점검수
  FROM 점검기록 WHERE 설비번호 = 1
  GROUP BY 1 ORDER BY 1`);
console.log("달력 없이 묶으면:", 달력없이.rows.length, "줄");
```

출력 달력 없이 묶으면: 4 줄

```
for (const 줄 of 달력없이.rows) console.log(`· ${줄.월} · ${Number(줄.점검수)}건`);
```

출력 · 2024-01 · 1건
 · 2024-02 · 1건
 · 2024-03 · 1건
 · 2024-04 · 1건

```
console.log("5월 줄이 있는가:", 달력없이.rows.some((줄) => 줄.월 === "2024-05"));
```

출력 5월 줄이 있는가: false

★ 5월에 점검이 없으니 5월 줄이 아예 안 생깁니다. 그래프를 그리면 5월이 통째로 빠집니다. 달력을 CROSS JOIN 으로 먼저 깔고 거기에 LEFT JOIN 을 붙이면 0 으로 채워집니다.

SELF JOIN — 자기 자신과 잇기

섹션 7

표를 두 번 써서 자기 자신과 잇습니다. 별칭을 반드시 달아야 구분이 됩니다. ① 같은 라인에 있는 설비끼리 짝짓기

```
const 짝짓기전부 = await db.query(`SELECT count(*) AS 개수 FROM 설비 a JOIN 설비 b ON
a.라인코드 = b.라인코드`);
const 짝짓기다름 = await db.query(`SELECT count(*) AS 개수 FROM 설비 a JOIN 설비 b ON
a.라인코드 = b.라인코드 AND a.설비번호 <> b.설비번호`);
const 짝짓기작음 = await db.query(`
  SELECT a.이름 AS 설비가, b.이름 AS 설비나
  FROM 설비 a JOIN 설비 b ON a.라인코드 = b.라인코드 AND a.설비번호 < b.설비번호
  ORDER BY a.설비번호, b.설비번호`);
```

```
console.log("조건 없이:", Number(짝짓기전부.rows[0].개수), "줄");
```

출력 조건 없이: 9 줄

```
console.log("a <> b:", Number(짝짓기다름.rows[0].개수), "줄");
```

출력 a <> b: 4 줄

```
console.log("a < b:", 짝짓기작음.rows.length, "줄");
```

출력 a < b: 2 줄

```
for (const 줄 of 짝짓기작음.rows) console.log(`· ${줄.설비가} ↔ ${줄.설비나}`);
```

출력 · 컨베이어 1호 ↔ 프레스 1호
 · 용접로봇 1호 ↔ 검사기 1호

★ 조건 없이 이으면 자기 자신끼리도 짝이 됩니다 (컨베이어 ↔ 컨베이어). <> 는 그건 빼 주지만 (1,2)와 (2,1)이 둘 다 나옵니다. 같은 짝인데 두 번입니다. < 를 쓰면 한 번만 나옵니다. 짝을 만들 때는 부등호를 쓰세요.

② 같은 설비의 바로 앞 점검을 찾아 간격 계산하기

```
const 직전점검 = await db.query(`
  SELECT to_char(이번.점검일, 'YYYY-MM-DD') AS 이번일,
         to_char(max(이전.점검일), 'YYYY-MM-DD') AS 직전일,
         (이번.점검일 - max(이전.점검일)) AS 간격일
  FROM 점검기록 이번
```

```
ON 이전.설비번호 = 이번.설비번호 AND 이전.점검일 < 이번.점검일
WHERE 이번.설비번호 = 1
GROUP BY 이번.점검번호, 이번.점검일
ORDER BY 이번.점검일`);
for (const 줄 of 직전점검.rows) {
  console.log(`· ${줄.이번일} · 직전 ${없으면(줄.직전일)} · ${없으면(줄.간격일)}일`);
}
```

```
출력      · 2024-01-05 · 직전 (없음) · (없음)일
          · 2024-02-05 · 직전 2024-01-05 · 31일
          · 2024-03-06 · 직전 2024-02-05 · 30일
          · 2024-04-05 · 직전 2024-03-06 · 30일
```

★ 첫 점검은 앞이 없어서 (없음) 입니다. LEFT JOIN 이라 줄은 살아 있습니다. INNER JOIN 이었으면 첫 점검이 사라집니다. 여기서도 같은 이야기입니다. ★ 이 질의가 까다로운 이유: “바로 앞” 을 찾으려고 ** 앞선 것 전부를 이어 놓고 그 중 최댓값**을 골랐습니다. 점검이 많아지면 이어지는 줄이 확 늘어납니다. LAG 라는 창 함수를 쓰면 훨씬 짧고 빠릅니다. 이 단원 개념05 에서 합니다. ★ 사원-상사 같은 계층 구조도 SELF JOIN 으로 편합니다. 단계가 정해져 있지 않으면 재귀 질의가 필요합니다. 그건 개념04 에서 합니다.

JOIN 종류별 결과 줄 수

섹션 8

설비(5줄) ↔ 점검기록(15줄) 을 설비번호로 이었을 때 몇 줄이 나오는지 전부 세어 봅니다.

```
const 종류들 = [
  ["INNER JOIN", "`, `설비 s JOIN", "점검기록 p ON s.설비번호 = p.설비번호"],
  ["LEFT JOIN", "`, `설비 s LEFT JOIN", "점검기록 p ON s.설비번호 = p.설비번호"],
  ["RIGHT JOIN", "`, `설비 s RIGHT JOIN", "점검기록 p ON s.설비번호 = p.설비번호"],
  ["FULL OUTER JOIN", "`, `설비 s FULL OUTER JOIN", "점검기록 p ON s.설비번호 = p.설비번호"],
  ["CROSS JOIN", "`, `설비 s CROSS JOIN", "점검기록 p`"],
];
for (const [이름, 절] of 종류들) {
  const 결과 = await db.query(`SELECT count(*) AS 개수 FROM ${절}`);
  console.log(`· ${이름} · ${Number(결과.rows[0].개수)}줄`);
}
```

```
출력      · INNER JOIN      · 15줄
          · LEFT JOIN       · 16줄
          · RIGHT JOIN      · 15줄
          · FULL OUTER JOIN · 16줄
          · CROSS JOIN      · 75줄
```

★ RIGHT 가 15줄인 이유: 점검기록은 전부 실제 설비를 가리켜서 짝 없는 점검기록이 없습니다. 그래서 INNER 와 같아집니다. 외래키가 지켜 주고 있기 때문입니다 (04단원). ★ CROSS 가 75줄인 이유: 5 × 15. ON 을 빠뜨리면 이 숫자가 나옵니다. 여기서는 75줄이지만 만 줄 × 만 줄이면 1억 줄입니다. 서버가 멈춥니다.

정리 — 언제 무엇을 쓰나

무엇 무슨 줄이 남나 설비↔점검기록

INNER JOIN 양쪽 다 짝이 있는 줄 15줄 LEFT JOIN 왼쪽 전부 + 짝 있으면 붙임 16줄 RIGHT JOIN 오른쪽 전부 + 짝 있으면 붙임 15줄 FULL OUTER JOIN 양쪽 전부 16줄 CROSS JOIN 모든 조합 (곱하기) 75줄 ★ 오른쪽 표 조건 → ON 에 짝 없는 줄이 삽니다 ★ 오른쪽 표 조건 → WHERE 에 ★★★ INNER 가 됩니다 ★ 왼쪽 표 조건 → WHERE 에 안전합니다 짝 없는 것만 찾기 LEFT JOIN ... WHERE 오른쪽.기본키 IS NULL 빈칸 채우기 달력 CROSS JOIN 대상 LEFT JOIN 사실 자기끼리 짝짓기 SELF JOIN ... ON a.키 < b.키

직접 해 볼 것

▫ 직접 해보기 1

작업자 표로 같은 것을 해 보세요. 점검을 한 번도 담당한 적 없는 작업자는? (정신입 하나만 나와야 합니다. 담당사번이 NULL 인 점검기록 2건에 낚이지 않도록 어느 칸으로 IS NULL 검사를 할지 잘 고르세요)

▫ 직접 해보기 2

섹션 4 의 ㉠ 을 고쳐서 설비별 불량 건수를 0 포함해서 세어 보세요. count(*) 로 세면 왜 틀리나요? 무엇으로 세야 하나요?

▫ 직접 해보기 3

섹션 4 의 ㉠ 에서 AND p.결과 = '불량' 을 ON 에서 WHERE 로 옮겨 보세요. 줄 수가 몇 줄로 바뀌나요? 왜 그런가요?

▫ 직접 해보기 4

라인 · 설비 · 점검기록 세 표를 이어서 "라인별 점검 건수" 를 만들어 보세요. 설비가 없는 라인 D 도 0 이어야 합니다. (힌트: LEFT JOIN 을 두 번. 중간에 INNER 를 쓰면 D 가 죽습니다)

▫ 직접 해보기 5

섹션 6 의 달력을 2024-01-01 ~ 2024-12-01 로 늘려 보세요. 설비 5대와 CROSS JOIN 하면 몇 줄인가요? 6월 이후는 몇 건인가요?

⇒ 직접 해보기

6

섹션 7 의 ㉠ 에서 < 를 <= 로 바꾸면 몇 줄이 되나요? 왜 늘어나나요?

⇒ 직접 해보기

7

설비 6번(라인 없음)을 다시 넣고 설비 LEFT JOIN 라인 을 해 보세요. ON s.라인코드 = l.라인코드 에서 NULL 은 무엇과도 안 맞습니다. 라인코드가 NULL 인 설비는? ★ 끝나면 꼭 지우세요.

자주 하는 실수

이런 실수를 합니다

★★★ LEFT JOIN 을 해 놓고 오른쪽 표 조건을 WHERE 에 씌

LEFT JOIN 점검기록 p ON ... WHERE p.결과 = '불량' → NULL 줄이 UNKNOWN 으로 걸려져서 INNER JOIN 이 됩니다. 오른쪽 표 조건은 ON 에, 왼쪽 표 조건만 WHERE 에 쓰세요.

이런 실수를 합니다

IS NULL 검사를 NULL 이 들어갈 수 있는 칸으로 함

WHERE p.점수 IS NULL 로 하면 점수를 안 적은 기록까지 달려옵니다. 오른쪽 표의 기본키로 검사하세요. 기본키에는 NULL 이 없습니다.

이런 실수를 합니다

짜 없는 줄을 count(*) 로 세서 1건이 됨

LEFT JOIN 이 만든 NULL 줄도 줄은 줄입니다. count(*) 는 1 을 셉니다. 0 으로 세려면 count(오른쪽표. 칸) 을 쓰세요. 집계는 개념03 에서 자세히 합니다.

이런 실수를 합니다

LEFT JOIN 을 여러 번 이으면서 중간에 INNER 를 섞음

라인 LEFT JOIN 설비 JOIN 점검기록 은 뒤의 INNER 가 앞의 LEFT 를 무효로 만듭니다. 라인 D 는 설비 칸이 NULL 인데, 다시 INNER 로 이으면 짝이 없어 죽습니다.

이런 실수를 합니다

RIGHT JOIN 을 섞어 써서 기준이 뭔지 알 수 없게 함

A LEFT JOIN B RIGHT JOIN C 는 못 따라갑니다. 표 순서를 바꿔 전부 LEFT 로 통일하세요.

이런 실수를 합니다

ON 을 빠뜨려서 **CROSS JOIN** 이 됨

FROM 설비 s, 점검기록 p 라고 쓰고 WHERE 조건을 잊으면 75줄이 나옵니다. 작은 표에서는 결과가 이상한 정도지만 큰 표에서는 서버가 멈춥니다. 조합이 진짜로 필요하면 **CROSS JOIN** 이라고 **밝혀서** 쓰세요.

```
await db.close();
```

묶어서 세기

RUN node 개념03_묶어서_세기.js

개념01 에서 표를 이었고, 개념02 에서 짝 없는 쪽도 살렸습니다. 이제 이어 놓은 줄들을 **묶어서 셉니다**.
 “설비마다 몇 번 점검했습니까”, “라인별 평균 점수가 얼마입니까” 같은 질문입니다. 줄을 하나씩 보는 게 아니라 뭉텅이로 줄여서 봅니다. 이것을 집계라고 합니다.

★ 05단원에서 사고가 제일 많이 나는 자리입니다. COUNT(*) 와 COUNT(칸) 이 다르고, 없는 값이 0 이 아니라 NULL 로 나옵니다. 전부 실제로 돌려서 눈으로 확인하겠습니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

```
await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    동        TEXT NOT NULL
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드),
    도입년도 INT NOT NULL
  );

  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드)
  );

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호),
    점검일    DATE NOT NULL,
    결과      TEXT NOT NULL,
    점수      INT,
    담당사번 INT REFERENCES 작업자(사번)
  );

  INSERT INTO 라인 VALUES
    ('A', '조립1라인', '1동'),
    ('B', '가공2라인', '1동'),
    ('C', '포장3라인', '2동'),
    ('D', '신설4라인', '2동');

  INSERT INTO 설비 VALUES
```



```
(1, '컨베이어 1호', 'A', 2015),
(2, '프레스 1호', 'A', 2019),
(3, '용접로봇 1호', 'B', 2021),
(4, '검사기 1호', 'B', 2018),
(5, '포장기 1호', 'C', 2022);
```

INSERT INTO 작업자 VALUES

```
(101, '김반장', 'A'),
(102, '이기사', 'B'),
(103, '박주임', 'B'),
(104, '최사원', 'C'),
(105, '정신입', NULL);
```

INSERT INTO 점검기록 VALUES

```
( 1, 1, '2024-01-05', '정상', 92, 101),
( 2, 1, '2024-02-05', '정상', 88, 101),
( 3, 1, '2024-03-06', '주의', 71, 102),
( 4, 1, '2024-04-05', '정상', 90, 101),
( 5, 2, '2024-01-12', '주의', 65, 102),
( 6, 2, '2024-02-14', '불량', 48, 103),
( 7, 2, '2024-03-15', '정상', 88, 101),
( 8, 3, '2024-01-20', '정상', 95, 103),
( 9, 3, '2024-02-20', '정상', NULL, 103),
(10, 3, '2024-03-21', '주의', 74, 102),
(11, 3, '2024-04-22', '정상', 90, NULL),
(12, 3, '2024-05-23', '불량', 52, 103),
(13, 4, '2024-01-28', '정상', NULL, 104),
(14, 4, '2024-03-29', '정상', 88, 104),
(15, 4, '2024-05-30', '주의', 69, NULL);
`);
```

집계는 여러 줄을 한 줄로 접는 것입니다

섹션 1

점검기록은 15줄입니다. 아래 SELECT 는 그 15줄을 한 줄로 접습니다. COUNT / SUM / MIN / MAX / AVG 를 집계 함수라고 부릅니다.

```
const 전체집계 = await db.query(
  `SELECT COUNT(*) AS 줄수, SUM(점수) AS 점수합, MIN(점수) AS 최저, MAX(점수) AS 최고
  FROM 점검기록`);
console.log("돌아온 줄 수:", 전체집계.rows.length);
```

출력 돌아온 줄 수: 1

```
const 점힌줄 = 전체집계.rows[0];
console.log(`· 줄수 ${점힌줄.줄수} · 점수합 ${점힌줄.점수합} · 최저 ${점힌줄.최저} ·
최고 ${점힌줄.최고}`);
```

출력 · 줄수 15 · 점수합 1010 · 최저 48 · 최고 95

★ 15줄이 1줄이 되었습니다. 이게 집계 of 전부입니다. 그래서 SELECT 점검번호, COUNT(*) 은 말이 안 됩니다. 한 줄에 점검번호 15개 중 뭘 넣을지 정할 수 없으니까요. (섹션 4 에서 확인) ★ COUNT 의 SQL 타입은 INT 가 아니라 BIGINT 입니다. 타입번호(OID)로 확인합니다.

```
console.log("COUNT(*) 의 타입번호:", 전체집계.fields[0].dataTypeID, "· 20 이면
BIGINT");
```

출력 COUNT(*) 의 타입번호: 20 · 20 이면 BIGINT

★★ 그런데 PGlite 는 BIGINT 를 JS number 에 안전하게 담기는 크기면 number 로, 넘어가면 bigint 로 줍니다. 같은 칸인데 값의 크기에 따라 타입이 달라집니다.

```
const 타입 = await db.query(
  `SELECT COUNT(*) AS 작은수, 9007199254740993::BIGINT AS 큰수 FROM 점검기록`);
console.log("작은 COUNT:", typeof 타입.rows[0].작은수, "· 아주 큰 BIGINT:", typeof
타입.rows[0].큰수);
```

출력 작은 COUNT: number · 아주 큰 BIGINT: bigint

★★★ 이게 왜 무섭습니까. 개발할 때는 줄이 적어 number 로 오다가, 운영에서 줄이 쌓이면 bigint 로 바뀝니다. 그 순간 JSON.stringify 가 터집니다. 어제까지 멀쩡하던 API 입니다.

```
let json결과;
try { json결과 = JSON.stringify(타입.rows[0]); }
catch (에러) { json결과 = `${에러.constructor.name}: ${에러.message}`; }
console.log("bigint 가 섞인 줄을 JSON.stringify 하면:", json결과);
```

출력 bigint 가 섞인 줄을 JSON.stringify 하면: TypeError: Do not know how to
serialize a BigInt

★ 그래서 집계 결과는 JS 로 넘기기 전에 Number() 로 감싸세요. number 면 아무 일도 없고 bigint 일 때만 살아납니다. 줄이 몇 개든 코드가 똑같이 돌아갑니다.

```
console.log("Number() 로 감싸서 더하기:", Number(점힌줄.줄수) + 1);
```

출력 Number() 로 감싸서 더하기: 16

★★ COUNT(*) 와 COUNT(칸) 과 COUNT(DISTINCT 칸)

섹션 2

이름이 다 COUNT 라 같아 보이지만 셋 다 다른 것을 썬니다. · COUNT(*) → 줄을 썬니다(NULL 상관없음) · COUNT(칸) → 그 칸이 NULL 이 아닌 줄만 · COUNT(DISTINCT 칸) → NULL 이 아닌 값 중 서로 다른 값의 개수

```
const 세는법 = await db.query(`
  SELECT COUNT(*) AS 별표, COUNT(점수) AS 점수칸, COUNT(DISTINCT 점수) AS 점수종류,
    COUNT(담당사번) AS 담당칸, COUNT(DISTINCT 담당사번) AS 담당종류
  FROM 점검기록`);
const L = 세는법.rows[0];
console.log(` · COUNT(*)                = ${L.별표}`);
```

```
출력      · COUNT(*)                = 15
```

```
console.log(` · COUNT(점수)            = ${L.점수칸}`);
```

```
출력      · COUNT(점수)            = 13
```

```
console.log(` · COUNT(DISTINCT 점수)    = ${L.점수종류}`);
```

```
출력      · COUNT(DISTINCT 점수)    = 10
```

```
console.log(` · COUNT(담당사번)        = ${L.담당칸}`);
```

```
출력      · COUNT(담당사번)        = 13
```

```
console.log(` · COUNT(DISTINCT 담당사번) = ${L.담당종류}`);
```

```
출력      · COUNT(DISTINCT 담당사번) = 4
```

15 = 줄 수 / 13 = 점수가 NULL 인 점검 9·13 을 뺀 것 / 10 = 점수 13개 중 88 이 3번, 90 이 2번 겹쳐 13-2-1 / 13 = 담당사번 NULL 인 11·15 를 뺀 것 / 4 = 101·102·103·104 (105 없음) ★ NULL 이 왜 안 세어지는지는 02단원의 “NULL 은 값이 아니라 모름” 과 같은 이야기입니다.

★★★ 사고 사례: 평균을 손으로 계산하기

“점수 합을 건수로 나누면 평균이지” — 이게 틀립니다. AVG 는 점수가 있는 13건으로 나누는데 COUNT(*) 는 15 이기 때문입니다.

```
const 평균비교 = await db.query(`
  SELECT AVG(점수) AS 진짜평균, SUM(점수) / COUNT(*) AS 그냥나눔,
         SUM(점수)::numeric / COUNT(*) AS 열다섯으로, SUM(점수)::numeric /
COUNT(점수) AS 열셋으로
  FROM 점검기록`);
const c = 평균비교.rows[0];
console.log("AVG(점수)           :", c.진짜평균);
```

```
출력      AVG(점수)           : 77.6923076923076923
```

```
console.log("SUM(점수) / COUNT(*)           :", c.그냥나눔);
```

```
출력      SUM(점수) / COUNT(*)           : 67
```

```
console.log("SUM(점수)::numeric / COUNT(*):", c.열다섯으로);
```

```
출력      SUM(점수)::numeric / COUNT(*) : 67.3333333333333333
```

```
console.log("SUM(점수)::numeric / COUNT(점수):", c.열셋으로);
```

```
출력      SUM(점수)::numeric / COUNT(점수) : 77.6923076923076923
```

```
console.log("AVG 와 'COUNT(점수)로 나눈 값'이 같은가:", c.진짜평균 === c.열셋으로);
```

```
출력      AVG 와 'COUNT(점수)로 나눈 값'이 같은가: true
```

★ 사고가 두 겹으로 났습니다. ① 15 로 나눠서 67.33 — 평균이 10점이나 낮게 나옵니다 ② 캐스팅을 안 하면 BIGINT ÷ BIGINT = 정수 나눗셈이라 소수점이 잘려 67 이 됩니다 ★★ 평균이 필요하다면 손으로 나누지 말고 그냥 AVG 를 쓰세요.

개념02 에서 예고한 것: 짝 없는 설비

설비 5(포장기 1호)는 점검기록이 한 건도 없습니다. LEFT JOIN 하면 오른쪽이 전부 NULL 인 “빈 짝” 한 줄이 생깁니다. 그 줄도 줄은 줄입니다.

```
const 짝없는설비 = await db.query(`
  SELECT s.설비번호, s.이름, COUNT(*) AS 별표로세기, COUNT(p.점검번호) AS 칸으로세기
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE s.설비번호 = 5 GROUP BY s.설비번호, s.이름`);
for (const 줄 of 짝없는설비.rows)
  console.log(` · ${줄.이름} · COUNT(*)=${줄.별표로세기} · COUNT(p.점검번호)=${줄.칸으로세기}`);
```

```
출력      · 포장기 1호 · COUNT(*)=1 · COUNT(p.점검번호)=0
```

★★★ 결론입니다. LEFT JOIN 위에서 세는 것은 COUNT(*) 가 아니라 COUNT(오른쪽칸) 입니다.

GROUP BY — 전체가 아니라 뭉텅이별로 접기

섹션 3

지금까지는 표 전체를 한 줄로 접었습니다. GROUP BY 를 붙이면 같은 값끼리 묶어서 묶음마다 한 줄로 접습니다.

```
const 설비별 = await db.query(`
  SELECT 설비번호, COUNT(*) AS 점검수 FROM 점검기록 GROUP BY 설비번호 ORDER BY
  설비번호`);
for (const 줄 of 설비별.rows) console.log(`· 설비 ${줄.설비번호} · ${줄.점검수}건`);
```

```
출력      · 설비 1 · 4건
          · 설비 2 · 3건
          · 설비 3 · 5건
          · 설비 4 · 3건
```

★ 설비 5 가 없습니다. 점검기록 안에 설비 5 라는 값이 없으니 묶을 뭉텅이가 안 생깁니다. 번호만 봐서는 무슨 설비인지 모릅니다. 개념01 의 JOIN 을 엮습니다. ★ SELECT 에 올린 s.이름 은 GROUP BY 에도 같이 적어 줍니다. (이유는 섹션 4)

```
const 이름까지 = await db.query(`
  SELECT s.설비번호, s.이름, COUNT(*) AS 점검수
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.설비번호, s.이름 ORDER BY s.설비번호`);
for (const 줄 of 이름까지.rows) console.log(`· ${줄.이름} · ${줄.점검수}건`);
```

```
출력      · 컨베이어 1호 · 4건
          · 프레스 1호 · 3건
          · 용접로봇 1호 · 5건
          · 검사기 1호 · 3건
```

★★ 제대로 된 집계: LEFT JOIN + COUNT(오른쪽칸)

위 결과에는 여전히 포장기 1호가 빠져 있습니다. JOIN 이 INNER 라서 그렇습니다. 개념02 의 LEFT JOIN 으로 바꾸고 세는 방법 두 가지를 나란히 놓아 봅니다.

```
const 제대로 = await db.query(`
  SELECT s.설비번호, s.이름, COUNT(*) AS 별표, COUNT(p.점검번호) AS 칸
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.설비번호, s.이름 ORDER BY s.설비번호`);
for (const 줄 of 제대로.rows) console.log(`· ${줄.이름} · COUNT(*)=${줄.별표} ·
COUNT(칸)=${줄.칸}`);
```

```
출력      · 컨베이어 1호 · COUNT(*)=4 · COUNT(칸)=4
          · 프레스 1호 · COUNT(*)=3 · COUNT(칸)=3
          · 용접로봇 1호 · COUNT(*)=5 · COUNT(칸)=5
          · 검사기 1호 · COUNT(*)=3 · COUNT(칸)=3
          · 포장기 1호 · COUNT(*)=1 · COUNT(칸)=0
```

★ 네 줄은 같고 마지막 한 줄만 다릅니다. 그 한 줄 때문에 보고서가 틀립니다. "점검 0건인 설비"를 찾는 게 목적이었는데 1건으로 보이면 영영 못 찾습니다. GROUP BY 에 칸을 두 개 적으면 그 두 값의 조합마다 한 줄이 나옵니다.

```
const 라인결과별 = await db.query(`
  SELECT s.라인코드, p.결과, COUNT(*) AS 건수
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.라인코드, p.결과 ORDER BY s.라인코드, p.결과`);
for (const 줄 of 라인결과별.rows) console.log(` ${줄.라인코드}라인 · ${줄.결과} ·
${줄.건수}건`);
```

```
출력      · A라인 · 불량 · 1건
          · A라인 · 정상 · 4건
          · A라인 · 주의 · 2건
          · B라인 · 불량 · 1건
          · B라인 · 정상 · 5건
          · B라인 · 주의 · 2건
```

★ GROUP BY 에 없는 칸을 SELECT 하면 에러

섹션 4

설비 1 에는 점검일이 4개 있습니다. 설비별로 한 줄만 남기기로 했는데 그 한 줄의 점검일 칸에 4개 중 무엇을 넣어야 합니까. 정할 수가 없습니다. 그래서 Postgres 는 아예 막습니다.

```
try {
  await db.query(`SELECT 설비번호, 점검일, COUNT(*) FROM 점검기록 GROUP BY 설비번호`);
} catch (에러) {
  console.log("e.code      :", 에러.code);
```

```
출력      e.code      : 42803
```

```
console.log("e.message :", 에러.message);
```

```
출력      e.message : column "점검기록.점검일" must appear in the GROUP BY clause
          or be used in an aggregate function
```

```
}
```

★ 메시지가 고치는 법을 그대로 알려 줍니다. ① GROUP BY 절에 넣든가 ② 집계로 감싸든가.

고치는 법 ① GROUP BY 에 넣기

점검일까지 묶음의 기준이 됩니다. "설비+날짜" 조합마다 한 줄이라 15줄이 그대로 15줄입니다. 접힌 게 없으니 집계한 보람이 없습니다.

```
const 고침1 = await db.query(`
  SELECT 설비번호, 점검일, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 설비번호, 점검일
`);
console.log("고침① 돌아온 줄 수:", 고침1.rows.length);
```

출력 고침① 돌아온 줄 수: 15

고치는 법 ② 집계로 감싸기

4개 중 무엇을 보여줄지 우리가 정해 주는 것입니다. 여기서는 처음 날짜와 마지막 날짜. ★ DATE 는 JS 에서 Date 객체로 옵니다. 시간대까지 붙어 지저분하니 TO_CHAR 로 문자열로 만듭니다.

```
const 고침2 = await db.query(`
  SELECT 설비번호, TO_CHAR(MIN(점검일), 'YYYY-MM-DD') AS 첫점검,
    TO_CHAR(MAX(점검일), 'YYYY-MM-DD') AS 마지막점검, COUNT(*) AS 건수
  FROM 점검기록 GROUP BY 설비번호 ORDER BY 설비번호`);
for (const 줄 of 고침2.rows)
  console.log(`설비 ${줄.설비번호} · ${줄.첫점검} ~ ${줄.마지막점검} · ${줄.건수}건`);
```

출력 · 설비 1 · 2024-01-05 ~ 2024-04-05 · 4건
 · 설비 2 · 2024-01-12 ~ 2024-03-15 · 3건
 · 설비 3 · 2024-01-20 ~ 2024-05-23 · 5건
 · 설비 4 · 2024-01-28 ~ 2024-05-30 · 3건

★ 기본키로 묶으면 나머지 칸은 봐줍니다

설비번호는 설비 표의 기본키입니다. 설비번호가 정해지면 이름도 도입년도도 하나로 정해집니다. 이것을 기능적 종속이라고 합니다(04단원). 정할 수 있으니 Postgres 가 통과시켜 줍니다.

```
const 기본키묶기 = await db.query(`
  SELECT s.설비번호, s.이름, s.도입년도, COUNT(p.점검번호) AS 점검수
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.설비번호 ORDER BY s.설비번호`);
for (const 줄 of 기본키묶기.rows) console.log(`· ${줄.이름} · ${줄.도입년도}년 · ${줄.점검수}건`);
```

출력

- 컨베이어 1호 · 2015년 · 4건
- 프레스 1호 · 2019년 · 3건
- 용접로봇 1호 · 2021년 · 5건
- 검사기 1호 · 2018년 · 3건
- 포장기 1호 · 2022년 · 0건

★ GROUP BY 에 s.설비번호 하나만 적었는데 s.이름 과 s.도입년도 가 통과했습니다. 기본키가 아니면 이 봐주기가 없습니다. 라인코드로 묶어 보면 바로 막힙니다.

```
try {
  await db.query(`SELECT s.라인코드, s.이름, COUNT(*) FROM 설비 s GROUP BY s.라인코드`);
} catch (에러) {
  console.log("라인코드로 묶으면:", 에러.code, ".", 에러.message);
}
```

출력 라인코드로 묶으면: 42803 · column "s.이름" must appear in the GROUP BY clause or be used in an aggregate function

}

MySQL 은 여기가 다릅니다

· MySQL 은 ONLY_FULL_GROUP_BY 를 끄면 안 막습니다. 4개 중 아무거나 하나가 슬쩍 나옵니다 ★
자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

★ WHERE 와 HAVING — 실행 순서가 전부입니다

섹션 5

SQL 은 적힌 순서대로 도는 게 아닙니다. 이 순서로 둡니다.

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

줄을 만든다 묶기 전 묶는다 묶은 뒤 별칭이 생기는 자리

거름 거름

WHERE 는 묶기 전에 줄을, HAVING 은 묶은 뒤에 묶음을 걸러냅니다. 둘 다 거르지만 거르는 대상이 줄이냐 묶음이냐가 다릅니다.

```
const 삼월이후 = await db.query(
  `SELECT COUNT(*) AS 건수 FROM 점검기록 WHERE 점검일 >= DATE '2024-03-01'`;
  console.log("3월 이후 점검 건수:", 삼월이후.rows[0].건수);
```

출력 3월 이후 점검 건수: 8

이제 그 8건을 설비별로 묶고, 2건 이상인 설비만 남깁니다.

```
const 둘다 = await db.query(`
  SELECT s.설비번호, s.이름, COUNT(*) AS 건수
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.점검일 >= DATE '2024-03-01'          -- 묶기 전에 줄을 거름
  GROUP BY s.설비번호, s.이름 HAVING COUNT(*) >= 2 -- 묶은 뒤에 묶음을 거름
  ORDER BY s.설비번호`);
for (const 줄 of 둘다.rows) console.log(`· ${줄.이름} · ${줄.건수}건`);
```

출력

- 컨베이어 1호 · 2건
- 용접로봇 1호 · 3건
- 검사기 1호 · 2건

★ 프레스 1호는 3월 이후 1건뿐이라 HAVING 에서 탈락, 포장기 1호는 INNER JOIN 이라 이미 없음.

★ WHERE 에 COUNT 를 쓰면 에러

WHERE 가 도는 시점에는 아직 GROUP BY 를 안 했습니다. 셀 묶음이 없습니다.

```
try {
  await db.query(`SELECT 설비번호, COUNT(*) FROM 점검기록 WHERE COUNT(*) > 2 GROUP BY
설비번호`);
} catch (에러) {
  console.log("WHERE 에 COUNT:", 에러.code, ".", 에러.message);
```

출력 WHERE 에 COUNT: 42803 · aggregate functions are not allowed in WHERE

```
}
```

★ 별칭을 WHERE/HAVING 에서 못 쓰는 것도 같은 이유입니다 (SELECT 가 더 나중이라 없습니다)

```
try {
  await db.query(
    `SELECT 설비번호, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 설비번호 HAVING 건수 >=
4`);
} catch (에러) {
  console.log("HAVING 에 별칭:", 에러.code, ".", 에러.message);
```

출력 HAVING 에 별칭: 42703 · column "건수" does not exist

```
}
try {
  await db.query(`SELECT 점검번호, 점수 AS 점 FROM 점검기록 WHERE 점 > 90`);
} catch (에러) {
  console.log("WHERE 에 별칭:", 에러.code, ".", 에러.message);
```

```
출력      WHERE 에 별칭: 42703 · column "점" does not exist
```

```
}
```

그런데 ORDER BY 와 GROUP BY 에서는 별칭이 됩니다

ORDER BY 는 SELECT 보다 나중이라 별칭이 이미 있습니다. GROUP BY 는 SELECT 보다 먼저지만 Postgres 가 특별히 봐주는 자리입니다. 외우지 말고 돌려 보고 확인하는 습관을 들이세요.

```
const 오더별칭 = await db.query(  
  `SELECT 설비번호, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 설비번호 ORDER BY 건수  
  DESC, 설비번호`);  
for (const 줄 of 오더별칭.rows) console.log(`설비 ${줄.설비번호} · ${줄.건수}건`);
```

```
출력      · 설비 3 · 5건  
          · 설비 1 · 4건  
          · 설비 2 · 3건  
          · 설비 4 · 3건
```

```
const 그룹별칭 = await db.query(  
  `SELECT 결과 AS 판정, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 판정 ORDER BY 판정`);  
for (const 줄 of 그룹별칭.rows) console.log(`${줄.판정} · ${줄.건수}건`);
```

```
출력      · 불량 · 2건  
          · 정상 · 9건  
          · 주의 · 4건
```

★ AVG 는 문자열로 돌아옵니다

섹션 6

여기서 한 번은 꼭 데입니다. 평균을 받아 더했는데 문자열 이어붙이기가 됩니다.

```
const 평균들 = await db.query(`  
  SELECT AVG(점수) AS 평균, ROUND(AVG(점수), 1) AS 반올림,  
         MIN(점수) AS 최저, MAX(점수) AS 최고 FROM 점검기록`);  
const ㄹ = 평균들.rows[0];  
console.log(`AVG(점수)          = ${ㄹ.평균} · 타입 ${typeof ㄹ.평균}`);
```

```
출력      · AVG(점수)          = 77.6923076923076923 · 타입 string
```

```
console.log(`ROUND(AVG(점수),1) = ${ㄹ.반올림} · 타입 ${typeof ㄹ.반올림}`);
```

```
출력      · ROUND(AVG(점수),1) = 77.7 · 타입 string
```

```
console.log(`MIN(점수)          = ${ㄹ.최저} · 타입 ${typeof ㄹ.최저}`);
```

```
출력      · MIN(점수)      = 48 · 타입 number
```

```
console.log(` · MAX(점수)      = ${ㄹ.최고} · 타입 ${typeof ㄹ.최고}`);
```

```
출력      · MAX(점수)      = 95 · 타입 number
```

★ 왜 이렇습니까. AVG(INT) 의 결과 타입은 NUMERIC 이고 NUMERIC 은 자릿수를 정확히 지킵니다. JS number(부동소수)로 바꾸면 정확도가 깨지므로 PGlite 는 문자열 그대로 넘깁니다. MIN/MAX 는 원래 칸의 타입(INT)을 물려받아 number 로 옵니다. (02단원과 이어집니다)

```
console.log("문자열끼리 그냥 더하면:", ㄹ.반올림 + 1);
```

```
출력      문자열끼리 그냥 더하면: 77.71
```

```
console.log("Number() 로 감싸면      :", Number(ㄹ.반올림) + 1);
```

```
출력      Number() 로 감싸면      : 78.7
```

★ ROUND 의 함정

ROUND(NUMERIC, 자릿수) 는 있는데 ROUND(double precision, 자릿수) 는 없습니다. 어딘가에서 float 으로 바뀌어 있으면 ROUND 가 통째로 실패합니다.

```
try {
  await db.query(`SELECT ROUND(AVG(점수)::double precision, 1) FROM 점검기록`);
} catch (에러) {
  console.log("float 에 ROUND:", 에러.code, ".", 에러.message);
```

```
출력      float 에 ROUND: 42883 · function round(double precision, integer) does
not exist
```

```
}
const 캐스팅 = await db.query(
  `SELECT ROUND(AVG(점수)::double precision::numeric, 1) AS 고침 FROM 점검기록`);
console.log("::numeric 을 거치면:", 캐스팅.rows[0].고침);
```

```
출력      ::numeric 을 거치면: 77.7
```

```
const 설비별평균 = await db.query(`
  SELECT s.이름, ROUND(AVG(p.점수), 1) AS 평균
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.설비번호, s.이름 ORDER BY s.설비번호`);
for (const 줄 of 설비별평균.rows) console.log(` · ${줄.이름} · 평균 ${줄.평균}`);
```

출력

- 컨베이어 1호 · 평균 85.3
- 프레스 1호 · 평균 67.0
- 용접로봇 1호 · 평균 77.8
- 검사기 1호 · 평균 78.5

★ 프레스 1호가 "67" 이 아니라 "67.0" 입니다. 문자열이라는 증거입니다.

FILTER — 조건별 집계를 한 줄에

섹션 7

"정상 몇 건, 주의 몇 건, 불량 몇 건" 을 쿼리 세 번 던져 뽑을 필요가 없습니다. 집계 함수마다 조건을 따로 붙일 수 있습니다.

```
const 필터 = await db.query(`
  SELECT COUNT(*) AS 전체,
         COUNT(*) FILTER (WHERE 결과 = '정상') AS 정상,
         COUNT(*) FILTER (WHERE 결과 = '주의') AS 주의,
         COUNT(*) FILTER (WHERE 결과 = '불량') AS 불량
  FROM 점검기록`);
const □ = 필터.rows[0];
console.log(` · 전체 ${□.전체} · 정상 ${□.정상} · 주의 ${□.주의} · 불량 ${□.불량}`);
```

출력

- 전체 15 · 정상 9 · 주의 4 · 불량 2

FILTER 가 없던 시절에는 CASE 로 이렇게 했습니다. 결과는 같습니다.

```
const 케이스 = await db.query(`
  SELECT COUNT(*) AS 전체,
         SUM(CASE WHEN 결과 = '정상' THEN 1 ELSE 0 END) AS 정상,
         SUM(CASE WHEN 결과 = '주의' THEN 1 ELSE 0 END) AS 주의,
         SUM(CASE WHEN 결과 = '불량' THEN 1 ELSE 0 END) AS 불량
  FROM 점검기록`);
const ▢ = 케이스.rows[0];
console.log(` · 전체 ${▢.전체} · 정상 ${▢.정상} · 주의 ${▢.주의} · 불량 ${▢.불량}`);
```

출력

- 전체 15 · 정상 9 · 주의 4 · 불량 2

```
console.log("FILTER 와 CASE 가 같은 답인가:", □.정상 === ▢.정상 && □.불량 === ▢.불량);
```

출력

- FILTER 와 CASE 가 같은 답인가: true

LEFT JOIN 과 합치면 점검 0건인 설비도 0/0/0 으로 자리를 지킵니다.

```
const 설비별판정 = await db.query(`
  SELECT s.이름, COUNT(p.점검번호) AS 전체,
    COUNT(*) FILTER (WHERE p.결과 = '정상') AS 정상,
    COUNT(*) FILTER (WHERE p.결과 = '주의') AS 주의,
    COUNT(*) FILTER (WHERE p.결과 = '불량') AS 불량
  FROM 설비 s LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY s.설비번호, s.이름 ORDER BY s.설비번호`);
for (const 줄 of 설비별판정.rows)
  console.log(` ${줄.이름} · 전체 ${줄.전체} · 정상 ${줄.정상} · 주의 ${줄.주의} ·
  불량 ${줄.불량}`);
```

```
출력      · 컨베이어 1호 · 전체 4 · 정상 3 · 주의 1 · 불량 0
          · 프레스 1호 · 전체 3 · 정상 1 · 주의 1 · 불량 1
          · 용접로봇 1호 · 전체 5 · 정상 3 · 주의 1 · 불량 1
          · 검사기 1호 · 전체 3 · 정상 2 · 주의 1 · 불량 0
          · 포장기 1호 · 전체 0 · 정상 0 · 주의 0 · 불량 0
```

05

MySQL 은 여기가 다릅니다

· MySQL 8.0 에는 FILTER 절이 없습니다. CASE 로 써야 합니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

ROLLUP — 소계와 총계를 한 번에

섹션 8

“라인별 건수 + 맨 아래 전체 합계” 를 쿼리 두 번 던져 붙이지 않아도 됩니다. GROUP BY ROLLUP(칸) 이라 쓰면 총계 줄이 한 줄 더 붙습니다. ★ 있다는 것만 알아 두세요.

```
const 롤업 = await db.query(`
  SELECT s.라인코드, GROUPING(s.라인코드) AS 합계줄, COUNT(*) AS 건수
  FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY ROLLUP(s.라인코드) ORDER BY GROUPING(s.라인코드), s.라인코드`);
for (const 줄 of 롤업.rows) console.log(` 라인코드 ${줄.라인코드} · 합계줄 ${줄.
합계줄} · ${줄.건수}건`);
```

```
출력      · 라인코드 A · 합계줄 0 · 7건
          · 라인코드 B · 합계줄 0 · 8건
          · 라인코드 null · 합계줄 1 · 15건
```

★ 합계 줄은 라인코드가 NULL 로 옵니다. 원래 데이터에 라인코드가 NULL 인 줄이 섞여 있으면 구분이 안 되므로 GROUPING(칸) 을 같이 뱉습니다. 1 이면 합계 줄, 0 이면 진짜 데이터입니다.

★★ 0 이 나올 때 · NULL 이 나올 때 · 아무것도 안 나올 때

섹션 9

조건에 맞는 줄이 하나도 없으면 무엇이 돌아옵니까. 함수마다 다릅니다.

```
const 없는결과 = await db.query(`
  SELECT COUNT(*) AS 건수, SUM(점수) AS 합, AVG(점수) AS 평균, MAX(점수) AS 최고
  FROM 점검기록 WHERE 결과 = '없는결과`);
console.log("돌아온 줄 수:", 없는결과.rows.length);
```

출력 돌아온 줄 수: 1

```
const ㅅ = 없는결과.rows[0];
console.log(`· COUNT(*) = ${ㅅ.건수} · SUM = ${ㅅ.합} · AVG = ${ㅅ.평균} · MAX =
${ㅅ.최고}`);
```

출력 · COUNT(*) = 0 · SUM = null · AVG = null · MAX = null

★★ COUNT 만 0 이고 나머지는 전부 NULL 입니다. COUNT 는 “몇 개냐” 니까 없으면 0 이 맞고, SUM 은 “더한 값”인데 더할 게 없으면 0 이 아니라 “값이 없음”입니다. 매출 0원과 매출 데이터가 아예 없는 것은 다른 이야기입니다. Postgres 는 그걸 구분합니다.

```
console.log("null + 1 을 하면:", ㅅ.합 + 1);
```

출력 null + 1 을 하면: 1

```
console.log("null 을 Number() 하면:", Number(ㅅ.합));
```

출력 null 을 Number() 하면: 0

★★★ JS 는 null 을 0 취급해서 조용히 넘어갑니다. 이게 더 무섭습니다. 에러가 안 나니까 “합계 1” 같은 엉뚱한 숫자가 그대로 보고서에 실립니다. SQL 에서 COALESCE 로 막으세요.

```
const 고친합 = await db.query(
  `SELECT COALESCE(SUM(점수), 0) AS 합 FROM 점검기록 WHERE 결과 = '없는결과`);
console.log("COALESCE(SUM(점수), 0):", 고친합.rows[0].합);
```

출력 COALESCE(SUM(점수), 0): 0

★★ 더 헛갈리는 것: GROUP BY 를 붙이면 줄이 아예 안 나옵니다

위에서는 한 줄은 나왔습니다. 묶을 값이 없으니 만들 묶음이 없어 그 한 줄도 사라집니다.

```
const 묶으면 = await db.query(
  `SELECT 설비번호, COUNT(*) AS 건수 FROM 점검기록 WHERE 결과 = '없는결과' GROUP BY
설비번호`);
console.log("GROUP BY 를 붙이면 줄 수:", 묶으면.rows.length);
```

출력 GROUP BY 를 붙이면 줄 수: 0

★★★ 그래서 “건수 0 인 것을 찾아라” 는 GROUP BY 만으로는 절대 못 합니다. 0 인 줄은 결과에 없습니다. 기준이 되는 표에서 시작해 LEFT JOIN 으로 이어야 0 이 보입니다. (개념02)

```
const 라인별 = await db.query(`
  SELECT l.라인코드, l.이름, COUNT(p.점검번호) AS 점검수
  FROM 라인 l LEFT JOIN 설비 s ON l.라인코드 = s.라인코드
           LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY l.라인코드, l.이름 ORDER BY l.라인코드`);
for (const 줄 of 라인별.rows) console.log(` · ${줄.이름} · ${줄.점검수}건`);
```

```
출력      · 조립1라인 · 7건
          · 가공2라인 · 8건
          · 포장3라인 · 0건
          · 신설4라인 · 0건
```

★ 포장3라인은 설비는 있는데(포장기 1호) 점검이 0건이고, 신설4라인은 설비 자체가 없어 0건입니다. 둘 다 0 으로 보이는 게 맞습니다. 구분하려면 COUNT(DISTINCT s.설비번호) 를 같이 뽑으세요.

정리

집계 함수와 NULL

함수	NULL 인 줄	맞는 줄이 0개	JS 로 오는 타입
COUNT(*)	센다	0	number

COUNT(칸) / DISTINCT 안 센다 0 number SUM(칸) / MIN/MAX(칸) 건너뛰다 NULL ★ number
AVG(칸) 건너뛰다 NULL ★ string ★★★ * COUNT 는 BIGINT. 작으면 number, 크면 bigint →
JSON.stringify 가 터짐. Number() 로 감싸기

COUNT 세 가지 (점검기록 15줄 기준)

COUNT(*) 15 (줄 수, 빈 짝도 1 로 셈 ★) · COUNT(점수) 13 (NULL 인 2건 제외) COUNT(DISTINCT
점수) 10 (겹치는 값을 접음) → LEFT JOIN 위에서는 반드시 COUNT(오른쪽칸)

WHERE 와 HAVING

WHERE HAVING	
시점	GROUP BY 앞 GROUP BY 뒤

거르는 것 줄 묶음 집계 함수 못 씬 (42803) 씬 SELECT 별칭 못 씬 (42703) 못 씬 (42703) 실행 순서:
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT ★
ORDER BY 와 GROUP BY 에서는 별칭을 쓸 수 있습니다

이 파일에서 만난 SQLSTATE

42803 GROUP BY 에 없는 칸 / WHERE 에 집계 함수
42703 없는 칸 (별칭을 WHERE·HAVING 에서 쓴 경우 포함)
42883 타입에 맞는 함수가 없음 (ROUND(double precision, int))

직접 해 볼 것

▣ 직접 해보기 1

담당자별 점검 건수

작업자 표를 기준으로 LEFT JOIN 해서 사번·이름·담당건수를 뽑아 보세요. 정신입(105)이 0건으로 나오면 맞습니다. COUNT(*) 로 하면 1건이 됩니다. 왜인지 설명해 보세요.

▣ 직접 해보기 2

점수가 비어 있는 점검이 있는 설비 찾기

설비별로 COUNT(*) 와 COUNT(점수) 를 같이 뽑고, 두 값이 다른 설비만 남겨 보세요. 힌트: HAVING COUNT(*) <> COUNT(점수)

▣ 직접 해보기 3

라인별 요약표

라인 표를 기준으로 라인이름·설비수·점검수·평균점수를 한 줄로 뽑아 보세요. 설비수는 COUNT(DISTINCT s.설비번호) 입니다. DISTINCT 를 빼고도 돌려서 왜 필요한지 보세요. 신설4라인의 평균점수가 무엇으로 나오는지 확인하고 COALESCE 로 0 으로 바꿔 보세요.

▣ 직접 해보기 4

WHERE 와 HAVING 자리 바꾸기

“불량이 1건 이상인 설비” 를 ① WHERE 결과 = ‘불량’ 로 거르고 HAVING COUNT(*) >= 1, ② WHERE 없이 HAVING COUNT(*) FILTER (WHERE 결과 = ‘불량’) >= 1 두 가지로 짜 보세요. 결과가 같은지 보고, ①로는 못 하는 것(정상 건수도 같이 보기)이 무엇인지 적어 보세요.

▣ 직접 해보기 5

월별 점검 건수

TO_CHAR(점검일, 'YYYY-MM') 으로 묶어 월별 건수를 뽑아 보세요. 점검이 하나도 없는 달이 결과에 나오는지 보세요. (안 나옵니다. 섹션 9 와 같은 이유입니다)

▹ 직접 해보기

6

동(棟)별 롤업

라인.동 으로 묶어 ROLLUP 으로 동별 소계와 전체 합계를 뽑고, 합계 줄을 GROUPING 으로 골라내 "합계" 로 바꿔 찍어 보세요. 힌트: CASE WHEN GROUPING(l.동) = 1 THEN '합계' ...

▹ 직접 해보기

7

평균 점수로 등급 매기기

설비별 평균 점수를 뽑고, JS 쪽에서 Number() 로 바꿔 85 이상 '양호', 70 이상 '보통', 그 아래는 '점검필요' 로 찍어 보세요. Number() 를 빼면 어떻게 되는지도 보고 오세요.

자주 하는 실수

이런 실수를 합니다

LEFT JOIN 해 놓고 COUNT(*) 로 세기

설비 LEFT JOIN 점검기록 에서 COUNT(*) 를 쓰면 점검 0건인 설비가 1건으로 나옵니다. 빈 짝도 "줄" 이기 때문입니다. 오른쪽 표의 칸을 세세요. COUNT(p.점검번호).

이런 실수를 합니다

평균을 SUM / COUNT(*) 로 손수 계산하기

AVG 는 NULL 을 뺀 개수로 나누는데 COUNT(*) 는 NULL 인 줄까지 셉니다. 이 데이터에서는 77.69 여야 할 평균이 67.33 이 되고, 캐스팅까지 빠뜨리면 정수 나눗셈으로 67 이 됩니다.

이런 실수를 합니다

집계 결과의 JS 타입을 지레짐작하기

AVG 는 NUMERIC 이라 문자열로 옵니다. 평균 + 1 은 78.7 이 아니라 "77.71" 입니다. COUNT 는 BIGINT 라 줄이 적을 땐 number, 2^53 을 넘으면 bigint 로 바뀌어 JSON.stringify 가 터집니다. MIN/MAX 만 늘 number 입니다. 셋 다 Number() 로 감싸 두면 고민할 일이 없습니다.

이런 실수를 합니다

SUM 이 NULL 인데 0 으로 믿기

맞는 줄이 없으면 SUM·AVG·MIN·MAX 는 0 이 아니라 NULL 입니다. JS 는 null 을 0 으로 슬쩍 바꿔 계산하므로 에러도 안 납니다. COALESCE(SUM(칸), 0) 으로 SQL 에서 막으세요.

이런 실수를 합니다

“건수 0”을 GROUP BY로 찾으려 하기

0인 묶음은 결과에 아예 없습니다. 없는 줄을 걸러낼 수는 없습니다. 기준이 되는 표(설비·라인·작업자)에서 시작해 LEFT JOIN으로 이어야 0이 보입니다.

이런 실수를 합니다

SELECT에 올린 별칭을 WHERE·HAVING에서 쓰기

SELECT는 WHERE·HAVING보다 나중에 돕니다. 별칭이 아직 없어 42703이 납니다. HAVING에는 COUNT(*)처럼 식을 다시 쓰세요. ORDER BY에서는 별칭이 됩니다.

이런 실수를 합니다

GROUP BY에 칸을 빠뜨리기

SELECT에 올린 칸은 GROUP BY에 있거나 집계로 감싸져 있어야 합니다 (42803). 기본키로 묶었을 때만 나머지 칸을 봐줍니다. ★ MySQL은 이걸 안 막고 아무 값이나 하나 줍니다. 그래서 틀린 줄도 모릅니다 → 09단원

```
await db.close();
```

쿼리 안의 쿼리

```
RUN node 개념04_쿼리_안의_쿼리.js
```

개념03 까지 JOIN 과 GROUP BY 로 나뉜 표를 다시 이어 봤습니다. 그런데 “평균보다 점수가 낮은 점검을 뽑아 주세요” 는 그것만으로 안 됩니다. 평균을 **먼저** 구해 놓아야 “평균보다 낮다” 를 따질 수 있으니까요. 쿼리 하나 안에 쿼리가 또 필요합니다. 이것을 서브쿼리라고 합니다.

이 파일에서 다루는 것: · 값 하나를 돌려주는 서브쿼리(스칼라), 목록을 돌려주는 서브쿼리(IN) · ★★★
NOT IN 에 NULL 이 섞이면 결과가 통째로 0건이 되는 사고 · EXISTS / NOT EXISTS, 상관 서브쿼리가
느려지는 지점, FROM 절 서브쿼리(인라인 뷰) · WITH(CTE) 와 WITH RECURSIVE

```
import { PGlite } from "@electric-sql/pglite";  
  
const db = await PGlite.create();
```

05

```
await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    동        TEXT NOT NULL
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드),
    도입년도 INT NOT NULL
  );

  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드)
  );

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호),
    점검일   DATE NOT NULL,
    결과     TEXT NOT NULL,
    점수     INT,
    담당사번 INT REFERENCES 작업자(사번)
  );

  INSERT INTO 라인 VALUES
    ('A', '조립1라인', '1동'),
    ('B', '가공2라인', '1동'),
    ('C', '포장3라인', '2동'),
    ('D', '신설4라인', '2동');

  INSERT INTO 설비 VALUES
```

```
(1, '컨베이어 1호', 'A', 2015),
(2, '프레스 1호', 'A', 2019),
(3, '용접로봇 1호', 'B', 2021),
(4, '검사기 1호', 'B', 2018),
(5, '포장기 1호', 'C', 2022);
```

```
INSERT INTO 작업자 VALUES
```

```
(101, '김반장', 'A'),
(102, '이기사', 'B'),
(103, '박주임', 'B'),
(104, '최사원', 'C'),
(105, '정신입', NULL);
```

```
INSERT INTO 점검기록 VALUES
```

```
( 1, 1, '2024-01-05', '정상', 92, 101),
( 2, 1, '2024-02-05', '정상', 88, 101),
( 3, 1, '2024-03-06', '주의', 71, 102),
( 4, 1, '2024-04-05', '정상', 90, 101),
( 5, 2, '2024-01-12', '주의', 65, 102),
( 6, 2, '2024-02-14', '불량', 48, 103),
( 7, 2, '2024-03-15', '정상', 88, 101),
( 8, 3, '2024-01-20', '정상', 95, 103),
( 9, 3, '2024-02-20', '정상', NULL, 103),
(10, 3, '2024-03-21', '주의', 74, 102),
(11, 3, '2024-04-22', '정상', 90, NULL),
(12, 3, '2024-05-23', '불량', 52, 103),
(13, 4, '2024-01-28', '정상', NULL, 104),
(14, 4, '2024-03-29', '정상', 88, 104),
(15, 4, '2024-05-30', '주의', 69, NULL);
`);
```

스칼라 서브쿼리: 값 하나를 돌려줍니다

섹션 1

스칼라(scalar) = 값 하나. 줄 하나 × 칸 하나. 값이 올 자리면 어디든 괄호로 싸서 넣습니다.

```
const 평균 = await db.query(`SELECT ROUND(AVG(점수), 2) AS 평균점수 FROM 점검기록`);
console.log("전체 평균 점수:", 평균.rows[0].평균점수);
```

출력 전체 평균 점수: 77.69

★ AVG(정수칸) 은 NUMERIC 이라 자바스크립트에는 문자열로 옵니다. Number(...) 를 거치세요.

```
console.log("자바스크립트에서 본 타입:", typeof 평균.rows[0].평균점수);
```

출력 자바스크립트에서 본 타입: string

이 평균을 WHERE 안에 통째로 끼워 넣습니다. 안쪽이 먼저 한 번 실행되어 값 하나가 되고, 바깥은 그 값과 비교만 합니다.

```
const 평균미만 = await db.query(`
  SELECT 점검번호, 설비번호, 점수 FROM 점검기록
  WHERE 점수 < (SELECT AVG(점수) FROM 점검기록)
  ORDER BY 점검번호`);
for (const 줄 of 평균미만.rows) console.log(` · 점검${줄.점검번호} · 설비${줄.설비번호} · ${줄.점수}점`);
```

```
출력      · 점검3 · 설비1 · 71점
          · 점검5 · 설비2 · 65점
          · 점검6 · 설비2 · 48점
          · 점검10 · 설비3 · 74점
          · 점검12 · 설비3 · 52점
          · 점검15 · 설비4 · 69점
```

★ 점수가 NULL 인 점검 9·13 은 안 나왔습니다. `NULL < 77.69` 는 거짓이 아니라 UNKNOWN 이고, WHERE 는 참만 통과시킵니다. (02단원 NULL 3값 논리. 섹션 3 에서 이게 사고를 냅니다)

```
console.log("평균 미만 줄 수:", 평균미만.rows.length);
```

```
출력      평균 미만 줄 수: 6
```

SELECT 절에도 씁니다. 안쪽에서 바깥 줄(s)을 참조하면 **상관 서브쿼리**입니다.

```
const 설비별 = await db.query(`
  SELECT s.이름,
    (SELECT COUNT(*) FROM 점검기록 p WHERE p.설비번호 = s.설비번호) AS 점검수,
    (SELECT MAX(점수) FROM 점검기록 p WHERE p.설비번호 = s.설비번호) AS 최고점
  FROM 설비 s ORDER BY s.설비번호`);
for (const 줄 of 설비별.rows) console.log(` · ${줄.이름} · 점검 ${줄.점검수}건 · 최고 ${줄.최고점}`);
```

```
출력      · 컨베이어 1호 · 점검 4건 · 최고 92
          · 프레스 1호 · 점검 3건 · 최고 88
          · 용접로봇 1호 · 점검 5건 · 최고 95
          · 검사기 1호 · 점검 3건 · 최고 88
          · 포장기 1호 · 점검 0건 · 최고 null
```

★ 포장기 1호는 점검기록이 0줄입니다. COUNT 는 0, MAX 는 null 입니다. 이 차이를 꼭 보세요. ★ COUNT(*) 는 BIGINT 입니다. PGlite 는 안전한 정수 범위면 number, 넘으면 bigint 로 줍니다. bigint 는 `JSON.stringify` 에서 터지니 `Number(...)` 로 감싸는 습관을 들이세요.

```
const 타입확인 = await db.query(`SELECT COUNT(*) AS 작은수, 9007199254740993::BIGINT
AS 큰수 FROM 점검기록`);
console.log("작은 COUNT:", typeof 타입확인.rows[0].작은수, "· 아주 큰 BIGINT:",
typeof 타입확인.rows[0].큰수);
```

출력 작은 COUNT: number · 아주 큰 BIGINT: bigint

★ 스칼라 자리에 두 줄 이상이 오면 에러입니다.

```
try {
  await db.query(`SELECT 이름, (SELECT 라인코드 FROM 설비) FROM 라인`);
} catch (e) {
  console.log("에러코드:", e.code, "·", e.message);
}
```

출력 에러코드: 21000 · more than one row returned by a subquery used as an expression

★ 그런데 0줄이면 에러가 아니라 조용히 NULL 이 됩니다. 이게 섹션 3 사고의 씨앗입니다.

```
const 영줄 = await db.query(`SELECT (SELECT 점수 FROM 점검기록 WHERE 설비번호 = 5) AS
값`);
console.log("0줄짜리 스칼라 서브쿼리의 값:", 영줄.rows[0].값);
```

출력 0줄짜리 스칼라 서브쿼리의 값: null

IN 서브쿼리: 목록을 돌려줍니다

섹션 2

스칼라는 값 하나였습니다. IN 뒤의 서브쿼리는 **줄 여러 개**를 돌려줍니다. 칸은 하나여야 합니다.

```
const 불량설비 = await db.query(`
  SELECT 설비번호, 이름 FROM 설비
  WHERE 설비번호 IN (SELECT 설비번호 FROM 점검기록 WHERE 결과 = '불량')
  ORDER BY 설비번호`);
for (const 줄 of 불량설비.rows) console.log(`· ${줄.설비번호} · ${줄.이름}`);
```

출력 · 2 · 프레스 1호
 · 3 · 용접로봇 1호

안쪽이 무엇을 돌려줬는지 직접 봅시다.

```
const 목록 = await db.query(`SELECT DISTINCT 설비번호 FROM 점검기록 WHERE 결과 =
'불량' ORDER BY 설비번호`);
console.log("서브쿼리가 돌려준 목록:", 목록.rows.map((줄) => 줄.설비번호).join(",
"));
```

출력 서버쿼리가 돌려준 목록: 2, 3

값 목록이든 서버쿼리든 IN 이 하는 일은 똑같습니다 — “이 중에 있나?”

```
const 값목록 = await db.query(`SELECT 설비번호 FROM 설비 WHERE 설비번호 IN (2, 3)
ORDER BY 설비번호`);
console.log("IN (2, 3) 과 결과가 같은가:",
  JSON.stringify(값목록.rows.map((줄) => 줄.설비번호)) ===
  JSON.stringify(불량설비.rows.map((줄) => 줄.설비번호)));
```

출력 IN (2, 3) 과 결과가 같은가: true

★ 차이는 “언제 정해지느냐” 뿐입니다 — 값 목록은 쿼리를 쓸 때, 서버쿼리는 실행할 때.

★★★ NOT IN 에 NULL 이 섞이면 결과가 0건이 됩니다

섹션 3

이 파일에서 제일 중요한 곳이고, 실무에서 사고가 제일 많이 나는 자리입니다. 하고 싶은 것: “점검을 한 번도
담당한 적 없는 작업자”. 눈으로 세면 정신입(105) 한 명입니다.

```
const 틀린것 = await db.query(`
  SELECT 사번, 이름 FROM 작업자
  WHERE 사번 NOT IN (SELECT 담당사번 FROM 점검기록)
  ORDER BY 사번`);
console.log("NOT IN 으로 찾은 줄 수:", 틀린것.rows.length);
```

출력 NOT IN 으로 찾은 줄 수: 0

0건입니다. 애러도 경고도 없습니다. 그냥 조용히 틀립니다. 안쪽이 돌려준 목록을 봅시다.

```
const 담당 = await db.query(`SELECT DISTINCT 담당사번 FROM 점검기록 ORDER BY 담당사번
`);
console.log("담당사번 목록:", 담당.rows.map((줄) => String(줄.담당사번)).join(", "));
```

출력 담당사번 목록: 101, 102, 103, 104, null

★ 끝에 null 이 있습니다. 점검 11 과 15 는 담당자가 안 정해졌습니다. 105 NOT IN (101, 102, 103, 104,
NULL) 을 SQL 은 이렇게 품니다.

105 <> 101 AND 105 <> 102 AND 105 <> 103 AND 105 <> 104 AND 105 <> NULL

참	참	참	참	???
---	---	---	---	-----

마지막 칸이 문제입니다. 실제로 찍어 봅시다.


```
const 논리 = await db.query(`
  SELECT (105 <> 101) AS 값과비교, (105 <> NULL) AS 널과비교,
         (true AND NULL) AS 참그리고널, (false AND NULL) AS 거짓그리고널`);
const L = 논리.rows[0];
console.log(`105 <> 101 → ${L.값과비교} · 105 <> NULL → ${L.널과비교}`);
```

출력 105 <> 101 → true · 105 <> NULL → null

```
console.log(`true AND NULL → ${L.참그리고널} · false AND NULL → ${L.
거짓그리고널}`);
```

출력 true AND NULL → null · false AND NULL → false

정리하면 · 105 <> NULL 은 참도 거짓도 아닌 **UNKNOWN** 입니다. NULL 은 “모름” 이니까요. · 참 AND UNKNOWN 은 **UNKNOWN** 입니다. 앞이 다 참이어도 소용없습니다. · WHERE 는 **참만** 통과시킵니다. UNKNOWN 은 버립니다. · 그리고 105 만이 아니라 **모든 줄이 똑같이** 됩니다. → 그래서 0건.

```
const 직접 = await db.query(`
  SELECT 105 NOT IN (SELECT 담당사번 FROM 점검기록) AS 백오번,
         104 IN     (SELECT 담당사번 FROM 점검기록) AS 백사번,
         999 IN     (SELECT 담당사번 FROM 점검기록) AS 구구구`);
const C = 직접.rows[0];
console.log(`105 NOT IN → ${C.백오번} · 104 IN → ${C.백사번} · 999 IN → ${C.
구구구}`);
```

출력 105 NOT IN → null · 104 IN → true · 999 IN → null

★ IN 은 “하나라도 참이면 참” 이라 104 는 무사히 true 가 나옵니다. 하지만 목록에 없는 999 는 **false 가 아니라 null** 입니다 (거짓 OR UNKNOWN = UNKNOWN). 결국 IN 도 NULL 이 섞이면 “없다” 를 딱 잘라 말하지 못합니다. 다만 WHERE 에서 IN 은 “찾은 것” 은 제대로 통과시키니 사고가 덜 보일 뿐입니다.

고치는 법 1 · NOT EXISTS ← ★ 이걸 쓰세요 (줄이 있냐 없냐만 보니 NULL 이 낄 틈이 없습니다)

```
const 방법1 = await db.query(`
  SELECT w.사번, w.이름 FROM 작업자 w
  WHERE NOT EXISTS (SELECT 1 FROM 점검기록 p WHERE p.담당사번 = w.사번)
  ORDER BY w.사번`);
for (const 줄 of 방법1.rows) console.log(`· ${줄.사번} · ${줄.이름}`);
```

출력 · 105 · 정신입

고치는 법 2 · 서브쿼리에서 NULL 을 먼저 걸어내기

```
const 방법2 = await db.query(`
  SELECT 사번, 이름 FROM 작업자
  WHERE 사번 NOT IN (SELECT 담당사번 FROM 점검기록 WHERE 담당사번 IS NOT NULL)
  ORDER BY 사번`);
```

고치는 법 3 · LEFT JOIN 하고 짝이 없는 줄만 (개념02 의 관용구)

```
const 방법3 = await db.query(`
  SELECT w.사번, w.이름 FROM 작업자 w LEFT JOIN 점검기록 p ON p.담당사번 = w.사번
  WHERE p.점검번호 IS NULL ORDER BY w.사번`);

console.log("세 방법의 결과가 모두 같은가:",
  JSON.stringify(방법1.rows) === JSON.stringify(방법2.rows) &&
  JSON.stringify(방법2.rows) === JSON.stringify(방법3.rows));
```

출력 세 방법의 결과가 모두 같은가: true

★★★ 결론: NOT IN 뒤에 서브쿼리를 쓰지 마세요. NOT EXISTS 를 쓰세요. 지금 그 칸에 NULL 이 없더라도, 내일 누가 NULL 을 하나 넣으면 그날부터 0건입니다. NOT IN 은 (1, 2, 3) 처럼 내가 손으로 적은 값 목록에만 쓰세요. (이 함정은 표준 SQL 의 규칙이라 MySQL 에서도 똑같이 일어납니다)

EXISTS 와 IN, 무엇을 언제 쓰나

섹션 4

EXISTS 는 거의 항상 상관 서브쿼리와 같이 씁니다. 안쪽에서 바깥 줄(s)을 참조해서 “이 줄에 딸린 게 있나?” 를 묻습니다.

```
const 이그1 = await db.query(`
  SELECT s.설비번호, s.이름 FROM 설비 s
  WHERE EXISTS (SELECT 1 FROM 점검기록 p WHERE p.설비번호 = s.설비번호 AND p.결과 =
  '불량')
  ORDER BY s.설비번호`);
for (const 줄 of 이그1.rows) console.log(`· ${줄.설비번호} · ${줄.이름}`);
```

출력 · 2 · 프레스 1호
 · 3 · 용접로봇 1호

★ SELECT 1 인 이유: EXISTS 는 무엇을 고르는지 안 봅니다. 줄이 있냐만 봅니다.

```
const 이그2 = await db.query(`
  SELECT s.설비번호, s.이름 FROM 설비 s
  WHERE EXISTS (SELECT * FROM 점검기록 p WHERE p.설비번호 = s.설비번호 AND p.결과 =
'불량')
  ORDER BY s.설비번호`);
console.log("SELECT 1 과 SELECT * 의 결과가 같은가:", JSON.stringify(이그1.rows) ===
JSON.stringify(이그2.rows));
```

출력 SELECT 1 과 SELECT * 의 결과가 같은가: true

같은 질문을 IN 으로도 써 봅니다.

```
const 인판 = await db.query(`
  SELECT 설비번호, 이름 FROM 설비
  WHERE 설비번호 IN (SELECT 설비번호 FROM 점검기록 WHERE 결과 = '불량')
  ORDER BY 설비번호`);
console.log("IN 과 EXISTS 의 결과가 같은가:", JSON.stringify(인판.rows) ===
JSON.stringify(이그1.rows));
```

출력 IN 과 EXISTS 의 결과가 같은가: true

언제 무엇을 쓰나

· NULL 이 있을 수 있다 → 무조건 EXISTS / NOT EXISTS · 목록이 고정된 짧은 값이다 → IN (1, 2, 3)
이 읽기 쉽습니다 · 안쪽에 바깥 줄을 참조할 게 없다 → IN 이 문장이 짧아집니다 ★ 요즘 Postgres 는 IN 과
EXISTS 를 비슷하게 최적화합니다. “어떻게” 는 06단원에서 봅니다.

상관 서브쿼리는 줄마다 다시 실행됩니다

섹션 5

상관 서브쿼리 = 안쪽에 바깥 줄을 참조하는 것. 바깥이 1000줄이면 안쪽이 **1000번** 돌 수 있습니다.
5줄짜리 표에서는 안 느껴집니다. 큰 표를 따로 만들어서 재 봅니다.

```
await db.exec(`
  CREATE TABLE 큰설비 AS
  SELECT g AS 설비번호, '설비' || g AS 이름 FROM generate_series(1, 1000) AS g;
  CREATE TABLE 큰점검 AS
  SELECT g AS 점검번호, (g % 1000) + 1 AS 설비번호, (g % 100) AS 점수
  FROM generate_series(1, 20000) AS g;
`);

const 상관쿼리 = `
  SELECT s.설비번호, (SELECT COUNT(*) FROM 큰점검 p WHERE p.설비번호 = s.설비번호) AS
  건수
```

```
FROM 큰설비 s ORDER BY s.설비번호`;
const 조인쿼리 = `
  SELECT s.설비번호, COUNT(p.점검번호) AS 건수
  FROM 큰설비 s LEFT JOIN 큰점검 p ON p.설비번호 = s.설비번호
  GROUP BY s.설비번호 ORDER BY s.설비번호`;
```

★ 먼저 두 방식이 **같은 답**을 내는지부터 확인합니다. 빠른 오답은 의미가 없습니다.

```
const 상관결과 = await db.query(상관쿼리);
const 조인결과 = await db.query(조인쿼리);
console.log("두 방식의 답이 같은가:", JSON.stringify(상관결과.rows) ===
  JSON.stringify(조인결과.rows));
```

출력 두 방식의 답이 같은가: true

시간은 기계마다 다릅니다. 한 번만 재면 튀니까 5번 재서 **중앙값**을 씁니다.

```
const 중앙값 = (목록) => [...목록].sort((ㄱ, ㄴ) => ㄱ - ㄴ)[Math.floor(목록.length /
2)];
async function 재기(sql, 횟수 = 5) {
  const 기록 = [];
  for (let i = 0; i < 횟수; i++) {
    const 시작 = performance.now();
    await db.query(sql);
    기록.push(performance.now() - 시작);
  }
  return 중앙값(기록);
}

const 상관ms = await 재기(상관쿼리);
const 조인ms = await 재기(조인쿼리);
console.log(`상관 서버쿼리: ${상관ms.toFixed(1)} ms`);
```

출력 상관 서버쿼리: 1705.1 ms
값이 다를 수 있음

```
console.log(`JOIN + GROUP BY: ${조인ms.toFixed(1)} ms`);
```

출력 JOIN + GROUP BY: 13.1 ms
값이 다를 수 있음

★ 판정은 시간이 아니라 **비교 결과**로 남깁니다. 시간은 기계마다 다르니까요.

```
console.log("상관 서버쿼리가 JOIN 보다 10배 넘게 느린가:", 상관ms > 조인ms * 10);
```

출력 상관 서버쿼리가 JOIN 보다 10배 넘게 느린가: true

설비 1000줄 × 점검 20000줄. 상관 쪽은 설비 한 줄마다 점검을 훑고, 조인 쪽은 한 번에 묶습니다. ★ “왜 그런가” 와 “색인을 걸면 어떻게 달라지는가” 는 06단원(색인과 실행계획) 것입니다. 여기서는 “줄마다 다시 도는 모양이면 의심하라” 만 기억하세요.

FROM 절 서브쿼리 (인라인 뷰)

섹션 6

서브쿼리를 표가 올 자리에 놓을 수도 있습니다. 임시로 만든 표처럼 쓰는 것으로, “집계한 결과를 다시 집계하거나 다시 거를 때” 필요합니다. 예를 들어 “설비 한 대당 점검이 평균 몇 건인가” 는 집계의 집계인데, 그냥 겹쳐 쓰면 에러가 납니다.

```
try {
  await db.query(`SELECT AVG(COUNT(*)) FROM 점검기록 GROUP BY 설비번호`);
} catch (e) {
  console.log("에러코드:", e.code, ".", e.message);
}
```

출력 에러코드: 42803 · aggregate function calls cannot be nested

안쪽에서 한 번 집계해 표로 만들고, 바깥에서 그 표를 다시 집계합니다.

```
const 집계의회계 = await db.query(`
  SELECT ROUND(AVG(집계.건수), 2) AS 설비당평균
  FROM (SELECT 설비번호, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 설비번호) AS 집계`);
console.log("설비 한 대당 평균 점검 건수:", 집계의회계.rows[0].설비당평균);
```

출력 설비 한 대당 평균 점검 건수: 3.75

★ 15건 / 4대 = 3.75. 4대인 이유는 점검기록이 0건인 포장기 1호가 GROUP BY 에 안 나와서입니다.

인라인 뷰를 JOIN 에 그대로 끼워 넣을 수도 있습니다.

```
const 인라인뷰 = await db.query(`
  SELECT s.이름, 집계.건수, 집계.평균 FROM 설비 s
  JOIN (SELECT 설비번호, COUNT(*) AS 건수, ROUND(AVG(점수), 1) AS 평균
        FROM 점검기록 GROUP BY 설비번호) AS 집계 ON 집계.설비번호 = s.설비번호
  ORDER BY s.설비번호`);
for (const 줄 of 인라인뷰.rows) console.log(`· ${줄.이름} · ${줄.건수}건 · 평균 ${줄.평균}`);
```

출력 · 컨베이어 1호 · 4건 · 평균 85.3
 · 프레스 1호 · 3건 · 평균 67.0
 · 용접로봇 1호 · 5건 · 평균 77.8
 · 검사기 1호 · 3건 · 평균 78.5

★ FROM 절 서브쿼리에는 AS 이름 으로 별칭을 붙이는 게 정석입니다. 예전 Postgres 는 별칭이 없으면 문법 어려웠습니다. 지금은 어떨까요? 돌려 봅시다.

```
const 별칭없이 = await db.query(`
  SELECT * FROM (SELECT 설비번호, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 설비번호)
  ORDER BY 설비번호`);
console.log("별칭 없이도 되는가 · 줄 수:", 별칭없이.rows.length);
```

출력 별칭 없이도 되는가 · 줄 수: 4

★ PostgreSQL 16 부터는 별칭이 없어도 통과합니다(우리 PGlite 는 18.3). 그래도 **붙이세요**. 칸을 집계. 건수 처럼 가리킬 수 없으면 조인할 때 바로 막힙니다.

MySQL 은 여기가 다릅니다

· MySQL 8.0 은 별칭이 **없으면 에러**입니다 ("Every derived table must have its own alias") ★
자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

★ CTE (WITH): 서브쿼리를 위에서 아래로 펴기

섹션 7

서브쿼리가 두세 겹 중첩되면 읽을 수가 없습니다. **안쪽부터** 읽어야 하니까요. 같은 질문 — “라인별 평균 점수가 전체 평균보다 높은 라인의 설비 목록” — 을 두 가지로 써 봅니다.

① 중첩 서브쿼리 버전 — 괄호 안에서 밖으로 읽어야 합니다

```
const 중첩버전 = await db.query(`
  SELECT s.설비번호, s.이름, s.라인코드 FROM 설비 s
  WHERE s.라인코드 IN (
    SELECT 라인평균.라인코드 FROM (
      SELECT s2.라인코드, AVG(p.점수) AS 평균 FROM 설비 s2
      JOIN 점검기록 p ON p.설비번호 = s2.설비번호 GROUP BY s2.라인코드
    ) AS 라인평균 WHERE 라인평균.평균 > (SELECT AVG(점수) FROM 점검기록)
  )
  ORDER BY s.설비번호`);
```

② CTE 버전 — 위에서 아래로 이름 붙여 가며 읽습니다

```
const 씨티이버전 = await db.query(`
  WITH 라인평균 AS (
    SELECT s.라인코드, AVG(p.점수) AS 평균 FROM 설비 s
    JOIN 점검기록 p ON p.설비번호 = s.설비번호 GROUP BY s.라인코드
  ),
  전체평균 AS (SELECT AVG(점수) AS 값 FROM 점검기록),
  잘하는라인 AS (
    SELECT 라인평균.라인코드 FROM 라인평균, 전체평균 WHERE 라인평균.평균 > 전체평균.
    값
  )
```

```
WHERE s.라인코드 IN (SELECT 라인코드 FROM 잘하는라인)
ORDER BY s.설비번호`);
```

```
for (const 줄 of 씨티이버전.rows) console.log(` ${줄.설비번호} · ${줄.이름} · ${줄.
라인코드}라인`);
```

```
출력      · 3 · 용접로봇 1호 · B라인
          · 4 · 검사기 1호 · B라인
```

```
console.log("중첩 버전과 CTE 버전의 결과가 같은가:", JSON.stringify(중첩버전.rows)
=== JSON.stringify(씨티이버전.rows));
```

```
출력      중첩 버전과 CTE 버전의 결과가 같은가: true
```

답이 맞는지 라인별 평균을 눈으로 확인합니다.

```
const 라인평균보기 = await db.query(`
  SELECT s.라인코드, ROUND(AVG(p.점수), 2) AS 평균 FROM 설비 s
  JOIN 점검기록 p ON p.설비번호 = s.설비번호 GROUP BY s.라인코드 ORDER BY s.라인코드
`);
console.log("라인별 평균:", 라인평균보기.rows.map((줄) => `${줄.라인코드}=${줄.
평균}`).join(" · "));
```

```
출력      라인별 평균: A=77.43 · B=78.00
```

★ 전체 평균 77.69 보다 높은 건 B라인뿐이라 B라인 설비 3·4 가 나왔습니다. C·D라인은 아예 안 보입니다
 — 포장기 1호는 점검기록이 없고 D라인은 설비가 없어 JOIN 에서 빠졌습니다. ★ 읽는 방향이 다릅니다.
 중첩 서브쿼리: 제일 안쪽 괄호부터 밖으로. 괄호를 세면서 읽어야 합니다. · CTE: 위에서 아래로. **라인평균**
 → 전체평균 → 잘하는라인 → 마지막 SELECT.

조각마다 ****이름****이 붙어 있어 이름만 읽어도 무슨 일인지 짐작이 됩니다.

★ CTE 는 쉽표로 잇습니다. 뒤 CTE 는 앞 CTE 를 볼 수 있지만(위 **잘하는라인**) 반대는 안 됩니다.

```
try {
  await db.query(`WITH ㄱ AS (SELECT 값 + 1 AS 값 FROM ㄴ), ㄴ AS (SELECT 1 AS 값)
SELECT * FROM ㄱ`);
} catch (e) {
  console.log("뒤 CTE 를 앞에서 참조 · 에러코드:", e.code, "·", e.message);
}
```

```
출력      뒤 CTE 를 앞에서 참조 · 에러코드: 42P01 · relation "ㄴ" does not exist
```

공장 조직처럼 “위-아래”가 여러 단계로 이어진 데이터는, 몇 단계인지 미리 모르면 JOIN을 몇 번 써야 할지도 모릅니다. 그때 재귀 CTE를 씁니다.

```
await db.exec(`
  CREATE TABLE 조직 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL, 상위번호 INT
  REFERENCES 조직(번호));

  INSERT INTO 조직 VALUES
    ( 1, '공장', NULL), ( 2, '1동', 1), ( 3, '2동', 1),
    ( 4, '조립1라인', 2), ( 5, '가공2라인', 2), ( 6, '포장3라인', 3),
    ( 7, '컨베이어 1호', 4), ( 8, '프레스 1호', 4), ( 9, '용접로봇 1호', 5),
    (10, '검사기 1호', 5), (11, '포장기 1호', 6), (12, '포장기 2호', 6);
`);
```

WITH RECURSIVE는 두 덩어리를 UNION ALL로 잇습니다. 더 붙일 게 없으면 저절로 멈춥니다.

기준줄: 어디서 출발하나 (여기서는 상위가 없는 줄 = 공장) · 재귀줄: 지금까지 나온 줄(훑기)에 딸린 아래 줄을 또 붙입니다

```
const 전개 = await db.query(`
  WITH RECURSIVE 훑기 AS (
    SELECT 번호, 이름, 1 AS 깊이, 이름::TEXT AS 경로
    FROM 조직 WHERE 상위번호 IS NULL
    UNION ALL
    SELECT 자식.번호, 자식.이름, 부모.깊이 + 1, 부모.경로 || ' > ' || 자식.이름
    FROM 조직 자식 JOIN 훑기 부모 ON 자식.상위번호 = 부모.번호
    WHERE 부모.깊이 < 5
  )
  SELECT 깊이, 경로 FROM 훑기 ORDER BY 경로`);
for (const 줄 of 전개.rows) console.log(`${ " ".repeat(줄.깊이 - 1)}· ${줄.경로}`);
```

```
출력      · 공장
          · 공장 > 1동
            · 공장 > 1동 > 가공2라인
              · 공장 > 1동 > 가공2라인 > 검사기 1호
              · 공장 > 1동 > 가공2라인 > 용접로봇 1호
            · 공장 > 1동 > 조립1라인
              · 공장 > 1동 > 조립1라인 > 컨베이어 1호
              · 공장 > 1동 > 조립1라인 > 프레스 1호
          · 공장 > 2동
            · 공장 > 2동 > 포장3라인
              · 공장 > 2동 > 포장3라인 > 포장기 1호
              · 공장 > 2동 > 포장3라인 > 포장기 2호
```


★ **깊이 + 1**로 단계를 세고 **경로 || ' > ' || 이름**으로 지나온 길을 이었습니다. 재귀줄 안에서는 지금까지 만든 **훅기**를 표처럼 씁니다. ★ 기준줄에 **이름::TEXT**로 타입을 못박은 이유: 안 박으면 경로가 짧게 잘릴 수 있습니다.

★★★ 종료 조건이 없으면 영원히 돌립니다. 위 쿼리의 **WHERE 부모.깊이 < 5**가 안전벨트입니다. 데이터에 순환 참조(A의 위가 B, B의 위가 A)가 있으면 안전벨트 없이는 안 멈춥니다. 일부러 순환을 만들어 봅시다.

```
await db.exec(`
  INSERT INTO 조직 VALUES (13, '임시반A', NULL), (14, '임시반B', 13);
  UPDATE 조직 SET 상위번호 = 14 WHERE 번호 = 13;
`);
```

★ Postgres 14 부터 CYCLE 절이 있습니다. 지나온 줄을 다시 만나면 표시하고 멈춥니다. (아래 쿼리에는 깊이 제한이 없습니다. CYCLE 절만으로 멈추는지 보는 겁니다)

```
const 순환 = await db.query(`
  WITH RECURSIVE 훅기 AS (
    SELECT 번호, 이름, 1 AS 깊이 FROM 조직 WHERE 번호 = 13
    UNION ALL
    SELECT 자식.번호, 자식.이름, 부모.깊이 + 1
    FROM 조직 자식 JOIN 훅기 부모 ON 자식.상위번호 = 부모.번호
  ) CYCLE 번호 SET 순환났음 USING 지나온길
  SELECT 이름, 깊이, 순환났음 FROM 훅기 ORDER BY 깊이`);
for (const 줄 of 순환.rows) console.log(`· ${줄.이름} · ${줄.깊이}단계 · 순환났음=${줄.순환났음}`);
```

```
출력      · 임시반A · 1단계 · 순환났음=false
          · 임시반B · 2단계 · 순환났음=false
          · 임시반A · 3단계 · 순환났음=true
```

★ 임시반A를 두 번째로 만난 줄에 **순환났음=true**가 붙고 거기서 멈춥니다. CYCLE 절도 없고 깊이 제한도 없었다면 이 쿼리는 안 끝납니다. 실무에서는 **둘 다** 거세요.

★ UNION은 중복을 지웁니다(정렬·해시 비용이 듭니다). UNION ALL은 그냥 다 붙입니다.

```
const 유니온 = await db.query(`
  SELECT (SELECT COUNT(*) FROM (SELECT 결과 FROM 점검기록 UNION ALL SELECT 결과 FROM
  점검기록) ⌋) AS 올붙임,
  (SELECT COUNT(*) FROM (SELECT 결과 FROM 점검기록 UNION      SELECT 결과 FROM
  점검기록) ⌋) AS 중복제거`);
console.log("UNION ALL:", 유니온.rows[0].올붙임, "· UNION:", 유니온.rows[0].
중복제거);
```

```
출력      UNION ALL: 30 · UNION: 3
```

★ 재귀 CTE에 UNION을 쓰면 중복이 지워져 순환이 멎기도 하지만 기대지 마세요.

★ “CTE 는 느리다” 는 옛날 이야기입니다

섹션 9

PostgreSQL 11 까지 CTE 는 **최적화의 벽**이었습니다. WITH 로 감싸면 무조건 따로 계산해서 임시 결과로 만들어 놓고 썼습니다. 그래서 옛날 자료에는 “CTE 는 느리다” 고 적혀 있습니다. PostgreSQL **12 부터** 바뀌었습니다. 조건이 맞으면 바깥 쿼리에 녹여 넣습니다(인라인). 이제는 직접 지정할 수도 있습니다. **·WITH x AS MATERIALIZED (...) → 무조건 따로 계산해 두기 (옛날 방식) ·WITH x AS NOT MATERIALIZED (...) → 무조건 녹여 넣기**

```
const 굳히기 = await db.query(`
  WITH 집계 AS MATERIALIZED (SELECT 설비번호, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 설비번호)
  SELECT COUNT(*) AS 줄수 FROM 집계`);
const 녹이기 = await db.query(`
  WITH 집계 AS NOT MATERIALIZED (SELECT 설비번호, COUNT(*) AS 건수 FROM 점검기록 GROUP BY 설비번호)
  SELECT COUNT(*) AS 줄수 FROM 집계`);
console.log("MATERIALIZED:", 굳히기.rows[0].줄수, "· NOT MATERIALIZED:", 녹이기.rows[0].줄수);
```

출력 MATERIALIZED: 4 · NOT MATERIALIZED: 4

★ 둘 다 문법이 통과하고 **답은 같습니다**. 달라지는 건 계산하는 방법뿐입니다. PGlite 는 PostgreSQL 18.3 이니 아무것도 안 붙이면 인라인됩니다. ★ 어느 쪽이 언제 빠르니, 어떻게 확인하는지는 06단원 (색인과 실행계획) 것입니다. ★ 지금 기억할 것: **읽기 좋게 CTE 로 쓰세요**. “CTE 라서 느리다” 는 이제 사실이 아닙니다.

정리 — 서브쿼리가 놓이는 다섯 자리

자리	모양	돌려주는 것	언제 쓰나
함정			

스칼라 WHERE 점수 < (SELECT AVG(...)) 값 하나 기준값 하나가 필요할 때 두 줄 오면 에러(21000)

	SELECT 이름, (SELECT COUNT(*)...)		줄마다 곱들일 값	0줄이면 조
IN	WHERE 번호 IN (SELECT ...)	한 칸 목록	"이 중에 있나"	NOT IN +
EXISTS	WHERE EXISTS (SELECT 1 ...)	참/거짓	"달린 게 있나"	상관이라
FROM	FROM (SELECT ...) AS 집계	표	집계를 다시 집계·거를 때	별칭 없으

WITH (CTE) WITH ㄱ AS (...), ㄴ AS (...) 이름 붙인 표 여러 겹을 펴서 읽을 때 뒤 CTE 를 앞에서 못 봄

★★★ 하나만 가져간다면: **NOT IN 뒤에 서브쿼리를 쓰지 말고 NOT EXISTS 를 쓰세요**.

직접 해 볼 것

▫ 직접 해보기 1

설비마다 “마지막 점검일” 을 SELECT 절 스칼라 서브쿼리로 붙여 보세요.

포장기 1호에는 무엇이 나오나요? 왜 그런가요?

▫ 직접 해보기 2

“점검기록이 한 건도 없는 설비” 를 NOT EXISTS 로 찾아보세요(답: 포장기 1호).

같은 것을 NOT IN 으로도 써 보세요. 이번에는 왜 NOT IN 도 잘 되나요? (힌트: 점검기록.설비번호 에 NULL 이 있나요? 위 담당사번과 비교해 보세요)

▫ 직접 해보기 3

UPDATE 점검기록 SET 설비번호 = NULL WHERE 점검번호 = 1; 뒤에

해보기 2 의 NOT IN 버전을 다시 돌려 보세요. NOT EXISTS 버전은 그대로인가요?

▫ 직접 해보기 4

“점검 건수가 설비 평균(3.75건)보다 많은 설비” 를 인라인 뷰로 찾아보세요.

HAVING 만으로는 왜 안 되는지 같이 생각해 보세요.

▫ 직접 해보기 5

“전체 평균보다 점수가 낮은 점검을 담당한 작업자 이름과 그 건수” 를

중첩 서브쿼리로, 또 CTE 로 써 보세요. 어느 쪽이 고치기 쉬운가요?

▫ 직접 해보기 6

조직표에서 “앞(자식이 하나도 없는 줄)만” 골라 보세요.

NOT EXISTS 를 쓰면 재귀 없이도 됩니다. 재귀 버전과 결과를 비교하세요.

▫ 직접 해보기 7

섹션 5 의 큰설비를 3000줄로 늘리면 상관 서브쿼리는 몇 배 느려지나요?

이런 실수를 합니다

★★★ 짝 없는 것을 찾는데 NOT IN 을 썼다

WHERE 사변 NOT IN (SELECT 담당사번 FROM 점검기록) → 0건. 목록에 NULL 이 하나만 섞여도 모든 줄이 UNKNOWN 이 되어 사라집니다. 에러도 경고도 없어서 배포된 뒤에야 압니다. → NOT EXISTS 를 쓰세요.

이런 실수를 합니다

스칼라 서브쿼리가 0줄이라 NULL 이 됐는데 모르고 지나갔다

(SELECT MAX(점수) FROM 점검기록 p WHERE p.설비번호 = s.설비번호) 는 짝이 없으면 에러가 아니라 NULL 입니다. 그대로 더하면 결과가 통째로 NULL 이 됩니다. → COALESCE(..., 0) 로 기본값을 정해 두세요.

이런 실수를 합니다

FROM 절 서브쿼리에 별칭을 안 붙였다

PostgreSQL 16 부터 문법은 통과하지만 집계.건수 로 가리킬 수가 없어서 JOIN 하는 순간 막힙니다. MySQL 8.0 에서는 아예 에러입니다. → 짧게라도 AS 집계 를 항상 붙이세요.

이런 실수를 합니다

스칼라 자리에 두 줄이 올 수 있는 서브쿼리를 넣었다

개발 데이터에서는 우연히 한 줄이라 잘 돌아다, 운영에서 두 줄이 되는 순간 21000 에러로 죽습니다. → 스칼라 자리에는 집계 함수를 쓰거나 ORDER BY ... LIMIT 1 로 한 줄을 보장하세요.

이런 실수를 합니다

상관 서브쿼리를 SELECT 절에 몇 개씩 쌓았다

SELECT (SELECT COUNT...), (SELECT MAX...), (SELECT MIN...) 은 바깥 줄 수 × 서브쿼리 개수만큼 다시 둡니다. 섹션 5 에서 잔 그 차이입니다. → 하나의 JOIN + GROUP BY 로 묶으세요.

이런 실수를 합니다

WITH RECURSIVE 에 종료 조건을 안 걸었다

데이터가 깨끗할 때는 잘 됩니다. 순환 참조가 하나 생기는 순간 쿼리가 안 끝나고 서버 하나를 통째로 먹습니다. → WHERE 깊이 < N 을 항상 걸고 CYCLE 절도 함께 쓰세요.

이런 실수를 합니다

“CTE 는 느리다” 는 옛날 글을 보고 읽기 어려운 중첩 서브쿼리로 되돌렸다

PostgreSQL 12 부터 CTE 는 인라인됩니다. 읽기 좋은 쪽으로 쓰고, 굳혀 두고 싶을 때만 MATERIALIZED 를 붙이세요.

```
await db.close();
```

줄마다 계산하기 (윈도우 함수)

```
RUN node 개념05_줄마다_계산하기.js
```

개념03 의 GROUP BY 는 여러 줄을 한 줄로 접습니다. 설비마다 평균 점수를 구하면 점검기록 15줄이 설비 4줄로 줄어듭니다. 그런데 현장 질문은 보통 이런 모양입니다. “이 점검 점수가 그 설비 평균보다 높은가?” 이걸 보려면 점검 15줄이 그대로 남고, 그 옆에 “이 설비의 평균” 이 붙어 있어야 합니다. 줄을 접으면 답할 수 없습니다. 윈도우 함수가 그것입니다. 줄은 그대로 두고 옆에 계산 칸을 하나 붙입니다. ★ 이 하나로 순위 매기기 · 직전 값과 비교 · 누적 합계 · 그룹마다 상위 N개가 전부 풀립니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

예제 데이터

```

await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    동        TEXT NOT NULL
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드),
    도입년도 INT NOT NULL
  );

  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드)
  );

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호),
    점검일   DATE NOT NULL,
    결과     TEXT NOT NULL,
    점수     INT,
    담당사번 INT REFERENCES 작업자(사번)
  );

  INSERT INTO 라인 VALUES
    ('A', '조립1라인', '1동'),
    ('B', '가공2라인', '1동'),
    ('C', '포장3라인', '2동'),
    ('D', '신설4라인', '2동');

  INSERT INTO 설비 VALUES

```

```
(1, '컨베이어 1호', 'A', 2015),
(2, '프레스 1호', 'A', 2019),
(3, '용접로봇 1호', 'B', 2021),
(4, '검사기 1호', 'B', 2018),
(5, '포장기 1호', 'C', 2022);
```

```
INSERT INTO 작업자 VALUES
```

```
(101, '김반장', 'A'),
(102, '이기사', 'B'),
(103, '박주임', 'B'),
(104, '최사원', 'C'),
(105, '정신입', NULL);
```

```
INSERT INTO 점검기록 VALUES
```

```
( 1, 1, '2024-01-05', '정상', 92, 101),
( 2, 1, '2024-02-05', '정상', 88, 101),
( 3, 1, '2024-03-06', '주의', 71, 102),
( 4, 1, '2024-04-05', '정상', 90, 101),
( 5, 2, '2024-01-12', '주의', 65, 102),
( 6, 2, '2024-02-14', '불량', 48, 103),
( 7, 2, '2024-03-15', '정상', 88, 101),
( 8, 3, '2024-01-20', '정상', 95, 103),
( 9, 3, '2024-02-20', '정상', NULL, 103),
(10, 3, '2024-03-21', '주의', 74, 102),
(11, 3, '2024-04-22', '정상', 90, NULL),
(12, 3, '2024-05-23', '불량', 52, 103),
(13, 4, '2024-01-28', '정상', NULL, 104),
(14, 4, '2024-03-29', '정상', 88, 104),
(15, 4, '2024-05-30', '주의', 69, NULL);
`);
```

이 개념에서만 쓰는 표를 하나 더 만듭니다. 누적 합계·이동 평균 실습용이라 값이 작습니다.


```
await db.exec(`
  CREATE TABLE 생산실적 (
    실적번호 INT PRIMARY KEY, 라인코드 TEXT NOT NULL, 생산일 DATE NOT NULL, 수량 INT
    NOT NULL
  );
  INSERT INTO 생산실적 VALUES
    (1, 'A', '2024-06-01', 10), (2, 'A', '2024-06-02', 20), (3, 'A', '2024-06-03', 30),
    (4, 'A', '2024-06-04', 20),
    (5, 'A', '2024-06-05', 40), (6, 'A', '2024-06-06', 20), (7, 'A', '2024-06-07', 50),
    (8, 'A', '2024-06-08', 30),
    (9, 'B', '2024-06-01', 5), (10, 'B', '2024-06-02', 15), (11, 'B', '2024-06-03', 10),
    (12, 'B', '2024-06-04', 25),
    (13, 'B', '2024-06-05', 10), (14, 'B', '2024-06-06', 20), (15, 'B', '2024-06-07', 15),
    (16, 'B', '2024-06-08', 5);
`);
```

DATE 는 JS 로 Date 객체로 옵니다. NULL 을 그냥 찍으면 "null" 이라 눈에 안 들어옵니다.

```
const 날짜 = (것) => (것 === null ? "(없음)" : 것.toISOString().slice(0, 10));
const 값 = (것) => (것 === null ? "(없음)" : String(것));
```

섹션 1 — GROUP BY 는 줄을 접고, 윈도우는 줄을 남깁니다

같은 것을 두 방식으로 구해 보겠습니다. "설비마다 평균 점수" 입니다.

① GROUP BY — 설비 하나가 한 줄이 됩니다

```
const 접은것 = await db.query(
  `SELECT 설비번호, ROUND(AVG(점수), 1) AS 평균
  FROM 점검기록 GROUP BY 설비번호 ORDER BY 설비번호`);
console.log("[① GROUP BY] 줄 수:", 접은것.rows.length);
```

출력 [① GROUP BY] 줄 수: 4

```
for (const 줄 of 접은것.rows) console.log(` · 설비${줄.설비번호} · 평균 ${줄.평균}`);
```

출력 · 설비1 · 평균 85.3
 · 설비2 · 평균 67.0
 · 설비3 · 평균 77.8
 · 설비4 · 평균 78.5

★ 점검기록 15줄이 4줄이 됐습니다. 개별 점검은 사라졌습니다. 설비5(포장기 1호)는 점검기록이 없어서 아예 안 나옵니다. 개념02 의 LEFT JOIN 이야기입니다. ★ ROUND(AVG(...), 1) 은 NUMERIC 이라 JS 로 문자열로 옵니다. "85.3" 이지 85.3 이 아닙니다.

② 윈도우 함수 — 줄은 그대로, 옆에 칸만 붙습니다

```
const 안점은것 = await db.query(
  `SELECT 점검번호, 설비번호, 점수,
    ROUND(AVG(점수) OVER (PARTITION BY 설비번호), 1) AS 설비평균
  FROM 점검기록 ORDER BY 점검번호`);
console.log("[② 윈도우] 줄 수:", 안점은것.rows.length);
```

출력 [② 윈도우] 줄 수: 15

```
for (const 줄 of 안점은것.rows.slice(0, 5)) // 앞 5줄만 봅니다 console.log(`· 점검
${줄.점검번호} · 설비${줄.설비번호} · 점수 ${줄.점수} · 설비평균 ${줄.
설비평균}`);
```

출력 · 점검1 · 설비1 · 점수 92 · 설비평균 85.3
 · 점검2 · 설비1 · 점수 88 · 설비평균 85.3
 · 점검3 · 설비1 · 점수 71 · 설비평균 85.3
 · 점검4 · 설비1 · 점수 90 · 설비평균 85.3
 · 점검5 · 설비2 · 점수 65 · 설비평균 67.0

★★ 같은 AVG(점수) 인데 결과 줄 수가 4 와 15 로 다릅니다. GROUP BY 는 “접어서 하나로”, 윈도우는 “옆에 붙이기” 입니다. 윈도우 함수는 줄을 절대 없애지 않습니다. 칸이 하나 늘 뿐입니다.

③ 그래서 “그 설비 평균보다 높은 점검” 을 한 쿼리로 씁니다 ★ WHERE 에는 윈도우 함수를 못 씁니다 (섹션 4). 서브쿼리로 한 번 감쌉니다.

```
const 평균초과 = await db.query(`
  SELECT 점검번호, 설비번호, 점수, ROUND(설비평균, 1) AS 설비평균
  FROM (
    SELECT 점검번호, 설비번호, 점수,
      AVG(점수) OVER (PARTITION BY 설비번호) AS 설비평균
    FROM 점검기록
  ) 불인표
  WHERE 점수 > 설비평균 ORDER BY 점검번호
`);
console.log("[③ 설비 평균보다 높은 점검] 줄 수:", 평균초과.rows.length);
```

출력 [③ 설비 평균보다 높은 점검] 줄 수: 7

```
for (const 줄 of 평균초과.rows) console.log(`· 점검${줄.점검번호} · 설비${줄.
설비번호} · ${줄.점수} > ${줄.설비평균}`);
```

```
출력      · 점검1 · 설비1 · 92 > 85.3
          · 점검2 · 설비1 · 88 > 85.3
          · 점검4 · 설비1 · 90 > 85.3
          · 점검7 · 설비2 · 88 > 67.0
          · 점검8 · 설비3 · 95 > 77.8
          · 점검11 · 설비3 · 90 > 77.8
          · 점검14 · 설비4 · 88 > 78.5
```

④ 윈도우 없이 하려면 — 평균을 GROUP BY 로 따로 구해서 원래 표에 다시 이어야 합니다

```
const 옛날방식 = await db.query(`
  SELECT 점.점검번호 FROM 점검기록 점
  JOIN (SELECT 설비번호, AVG(점수) AS 평균 FROM 점검기록 GROUP BY 설비번호) 평
  ON 평.설비번호 = 점.설비번호
  WHERE 점.점수 > 평.평균 ORDER BY 점.점검번호
`);
console.log("[④ GROUP BY + JOIN] 결과 줄 수가 ③과 같은가:", 옛날방식.rows.length
=== 평균초과.rows.length);
```

```
출력      [④ GROUP BY + JOIN] 결과 줄 수가 ③과 같은가: true
```

★ 답은 같습니다. 그런데 ④는 점검기록을 두 번 읽고 JOIN 을 한 번 더 합니다. 윈도우는 한 번만 읽습니다.

섹션 2 — OVER (PARTITION BY ... ORDER BY ...)

OVER 괄호 안이 “창(window)” 의 정의입니다. 이 줄에서 어디까지를 볼 것인가입니다.

① OVER () — 괄호가 비면 표 전체가 한 창입니다

```
const 전체창 = await db.query(
  `SELECT 점검번호, 점수, ROUND(AVG(점수) OVER (), 1) AS 전체평균
  FROM 점검기록 ORDER BY 점검번호 LIMIT 3`);
for (const 줄 of 전체창.rows) console.log(`· 점검${줄.점검번호} · 점수 ${줄.점수} ·
전체평균 ${줄.전체평균}`);
```

```
출력      · 점검1 · 점수 92 · 전체평균 77.7
          · 점검2 · 점수 88 · 전체평균 77.7
          · 점검3 · 점수 71 · 전체평균 77.7
```

② PARTITION BY — 창을 나눕니다. 묶는 뜻은 GROUP BY 와 같지만 줄은 안 접습니다. 섹션 1 ②에서 이미 썼습니다.

③ ★★ OVER 안에 ORDER BY 를 넣으면 집계적의 뜻이 바뀝니다 안 넣으면 “창 전체의 합”, 넣으면 “처음부터 지금 줄까지의 합” 이 됩니다.

```
const 합비교 = await db.query(`
  SELECT 점검일, 점수,
         SUM(점수) OVER (PARTITION BY 설비번호) AS 설비합,
         SUM(점수) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS 누적합
  FROM 점검기록 WHERE 설비번호 = 1 ORDER BY 점검일
`);
for (const 줄 of 합비교.rows) console.log(` · ${날짜(줄.점검일)} · 점수 ${줄.점수} ·
설비합 ${줄.설비합} · 누적합 ${줄.누적합}`);
```

```
출력      · 2024-01-05 · 점수 92 · 설비합 341 · 누적합 92
          · 2024-02-05 · 점수 88 · 설비합 341 · 누적합 180
          · 2024-03-06 · 점수 71 · 설비합 341 · 누적합 251
          · 2024-04-05 · 점수 90 · 설비합 341 · 누적합 341
```

★★ 설비합은 네 줄 모두 341로 같습니다. 누적합은 92 → 180 → 251 → 341로 자랍니다. ORDER BY 한 마디 차이인데 뜻이 완전히 다릅니다. 이유는 “프레임”입니다. 섹션 6에서 봅니다.

④ 같은 창을 여러 번 쓸 때는 WINDOW 절로 이름을 붙입니다

```
const 이름붙인창 = await db.query(`
  SELECT 점검번호, 점수,
         ROUND(AVG(점수) OVER 설비창, 1) AS 평균, MAX(점수) OVER 설비창 AS 최고,
         COUNT(점수) OVER 설비창 AS 점수있는건수, COUNT(*) OVER 설비창 AS 전체건수
  FROM 점검기록 WHERE 설비번호 = 3
  WINDOW 설비창 AS (PARTITION BY 설비번호)
  ORDER BY 점검번호 LIMIT 3
`);
for (const 줄 of 이름붙인창.rows)
  console.log(` · 점검${줄.점검번호} · 점수 ${값(줄.점수)} · 평균 ${줄.평균} · 최고
${줄.최고} · 점수있는건수 ${줄.점수있는건수} · 전체건수 ${줄.전체건수}`);
```

```
출력      · 점검8 · 점수 95 · 평균 77.8 · 최고 95 · 점수있는건수 4 · 전체건수 5
          · 점검9 · 점수 (없음) · 평균 77.8 · 최고 95 · 점수있는건수 4 · 전체건수 5
          · 점검10 · 점수 74 · 평균 77.8 · 최고 95 · 점수있는건수 4 · 전체건수 5
```

★ 설비3은 5줄인데 점수있는건수(COUNT(점수))는 4입니다. 점검9의 점수가 NULL이라서 그렇습니다. COUNT(*)는 줄을, COUNT(칸)은 NULL 아닌 값을 셉니다. 개념03에서 본 그대로입니다.

★ COUNT의 SQL 타입은 BIGINT입니다. 그런데 PGlite는 안전하게 담기는 크기면 number로, 넘어가면 bigint로 줍니다. 지금은 5라서 number지만 줄이 쌓이면 바뀝니다. (개념01에서 재현합니다)

```
console.log("COUNT 결과의 자바스크립트 타입:", typeof 이름붙인창.rows[0].전체건수);
```

```
출력      COUNT 결과의 자바스크립트 타입: number
```

```
console.log("숫자로 쓰려면 Number()로 바꿉니다:", Number(이름붙인창.rows[0].
전체건수) + 1);
```

출력 숫자로 쓰려면 Number() 로 바꿉니다: 6

MySQL 은 여기가 다릅니다

· MySQL 은 8.0 부터 윈도우 함수가 있습니다. 5.7 에는 없어서 변수(@)로 흉내 내야 했습니다 ★ 자세한 비교는 09단원에서 둘 다 띄워 놓고 합니다

섹션 3 — ROW_NUMBER / RANK / DENSE_RANK

순위를 매기는 함수가 셋 있습니다. 동점일 때 행동이 다릅니다. 이 표에는 동점이 일부러 들어 있습니다. 90점이 2건, 88점이 3건입니다.

```
const 순위셋 = await db.query(`
  SELECT 점검번호, 점수,
         ROW_NUMBER() OVER (ORDER BY 점수 DESC NULLS LAST, 점검번호) AS 행번호,
         RANK()          OVER (ORDER BY 점수 DESC NULLS LAST)          AS 순위,
         DENSE_RANK() OVER (ORDER BY 점수 DESC NULLS LAST)          AS 촘촘순위
  FROM 점검기록 ORDER BY 점수 DESC NULLS LAST, 점검번호
`);
for (const 줄 of 순위셋.rows.slice(0, 9)) // 위 9줄만 봅니다 console.log(`· 점검
${줄.점검번호} · 점수 ${줄.점수} · 행번호 ${줄.행번호} · 순위 ${줄.순위} ·
촘촘순위 ${줄.촘촘순위}`);
```

출력	· 점검8 · 점수 95 · 행번호 1 · 순위 1 · 촘촘순위 1
	· 점검1 · 점수 92 · 행번호 2 · 순위 2 · 촘촘순위 2
	· 점검4 · 점수 90 · 행번호 3 · 순위 3 · 촘촘순위 3
	· 점검11 · 점수 90 · 행번호 4 · 순위 3 · 촘촘순위 3
	· 점검2 · 점수 88 · 행번호 5 · 순위 5 · 촘촘순위 4
	· 점검7 · 점수 88 · 행번호 6 · 순위 5 · 촘촘순위 4
	· 점검14 · 점수 88 · 행번호 7 · 순위 5 · 촘촘순위 4
	· 점검10 · 점수 74 · 행번호 8 · 순위 8 · 촘촘순위 5
	· 점검3 · 점수 71 · 행번호 9 · 순위 9 · 촘촘순위 6

★ 90점 두 줄, 88점 세 줄을 보세요. ROW_NUMBER : 동점이어도 1,2,3,4... 같은 번호가 없습니다
RANK : 90점은 둘 다 3위, 다음 88점은 5위로 건너뛴니다 (3,3,5) DENSE_RANK : 90점은 둘 다 3위,
다음 88점은 4위 (3,3,4). 안 건너뛴니다

★★★ ROW_NUMBER 는 동점일 때 “어느 줄이 먼저인지” 를 스스로 정하지 않습니다. ORDER BY 점수
DESC

만 쓰면 88점 세 줄의 앞뒤가 실행할 때마다 달라질 수 있습니다. 그래서 위 쿼리는
ORDER BY 에
점검번호를 하나 더 넣었습니다. 이것 tie-breaker 라고 합니다.
* 순위·정렬 쿼리에는 값이 겹치지 않는 칸을 반드시 하나 더 넣으세요. 결과가
고정됩니다.

★ NULL 은 어디로 가는가 — Postgres 는 DESC 면 NULL 을 앞에 둡니다(NULLS FIRST 가 기본).

```
const 널앞 = await db.query(
  `SELECT 점검번호, 점수, RANK() OVER (ORDER BY 점수 DESC) AS 순위
  FROM 점검기록 ORDER BY 점수 DESC, 점검번호 LIMIT 3`);
for (const 줄 of 널앞.rows) console.log(` · 점검${줄.점검번호} · 점수 ${줄.점수} ·
순위 ${줄.순위}`);
```

```
출력      · 점검9 · 점수 (없음) · 순위 1
          · 점검13 · 점수 (없음) · 순위 1
          · 점검8 · 점수 95 · 순위 3
```

★★ 점수가 없는 두 건이 1위가 됐습니다. 진짜 1등인 95점이 3위로 밀렸습니다. “점수 높은 순” 인데 점수 없는 줄이 맨 위에 옵니다. NULLS LAST 로 고칩니다. (ASC 는 기본이 NULLS LAST 라 반대)

```
const 널뒤 = await db.query(
  `SELECT 점검번호, 점수, RANK() OVER (ORDER BY 점수 DESC NULLS LAST) AS 순위
  FROM 점검기록 ORDER BY 점수 DESC NULLS LAST, 점검번호 LIMIT 3`);
for (const 줄 of 널뒤.rows) console.log(` · 점검${줄.점검번호} · 점수 ${줄.점수} ·
순위 ${줄.순위}`);
```

```
출력      · 점검8 · 점수 95 · 순위 1
          · 점검1 · 점수 92 · 순위 2
          · 점검4 · 점수 90 · 순위 3
```

섹션 4 — ★ 그룹마다 상위 N개 뽑기

“설비마다 점수가 높은 점검 2건씩” — 실무에서 가장 자주 필요한 모양입니다. 한 번에는 안 됩니다. 1단계 설비마다 순위를 매기고, 2단계 순위 2 이하만 남깁니다.

★ 먼저 안 되는 쪽부터 봅니다. WHERE 에 윈도우 함수를 쓰면 어떻게 되는가

```
try {
  await db.query(
    `SELECT 점검번호 FROM 점검기록
    WHERE ROW_NUMBER() OVER (PARTITION BY 설비번호 ORDER BY 점수 DESC) <= 2`);
  console.log("WHERE 에 윈도우 함수 - 그냥 통과했습니다");
} catch (에러) {
  console.log("WHERE 에 윈도우 함수 · code:", 에러.code);
```

```
출력      WHERE 에 윈도우 함수 · code: 42P20
```

```
console.log("WHERE 에 윈도우 함수 · message:", 에러.message);
```

```
출력      WHERE 에 윈도우 함수 · message: window functions are not allowed in WHERE
```

```
}
```

★★★ 이유는 실행 순서입니다. FROM → WHERE → GROUP BY → HAVING → 윈도우 → SELECT → ORDER BY WHERE 가 도는 시점에는 순위가 아직 계산되지 않았습니다. 없는 값을 조건에 쓸 수는 없습니다. 그래서 “순위를 다 매긴 결과” 를 한 번 감싼 뒤, 바깥에서 WHERE 를 겁니다.

방법 ① 서브쿼리로 감싸기

```
const 상위2 = await db.query(`
  SELECT 설비번호, 점검번호, 점수, 순위
  FROM (
    SELECT 설비번호, 점검번호, 점수,
           ROW_NUMBER() OVER (PARTITION BY 설비번호
                               ORDER BY 점수 DESC NULLS LAST, 점검번호) AS 순위
    FROM 점검기록
  ) 순위표
  WHERE 순위 <= 2 ORDER BY 설비번호, 순위
`);
console.log("[설비마다 상위 2건] 줄 수:", 상위2.rows.length);
```

출력 [설비마다 상위 2건] 줄 수: 8

```
for (const 줄 of 상위2.rows) console.log(` · 설비${줄.설비번호} · ${줄.순위}위 · 점검
${줄.점검번호} · ${줄.점수}점`);
```

출력 · 설비1 · 1위 · 점검1 · 92점
 · 설비1 · 2위 · 점검4 · 90점
 · 설비2 · 1위 · 점검7 · 88점
 · 설비2 · 2위 · 점검5 · 65점
 · 설비3 · 1위 · 점검8 · 95점
 · 설비3 · 2위 · 점검11 · 90점
 · 설비4 · 1위 · 점검14 · 88점
 · 설비4 · 2위 · 점검15 · 69점

방법 ② CTE(WITH)로 감싸기 — 개념04 에서 본 그 WITH 입니다. 읽기가 훨씬 낫습니다.

```
const 상위2CTE = await db.query(`
  WITH 순위매김 AS (
    SELECT 설비번호, 점검번호, 점수,
           ROW_NUMBER() OVER (PARTITION BY 설비번호
                               ORDER BY 점수 DESC NULLS LAST, 점검번호) AS 순위
    FROM 점검기록
  )
  SELECT 설비번호, 점검번호, 점수, 순위 FROM 순위매김
  WHERE 순위 <= 2 ORDER BY 설비번호, 순위
`);
console.log("서브쿼리와 CTE 결과가 같은가:", JSON.stringify(상위2.rows) ===
JSON.stringify(상위2CTE.rows));
```

출력 서브쿼리와 CTE 결과가 같은가: true

★ LIMIT 은 이 일을 못 합니다. LIMIT 2 는 “전체에서 2줄” 이지 “설비마다 2줄” 이 아닙니다.

```
const 리밋 = await db.query(`
  SELECT 설비번호, 점검번호, 점수 FROM 점검기록
  ORDER BY 점수 DESC NULLS LAST, 점검번호 LIMIT 2`);
for (const 줄 of 리밋.rows) console.log(` · 설비${줄.설비번호} · 점검${줄.점검번호} ·
${줄.점수}점`);
```

출력 · 설비3 · 점검8 · 95점
 · 설비1 · 점검1 · 92점

★★ 설비3 과 설비1 만 나왔습니다. 설비2·설비4 는 한 줄도 못 나왔습니다. “그룹마다 N개” 는 LIMIT 으로 절대 안 됩니다. 순위를 매기고 감싸는 것이 유일한 길입니다.

섹션 5 — LAG / LEAD (이전 줄 · 다음 줄 보기)

LAG 는 창 안에서 “한 줄 위” 값을, LEAD 는 “한 줄 아래” 값을 지금 줄 옆에 붙여 줍니다. ★ 개념02 에서 SELF JOIN 으로 “직전 점검” 을 찾으려면 조건이 꽤 까다로웠습니다. LAG 는 한 마디입니다.

```
const 간격 = await db.query(`
  SELECT 설비번호, 점검일,
         LAG(점검일) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS 이전점검일,
         점검일 - LAG(점검일) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS
  며칠만에
  FROM 점검기록 WHERE 설비번호 IN (1, 4) ORDER BY 설비번호, 점검일
`);
for (const 줄 of 간격.rows)
  console.log(` · 설비${줄.설비번호} · ${날짜(줄.점검일)} · 이전 ${날짜(줄.
이전점검일)} · ${값(줄.며칠만에)}일만에`);
```

출력 · 설비1 · 2024-01-05 · 이전 (없음) · (없음)일만에
 · 설비1 · 2024-02-05 · 이전 2024-01-05 · 31일만에
 · 설비1 · 2024-03-06 · 이전 2024-02-05 · 30일만에
 · 설비1 · 2024-04-05 · 이전 2024-03-06 · 30일만에
 · 설비4 · 2024-01-28 · 이전 (없음) · (없음)일만에
 · 설비4 · 2024-03-29 · 이전 2024-01-28 · 61일만에
 · 설비4 · 2024-05-30 · 이전 2024-03-29 · 62일만에

★ DATE 끼리 빼면 Postgres 는 “며칠” 이라는 정수를 줍니다. 설비4 는 두 달씩 벌어져 있습니다. ★ 각 설비의 첫 줄은 이전 값이 없어 NULL 입니다. PARTITION 이 바뀌면 LAG 도 다시 비었습니다. 기본값을 주고 싶으면 세 번째 인자를 씁니다: LAG(칸, 몇칸위, 기본값)


```
const 앞뒤 = await db.query(`
  SELECT 점검번호, 점수,
    LAG(점수) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS 이전점수,
    LAG(점수, 1, 0) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS
이전점수기본0,
    LEAD(점수) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS 다음점수,
    점수 - LAG(점수) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS 점수변화
  FROM 점검기록 WHERE 설비번호 = 3 ORDER BY 점검일
`);
for (const 줄 of 앞뒤.rows)
  console.log(` · 점검${줄.점검번호} · 점수 ${줄.점수} · 이전 ${줄.이전점수} ·
이전(기본0) ${줄.이전점수기본0} · 다음 ${줄.다음점수} · 변화 ${줄.
점수변화}`);
```

출력

- 점검8 · 점수 95 · 이전 (없음) · 이전(기본0) 0 · 다음 (없음) · 변화 (없음)
- 점검9 · 점수 (없음) · 이전 95 · 이전(기본0) 95 · 다음 74 · 변화 (없음)
- 점검10 · 점수 74 · 이전 (없음) · 이전(기본0) (없음) · 다음 90 · 변화 (없음)
- 점검11 · 점수 90 · 이전 74 · 이전(기본0) 74 · 다음 52 · 변화 16
- 점검12 · 점수 52 · 이전 90 · 이전(기본0) 90 · 다음 (없음) · 변화 -38

★★ 점검10 을 보세요. 이전(기본0)이 0 이 아니라 (없음) 입니다. 기본값은 “위에 줄이 아예 없을 때” 만 씁니다. 위에 줄은 있는데 그 값이 NULL 이면 NULL 그대로입니다. ★ 그래서 점수변화도 NULL 이 두 번 나옵니다. NULL 이 섞인 뽀렘은 결과도 NULL 입니다.

섹션 6 — 누적 합계와 이동 평균 (프레임)

창 안에서 “지금 줄 기준으로 어디부터 어디까지” 를 정하는 것이 프레임(frame)입니다.

```
const 누적 = await db.query(`
  SELECT 라인코드, 생산일, 수량,
    SUM(수량) OVER (PARTITION BY 라인코드 ORDER BY 생산일) AS 누적수량,
    ROUND(AVG(수량) OVER (PARTITION BY 라인코드 ORDER BY 생산일
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 1) AS
사흘평균
  FROM 생산실적 ORDER BY 라인코드, 생산일
`);
for (const 줄 of 누적.rows.slice(0, 10)) // 라인 A 8줄 + 라인 B 앞
2줄 console.log(` · ${줄.라인코드} · ${줄.날짜(줄.생산일)} · 수량 ${줄.수량} · 누적 ${줄.
누적수량} · 3일평균 ${줄.사흘평균}`);
```

출력	· A · 2024-06-01 · 수량 10 · 누적 10 · 3일평균 10.0
	· A · 2024-06-02 · 수량 20 · 누적 30 · 3일평균 15.0
	· A · 2024-06-03 · 수량 30 · 누적 60 · 3일평균 20.0
	· A · 2024-06-04 · 수량 20 · 누적 80 · 3일평균 23.3
	· A · 2024-06-05 · 수량 40 · 누적 120 · 3일평균 30.0
	· A · 2024-06-06 · 수량 20 · 누적 140 · 3일평균 26.7
	· A · 2024-06-07 · 수량 50 · 누적 190 · 3일평균 36.7
	· A · 2024-06-08 · 수량 30 · 누적 220 · 3일평균 33.3
	· B · 2024-06-01 · 수량 5 · 누적 5 · 3일평균 5.0
	· B · 2024-06-02 · 수량 15 · 누적 20 · 3일평균 10.0

★ 라인 A 의 누적: 10 → 30 → 60 → 80 → 120 → 140 → 190 → 220. 손으로 더해도 맞습니다. ★
라인 B 로 넘어가면 누적이 5 부터 다시 시작합니다. PARTITION 이 바뀌면 창도 새로 열립니다. ★
3일평균 첫 줄은 자기 값, 둘째 줄은 두 개 평균입니다. ROWS BETWEEN 2 PRECEDING AND
CURRENT ROW = “위로 두 줄 + 나 자신” 입니다.

★★ ORDER BY 만 쓰고 프레임 안 적으면 기본값은 RANGE BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW (“창 시작부터 지금 줄까지”) 입니다. 그래서 섹션 2 ㉔ 에서 SUM 이
누적이 됐습니다. ORDER BY 도 없으면 창 전체가 프레임입니다. 그때는 합이 하나로 고정됐습니다. ★★
ROWS 와 RANGE 는 세는 단위가 다릅니다. ROWS = 줄 수, RANGE = ORDER BY 한 칸의 “값”. 값이
같은 줄들은 RANGE 에서 한 덩어리입니다. 그래서 동점이 있으면 결과가 달라집니다.

```
const 결과값 = await db.query(`
  SELECT 수량,
         SUM(수량) OVER (ORDER BY 수량 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS 줄기준,
         SUM(수량) OVER (ORDER BY 수량 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS 값기준
  FROM 생산실적 WHERE 라인코드 = 'A' ORDER BY 수량, 줄기준
`);
for (const 줄 of 결과값.rows) console.log(` · 수량 ${줄.수량} · ROWS ${줄.줄기준} ·
RANGE ${줄.값기준}`);
```

출력	· 수량 10 · ROWS 10 · RANGE 10
	· 수량 20 · ROWS 30 · RANGE 70
	· 수량 20 · ROWS 50 · RANGE 70
	· 수량 20 · ROWS 70 · RANGE 70
	· 수량 30 · ROWS 100 · RANGE 130
	· 수량 30 · ROWS 130 · RANGE 130
	· 수량 40 · ROWS 170 · RANGE 170
	· 수량 50 · ROWS 220 · RANGE 220

★★ 수량 20 짜리 세 줄을 보세요. ROWS 는 30 · 50 · 70 으로 한 줄씩 더해 갑니다. RANGE 는 셋 다 70
입니다. “20 이하인 값 전부” 를 한꺼번에 보기 때문입니다. ★ 줄 단위로 세려면 ROWS 라고 반드시
적으세요. 안 적으면 RANGE 라 동점이 뭉개집니다. ROWS 는 같은 값끼리의 앞뒤가 안 정해져서, 위에서는
ORDER BY 수량, 줄기준 으로 고정했습니다.

섹션 7 — ★★ FIRST_VALUE 는 되는데 LAST_VALUE 는 안 되는 이유

```
const 첫끝 = await db.query(`
  SELECT 생산일, 수량,
    FIRST_VALUE(수량) OVER (PARTITION BY 라인코드 ORDER BY 생산일) AS 첫값,
    LAST_VALUE(수량) OVER (PARTITION BY 라인코드 ORDER BY 생산일) AS 끝값_그냥,
    LAST_VALUE(수량) OVER (PARTITION BY 라인코드 ORDER BY 생산일
                          ROWS BETWEEN UNBOUNDED PRECEDING
                          AND UNBOUNDED FOLLOWING) AS 끝값_프레임명시
  FROM 생산실적 WHERE 라인코드 = 'A' ORDER BY 생산일 LIMIT 5
`);
for (const 줄 of 첫끝.rows)
  console.log(` ${날짜(줄.생산일)} · 수량 ${줄.수량} · 첫값 ${줄.첫값} · 끝값(그냥)
${줄.끝값_그냥} · 끝값(프레임명시) ${줄.끝값_프레임명시}`);
```

출력

- 2024-06-01 · 수량 10 · 첫값 10 · 끝값(그냥) 10 · 끝값(프레임명시) 30
- 2024-06-02 · 수량 20 · 첫값 10 · 끝값(그냥) 20 · 끝값(프레임명시) 30
- 2024-06-03 · 수량 30 · 첫값 10 · 끝값(그냥) 30 · 끝값(프레임명시) 30
- 2024-06-04 · 수량 20 · 첫값 10 · 끝값(그냥) 20 · 끝값(프레임명시) 30
- 2024-06-05 · 수량 40 · 첫값 10 · 끝값(그냥) 40 · 끝값(프레임명시) 30

★★★ 「끝값(그냥)」 칸을 보세요. 매 줄이 자기 수량과 똑같습니다. 라인 A 의 마지막 값 30 이 아닙니다.

ORDER BY 가 있으면 프레임 기본값이 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
라서
창이 아직 "지금 줄" 까지만 열려 있습니다. 그 안에서 마지막 줄은 곧 자기 자신입니다.
버그가 아니라 정의대로입니다. FIRST_VALUE 는 창 시작이 안 잘려서 우연히 잘 맞았을
뿐입니다.
고치는 법은 프레임을 ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING 이라고
직접 적는 것뿐입니다. 그러면 「끝값(프레임명시)」 처럼 모든 줄이 30 이 됩니다.

★ NTH_VALUE 도 똑같은 함정입니다. 프레임이 안 열려 있으면 그 번째 줄이 없어 NULL 입니다.

섹션 8 — 실전: 설비마다 마지막 점검 + 라인 평균과의 차이

지금까지 것을 다 씁니다. CTE 로 순위를 매기고, 감싸서 1위만 남기고, 그 위에 윈도우를 또 엮습니다.

```
const 현황판 = await db.query(`
  WITH 최신순 AS (
    SELECT 설비번호, 점검일, 결과, 점수,
      ROW_NUMBER() OVER (PARTITION BY 설비번호
                        ORDER BY 점검일 DESC, 점검번호 DESC) AS 최근순
    FROM 점검기록
  ),
  마지막점검 AS (
    SELECT 설.설비번호, 설.이름 AS 설비이름, 라.라인코드, 라.이름 AS 라인이름,
      최.점검일, 최.결과, 최.점수
```

```

JOIN 라인 라 ON 라.라인코드 = 설.라인코드
WHERE 최.최근순 = 1
)
SELECT 라인이름, 설비이름, 점검일, 결과, 점수,
       ROUND(AVG(점수) OVER (PARTITION BY 라인코드), 1) AS 라인평균,
       ROUND(점수 - AVG(점수) OVER (PARTITION BY 라인코드), 1) AS 평균대비,
       RANK() OVER (ORDER BY 점수 DESC NULLS LAST) AS 전체순위
FROM 마지막점검 ORDER BY 라인코드, 설비번호
`);
console.log("[설비별 마지막 점검 현황] 줄 수:", 현황판.rows.length);

```

출력 [설비별 마지막 점검 현황] 줄 수: 4

```

for (const 줄 of 현황판.rows)
  console.log(`· ${줄.라인이름} · ${줄.설비이름} · ${날짜(줄.점검일)} · ${줄.
결과} ${줄.점수}점 · 라인평균 ${줄.라인평균} · 대비 ${줄.평균대비} · 전체 ${줄.
전체순위}위`);

```

출력

- 조립1라인 · 컨베이어 1호 · 2024-04-05 · 정상 90점 · 라인평균 89.0 · 대비 1.0 · 전체 1위
- 조립1라인 · 프레스 1호 · 2024-03-15 · 정상 88점 · 라인평균 89.0 · 대비 -1.0 · 전체 2위
- 가공2라인 · 용접로봇 1호 · 2024-05-23 · 불량 52점 · 라인평균 60.5 · 대비 -8.5 · 전체 4위
- 가공2라인 · 검사기 1호 · 2024-05-30 · 주의 69점 · 라인평균 60.5 · 대비 8.5 · 전체 3위

★ 설비가 5개인데 4줄만 나왔습니다. 포장기 1호는 점검기록이 없어서, 라인 D 는 설비가 없어서 빠졌습니다. 빠진 설비까지 보이려면 설비를 왼쪽에 두고 LEFT JOIN 해야 합니다. 개념02 이야기입니다.

정리

GROUP BY 윈도우 함수 OVER(...)

줄 수 묶은 개수만큼 줄어듦 입력 줄 수 그대로 개별 값 사라짐 그대로 남음 묶는 말 GROUP BY 칸 PARTITION BY 칸 조건 거는 곳 HAVING 바깥으로 감싼 뒤 WHERE (SELECT 안에서만 씀)

순위 세 함수 — 점수가 90, 90, 88 일 때

ROW_NUMBER()	1, 2, 3	동점 없음. 순서는 tie-breaker 로 직접 정할 것
RANK()	1, 1, 3	동점 뒤가 건너뛴
DENSE_RANK()	1, 1, 2	동점 뒤가 안 건너뛴

프레임 기본값 — OVER 안에 뭐가 있느냐로 정해집니다 ORDER BY 없음 창 전체 (전체 합·전체 평균) ORDER BY 있음 RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW → 누적

ROWS n PRECEDING 줄 수로 셈 (이동 평균은 보통 이것) RANGE n PRECEDING 값으로 셈. 동점은 한 덩어리

실행 순서 — 윈도우는 한참 뒤입니다 FROM → WHERE → GROUP BY → HAVING → 윈도우 함수 → SELECT → ORDER BY

직접 해 볼 것

▫ 직접 해보기 1

설비마다 “그 설비 최고 점수” 를 옆에 붙이고, 최고 점수를 받은 줄만 남기세요.

힌트: MAX(점수) OVER (PARTITION BY 설비번호) 를 붙인 뒤 감싸서 점수 = 최고점수.

▫ 직접 해보기 2

작업자별로 담당한 점검을 점검일 순서로 늘어놓고 몇 번째인지 번호를 매기세요.

담당사번이 NULL 인 점검 2건이 어느 창에 들어가는지 확인하세요. NULL 도 하나의 창이 됩니다.

▫ 직접 해보기 3

라인마다 점수 상위 3건을 뽑으세요. 설비가 아니라 라인 기준입니다.

점검기록 → 설비 → 라인 으로 이은 뒤 PARTITION BY 라인코드 로. tie-breaker 를 꼭 넣으세요.

▫ 직접 해보기 4

생산실적에 “전날 대비 증감” 칸을 만드세요.

수량 - LAG(수량) OVER (PARTITION BY 라인코드 ORDER BY 생산일) 입니다. 증감이 양수인 날만 남기세요. WHERE 에 바로 못 쓴다는 것을 확인하세요.

▫ 직접 해보기 5

라인마다 “그 라인 전체 합계 대비 그날의 비중(%)” 을 구하세요.

ROUND(100.0 * 수량 / SUM(수량) OVER (PARTITION BY 라인코드), 1) 입니다. ★ 100 대신 100.0 을 쓰는 이유를 생각해 보세요(정수 나눗셈).

▫ 직접 해보기 6

섹션 7 의 LAST_VALUE 프레임을 RANGE BETWEEN UNBOUNDED PRECEDING

AND UNBOUNDED FOLLOWING 으로 바꿔 보세요. ROWS 로 쓴 것과 결과가 같습니까?

결과별(정상/주의/불량) 건수를 세되 줄은 15줄 그대로 두세요.

COUNT(*) OVER (PARTITION BY 결과) 입니다. GROUP BY 로 하면 몇 줄인지 대조해 보세요.

자주 하는 실수

이런 실수를 합니다

WHERE 에 윈도우 함수를 썼습니다

WHERE ROW_NUMBER() OVER (...) <= 2 → 42P20 에러입니다. WHERE 는 윈도우보다 먼저 돕니다. ★ 서브쿼리나 CTE 로 한 번 감싼 뒤 바깥에서 거세요. HAVING 도 마찬가지로 안 됩니다.

이런 실수를 합니다

LAST_VALUE 가 마지막 값인 줄 알았습니다

ORDER BY 를 넣는 순간 프레임이 “지금 줄까지” 로 잘립니다. 그 안의 마지막은 자기 자신입니다. ★ ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING 을 반드시 적으세요. ★ FIRST_VALUE 는 멀쩡해 보여서 더 헛갈립니다. 창 시작은 안 잘립니다.

이런 실수를 합니다

ORDER BY 를 넣었더니 SUM 이 누적이 됐습니다

SUM(점수) OVER (PARTITION BY 설비번호) → 설비 전체 합 (모든 줄 같은 값) SUM(점수) OVER (PARTITION BY 설비번호 ORDER BY 점검일) → 처음부터 지금 줄까지의 누적 ★ “정렬만 하려고” OVER 안에 ORDER BY 를 넣으면 뜻이 바뀝니다. 정렬은 쿼리 맨 끝에서 하세요.

이런 실수를 합니다

동점 tie-breaker 를 안 넣어서 순위가 매번 달라졌습니다

ROW_NUMBER() OVER (ORDER BY 점수 DESC) 는 88점 세 줄의 앞뒤를 정해 주지 않습니다. 테스트는 통과했는데 운영에서 순서가 바뀌는 버그입니다. ★ ORDER BY 점수 DESC, 점검번호 처럼 값이 겹치지 않는 칸을 항상 하나 더 넣으세요.

이런 실수를 합니다

“점수 높은 순” 인데 점수 없는 줄이 1위로 올라왔습니다

Postgres 는 DESC 정렬에서 NULL 을 앞에 돕니다(NULLS FIRST 가 기본). ★ ORDER BY 점수 DESC NULLS LAST 라고 적으세요. 창 안과 쿼리 끝 둘 다 고쳐야 합니다.

이런 실수를 합니다

그룹마다 N개를 LIMIT 으로 뽑으려 했습니다

LIMIT 2 는 결과 전체에서 2줄입니다. 설비마다 2줄이 아닙니다. ★ PARTITION BY 로 순위를 매기고
감싸서 WHERE 순위 <= N 하세요.

이런 실수를 합니다

COUNT(*) OVER (...) 결과를 그대로 더했습니다

COUNT 의 SQL 타입은 BIGINT 입니다. 작을 때는 number 로 오다가 안전 정수 범위를 넘으면 bigint 로
웁니다. 그때 $5n + 1$ 은 타입 에러가 납니다. 개발에선 멀쩡하다 운영에서 터집니다. ★ Number() 로 바꾸고
쓰세요.

```
await db.close();
```

여러 표를 잇기

RUN node 연습문제.js

★ 처음 실행하면 PGlite 를 켜느라 1초쯤 걸립니다.

푸는 법

① 각 문제 아래의 **문제N** 안에 SQL 을 씁니다. ② 이 파일을 실행합니다. ③ 채점 코드가 알아서 돌려 보고 정답인지 알려 줍니다.

★ **TODO** 라는 글자가 남아 있으면 “아직 안 풀었습니다” 로 넘어갑니다. 답을 쓸 때 주석째로 지우세요.

★ 칸 이름은 자유입니다. **칸의 순서와 값**, 그리고 **줄의 순서**만 봅니다. 그래서 문제마다 “칸 순서” 와 “줄 순서” 를 적어 두었습니다. 그대로 맞추세요.

★ 막히면 개념01~05 를 다시 보세요. 정답은 연습문제_정답.js 에 있습니다. 먼저 30분은 혼자 붙잡고 있어 보세요. 답을 보고 이해한 것은 이틀이면 사라집니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```

예제 데이터

개념01~05 에서 쓰던 것과 같은 표입니다. (조직 표만 15번 문제용으로 더 있습니다)

라인 4개 · D(신설4라인) 은 설비가 하나도 없습니다 설비 5대 · 5번(포장기 1호) 은 점검기록이 하나도 없습니다 작업자 5명 · 105번(정신입) 은 점검을 담당한 적이 없습니다 점검기록 15건 · 점수에 NULL 2건, 담당사번에 NULL 2건이 섞여 있습니다


```
await db.exec(`
  CREATE TABLE 라인 (
    라인코드 TEXT PRIMARY KEY,
    이름      TEXT NOT NULL,
    동        TEXT NOT NULL
  );

  CREATE TABLE 설비 (
    설비번호 INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드),
    도입년도 INT NOT NULL
  );

  CREATE TABLE 작업자 (
    사번      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드)
  );

  CREATE TABLE 점검기록 (
    점검번호 INT PRIMARY KEY,
    설비번호 INT REFERENCES 설비(설비번호),
    점검일   DATE NOT NULL,
    결과     TEXT NOT NULL,
    점수     INT,
    담당사번 INT REFERENCES 작업자(사번)
  );

  CREATE TABLE 조직 (
    번호      INT PRIMARY KEY,
    이름      TEXT NOT NULL,
    상위번호 INT REFERENCES 조직(번호)
  );

  INSERT INTO 라인 VALUES
```

```
('A', '조립1라인', '1동'),  
( 'B', '가공2라인', '1동'),  
( 'C', '포장3라인', '2동'),  
( 'D', '신설4라인', '2동');
```

INSERT INTO 설비 VALUES

```
(1, '컨베이어 1호', 'A', 2015),  
(2, '프레스 1호', 'A', 2019),  
(3, '용접로봇 1호', 'B', 2021),  
(4, '검사기 1호', 'B', 2018),  
(5, '포장기 1호', 'C', 2022);
```

INSERT INTO 작업자 VALUES

```
(101, '김반장', 'A'),  
(102, '이기사', 'B'),  
(103, '박주임', 'B'),  
(104, '최사원', 'C'),  
(105, '정신입', NULL);
```

INSERT INTO 점검기록 VALUES

```
( 1, 1, '2024-01-05', '정상', 92, 101),  
( 2, 1, '2024-02-05', '정상', 88, 101),  
( 3, 1, '2024-03-06', '주의', 71, 102),  
( 4, 1, '2024-04-05', '정상', 90, 101),  
( 5, 2, '2024-01-12', '주의', 65, 102),  
( 6, 2, '2024-02-14', '불량', 48, 103),  
( 7, 2, '2024-03-15', '정상', 88, 101),  
( 8, 3, '2024-01-20', '정상', 95, 103),  
( 9, 3, '2024-02-20', '정상', NULL, 103),  
(10, 3, '2024-03-21', '주의', 74, 102),  
(11, 3, '2024-04-22', '정상', 90, NULL),  
(12, 3, '2024-05-23', '불량', 52, 103),  
(13, 4, '2024-01-28', '정상', NULL, 104),  
(14, 4, '2024-03-29', '정상', 88, 104),  
(15, 4, '2024-05-30', '주의', 69, NULL);
```

```

INSERT INTO 조직 VALUES
  ( 1, '조립1라인', NULL),
  ( 2, '가공2라인', NULL),
  (10, '투입공정', 1),
  (11, '조립공정', 1),
  (12, '검사공정', 1),
  (20, '절삭공정', 2),
  (21, '용접공정', 2),
  (100, '컨베이어 1호', 10),
  (101, '프레스 1호', 11),
  (102, '검사기 1호', 12),
  (200, '선반 1호', 20),
  (201, '용접로봇 1호', 21),
  (202, '용접로봇 2호', 21);
`);

```

채점 도구

결과를 한 줄짜리 글자로 눌러서 기대값과 그대로 비교합니다. · NULL 은 "NULL" 로 · DATE 는 "2024-01-05" 로 (Date 객체로 오기 때문에 다듬습니다) · BIGINT(COUNT 결과) 는 "15" 로 · 칸 사이는 쉼표, 줄 사이는 " | "

```

function 값글자(값) {
  if (값 === null) return "NULL";
  if (값 instanceof Date) return 값.toISOString().slice(0, 10);
  return String(값);
}

function 눌러쓰기(줄들) {
  return 줄들.map((줄) => Object.values(줄).map(값글자).join(",")).join(" | ");
}

const 문제 = [];
let 맞힌수 = 0;

async function 채점(번호) {
  const { 제목, 답, 기대 } = 문제[번호 - 1];

  if (답.includes("TODO")) {
    console.log(`${번호}. ${제목} - 아직 안 풀었습니다`);
    return;
  }

  let 내것;

  try {
    내것 = 눌러쓰기((await db.query(답)).rows);
  } catch (에러) {
    console.log(`${번호}. ${제목} - SQL 에러 (${에러.code})`);
    return;
  }

  if (내것 === 기대) {
    맞힌수 += 1;
    console.log(`${번호}. ${제목} - 정답`);
  } else {
    console.log(`${번호}. ${제목} - 오답`);
  }
}

```

문제 1 — 두 표 잇기

설비 1번과 2번의 점검기록을 봅니다. 설비 **이름**이 같이 나와야 합니다.

칸 순서 : 설비이름, 점검일, 결과 줄 순서 : 점검번호 오름차순 대상 : 설비번호 1, 2 만

(개념01 — INNER JOIN)

```
문제.push({
  제목: "두 표 잇기",
  기대: "컨베이어 1호,2024-01-05,정상 | 컨베이어 1호,2024-02-05,정상 | 컨베이어 1호,2024-03-06,주의 | 컨베이어 1호,2024-04-05,정상 | 프레스 1호,2024-01-12,주의 | 프레스 1호,2024-02-14,불량 | 프레스 1호,2024-03-15,정상",
  답: `
    -- TODO: 점검기록과 설비를 이어서 설비 이름 · 점검일 · 결과를 뽑으세요
  `,
});
```

문제 2 — 세 표 잇기와 별칭

점검번호 1~6 에 대해 점검번호 · 설비이름 · 담당자이름 · 라인이름을 뽑습니다.

칸 순서: 점검번호, 설비이름, 담당자이름, 라인이름 줄 순서: 점검번호 오름차순 대상: 점검번호 6 이하

★ 설비에도 이름 칸이 있고 작업자에도 이름 칸이 있습니다. 별칭(alias)을 안 쓰면 “이름이 어느 쪽 것이냐”는 애러가 납니다.

(개념01 — 별칭, 여러 표 잇기)

```
문제.push({
  제목: "세 표 잇기와 별칭",
  기대: "1,컨베이어 1호,김반장,조립1라인 | 2,컨베이어 1호,김반장,조립1라인 | 3,컨베이어 1호,이기사,조립1라인 | 4,컨베이어 1호,김반장,조립1라인 | 5,프레스 1호,이기사,조립1라인 | 6,프레스 1호,박주임,조립1라인",
  답: `
    -- TODO: 점검기록 · 설비 · 라인 · 작업자 네 표를 이으세요
  `,
});
```

문제 3 — 짝이 없는 것만 찾기

점검을 한 번도 받지 않은 설비를 찾으세요.

칸 순서: 설비번호, 설비이름 줄 순서: 설비번호 오름차순

★ INNER JOIN 으로는 절대 못 찾습니다. 짝이 없어서 사라지니까요.

(개념02 — 짝이 없는 것만 찾기)

```
문제.push({
  제목: "짝이 없는 것만 찾기",
  기대: "5,포장기 1호",
  답: `
    -- TODO: LEFT JOIN 하고, 오른쪽 표의 기본키가 NULL 인 줄만 남기세요
  `,
});
```

문제 4 — 0건도 나오는 집계

설비 5대 **전부**에 대해 점검 건수를 세세요. 점검이 없는 설비는 0 이어야 합니다.

칸 순서 : 설비이름, 점검건수 줄 순서 : 설비번호 오름차순

★ **COUNT(*)** 로 세면 포장기 1호가 **1**로 나옵니다. 왜 그런지 생각해 보세요.

(개념02 LEFT JOIN + 개념03 GROUP BY)

```
문제.push({
  제목: "0건도 나오는 집계",
  기대: "컨베이어 1호,4 | 프레스 1호,3 | 용접로봇 1호,5 | 검사기 1호,3 | 포장기 1호,0",
  답: `
    -- TODO: 설비를 왼쪽에 두고 LEFT JOIN 한 뒤 GROUP BY 로 묶으세요
  `,
});
```

문제 5 — COUNT 세 가지

점검기록 표에서 세 가지를 한 줄로 뽑으세요.

① 전체 줄 수 ② 점수가 들어 있는 줄 수 ③ 서로 다른 점수의 가짓수

칸 순서 : 전체, 점수있음, 서로다른점수 줄 수 : 1줄

(개념03 — COUNT 세 가지)

```
문제.push({
  제목: "COUNT 세 가지",
  기대: "15,13,10",
  답: `
    -- TODO: COUNT(*), COUNT(칸), COUNT(DISTINCT 칸) 을 한 줄에 쓰세요
  `,
});
```

문제 6 — HAVING 으로 그룹 거르기

점검을 **4건 이상** 받은 설비만 뽑으세요.

칸 순서 : 설비이름, 건수 줄 순서 : 설비번호 오름차순

★ **WHERE COUNT(*) >= 4** 는 에러입니다. 왜 그런지는 개념03 의 WHERE 와 HAVING 을 보세요.

(개념03 — WHERE 와 HAVING)

```
문제.push({
  제목: "HAVING 으로 그룹 거르기",
  기대: "컨베이어 1호,4 | 용접로봇 1호,5",
  답: `
    -- TODO: GROUP BY 로 묶고 HAVING 으로 거르세요
  `,
});
```

문제 7 — ★★★★★ LEFT JOIN 과 조건의 자리

설비 5대 **전부**에 대해 '불량' 점검이 몇 건인지 세세요. 불량이 하나도 없는 설비는 0 이어야 합니다.

칸 순서: 설비이름, 불량건수 줄 순서: 설비번호 오름차순

★★★★ 여기가 이 단원에서 가장 많이 틀리는 곳입니다. LEFT JOIN 점검기록 p ON ... WHERE p.결과 = '불량' 이라고 쓰면 LEFT JOIN 이 **INNER JOIN 이 되어 버립니다**. 설비 3대가 사라집니다. 조건을 어디에 써야 할까요?

(개념02 — LEFT JOIN 뒤의 WHERE 함정)

```
문제.push({
  제목: "LEFT JOIN 과 조건의 자리",
  기대: "컨베이어 1호,0 | 프레스 1호,1 | 용접로봇 1호,1 | 검사기 1호,0 | 포장기 1호,0",
  답: `
    -- TODO: '불량' 조건을 WHERE 가 아니라 어디에 넣어야 할까요
  `,
});
```

문제 8 — ★★★★★ 담당한 적 없는 작업자

점검을 **한 번도 담당한 적 없는** 작업자를 찾으세요.

칸 순서: 사번, 이름 줄 순서: 사번 오름차순

★★★★ WHERE 사번 NOT IN (SELECT 담당사번 FROM 점검기록) 은 **0건**이 나옵니다. 에러도 안 납니다. 담당사번에 NULL 이 섞여 있기 때문입니다. 안전한 방법으로 쓰세요.

(개념04 — NOT IN 함정)

```
문제.push({
  제목: "담당한 적 없는 작업자",
  기대: "105,정신입",
  답: `
    -- TODO: NOT IN 말고 안전한 것을 쓰세요
  `,
});
```

문제 9 — FILTER 로 조건별 세기

라인별로 정상 · 주의 · 불량 건수를 **한 줄씩** 뽑으세요.

칸 순서 : 라인코드, 정상, 주의, 불량 줄 순서 : 라인코드 오름차순 대상 : 점검기록이 있는 라인만 (A, B)

(개념03 — FILTER)

```
문제.push({
  제목: "FILTER 로 조건별 세기",
  기대: "A,4,2,1 | B,5,2,1",
  답: `
    -- TODO: COUNT(*) FILTER (WHERE ...) 를 세 번 쓰세요
  `,
});
```

문제 10 — 평균보다 높은 점검

점수가 **전체 평균보다 높은** 점검을 뽑으세요.

칸 순서 : 점검번호, 점수 줄 순서 : 점검번호 오름차순

★ 평균이 얼마인지 미리 계산해서 숫자로 박아 넣으면 안 됩니다. 데이터가 바뀌면 틀린 답이 됩니다.
서브쿼리로 쓰세요.

(개념04 — 스칼라 서브쿼리)

```
문제.push({
  제목: "평균보다 높은 점검",
  기대: "1,92 | 2,88 | 4,90 | 7,88 | 8,95 | 11,90 | 14,88",
  답: `
    -- TODO: WHERE 점수 > (평균을 구하는 서브쿼리)
  `,
});
```

문제 11 — CTE 로 최고 라인 찾기

라인별 평균 점수를 구하고, 그중 **가장 높은 라인 한 줄**만 뽑으세요.

칸 순서 : 라인코드, 평균점수 줄 수 : 1줄 평균점수: 소수점 둘째 자리까지 (ROUND(..., 2)) 대상 :
점검기록이 있는 라인만

★ WITH 로 라인별 평균을 먼저 만들어 두면 읽기 쉽습니다. ★ 평균점수는 NUMERIC 이라 **문자열**로
옵니다. "78.00" 처럼 나옵니다.

(개념03 AVG 와 NUMERIC + 개념04 CTE)


```
문제.push({
  제목: "CTE 로 최고 라인 찾기",
  기대: "B,78.00",
  답: `
    -- TODO: WITH 라인평균 AS (...) 로 시작해 보세요
  `,
});
```

문제 12 — RANK 로 순위 매기기

설비 1번과 2번의 점검을 점수 높은 순으로 순위를 매기세요. **동점은 같은 순위**입니다.

칸 순서: 점검번호, 점수, 순위 줄 순서: 순위 오름차순, 같으면 점검번호 오름차순 대상: 설비번호 1, 2 만

★ 점수 88 이 두 건 있습니다. 둘 다 3위가 되고, 그다음은 4위가 아니라 **5위**입니다. 그렇게 되는 함수를 고르세요.

(개념05 — 순위 함수 세 가지)

```
문제.push({
  제목: "RANK 로 순위 매기기",
  기대: "1,92,1 | 4,90,2 | 2,88,3 | 7,88,3 | 3,71,5 | 5,65,6 | 6,48,7",
  답: `
    -- TODO: 순위를 매기는 윈도우 함수를 쓰세요
  `,
});
```

문제 13 — [도전] 설비마다 점수 상위 2건

설비마다 점수가 높은 점검 2건씩 뽑으세요.

칸 순서: 설비번호, 점검번호, 점수 줄 순서: 설비번호 오름차순, 그 안에서 순위 오름차순 대상: 점검기록이 있는 설비 (1~4)

★ **LIMIT 2** 는 전체에서 2건입니다. 설비마다 2건이 아닙니다. ★ 점수가 NULL 인 점검(9번, 13번)은 맨 뒤로 보내세요 (**NULLS LAST**). ★ 점수가 같으면 점검번호가 작은 쪽을 위로 하세요. (안 그러면 답이 매번 달라집니다) ★ 윈도우 함수는 WHERE 에서 못 씁니다. 한 겹 감싸야 합니다.

(개념05 — 그룹마다 상위 N개)

```
문제.push({
  제목: "[도전] 설비마다 점수 상위 2건",
  기대: "1,1,92 | 1,4,90 | 2,7,88 | 2,5,65 | 3,8,95 | 3,11,90 | 4,14,88 | 4,15,69",
  답: `
    -- TODO: ROW_NUMBER 로 설비마다 번호를 매기고, 서브쿼리로 감싸서 걸러 내세요
  `,
});
```

문제 14 — [도전] 점검 간격 구하기

설비 3번의 점검이 **며칠 만에** 이루어졌는지 구하세요.

칸 순서 : 점검번호, 설비번호, 간격 줄 순서 : 점검일 오름차순 대상 : 설비번호 3 만

★ 첫 점검은 앞이 없으니 간격이 NULL 입니다. 그대로 두세요. ★ DATE 끼리 빼면 **일수(정수)** 가 나옵니다.

(개념05 — LAG 와 LEAD)

```
문제.push({
  제목: "[도전] 점검 간격 구하기",
  기대: "8,3,NULL | 9,3,31 | 10,3,30 | 11,3,32 | 12,3,31",
  답: `
    -- TODO: 이전 줄의 값을 가져오는 윈도우 함수를 쓰세요
  `,
});
```

문제 15 — [도전] 계층 훑기

조직 표는 라인 → 공정 → 설비 로 3단계입니다. **3단계(설비)** 만 뽑되, 맨 위에서부터의 경로를 함께 보이세요.

칸 순서 : 깊이, 경로 줄 순서 : 경로 오름차순 경로 모양: "조립1라인 > 투입공정 > 컨베이어 1호" (구분자는 ">") 깊이 : 맨 위가 1

★ **WITH RECURSIVE** 를 씁니다. ★ 경로를 이을 때 첫 줄에서 **이름::TEXT** 로 타입을 맞춰야 합니다. 안 그러면 "recursive query column ... has type character varying" 에러가 납니다. ★ 깊이 제한(**WHERE 깊이 < 5**)을 꼭 거세요. 순환 참조가 생기면 영원히 돌립니다.

(개념04 — WITH RECURSIVE)

```
문제.push({
  제목: "[도전] 계층 훑기",
  기대: "3,가공2라인 > 용접공정 > 용접로봇 1호 | 3,가공2라인 > 용접공정 > 용접로봇 2호 | 3,가공2라인 > 절삭공정 > 선반 1호 | 3,조립1라인 > 검사공정 > 검사기 1호 | 3, 조립1라인 > 조립공정 > 프레스 1호 | 3,조립1라인 > 투입공정 > 컨베이어 1호",
  답: `
    -- TODO: WITH RECURSIVE 훑기 AS ( 기준줄 UNION ALL 재귀줄 ) SELECT ...
  `,
});
```

채점

```
console.log("— 05단원 연습문제 채점 —");
```

```
출력      — 05단원 연습문제 채점 —
```

```
for (let 번호 = 1; 번호 <= 문제.length; 번호 += 1) {
  await 채점(번호);
}
```

출력

1. 두 표 잇기 - 아직 안 풀었습니다
2. 세 표 잇기와 별칭 - 아직 안 풀었습니다
3. 짝이 없는 것만 찾기 - 아직 안 풀었습니다
4. 0건도 나오는 집계 - 아직 안 풀었습니다
5. COUNT 세 가지 - 아직 안 풀었습니다
6. HAVING 으로 그룹 거르기 - 아직 안 풀었습니다
7. LEFT JOIN 과 조건의 자리 - 아직 안 풀었습니다
8. 담당할 적 없는 작업자 - 아직 안 풀었습니다
9. FILTER 로 조건별 세기 - 아직 안 풀었습니다
10. 평균보다 높은 점검 - 아직 안 풀었습니다
11. CTE 로 최고 라인 찾기 - 아직 안 풀었습니다
12. RANK 로 순위 매기기 - 아직 안 풀었습니다
13. [도전] 설비마다 점수 상위 2건 - 아직 안 풀었습니다
14. [도전] 점검 간격 구하기 - 아직 안 풀었습니다
15. [도전] 계층 훑기 - 아직 안 풀었습니다

```
console.log(`— ${문제.length}문제 중 ${맞힌수}문제 정답 —`);
```

출력 — 15문제 중 0문제 정답 —

막혔을 때 —————

힌트 1 · 결과가 예상보다 **많이** 나온다

1:N 을 이어서 왼쪽 줄이 반복된 것입니다. (개념01 — 1:N 을 이으면 줄이 늘어납니다) 세는 것이 목적이라면 `COUNT(DISTINCT ...)` 나 `GROUP BY` 를 쓰세요.

힌트 2 · 결과가 예상보다 **적게** 나온다

① `INNER JOIN` 이라 짝 없는 것이 빠졌거나 (개념02 — `INNER JOIN` 이 빠뜨리는 것) ② `LEFT JOIN` 뒤 `WHERE` 가 다시 죽였거나 (개념02 — `LEFT JOIN` 뒤의 `WHERE` 함정) ③ `NULL` 이 섞인 `NOT IN` 이거나 (개념04 — `NOT IN` 함정)

힌트 3 · "column ... must appear in the GROUP BY clause" 에러

묵지 않은 칸을 `SELECT` 했습니다. `GROUP BY` 에 넣거나 집계로 감싸세요. (개념03 — `GROUP BY` 에 없는 칸)

힌트 4 · "window functions are not allowed in WHERE" 에러

윈도우 함수는 WHERE 보다 늦게 계산됩니다. 서브쿼리나 CTE 로 한 겹 감싸세요. (개념05 — 그룹마다 상위 N개)

힌트 5 · 줄 순서가 매번 다르다

ORDER BY 를 안 썼거나, 동점을 가를 칸이 없습니다. 순서를 정하고 싶으면 **유일한 칸**까지 ORDER BY 에 넣으세요.

힌트 6 · "오답" 인데 눈으로 보면 맞는 것 같다

칸 **순서**가 다를 수 있습니다. 문제에 적힌 칸 순서를 그대로 맞추세요. 숫자가 문자열로 오는 경우도 있습니다 (NUMERIC → "78.00").

```
await db.close();
```

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

01단원 연습문제 정답 왜 데이터베이스인가

RUN node 연습문제_정답.js

★ 문제를 다 풀어 본 다음에 보세요. 답만 베끼면 남는 게 없습니다.

★ 채점에 몇 초 걸립니다. 13번이 10만 건을 진짜로 넣고 잡니다.

답만 적어 두지 않았습니다. 왜 그런지와 어디서 틀리기 쉬운지를 적었습니다. 맞힌 문제도 설명은 읽어 보세요. 맞힌 이유가 틀린 경우가 많습니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();
```

실습용 데이터 (연습문제.js 와 같습니다)

```
await db.exec(`
  CREATE TABLE 설비 (
    id INT PRIMARY KEY, 이름 TEXT NOT NULL, 라인 TEXT NOT NULL, 상태 TEXT NOT NULL
  );
  INSERT INTO 설비 VALUES
    (1, '컨베이어 1호', 'A', '가동'), (2, '컨베이어 2호', 'A', '정지'),
    (3, '프레스 1호', 'B', '가동'), (4, '프레스 2호', 'B', '가동'),
    (5, '용접로봇 1호', 'C', '정지'), (6, '도장기 1호', 'A', '가동');

  CREATE TABLE 점검기록 (
    id SERIAL PRIMARY KEY, 설비명 TEXT NOT NULL, 라인 TEXT NOT NULL,
    점검자 TEXT NOT NULL, 소요분 INT NOT NULL, 결과 TEXT NOT NULL
  );
  INSERT INTO 점검기록 (설비명, 라인, 점검자, 소요분, 결과) VALUES
    ('컨베이어 1호', 'A', '김반장', 35, '정상'), ('컨베이어 2호', 'A', '김반장', 50,
    '이상'),
    ('프레스 1호', 'B', '이반장', 20, '정상'), ('프레스 2호', 'B', '이반장', 65,
    '이상'),
    ('용접로봇 1호', 'C', '박반장', 15, '정상'), ('용접로봇 2호', 'C', '박반장', 80,
    '이상'),
    ('도장기 1호', 'A', '김반장', 45, '정상');

  CREATE TABLE 설비사양 (id INT PRIMARY KEY, 이름 TEXT NOT NULL, 사양 JSONB NOT
  NULL);
  INSERT INTO 설비사양 VALUES
```

```
(2, '프레스 1호', '{"제조사":"대동프레스","압력톤":250}');
`);

const 답 = {};
```

문제 1 — 설비를 전부 가져오기

```
답.문제1 = "SELECT * FROM 설비 ORDER BY id";
```

★★ **ORDER BY id** 를 왜 붙였을까요? **SELECT * FROM 설비** 만 해도 지금은 1~6 순서로 나옵니다. 그런데 그건 보장이 아닙니다. SQL 표준에서 ORDER BY 없는 결과의 순서는 **정해져 있지 않습니다**. 지금은 넣은 순서대로 저장돼 있어서 그렇게 나올 뿐입니다. 누가 4번 줄을 UPDATE 하면 그 줄이 맨 뒤로 갈 수도 있습니다. (PostgreSQL 의 UPDATE 는 원래 줄을 지우고 새 줄을 뒤에 붙이는 방식입니다)

★★★ 순서가 중요하면 반드시 **ORDER BY** 를 적으세요.

"테스트에서는 잘 됐는데 운영에서 순서가 뒤집혔다" 는 사고가 여기서 납니다.

★ **SELECT *** 는 연습용입니다. 진짜 코드에서는 필요한 칸만 적으세요. 나중에 칸이 하나 추가되면 * 는 그것까지 실어 나릅니다.

문제 2 — 조건으로 거르고 순서 정하기

```
답.문제2 = "SELECT 이름 FROM 설비 WHERE 라인 = 'A' ORDER BY id";
```

★★ 틀리기 쉬운 곳 ①: 등호가 하나입니다. 자바스크립트는 **===** 지만 SQL 은 **=** 하나입니다. **==** 는 42601 문법 에러입니다.

★★ 틀리기 쉬운 곳 ②: 값은 작은따옴표입니다. **WHERE 라인 = "A"** 라고 쓰면 PostgreSQL 은 "A" 를 칸 이름으로 봅니다. 그래서 42703 (column "A" does not exist) 이 납니다.

작은따옴표 'A' → 글자 값
큰따옴표 "A" → 칸이나 표의 이름

자바스크립트에서는 둘이 같아서 여기서 정말 많이 헛갈립니다.

★ **ORDER BY** 에 쓴 **id** 는 **SELECT** 에 없습니다. 그래도 됩니다. 보여 줄 칸과 정렬할 칸은 별개입니다.

문제 3 — 세기

답.문제3 = "SELECT count(*)::int AS 건수 FROM 설비 WHERE 상태 = '가동';";

★★★ ::int 를 왜 붙였나요? 이 문제의 진짜 요점입니다. count(*) 없이 그냥 써도 이 문제는 통과합니다. 값이 4 라서 그렇습니다. 그게 함정입니다.

count(*) 의 타입은 **BIGINT** 이고, PGLite 는 BIGINT 를 이렇게 줍니다.

- 안전 정수(2의 53승 - 1) 안이면 → 그냥 `number`
- 그 범위를 넘으면 → `bigint` (`9007199254740993n` 처럼 n 이 붙음)

그리고 bigint 는 `JSON.stringify` 가 처리하지 못합니다.

```
JSON.stringify({건수: 9007199254740993n})
→ Do not know how to serialize a BigInt
```

★★★ 작을 때는 멀쩡하다가 커지면 터집니다.

테스트 데이터로는 절대 안 걸립니다. 운영에서 몇 년 뒤에 처음 납니다.
그래서 처음부터 `::int` 로 못을 박아 두는 것입니다.
(`CAST(count(\*) AS INT)` 와 같은 뜻입니다)

★ 드라이버마다 다르기도 합니다. 08단원에서 pg 로 진짜 서버에 붙을 때

BIGINT 가 또 어떻게 오는지 봅니다. `::int` 를 붙여 두면 어디서든 같습니다.
02단원에서 BIGINT·NUMERIC 을 정면으로 다룹니다.

★ count(*) 와 count(칸이름) 은 다릅니다. count(*) 는 줄 수, count(상태) 는 상태가 NULL 이 아닌 줄 수입니다.

문제 4 — 정렬하고 자르기

답.문제4 = "SELECT 설비명 FROM 점검기록 ORDER BY 소요분 DESC LIMIT 3";

★ DESC 는 내림차순, ASC 는 오름차순. 안 쓰면 ASC 입니다.

★★ ORDER BY 없이 LIMIT 만 쓰면 안 됩니다. 그러면 "아무 3건" 이 나옵니다. "위 3건" 이 아닙니다.
LIMIT 은 자르기만 합니다. 무엇을 자를지는 ORDER BY 가 정합니다.

★ 실행 순서로 보면 이렇습니다.


```
FROM (7건) → ORDER BY (80,65,50,45,35,20,15) → LIMIT 3 (80,65,50)
```

LIMIT 이 맨 마지막이라 정렬 다음에 잘립니다.

★★ 색인이 없으면 3건만 필요해도 7건을 **전부 정렬합니다**. 10만 건이면 10만 건을 정렬하고 3건만 씁니다. 06단원에서 이걸 고칩니다.

문제 5 — 묶어서 세기

```
답.문제5 =
"SELECT 점검자, count(*)::int AS 건수 FROM 점검기록 GROUP BY 점검자 ORDER BY
점검자";
```

★ GROUP BY 점검자 는 “점검자가 같은 줄끼리 묶어라” 입니다. 7줄이 3덩어리(김반장 3, 이반장 2, 박반장 2)가 됩니다.

★★★ GROUP BY 를 쓰면 SELECT 에 아무 칸이나 못 씁니다. 쓸 수 있는 것은 두 가지뿐입니다.

- ① GROUP BY 에 적은 칸 (여기서는 점검자)
- ② 집계 함수 (count, sum, avg, min, max ...)

SELECT 점검자, 설비명, count(*) ... GROUP BY 점검자 는 42803 입니다. 김반장 덩어리 안에 설비명이 셋인데 무엇을 보여 줘야 할지 알 수 없으니까요.

★ MySQL 은 설정에 따라 **아무거나 하나 골라서 보여 줍니다**.

에러가 안 나니 더 위험합니다. 09단원에서 직접 봅니다.

★ 김반장 → 박반장 → 이반장 은 한글 사전 순서(ㄱ, ㅂ, ㅇ)입니다.

문제 6 — 묶은 것 중에서 고르기

```
답.문제6 =
"SELECT 점검자 FROM 점검기록 GROUP BY 점검자 HAVING avg(소요분) >= 40 ORDER BY
avg(소요분) DESC";
```

★ 계산해 보면 김반장 43.33 / 이반장 42.5 / 박반장 47.5 입니다. 셋 다 40 이상이라 전부 나오고, 큰 순서로 박반장 → 김반장 → 이반장 입니다.

★★★ 여기가 이 단원에서 가장 많이 틀리는 곳입니다.

- ① WHERE avg(소요분) >= 40 이라고 쓰면?

→ 42803 "aggregate functions are not allowed in WHERE"

WHERE 는 GROUP BY 보다 ****먼저**** 둡니다. 아직 안 묶였으니 평균이 없습니다.

② SELECT ... avg(소요분) AS 평균 ... HAVING 평균 >= 40 이라고 쓰면?

→ 42703 "column 평균 does not exist"

HAVING 도 SELECT 보다 ****먼저**** 둡니다. 별칭이 아직 없습니다.

③ 그런데 ORDER BY 에서 별칭을 쓰면? → **됩니다**. SELECT 다음에 돌기 때문입니다.

★★ 전부 이 한 줄로 설명됩니다.

****FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT****

★ WHERE 와 HAVING 을 같이 쓰는 것도 혼합니다.

WHERE 결과 = '이상' ← 먼저 줄을 거르고

GROUP BY 점검자 ← 묶고

HAVING count(*) >= 2 ← 묶음을 거른다

★★ 성능 면에서는 **WHERE** 로 먼저 거르는 게 유리합니다. 묶을 줄이 줄어듭니다.

문제 7 — 에러 코드 맞추기

답.문제7 = { 오타: "42601", 없는표: "42P01", WHERE별칭: "42703" };

★ 하나씩 봅시다. ① SELEC * FROM 설비 → 42601 syntax error at or near "SELEC"

****문법 에러****입니다. "at or near" 뒤의 단어가 힌트인데,
진짜 원인은 그 ****직전****일 때가 많습니다 (coma 빠짐, 괄호 안 닫음).

② SELECT * FROM 없는표 → 42P01 relation "없는표" does not exist

``relation`` 은 '표' 라는 뜻입니다.

③ 별칭을 WHERE 에서 씀 → 42703 column "소요초" does not exist

★★ 앞 두 글자로 갈래를 잡으세요.

42 → 내가 문장을 잘못 썼습니다 (문법·이름·규칙)
23 → 문장은 맞는데 제약에 걸렸습니다 (중복·NULL·외래키)
22 → 값의 타입이나 범위가 문제입니다

★ 메시지가 아니라 코드로 검색하세요. 메시지에는 내 표 이름이 섞여 있고, 버전에 따라 바뀝니다. **코드는 안 바뀝니다.**

문제 8 — 실행 순서

답.문제8 = ["FROM", "WHERE", "GROUP BY", "HAVING", "SELECT", "ORDER BY", "LIMIT"];

★★★ 이 자료에서 **가장 중요한 한 줄**입니다.

적는 순서: SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT 실행
순서: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

차이는 **SELECT** 의 위치 하나입니다. 적을 때는 맨 앞, 돌 때는 뒤에서 세 번째입니다.

★ 서류 정리에 비유하면 자연스럽습니다.

- ① 어느 서류함에서 (FROM) → ② 필요 없는 종이를 버리고 (WHERE)
- ③ 비슷한 것끼리 쌓고 (GROUP BY) → ④ 작은 더미는 버리고 (HAVING)
- ⑤ 각 더미에서 볼 항목을 적고 (SELECT) → ⑥ 늘어놓고 (ORDER BY)
- ⑦ 위에서 몇 장만 가져옵니다 (LIMIT)

서류를 다 정리하기 전에 “무엇을 적을지” 는 정할 수 없습니다. 그래서 SELECT 가 뒤입니다.

★ 데이터베이스는 실제로 이 순서를 그대로 따르지 않습니다. 더 빠른 순서로 바뀌서 돕니다. **결과만 이 순서대로 한 것과 같게** 보장할 뿐입니다. 06단원에서 봅니다.

문제 9 — 별칭이 통하는 곳

답.문제9 = "ORDER BY";

★ 문제 8 의 순서에서 **SELECT** 보다 뒤에 있는 것만 별칭을 씁니다.

★★ 헛갈리는 게 하나 있습니다. **GROUP BY** 에서도 별칭이 됩니다.

SELECT 라인 AS 공정, count(*) FROM 점검기록 GROUP BY 공정 ← 이게 됩니다

순서상 GROUP BY 가 SELECT 보다 먼저인데 왜 될까요? PostgreSQL 이 편의로 봐 주는 것입니다. 표준 SQL 에는 없는 동작입니다.

★★★ 의존하지 마세요. 다른 데이터베이스로 옮기면 깨집니다.

별칭과 같은 이름의 칸이 있으면 어느 쪽인지도 헷갈립니다.

문제 10 — JSONB 안을 들여다보기

답.문제10 = "SELECT 이름 FROM 설비사양 WHERE 사양->>'제조사' = '한성기계'";

★ 이것도 정답입니다.

SELECT 이름 FROM 설비사양 WHERE 사양 @> '{"제조사":"한성기계"}'

★★ 두 방식의 차이 사양->>'제조사' = '한성기계' 키 하나를 꺼내 글자로 비교. 읽기 쉽습니다 사양 @> '{...}' "이 JSON 을 품고 있느냐". 조건 여러 개를 한 번에

넣을 수 있고, JSONB 색인(GIN)이 이 연산자를 탐니다

★★★ 화살표 개수에 주의하세요.

`->` 결과가 **JSON**, `'"한성기계"'` 처럼 따옴표까지 딸려옵니다
`->>` 결과가 **글자(text)**, `한성기계` 입니다

사양->'제조사' = '한성기계' 는 왼쪽이 JSON 이라 오른쪽도 JSON 으로 읽으려 합니다. 그런데 한성기계 는 따옴표 없는 글자라 JSON 이 아닙니다. → 22P02 invalid input syntax for type json (직접 재 본 값입니다)

★ ->> 로 꺼낸 숫자는 문자열입니다. (사양->>'최대속도')::int 로 바꿔야 계산됩니다.

문제 11 — query 인가 exec 인가

답.문제11 = "exec";

★ db.query() 에 두 문장을 넣으면 이렇게 납니다.

[42601] cannot insert multiple commands into a prepared statement

★★ 왜일까요? query 는 문장을 **준비된 문장(prepared statement)** 으로 만듭니다. 값이 들어갈 자리 (\$1, \$2)를 미리 뚫어 놓고 값은 따로 보내는 방식입니다. 그래야 값이 SQL 문법으로 해석되지 않아 인젝션이 막힙니다. 그런데 준비된 문장은 **한 번에 하나만** 만들 수 있습니다.

★★★ 그래서 규칙이 이렇게 갑니다.

```
`db.exec()`   여러 문장 0 . **$1 못 씀** . 표 만들기·실습 데이터 넣기용
`db.query()` 한 문장만 . $1 씀 . **사용자 값이 들어가는 모든 곳**
```

★ 값이 들어가면 무조건 query + \$1 입니다. 예외는 없습니다.

exec 로 사용자 값을 보내면 인젝션이 뚫립니다. 08단원에서 진짜로 뚫어 보니다.

MySQL 은 여기가 다릅니다

· 자리 표시가 \$1 이 아니라 ? 입니다 ★ 자세한 비교는 09단원

문제 12 [도전] — 묶고, 거르고, 타입까지 맞추기

```
답.문제12 = `
SELECT 라인,
       count(*)::int          AS 건수,
       round(avg(소요분), 1)::float AS 평균분
FROM   점검기록
GROUP BY 라인
HAVING count(*) >= 2
ORDER BY 라인
`;
```

★ 답: A — 3건 43.3 / B — 2건 42.5 / C — 2건 47.5

★★★ 이 문제의 함정은 ::float 입니다. round(avg(소요분), 1) 의 타입은 **NUMERIC** 이고, NUMERIC 은 자바스크립트로 올 때 **문자열**이 됩니다. “43.3” 처럼요.

```
"43.3" === 43.3 → false
"43.3" + 1      → "43.31" ← 숫자가 아니라 이어 붙습니다
```

왜 문자열일까요? NUMERIC 은 **정확한 십진 소수**입니다. 자바스크립트의 number 는 이진 부동소수점이라 정확히 담을 수 없습니다. (0.1 + 0.2 가 0.30000000000000004 가 되는 그 문제입니다) 그래서 드라이버가 “정확도를 깨뜨리느니 문자열로 주겠다” 고 하는 것입니다.

★★ 돈 계산에서는 절대 ::float 를 붙이지 마세요.

여기서는 채점 편의로 숫자를 요구했지만,
금액·수량은 문자열로 받아 정확한 십진 계산으로 다뤄야 합니다. 02단원에서 합니다.

★ HAVING count(*) >= 2 에서 count(*) 를 또 썼습니다. 별칭은 못 쓰니까요. 이 데이터에서는 HAVING 이 아무것도 안 버리지만, 그래도 붙여야 합니다. “지금 결과가 같으니 빼도 된다” 는 위험한 생각입니다.

문제 13 [도전] — 어느 쪽이 빠를지 예측하기

답.문제13 = "id";

★ id 가 훨씬 빠릅니다. 채점기가 실제로 재서 확인합니다.

★★ id 에는 색인이 있고 이름에는 없기 때문입니다. id INT PRIMARY KEY 라고 적으면 PostgreSQL 이 큰 표_pkey 같은 색인을 자동으로 만듭니다. 따로 만들지 않아도 생깁니다.

id 로 찾기 → 색인을 타고 17번쯤 비교하면 도착
이름으로 찾기 → 10만 줄을 처음부터 끝까지 훑음

★ 개념02 에서 본 그 60배 차이가 바로 이것입니다.

`WHERE id = 77777` 0.25 ms ← 색인 있음
`WHERE 라인='A' AND 상태='가동'` 16 ms ← 색인 없음

★★ 그럼 모든 칸에 색인을 만들면 될까요? 안 됩니다. 색인은 공짜가 아닙니다.

- 자리를 차지합니다
- **넣고 고치고 지울 때마다 색인도 같이 고쳐야 합니다** (쓰기가 느려집니다)
- 색인이 있어도 데이터베이스가 안 쓰기로 결정할 때가 있습니다

어디에 만들고, 만든 색인을 진짜 쓰는지 확인하는 법은 06단원에서 합니다.

★ 여러분 컴퓨터에서는 숫자가 다르게 나옵니다. 그런데 "id 가 빠르다" 는 안 뒤집힙니다. 측정값은 다르고 판정은 같아야 한다 — 개념05 에서 본 그 원칙입니다.

문제 14 [도전] — 오타 난 JSON 키

답.문제14 = "null";

★★★ 예러가 안 납니다. 조용히 null 이 됩니다. 이게 정답이자 요점입니다. JSONB 에는 "이 키가 있어야 한다" 는 규칙이 없습니다. 압력톤수 라고 오타를 내도 아무도 안 막고, ->> 는 "없으면 null" 이라고 답합니다.

★ 칸으로 만들었다면 어땠을까요?

INSERT INTO 설비 (압력톤수) VALUES (300)
→ 42703 column "압력톤수" of relation "설비" does not exist

넣는 순간 거절당합니다. 오타를 그 자리에서 잡습니다.

★★ 이게 개념01 에서 말한 **조용한 버그**입니다. 에러도 로그도 없습니다. 보고서에 압력톤이 빈칸으로 나오고, 몇 달 뒤에 누가 “이거 왜 비어 있죠?” 합니다. 그때는 원인 시점도 못 찾습니다.

★ 막는 방법

- 애초에 ****칸으로 만듭니다**** (제일 확실합니다)
- 넣을 때 애플리케이션에서 검사합니다 (사람이 빼먹을 수 있습니다)
- 제약을 겁니다: ``CHECK (사양 ? '압력톤')`` (02단원에서 합니다)

★★ 결론은 개념03 과 같습니다. 틀리면 안 되는 것은 칸으로, 자주 바뀌는 부가 정보만 JSONB 로.

채점 코드 (연습문제.js 와 같습니다)

```

const 채점결과 = [];
const 같나 = (가, 나) => JSON.stringify(가) === JSON.stringify(나);

async function 채점(번호, 제목, 검사) {
  if (답[`문제${번호}`] === null || 답[`문제${번호}`] === undefined) {
    채점결과.push([번호, 제목, "■", "아직 안 풀었습니다"]);
    return;
  }
  try {
    const 판정 = await 검사();
    채점결과.push([번호, 제목, 판정 === true ? "○", "맞았습니다"] : [번호, 제목, "✗", 판정]);
  } catch (에러) {
    채점결과.push([번호, 제목, "✗", `에러 [${에러.code ?? "?"}] ${에러.message}`]);
  }
}

async function 준비교(번호, 기대) {
  const 실제 = (await db.query(답[`문제${번호}`])).rows;
  return 같나(실제, 기대) ? true : `나온 값: ${JSON.stringify(실제)}`;
}

const 값비교 = (번호, 기대) =>
  같나(답[`문제${번호}`], 기대) ? true : `나온 값: ${JSON.stringify(답[`문제${번호}`])}`;

await 채점(1, "설비 전부 가져오기", async () => {
  const 실제 = (await db.query(답.문제1)).rows;
  if (실제.length !== 6) return `6줄이어야 하는데 ${실제.length}줄입니다`;
  return 같나(실제[0], { id: 1, 이름: "컨베이어 1호", 라인: "A", 상태: "가동" })
    ? true
    : `첫 줄이 다릅니다: ${JSON.stringify(실제[0])}`;
});

await 채점(2, "A라인 설비 이름", () =>
  준비교(2, [{ 이름: "컨베이어 1호" }, { 이름: "컨베이어 2호" }, { 이름: "도장기 1호"
}]),
);

await 채점(3, "가동 중인 설비 수", () => 준비교(3, [{ 건수: 4 }]));
await 채점(4, "오래 걸린 점검 3건", () =>

```



```

    준비교(4, [{ 설비명: "용접로봇 2호" }, { 설비명: "프레스 2호" }, { 설비명:
"컨베이어 2호" }]),
);
await 채점(5, "점검자별 건수", () =>
    준비교(5, [
        { 점검자: "김반장", 건수: 3 },
        { 점검자: "박반장", 건수: 2 },
        { 점검자: "이반장", 건수: 2 },
    ]),
);
await 채점(6, "평균 40분 이상 점검자", () =>
    준비교(6, [{ 점검자: "박반장" }, { 점검자: "김반장" }, { 점검자: "이반장" }]),
);
await 채점(7, "에러 코드 세 개", async () =>
    값비교(7, { 오타: "42601", 없는표: "42P01", WHERE별칭: "42703" }),
);
await 채점(8, "실행 순서", async () =>
    값비교(8, ["FROM", "WHERE", "GROUP BY", "HAVING", "SELECT", "ORDER BY", "LIMIT"]),
);
await 채점(9, "별칭이 통하는 절", async () => 값비교(9, "ORDER BY"));
await 채점(10, "JSONB 로 제조사 찾기", () => 준비교(10, [{ 이름: "컨베이어 1호" }]));
await 채점(11, "query 인가 exec 인가", async () => 값비교(11, "exec"));
await 채점(12, "[도전] 라인별 건수와 평균", () =>
    준비교(12, [
        { 라인: "A", 건수: 3, 평균분: 43.3 },
        { 라인: "B", 건수: 2, 평균분: 42.5 },
        { 라인: "C", 건수: 2, 평균분: 47.5 },
    ]),
);
await 채점(13, "[도전] 어느 쪽이 빠른가", async () => {
    await db.exec(`
        CREATE TABLE 큰표 (id INT PRIMARY KEY, 이름 TEXT NOT NULL);
        INSERT INTO 큰표 SELECT 번호, '설비' || 번호 FROM generate_series(1, 100000) AS
        번호;
    `);
    const 재기 = async (문장, 값) => {
        const 잔것 = [];
        for (let 회차 = 0; 회차 < 5; 회차 += 1) {

```

```

const 시작 = performance.now();
  await db.query(문장, [값]);
  잰것.push(performance.now() - 시작);
}
return 잰것.sort((가, 나) => 가 - 나)[2];
};

const idms = await 재기("SELECT * FROM 큰표 WHERE id = $1", 77777);
const 이름ms = await 재기("SELECT * FROM 큰표 WHERE 이름 = $1", "설비77777");
const 빠른쪽 = idms < 이름ms ? "id" : "이름";
return 답.문제13 === 빠른쪽 ? true : `실제로 빠른 쪽은 "${빠른쪽}" 입니다`;
});

await 채점(14, "[도전] 오타 난 JSON 키", async () => {
  await db.exec(`INSERT INTO 설비사양 VALUES (3, '프레스 3호', '{"압력톤수":
300}')`);
  const 결과 = await db.query("SELECT 사양->>'압력톤' AS 값 FROM 설비사양 WHERE id =
3");
  const 진짜 = 결과.rows[0].값 === null ? "null" : String(결과.rows[0].값);
  return 답.문제14 === 진짜 ? true : `실제로는 "${진짜}" 입니다`;
});

console.log("===== 01단원 연습문제 정답 채점 =====");

```

```
출력      ===== 01단원 연습문제 정답 채점 =====
```

```

for (const [번호, 제목, 표시, 설명] of 채점결과) {
  console.log(`${String(번호).padStart(2, "0")}. ${표시} ${제목} - ${설명}`);
}

```

```

출력      01. ○ 설비 전부 가져오기 - 맞았습니다
          02. ○ A라인 설비 이름 - 맞았습니다
          03. ○ 가동 중인 설비 수 - 맞았습니다
          04. ○ 오래 걸린 점검 3건 - 맞았습니다
          05. ○ 점검자별 건수 - 맞았습니다
          06. ○ 평균 40분 이상 점검자 - 맞았습니다
          07. ○ 에러 코드 세 개 - 맞았습니다
          08. ○ 실행 순서 - 맞았습니다
          09. ○ 별칭이 통하는 절 - 맞았습니다
          10. ○ JSONB 로 제조사 찾기 - 맞았습니다
          11. ○ query 인가 exec 인가 - 맞았습니다
          12. ○ [도전] 라인별 건수와 평균 - 맞았습니다
          13. ○ [도전] 어느 쪽이 빠른가 - 맞았습니다
          14. ○ [도전] 오타 난 JSON 키 - 맞았습니다

```

```

const 맞은수 = 채점결과.filter(([, , 표시]) => 표시 === "○").length;

console.log(`===== ${맞은수} / ${채점결과.length} 맞았습니다 =====`);

```

```
출력      ===== 14 / 14 맞았습니다 =====

console.log("02단원에서 표 설계를 제대로 합니다.");

출력      02단원에서 표 설계를 제대로 합니다.

await db.close();
```

자주 틀린 곳 정리

문제 틀리는 이유 기억할 것

1 ORDER BY 를 안 붙임	순서가 중요하면 반드시 적는다
2 큰따옴표로 값을 감쌘	값은 '작은따옴표', 이름은 "큰따옴표"
3 ::int 를 안 붙임	BIGINT 는 커지면 bigint 로 바뀐다
4 ORDER BY 없이 LIMIT 만 씀	LIMIT 은 자르기만 한다
5 GROUP BY 에 없는 칸을 SELECT 에 씀	묶은 칸이나 집계 함수만
6 WHERE 에 집계 · HAVING 에 별칭	WHERE 는 줄, HAVING 은 묶음
9 GROUP BY 별칭이 되는 걸 표준으로 알	PostgreSQL 이 봐 주는 것뿐
10 -> 와 ->> 를 헷갈림	->> 가 글자, -> 가 JSON
12 NUMERIC 이 문자열로 오는 걸 모름	"43.3" !== 43.3
14 오타 난 키에 에러가 날 거라 생각함	조용히 null 이 된다

★★★ 이 단원에서 딱 하나만 가져간다면 이것입니다. **FROM → WHERE → GROUP BY → HAVING**
→ SELECT → ORDER BY → LIMIT 6번·7번·9번·12번이 전부 이 한 줄에서 나왔습니다.

02단원 연습문제 정답 표 설계하기

RUN node 연습문제_정답.js

답만 보지 말고 **왜 그런지**를 읽으세요. 이 단원의 문제들은 “문법을 아느냐”가 아니라 “나중에 무엇이 터지는지 아느냐”를 묻는 것입니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();

const 답 = {};
```

문제 1 — 표 만들기

```
답.문제1 = `
CREATE TABLE 작업자 (
  작업자번호 INT,
  이름 TEXT,
  입사일 DATE
)
`;
```

왜 이렇게 쓰나

칸 하나가 “이름 타입” 한 쌍입니다. 쉼표로 잇고, **마지막에는 쉼표를 붙이지 않습니다**. 자바스크립트 배열은 [1, 2,] 를 받아 주지만 SQL 은 42601 (문법 오류) 를 냅니다.

★ INT 는 INTEGER 의 줄임말입니다. information_schema 에서는 integer 로 나옵니다. ★ 글자는 TEXT 를 쓰세요. Postgres 에서 TEXT 는 VARCHAR 보다 느리지 않습니다. VARCHAR(50) 을 쓰면 나중에 50자를 넘는 이름이 들어올 때 22001 로 막힙니다. 길이를 정말 제한하고 싶으면 TEXT + CHECK (char_length(이름) <= 50) 이 낫습니다.

★★ 이 표에는 제약이 하나도 없습니다. 그래서 이런 게 다 들어갑니다.

작업자번호가 NULL 인 줄 · 같은 번호가 두 개 · 이름이 빈 문자열

실제 표라면 문제 7, 9, 10 처럼 제약을 걸어야 합니다.

문제 2 — 타입 고르기

```

답.문제2 = {
  설비단가: "NUMERIC(12,2)",
  설비온도: "REAL",
  가동여부: "BOOLEAN",
  점검기록시각: "TIMESTAMP TZ",
};

```

왜 이렇게 고르나

설비단가 → NUMERIC. “1원도 틀리면 안 됩니다”가 결정적인 단서입니다. REAL / DOUBLE PRECISION 은 2진수로 근사해서 담기 때문에 쌓으면 틀어집니다. 개념02 에서 0.1 을 1,000번 더했더니 **99.99905** 가 나왔습니다. 회계 담당자는 이 차이를 반드시 찾아냅니다. 그리고 원인은 못 찾습니다. ★ 돈·단가·세금·비율은 예외 없이 NUMERIC 입니다.

설비온도 → REAL. 반대로 온도는 원래 근사값입니다. 센서가 읽은 값 자체가 ±0.5도쯤 오차가 있습니다. 여기에 NUMERIC 을 쓰면 필요 없는 정확도에 비용만 납니다. ★ “원래 근사값인 측정치” 는 REAL / DOUBLE 이 맞습니다.

가동여부 → BOOLEAN. INT 로 0/1 을 쓰지 마세요. BOOLEAN 은 1바이트고, WHERE 가동여부 만으로 조회가 됩니다. 0/1 로 하면 “2가 들어오면 뭔가요?” 라는 질문이 언젠가 생깁니다.

점검기록시각 → TIMESTAMP TZ. “해외 지사와 같이 봅니다”가 결정적입니다. TIMESTAMP 는 시간대 정보가 없어서 뉴욕 사람이 “밤 11시 30분에 점검했구나” 하고 오해합니다. TIMESTAMPTZ 는 어디서 봐도 **같은 순간**입니다. ★ 사실 해외가 아니어도 TIMESTAMPTZ 를 쓰세요. 서머타임이나 서버 이전 때 터집니다.

문제 3 — 어느 타입이 정확한가

```

답.문제3 = "NUMERIC";

```

왜 그런가

NUMERIC 은 10진수를 **그대로** 담습니다. 사람이 종이에 계산하는 것과 같습니다. REAL / DOUBLE 은 2진수로 근사합니다. 0.1 은 2진수로 딱 떨어지지 않습니다. (1/3 을 소수로 적으면 0.3333... 으로 끝나지 않는 것과 같은 일입니다)

★★ 개념02 에서 재 보고 놀란 것이 하나 있습니다. `0.1::real + 0.2::real = 0.3::real` 은 **true** 가 나왔습니다. REAL 은 4바이트라 정밀도가 낮아서 오차가 반올림에 묻힌 것입니다.

그런데 1,000번 더하면 99.99905 가 됩니다. ★ 오차가 **없는** 게 아니라 **안 보이는** 것뿐입니다. 쌓으면 드러납니다. 한 번 재 보고 “괜찮네” 하고 넘어가면 안 되는 이유입니다.

문제 4 — BIGINT 를 안전하게 꺼내기

```
답.문제4 = `SELECT 큰번호::text AS 큰번호 FROM 번호창고`;
```

왜 이렇게 하나

PGlite 는 BIGINT 를 꺼낼 때, 값이 자바스크립트 number 에 안전하게 담기면 number 로, 넘으면 bigint 로 줍니다. 그리고 JSON.stringify 는 bigint 를 못 다룹니다.

```
TypeError: Do not know how to serialize a BigInt
```

★★★ 이 사고의 진짜 무서운 점은 개발할 때 절대 재현이 안 된다는 것입니다. 개발 DB 의 id 는 1, 2, 3 이라 number 로 옵니다. 잘 돌아갑니다. 운영에서 id 가 9,007,199,254,740,991 을 넘는 순간 bigint 가 되고, res.json(줄) 안의 JSON.stringify 가 터지면서 API 가 500 을 냅니다.

★ ::text 로 캐스팅하면 처음부터 문자열로 옵니다. 크기와 무관하게 안전합니다.

★★ Number(값) 으로 바꾸면 안 됩니다. 값이 조용히 망가집니다.

```
Number("9007199254740993") === 9007199254740992 ← 마지막 자리가 바뀝니다
```

에러도 안 나서 더 위험합니다. 자바스크립트 number 는 2^{53} 까지만 정확합니다.

문제 5 — 날짜를 하루 안 밀리게 꺼내기

```
답.문제5 = `SELECT 마감일::text AS 마감일 FROM 일정`;
```

왜 이렇게 하나

DATE 는 자바스크립트로 오면 UTC 자정의 Date 객체가 됩니다. 2026-03-15 는 2026-03-15T00:00:00.000Z 입니다.

여기에 getDate() 나 toLocaleDateString() 을 그냥 쓰면 음수 시간대(미국·유럽 일부)에서 하루가 밀립니다.

```
서울에서 보면 2026-03-15
뉴욕에서 보면 2026-03-14 ← 같은 값인데 하루 전
```

3월 15일 점검 기록이 3월 14일로 보이는 것입니다.

★ ::text 로 꺼내면 애초에 Date 객체를 안 만듭니다. 밀릴 일이 없습니다. ★ 이미 Date 로 받았다면 toISOString().slice(0,10) 이나 getUTCDate() 를 쓰세요. UTC 기준으로 읽는 것이 핵심입니다.

문제 6 — 표 고치기

```

답.문제6 = `
ALTER TABLE 설비목록 ADD COLUMN 도입일 DATE;
ALTER TABLE 설비목록 RENAME COLUMN 이름 TO 설비명;
`;

```

왜 이렇게 하나

표를 다시 만들 필요는 없습니다. ALTER 로 고칩니다.

ADD COLUMN 칸 추가 (기본값 없으면 거의 즉시. 안전합니다) DROP COLUMN 칸 삭제 (★ 데이터가 사라집니다) RENAME COLUMN A TO B 칸 이름 변경 ALTER COLUMN c TYPE t 칸 타입 변경 (★ 표 전체를 다시 씁니다. 큰 표면 멈춥니다) RENAME TO 표 이름 변경

★ 두 문장이므로 db.exec 를 써야 합니다. db.query 는 문장 하나만 받습니다.

```
"cannot insert multiple commands into a prepared statement"
```

★★ 새로 넣은 도입일 은 맨 뒤로 갑니다. Postgres 에서 칸 순서는 못 바꿉니다. 불편해 보이지만 SELECT 에서 칸 이름을 적으면 순서는 상관없습니다.

★★★ ALTER 로 고칠 수 있다고 대충 만들면 안 됩니다. RENAME COLUMN 을 하면 그 이름을 쓰던 모든 코드가 조용히 깨집니다. 처음에 잘 만드는 게 훨씬 쉽습니다.

문제 7 — 제약 걸기

```

답.문제7 = `
CREATE TABLE 점검 (
  점검번호 INT PRIMARY KEY,
  설비번호 INT NOT NULL,
  결과 TEXT NOT NULL CHECK (결과 IN ('정상', '주의', '불량')),
  비고 TEXT
)
`;

```

왜 이렇게 하나

요구사항 하나가 제약 하나에 그대로 대응합니다.

“기본키다” → PRIMARY KEY (= NOT NULL + UNIQUE + 색인) “비울 수 없다” → NOT NULL “셋 중 하나만” → CHECK (... IN (...)) “비워도 됨” → 아무것도 안 씀

★★ 결과 에 NOT NULL 과 CHECK 를 둘 다 건 것에 주목하세요. CHECK 만 걸면 NULL 이 통과합니다. NULL IN ('정상', '주의', '불량') 이 거짓이 아니라 UNKNOWN 이고, CHECK 는 거짓일 때만 막기 때문입니다. ★ CHECK 와 NOT NULL 은 서로 다른 일을 합니다. 둘 다 필요합니다.

★ 왜 애플리케이션 코드로 하지 않나요? 같은 검사를 자바스크립트로 쓰면 15줄쯤 됩니다. 줄 수가 문제가 아닙니다. 그 함수를 부르는 걸 잊는 순간 뚫린다는 게 문제입니다. 관리자 화면에서는 부르고, 엑셀 일괄 등록에서는 깜빡하고, 배치 스크립트는 몰랐고, 개발자가 psql 로 직접 넣으면 당연히 안 부릅니다. 제약은 누가 어떤 경로로 넣든 통합니다.

★ ENUM 타입(CREATE TYPE 결과종류 AS ENUM (...))도 있습니다. 그런데 값을 추가·삭제하기가 CHECK 보다 번거로워서, 처음에는 CHECK 를 권합니다.

문제 8 — 에러 코드

```
답.문제8 = {
  낫널위반: "23502",
  유니크위반: "23505",
  체크위반: "23514",
};
```

왜 외워야 하나

이 세 개는 사용자에게 보여 줄 메시지를 고르는 데 씁니다. 외워 두면 편합니다.

```
23502 NOT NULL 위반   e.column 에 칸 이름이 들어옵니다 (e.constraint 는 없습니다)
23505 UNIQUE / PK 중복 e.constraint 에 제약 이름
23514 CHECK 위반      e.constraint 에 제약 이름
23503 외래키 위반 (04단원)
22P02 타입에 안 맞는 글자
22003 범위 초과 (INT 넘침, NUMERIC 자릿수 넘침)
22001 길이 초과 (VARCHAR(n))
42601 문법 오류 · 42P01 없는 표 · 42P07 이미 있는 표
```

★★ e.message 와 e.detail 을 사용자에게 그대로 보여 주면 안 됩니다.

```
duplicate key value violates unique constraint "설비_이름_key"
Key ("이름")=(컨베이어 1호) already exists.
```

표 구조와 데이터가 그대로 새어 나갑니다. 로그에만 남기고 화면에는 “같은 이름의 설비가 이미 있습니다.” 처럼 우리 말로 바꿔서 보여 주세요.

★ 그러려면 제약에 이름을 붙여 두는 것이 좋습니다.

```
CONSTRAINT 상태_확인 CHECK (...)
```

자동 이름(표_칸_check)에 의존하면 칸 이름을 바꿨을 때 if (e.constraint === “설비_라인_check”) 가 조용히 헛됩니다.

문제 9 — DEFAULT


```

답.문제9 = `
CREATE TABLE 주문 (
  주문번호 INT PRIMARY KEY,
  상태 TEXT NOT NULL DEFAULT '접수',
  접수시각 TIMESTAMPTZ NOT NULL DEFAULT now()
)
`;

```

왜 이렇게 하나

“안 적으면 ~가 들어가야 함” 이 DEFAULT 입니다. “해외에서도 같은 순간으로 보여야 합니다” 가 TIMESTAMPTZ 입니다.

★★ DEFAULT 는 거의 항상 **NOT NULL** 과 짝으로 씁니다. 이유가 있습니다.

DEFAULT 는 “그 칸을 **아예 안 적었을 때**” 만 동작합니다. `INSERT INTO 주문 (주문번호, 상태) VALUES (1, NULL)` 처럼 **NULL** 을 직접 적으면 **DEFAULT** 를 지나치고 **NULL** 이 들어갑니다.

자바스크립트에서 { 상태: null } 을 그대로 넘기면 딱 이렇게 됩니다. NOT NULL 을 같이 걸어 두면 그 순간 23502 로 막힙니다.

★ 시각 기본값의 선택지

now()	동작 중인 트랜잭션이 **시작한** 시각 (같은 트랜잭션 안에서 항상 같음)
clock_timestamp()	진짜 지금 시각 (부를 때마다 다름)
CURRENT_TIMESTAMP	now() 와 같습니다
localtimestamp	★ TIMESTAMP 입니다. 시간대가 없습니다. 쓰지 마세요

보통은 now() 로 충분합니다.

문제 10 — 기본키를 자동으로

```

답.문제10 = `
CREATE TABLE 알림 (
  알림번호 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  내용 TEXT NOT NULL
)
`;

```

왜 SERIAL 이 아니라 IDENTITY 인가

BIGSERIAL 로도 자동 번호는 됩니다. 그런데 두 가지가 다릅니다.

① **SERIAL** 은 값을 직접 넣을 수 있습니다. 그게 사고의 원인입니다.

데이터를 옮기면서 번호를 직접 넣으면, 시퀀스는 그걸 **모릅니다**.

표에는 1,2,3 이 있는데 시퀀스는 여전히 1
→ 다음에 자동으로 넣으면 2번을 주려다 **23505 (중복)**

★★★ 이 사고의 지독한 점은 **며칠 뒤에 터진다**는 것입니다.

이관한 날은 멀쩡합니다. 나중에 사용자가 새로 등록할 때 처음 터집니다.
그때는 아무도 이관 작업을 떠올리지 못합니다.

GENERATED ALWAYS AS IDENTITY 는 직접 넣기를 아예 막습니다 (428C9). 정말 필요하면 OVERRIDING SYSTEM VALUE 를 **일부러 길게 적어야** 뚫립니다. 실수로는 안 뚫립니다.

(이미 SERIAL 로 만든 표를 옮겼다면 이걸 꼭 하세요)

```
SELECT setval('표_칸_seq', (SELECT max(칸) FROM 표));
```

② IDENTITY 는 SQL 표준입니다. SERIAL 은 Postgres 만의 것입니다.

★ 왜 INT 가 아니라 BIGINT 인가 INT 는 약 21억에서 잡니다. 초당 100건 쌓이는 로그면 250일에 다 씁니다. 다 찬 뒤에 BIGINT 로 바꾸려면 표 전체를 다시 써야 합니다. 몇 시간씩 멈춥니다. 4바이트 아끼려다 새벽에 일어납니다. ★ 키는 처음부터 BIGINT 로 잡으세요.

문제 11 — NULL 을 찾기

```
답.문제11 = `SELECT 설비번호 FROM 설비 WHERE 담당자 IS NULL ORDER BY 설비번호`;
```

왜 = NULL 이 아닌가 —————

WHERE 담당자 = NULL 은 문법 오류가 안 납니다. 그냥 항상 0건입니다.

NULL 은 “모른다” 라서, 무엇과 비교해도 결과가 UNKNOWN 입니다. NULL = NULL 조차 참이 아닙니다. 그리고 WHERE 는 TRUE 인 **줄만** 통과시킵니다. UNKNOWN 은 FALSE 와 똑같이 버려집니다.

★ IS NULL / IS NOT NULL 은 언제나 true 나 false 를 줍니다. UNKNOWN 이 없습니다.

★★ 파라미터로 넘길 때도 같은 함정이 있습니다.

```
db.query("... WHERE 담당자 = $1", [null]) → **항상 0건**  
db.query("... WHERE 담당자 IS NOT DISTINCT FROM $1", [null]) → 제대로 나옵니다
```

검색 화면에서 빈 칸을 넘길 때 자주 겪습니다. undefined 를 넘겨도 똑같이 0건입니다. 오타로 req.body.담당자 를 잘못 써도 에러 없이 0건이 나옵니다. 조용히 틀리는 종류입니다.

문제 12 — ★ NOT IN 을 고치기

```

답.문제12 = `
SELECT 설비번호 FROM 설비 가
WHERE NOT EXISTS (SELECT 1 FROM 정비중 나 WHERE 나.설비번호 = 가.설비번호)
ORDER BY 설비번호
`;

```

왜 NOT IN 이 0건이 되나

이 단원에서 **실무에서 가장 자주 터지는** 사고입니다.

설비번호 NOT IN (1, NULL) 은 이렇게 풀립니다.

```
NOT (설비번호 = 1 OR 설비번호 = NULL)
```

2번 설비로 따라가 봅시다.

```

설비번호 = 1      → false
설비번호 = NULL   → UNKNOWN
false OR UNKNOWN  → UNKNOWN
NOT UNKNOWN       → UNKNOWN
WHERE 는 TRUE 만 통과 → **버려집니다**

```

NULL 이 하나라도 섞이면 **모든 줄이 UNKNOWN** 이 되어 결과가 0건입니다. 에러도, 경고도 없습니다.
 “오늘 가동 가능한 설비” 화면이 그냥 텅 빕니다.

고치는 세 가지

① NOT EXISTS 를 쓴다 ← ★ 이게 정답입니다

NULL 이 있어도 안전하고, 대개 더 빠릅니다

② 안쪽에 IS NOT NULL 을 붙인다

```
NOT IN (SELECT 설비번호 FROM 정비중 WHERE 설비번호 IS NOT NULL)
```

됩니다. 다만 다음 사람이 왜 붙였는지 모르고 지웁니다

③ 애초에 그 칸에 NOT NULL 을 건다 ← ★ 근본 해결

```
`정비중.설비번호 INT NOT NULL` 이었으면 이 사고 자체가 없습니다
```

★★★ 습관을 하나 만드세요.

****NOT IN 에 서브쿼리를 쓰면 NOT EXISTS 로 바꿔 쓴다.****

그러면 이 사고를 평생 안 만납니다. (IN 은 괜찮습니다. NULL 이 섞여도 있는 것은 제대로 찾아 줍니다)

문제 13 — [도전] 집계와 NULL

답. 문제13 = ``SELECT avg(coalesce(수량, 0))::int AS 평균 FROM 생산`;`

왜 그냥 avg 를 쓰면 안 되나

생산 표에는 네 줄이 있고, 그중 두 줄의 수량이 NULL 입니다.

(1,100) (2,NULL) (3,200) (4,NULL)

``avg(수량)`` → (100+200) / **2** = 150
``avg(coalesce(수량,0))`` → (100+0+200+0) / **4** = 75

★★ avg 는 NULL 인 줄을 분모에서도 뺍니다. 이게 핵심입니다. sum 도 NULL 을 건너뛰니다. 그래서 `sum(수량) / count(*)` 로 하면 75가 나옵니다.

★★★ 150 이냐 75 나 는 업무가 정할 문제입니다. “수량을 아직 안 적은 설비” 를 평균에 넣을 것인가, 빼 것인가. SQL 의 기본은 “뺀다” 입니다. 중요한 건 어느 쪽을 원하는지 알고 쓰는 것입니다. 모르고 쓰면 보고서 숫자가 틀리고, 아무도 눈치 못 찻니다.

★ 관련해서 같이 기억할 것

`count(*)` 줄 수를 셉니다. NULL 과 무관
`count(칸)` 그 칸이 NULL 이 아닌 줄만 셉니다
`sum(칸)` 한 줄도 없으면 0 이 아니라 **NULL** 입니다
→ ``coalesce(sum(칸), 0)`` 으로 감싸세요

문제 14 — [도전] 요구사항대로 표 설계하기

답. 문제14 = ```
`CREATE TABLE 작업지시 (`
`지시번호 BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,`
`설비번호 INT NOT NULL,`
`라인 TEXT NOT NULL CHECK (라인 IN ('A','B','C')),`
`지시일 DATE NOT NULL DEFAULT CURRENT_DATE,`
`목표수량 INT NOT NULL CHECK (목표수량 > 0),`
`완료수량 INT NOT NULL DEFAULT 0 CHECK (완료수량 >= 0),`
`CONSTRAINT 수량_확인 CHECK (완료수량 <= 목표수량),`
`UNIQUE (설비번호, 지시일)`
`)`
``;`

요구사항이 어디로 갔는지 한 줄씩

“BIGINT 로 자동, 직접 못 넣음, 기본키”

→ BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

“반드시 있어야 한다” → NOT NULL “‘A,’B,’C’ 중 하나” → CHECK (라인 IN (‘A,’B,’C’)) “안 적으면 오늘” → DEFAULT CURRENT_DATE (+ NOT NULL 짝) “0보다 커야 한다” → CHECK (목표수량 > 0) “0 이상, 안 적으면 0” → CHECK (완료수량 >= 0) + DEFAULT 0

“완료수량이 목표수량보다 클 수 없다”

→ CONSTRAINT 수량_확인 CHECK (완료수량 <= 목표수량)

“같은 설비에 같은 날짜의 지시가 두 번”

→ UNIQUE (설비번호, 지시일)

★ 여기서 배울 세 가지

① 칸 두 개를 비교하는 **CHECK** 는 칸 정의 안에 못 씁니다. 완료수량 INT CHECK (완료수량 <= 목표수량) 은 문법은 통과하지만 의미가 어색합니다. 표 수준(테이블 제약)으로 따로 적는 것이 정석입니다. 종료일 >= 시작일 같은 것도 전부 이 자리에 씁니다.

② **UNIQUE (설비번호, 지시일)** 은 두 칸을 묶어서 유일하게 봅니다. 설비 1번의 3월 15일 지시와 설비 2번의 3월 15일 지시는 둘 다 들어갑니다. 설비번호 INT UNIQUE, 지시일 DATE UNIQUE 로 따로 걸면 완전히 다른 뜻이 됩니다. (설비 1번은 평생 지시를 한 번만 받게 됩니다)

③ **제약에 이름을 붙였습니다.** CONSTRAINT 수량_확인 이름을 안 붙이면 작업지시_check 처럼 자동 이름이 붙습니다. 에러를 사람 말로 바꿀 때 e.constraint === ‘수량_확인’ 으로 잡을 수 있어야 “완료수량은 목표수량을 넘을 수 없습니다.” 를 보여 줄 수 있습니다.

★★ 이 표 정의 하나가 자바스크립트 검사 코드 20줄을 대체합니다. 그리고 관리자 화면·배치·엑셀 업로드 ·psql 어디서 넣어도 똑같이 통합합니다. 동시에 열 명이 같은 설비·같은 날짜를 넣어도 UNIQUE 가 하나만 통과시킵니다. “조회해서 없으면 INSERT” 로는 이걸 못 막습니다.

문제 15 — [도전] 더러운 데이터를 정리하고 제약 걸기

답. 문제 15 = `

```
UPDATE 옛날설비 SET 라인 = 'A' WHERE 라인 IS NULL OR 라인 NOT IN ('A','B','C');
ALTER TABLE 옛날설비 ADD CONSTRAINT 라인_확인 CHECK (라인 IN ('A','B','C'));
`;
```

왜 순서가 중요한가

제약을 먼저 걸면 **실패합니다**.

```
ERROR: check constraint "라인_확인" of relation "옛날설비" is violated by some row
(SQLSTATE 23514)
```

★★★ 이미 어긴 데이터가 있으면 제약을 걸 수 없습니다. 이게 이 단원 전체가 “처음에 잘 만들라” 고 하는 이유입니다.

★ UPDATE 의 WHERE 에 주목하세요

```
WHERE 라인 IS NULL OR 라인 NOT IN ('A','B','C')
```

WHERE 라인 NOT IN ('A','B','C') 만 쓰면 **NULL 인 줄이 안 걸립니다**. NULL NOT IN (...) 이 UNKNOWN 이라 WHERE 를 통과하지 못하기 때문입니다. 그러면 UPDATE 는 잘 끝나는데 ALTER 가 여전히 실패합니다. “분명히 다 고쳤는데 왜 안 되지?” 하고 한참 헤맵니다. 문제 12 와 같은 뿌리입니다.

★ 빈 문자열 '' 도 잊지 마세요. NULL 이 아니라서 IS NULL 로는 안 걸리고,

```
`NOT IN ('A','B','C')` 으로 걸립니다. 이 문제에서는 5번 줄이 그렇습니다.
```

운영에서는 이 순서를 밟습니다

① 어긴 줄이 몇 개인지 센다 ② 그 줄들을 어떻게 할지 **업무 담당자와 정한다** ← 여기가 몇 주씩 걸립니다 ③ 고친다 ④ 제약을 건다 ⑤ 그 사이에 새로 들어온 어긴 줄이 없는지 다시 본다

★ 이 문제에서는 “전부 A 로” 라고 정해 줬지만, 실제로는

```
"999 라인은 뭐예요?" 를 물어볼 사람이 이미 퇴사한 경우가 많습니다.
```

★★ 큰 표에서는 NOT VALID 를 씁니다.

```
ALTER TABLE 옛날설비 ADD CONSTRAINT 라인_확인 CHECK (...) NOT VALID;
```

기존 줄은 안 보고 **새로 들어오는 것만** 막습니다. 표를 오래 잠그지 않습니다. 데이터를 다 고친 뒤 한가한 시간에 **VALIDATE CONSTRAINT** 로 전체를 검사합니다.

↓↓↓ 아래는 채점 코드입니다 (연습문제.js 와 같습니다) ↓↓↓

```

await db.exec(`
  CREATE TABLE 번호창고 (큰번호 BIGINT);
  INSERT INTO 번호창고 VALUES (9007199254740993);

  CREATE TABLE 일정 (마감일 DATE);
  INSERT INTO 일정 VALUES ('2026-03-15');

  CREATE TABLE 설비목록 (번호 INT, 이름 TEXT);

  CREATE TABLE 설비 (설비번호 INT PRIMARY KEY, 담당자 TEXT);
  INSERT INTO 설비 VALUES (1, '김반장'), (2, NULL), (3, '이반장'), (4, NULL);

  CREATE TABLE 정비중 (설비번호 INT);
  INSERT INTO 정비중 VALUES (1), (NULL);

  CREATE TABLE 생산 (설비번호 INT, 수량 INT);
  INSERT INTO 생산 VALUES (1, 100), (2, NULL), (3, 200), (4, NULL);

  CREATE TABLE 옛날설비 (번호 INT, 라인 TEXT);
  INSERT INTO 옛날설비 VALUES (1, 'A'), (2, '999'), (3, NULL), (4, 'B'), (5, '');
`);

let 맞은수 = 0;

async function 채점(번호, 확인) {
  const 답안 = 답[`문제${번호}`];

  const 안풀었나 =
    답안 === null ||
    답안 === undefined ||
    (typeof 답안 === "object" && Object.values(답안).every((값) => 값 === null));

  if (안풀었나) {
    console.log(`문제 ${번호} - 아직 안 풀었습니다`);
    return;
  }
}

```

```

try {
  const 통과 = await 확인(답안);
  if (통과) 맞은수 += 1;
  console.log(`문제 ${번호} - ${통과 ? "✅ 정답" : "❌ 틀렸습니다"}`);
} catch (에러) {
  console.log(`문제 ${번호} - ❌ 에러 (${에러.code ?? 에러.name})`);
}
}

async function 굴러보기(sql, 확인) {
  let 통과 = false;
  try {
    await db.transaction(async (tx) => {
      await tx.exec(sql);
      통과 = await 확인(tx);
      throw new Error("되돌리기");
    });
  } catch (에러) {
    if (에러.message !== "되돌리기") throw 에러;
  }
  return 통과;
}

async function 칸모양(tx, 표이름) {
  const 결과 = await tx.query(
    `SELECT column_name, data_type, is_nullable, column_default, is_identity
    FROM information_schema.columns WHERE table_name = $1 ORDER BY
    ordinal_position`,
    [표이름],
  );
  return 결과.rows;
}

```

성공하면 그 줄을 그대로 둡니다. 뒤에서 중복 검사를 해야 하기 때문입니다.


```

async function 막히나(tx, sql) {
  await tx.query(`SAVEPOINT 시험`);
  try {
    await tx.query(sql);
    await tx.query(`RELEASE SAVEPOINT 시험`);
    return false;
  } catch {
    await tx.query(`ROLLBACK TO SAVEPOINT 시험`);
    return true;
  }
}

console.log("===== 02단원 연습문제 채점 =====");

await 채점(1, (답안) =>
  쿨려보기(답안, async (tx) => {
    const 칸 = await 칸모양(tx, "작업자");
    return (
      칸.length === 3 &&
      칸[0].column_name === "작업자번호" && 칸[0].data_type === "integer" &&
      칸[1].column_name === "이름" && ["text", "character varying"].includes(칸
[1].data_type) &&
      칸[2].column_name === "입사일" && 칸[2].data_type === "date"
    );
  }),
);

await 채점(2, (답안) =>
  답안.설비단가 === "NUMERIC(12,2)" &&
  답안.설비온도 === "REAL" &&
  답안.가동여부 === "BOOLEAN" &&
  답안.점검기록시각 === "TIMESTAMP TZ",
);

await 채점(3, (답안) => 답안 === "NUMERIC");

await 채점(4, async (답안) => {

```

```

const 결과 = await db.query(답안);
const 값 = 결과.rows[0].큰번호;
return typeof 값 === "string" && 값 === "9007199254740993" &&
JSON.stringify(결과.rows).length > 0;
});

await 채점(5, async (답안) => {
const 값 = (await db.query(답안)).rows[0].마감일;
return 값 === "2026-03-15";
});

await 채점(6, (답안) =>
  둘러보기(답안, async (tx) => {
const 칸 = await 칸모양(tx, "설비목록");
const 이름들 = 칸.map((하나) => 하나.column_name);
return (
  이름들.includes("설비명") && !이름들.includes("이름") &&
  칸.some((하나) => 하나.column_name === "도입일" && 하나.data_type === "date")
);
}),
);

await 채점(7, (답안) =>
  둘러보기(답안, async (tx) => {
const 정상 = !(await 막히나(tx, `INSERT INTO 점검 VALUES (1, 10, '정상',
NULL)`));
const 결과오타 = await 막히나(tx, `INSERT INTO 점검 VALUES (2, 10, '가동',
NULL)`);
const 설비널 = await 막히나(tx, `INSERT INTO 점검 VALUES (3, NULL, '정상',
NULL)`);
const 번호중복 = await 막히나(tx, `INSERT INTO 점검 VALUES (1, 11, '주의',
NULL)`);
return 정상 && 결과오타 && 설비널 && 번호중복;
}),
);

await 채점(8, (답안) =>
  답안.낫날위반 === "23502" && 답안.유니크위반 === "23505" && 답안.체크위반 ===
"23514",
);

await 채점(9, (답안) =>

```

```

    쿨러보기(답안, async (tx) => {
      await tx.query(`INSERT INTO 주문 (주문번호) VALUES (1)`);
      const 줄 = (await tx.query(`SELECT 상태, 접수시각 FROM 주문 WHERE 주문번호 =
1`)).rows[0];
      const 칸 = await 칸모양(tx, "주문");
      const 시각칸 = 칸.find((하나) => 하나.column_name === "접수시각");
      return (
        줄.상태 === "접수" &&
        줄.접수시각 instanceof Date &&
        시각칸.data_type === "timestamp with time zone" &&
        시각칸.is_nullable === "NO"
      );
    }),
  );

  await 채점(10, (답안) =>
    쿨러보기(답안, async (tx) => {
      const 칸 = await 칸모양(tx, "알림");
      const 번호칸 = 칸.find((하나) => 하나.column_name === "알림번호");
      if (!번호칸 || 번호칸.data_type !== "bigint" || 번호칸.is_identity !== "YES")
      return false;
      await tx.query(`INSERT INTO 알림 (내용) VALUES ('첫 알림')`);
      const 자동 = (await tx.query(`SELECT 알림번호 FROM 알림`)).rows[0].알림번호;
      const 직접막힘 = await 막히나(tx, `INSERT INTO 알림 VALUES (99, '몰래')`);
      return Number(자동) === 1 && 직접막힘;
    }),
  );

  await 채점(11, async (답안) => {
    const 줄들 = (await db.query(답안)).rows.map((줄) => 줄.설비번호);
    return JSON.stringify(줄들) === JSON.stringify([2, 4]);
  });

  await 채점(12, async (답안) => {
    const 줄들 = (await db.query(답안)).rows.map((줄) => 줄.설비번호);
    return JSON.stringify(줄들) === JSON.stringify([2, 3, 4]);
  });

```

```

await 채점(13, async (답안) => {
  const 값 = (await db.query(답안)).rows[0].평균;
  return Number(값) === 75;
});

await 채점(14, (답안) =>
  둘러보기(답안, async (tx) => {
    const 칸 = await 칸모양(tx, "작업지시");
    const 지시번호 = 칸.find((하나) => 하나.column_name === "지시번호");
    if (!지시번호 || 지시번호.data_type !== "bigint" || 지시번호.is_identity !==
"YES") return false;

    const 정상 = !(await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (1,'A',100)`));
    const 라인틀림 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (2,'999',100)`);
    const 목표영 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (3,'A',0)`);
    const 완료초과 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량,
완료수량) VALUES (4,'B',10,11)`);
    const 중복 = await 막히나(tx, `INSERT INTO 작업지시 (설비번호,라인,목표수량)
VALUES (1,'A',50)`);

    const 첫줄 = (await tx.query(`SELECT 완료수량, 지시일 FROM 작업지시 WHERE
설비번호 = 1`)).rows[0];
    const 기본값 = 첫줄 && 첫줄.완료수량 === 0 && 첫줄.지시일 instanceof Date;

    const 이름불임 = (await tx.query(
      `SELECT count(*)::int AS 개수 FROM pg_constraint
      WHERE conrelid = '작업지시'::regclass AND conname = '수량_확인'`,
    )).rows[0].개수 === 1;

    return 정상 && 라인틀림 && 목표영 && 완료초과 && 중복 && 기본값 && 이름불임;
  }),
);

await 채점(15, (답안) =>
  둘러보기(답안, async (tx) => {
    const 줄수 = (await tx.query(`SELECT count(*)::int AS 개수 FROM 옛날설비
`)).rows[0].개수;
    const 나쁜것 = (await tx.query(
      `SELECT count(*)::int AS 개수 FROM 옛날설비 WHERE 라인 IS NULL OR 라인 NOT IN
('A','B','C')`,
    )).rows[0].개수;
  });

```

```

const 제약있나 = (await tx.query(
  `SELECT count(*)::int AS 개수 FROM pg_constraint
    WHERE conrelid = '옛날설비'::regclass AND conname = '라인_확인'`,
)).rows[0].개수 === 1;
const 막히나결과 = await 막히나(tx, `INSERT INTO 옛날설비 VALUES (9, 'Z')`);
return 줄수 === 5 && 나쁜것 === 0 && 제약있나 && 막히나결과;
}),
);

console.log(`===== 15문제 중 ${맞은수}개 정답 =====`);

```

```

출력      ===== 02단원 연습문제 채점 =====
문제 1 - ☒ 정답
문제 2 - ☒ 정답
문제 3 - ☒ 정답
문제 4 - ☒ 정답
문제 5 - ☒ 정답
문제 6 - ☒ 정답
문제 7 - ☒ 정답
문제 8 - ☒ 정답
문제 9 - ☒ 정답
문제 10 - ☒ 정답
문제 11 - ☒ 정답
문제 12 - ☒ 정답
문제 13 - ☒ 정답
문제 14 - ☒ 정답
문제 15 - ☒ 정답
===== 15문제 중 15개 정답 =====

```

```

await db.close();

```

03단원 연습문제 정답

넣고 읽고 고치고 지우기

RUN node 연습문제_정답.js

답만 보지 마세요. 왜 그렇게 쓰는지 이 파일의 본문입니다.

특히 12·13번(인젝션)은 답보다 설명이 더 중요합니다.

```
import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();

await db.exec(`
  CREATE TABLE 설비 (
    id          SERIAL PRIMARY KEY,
    이름        TEXT NOT NULL UNIQUE,
    라인        TEXT NOT NULL,
    상태        TEXT NOT NULL DEFAULT '정지',
    담당자      TEXT,
    도입연도    INT
  );

  INSERT INTO 설비 (이름, 라인, 상태, 담당자, 도입연도) VALUES
    ('컨베이어 1호', 'A', '가동', '김반장', 2019),
    ('컨베이어 2호', 'A', '정지', NULL, 2021),
    ('프레스 1호', 'B', '가동', '이기사', 2018),
    ('프레스 2호', 'B', '점검', '김반장', 2023),
    ('용접로봇 1호', 'C', '가동', '박주임', 2022),
    ('CNC 선반 1호', 'C', '고장', NULL, 2017);

  CREATE TABLE 점검기록 (
    id          SERIAL PRIMARY KEY,
    설비명      TEXT NOT NULL,
    결과        TEXT NOT NULL,
    점검일      DATE NOT NULL
  );

  INSERT INTO 점검기록 (설비명, 결과, 점검일)
  SELECT '설비' || (i % 10 + 1),
         (ARRAY['정상', '주의', '불량'])[i % 3 + 1],
         DATE '2026-06-01' + (i % 30)
  FROM generate_series(1, 300) AS i;

  CREATE TABLE 일일생산 (
```

```

    설비명 TEXT NOT NULL,
    날짜 DATE NOT NULL,
    수량 INT NOT NULL,
    PRIMARY KEY (설비명, 날짜)
);

CREATE TABLE 작업자 (
    사번 TEXT PRIMARY KEY,
    이름 TEXT NOT NULL,
    비밀번호 TEXT NOT NULL
);

INSERT INTO 작업자 VALUES
    ('A1001', '김반장', 'gongjang1'),
    ('A1002', '이기사', 'press2024');
`);

const 결과들 = [];

async function 채점(번호, 제목, 검사) {
    let 통과 = false;

    try {
        통과 = (await 검사()) === true;
    } catch {
        통과 = false;
    }

    결과들.push(통과);
    console.log(`${String(번호).padStart(2)}. ${제목} - ${통과 ? "✅ 통과" : "❌
아직"}`);
}

```

정답 1 — INSERT 한 건

```

const 문제1 = `
    INSERT INTO 설비 (이름, 라인, 상태, 도입연도)
    VALUES ('포장기 1호', 'A', '가동', 2025)
`;

await 채점(1, "INSERT 한 건", async () => {
    await db.query(문제1);
    const 확인 = await db.query("SELECT 라인, 상태, 도입연도 FROM 설비 WHERE 이름 =
'포장기 1호'");
    const 줄 = 확인.rows[0];
    return 줄?.라인 === "A" && 줄?.상태 === "가동" && 줄?.도입연도 === 2025;
});

```

★ 왜 칸 이름을 적나 INSERT INTO 설비 VALUES ('포장기 1호', 'A', '가동', 2025) 라고 써도 지금은 됩니다. 그런데 설비 표의 첫 칸은 id 입니다. 저렇게 쓰면 id 자리에 '포장기 1호' 가 갑니다. 운 좋게 타입이 안 맞아 에러가 나지만, 타입이 맞으면 **조용히 엉뚱한 칸에 들어갑니다**. 개념01 섹션 4 에서 그 사고를 직접 재현했습니다.

그리고 칸 이름을 적으면 순서를 신경 쓸 필요가 없습니다. 나중에 표에 칸이 추가돼도 이 코드는 그대로 둡니다.

★ 담당자와 id 는 안 적었습니다. id 는 SERIAL 이라 데이터베이스가 정하고, 담당자는 NULL 이 됩니다. 상태를 안 적었으면 DEFAULT 인 '정지' 가 들어갔을 것입니다.

정답 2 — 여러 건을 한 문장으로

```
const 문제2 = `
  INSERT INTO 설비 (이름, 라인, 도입연도)
  VALUES
    ('검사기 1호', 'A', 2024),
    ('검사기 2호', 'B', 2024),
    ('검사기 3호', 'C', 2025)
`;

await 채점(2, "여러 건을 한 문장으로", async () => {
  const 넣기 = await db.query(문제2);
  const 확인 = await db.query("SELECT 상태 FROM 설비 WHERE 이름 LIKE '검사기%' ORDER BY 이름");
  return 넣기.affectedRows === 3 && 확인.rows.length === 3 && 확인.rows.every((줄) =>
    줄.상태 === "정지");
});
```

★ 왜 한 문장으로 묶나 한 건씩 세 번 보내면 서버와 세 번 왕복합니다. 한 번에 보내면 한 번입니다. 3건이면 차이가 안 느껴지지만 10만 건이면 수십 배가 됩니다. 개념05 에서 진짜로 했습니다. 16초 대 0.7초였습니다.

★ 상태를 아예 안 적었습니다. 그래서 세 줄 모두 DEFAULT 인 '정지' 가 들어갔습니다. 상태 자리에 DEFAULT 라고 직접 써도 같습니다.

★★ 세 줄은 전부 되거나 전부 안 됩니다. 가운데 줄이 UNIQUE 를 어기면 첫 줄도 안 들어갑니다. 한 문장이 곧 하나의 트랜잭션이기 때문입니다.

정답 3 — RETURNING 으로 id 받기

```
async function 설비추가(이름, 도입연도) {
  const 결과 = await db.query(
```



```
VALUES ($1, 'C', $2)
  RETURNING id, 이름`,
  [이름, 도입연도],
);

return 결과.rows[0];
}

await 채점(3, "RETURNING 으로 id 받기", async () => {
  const 돌아온것 = await 설비추가("레이저 커터 1호", 2020);
  const 확인 = await db.query("SELECT id FROM 설비 WHERE 이름 = '레이저 커터 1호'");
  return typeof 돌아온것?.id === "number" && 돌아온것.id === 확인.rows[0]?.id;
});
```

출력 3. RETURNING 으로 id 받기 -  통과

★ RETURNING 이 없으면 어떻게 해야 하나 INSERT 한 뒤 `SELECT max(id) FROM 설비` 로 찾고 싶어집니다. 이게 위험합니다. 내가 넣은 직후에 남이 하나 더 넣으면, `max(id)` 는 **남의 줄**을 가리킵니다. 그 id 로 점검기록을 만들면 엉뚱한 설비에 붙습니다.

RETURNING 은 “이 INSERT 가 만든 줄” 을 돌려줍니다. 남의 줄이 섞일 수가 없습니다. 그리고 왕복이 한 번입니다.

★★ MySQL 8.4 에는 RETURNING 이 없습니다. `LAST_INSERT_ID()` 를 따로 부릅니다. 여러 줄을 넣으면 **첫 줄의 id** 만 줍니다. 09단원에서 비교합니다.

★ 파라미터를 쓴 이유 설비 이름은 사람이 입력합니다. 이어 붙이면 그대로 인젝션 통로가 됩니다. 값이 하나뿐이어도 예외 없이 파라미터로 보냅니다.

정답 4 — WHERE 와 IN

```
const 문제4 = `
  SELECT 이름
  FROM 설비
  WHERE 상태 IN ('점검', '고장')
  ORDER BY id
`;

await 채점(4, "WHERE 와 IN", async () => {
  const 결과 = await db.query(문제4);
  const 이름들 = 결과.rows.map((줄) => 줄.이름).join(",");
  const 칸들 = Object.keys(결과.rows[0] ?? {});
  return 이름들 === "프레스 2호,CNC 선반 1호" && 칸들.length === 1;
});
```

출력 4. WHERE 와 IN -  통과

★ IN 대신 OR 로 써도 결과는 같습니다.

```
WHERE 상태 = '점검' OR 상태 = '고장'
```

값이 셋 넷으로 늘면 IN 이 훨씬 읽기 좋습니다. 괄호 실수도 안 납니다.

★★ 목록에 NULL 이 섞이면 조심하세요. 상태 NOT IN ('점검', NULL) 은 **언제나 0건**입니다. 에러도 안 납니다. 'NULL 이 아닌가?' 의 답이 '모른다' 라서 모든 줄이 탈락합니다. NOT IN 을 쓸 때는 목록에 NULL 이 있는지 반드시 확인하세요.

★ SELECT * 가 아니라 이름만 골랐습니다. 안 쓰는 칸을 실어 나르지 않고, 표에 칸이 늘어도 응답이 안 바뀝니다.

★ 값이 프로그램에서 온다면 파라미터로 넘깁니다.

```
WHERE 상태 = ANY($1) ← 배열을 그대로 넘길 수 있습니다
```

정답 5 — NULL 찾기

```
const 문제5 = `
  SELECT count(*)::int AS 건수
  FROM 설비
  WHERE 담당자 IS NULL
`;

await 채점(5, "NULL 찾기", async () => {
  const 결과 = await db.query(문제5);
  const 기대 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 담당자 IS NULL");
  return 결과.rows[0]?.건수 === 기대.rows[0].건수 && 기대.rows[0].건수 > 0;
});
```

출력 5. NULL 찾기 -  통과

★★★ 담당자 = NULL 로 쓰면 **에러가 아니라 0건**입니다. NULL 은 “값이 없다” 가 아니라 “모른다” 입니다. 모르는 것끼리 같은지 물으면 답도 “모른다” 이고, 참이 아니니 그 줄은 안 나옵니다. 에러가 났으면 금방 고쳤을 텐데, 조용히 0건이라 한참 뒤에 발견됩니다.

★ 빈 글자("")와 NULL 도 다릅니다. 담당자 = " 는 “빈 글자가 들어 있다”, 담당자 IS NULL 은 “아무것도 없다” 입니다. 표를 설계할 때 둘 중 하나로 정하고 섞어 쓰지 마세요.

★ ::int 를 붙인 이유 count(*) 는 BIGINT 라서 자바스크립트에 **bigint** 로 옵니다. 그대로 두면 JSON.stringify 가 터지고, === 2 비교도 false 가 됩니다. ::int 로 바꾸면 number 로 옵니다. (02단원에서 다뤘습니다)

정답 6 — LIKE 로 검색하기

```
async function 설비검색(검색어) {
  const 결과 = await db.query(
    "SELECT 이름 FROM 설비 WHERE 이름 ILIKE $1 ORDER BY id",
    [`%${검색어}%`],
  );

  return 결과.rows.map((줄) => 줄.이름);
}

await 채점(6, "LIKE 로 검색하기", async () => {
  const 찾은것 = await 설비검색("cnc");
  const 없는것 = await 설비검색("zzz");
  return 찾은것.join(",") === "CNC 선반 1호" && 없는것.length === 0;
});
```

출력 6. LIKE 로 검색하기 -  통과

★ % 는 **값** 쪽에 붙였습니다. ILIKE ‘%’ || \$1 || ‘%’ 처럼 SQL 안에서 이어도 되지만, 자바스크립트에서 `%${검색어}%` 로 만들어 넘기는 편이 읽기 좋습니다. 어느 쪽이든 **검색어 자체는 파라미터**라는 점이 중요합니다.

★★ LIKE 는 대소문자를 가리고 ILIKE 는 안 가립니다. ‘cnc’ 로 ‘CNC 선반 1호’ 를 찾으려면 ILIKE 여야 합니다. 한글에는 대소문자가 없어서 한글만 찾을 때는 둘이 같습니다.

★★ 사용자가 % 나 _ 를 치면 와일드카드가 됩니다. ‘95%’ 로 검색하면 ‘95’ 로 시작하는 것이 전부 걸립니다. 막으려면 값에서 \ % _ 앞에 백슬래시를 붙여 주세요. (개념02 섹션 3)

★ 앞에 % 가 붙은 LIKE 는 색인을 못 씁니다. 10만 건이면 10만 건을 전부 훑습니다. 06단원에서 왜 그런지 봅니다.

정답 7 — 정렬과 자르기

```
const 문제7 = `
  SELECT 이름
  FROM 설비
  ORDER BY 도입연도 ASC, id
  LIMIT 3
`;

await 채점(7, "정렬과 자르기", async () => {
  const 결과 = await db.query(문제7);
  const 이름들 = 결과.rows.map((줄) => 줄.이름).join(",");
  return 이름들 === "CNC 선반 1호,프레스 1호,컨베이어 1호" &&
  /ORDER\s+BY[\s\S]*,\s*id/i.test(문제7);
});
```

출력 7. 정렬과 자르기 -  통과

★★★ 왜 id 를 덧붙이나 도입연도가 같은 설비가 둘 있으면(2024년 검사기 1호·2호), 그 둘 사이의 순서는 정해져 있지 않습니다. 매번 달라도 규칙 위반이 아닙니다.

목록 화면에서 쪽을 나눌 때 이게 사고가 됩니다. 1쪽에서 본 줄이 2쪽에 또 나오고, 어떤 줄은 아무리 넘겨도 안 보입니다. 개념02 섹션 5 에서 10개 중 9개가 겹치는 것을 직접 재현했습니다.

ORDER BY 끝에 고유한 칸(보통 id)을 하나 붙이면 순서가 완전히 정해집니다. 비용은 거의 없습니다.

습관으로 만드세요.

★ ORDER BY 없이 LIMIT 을 쓰면 “아무 3건” 이 나옵니다. 지금은 잘 나오다가 데이터가 늘거나 색인이 생기면 달라집니다.

★ 도입연도가 NULL 인 줄이 있으면 ASC 에서는 맨 뒤로 갑니다. DESC 로 하면 맨 앞으로 옵니다. 원하는 자리가 있으면 NULLS LAST 를 적으세요.

정답 8 — 별칭과 계산 칸

```
const 문제8 = `
  SELECT 이름, 2026 - 도입연도 AS 사용연수
  FROM 설비
  WHERE 도입연도 IS NOT NULL
  ORDER BY 사용연수 DESC, id
  LIMIT 2
`;

await 채점(8, "별칭과 계산 칸", async () => {
  const 결과 = await db.query(문제8);
  const 줄들 = 결과.rows.map((줄) => `${줄.이름}:${줄.사용연수}`).join(",");
  return 줄들 === "CNC 선반 1호:9,프레스 1호:8";
});
```

출력 8. 별칭과 계산 칸 -  통과

★★ 별칭은 ORDER BY 에서는 쓰이고 WHERE 에서는 안 쓰입니다. 실행 순서가 그렇기 때문입니다.

FROM → WHERE → GROUP BY → SELECT → ORDER BY → LIMIT

WHERE 를 볼 때는 사용연수라는 이름이 아직 안 만들어져 있습니다. WHERE 에서 걸러야 하면 계산식을 그대로 쓰세요.

```
WHERE 2026 - 도입연도 > 5
```

★ 도입연도가 NULL 인 줄을 왜 뺐나 2026 - NULL 은 NULL 입니다. 그리고 DESC 정렬에서 NULL 은 맨 앞입니다. 그러면 도입연도를 모르는 설비가 “가장 오래된 설비” 로 화면 맨 위에 뜹니다. WHERE 로 빼거나 ORDER BY ... DESC NULLS LAST 를 쓰세요.

★ 별칭에 띄어쓰기나 특수문자를 쓰려면 큰따옴표가 필요합니다. AS “사용 연수” 처럼요. 작은따옴표는 값이라서 안 됩니다.

정답 9 — UPDATE 와 affectedRows

```
async function 라인점검(라인) {
  const 결과 = await db.query(
    "UPDATE 설비 SET 상태 = '점검' WHERE 라인 = $1",
    [라인],
  );

  return 결과.affectedRows;
}

await 채점(9, "UPDATE 와 affectedRows", async () => {
  const 대상 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 라인 = 'C'");
  const 바뀐수 = await 라인점검("C");
  const 확인 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 라인 = 'C' AND 상태 = '점검'");
  const 다른라인 = await db.query("SELECT count(*)::int AS 건수 FROM 설비 WHERE 라인 <> 'C' AND 상태 = '점검'");

  return 바뀐수 === 대상.rows[0].건수 && 확인.rows[0].건수 === 대상.rows[0].건수 && 다른라인.rows[0].건수 === 1;
});
```

출력 9. UPDATE 와 affectedRows -  통과

★★★ WHERE 를 빠뜨리면 표 전체가 바뀝니다. 에러도 경고도 없습니다. 그리고 바뀌기 전 값은 어디에도 안 남습니다. **되돌릴 수 없습니다.** 개념04 섹션 2 에서 여섯 대가 한 번에 ‘정지’ 가 되는 것을 재현했습니다.

습관 세 가지를 권합니다.

- ① 같은 WHERE 로 SELECT 를 먼저 돌려 몇 건인지 확인합니다
- ② 운영 데이터는 BEGIN ... ROLLBACK/COMMIT 안에서 고칩니다
- ③ UPDATE 를 칠 때 WHERE 를 ****먼저**** 쓰고 SET 을 나중에 채웁니다

★ affectedRows 를 왜 돌려주나 0 건이어도 에러가 아닙니다. 조건에 맞는 줄이 없었을 뿐입니다. 이걸 안 보고 “수정했습니다” 를 띄우면, 아무 일도 안 일어났는데 성공으로 보입니다.

★ 라인을 파라미터로 받았습니다. ‘C’ 를 이어 붙이면 ‘C’; DROP TABLE 설비 -- 같은 값이 들어올 수 있습니다.

정답 10 — DELETE 와 RETURNING

```
async function 불량기록지우기() {
  const 결과 = await db.query(
```

04단원 연습문제 정답

데이터 모델링과 정규화

RUN node 연습문제_정답.js

답만 보지 마세요. 왜 그렇게 나누는지가 이 파일의 본문입니다.

특히 9·13번(설계)은 답보다 “왜 이 모양인가” 가 훨씬 중요합니다.

★★★ 설계 문제에는 정답이 여럿입니다.

채점 코드는 하나만 통과시키지만, 현장에서는 다른 답도 맞습니다.

그래서 각 설계 문제마다 “다른 합리적인 설계” 와 “무엇을 기준으로 고르는가” 를 적었습니다.

```

import { PGLite } from "@electric-sql/pglite";

const db = await PGLite.create();

await db.exec(`
  CREATE TABLE 점검대장 (
    점검번호      SERIAL PRIMARY KEY,
    설비이름      TEXT NOT NULL,
    설비도입년도  INT  NOT NULL,
    라인코드      TEXT NOT NULL,
    라인이름      TEXT NOT NULL,
    라인동        TEXT NOT NULL,
    담당자사번    INT  NOT NULL,
    담당자이름    TEXT NOT NULL,
    담당자연락처  TEXT NOT NULL,
    점검일        DATE NOT NULL,
    결과          TEXT NOT NULL,
    점수          INT,
    점검내용      TEXT NOT NULL
  );

  INSERT INTO 점검대장
    (설비이름, 설비도입년도, 라인코드, 라인이름, 라인동,
     담당자사번, 담당자이름, 담당자연락처, 점검일, 결과, 점수, 점검내용) VALUES
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-01-05', '정상', 92, '벨트 장력 정상'),
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-02-05', '정상', 88, '롤러 윤활 완료'),
    ('컨베이어 1호', 2015, 'A', '조립1라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-03-05', '주의', 71, '벨트 마모 관찰'),
    ('프레스 1호', 2019, 'A', '조립1라인', '1동', 101, '김반장', '010-1111-2222',
     '2024-01-12', '정상', 95, '유압 압력 정상'),
    ('프레스 1호', 2019, 'A', '조립1라인', '1동', 103, '박주임', '010-5555-6666',
     '2024-02-12', '정상', 90, '금형 교체'),
    ('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-01-20', '정상', 97, '토치 팁 교체'),
    ('용접로봇 1호', 2021, 'B', '가공2라인', '1동', 102, '이기사', '010-3333-4444',
     '2024-02-20', '주의', 68, '아크 불안정'),
    ('검사기 1호', 2018, 'B', '가공2라인', '1동', 103, '박주임', '010-5555-6666',
     '2024-01-25', '정상', 85, '센서 교정'),
    ('포장기 1호', 2022, 'C', '포장3라인', '2동', 104, '최사원', '010-7777-8888',
     '2024-02-28', '정상', 91, '실링 온도 조정');

  CREATE TABLE 작업자라인 (
    사번 INT PRIMARY KEY, 이름 TEXT NOT NULL, 라인목록 TEXT NOT NULL
  );

```

```
INSERT INTO 작업자라인 VALUES
```

```
(101, '김반장', 'A,B'), (102, '이기사', 'B'), (103, '박주임', 'A,BC'),  
(104, '최사원', 'BC'), (105, '정신입', 'C');
```

```
CREATE TABLE 설비부품 (
```

```
설비번호 INT NOT NULL, 부품코드 TEXT NOT NULL, 부품이름 TEXT NOT NULL,  
부품단가 INT NOT NULL, 수량 INT NOT NULL,  
PRIMARY KEY (설비번호, 부품코드)
```

```
);
```

```
INSERT INTO 설비부품 VALUES
```

```
(1, 'P-100', '베어링', 12000, 4), (1, 'P-200', '벨트', 35000, 2),  
(2, 'P-100', '베어링', 12000, 6), (3, 'P-300', '토치팁', 8000, 10),  
(3, 'P-100', '베어링', 12000, 2);
```

```
CREATE TABLE 설비대장 (
```

```
설비번호 INT PRIMARY KEY, 설비이름 TEXT NOT NULL, 도입년도 INT NOT NULL,  
라인코드 TEXT NOT NULL, 라인이름 TEXT NOT NULL, 라인동 TEXT NOT NULL
```

```
);
```

```
INSERT INTO 설비대장 VALUES
```

```
(1, '컨베이어 1호', 2015, 'A', '조립1라인', '1동'),  
(2, '프레스 1호', 2019, 'A', '조립1라인', '1동'),  
(3, '용접로봇 1호', 2021, 'B', '가공2라인', '1동'),  
(4, '검사기 1호', 2018, 'B', '가공2라인', '1동'),  
(5, '포장기 1호', 2022, 'C', '포장3라인', '2동');
```

```
CREATE TABLE 생산실적 (
```

```
실적번호 SERIAL PRIMARY KEY, 설비번호 INT NOT NULL, 날짜 DATE NOT NULL, 수량  
INT NOT NULL
```

```
);
```

```
INSERT INTO 생산실적 (설비번호, 날짜, 수량) VALUES
```

```
( 1, '2024-03-01', 120), ( 2, '2024-03-01', 80), (99, '2024-03-01', 45),  
( 3, '2024-03-02', 150), (77, '2024-03-02', 30), (99, '2024-03-03', 60),  
( 5, '2024-03-03', 95);
```

```
CREATE TABLE 정책라인 (라인코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL);
```

```
CREATE TABLE 정책설비 (
```

```
설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
```



```

라인코드 TEXT REFERENCES 정책라인(라인코드) ON DELETE RESTRICT
);
INSERT INTO 정책라인 VALUES ('A', '조립1라인'), ('B', '가공2라인');
INSERT INTO 정책설비 VALUES (1, '컨베이어 1호', 'A'), (2, '프레스 1호', 'A');

CREATE TABLE 사원 (사번 INT PRIMARY KEY, 이름 TEXT NOT NULL);
INSERT INTO 사원 VALUES
  (101, '김반장'), (102, '이기사'), (103, '박주임'), (104, '최사원'), (105,
'정신입');
CREATE TABLE 자격증 (자격코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL);
INSERT INTO 자격증 VALUES
  ('WELD', '용접기능사'), ('CRANE', '크레인운전'), ('SAFE', '산업안전기사');

CREATE TABLE 설비집계 (설비이름 TEXT PRIMARY KEY, 점검횟수 INT NOT NULL);
INSERT INTO 설비집계 VALUES
  ('컨베이어 1호', 3), ('프레스 1호', 2), ('용접로봇 1호', 5),
  ('검사기 1호', 0), ('포장기 1호', 1);
`);

const 결과들 = [];

async function 채점(번호, 제목, 검사) {
  let 통과 = false;

  try {
    통과 = (await 검사()) === true;
  } catch {
    통과 = false;
  }

  결과들.push(통과);
  console.log(`${String(번호).padStart(2)}. ${제목} - ${통과 ? "✅ 통과" : "❌
아직"}`);
}

const 다듬기 = (값) => String(값 ?? "").replace(/\s+/g, "").toUpperCase();
const 같은목록 = (내답, 정답) =>
  Array.isArray(내답) &&
  내답.length === 정답.length &&
  내답.every((값, 자리) => 다듬기(값) === 다듬기(정답[자리]));

```

정답 1 — 연락처 하나 바꾸는 데 몇 줄인가

```

async function 연락처바꾸기(사번, 새번호) {
  const 결과 = await db.query(
    "UPDATE 점검대장 SET 담당자연락처 = $2 WHERE 담당자사번 = $1",

```

```

);

return 결과.affectedRows;
}

await 채점(1, "연락처 하나 바꾸는 데 몇 줄인가", async () => {
  const 바뀐수 = await 연락처바꾸기(101, "010-9999-0000");
  const 가짓수 = await db.query(
    "SELECT count(DISTINCT 담당자연락처)::int AS 가짓수 FROM 점검대장 WHERE 담당자사번 = 101",
  );
  const 남의것 = await db.query(
    "SELECT count(*)::int AS 건수 FROM 점검대장 WHERE 담당자사번 <> 101 AND 담당자연락처 = '010-9999-0000'",
  );
  return 바뀐수 === 3 && 가짓수.rows[0].가짓수 === 1 && 남의것.rows[0].건수 === 0;
});

```

출력 1. 연락처 하나 바꾸는 데 몇 줄인가 - ☒ 통과

★ 이게 갱신 이상입니다. 김반장은 한 사람인데 연락처는 세 군데에 적혀 있습니다. 사실 하나를 고치는데 세 줄을 건드려야 합니다. 3년 치가 쌓여 300줄이 되면 어떨까요.

★★ 흔한 오답 ① — 이름으로 찾기

```
UPDATE 점검대장 SET 담당자연락처 = '...' WHERE 담당자이름 = '김반장'
```

지금은 통과합니다. 김반장이 한 명 더 들어오면 남의 연락처까지 바꿉니다. 사람을 가리키는 값은 이름이 아니라 사번입니다.

★★ 흔한 오답 ② — 한 줄만 고치기 (WHERE 점검번호 = 1) 화면에서 “김반장 정보 수정”을 눌러 저장하면 딱 이 모양이 됩니다. 같은 사번에 연락처가 두 가지가 되고, 어느 쪽이 맞는지 아무도 모릅니다. 채점의 count(DISTINCT 담당자연락처)가 이걸 잡아냅니다.

★ affectedRows를 꼭 보세요. 3이 아니라 1이면 조건이 잘못된 것입니다. UPDATE는 조용히 성공해서, 개수를 확인하지 않으면 알 수가 없습니다.

★ 9번에서 표를 나누면 이 작업은 UPDATE 작업자 ... WHERE 사번 = 101 한 줄이 됩니다. 고칠 곳이 하나뿐이면 어긋날 수가 없습니다. 그게 정규화의 목적입니다.

정답 2 — 이 사고들의 이름은 무엇인가

```

const 답2 = ["갱신 이상", "삽입 이상", "삭제 이상"];

await 채점(2, "이 사고들의 이름은 무엇인가", async () =>
  같은목록(답2, ["갱신 이상", "삽입 이상", "삭제 이상"]),
);

```

★ 세 가지 이상(anomaly)은 외울 게 아니라 알아보는 것입니다. 갱신 이상 — 사실 하나를 고치는데 여러 줄을 고쳐야 하고, 놓치면 말이 어긋납니다 삽입 이상 — 아직 일어나지 않은 일 때문에 아는 사실을 못 넣습니다 (라인 D) 삭제 이상 — 하나를 지웠더니 팔려 있던 다른 사실까지 사라집니다 (포장기 1호)

★★ 흔한 오답 — (나)를 갱신 이상이라고 답하기 “라인을 새로 등록하는 일” 이라 갱신 같아 보입니다. 못 넣는 것은 삽입 이상입니다. 기준은 단순합니다. 고칠 때 나면 갱신, 넣을 때 나면 삽입, 지울 때 나면 삭제입니다.

★ 세 이상의 뿌리는 하나입니다. **한 표에 서로 다른 주제를 섞어 놓았기 때문**입니다. 점검대장에는 라인·설비·사람·점검 이야기가 다 들어 있습니다. 주제별로 나누면 세 이상이 한꺼번에 없어집니다. 그게 9번입니다.

★ 이름 붙이는 순서를 뒤집지 마세요. 사고를 먼저 겪고 그 다음에 이름을 붙여야 남습니다.

정답 3 — 한 칸에 값을 여러 개 넣으면

```
async function B라인작업자() {
  await db.exec(`
    CREATE TABLE 작업자라인_1정규형 (
      사번      INT NOT NULL,
      라인코드 TEXT NOT NULL,
      PRIMARY KEY (사번, 라인코드)
    );

    INSERT INTO 작업자라인_1정규형 (사번, 라인코드)
    SELECT 사번, unnest(string_to_array(라인목록, ',')) FROM 작업자라인;
  `);

  const 결과 = await db.query(
    "SELECT 사번 FROM 작업자라인_1정규형 WHERE 라인코드 = $1 ORDER BY 사번",
    ["B"],
  );

  return 결과.rows.map((줄) => 줄.사번);
}

await 채점(3, "한 칸에 값을 여러 개 넣으면", async () => {
  const 사번들 = await B라인작업자();
  const 줄수 = await db.query("SELECT count(*)::int AS 건수 FROM 작업자라인_1정규형");
  const 오답 = await db.query("SELECT count(*)::int AS 건수 FROM 작업자라인 WHERE 라인목록 LIKE '%B%'");
  return (
    Array.isArray(사번들) &&
    사번들.join(",") === "101,102" &&
  );
});
```

```

줄수.rows[0].건수 === 7 &&
  오타م.rows[0].건수 === 4
);
});

```

출력 3. 한 칸에 값을 여러 개 넣으면 - ☒ 통과

★ 제1정규형은 “한 칸에 값 하나” 입니다. ‘A,B’ 는 값 하나가 아니라 값 둘을 글자로 이어 붙인 것입니다. 데이터베이스는 이걸 그냥 긴 글자로만 봅니다. 안에 뭐가 들었는지 모릅니다.

★★ 흔한 오답 ① — LIKE 를 더 정교하게 만들기

```

WHERE 라인목록 LIKE '%B%' → 4명 (오타 2명)
WHERE 라인목록 = 'B' OR LIKE 'B,%' OR LIKE '%,B' OR LIKE '%,B,%' → 맞긴 맞습니다

```

두 번째가 바로 **정규화를 안 해서 치르는 대가**입니다. 조건이 네 개로 늘었고, 라인 이름에 쉼표가 하나라도 들어가면 다시 깨집니다. 그리고 앞에 % 가 붙은 LIKE 는 색인을 못 씁니다. 10만 줄이면 10만 줄을 다 훑습니다.

★★ 흔한 오답 ② — 라인1·라인2·라인3 처럼 칸을 늘리기 네 번째 라인이 생기면 표 구조를 고쳐야 하고, 찾을 때도 칸 셋을 다 봐야 합니다. 이것도 1정규형 위반입니다.

★ `unnest(string_to_array(...))` 는 쉼표로 나눠 여러 줄로 퍼 줍니다. 손으로 일곱 줄을 적어 넣어도 정답입니다. 결과가 같으면 됩니다.

★★★ 정답이 여럿입니다 — 배열 칸(TEXT[])과 JSONB 도 현장에서 씁니다. GIN 색인도 됩니다. 기준 하나면 됩니다. **그 값으로 다른 표를 잇는가**. 라인코드는 라인 표를 가리키는 값인데 배열 안에는 외래키를 걸 수 없습니다. 반대로 ‘태그’ 처럼 아무 표도 안 가리키면 배열이 편합니다. (개념05 에서 JSONB 를 봅니다)

정답 4 — 기본키의 절반에만 매달린 칸

```

const 답4 = { 정규형: 1, 부분종속칸: ["부품단가", "부품이름"] };

await 채점(4, "기본키의 절반에만 매달린 칸", async () =>
  답4.정규형 === 1 && 같은목록(답4.부분종속칸, ["부품단가", "부품이름"]),
);

```

출력 4. 기본키의 절반에만 매달린 칸 - ☒ 통과

★ 왜 1정규형인가 — 칸마다 값이 하나씩이니 1정규형은 만족합니다. 2정규형을 어깁니다.

★ 부분 종속이 무엇인가 기본키가 (설비번호, 부품코드) 인데 부품이름·부품단가는 **부품코드만 알면** 정해집니다. 기본키의 일부에만 매달려 있습니다. 이게 부분 종속입니다. 확인해 보세요. P-100 이 세 줄에 나오는데 ‘베어링 · 12000’ 이 세 번 적혔습니다. 단가가 오르면 세 줄을 고쳐야 합니다. 또 갱신 이상입니다.

★★ 흔한 오답 ① — 수량도 부분 종속이라고 답하기 수량은 두 칸이 다 있어야 정해집니다. P-100 이 1번엔 4개, 2번엔 6개입니다. 이건 온전한 종속(완전 함수 종속)입니다.

★★ 흔한 오답 ② — 부분종속간에 설비번호나 부품코드를 넣기 그 둘은 기본키입니다. 종속되는 쪽이 아니라 결정하는 쪽입니다. “무엇이 무엇을 정하는가” 에서 화살표 **오른쪽**만 답에 씁니다.

★ 2정규형으로 고치면 이렇게 됩니다.

부품 (부품코드 TEXT 기본키, 부품이름 TEXT, 부품단가 INT)
설비부품 (설비번호 INT, 부품코드 TEXT, 수량 INT, PRIMARY KEY (설비번호, 부품코드))

단가를 바꾸는 일이 세 줄에서 한 줄이 됩니다.

★ 부분 종속은 **기본키가 두 칸 이상일 때만** 생깁니다. 한 칸이면 2정규형은 자동입니다. 그래서 실무에서 2정규형이 문제가 되는 곳은 거의 언제나 중간 표입니다.

정답 5 — 키가 아닌 칸에 매달린 칸

```
const 답5 = { 정규형: 2, 이행종속간: ["라인동", "라인이름"] };

await 채점(5, "키가 아닌 칸에 매달린 칸", async () =>
  답5.정규형 === 2 && 같은목록(답5.이행종속간, ["라인동", "라인이름"]),
);
```

출력 5. 키가 아닌 칸에 매달린 칸 -  통과

★ 왜 2정규형인가 — 기본키가 설비번호 한 칸이라 부분 종속이 있을 수 없습니다.

★ 이행 종속이 무엇인가

설비번호 → 라인코드 → 라인이름

설비번호로 라인이름이 정해지긴 하는데 **라인코드를 한 번 거칩니다**. 그中间的의 라인코드는 기본키가 아닙니다. 이걸 이행 종속이라고 합니다.

★★ 흔한 오답 — 이행종속간에 라인코드를 넣기 라인코드는 여기서 **결정하는 쪽**이고 설비번호에 직접 매달려 있습니다. 설비 한 대가 어느 라인에 속하는지는 설비 자신의 사실이라 남아야 합니다. 따라 나가야 하는 것은 “라인의 사실” 인 라인이름과 라인동입니다.

★ 3정규형으로 고치면 9번에서 만들 표가 그대로 나옵니다.

라인 (라인코드 TEXT 기본키, 이름 TEXT, 동 TEXT)
설비 (설비번호 INT 기본키, 설비이름 TEXT, 도입년도 INT, 라인코드 TEXT → 라인 참조)

★ 한 줄로 외우면 “키에 전부, 키에만, 다른 데는 말고.” 전부 = 2정규형(부분 종속 없음), 다른 데는 말고 = 3정규형(이행 종속 없음).

★ BCNF 는 이름만 알아 두세요. “결정하는 쪽은 전부 후보키여야 한다” 는 더 조인 규칙입니다. 후보키가 여럿이고 서로 겹칠 때만 문제가 됩니다. 보통 3정규형까지 맞추고, 필요하면 12번처럼 일부러 깎니다.

```
const 답6정책 = ["RESTRICT", "CASCADE", "SET NULL"];

async function 라인지워보기() {
  try {
    await db.query("DELETE FROM 정책라인 WHERE 라인코드 = 'A'");
    return null;
  } catch (에러) {
    return 에러.code;
  }
}

await 채점(6, "ON DELETE 를 상황에 맞게 고르기", async () => {
  const 코드 = await 라인지워보기();
  const 남았나 = await db.query("SELECT count(*)::int AS 건수 FROM 정책라인 WHERE 라인코드 = 'A'");
  return (
    같은목록(답6정책, ["RESTRICT", "CASCADE", "SET NULL"]) &&
    코드 === "23001" &&
    남았나.rows[0].건수 === 1
  );
});
```

출력 6. ON DELETE 를 상황에 맞게 고르기 -  통과

★ 다섯 가지 정책 RESTRICT 자식이 있으면 부모 삭제를 즉시 막습니다 NO ACTION 자식이 있으면 막습니다 (기본값). RESTRICT 와 거의 같습니다 CASCADE 부모를 지우면 자식도 같이 지웁니다 SET NULL 자식의 그 칸을 NULL 로 바꿉니다 SET DEFAULT 자식의 그 칸을 DEFAULT 값으로 바꿉니다

★★★ 여기가 핵심입니다. SQLSTATE 가 서로 다릅니다.

```
ON DELETE RESTRICT 위반 → 23001
ON DELETE NO ACTION 위반 → 23503
```

외래키니까 당연히 23503 이겠거니 하고 적으면 틀립니다. 돌려 보지 않으면 절대 모릅니다. 에러 코드로 분기하는 코드를 짜다가 이걸로 반나절씩 날립니다.

★★ 둘의 진짜 차이는 미룰 수 있느냐입니다. DEFERRABLE INITIALLY DEFERRED 를 걸면 NO ACTION 은 **트랜잭션 끝까지 미뤄져 통과**하고, RESTRICT 는 미뤄도 **그 자리에서 막힙니다**. (트랜잭션은 07단원에서 제대로 합니다)

★★ CASCADE 는 편한 만큼 위험합니다. 라인을 지우면 설비가, 그 설비의 점검기록이 줄줄이 따라갑니다. DELETE 한 줄에 몇 만 줄이 사라질 수 있고 되돌릴 수 없습니다. 무엇이 떨어져 나가는지 모르면 걸지 마세요.

★ SET DEFAULT 에는 함정이 하나 더 있습니다. 기본값이 **부모 표에 없으면** 23503 입니다. 기본값이 '미상' 이면 부모 표에 '미상' 줄을 미리 넣어 두어야 합니다.

★★★ 정답이 여럿입니다 — (나)를 SET NULL 로 해도 맞습니다. “설비를 폐기해도 점검 이력은 감사 때문에 남긴다” 는 현상이 많습니다. 그러면 SET NULL 이거나, 아예 안 지우고 폐기여부 칸을 두는 논리 삭제를 씁니다. 무엇을 기준으로 고르는가: **자식이 부모 없이도 의미가 있는가**. 장바구니에 담긴 물건은 의미가 없습니다 → CASCADE. 매출 전표는 담당자가 퇴사해도 의미가 있습니다 → SET NULL.

정답 7 — 관계의 모양 고르기

```
const 답7 = ["1:N", "N:M", "1:1", "N:M", "1:N"];

await 채점(7, "관계의 모양 고르기", async () =>
  같은목록(답7, ["1:N", "N:M", "1:1", "N:M", "1:N"]),
);
```

출력 7. 관계의 모양 고르기 -  통과

★ 모양을 가리는 질문은 언제나 두 개입니다. ① 왼쪽 하나에 오른쪽이 몇 개 붙는가 ② 오른쪽 하나에 왼쪽이 몇 개 붙는가 둘 다 하나면 1:1, 한쪽만 여럿이면 1:N, 둘 다 여럿이면 N:M 입니다.

★ 모양이 정해지면 표 만드는 법도 따라 정해집니다. 1:N → **N쪽(자식)에 외래키 칸**을 둡니다 (설비에 라인코드) N:M → 중간 표를 하나 더 만들고 양쪽으로 외래키를 겁니다 (10번) 1:1 → 한쪽에 외래키를 두고 그 칸에 **UNIQUE** 를 겁니다

★★ 흔한 오답 ① — (다)를 1:N 이라고 답하기 문제가 “딱 한 장” 이라고 못 박았으면 1:1 입니다. 구현은 보증서.설비번호 에 UNIQUE. ★ UNIQUE 하나가 1:1 과 1:N 을 가릅니다.

★★ 흔한 오답 ② — (마)를 “자기 참조” 라고 답하기 자기 참조는 **가리키는 곳을** 말하는 말이지 모양의 이름이 아닙니다. 모양은 그냥 1:N 입니다.

```
ALTER TABLE 작업자 ADD 사수사번 INT REFERENCES 작업자(사번);
```

★ 사수사번은 NULL 을 허용해야 합니다. 맨 위 사람은 사수가 없습니다.

★★★ 정답이 여럿입니다 — 1:1 은 표를 합치기도 합니다. 보증서 칸들을 설비 표에 그냥 넣어도 됩니다. 언제 나눌까요.

- 보증서가 **없는 설비가 많은가** → 많으면 나눕니다 (NULL 발이 안 생깁니다)
- 보증서 칸을 **거의 안 읽는가** → 안 읽으면 나눕니다 (한 줄이 가벼워집니다)

늘 같이 읽고 대부분 값이 있으면 합치는 편이 낫습니다.

정답 8 — 외래키에 색인이 생겼을까

```
async function 색인확인() {
  const 처음결과 = await db.query(
```

05단원 연습문제 정답 여러 표를 잇기

RUN node 연습문제_정답.js

★ 먼저 연습문제.js 를 직접 풀어 보고 오세요. 답을 보고 “아 그렇구나” 한 것은 이틀이면 사라집니다. 틀린 답을 직접 돌려 보고 왜 틀렸는지 확인한 것만 남습니다.

이 파일은 정답만 적어 두지 않았습니다. 틀린 답도 같이 돌려서 무엇이 어떻게 달라지는지 보여 줍니다.

```
import { PGlite } from "@electric-sql/pglite";

const db = await PGlite.create();
```


예제 데이터 (연습문제.js 와 같습니다)

```

await db.exec(`
  CREATE TABLE 라인 (라인코드 TEXT PRIMARY KEY, 이름 TEXT NOT NULL, 동 TEXT NOT NULL);
  CREATE TABLE 설비 (설비번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
    라인코드 TEXT REFERENCES 라인(라인코드), 도입년도 INT NOT NULL);
  CREATE TABLE 작업자 (사번 INT PRIMARY KEY, 이름 TEXT NOT NULL,
    소속라인 TEXT REFERENCES 라인(라인코드));
  CREATE TABLE 점검기록 (점검번호 INT PRIMARY KEY, 설비번호 INT REFERENCES 설비(설비번호),
    점검일 DATE NOT NULL, 결과 TEXT NOT NULL, 점수 INT,
    담당사번 INT REFERENCES 작업자(사번));
  CREATE TABLE 조직 (번호 INT PRIMARY KEY, 이름 TEXT NOT NULL,
    상위번호 INT REFERENCES 조직(번호));

  INSERT INTO 라인 VALUES ('A','조립1라인','1동'),('B','가공2라인','1동'),
    ('C','포장3라인','2동'),('D','신설4라인','2동');

  INSERT INTO 설비 VALUES (1,'컨베이어 1호','A',2015),(2,'프레스 1호','A',2019),
    (3,'용접로봇 1호','B',2021),(4,'검사기 1호','B',2018),
    (5,'포장기 1호','C',2022);

  INSERT INTO 작업자 VALUES (101,'김반장','A'),(102,'이기사','B'),(103,'박주임','B'),
    (104,'최사원','C'),(105,'정신입',NULL);

  INSERT INTO 점검기록 VALUES
    ( 1,1,'2024-01-05','정상', 92, 101),( 2,1,'2024-02-05','정상', 88, 101),
    ( 3,1,'2024-03-06','주의', 71, 102),( 4,1,'2024-04-05','정상', 90, 101),
    ( 5,2,'2024-01-12','주의', 65, 102),( 6,2,'2024-02-14','불량', 48, 103),
    ( 7,2,'2024-03-15','정상', 88, 101),( 8,3,'2024-01-20','정상', 95, 103),
    ( 9,3,'2024-02-20','정상',NULL, 103),(10,3,'2024-03-21','주의', 74, 102),
    (11,3,'2024-04-22','정상', 90,NULL),(12,3,'2024-05-23','불량', 52, 103),
    (13,4,'2024-01-28','정상',NULL, 104),(14,4,'2024-03-29','정상', 88, 104),
    (15,4,'2024-05-30','주의', 69,NULL);

  INSERT INTO 조직 VALUES
    ( 1,'조립1라인',NULL),( 2,'가공2라인',NULL),
    ( 10,'투입공정', 1),( 11,'조립공정', 1),( 12,'검사공정', 1),
    ( 20,'절삭공정', 2),( 21,'용접공정', 2),
    (100,'컨베이어 1호',10),(101,'프레스 1호',11),(102,'검사기 1호',12),
    (200,'선반 1호', 20),(201,'용접로봇 1호',21),(202,'용접로봇 2호',21);
`);

```

채점 도구 (연습문제.js 와 같습니다)

```
function 값글자(값) {
  if (값 === null) return "NULL";
  if (값 instanceof Date) return 값.toISOString().slice(0, 10);
  return String(값);
}

function 눌러쓰기(줄들) {
  return 줄들.map((줄) => Object.values(줄).map(값글자).join(",")).join(" | ");
}

let 번호 = 0;
let 맞힌수 = 0;

async function 정답확인(제목, 기대, 답) {
  번호 += 1;

  const 내것 = 눌러쓰기((await db.query(답)).rows);

  if (내것 === 기대) 맞힌수 += 1;

  console.log(`${번호}. ${제목} - ${내것 === 기대 ? "정답" : "오답"}`);
}
```

틀린 답을 돌려서 무엇이 나오는지 보여 줍니다.

```
async function 돌려보기(꼬리표, 답) {
  const 줄들 = (await db.query(답)).rows;

  console.log(` ${꼬리표}: ${줄들.length}줄 · ${누려쓰기(줄들) || "(빈 결과)"} `);
}

console.log("— 05단원 연습문제 정답 —");
```

출력 — 05단원 연습문제 정답 —

정답 1 — 두 표 잇기

await 정답확인(

"두 표 잇기",

"컨베이어 1호,2024-01-05,정상 | 컨베이어 1호,2024-02-05,정상 | 컨베이어 1호,2024-03-06,주의 | 컨베이어 1호,2024-04-05,정상 | 프레스 1호,2024-01-12,주의 | 프레스 1호,2024-02-14,불량 | 프레스 1호,2024-03-15,정상",

`

```
SELECT s.이름, p.점검일, p.결과
FROM 점검기록 p
JOIN 설비 s ON p.설비번호 = s.설비번호
WHERE p.설비번호 IN (1, 2)
ORDER BY p.점검번호
```

`

);

출력 1. 두 표 잇기 - 정답

왜 그런가

ON p.설비번호 = s.설비번호 가 **짜짓는 규칙**입니다. 점검기록의 한 줄마다 설비 표에서 설비번호가 같은 줄을 찾아 **옆에 붙입니다**.

설비번호는 설비 표의 기본키라서 짝이 **딱 하나**입니다. 그래서 줄 수가 안 늘어납니다. 점검기록 7줄이면 결과도 7줄입니다.

★ 어느 쪽을 FROM 에 쓰든 결과는 같습니다. FROM 설비 s JOIN 점검기록 p ON ... 이라고 써도 됩니다. INNER JOIN 은 좌우가 대칭입니다. (LEFT JOIN 은 아닙니다 — 7번 문제에서 봅니다)

★ JOIN 은 INNER JOIN 의 줄임말입니다. 둘은 완전히 같습니다.

정답 2 — 세 표 잇기와 별칭

await 정답확인(

"세 표 잇기와 별칭",

"1,컨베이어 1호,김반장,조립1라인 | 2,컨베이어 1호,김반장,조립1라인 | 3,컨베이어 1호,이기사,조립1라인 | 4,컨베이어 1호,김반장,조립1라인 | 5,프레스 1호,이기사,조립1라인 | 6,프레스 1호,박주임,조립1라인",

`

```
SELECT p.점검번호, s.이름 AS 설비이름, w.이름 AS 담당자, l.이름 AS 라인이름
FROM 점검기록 p
JOIN 설비 s ON p.설비번호 = s.설비번호
JOIN 라인 l ON s.라인코드 = l.라인코드
JOIN 작업자 w ON p.담당사번 = w.사번
WHERE p.점검번호 <= 6
ORDER BY p.점검번호
```

`

);

왜 그런가

★ 별칭이 없으면 아예 안 돌아갑니다. 설비에도 이름 이 있고, 작업자에도 이름 이 있고, 라인에도 이름 이 있습니다. 그냥 SELECT 이름 이라고 쓰면 Postgres 는 어느 쪽인지 모릅니다. column reference “이름” is ambiguous (42702) 에러가 납니다.

★ AS 는 결과에 붙는 이름을 바꿉니다. 원래 표의 칸 이름은 안 바꿉니다. 세 개가 다 이름 이면 받는 쪽 (자바스크립트)에서 키가 겹쳐 덮어써집니다. 그래서 AS 로 서로 다르게 붙여야 합니다.

★ 표를 잇는 순서를 보세요. 점검기록 → 설비 → 라인 으로 타고 갑니다. 라인을 이으려면 라인코드가 필요한데, 그건 설비가 갖고 있습니다. 그래서 설비를 먼저 이어야 s.라인코드 를 쓸 수 있습니다.

★ 작업자는 점검기록에서 바로 갑니다 (p.담당사번 = w.사번). 순서가 늘 한 줄로 이어지는 것은 아닙니다. 어디에 그 값이 있는지를 보세요.

정답 3 — 짝이 없는 것만 찾기

```
await 정답확인(
  "짝이 없는 것만 찾기",
  "5,포장기 1호",
  `
    SELECT s.설비번호, s.이름
    FROM 설비 s
    LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
    WHERE p.점검번호 IS NULL
    ORDER BY s.설비번호
  `,
  );
```

왜 그런가

세 걸음으로 나눠 보세요.

① LEFT JOIN 이 설비 5대를 전부 살려 둡니다.

짝이 없는 포장기 1호는 오른쪽 칸이 **전부 NULL** 인 줄로 남습니다.

② 짝이 있는 줄은 p.점검번호 에 진짜 값이 있습니다. ③ 그러니 p.점검번호 IS NULL 인 줄만 남기면 짝이 없던 것만 남습니다.

★★ NULL 검사는 반드시 오른쪽 표의 NOT NULL 칸으로 하세요. 보통 기본키입니다. p.점수 IS NULL 로 하면 이렇게 됩니다:

```
await 돌러보기("p.점수 IS NULL 로 하면", `
SELECT s.설비번호, s.이름
FROM 설비 s
LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
WHERE p.점수 IS NULL
ORDER BY s.설비번호
`);
```

출력 p.점수 IS NULL 로 하면: 3줄 · 3,용접로봇 1호 | 4,검사기 1호 | 5,포장기 1호

용접로봇 1호와 검사기 1호가 떨어져 나왔습니다. 이 둘은 점검을 받았는데 그중 점수가 비어 있는 점검(9번, 13번)이 있을 뿐입니다.

★ 점수 는 원래 NULL 이 들어갈 수 있는 칸입니다. 그래서 “짜이 없다” 는 뜻이 안 됩니다. 기본키는 절대 NULL 이 아니므로, 거기가 NULL 이면 **짜이 없다는 뜻**입니다.

★ 같은 것을 NOT EXISTS 로도 씁니다 (8번 정답 참고). 취향껏 고르세요.

정답 4 — 0건도 나오는 집계

```
await 정답확인(
    "0건도 나오는 집계",
    "컨베이어 1호,4 | 프레스 1호,3 | 용접로봇 1호,5 | 검사기 1호,3 | 포장기 1호,0",
    `
SELECT s.이름, COUNT(p.점검번호) AS 점검건수
FROM 설비 s
LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
GROUP BY s.설비번호, s.이름
ORDER BY s.설비번호
`,
);
```

출력 4. 0건도 나오는 집계 - 정답

왜 그런가

두 가지를 동시에 맞춰야 합니다.

① LEFT JOIN — 안 그러면 포장기 1호는 줄 자체가 사라집니다 ② COUNT(p.점검번호) — COUNT(*) 로 세면 안 됩니다

★★ COUNT(*) 로 세면 어떻게 되는지 보세요:

```
await 돌러보기("COUNT(*) 로 세면", `
SELECT s.이름, COUNT(*) AS 점검건수
FROM 설비 s
LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
GROUP BY s.설비번호, s.이름
ORDER BY s.설비번호
`);
```

출력 COUNT(*) 로 세면: 5줄 · 컨베이어 1호,4 | 프레스 1호,3 | 용접로봇 1호,5
| 검사기 1호,3 | 포장기 1호,1

포장기 1호가 1 이 되었습니다. 점검을 한 번도 안 받았는데요.

★ 이유: COUNT(*) 는 **줄 수**를 셉니다. LEFT JOIN 이 포장기 1호를 살려 두느라 **NULL 로 채운 줄 한 개**를 만들었고, COUNT(*) 는 그 줄도 한 줄이라고 셉니다.

★ COUNT(p.점검번호) 는 **그 칸이 NULL 이 아닌 줄만** 셉니다. NULL 로 채워진 줄은 안 셉니다. 그래서 0 이 나옵니다.

★ 이걸 실무 보고서에서 진짜로 터지는 실수입니다. “미점검 설비 0대” 라고 보고했는데 알고 보니 1대씩 세고 있던 것입니다.

정답 5 — COUNT 세 가지

```
await 정답확인(
  "COUNT 세 가지",
  "15,13,10",
  `
SELECT COUNT(*)           AS 전체,
       COUNT(점수)         AS 점수있음,
       COUNT(DISTINCT 점수) AS 서로다른점수
FROM 점검기록
`,
);
```

출력 5. COUNT 세 가지 - 정답

왜 그런가

COUNT(*)	15	줄 수를 셉니다. NULL 이 있든 없든 상관없습니다
COUNT(점수)	13	점수가 NULL 인 9번·13번을 안 셉니다

COUNT(DISTINCT 점수) 10 88 이 세 번(2·7·14), 90 이 두 번(4·11) 나오므로

★ 셋이 다르다는 것을 모르면 평균이 틀립니다. 직접 보세요:

```
await 돌러보기("AVG(점수) vs SUM(점수)/COUNT(*)", `
  SELECT AVG(점수) AS 진짜평균, SUM(점수)::numeric / COUNT(*) AS 틀린평균
  FROM 점검기록
`);
```

```
출력      AVG(점수) vs SUM(점수)/COUNT(*): 1줄
          77.6923076923076923,67.333333333333333
```

10점 넘게 차이가 납니다.

★ **AVG(점수)** 는 **점수가 있는 13건**으로 나눕니다. **SUM(점수) / COUNT(*)** 는 **15건**으로 나눕니다. 점수를 안 매긴 점검까지 0점처럼 취급한 셈입니다.

★ 어느 쪽이 “맞다” 고 정해진 것은 없습니다. **무엇을 보고할지**에 달렸습니다. 다만 둘이 다르다는 것을 모르고 쓰면 사고입니다.

★ 결과가 77.6923076923076923 처럼 긴 것은 **NUMERIC** 이기 때문입니다. 자바스크립트로는 **문자열**로 옵니다. **Number()** 로 바꿔야 계산이 됩니다.

정답 6 — HAVING 으로 그룹 거르기

```
await 정답확인(
  "HAVING 으로 그룹 거르기",
  "컨베이어 1호,4 | 용접로봇 1호,5",
  `
    SELECT s.이름, COUNT(*) AS 건수
    FROM 설비 s
    JOIN 점검기록 p ON s.설비번호 = p.설비번호
    GROUP BY s.설비번호, s.이름
    HAVING COUNT(*) >= 4
    ORDER BY s.설비번호
  `
);
```

```
출력      6. HAVING 으로 그룹 거르기 - 정답
```

왜 그런가

실행 순서를 떠올리세요.

FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

WHERE 는 **묶기 전에** 줄을 버립니다. 그 시점에는 아직 **COUNT** 가 없습니다. 그래서 **WHERE COUNT(*) >= 4** 는 에러입니다 (aggregate functions are not allowed in WHERE).

HAVING 은 **묶은 뒤에** 그룹을 버립니다. 그때는 **COUNT** 가 있습니다.

★ 한 문장으로: **줄을 거르면 WHERE, 그룹을 거르면 HAVING.**

★ 여기서는 INNER JOIN 을 썼습니다. 4건 이상만 찾으니 0건짜리는 어차피 탈락입니다. LEFT JOIN 을 써도 답은 같습니다. 짧은 쪽을 골랐습니다.

정답 7 — ★★★ LEFT JOIN 과 조건의 자리

```
await 정답확인(
  "LEFT JOIN 과 조건의 자리",
  "컨베이어 1호,0 | 프레스 1호,1 | 용접로봇 1호,1 | 검사기 1호,0 | 포장기 1호,0",
  `
    SELECT s.이름, COUNT(p.점검번호) AS 불량건수
    FROM 설비 s
    LEFT JOIN 점검기록 p
      ON s.설비번호 = p.설비번호
     AND p.결과 = '불량'
    GROUP BY s.설비번호, s.이름
    ORDER BY s.설비번호
  `
);
```

출력 7. LEFT JOIN 과 조건의 자리 - 정답

왜 그런가 —————

조건을 WHERE 에 쓰면 이렇게 됩니다:

```
await 돌려보기("WHERE p.결과 = '불량' 으로 쓰면", `
  SELECT s.이름, COUNT(p.점검번호) AS 불량건수
  FROM 설비 s
  LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.결과 = '불량'
  GROUP BY s.설비번호, s.이름
  ORDER BY s.설비번호
`);
```

출력 WHERE p.결과 = '불량' 으로 쓰면: 2줄 · 프레스 1호,1 | 용접로봇 1호,1

설비 5대 중 **2대만** 남았습니다. 불량이 0건인 설비 3대가 통째로 사라졌습니다.

★★★ 무슨 일이 일어났나 — 순서대로 보세요.

① LEFT JOIN 이 짝을 짓습니다.

컨베이어 1호는 점검이 4건 있지만 전부 불량이 아닙니다. → 4줄이 붙습니다
포장기 1호는 짝이 없습니다. → 오른쪽이 전부 NULL 인 줄 1개가 만들어집니다

② 그다음 WHERE 가 p.결과 = '불량' 로 줄을 거릅니다.

컨베이어 1호의 4줄은 결과가 '정상'/'주의' 라 전부 탈락합니다.
 포장기 1호의 NULL 줄은 `NULL = '불량'` 인데, 이건 거짓이 아니라 **UNKNOWN** 입니다.
 WHERE 는 **참인 것만** 통과시킵니다. UNKNOWN 도 탈락입니다.
 (02단원에서 배운 NULL 3값 논리입니다)

③ 결국 불량이 있는 설비만 남습니다. **LEFT JOIN 이 INNER JOIN 이 되어 버렸습니다.**

★ 고치는 방법: 조건을 **ON** 으로 옮깁니다.

ON 은 **짜을 지을 때** 쓰는 규칙입니다. "설비번호가 같고 **그리고 결과가 불량인** 점검만 짝으로 인정한다" 는 뜻이 됩니다. 조건에 안 맞으면 짝이 없는 것으로 처리되고, LEFT JOIN 이 왼쪽 줄을 살려 둡니다.

한 문장으로: **ON** 은 **짜짓는 규칙**, **WHERE** 는 **다 짓고 나서 버리는 규칙**.

★ NULL 을 봐주면 살아나기는 합니다. 그런데 읽기가 어렵습니다:

```
await 돌러보기("WHERE 에 IS NULL 을 더하면", `
  SELECT s.이름, COUNT(p.점검번호) AS 불량건수
  FROM 설비 s
  LEFT JOIN 점검기록 p ON s.설비번호 = p.설비번호
  WHERE p.결과 = '불량' OR p.결과 IS NULL
  GROUP BY s.설비번호, s.이름
  ORDER BY s.설비번호
`);
```

```
출력      WHERE 에 IS NULL 을 더하면: 3줄 · 프레스 1호,1 | 용접로봇 1호,1 |
          포장기 1호,0
```

포장기 1호는 살아났지만 컨베이어 1호와 검사기 1호는 여전히 없습니다. 이 둘은 점검을 받았고(그래서 NULL 줄이 안 생겼고) 불량이 없을 뿐입니다.

★ 결론: **오른쪽 표에 거는 조건은 ON** 에. 예외를 만들지 마세요. 왼쪽 표에 거는 조건(**WHERE s.라인코드 = 'A'**)은 WHERE 에 써도 안전합니다.

정답 8 — ★★★ 담당하 적 없는 작업자

```
await 정답확인(
  "담당하 적 없는 작업자",
  "105,정신입",
  `
    SELECT w.사번, w.이름
    FROM 작업자 w
    WHERE NOT EXISTS (
      SELECT 1 FROM 점검기록 p WHERE p.담당사번 = w.사번
    )
    ORDER BY w.사번
  `,
  `);
```

왜 그런가

먼저 **NOT IN** 으로 쓴 것을 돌려 보세요:

```
await 돌러보기("NOT IN 으로 쓰면", `
    SELECT w.사번, w.이름
    FROM 작업자 w
    WHERE w.사번 NOT IN (SELECT 담당사번 FROM 점검기록)
    ORDER BY w.사번
`);
```

0건입니다. 에러도 안 납니다. 그냥 아무것도 없다고 나옵니다.

★★★ 왜 그런가 — 한 줄씩 풀어 보세요.

점검기록의 담당사번을 모으면 101, 102, 103, 104, 그리고 **NULL** 입니다. (11번·15번 점검은 담당자가 안 적혀 있습니다)

정신입(105)에 대해 **105 NOT IN (101, 102, 103, 104, NULL)** 을 계산하면

```
105 <> 101 → 참
105 <> 102 → 참
105 <> 103 → 참
105 <> 104 → 참
105 <> NULL → * UNKNOWN ← "모르는 값과 다른가?" 는 모릅니다
```

```
참 AND 참 AND 참 AND 참 AND UNKNOWN = UNKNOWN
```

WHERE 는 참만 통과시킵니다. UNKNOWN 은 버립니다. 그래서 정신입도 탈락합니다. 그리고 **모든 줄이 똑같이 UNKNOWN** 이 됩니다. 결과는 언제나 0건입니다.

★ 직접 확인해 보세요:

```
await 돌러보기("105 NOT IN (...) 의 값", `
    SELECT (105 NOT IN (SELECT 담당사번 FROM 점검기록)) AS 결과
`);
```

true 도 false 도 아닌 **NULL** 입니다.

★ 반대로 **IN** 은 괜찮습니다. 하나라도 참이면 참이니까요:

```
await 돌러보기("104 IN (...) 과 999 IN (...)", `
    SELECT (104 IN (SELECT 담당사번 FROM 점검기록)) AS 있는사번,
           (999 IN (SELECT 담당사번 FROM 점검기록)) AS 없는사번
`);
```

출력 104 IN (...) 과 999 IN (...): 1줄 · true,NULL

★ 찾으면 true 로 잘 나옵니다. 그런데 못 찾으면 false 가 아니라 NULL 입니다. IN 도 NULL 이 섞이면 완전히 안전하지는 않습니다.

고치는 방법 세 가지

① NOT EXISTS ← 권장

EXISTS 는 "그런 줄이 있냐 없냐" 만 봅니다. 값을 비교하지 않으니 NULL 이 끼어들 틈이 없습니다.
 `SELECT 1` 인 이유도 같습니다. 무엇을 고르는지 아무도 안 봅니다.

② 서브쿼리에서 NULL 을 미리 빼기

```
await 돌러보기("NOT IN + IS NOT NULL", `
    SELECT w.사번, w.이름
    FROM 작업자 w
    WHERE w.사번 NOT IN (SELECT 담당사번 FROM 점검기록 WHERE 담당사번 IS NOT NULL)
    ORDER BY w.사번
`);
```

출력 NOT IN + IS NOT NULL: 1줄 · 105,정신입

됩니다. 그런데 **까먹으면 그날로 0건**입니다. 사람이 기억해야 하는 방법은 나쁜 방법입니다.

③ LEFT JOIN ... IS NULL (3번 문제의 관용구)

```
await 돌러보기("LEFT JOIN 으로 풀면", `
    SELECT w.사번, w.이름
    FROM 작업자 w
    LEFT JOIN 점검기록 p ON p.담당사번 = w.사번
    WHERE p.점검번호 IS NULL
    ORDER BY w.사번
`);
```

출력 LEFT JOIN 으로 풀면: 1줄 · 105,정신입

★★★ 외울 것: 서브쿼리에 NOT IN 을 쓰지 마세요. NOT EXISTS 를 쓰세요. 지금은 그 칸에 NULL 이 없더라도, 반년 뒤에 누가 NULL 을 넣으면 쿼리는 에러 없이 조용히 0건이 됩니다. 아무도 모릅니다.

```
await 정답확인(
  "FILTER 로 조건별 세기",
  "A,4,2,1 | B,5,2,1",
  `
    SELECT l.라인코드,
           COUNT(*) FILTER (WHERE p.결과 = '정상') AS 정상,
           COUNT(*) FILTER (WHERE p.결과 = '주의') AS 주의,
           COUNT(*) FILTER (WHERE p.결과 = '불량') AS 불량
    FROM 라인 l
    JOIN 설비 s      ON l.라인코드 = s.라인코드
    JOIN 점검기록 p ON s.설비번호 = p.설비번호
    GROUP BY l.라인코드
    ORDER BY l.라인코드
  `;
);
```

출력 9. FILTER 로 조건별 세기 - 정답

왜 그런가

WHERE p.결과 = '정상' 을 쓰면 정상 건수밖에 못 셉니다. 나머지 줄이 사라지니까요. FILTER 는 **집계 함수 하나하나에** 조건을 겁니다. 줄은 그대로 둔 채로요.

★ FILTER 가 없던 시절에는 이렇게 썼습니다. 결과는 같습니다:

```
await 돌려보기("SUM(CASE WHEN ...) 으로 쓰면", `
  SELECT l.라인코드,
         SUM(CASE WHEN p.결과 = '정상' THEN 1 ELSE 0 END) AS 정상,
         SUM(CASE WHEN p.결과 = '주의' THEN 1 ELSE 0 END) AS 주의,
         SUM(CASE WHEN p.결과 = '불량' THEN 1 ELSE 0 END) AS 불량
  FROM 라인 l
  JOIN 설비 s      ON l.라인코드 = s.라인코드
  JOIN 점검기록 p ON s.설비번호 = p.설비번호
  GROUP BY l.라인코드
  ORDER BY l.라인코드
`);
```

출력 SUM(CASE WHEN ...) 으로 쓰면: 2줄 · A,4,2,1 | B,5,2,1

★ FILTER 쪽이 짧고 뜻이 분명합니다. Postgres 를 쓴다면 FILTER 를 쓰세요. MySQL 에는 FILTER 가 없어서 SUM(CASE ...) 를 써야 합니다. (09단원에서 다룹니다)

★ C 라인과 D 라인은 결과에 없습니다. INNER JOIN 이라 점검기록이 없는 쪽이 빠졌기 때문입니다. 전부 보고 싶으면 LEFT JOIN 으로 바꾸세요 (직접 해 보세요).

정답 10 — 평균보다 높은 점검

```
await 정답확인(
  "평균보다 높은 점검",
  "1,92 | 2,88 | 4,90 | 7,88 | 8,95 | 11,90 | 14,88",
  `
    SELECT 점검번호, 점수
    FROM 점검기록
    WHERE 점수 > (SELECT AVG(점수) FROM 점검기록)
    ORDER BY 점검번호
  `,
);
```

출력 10. 평균보다 높은 점검 - 정답

왜 그런가

괄호 안의 `SELECT AVG(점수) FROM 점검기록` 은 값 하나를 돌려줍니다. 이런 것을 스칼라 서브쿼리라고 합니다. 값 하나니까 > 오른쪽에 그대로 놓을 수 있습니다.

★ 평균은 77.69 입니다. 그보다 큰 점수는 88, 90, 92, 95 입니다. 7건입니다.

★ 점수가 NULL 인 9번·13번은 결과에 없습니다. `NULL > 77.69` 는 거짓이 아니라 **UNKNOWN** 이라 통과하지 못합니다. 여기서는 그게 원하는 동작입니다. 점수를 안 매겼으니 “평균보다 높다” 고 할 수 없으니까요. ★ 하지만 항상 그런 것은 아닙니다. NULL 을 어떻게 다룰지는 **매번 정해야** 합니다.

★ “77.69 보다 큰 것” 이라고 숫자를 박아 넣으면 안 됩니다. 점검을 하나 더 넣는 순간 평균이 바뀌는데, 쿼리는 옛날 숫자를 씁니다. **에러가 안 나서** 더 위험합니다.

★ 서브쿼리가 두 줄 이상을 돌려주면 에러입니다: `more than one row returned by a subquery used as an expression` (21000) 반대로 **0줄이면 에러가 아니라 NULL** 입니다. 그러면 WHERE 가 전부 탈락시킵니다. 0건이 나왔는데 이유를 모르겠으면 서브쿼리부터 따로 돌려 보세요.

정답 11 — CTE 로 최고 라인 찾기

```
await 정답확인(
  "CTE 로 최고 라인 찾기",
  "B,78.00",
  `
    WITH 라인평균 AS (
      SELECT s.라인코드, AVG(p.점수) AS 평균점수
      FROM 설비 s
      JOIN 점검기록 p ON s.설비번호 = p.설비번호
      GROUP BY s.라인코드
    )
    SELECT 라인코드, ROUND(평균점수, 2) AS 평균점수
  `
);
```

```
ORDER BY 평균점수 DESC, 라인코드
LIMIT 1
\,
);
```

출력 11. CTE 로 최고 라인 찾기 - 정답

왜 그런가

WITH 이름 AS (...) 는 **이름 붙인 임시 결과**입니다. 아래 SELECT 에서 그냥 표처럼 씁니다.

★ 같은 것을 서브쿼리로도 쓸 수 있습니다. 결과는 같습니다:

```
await 돌러보기("FROM 절 서브쿼리로 쓰면", `
SELECT 라인코드, ROUND(평균점수, 2) AS 평균점수
FROM (
    SELECT s.라인코드, AVG(p.점수) AS 평균점수
    FROM 설비 s JOIN 점검기록 p ON s.설비번호 = p.설비번호
    GROUP BY s.라인코드
) AS 라인평균
ORDER BY 평균점수 DESC, 라인코드
LIMIT 1
`);
```

출력 FROM 절 서브쿼리로 쓰면: 1줄 · B,78.00

★ 지금은 한 겹이라 별 차이가 없습니다. 두세 겹이 되면 **서브쿼리는 안에서 밖으로, CTE 는 위에서 아래로** 읽게 됩니다. 사람은 위에서 아래로 읽는 데 익숙합니다. 그래서 CTE 가 이깁니다.

★ FROM 절 서브쿼리에는 별칭(**AS 라인평균**)을 꼭 붙이세요. PostgreSQL 15 까지는 없으면 **subquery in FROM must have an alias** 에러였습니다. 16 부터 선택 사항이 되어서 우리가 쓰는 18.3 에서는 그냥 통과합니다. 그래도 붙이세요. 별칭이 없으면 그 결과의 칸을 **라인평균.평균점수** 처럼 가리킬 수 없고, MySQL 8.0 에서는 여전히 에러입니다. (개념04 에서 실측합니다)

★ ORDER BY 평균점수 DESC 뒤에 , 라인코드 를 붙인 이유: 평균이 같은 라인이 둘이면 어느 쪽이 1등인지 **정해지지 않습니다**. 실행할 때마다 답이 달라질 수 있습니다. 유일한 칸으로 반드시 순서를 못 박으세요.

★ 결과가 78.00 이지 78 이 아닙니다. ROUND(numeric, 2) 는 소수 두 자리를 **유지한 NUMERIC** 을 돌려주고, PGlite 는 NUMERIC 을 **문자열**로 줍니다. typeof 를 찍어 보면 string 입니다.

★ C 라인은 점검기록이 없어서 평균 자체가 없습니다. D 라인은 설비도 없습니다. 둘 다 INNER JOIN 에서 빠졌습니다.

정답 12 — RANK 로 순위 매기기

```
await 정답확인(
    "RANK 로 순위 매기기",
    "1,92,1 | 4,90,2 | 2,88,3 | 7,88,3 | 3,71,5 | 5,65,6 | 6,48,7",
    `
        SELECT 점검번호, 점수, RANK() OVER (ORDER BY 점수 DESC NULLS LAST) AS 순위
        FROM 점검기록
        WHERE 설비번호 IN (1, 2)
        ORDER BY 순위, 점검번호
    `,
);
```

출력 12. RANK 로 순위 매기기 - 정답

왜 그런가

순위를 매기는 함수는 셋입니다. 같은 데이터로 나란히 보세요.

```
await 돌려보기("ROW_NUMBER 로 매기면", `
    SELECT 점검번호, 점수, ROW_NUMBER() OVER (ORDER BY 점수 DESC NULLS LAST, 점검번호)
    AS 순위
    FROM 점검기록 WHERE 설비번호 IN (1, 2)
    ORDER BY 순위
`);
```

출력 ROW_NUMBER 로 매기면: 7줄 · 1,92,1 | 4,90,2 | 2,88,3 | 7,88,4 |
3,71,5 | 5,65,6 | 6,48,7

```
await 돌려보기("DENSE_RANK 로 매기면", `
    SELECT 점검번호, 점수, DENSE_RANK() OVER (ORDER BY 점수 DESC NULLS LAST) AS 순위
    FROM 점검기록 WHERE 설비번호 IN (1, 2)
    ORDER BY 순위, 점검번호
`);
```

출력 DENSE_RANK 로 매기면: 7줄 · 1,92,1 | 4,90,2 | 2,88,3 | 7,88,3 |
3,71,4 | 5,65,5 | 6,48,6

88점짜리 두 건(2번·7번)을 보세요.

```
ROW_NUMBER 3, 4   동점을 **무시하고** 그냥 번호를 붙입니다
RANK 3, 3   동점은 같은 순위. 다음은 4위를 **건너뛰고** 5위
DENSE_RANK 3, 3   동점은 같은 순위. 다음은 **건너뛰지 않고** 4위
```

★ 어느 것이 맞느냐가 아니라 무엇을 세느냐의 문제입니다. “몇 등이나” 를 사람이 이해하는 방식은 보통 RANK 입니다 (공동 3위 다음은 5위). “서로 다른 등급이 몇 개냐” 를 보려면 DENSE_RANK 입니다. “그냥 줄 번호” 가 필요하면 ROW_NUMBER 입니다.

★★ ROW_NUMBER 를 쓸 때는 동점을 가를 칸을 반드시 ORDER BY 에 넣으세요. 위 예제에도 , 점검번호 가 들어 있습니다. 안 넣으면 2번과 7번 중 누가 3위인지 정해지지 않아 실행할 때마다 답이 달라질 수 있습니다.

★ NULLS LAST 를 붙인 이유: Postgres 는 ORDER BY ... DESC 일 때 NULL 을 맨 앞에 둡니다. 점수가 없는 점검이 1위가 되어 버립니다. 여기서는 설비 1·2만 봐서 NULL 이 없지만, 습관을 들여 두는 편이 안전합니다.

정답 13 — [도전] 설비마다 점수 상위 2건

```
await 정답확인(
  "[도전] 설비마다 점수 상위 2건",
  "1,1,92 | 1,4,90 | 2,7,88 | 2,5,65 | 3,8,95 | 3,11,90 | 4,14,88 | 4,15,69",
  `
    SELECT 설비번호, 점검번호, 점수
    FROM (
      SELECT 설비번호, 점검번호, 점수,
             ROW_NUMBER() OVER (
               PARTITION BY 설비번호
               ORDER BY 점수 DESC NULLS LAST, 점검번호
             ) AS 순위
      FROM 점검기록
    ) AS 매긴것
    WHERE 순위 <= 2
    ORDER BY 설비번호, 순위
  `
);
```

출력 13. [도전] 설비마다 점수 상위 2건 - 정답

왜 그런가 —————

이건 실무에서 가장 자주 필요한 모양입니다. “고객마다 최근 주문 3건”, “라인마다 불량 상위 5개” 전부 같은 꼴입니다.

세 걸음입니다.

① PARTITION BY 설비번호 로 설비마다 따로 창을 엽니다 ② 그 안에서 ORDER BY 점수 DESC 로 줄을 세우고 ROW_NUMBER() 로 번호를 붙입니다 ③ 그 결과를 한 겹 감싸서 WHERE 순위 <= 2 로 거릅니다

★★ ③에서 감싸는 것이 핵심입니다. 안 감싸면 안 됩니다:

```
SELECT ... FROM 점검기록
WHERE ROW_NUMBER() OVER (...) <= 2      ← 에러
→ window functions are not allowed in WHERE (42P20)
```

실행 순서 때문입니다.


```
await 정답확인(
    "[도전] 점검 간격 구하기",
    "8,3,NULL | 9,3,31 | 10,3,30 | 11,3,32 | 12,3,31",
    `
        SELECT 점검번호,
               설비번호,
               점검일 - LAG(점검일) OVER (PARTITION BY 설비번호 ORDER BY 점검일) AS 간격
        FROM 점검기록
        WHERE 설비번호 = 3
        ORDER BY 점검일
    `,
    );
```

출력 14. [도전] 점검 간격 구하기 - 정답

왜 그런가

LAG(칸) 은 같은 창 안에서 한 줄 앞의 값을 가져옵니다. ORDER BY 점검일 이 “앞” 이 무엇인지 정해 줍니다.

점검일	LAG(점검일)	빼면
2024-01-20	NULL	NULL ← 앞이 없습니다
2024-02-20	2024-01-20	31
2024-03-21	2024-02-20	30
2024-04-22	2024-03-21	32
2024-05-23	2024-04-22	31

★ DATE - DATE 는 정수(일수) 를 돌려줍니다. INTERVAL 이 아닙니다. 자바스크립트로도 number 로 옵니다.

★ 첫 줄이 NULL 인 게 싫으면 LAG(점검일, 1, 기준일) 처럼 기본값을 줄 수 있습니다. 하지만 여기서는 NULL 이 정직합니다. “앞 점검이 없다” 는 사실 그대로니까요.

★ PARTITION BY 설비번호 를 빼면 어떻게 될까요? 설비가 섞여서, 3번 설비의 첫 점검이 2번 설비의 마지막 점검과 비교됩니다. 지금은 WHERE 설비번호 = 3 이라 티가 안 나지만, 전체를 볼 때는 반드시 필요합니다.

★ 개념02 에서 이걸 SELF JOIN 으로 하려면 “바로 앞 줄” 을 찾는 조건이 꽤 복잡했습니다. 윈도우 함수로는 한 줄입니다. 이게 윈도우 함수를 쓰는 이유입니다.

정답 15 — [도전] 계층 훑기

```
await 정답확인(
    "[도전] 계층 훑기",
    "3,가공2라인 > 용접공정 > 용접로봇 1호 | 3,가공2라인 > 용접공정 > 용접로봇 2호 |
```

3,가공2라인 > 절삭공정 > 선반 1호 | 3,조립1라인 > 검사공정 > 검사기 1호 | 3,조립1라인 > 조립공정 > 프레스 1호 | 3,조립1라인 > 투입공정 > 컨베이어 1호",

```
WITH RECURSIVE 훑기 AS (
    SELECT 번호, 이름, 1 AS 깊이, 이름::TEXT AS 경로
    FROM 조직
    WHERE 상위번호 IS NULL

    UNION ALL

    SELECT 아래.번호, 아래.이름, 위.깊이 + 1, 위.경로 || ' > ' || 아래.이름
    FROM 조직 아래
    JOIN 훑기 위 ON 아래.상위번호 = 위.번호
    WHERE 위.깊이 < 5
)
SELECT 깊이, 경로
FROM 훑기
WHERE 깊이 = 3
ORDER BY 경로
);
```

출력 15. [도전] 계층 훑기 - 정답

왜 그런가

WITH RECURSIVE 는 두 부분으로 되어 있습니다. 가운데의 UNION ALL 이 경계입니다.

① **기준줄** — 맨 위(상위번호가 NULL 인 것)를 고릅니다. 여기가 시작점입니다. ② **재귀줄** — 방금 찾은 것들(훑기)의 자식을 찾습니다.

어떻게 도는지 보세요.

1회차: 기준줄이 조립1라인, 가공2라인 을 만듭니다 (깊이 1, 2줄) 2회차: 그 둘의 자식인 공정 5개를 찾습니다 (깊이 2, 5줄) 3회차: 공정들의 자식인 설비 6개를 찾습니다 (깊이 3, 6줄) 4회차: 설비의 자식은 없습니다 → 아무것도 안 나옵니다 → 멈춥니다

★ 멈추는 조건은 “더 나올 게 없을 때” 입니다. 자동입니다. 문제는 **순환 참조**입니다. A의 상위가 B, B의 상위가 A 라면 영원히 돌립니다. 그래서 WHERE 위.깊이 < 5 같은 안전장치를 겁니다. Postgres 14 부터는 CYCLE 번호 SET 순환 USING 지나온길 로 더 깔끔하게 막을 수 있습니다.

★ 이름::TEXT 로 캐스팅한 이유: 기준줄과 재귀줄은 **칸의 타입이 정확히 같아야** 합니다. 이름 은 TEXT 인데, 경로 || ' > ' || 이름 도 TEXT 라 사실 여기서는 없어도 됩니다. 그런데 칸을 VARCHAR(50) 으로 만든 표에서는 길이가 안 맞아 예러가 납니다. 습관적으로 ::TEXT 를 붙여 두면 안전합니다.

★ UNION ALL 이지 UNION 이 아닙니다. UNION 은 중복을 제거하느라 매번 전체를 비교합니다. 느립니다. 계층을 훑을 때는 같은 줄이 두 번 나올 일이 없으니 UNION ALL 이 맞습니다.

★ **ORDER BY** 경로로 정렬했습니다. 재귀 결과의 순서는 **보장되지 않습니다**. 깊이 순으로 나올 것 같지만, 그건 우연입니다. 순서가 필요하면 반드시 **ORDER BY** 를 쓰세요.

채점 결과

```
console.log(`— ${번호}문제 중 ${맞힌수}문제 정답 —`);
```

```
출력      — 15문제 중 15문제 정답 —
```

정리 — 이 단원에서 반드시 몸에 붙여야 할 것

상황 쓸 것 틀리면

— —

짜 있는 것만 **INNER JOIN** — 짜 없는 쪽도 보고 싶다 **LEFT JOIN** 3대가 조용히 사라짐 오른쪽 표에 조건을 건다 **LEFT JOIN ... ON ... WHERE** 에 쓰면 **INNER** 가 됨 짜이 없는 것만 찾는다 **LEFT JOIN + 기본키 IS NULL NULL** 가능 칸으로 하면 오답 0건도 나오게 센다 **COUNT(오른쪽칸) COUNT(*)** 면 0 이 1 이 됨 그룹을 거른다 **HAVING WHERE** 에 쓰면 에러 “~한 적 없는” 을 찾는다 **NOT EXISTS NOT IN** 이면 0건 그룹마다 상위 **N ROW_NUMBER + 감싸기 LIMIT** 이면 전체에서 **N건** 줄을 유지한 채 계산 윈도우 함수 **GROUP BY** 면 줄이 접힘

★ 이 중 **LEFT JOIN + WHERE** 와 **NOT IN + NULL** 두 가지는 에러가 안 나고 결과만 조용히 틀립니다. 그래서 제일 위험합니다. 쿼리를 쓸 때마다 이 둘을 의심하세요.

```
await db.close();
```

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.